

Directory: system

.lsystem\config.py

```
import os
from pathlib import Path
from pydantic import Field, computed_field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Config(BaseSettings):
    """MIRA environment configurations using pydantic-settings"""

    model_config = SettingsConfigDict(
        env_file='.env',
        env_file_encoding='utf-8',
        case_sensitive=True,
        extra='ignore'
    )

    # Logging & Telemetry
    BROWSER_USE_LOGGING_LEVEL: str = 'info'
    ANONYMIZED_TELEMETRY: bool = False

    # LLM API Keys
    OPENAI_API_KEY: str | None = None
    ANTHROPIC_API_KEY: str | None = None
    GOOGLE_API_KEY: str | None = None
    SKIP_LLM_API_KEY_VERIFICATION: bool = False
    BROWSER_USE_VERSION_CHECK: bool = True

    # Browser configurations
    CHROME_PATH: str | None = None

    # Hardcoded or env fields
    IN_DOCKER: bool = Field(default_factory=lambda: os.getenv('IN_DOCKER', 'false').lower()[0] in 'ty1')

    @computed_field
    @property
    def XDG_CONFIG_HOME(self) -> Path:
        return Path(os.getenv('XDG_CONFIG_HOME', '~/.config')).expanduser().resolve()

    @computed_field
    @property
    def XDG_CACHE_HOME(self) -> Path:
        return Path(os.getenv('XDG_CACHE_HOME', '~/.cache')).expanduser().resolve()

    # The remaining properties used by profile.py
    @computed_field
    @property
    def BROWSER_USE_CONFIG_DIR(self) -> Path:
        path = Path(os.getenv('BROWSER_USE_CONFIG_DIR', str(self.XDG_CONFIG_HOME / 'browseruse'))).expanduser()
        path.mkdir(parents=True, exist_ok=True)
        return path

    @computed_field
    @property
    def BROWSER_USE_PROFILES_DIR(self) -> Path:
        return self.BROWSER_USE_CONFIG_DIR / 'profiles'

    @computed_field
    @property
    def BROWSER_USE_DEFAULT_USER_DATA_DIR(self) -> Path:
        return self.BROWSER_USE_PROFILES_DIR / 'default'

    @computed_field
    @property
```

```

def BROWSER_USE_EXTENSIONS_DIR(self) -> Path:
    path = self.BROWSER_USE_CONFIG_DIR / 'extensions'
    path.mkdir(parents=True, exist_ok=True)
    return path

# Create singleton
CONFIG = Config()

```

.\system\logging_config.py

```

"""Logging configuration for MIRA browser-use clone."""
import logging
import sys

def setup_logging(stream=None, log_level=logging.INFO):
    """Setup basic logging configuration for the agent."""
    logger = logging.getLogger('system')

    # Check if handlers are already set up
    if logger.hasHandlers():
        return logger

    # Clear existing handlers
    logger.handlers = []

    class MIRAFormatter(logging.Formatter):
        def format(self, record):
            # Clean up long component names
            if isinstance(record.name, str) and record.name.startswith('system.'):
                parts = record.name.split('.')
                if len(parts) >= 2:
                    record.name = parts[-1]
            return super().format(record)

    # Setup console handler
    console = logging.StreamHandler(stream or sys.stdout)
    console.setLevel(log_level)
    console.setFormatter(MIRAFormatter('%(levelname)-8s [%(name)s] %(message)s'))

    logger.addHandler(console)
    logger.setLevel(log_level)
    logger.propagate = False

    # Silence noisy third-party loggers
    third_party_loggers = [
        'playwright',
        'asyncio',
        'openai',
        'httpx',
        'httpcore'
    ]
    for logger_name in third_party_loggers:
        third_party = logging.getLogger(logger_name)
        third_party.setLevel(logging.ERROR)
        third_party.propagate = False

    return logger

```

.\system\observability.py

```

# @file purpose: Observability module for browser-use that handles optional lmnr integration with debug
"""
Observability module for browser-use
This module provides observability decorators that optionally integrate with lmnr (Laminar) for tracing.

```

If `lmnr` is not installed, it provides no-op wrappers that accept the same parameters.

Features:

- Optional `lmnr` integration - works with or without `lmnr` installed
- Debug mode support - `observe_debug` only traces when in debug mode
- Full parameter compatibility with `lmnr` `observe` decorator
- No-op fallbacks when `lmnr` is unavailable

"""

```
import logging
import os
from collections.abc import Callable
from functools import wraps
from typing import Any, Literal, TypeVar, cast
```

```
logger = logging.getLogger(__name__)
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
# Type definitions
```

```
F = TypeVar('F', bound=Callable[..., Any])
```

```
# Check if we're in debug mode
```

```
def _is_debug_mode() -> bool:
```

```
    """Check if we're in debug mode based on environment variables or logging level."""
```

```
    _lmnr_debug_mode = os.getenv('LMNR_LOGGING_LEVEL', '').lower()
```

```
    if _lmnr_debug_mode == 'debug':
```

```
        # logger.info('Debug mode is enabled for observability')
```

```
        return True
```

```
    # logger.info('Debug mode is disabled for observability')
```

```
    return False
```

```
# Try to import lmnr observe
```

```
_LMNR_AVAILABLE = False
```

```
_lmnr_observe = None
```

```
try:
```

```
    from lmnr import observe as _lmnr_observe # type: ignore
```

```
    if os.environ.get('BROWSER_USE_VERBOSE_OBSERVABILITY', 'false').lower() == 'true':
```

```
        logger.debug('Lmnr is available for observability')
```

```
        _LMNR_AVAILABLE = True
```

```
    except ImportError:
```

```
        if os.environ.get('BROWSER_USE_VERBOSE_OBSERVABILITY', 'false').lower() == 'true':
```

```
            logger.debug('Lmnr is not available for observability')
```

```
        _LMNR_AVAILABLE = False
```

```
def _create_no_op_decorator(
```

```
    name: str | None = None,
```

```
    ignore_input: bool = False,
```

```
    ignore_output: bool = False,
```

```
    metadata: dict[str, Any] | None = None,
```

```
    **kwargs: Any,
```

```
) -> Callable[[F], F]:
```

```
    """Create a no-op decorator that accepts all lmnr observe parameters but does nothing."""
```

```
    import asyncio
```

```
    def decorator(func: F) -> F:
```

```
        if asyncio.iscoroutinefunction(func):
```

```
            @wraps(func)
```

```
            async def async_wrapper(*args, **kwargs):
```

```
                return await func(*args, **kwargs)
```

```
            return cast(F, async_wrapper)
```

```
        else:
```

```
            @wraps(func)
```

```

def sync_wrapper(*args, **kwargs):
    return func(*args, **kwargs)

return cast(F, sync_wrapper)

return decorator

def observe(
    name: str | None = None,
    ignore_input: bool = False,
    ignore_output: bool = False,
    metadata: dict[str, Any] | None = None,
    span_type: Literal['DEFAULT', 'LLM', 'TOOL'] = 'DEFAULT',
    **kwargs: Any,
) -> Callable[[F], F]:
    """
    Observability decorator that traces function execution when lmnr is available.

    This decorator will use lmnr's observe decorator if lmnr is installed,
    otherwise it will be a no-op that accepts the same parameters.

    Args:
        name: Name of the span/trace
        ignore_input: Whether to ignore function input parameters in tracing
        ignore_output: Whether to ignore function output in tracing
        metadata: Additional metadata to attach to the span
        **kwargs: Additional parameters passed to lmnr observe

    Returns:
        Decorated function that may be traced depending on lmnr availability

    Example:
        @observe(name="my_function", metadata={"version": "1.0"})
        def my_function(param1, param2):
            return param1 + param2
        """
    kwargs = {
        'name': name,
        'ignore_input': ignore_input,
        'ignore_output': ignore_output,
        'metadata': metadata,
        'span_type': span_type,
        'tags': ['observe', 'observe_debug'], # important: tags need to be created on laminar first
        **kwargs,
    }

    if _LMNR_AVAILABLE and _lmnr_observe:
        # Use the real lmnr observe decorator
        return cast(Callable[[F], F], _lmnr_observe(**kwargs))
    else:
        # Use no-op decorator
        return _create_no_op_decorator(**kwargs)

def observe_debug(
    name: str | None = None,
    ignore_input: bool = False,
    ignore_output: bool = False,
    metadata: dict[str, Any] | None = None,
    span_type: Literal['DEFAULT', 'LLM', 'TOOL'] = 'DEFAULT',
    **kwargs: Any,
) -> Callable[[F], F]:
    """
    Debug-only observability decorator that only traces when in debug mode.

    This decorator will use lmnr's observe decorator if both lmnr is installed
    AND we're in debug mode, otherwise it will be a no-op.

    Debug mode is determined by:
    - DEBUG environment variable set to 1/true/yes/on
    - BROWSER_USE_DEBUG environment variable set to 1/true/yes/on

```

■- Root logging level set to DEBUG or lower

■Args:

- name: Name of the span/trace
- ignore_input: Whether to ignore function input parameters in tracing
- ignore_output: Whether to ignore function output in tracing
- metadata: Additional metadata to attach to the span
- **kwargs: Additional parameters passed to `lmnr observe`

■Returns:

- Decorated function that may be traced only in debug mode

■Example:

```
■ @observe_debug(ignore_input=True, ignore_output=True, name="debug_function", metadata={"debug": True})
■ def debug_function(param1, param2):
■     return param1 + param2
```

■"""

■kwargs = {

■ 'name': name,

■ 'ignore_input': ignore_input,

■ 'ignore_output': ignore_output,

■ 'metadata': metadata,

■ 'span_type': span_type,

■ 'tags': ['observe_debug'], # important: tags need to be created on laminar first

■ **kwargs,

■}

■if _LMNR_AVAILABLE and _lmnr_observe and _is_debug_mode():

■ # Use the real lmnr observe decorator only in debug mode

■ return cast(Callable[[F], F], _lmnr_observe(**kwargs))

■else:

■ # Use no-op decorator (either not in debug mode or lmnr not available)

■ return _create_no_op_decorator(**kwargs)

Convenience functions for checking availability and debug status

def is_lmnr_available() -> bool:

■ """Check if lmnr is available for tracing."""

■ return _LMNR_AVAILABLE

def is_debug_mode() -> bool:

■ """Check if we're currently in debug mode."""

■ return _is_debug_mode()

def get_observability_status() -> dict[str, bool]:

■ """Get the current status of observability features."""

■ return {

■ 'lmnr_available': _LMNR_AVAILABLE,

■ 'debug_mode': _is_debug_mode(),

■ 'observe_active': _LMNR_AVAILABLE,

■ 'observe_debug_active': _LMNR_AVAILABLE and _is_debug_mode(),

■ }

.\systemutils.py

```
import asyncio
import logging
import os
import platform
import re
import signal
import time
from collections.abc import Callable, Coroutine
from fnmatch import fnmatch
from functools import cache, wraps
from pathlib import Path
```

```

from sys import stderr
from typing import Any, ParamSpec, TypeVar
from urllib.parse import urlparse

import httpx
from dotenv import load_dotenv

load_dotenv()

# Pre-compiled regex for URL detection - used in URL shortening
URL_PATTERN = re.compile(r'https?://[^\s<>"\']+|www\.[^\s<>"\']+[^\s<>"\']+\.[a-z]{2,}?(?:/[^\s<>"\']*)?')

logger = logging.getLogger(__name__)

# Lazy import for error types
# Use sentinel to avoid retrying import when package is not installed
_IMPORT_NOT_FOUND: type = type('_ImportNotFound', (), {})
_openai_bad_request_error: type | None = None
_groq_bad_request_error: type | None = None

def _get_openai_bad_request_error() -> type | None:
    """Lazy loader for OpenAI BadRequestError."""
    global _openai_bad_request_error
    if _openai_bad_request_error is None:
        try:
            from openai import BadRequestError

            _openai_bad_request_error = BadRequestError
        except ImportError:
            _openai_bad_request_error = _IMPORT_NOT_FOUND
    return _openai_bad_request_error if _openai_bad_request_error is not _IMPORT_NOT_FOUND else None

def _get_groq_bad_request_error() -> type | None:
    """Lazy loader for Groq BadRequestError."""
    global _groq_bad_request_error
    if _groq_bad_request_error is None:
        try:
            from groq import BadRequestError # type: ignore[import-not-found]

            _groq_bad_request_error = BadRequestError
        except ImportError:
            _groq_bad_request_error = _IMPORT_NOT_FOUND
    return _groq_bad_request_error if _groq_bad_request_error is not _IMPORT_NOT_FOUND else None

# Global flag to prevent duplicate exit messages
_exiting = False

# Define generic type variables for return type and parameters
R = TypeVar('R')
T = TypeVar('T')
P = ParamSpec('P')

class SignalHandler:
    """
    A modular and reusable signal handling system for managing SIGINT (Ctrl+C), SIGTERM,
    and other signals in asyncio applications.

    This class provides:
    - Configurable signal handling for SIGINT and SIGTERM
    - Support for custom pause/resume callbacks
    - Management of event loop state across signals
    - Standardized handling of first and second Ctrl+C presses
    - Cross-platform compatibility (with simplified behavior on Windows)
    """

    def __init__(
        self,

```

```

    loop: asyncio.AbstractEventLoop | None = None,
    pause_callback: Callable[[], None] | None = None,
    resume_callback: Callable[[], None] | None = None,
    custom_exit_callback: Callable[[], None] | None = None,
    exit_on_second_int: bool = True,
    interruptible_task_patterns: list[str] | None = None,
    ):
    """
    Initialize the signal handler.

    Args:
        loop: The asyncio event loop to use. Defaults to current event loop.
        pause_callback: Function to call when system is paused (first Ctrl+C)
        resume_callback: Function to call when system is resumed
        custom_exit_callback: Function to call on exit (second Ctrl+C or SIGTERM)
        exit_on_second_int: Whether to exit on second SIGINT (Ctrl+C)
        interruptible_task_patterns: List of patterns to match task names that should be
        canceled on first Ctrl+C (default: ['step', 'multi_act', 'get_next_action'])
    """
    self.loop = loop or asyncio.get_event_loop()
    self.pause_callback = pause_callback
    self.resume_callback = resume_callback
    self.custom_exit_callback = custom_exit_callback
    self.exit_on_second_int = exit_on_second_int
    self.interruptible_task_patterns = interruptible_task_patterns or ['step', 'multi_act', 'get_next_act.
    self.is_windows = platform.system() == 'Windows'

    ## Initialize loop state attributes
    self._initialize_loop_state()

    ## Store original signal handlers to restore them later if needed
    self.original_sigint_handler = None
    self.original_sigterm_handler = None

    def _initialize_loop_state(self) -> None:
        """Initialize loop state attributes used for signal handling."""
        setattr(self.loop, 'ctrl_c_pressed', False)
        setattr(self.loop, 'waiting_for_input', False)

    def register(self) -> None:
        """Register signal handlers for SIGINT and SIGTERM."""
        try:
            if self.is_windows:
                ## On Windows, use simple signal handling with immediate exit on Ctrl+C
                def windows_handler(sig, frame):
                    print('\n\n Got Ctrl+C. Exiting immediately on Windows...\n', file=stderr)
                    ## Run the custom exit callback if provided
                    if self.custom_exit_callback:
                        self.custom_exit_callback()
                    os._exit(0)

                self.original_sigint_handler = signal.signal(signal.SIGINT, windows_handler)
            else:
                ## On Unix-like systems, use asyncio's signal handling for smoother experience
                self.original_sigint_handler = self.loop.add_signal_handler(signal.SIGINT, lambda: self.sigint_hanc
                self.original_sigterm_handler = self.loop.add_signal_handler(signal.SIGTERM, lambda: self.sigterm_h

        except Exception:
            ## there are situations where signal handlers are not supported, e.g.
            ## - when running in a thread other than the main thread
            ## - some operating systems
            ## - inside jupyter notebooks
            pass

    def unregister(self) -> None:
        """Unregister signal handlers and restore original handlers if possible."""
        try:
            if self.is_windows:
                ## On Windows, just restore the original SIGINT handler
                if self.original_sigint_handler:
                    signal.signal(signal.SIGINT, self.original_sigint_handler)
            else:

```

```

##### On Unix-like systems, use asyncio's signal handler removal
#####self.loop.remove_signal_handler(signal.SIGINT)
#####self.loop.remove_signal_handler(signal.SIGTERM)

##### Restore original handlers if available
#####if self.original_sigint_handler:
#####    signal.signal(signal.SIGINT, self.original_sigint_handler)
#####if self.original_sigterm_handler:
#####    signal.signal(signal.SIGTERM, self.original_sigterm_handler)
#####except Exception as e:
#####    logger.warning(f'Error while unregistering signal handlers: {e}')

def _handle_second_ctrl_c(self) -> None:
    """
    Handle a second Ctrl+C press by performing cleanup and exiting.
    This is shared logic used by both sigint_handler and wait_for_resume.
    """
    global _exiting

    if not _exiting:
        _exiting = True

    # Call custom exit callback if provided
    if self.custom_exit_callback:
        try:
            self.custom_exit_callback()
        except Exception as e:
            logger.error(f'Error in exit callback: {e}')

    # Force immediate exit - more reliable than sys.exit()
    print('\n\n Got second Ctrl+C. Exiting immediately...\n', file=stderr)

    # Reset terminal to a clean state by sending multiple escape sequences
    # Order matters for terminal resets - we try different approaches

    # Reset terminal modes for both stdout and stderr
    print('\033[?25h', end='', flush=True, file=stderr) # Show cursor
    print('\033[?25h', end='', flush=True) # Show cursor

    # Reset text attributes and terminal modes
    print('\033[0m', end='', flush=True, file=stderr) # Reset text attributes
    print('\033[0m', end='', flush=True) # Reset text attributes

    # Disable special input modes that may cause arrow keys to output control chars
    print('\033[?1l', end='', flush=True, file=stderr) # Reset cursor keys to normal mode
    print('\033[?1l', end='', flush=True) # Reset cursor keys to normal mode

    # Disable bracketed paste mode
    print('\033[?2004l', end='', flush=True, file=stderr)
    print('\033[?2004l', end='', flush=True)

    # Carriage return helps ensure a clean line
    print('\r', end='', flush=True, file=stderr)
    print('\r', end='', flush=True)

    # these ^^ attempts dont work as far as we can tell
    # we still dont know what causes the broken input, if you know how to fix it, please let us know
    print('(tip: press [Enter] once to fix escape codes appearing after chrome exit)', file=stderr)

    os._exit(0)

def sigint_handler(self) -> None:
    """
    SIGINT (Ctrl+C) handler.

    First Ctrl+C: Cancel current step and pause.
    Second Ctrl+C: Exit immediately if exit_on_second_int is True.
    """
    global _exiting

    if _exiting:
        # Already exiting, force exit immediately

```



```

    os._exit(0)

    if getattr(self.loop, 'ctrl_c_pressed', False):
        # If we're in the waiting for input state, let the pause method handle it
        if getattr(self.loop, 'waiting_for_input', False):
            return

    # Second Ctrl+C - exit immediately if configured to do so
    if self.exit_on_second_int:
        self._handle_second_ctrl_c()

    # Mark that Ctrl+C was pressed
    setattr(self.loop, 'ctrl_c_pressed', True)

    # Cancel current tasks that should be interruptible - this is crucial for immediate pausing
    self._cancel_interruptible_tasks()

    # Call pause callback if provided - this sets the paused flag
    if self.pause_callback:
        try:
            self.pause_callback()
        except Exception as e:
            logger.error(f'Error in pause callback: {e}')

    # Log pause message after pause_callback is called (not before)
    print('-----', file=stderr)

    def sigterm_handler(self) -> None:
        """
        SIGTERM handler.

        Always exits the program completely.
        """
        global _exiting
        if not _exiting:
            _exiting = True
            print('\n\nSIGTERM received. Exiting immediately...\n\n', file=stderr)

    # Call custom exit callback if provided
    if self.custom_exit_callback:
        self.custom_exit_callback()

    os._exit(0)

    def _cancel_interruptible_tasks(self) -> None:
        """Cancel current tasks that should be interruptible."""
        current_task = asyncio.current_task(self.loop)
        for task in asyncio.all_tasks(self.loop):
            if task != current_task and not task.done():
                task_name = task.get_name() if hasattr(task, 'get_name') else str(task)
                # Cancel tasks that match certain patterns
                if any(pattern in task_name for pattern in self.interruptible_task_patterns):
                    logger.debug(f'Cancelling task: {task_name}')
                    task.cancel()
                # Add exception handler to silence "Task exception was never retrieved" warnings
                task.add_done_callback(lambda t: t.exception() if t.cancelled() else None)

    # Also cancel the current task if it's interruptible
    if current_task and not current_task.done():
        task_name = current_task.get_name() if hasattr(current_task, 'get_name') else str(current_task)
        if any(pattern in task_name for pattern in self.interruptible_task_patterns):
            logger.debug(f'Cancelling current task: {task_name}')
            current_task.cancel()

    def wait_for_resume(self) -> None:
        """
        Wait for user input to resume or exit.

        This method should be called after handling the first Ctrl+C.
        It temporarily restores default signal handling to allow catching
        a second Ctrl+C directly.
        """

```

```

    ## Set flag to indicate we're waiting for input
    setattr(self.loop, 'waiting_for_input', True)

    ## Temporarily restore default signal handling for SIGINT
    ## This ensures KeyboardInterrupt will be raised during input()
    original_handler = signal.getsignal(signal.SIGINT)
    try:
        signal.signal(signal.SIGINT, signal.default_int_handler)
    except ValueError:
        ## we are running in a thread other than the main thread
        ## or signal handlers are not supported for some other reason
        pass

    green = '\x1b[32;1m'
    red = '\x1b[31m'
    blink = '\033[33;5m'
    unblink = '\033[0m'
    reset = '\x1b[0m'

    try: # escape code is to blink the ...
        print(
            f'➡ Press {green}[Enter]{reset} to resume or {red}[Ctrl+C]{reset} again to exit{blink}...{unblink}'
            end='',
            flush=True,
            file=stderr,
        )
        input() # This will raise KeyboardInterrupt on Ctrl+C

        ## Call resume callback if provided
        if self.resume_callback:
            self.resume_callback()
        except KeyboardInterrupt:
            ## Use the shared method to handle second Ctrl+C
            self._handle_second_ctrl_c()
        finally:
            try:
                ## Restore our signal handler
                signal.signal(signal.SIGINT, original_handler)
                setattr(self.loop, 'waiting_for_input', False)
            except Exception:
                pass

    def reset(self) -> None:
        """Reset state after resuming."""
        ## Clear the flags
        if hasattr(self.loop, 'ctrl_c_pressed'):
            setattr(self.loop, 'ctrl_c_pressed', False)
        if hasattr(self.loop, 'waiting_for_input'):
            setattr(self.loop, 'waiting_for_input', False)

def time_execution_sync(additional_text: str = '') -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            start_time = time.time()
            result = func(*args, **kwargs)
            execution_time = time.time() - start_time
            ## Only log if execution takes more than 0.25 seconds
            if execution_time > 0.25:
                self_has_logger = args and getattr(args[0], 'logger', None)
                if self_has_logger:
                    logger = getattr(args[0], 'logger')
                elif 'agent' in kwargs:
                    logger = getattr(kwargs['agent'], 'logger')
                elif 'browser_session' in kwargs:
                    logger = getattr(kwargs['browser_session'], 'logger')
                else:
                    logger = logging.getLogger(__name__)
                logger.debug(f'➡ {additional_text.strip("-")} took {execution_time:.2f}s')
            return result
        return wrapper

```

■return decorator

```
def time_execution_async(
    ■additional_text: str = ''
) -> Callable[[Callable[P, Coroutine[Any, Any, R]]], Callable[P, Coroutine[Any, Any, R]]]:
    ■def decorator(func: Callable[P, Coroutine[Any, Any, R]]) -> Callable[P, Coroutine[Any, Any, R]]:
        ■@wraps(func)
        ■■async def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            ■■■start_time = time.time()
            ■■■result = await func(*args, **kwargs)
            ■■■execution_time = time.time() - start_time
            ■■■# Only log if execution takes more than 0.25 seconds to avoid spamming the logs
            ■■■# you can lower this threshold locally when you're doing dev work to performance optimize stuff
            ■■■if execution_time > 0.25:
                ■■■■self_has_logger = args and getattr(args[0], 'logger', None)
                ■■■■if self_has_logger:
                    ■■■■■logger = getattr(args[0], 'logger')
                    ■■■■elif 'agent' in kwargs:
                        ■■■■■logger = getattr(kwargs['agent'], 'logger')
                    ■■■■elif 'browser_session' in kwargs:
                        ■■■■■logger = getattr(kwargs['browser_session'], 'logger')
                    ■■■■else:
                        ■■■■■logger = logging.getLogger(__name__)
                ■■■■logger.debug(f'■ {additional_text.strip("-")} took {execution_time:.2f}s')
            ■■■return result
```

■return wrapper

■return decorator

```
def singleton(cls):
    ■instance = [None]

    ■def wrapper(*args, **kwargs):
        ■■if instance[0] is None:
            ■■■instance[0] = cls(*args, **kwargs)
        ■■return instance[0]
```

■return wrapper

```
def check_env_variables(keys: list[str], any_or_all=all) -> bool:
    ■"""Check if all required environment variables are set"""
    ■return any_or_all(os.getenv(key, '').strip() for key in keys)
```

```
def is_unsafe_pattern(pattern: str) -> bool:
    ■"""
    ■Check if a domain pattern has complex wildcards that could match too many domains.
```

■Args:

■pattern: The domain pattern to check

■Returns:

■bool: True if the pattern has unsafe wildcards, False otherwise

■"""

■# Extract domain part if there's a scheme

■if '://' in pattern:

■_, pattern = pattern.split('://', 1)

■# Remove safe patterns (*.domain and domain.*)

■bare_domain = pattern.replace('.', '*').replace('*', '')

■# If there are still wildcards, it's potentially unsafe

■return '*' in bare_domain

```
def is_new_tab_page(url: str) -> bool:
```

■"""

■Check if a URL is a new tab page (about:blank, chrome://new-tab-page, or chrome://newtab).

■Args:
■url: The URL to check

■Returns:
■bool: True if the URL is a new tab page, False otherwise
■"""

■return url in ('about:blank', 'chrome://new-tab-page/', 'chrome://new-tab-page', 'chrome://newtab/', 'chrome://newtab')

def match_url_with_domain_pattern(url: str, domain_pattern: str, log_warnings: bool = False) -> bool:

■"""

■Check if a URL matches a domain pattern. SECURITY CRITICAL.

■Supports optional glob patterns and schemes:

■- *.example.com will match sub.example.com and example.com

■- *google.com will match google.com, agoogle.com, and www.google.com

■- http*://example.com will match http://example.com, https://example.com

■- chrome-extension://* will match chrome-extension://aaaaaaaaaaaa and chrome-extension://bbbbbbbbbbbbbb

■When no scheme is specified, https is used by default for security.

■For example, 'example.com' will match 'https://example.com' but not 'http://example.com'.

■Note: New tab pages (about:blank, chrome://new-tab-page) must be handled at the callsite, not inside this function.

■Args:

■url: The URL to check

■domain_pattern: Domain pattern to match against

■log_warnings: Whether to log warnings about unsafe patterns

■Returns:

■bool: True if the URL matches the pattern, False otherwise

■"""

■try:

■# Note: new tab pages should be handled at the callsite, not here

■if is_new_tab_page(url):

■return False

■parsed_url = urlparse(url)

■# Extract only the hostname and scheme components

■scheme = parsed_url.scheme.lower() if parsed_url.scheme else ''

■domain = parsed_url.hostname.lower() if parsed_url.hostname else ''

■if not scheme or not domain:

■return False

■# Normalize the domain pattern

■domain_pattern = domain_pattern.lower()

■# Handle pattern with scheme

■if '://' in domain_pattern:

■pattern_scheme, pattern_domain = domain_pattern.split('://', 1)

■else:

■pattern_scheme = 'https' # Default to matching only https for security

■pattern_domain = domain_pattern

■# Handle port in pattern (we strip ports from patterns since we already

■# extracted only the hostname from the URL)

■if ':' in pattern_domain and not pattern_domain.startswith(':'):

■pattern_domain = pattern_domain.split(':', 1)[0]

■# If scheme doesn't match, return False

■if not fnmatch(scheme, pattern_scheme):

■return False

■# Check for exact match

■if pattern_domain == '*' or domain == pattern_domain:

■return True

■# Handle glob patterns

■if '*' in pattern_domain:

■# Check for unsafe glob patterns

```

##### First, check for patterns like *.*.domain which are unsafe
#####if pattern_domain.count('*.') > 1 or pattern_domain.count('*.') > 1:
#####if log_warnings:
#####logger = logging.getLogger(__name__)
#####logger.error(f'##### Multiple wildcards in pattern=[{domain_pattern}] are not supported')
#####return False # Don't match unsafe patterns

##### Check for wildcards in TLD part (example.*)
#####if pattern_domain.endswith('*.'):
#####if log_warnings:
#####logger = logging.getLogger(__name__)
#####logger.error(f'##### Wildcard TLDs like in pattern=[{domain_pattern}] are not supported for security')
#####return False # Don't match unsafe patterns

##### Then check for embedded wildcards
#####bare_domain = pattern_domain.replace('*.', '')
#####if '*' in bare_domain:
#####if log_warnings:
#####logger = logging.getLogger(__name__)
#####logger.error(f'##### Only *.domain style patterns are supported, ignoring pattern=[{domain_pattern}]')
#####return False # Don't match unsafe patterns

##### Special handling so that *.google.com also matches bare google.com
#####if pattern_domain.startswith('*.'):
#####parent_domain = pattern_domain[2:]
#####if domain == parent_domain or fnmatch(domain, parent_domain):
#####return True

##### Normal case: match domain against pattern
#####if fnmatch(domain, pattern_domain):
#####return True

#####return False
#####except Exception as e:
#####logger = logging.getLogger(__name__)
#####logger.error(f'##### Error matching URL {url} with pattern {domain_pattern}: {type(e).__name__}: {e}')
#####return False

def merge_dicts(a: dict, b: dict, path: tuple[str, ...] = ()):
    for key in b:
        if key in a:
            if isinstance(a[key], dict) and isinstance(b[key], dict):
                merge_dicts(a[key], b[key], path + (str(key),))
            elif isinstance(a[key], list) and isinstance(b[key], list):
                a[key] = a[key] + b[key]
            elif a[key] != b[key]:
                raise Exception('Conflict at ' + '.'.join(path + (str(key),)))
            else:
                a[key] = b[key]
    return a

@cache
def get_browser_use_version() -> str:
    """Get the browser-use package version using the same logic as Agent._set_browser_use_version_and_source"""
    try:
        package_root = Path(__file__).parent.parent
        pyproject_path = package_root / 'pyproject.toml'

        ##### Try to read version from pyproject.toml
        if pyproject_path.exists():
            import re

            with open(pyproject_path, encoding='utf-8') as f:
                content = f.read()
                match = re.search(r'version\s*=\s*"([^\"]+)"', content)
                if match:
                    version = f'{match.group(1)}'
                    os.environ['LIBRARY_VERSION'] = version # used by bubus event_schema so all Event schemas include
                    return version
            ##### If pyproject.toml doesn't exist, try getting version from pip

```

```

from importlib.metadata import version as get_version

version = str(get_version('browser-use'))
os.environ['LIBRARY_VERSION'] = version
return version

except Exception as e:
    logger.debug(f'Error detecting browser-use version: {type(e).__name__}: {e}')
    return 'unknown'

async def check_latest_browser_use_version() -> str | None:
    """Check the latest version of browser-use from PyPI asynchronously.

    Returns:
    The latest version string if successful, None if failed
    """
    try:
        async with httpx.AsyncClient(timeout=3.0) as client:
            response = await client.get('https://pypi.org/pypi/browser-use/json')
            if response.status_code == 200:
                data = response.json()
                return data['info']['version']
            except Exception:
                # Silently fail - we don't want to break agent startup due to network issues
                pass
    return None

@cache
def get_git_info() -> dict[str, str] | None:
    """Get git information if installed from git repository"""
    try:
        import subprocess

        package_root = Path(__file__).parent.parent
        git_dir = package_root / '.git'
        if not git_dir.exists():
            return None

        # Get git commit hash
        commit_hash = (
            subprocess.check_output(['git', 'rev-parse', 'HEAD'], cwd=package_root, stderr=subprocess.DEVNULL).d
        )

        # Get git branch
        branch = (
            subprocess.check_output(['git', 'rev-parse', '--abbrev-ref', 'HEAD'], cwd=package_root, stderr=subprocess
            ).decode()
            .strip()
        )

        # Get remote URL
        remote_url = (
            subprocess.check_output(['git', 'config', '--get', 'remote.origin.url'], cwd=package_root, stderr=subprocess
            ).decode()
            .strip()
        )

        # Get commit timestamp
        commit_timestamp = (
            subprocess.check_output(['git', 'show', '-s', '--format=%ci', 'HEAD'], cwd=package_root, stderr=subprocess
            ).decode()
            .strip()
        )

        return {'commit_hash': commit_hash, 'branch': branch, 'remote_url': remote_url, 'commit_timestamp': commit_timestamp}
    except Exception as e:
        logger.debug(f'Error getting git info: {type(e).__name__}: {e}')
        return None
def _log_pretty_path(path: str | Path | None) -> str:
    """Pretty-print a path, shorten home dir to ~ and cwd to ."""

```

```

■if not path or not str(path).strip():
■return '' # always falsy in -> falsy out so it can be used in ternaries

■# dont print anything thats not a path
■if not isinstance(path, (str, Path)):
■# no other types are safe to just str(path) and log to terminal unless we know what they are
■# e.g. what if we get storage_date=dict | Path and the dict version could contain real cookies
■return f'<{type(path).__name__}>'

■# replace home dir and cwd with ~ and .
■pretty_path = str(path).replace(str(Path.home()), '~').replace(str(Path.cwd()).resolve(), '.')

■# wrap in quotes if it contains spaces
■if pretty_path.strip() and ' ' in pretty_path:
■pretty_path = f'"{pretty_path}"'

■return pretty_path

def _log_pretty_url(s: str, max_len: int | None = 22) -> str:
■"""Truncate/pretty-print a URL with a maximum length, removing the protocol and www. prefix"""
■s = s.replace('https://', '').replace('http://', '').replace('www.', '')
■if max_len is not None and len(s) > max_len:
■return s[:max_len] + '...'
■return s

def create_task_with_error_handling(
■coro: Coroutine[Any, Any, T],
■*,
■name: str | None = None,
■logger_instance: logging.Logger | None = None,
■suppress_exceptions: bool = False,
) -> asyncio.Task[T]:
■"""
■Create an asyncio task with proper exception handling to prevent "Task exception was never retrieved" v

■Args:
■coro: The coroutine to wrap in a task
■name: Optional name for the task (useful for debugging)
■logger_instance: Optional logger instance to use. If None, uses module logger.
■suppress_exceptions: If True, logs exceptions at ERROR level. If False, logs at WARNING level
■and exceptions remain retrievable via task.exception() if the caller awaits the task.
■Default False.

■Returns:
■asyncio.Task: The created task with exception handling callback

■Example:
■# Fire-and-forget with suppressed exceptions
■create_task_with_error_handling(some_async_function(), name="my_task", suppress_exceptions=True)

■# Task with retrievable exceptions (if you plan to await it)
■task = create_task_with_error_handling(critical_function(), name="critical")
■result = await task # Will raise the exception if one occurred
■"""
■task = asyncio.create_task(coro, name=name)
■log = logger_instance or logger

■def _handle_task_exception(t: asyncio.Task[T]) -> None:
■"""Callback to handle task exceptions"""
■exc_to_raise = None
■try:
■# This will raise if the task had an exception
■exc = t.exception()
■if exc is not None:
■task_name = t.get_name() if hasattr(t, 'get_name') else 'unnamed'
■if suppress_exceptions:
■log.error(f'Exception in background task [{task_name}]: {type(exc).__name__}: {exc}', exc_info=exc)
■else:
■# Log at warning level then mark for re-raising
■log.warning(

```

```

        """If Exception in background task [{task_name}]: {type(exc).__name__}: {exc}',
        exc_info=exc,
        )
        exc_to_raise = exc
    except asyncio.CancelledError:
        """# Task was cancelled, this is normal behavior
        pass
    except Exception as e:
        """# Catch any other exception during exception handling (e.g., t.exception() itself failing)
        task_name = t.get_name() if hasattr(t, 'get_name') else 'unnamed'
        log.error(f'Error handling exception in task [{task_name}]: {type(e).__name__}: {e}')

    """# Re-raise outside the try-except block so it propagates to the event loop
    if exc_to_raise is not None:
        raise exc_to_raise

    task.add_done_callback(_handle_task_exception)
    return task

def sanitize_surrogates(text: str) -> str:
    """Remove surrogate characters that can't be encoded in UTF-8.

    Surrogate pairs (U+D800 to U+DFFF) are invalid in UTF-8 when unpaired.
    These often appear in DOM content from mathematical symbols or emojis.

    Args:
        text: The text to sanitize

    Returns:
        Text with surrogate characters removed
    """
    return text.encode('utf-8', errors='ignore').decode('utf-8')

```

.system__init__.py

```

import os
from typing import TYPE_CHECKING

from system.logging_config import setup_logging

# Only set up logging if not in MCP mode or if explicitly requested
if os.environ.get('BROWSER_USE_SETUP_LOGGING', 'true').lower() != 'false':
    import logging
    from system.config import CONFIG

    """# Get log level from config/environment (default to INFO)
    log_level = logging.DEBUG if CONFIG.IN_DOCKER else logging.INFO

    """# Set up logging with file handlers if specified
    logger = setup_logging(log_level=log_level)
    else:
        import logging

    logger = logging.getLogger('system')

# Monkeypatch BaseSubprocessTransport.__del__ to handle closed event loops gracefully
from asyncio import base_subprocess

_original_del = base_subprocess.BaseSubprocessTransport.__del__

def _patched_del(self):
    """Patched __del__ that handles closed event loops without throwing noisy red-herring errors like RuntimeWarning: Task > was never awaited
    try:
        """# Check if the event loop is closed before calling the original
        if hasattr(self, '_loop') and self._loop and self._loop.is_closed():
            """# Event loop is closed, skip cleanup that requires the loop

```



```

    return
    _original_del(self)
except RuntimeError as e:
    if 'Event loop is closed' in str(e):
        # Silently ignore this specific error
        pass
    else:
        raise

```

```

base_subprocess.BaseSubprocessTransport.__del__ = _patched_del

```

```

# Type stubs for lazy imports - fixes linter warnings

```

```

if TYPE_CHECKING:

```

```

    from system.agent.prompts import SystemPrompt
    from system.agent.service import Agent

```

```

    # from system.agent.service import Agent
    from system.agent.views import ActionModel, ActionResult, AgentHistoryList
    from system.browser import BrowserProfile, BrowserSession
    from system.browser import BrowserSession as Browser
    from system.code_use.service import CodeAgent
    from system.dom.service import DomService
    from system.llm import models
    from system.llm.azure.chat import ChatAzureOpenAI
    from system.llm.system.chat import ChatBrowserUse
    from system.llm.google.chat import ChatGoogle
    from system.llm.groq.chat import ChatGroq
    from system.llm.mistral.chat import ChatMistral
    from system.llm.oci_raw.chat import ChatOCIRaw
    from system.llm.ollama.chat import ChatOllama
    from system.llm.openai.chat import ChatOpenAI
    from system.llm.vercel.chat import ChatVercel
    from system.sandbox import sandbox
    from system.tools.service import Controller, Tools

```

```

    # Lazy imports mapping - only import when actually accessed

```

```

    _LAZY_IMPORTS = {

```

```

        # Agent service (heavy due to dependencies)
        # 'Agent': ('system.agent.service', 'Agent'),
        # Code-use agent (Jupyter notebook-like execution)
        # 'CodeAgent': ('system.code_use.service', 'CodeAgent'),
        # 'Agent': ('system.agent.service', 'Agent'),
        # System prompt (moderate weight due to agent.views imports)
        # 'SystemPrompt': ('system.agent.prompts', 'SystemPrompt'),
        # Agent views (very heavy - over 1 second!)
        # 'ActionModel': ('system.agent.views', 'ActionModel'),
        # 'ActionResult': ('system.agent.views', 'ActionResult'),
        # 'AgentHistoryList': ('system.agent.views', 'AgentHistoryList'),
        # 'BrowserSession': ('system.browser', 'BrowserSession'),
        # 'Browser': ('system.browser', 'BrowserSession'), # Alias for BrowserSession
        # 'BrowserProfile': ('system.browser', 'BrowserProfile'),
        # Tools (moderate weight)
        # 'Tools': ('system.tools.service', 'Tools'),
        # 'Controller': ('system.tools.service', 'Controller'), # alias
        # DOM service (moderate weight)
        # 'DomService': ('system.dom.service', 'DomService'),
        # Chat models (very heavy imports)
        # 'ChatOpenAI': ('system.llm.openai.chat', 'ChatOpenAI'),
        # 'ChatGoogle': ('system.llm.google.chat', 'ChatGoogle'),
        # 'ChatBrowserUse': ('system.llm.system.chat', 'ChatBrowserUse'),
        # 'ChatGroq': ('system.llm.groq.chat', 'ChatGroq'),
        # 'ChatMistral': ('system.llm.mistral.chat', 'ChatMistral'),
        # 'ChatAzureOpenAI': ('system.llm.azure.chat', 'ChatAzureOpenAI'),
        # 'ChatOCIRaw': ('system.llm.oci_raw.chat', 'ChatOCIRaw'),
        # 'ChatOllama': ('system.llm.ollama.chat', 'ChatOllama'),
        # 'ChatVercel': ('system.llm.vercel.chat', 'ChatVercel'),
        # LLM models module
        # 'models': ('system.llm.models', None),
        # Sandbox execution
        # 'sandbox': ('system.sandbox', 'sandbox'),
    }

```

```

}

def __getattr__(name: str):
    """Lazy import mechanism - only import modules when they're actually accessed."""
    if name in _LAZY_IMPORTS:
        module_path, attr_name = _LAZY_IMPORTS[name]
        try:
            from importlib import import_module

            module = import_module(module_path)
            if attr_name is None:
                # For modules like 'models', return the module itself
                attr = module
            else:
                attr = getattr(module, attr_name)
            # Cache the imported attribute in the module's globals
            globals()[name] = attr
            return attr
        except ImportError as e:
            raise ImportError(f'Failed to import {name} from {module_path}: {e}') from e

    raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

__all__ = [
    'Agent',
    'CodeAgent',
    # 'CodeAgent',
    'BrowserSession',
    'Browser', # Alias for BrowserSession
    'BrowserProfile',
    'Controller',
    'DomService',
    'SystemPrompt',
    'ActionResult',
    'ActionModel',
    'AgentHistoryList',
    # Chat models
    'ChatOpenAI',
    'ChatGoogle',
    'ChatBrowserUse',
    'ChatGrok',
    'ChatMistral',
    'ChatAzureOpenAI',
    'ChatOCIRaw',
    'ChatOllama',
    'ChatVercel',
    'Tools',
    'Controller',
    # LLM models module
    'models',
    # Sandbox execution
    'sandbox',
]

```

.lsystem\actor\element.py

```

"""Element class for element operations."""

import asyncio
from typing import TYPE_CHECKING, Literal, Union

from cdp_use.client import logger
from typing_extensions import TypedDict

if TYPE_CHECKING:
    from cdp_use.cdp.dom.commands import (

```

```

    DescribeNodeParameters,
    FocusParameters,
    GetAttributesParameters,
    GetBoxModelParameters,
    PushNodesByBackendIdsToFrontendParameters,
    RequestChildNodesParameters,
    ResolveNodeParameters,
)
from cdp_use.cdp.input.commands import (
    DispatchMouseEventParameters,
)
from cdp_use.cdp.input.types import MouseButton
from cdp_use.cdp.page.commands import CaptureScreenshotParameters
from cdp_use.cdp.page.types import Viewport
from cdp_use.cdp.runtime.commands import CallFunctionOnParameters

from system.browser.session import BrowserSession

# Type definitions for element operations
ModifierType = Literal['Alt', 'Control', 'Meta', 'Shift']

class Position(TypedDict):
    """2D position coordinates."""

    x: float
    y: float

class BoundingBox(TypedDict):
    """Element bounding box with position and dimensions."""

    x: float
    y: float
    width: float
    height: float

class ElementInfo(TypedDict):
    """Basic information about a DOM element."""

    backendNodeId: int
    nodeId: int | None
    nodeName: str
    nodeType: int
    nodeValue: str | None
    attributes: dict[str, str]
    boundingBox: BoundingBox | None
    error: str | None

class Element:
    """Element operations using BackendNodeId."""

    def __init__(
        self,
        browser_session: 'BrowserSession',
        backend_node_id: int,
        session_id: str | None = None,
    ):
        self._browser_session = browser_session
        self._client = browser_session.cdp_client
        self._backend_node_id = backend_node_id
        self._session_id = session_id

    async def _get_node_id(self) -> int:
        """Get DOM node ID from backend node ID."""
        params: 'PushNodesByBackendIdsToFrontendParameters' = {'backendNodeIds': [self._backend_node_id]}
        result = await self._client.send.DOM.pushNodesByBackendIdsToFrontend(params, session_id=self._session_id)
        return result['nodeIds'][0]

    async def _get_remote_object_id(self) -> str | None:

```

```

    """Get remote object ID for this element."""
    node_id = await self._get_node_id()
    params: 'ResolveNodeParameters' = {'nodeId': node_id}
    result = await self._client.send.DOM.resolveNode(params, session_id=self._session_id)
    object_id = result['object'].get('objectId', None)

    if not object_id:
        return None
    return object_id

    async def click(
        self,
        button: 'MouseButton' = 'left',
        click_count: int = 1,
        modifiers: list[ModifierType] | None = None,
    ) -> None:
        """Click the element using the advanced watchdog implementation."""

        try:
            # Get viewport dimensions for visibility checks
            layout_metrics = await self._client.send.Page.getLayoutMetrics(session_id=self._session_id)
            viewport_width = layout_metrics['layoutViewport']['clientWidth']
            viewport_height = layout_metrics['layoutViewport']['clientHeight']

            # Try multiple methods to get element geometry
            quads = []

            # Method 1: Try DOM.getContentQuads first (best for inline elements and complex layouts)
            try:
                content_quads_result = await self._client.send.DOM.getContentQuads(
                    params={'backendNodeId': self._backend_node_id}, session_id=self._session_id
                )
                if 'quads' in content_quads_result and content_quads_result['quads']:
                    quads = content_quads_result['quads']
            except Exception:
                pass

            # Method 2: Fall back to DOM.getBoxModel
            if not quads:
                try:
                    box_model = await self._client.send.DOM.getBoxModel(
                        params={'backendNodeId': self._backend_node_id}, session_id=self._session_id
                    )
                    if 'model' in box_model and 'content' in box_model['model']:
                        content_quad = box_model['model']['content']
                        if len(content_quad) >= 8:
                            # Convert box model format to quad format
                            quads = [
                                [
                                    content_quad[0],
                                    content_quad[1], # x1, y1
                                    content_quad[2],
                                    content_quad[3], # x2, y2
                                    content_quad[4],
                                    content_quad[5], # x3, y3
                                    content_quad[6],
                                    content_quad[7], # x4, y4
                                ]
                            ]
                except Exception:
                    pass

            # Method 3: Fall back to JavaScript getBoundingClientRect
            if not quads:
                try:
                    result = await self._client.send.DOM.resolveNode(
                        params={'backendNodeId': self._backend_node_id}, session_id=self._session_id
                    )
                    if 'object' in result and 'objectId' in result['object']:
                        object_id = result['object']['objectId']

                # Get bounding rect via JavaScript

```

```

#####bounds_result = await self._client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': """
#####function() {
#####const rect = this.getBoundingClientRect();
#####return {
#####x: rect.left,
#####y: rect.top,
#####width: rect.width,
#####height: rect.height
#####};
#####}
#####""",
#####'objectId': object_id,
#####'returnByValue': True,
#####},
#####session_id=self._session_id,
#####)

#####if 'result' in bounds_result and 'value' in bounds_result['result']:
#####rect = bounds_result['result']['value']
###### Convert rect to quad format
#####x, y, w, h = rect['x'], rect['y'], rect['width'], rect['height']
#####quads = [
#####[
#####x,
#####y, # top-left
#####x + w,
#####y, # top-right
#####x + w,
#####y + h, # bottom-right
#####x,
#####y + h, # bottom-left
#####]
#####]
#####except Exception:
#####pass

###### If we still don't have quads, fall back to JS click
#####if not quads:
#####try:
#####result = await self._client.send.DOM.resolveNode(
#####params={'backendNodeId': self._backend_node_id}, session_id=self._session_id
#####)
#####if 'object' not in result or 'objectId' not in result['object']:
#####raise Exception('Failed to find DOM element based on backendNodeId, maybe page content changed?')
#####object_id = result['object']['objectId']

#####await self._client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': 'function() { this.click(); }',
#####'objectId': object_id,
#####},
#####session_id=self._session_id,
#####)
#####await asyncio.sleep(0.05)
#####return
#####except Exception as js_e:
#####raise Exception(f'Failed to click element: {js_e}')

###### Find the largest visible quad within the viewport
#####best_quad = None
#####best_area = 0

#####for quad in quads:
#####if len(quad) < 8:
#####continue

###### Calculate quad bounds
#####xs = [quad[i] for i in range(0, 8, 2)]
#####ys = [quad[i] for i in range(1, 8, 2)]
#####min_x, max_x = min(xs), max(xs)

```

```

min_y, max_y = min(ys), max(ys)

# Check if quad intersects with viewport
if max_x < 0 or max_y < 0 or min_x > viewport_width or min_y > viewport_height:
    continue # Quad is completely outside viewport

# Calculate visible area (intersection with viewport)
visible_min_x = max(0, min_x)
visible_max_x = min(viewport_width, max_x)
visible_min_y = max(0, min_y)
visible_max_y = min(viewport_height, max_y)

visible_width = visible_max_x - visible_min_x
visible_height = visible_max_y - visible_min_y
visible_area = visible_width * visible_height

if visible_area > best_area:
    best_area = visible_area
    best_quad = quad

if not best_quad:
    # No visible quad found, use the first quad anyway
    best_quad = quads[0]

# Calculate center point of the best quad
center_x = sum(best_quad[i] for i in range(0, 8, 2)) / 4
center_y = sum(best_quad[i] for i in range(1, 8, 2)) / 4

# Ensure click point is within viewport bounds
center_x = max(0, min(viewport_width - 1, center_x))
center_y = max(0, min(viewport_height - 1, center_y))

# Scroll element into view
try:
    await self._client.send.DOM.scrollToViewIfNeeded(
        params={'backendNodeId': self._backend_node_id, 'session_id':self._session_id}
    )
    await asyncio.sleep(0.05) # Wait for scroll to complete
except Exception:
    pass

# Calculate modifier bitmask for CDP
modifier_value = 0
if modifiers:
    modifier_map = {'Alt': 1, 'Control': 2, 'Meta': 4, 'Shift': 8}
    for mod in modifiers:
        modifier_value |= modifier_map.get(mod, 0)

# Perform the click using CDP
try:
    # Move mouse to element
    await self._client.send.Input.dispatchMouseEvent(
        params={
            'type': 'mouseMoved',
            'x': center_x,
            'y': center_y,
        },
        session_id=self._session_id,
    )
    await asyncio.sleep(0.05)

# Mouse down
try:
    await asyncio.wait_for(
        self._client.send.Input.dispatchMouseEvent(
            params={
                'type': 'mousePressed',
                'x': center_x,
                'y': center_y,
                'button': button,
                'clickCount': click_count,
                'modifiers': modifier_value,
            }
        )
    )

```

```

        },
        session_id=self._session_id,
    ),
    timeout=1.0, # 1 second timeout for mousePressed
)
    await asyncio.sleep(0.08)
except TimeoutError:
    pass # Don't sleep if we timed out

# Mouse up
try:
    await asyncio.wait_for(
        self._client.send.Input.dispatchMouseEvent(
            params={
                'type': 'mouseReleased',
                'x': center_x,
                'y': center_y,
                'button': button,
                'clickCount': click_count,
                'modifiers': modifier_value,
            },
            session_id=self._session_id,
        ),
        timeout=3.0, # 3 second timeout for mouseReleased
    )
except TimeoutError:
    pass

except Exception as e:
    # Fall back to JavaScript click via CDP
    try:
        result = await self._client.send.DOM.resolveNode(
            params={'backendNodeId': self._backend_node_id, session_id=self._session_id}
        )
        if 'object' not in result or 'objectId' not in result['object']:
            raise Exception('Failed to find DOM element based on backendNodeId, maybe page content changed?')
        object_id = result['object']['objectId']

        await self._client.send.Runtime.callFunctionOn(
            params={
                'functionDeclaration': 'function() { this.click(); }',
                'objectId': object_id,
            },
            session_id=self._session_id,
        )
        await asyncio.sleep(0.1)
        return
    except Exception as js_e:
        raise Exception(f'Failed to click element: {e}')

except Exception as e:
    # Extract key element info for error message
    raise RuntimeError(f'Failed to click element: {e}')

async def fill(self, value: str, clear: bool = True) -> None:
    """Fill the input element using proper CDP methods with improved focus handling."""
    try:
        # Use the existing CDP client and session
        cdp_client = self._client
        session_id = self._session_id
        backend_node_id = self._backend_node_id

        # Track coordinates for metadata
        input_coordinates = None

        # Scroll element into view
        try:
            await cdp_client.send.DOM.scrollToViewIfNeeded(params={'backendNodeId': backend_node_id, session_id=self._session_id},
                timeout=0.01)
        except Exception as e:
            logger.warning(f'Failed to scroll element into view: {e}')
        # Get object ID for the element

```

```

result = await cdp_client.send.DOM.resolveNode(
    params={'backendNodeId': backend_node_id,
    session_id=session_id,
    })
    if 'object' not in result or 'objectId' not in result['object']:
        raise RuntimeError('Failed to get object ID for element')
    object_id = result['object']['objectId']

    # Get element coordinates for focus
    try:
        bounds_result = await cdp_client.send.Runtime.callFunctionOn(
            params={
                'functionDeclaration': 'function() { return this.getBoundingClientRect(); }',
                'objectId': object_id,
                'returnByValue': True,
            },
            session_id=session_id,
        )
        if bounds_result.get('result', {}).get('value'):
            bounds = bounds_result['result']['value'] # type: ignore
            center_x = bounds['x'] + bounds['width'] / 2
            center_y = bounds['y'] + bounds['height'] / 2
            input_coordinates = {'input_x': center_x, 'input_y': center_y}
            logger.debug(f'Using element coordinates: x={center_x:.1f}, y={center_y:.1f}')
        except Exception as e:
            logger.debug(f'Could not get element coordinates: {e}')

    # Ensure session_id is not None
    if session_id is None:
        raise RuntimeError('Session ID is required for fill operation')

    # Step 1: Focus the element
    focused_successfully = await self._focus_element_simple(
        backend_node_id=backend_node_id,
        object_id=object_id,
        cdp_client=cdp_client,
        session_id=session_id,
        input_coordinates=input_coordinates,
    )

    # Step 2: Clear existing text if requested
    if clear:
        cleared_successfully = await self._clear_text_field(
            object_id=object_id, cdp_client=cdp_client, session_id=session_id
        )
        if not cleared_successfully:
            logger.warning('Text field clearing failed, typing may append to existing text')

    # Step 3: Type the text character by character using proper human-like key events
    logger.debug(f'Typing text character by character: "{value}"')

    for i, char in enumerate(value):
        # Handle newline characters as Enter key
        if char == '\n':
            # Send proper Enter key sequence
            await cdp_client.send.Input.dispatchKeyEvent(
                params={
                    'type': 'keyDown',
                    'key': 'Enter',
                    'code': 'Enter',
                    'windowsVirtualKeyCode': 13,
                },
                session_id=session_id,
            )

        # Small delay to emulate human typing speed
        await asyncio.sleep(0.001)

        # Send char event with carriage return
        await cdp_client.send.Input.dispatchKeyEvent(
            params={
                'type': 'char',

```



```

        'text': '\r',
        'key': 'Enter',
    },
    session_id=session_id,
)

# Send keyUp event
await cdp_client.send.Input.dispatchKeyEvent(
    params={
        'type': 'keyUp',
        'key': 'Enter',
        'code': 'Enter',
        'windowsVirtualKeyCode': 13,
    },
    session_id=session_id,
)
else:
    # Handle regular characters
    # Get proper modifiers, VK code, and base key for the character
    modifiers, vk_code, base_key = self._get_char_modifiers_and_vk(char)
    key_code = self._get_key_code_for_char(base_key)

    # Step 1: Send keyDown event (NO text parameter)
    await cdp_client.send.Input.dispatchKeyEvent(
        params={
            'type': 'keyDown',
            'key': base_key,
            'code': key_code,
            'modifiers': modifiers,
            'windowsVirtualKeyCode': vk_code,
        },
        session_id=session_id,
    )

    # Small delay to emulate human typing speed
    await asyncio.sleep(0.001)

    # Step 2: Send char event (WITH text parameter) - this is crucial for text input
    await cdp_client.send.Input.dispatchKeyEvent(
        params={
            'type': 'char',
            'text': char,
            'key': char,
        },
        session_id=session_id,
    )

    # Step 3: Send keyUp event (NO text parameter)
    await cdp_client.send.Input.dispatchKeyEvent(
        params={
            'type': 'keyUp',
            'key': base_key,
            'code': key_code,
            'modifiers': modifiers,
            'windowsVirtualKeyCode': vk_code,
        },
        session_id=session_id,
    )

    # Add 18ms delay between keystrokes
    await asyncio.sleep(0.018)

except Exception as e:
    raise Exception(f'Failed to fill element: {str(e)}')

async def hover(self) -> None:
    """Hover over the element."""
    box = await self.get_bounding_box()
    if not box:
        raise RuntimeError('Element is not visible or has no bounding box')

    x = box['x'] + box['width'] / 2

```

```

    y = box['y'] + box['height'] / 2

    params: 'DispatchMouseEventParameters' = {'type': 'mouseMoved', 'x': x, 'y': y}
    await self._client.send.Input.dispatchMouseEvent(params, session_id=self._session_id)

    async def focus(self) -> None:
        """Focus the element."""
        node_id = await self._get_node_id()
        params: 'FocusParameters' = {'nodeId': node_id}
        await self._client.send.DOM.focus(params, session_id=self._session_id)

    async def check(self) -> None:
        """Check or uncheck a checkbox/radio button."""
        await self.click()

    async def select_option(self, values: str | list[str]) -> None:
        """Select option(s) in a select element."""
        if isinstance(values, str):
            values = [values]

        ## Focus the element first
        try:
            await self.focus()
        except Exception:
            logger.warning('Failed to focus element')

        ## For select elements, we need to find option elements and click them
        ## This is a simplified approach - in practice, you might need to handle
        ## different select types (single vs multi-select) differently
        node_id = await self._get_node_id()

        ## Request child nodes to get the options
        params: 'RequestChildNodesParameters' = {'nodeId': node_id, 'depth': 1}
        await self._client.send.DOM.requestChildNodes(params, session_id=self._session_id)

        ## Get the updated node description with children
        describe_params: 'DescribeNodeParameters' = {'nodeId': node_id, 'depth': 1}
        describe_result = await self._client.send.DOM.describeNode(describe_params, session_id=self._session_id)

        select_node = describe_result['node']

        ## Find and select matching options
        for child in select_node.get('children', []):
            if child.get('nodeName', '').lower() == 'option':
                ## Get option attributes
                attrs = child.get('attributes', [])
                option_attrs = {}
                for i in range(0, len(attrs), 2):
                    if i + 1 < len(attrs):
                        option_attrs[attrs[i]] = attrs[i + 1]

                option_value = option_attrs.get('value', '')
                option_text = child.get('nodeValue', '')

                ## Check if this option should be selected
                should_select = option_value in values or option_text in values

                if should_select:
                    ## Click the option to select it
                    option_node_id = child.get('nodeId')
                    if option_node_id:
                        ## Get backend node ID for the option
                        option_describe_params: 'DescribeNodeParameters' = {'nodeId': option_node_id}
                        option_backend_result = await self._client.send.DOM.describeNode(
                            option_describe_params, session_id=self._session_id
                        )
                        option_backend_id = option_backend_result['node']['backendNodeId']

                        ## Create an Element for the option and click it
                        option_element = Element(self._browser_session, option_backend_id, self._session_id)
                        await option_element.click()

    async def drag_to(

```

```

    self,
    target: Union['Element', Position],
    source_position: Position | None = None,
    target_position: Position | None = None,
    ) -> None:
    """Drag this element to another element or position."""
    # Get source coordinates
    if source_position:
        source_x = source_position['x']
        source_y = source_position['y']
    else:
        source_box = await self.get_bounding_box()
        if not source_box:
            raise RuntimeError('Source element is not visible')
        source_x = source_box['x'] + source_box['width'] / 2
        source_y = source_box['y'] + source_box['height'] / 2

    # Get target coordinates
    if isinstance(target, dict) and 'x' in target and 'y' in target:
        target_x = target['x']
        target_y = target['y']
    else:
        if target_position:
            target_box = await target.get_bounding_box()
            if not target_box:
                raise RuntimeError('Target element is not visible')
            target_x = target_box['x'] + target_position['x']
            target_y = target_box['y'] + target_position['y']
        else:
            target_box = await target.get_bounding_box()
            if not target_box:
                raise RuntimeError('Target element is not visible')
            target_x = target_box['x'] + target_box['width'] / 2
            target_y = target_box['y'] + target_box['height'] / 2

    # Perform drag operation
    await self._client.send.Input.dispatchMouseEvent(
        {'type': 'mousePressed', 'x': source_x, 'y': source_y, 'button': 'left'},
        session_id=self._session_id,
    )

    await self._client.send.Input.dispatchMouseEvent(
        {'type': 'mouseMoved', 'x': target_x, 'y': target_y},
        session_id=self._session_id,
    )

    await self._client.send.Input.dispatchMouseEvent(
        {'type': 'mouseReleased', 'x': target_x, 'y': target_y, 'button': 'left'},
        session_id=self._session_id,
    )

    # Element properties and queries
    async def get_attribute(self, name: str) -> str | None:
        """Get an attribute value."""
        node_id = await self._get_node_id()
        params: 'GetAttributesParameters' = {'nodeId': node_id}
        result = await self._client.send.DOM.getAttributes(params, session_id=self._session_id)

        attributes = result['attributes']
        for i in range(0, len(attributes), 2):
            if attributes[i] == name:
                return attributes[i + 1]
        return None

    async def get_bounding_box(self) -> BoundingBox | None:
        """Get the bounding box of the element."""
        try:
            node_id = await self._get_node_id()
            params: 'GetBoxModelParameters' = {'nodeId': node_id}
            result = await self._client.send.DOM.getBoxModel(params, session_id=self._session_id)

            if 'model' not in result:

```

```

    return None

    # Get content box (first 8 values are content quad: x1,y1,x2,y2,x3,y3,x4,y4)
    content = result['model']['content']
    if len(content) < 8:
        return None

    # Calculate bounding box from quad
    x_coords = [content[i] for i in range(0, 8, 2)]
    y_coords = [content[i] for i in range(1, 8, 2)]

    x = min(x_coords)
    y = min(y_coords)
    width = max(x_coords) - x
    height = max(y_coords) - y

    return BoundingBox(x=x, y=y, width=width, height=height)

except Exception:
    return None

async def screenshot(self, format: str = 'png', quality: int | None = None) -> str:
    """Take a screenshot of this element and return base64 encoded image.

    Args:
        format: Image format ('jpeg', 'png', 'webp')
        quality: Quality 0-100 for JPEG format

    Returns:
        Base64-encoded image data
    """
    # Get element's bounding box
    box = await self.get_bounding_box()
    if not box:
        raise RuntimeError('Element is not visible or has no bounding box')

    # Create viewport clip for the element
    viewport: 'Viewport' = {'x': box['x'], 'y': box['y'], 'width': box['width'], 'height': box['height']}

    # Prepare screenshot parameters
    params: 'CaptureScreenshotParameters' = {'format': format, 'clip': viewport}

    if quality is not None and format.lower() == 'jpeg':
        params['quality'] = quality

    # Take screenshot
    result = await self._client.send.Page.captureScreenshot(params, session_id=self._session_id)

    return result['data']

async def evaluate(self, page_function: str, *args) -> str:
    """Execute JavaScript code in the context of this element.

    The JavaScript code executes with 'this' bound to the element, allowing direct
    access to element properties and methods.

    Args:
        page_function: JavaScript code that MUST start with (...args) => format
        *args: Arguments to pass to the function

    Returns:
        String representation of the JavaScript execution result.
        Objects and arrays are JSON-stringified.

    Example:
        # Get element's text content
        text = await element.evaluate("() => this.textContent")

        # Set style with argument
        await element.evaluate("(color) => this.style.color = color", "red")

        # Get computed style

```

```

■■■color = await element.evaluate("() => getComputedStyle(this).color")

■■■# Async operations
■■■result = await element.evaluate("async () => { await new Promise(r => setTimeout(r, 100)); return th
■■■"""
■■■# Get remote object ID for this element
■■■object_id = await self._get_remote_object_id()
■■■if not object_id:
■■■raise RuntimeError('Element has no remote object ID (element may be detached from DOM)')

■■■# Validate arrow function format (allow async prefix)
■■■page_function = page_function.strip()
■■■# Check for arrow function with optional async prefix
■■■if not ('=>' in page_function and (page_function.startswith('(') or page_function.startswith('async')))
■■■raise ValueError(
■■■    f'JavaScript code must start with (...args) => or async (...args) => format. Got: {page_function[:5
■■■})

■■■# Convert arrow function to function declaration for CallFunctionOn
■■■# CallFunctionOn expects 'function(...args) { ... }' format, not arrow functions
■■■# We need to convert: '() => expression' to 'function() { return expression; }'
■■■# or: '(x, y) => { statements }' to 'function(x, y) { statements }'

■■■# Extract parameters and body from arrow function
■■■import re

■■■# Check if it's an async arrow function
■■■is_async = page_function.strip().startswith('async')
■■■async_prefix = 'async ' if is_async else ''

■■■# Match: (params) => body or async (params) => body
■■■# Strip 'async' prefix if present for parsing
■■■func_to_parse = page_function.strip()
■■■if is_async:
■■■    func_to_parse = func_to_parse[5:].strip() # Remove 'async' prefix

■■■arrow_match = re.match(r'\s*(((\[^\])*\])\s*>)\s*(.+)', func_to_parse, re.DOTALL)
■■■if not arrow_match:
■■■    raise ValueError(f'Could not parse arrow function: {page_function[:50]}...')

■■■params_str = arrow_match.group(1).strip() # e.g., '', 'x', 'x, y'
■■■body = arrow_match.group(2).strip()

■■■# If body doesn't start with {, it's an expression that needs implicit return
■■■if not body.startswith('{'):
■■■    function_declaration = f'{async_prefix}function({params_str}) {{ return {body}; }}'
■■■else:
■■■    # Body already has braces, use as-is
■■■    function_declaration = f'{async_prefix}function({params_str}) {body}'

■■■# Build CallArgument list for args if provided
■■■call_arguments = []
■■■if args:
■■■    from cdp_use.cdp.runtime.types import CallArgument

■■■    for arg in args:
■■■        # Convert Python values to CallArgument format
■■■        call_arguments.append(CallArgument(value=arg))

■■■# Prepare CallFunctionOn parameters

■■■params: 'CallFunctionOnParameters' = {
■■■    'functionDeclaration': function_declaration,
■■■    'objectId': object_id,
■■■    'returnByValue': True,
■■■    'awaitPromise': True,
■■■}

■■■if call_arguments:
■■■    params['arguments'] = call_arguments

■■■# Execute the function on the element

```

```

    result = await self._client.send.Runtime.callFunctionOn(
        params,
        session_id=self._session_id,
    )

    ## Handle exceptions
    if 'exceptionDetails' in result:
        raise RuntimeError(f'JavaScript evaluation failed: {result["exceptionDetails"]}')

    ## Extract and return value
    value = result.get('result', {}).get('value')

    ## Return string representation (matching Page.evaluate behavior)
    if value is None:
        return ''
    elif isinstance(value, str):
        return value
    else:
        ## Convert objects, numbers, booleans to string
        import json

        try:
            return json.dumps(value) if isinstance(value, (dict, list)) else str(value)
        except (TypeError, ValueError):
            return str(value)

    ## Helpers for modifiers etc
    def _get_char_modifiers_and_vk(self, char: str) -> tuple[int, int, str]:
        """Get modifiers, virtual key code, and base key for a character.

        Returns:
            (modifiers, windowsVirtualKeyCode, base_key)
        """
        ## Characters that require Shift modifier
        shift_chars = {
            '!': ('1', 49),
            '@': ('2', 50),
            '#': ('3', 51),
            '$': ('4', 52),
            '%': ('5', 53),
            '^': ('6', 54),
            '&': ('7', 55),
            '*': ('8', 56),
            '(': ('9', 57),
            ')': ('0', 48),
            '_': ('-', 189),
            '+': ('=', 187),
            '{': ('[', 219),
            '}': (']', 221),
            '|': ('\\', 220),
            ':': (';', 186),
            '"': ('"', 222),
            '<': ('_', 188),
            '>': ('.', 190),
            '?': ('/', 191),
            '~': ('`', 192),
        }

        ## Check if character requires Shift
        if char in shift_chars:
            base_key, vk_code = shift_chars[char]
            return (8, vk_code, base_key) # Shift=8

        ## Uppercase letters require Shift
        if char.isupper():
            return (8, ord(char), char.lower()) # Shift=8

        ## Lowercase letters
        if char.islower():
            return (0, ord(char.upper()), char)

        ## Numbers

```

```

    if char.isdigit():
        return (0, ord(char), char)

    # Special characters without Shift
    no_shift_chars = {
        ' ': 32,
        '-': 189,
        '=': 187,
        '[': 219,
        ']': 221,
        '\\': 220,
        ';': 186,
        '"': 222,
        ',': 188,
        '.': 190,
        '/': 191,
        '`': 192,
    }

    if char in no_shift_chars:
        return (0, no_shift_chars[char], char)

    # Fallback
    return (0, ord(char.upper()) if char.isalpha() else ord(char), char)

def _get_key_code_for_char(self, char: str) -> str:
    """Get the proper key code for a character (like Playwright does)."""
    # Key code mapping for common characters (using proper base keys + modifiers)
    key_codes = {
        ' ': 'Space',
        '.': 'Period',
        ',': 'Comma',
        '-': 'Minus',
        '_': 'Minus', # Underscore uses Minus with Shift
        '@': 'Digit2', # @ uses Digit2 with Shift
        '!': 'Digit1', # ! uses Digit1 with Shift (not 'Exclamation')
        '?': 'Slash', # ? uses Slash with Shift
        ':': 'Semicolon', # : uses Semicolon with Shift
        ';': 'Semicolon',
        '(': 'Digit9', # ( uses Digit9 with Shift
        ')': 'Digit0', # ) uses Digit0 with Shift
        '[': 'BracketLeft',
        ']': 'BracketRight',
        '{': 'BracketLeft', # { uses BracketLeft with Shift
        '}': 'BracketRight', # } uses BracketRight with Shift
        '/': 'Slash',
        '\\': 'Backslash',
        '=': 'Equal',
        '+': 'Equal', # + uses Equal with Shift
        '*': 'Digit8', # * uses Digit8 with Shift
        '&': 'Digit7', # & uses Digit7 with Shift
        '%': 'Digit5', # % uses Digit5 with Shift
        '$': 'Digit4', # $ uses Digit4 with Shift
        '#': 'Digit3', # # uses Digit3 with Shift
        '^': 'Digit6', # ^ uses Digit6 with Shift
        '~': 'Backquote', # ~ uses Backquote with Shift
        '`': 'Backquote',
        '"': 'Quote', # " uses Quote with Shift
        "'": 'Quote',
        '<': 'Comma', # < uses Comma with Shift
        '>': 'Period', # > uses Period with Shift
        '|': 'Backslash', # | uses Backslash with Shift
    }

    if char in key_codes:
        return key_codes[char]
    elif char.isalpha():
        return f'Key{char.upper()}'
    elif char.isdigit():
        return f'Digit{char}'
    else:
        # Fallback for unknown characters

```

```

    return f'Key{char.upper()}' if char.isascii() and char.isalpha() else 'Unidentified'

    async def _clear_text_field(self, object_id: str, cdp_client, session_id: str) -> bool:
        """Clear text field using multiple strategies, starting with the most reliable."""
        try:
            # Strategy 1: Direct JavaScript value setting (most reliable for modern web apps)
            logger.debug('Clearing text field using JavaScript value setting')

            await cdp_client.send.Runtime.callFunctionOn(
                params={
                    'functionDeclaration': """
                    function() {
                        // Try to select all text first (only works on text-like inputs)
                        // This handles cases where cursor is in the middle of text
                        try {
                            this.select();
                        } catch (e) {
                            // Some input types (date, color, number, etc.) don't support select()
                            // That's fine, we'll just clear the value directly
                        }
                        // Set value to empty
                        this.value = "";
                        // Dispatch events to notify frameworks like React
                        this.dispatchEvent(new Event("input", { bubbles: true }));
                        this.dispatchEvent(new Event("change", { bubbles: true }));
                        return this.value;
                    }
                    """,
                    'objectId': object_id,
                    'returnByValue': True,
                },
                session_id=session_id,
            )

            # Verify clearing worked by checking the value
            verify_result = await cdp_client.send.Runtime.callFunctionOn(
                params={
                    'functionDeclaration': 'function() { return this.value; }',
                    'objectId': object_id,
                    'returnByValue': True,
                },
                session_id=session_id,
            )

            current_value = verify_result.get('result', {}).get('value', '')
            if not current_value:
                logger.debug('Text field cleared successfully using JavaScript')
                return True
            else:
                logger.debug(f'JavaScript clear partially failed, field still contains: "{current_value}"')

        except Exception as e:
            logger.debug(f'JavaScript clear failed: {e}')

        # Strategy 2: Triple-click + Delete (fallback for stubborn fields)
        try:
            logger.debug('Fallback: Clearing using triple-click + Delete')

            # Get element center coordinates for triple-click
            bounds_result = await cdp_client.send.Runtime.callFunctionOn(
                params={
                    'functionDeclaration': 'function() { return this.getBoundingClientRect(); }',
                    'objectId': object_id,
                    'returnByValue': True,
                },
                session_id=session_id,
            )

            if bounds_result.get('result', {}).get('value'):
                bounds = bounds_result['result']['value'] # type: ignore # type: ignore
                center_x = bounds['x'] + bounds['width'] / 2
                center_y = bounds['y'] + bounds['height'] / 2

```



```

##### # Triple-click to select all text
##### await cdp_client.send.Input.dispatchMouseEvent(
#####     params={
#####         'type': 'mousePressed',
#####         'x': center_x,
#####         'y': center_y,
#####         'button': 'left',
#####         'clickCount': 3,
#####     },
#####     session_id=session_id,
##### )
##### await cdp_client.send.Input.dispatchMouseEvent(
#####     params={
#####         'type': 'mouseReleased',
#####         'x': center_x,
#####         'y': center_y,
#####         'button': 'left',
#####         'clickCount': 3,
#####     },
#####     session_id=session_id,
##### )

##### # Delete selected text
##### await cdp_client.send.Input.dispatchKeyEvent(
#####     params={
#####         'type': 'keyDown',
#####         'key': 'Delete',
#####         'code': 'Delete',
#####     },
#####     session_id=session_id,
##### )
##### await cdp_client.send.Input.dispatchKeyEvent(
#####     params={
#####         'type': 'keyUp',
#####         'key': 'Delete',
#####         'code': 'Delete',
#####     },
#####     session_id=session_id,
##### )

##### logger.debug('Text field cleared using triple-click + Delete')
##### return True

##### except Exception as e:
#####     logger.debug(f'Triple-click clear failed: {e}')

##### # If all strategies failed
##### logger.warning('All text clearing strategies failed')
##### return False

##### async def _focus_element_simple(
#####     self, backend_node_id: int, object_id: str, cdp_client, session_id: str, input_coordinates=None
##### ) -> bool:
#####     """Focus element using multiple strategies with robust fallbacks."""
#####     try:
#####         # Strategy 1: CDP focus (most reliable)
#####         logger.debug('Focusing element using CDP focus')
#####         await cdp_client.send.DOM.focus(params={'backendNodeId': backend_node_id}, session_id=session_id)
#####         logger.debug('Element focused successfully using CDP focus')
#####         return True
#####     except Exception as e:
#####         logger.debug(f'CDP focus failed: {e}, trying JavaScript focus')

#####     try:
#####         # Strategy 2: JavaScript focus (fallback)
#####         logger.debug('Focusing element using JavaScript focus')
#####         await cdp_client.send.Runtime.callFunctionOn(
#####             params={
#####                 'functionDeclaration': 'function() { this.focus(); }',
#####                 'objectId': object_id,
#####             },
#####             session_id=session_id,

```

```

    )
    logger.debug('Element focused successfully using JavaScript')
    return True
except Exception as e:
    logger.debug(f'JavaScript focus failed: {e}, trying click focus')

    try:
        # Strategy 3: Click to focus (last resort)
        if input_coordinates:
            logger.debug(f'Focusing element by clicking at coordinates: {input_coordinates}')
            center_x = input_coordinates['input_x']
            center_y = input_coordinates['input_y']

            # Click on the element to focus it
            await cdp_client.send.Input.dispatchMouseEvent(
                params={
                    'type': 'mousePressed',
                    'x': center_x,
                    'y': center_y,
                    'button': 'left',
                    'clickCount': 1,
                },
                session_id=session_id,
            )
            await cdp_client.send.Input.dispatchMouseEvent(
                params={
                    'type': 'mouseReleased',
                    'x': center_x,
                    'y': center_y,
                    'button': 'left',
                    'clickCount': 1,
                },
                session_id=session_id,
            )
            logger.debug('Element focused using click')
            return True
        else:
            logger.debug('No coordinates available for click focus')
            except Exception as e:
                logger.warning(f'All focus strategies failed: {e}')
            return False

    async def get_basic_info(self) -> ElementInfo:
        """Get basic information about the element including coordinates and properties."""
        try:
            # Get basic node information
            node_id = await self._get_node_id()
            describe_result = await self._client.send.DOM.describeNode({'nodeId': node_id}, session_id=self._ses

            node_info = describe_result['node']

            # Get bounding box
            bounding_box = await self.get_bounding_box()

            # Get attributes as a proper dict
            attributes_list = node_info.get('attributes', [])
            attributes_dict: dict[str, str] = {}
            for i in range(0, len(attributes_list), 2):
                if i + 1 < len(attributes_list):
                    attributes_dict[attributes_list[i]] = attributes_list[i + 1]

            return ElementInfo(
                backendNodeId=self._backend_node_id,
                nodeId=node_id,
                nodeName=node_info.get('nodeName', ''),
                nodeType=node_info.get('nodeType', 0),
                nodeValue=node_info.get('nodeValue'),
                attributes=attributes_dict,
                boundingBox=bounding_box,
                error=None,
            )
        except Exception as e:

```

```

    return ElementInfo(
        backendNodeId=self._backend_node_id,
        nodeId=None,
        nodeName='',
        nodeType=0,
        nodeValue=None,
        attributes={},
        boundingBox=None,
        error=str(e),
    )

```

.\system\actor\mouse.py

```

"""Mouse class for mouse operations."""

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from cdp_use.cdp.input.commands import DispatchMouseEventParameters, SynthesizeScrollGestureParameters
    from cdp_use.cdp.input.types import MouseButton

    from system.browser.session import BrowserSession

class Mouse:
    """Mouse operations for a target."""

    def __init__(self, browser_session: 'BrowserSession', session_id: str | None = None, target_id: str | None = None):
        self._browser_session = browser_session
        self._client = browser_session.cdp_client
        self._session_id = session_id
        self._target_id = target_id

    async def click(self, x: int, y: int, button: 'MouseButton' = 'left', click_count: int = 1) -> None:
        """Click at the specified coordinates."""
        # Mouse press
        press_params: 'DispatchMouseEventParameters' = {
            'type': 'mousePressed',
            'x': x,
            'y': y,
            'button': button,
            'clickCount': click_count,
        }
        await self._client.send.Input.dispatchMouseEvent(
            press_params,
            session_id=self._session_id,
        )

        # Mouse release
        release_params: 'DispatchMouseEventParameters' = {
            'type': 'mouseReleased',
            'x': x,
            'y': y,
            'button': button,
            'clickCount': click_count,
        }
        await self._client.send.Input.dispatchMouseEvent(
            release_params,
            session_id=self._session_id,
        )

    async def down(self, button: 'MouseButton' = 'left', click_count: int = 1) -> None:
        """Press mouse button down."""
        params: 'DispatchMouseEventParameters' = {
            'type': 'mousePressed',
            'x': 0, # Will use last mouse position
            'y': 0,
            'button': button,

```

```

    clickCount': click_count,
    }
    await self._client.send.Input.dispatchMouseEvent(
        params,
        session_id=self._session_id,
    )

    async def up(self, button: 'MouseButton' = 'left', click_count: int = 1) -> None:
        """Release mouse button."""
        params: 'DispatchMouseEventParameters' = {
            'type': 'mouseReleased',
            'x': 0, # Will use last mouse position
            'y': 0,
            'button': button,
            'clickCount': click_count,
        }
        await self._client.send.Input.dispatchMouseEvent(
            params,
            session_id=self._session_id,
        )

    async def move(self, x: int, y: int, steps: int = 1) -> None:
        """Move mouse to the specified coordinates."""
        # TODO: Implement smooth movement with multiple steps if needed
        _ = steps # Acknowledge parameter for future use

        params: 'DispatchMouseEventParameters' = {'type': 'mouseMoved', 'x': x, 'y': y}
        await self._client.send.Input.dispatchMouseEvent(params, session_id=self._session_id)

    async def scroll(self, x: int = 0, y: int = 0, delta_x: int | None = None, delta_y: int | None = None) -> None:
        """Scroll the page using robust CDP methods."""
        if not self._session_id:
            raise RuntimeError('Session ID is required for scroll operations')

        # Method 1: Try mouse wheel event (most reliable)
        try:
            # Get viewport dimensions
            layout_metrics = await self._client.send.Page.getLayoutMetrics(session_id=self._session_id)
            viewport_width = layout_metrics['layoutViewport']['clientWidth']
            viewport_height = layout_metrics['layoutViewport']['clientHeight']

            # Use provided coordinates or center of viewport
            scroll_x = x if x > 0 else viewport_width / 2
            scroll_y = y if y > 0 else viewport_height / 2

            # Calculate scroll deltas (positive = down/right)
            scroll_delta_x = delta_x or 0
            scroll_delta_y = delta_y or 0

            # Dispatch mouse wheel event
            await self._client.send.Input.dispatchMouseEvent(
                params={
                    'type': 'mouseWheel',
                    'x': scroll_x,
                    'y': scroll_y,
                    'deltaX': scroll_delta_x,
                    'deltaY': scroll_delta_y,
                },
                session_id=self._session_id,
            )
            return

        except Exception:
            pass

        # Method 2: Fallback to synthesizeScrollGesture
        try:
            params: 'SynthesizeScrollGestureParameters' = {'x': x, 'y': y, 'xDistance': delta_x or 0, 'yDistance': delta_y or 0}
            await self._client.send.Input.synthesizeScrollGesture(
                params,
                session_id=self._session_id,
            )

```

```

    except Exception:
        # Method 3: JavaScript fallback
        scroll_js = f'window.scrollBy({delta_x or 0}, {delta_y or 0})'
        await self._client.send.Runtime.evaluate(
            params={'expression': scroll_js, 'returnByValue': True},
            session_id=self._session_id,
        )

```

.system\actor\page.py

```

"""Page class for page-level operations."""

from typing import TYPE_CHECKING, TypeVar

from pydantic import BaseModel

from system import logger
from system.actor.utils import get_key_info
from system.dom.serializer.serializer import DOMTreeSerializer
from system.dom.service import DomService
from system.llm.messages import SystemMessage, UserMessage

T = TypeVar('T', bound=BaseModel)

if TYPE_CHECKING:
    from cdp_use.cdp.dom.commands import (
        DescribeNodeParameters,
        QuerySelectorAllParameters,
    )
    from cdp_use.cdp.emulation.commands import SetDeviceMetricsOverrideParameters
    from cdp_use.cdp.input.commands import (
        DispatchKeyEventParameters,
    )
    from cdp_use.cdp.page.commands import CaptureScreenshotParameters, NavigateParameters, NavigateToHistoryEntryParameters
    from cdp_use.cdp.runtime.commands import EvaluateParameters
    from cdp_use.cdp.target.commands import (
        AttachToTargetParameters,
        GetTargetInfoParameters,
    )
    from cdp_use.cdp.target.types import TargetInfo

    from system.browser.session import BrowserSession
    from system.llm.base import BaseChatModel

    from .element import Element
    from .mouse import Mouse

class Page:
    """Page operations (tab or iframe)."""

    def __init__(
        self, browser_session: 'BrowserSession', target_id: str, session_id: str | None = None, llm: 'BaseChatModel' = None
    ):
        self._browser_session = browser_session
        self._client = browser_session.cdp_client
        self._target_id = target_id
        self._session_id: str | None = session_id
        self._mouse: 'Mouse | None' = None

        self._llm = llm

    async def _ensure_session(self) -> str:
        """Ensure we have a session ID for this target."""
        if not self._session_id:
            params: 'AttachToTargetParameters' = {'targetId': self._target_id, 'flatten': True}
            result = await self._client.send.Target.attachToTarget(params)
            self._session_id = result['sessionId']

```

```

#### Enable necessary domains
####import asyncio

####await asyncio.gather(
#####self._client.send.Page.enable(session_id=self._session_id),
#####self._client.send.DOM.enable(session_id=self._session_id),
#####self._client.send.Runtime.enable(session_id=self._session_id),
#####self._client.send.Network.enable(session_id=self._session_id),
#####)

####return self._session_id

####@property
####async def session_id(self) -> str:
####    """Get the session ID for this target.

####@dev Pass this to an arbitrary CDP call
####    """
####    return await self._ensure_session()

####@property
####async def mouse(self) -> 'Mouse':
####    """Get the mouse interface for this target."""
####    if not self._mouse:
####        session_id = await self._ensure_session()
####        from .mouse import Mouse

####        self._mouse = Mouse(self._browser_session, session_id, self._target_id)
####    return self._mouse

####async def reload(self) -> None:
####    """Reload the target."""
####    session_id = await self._ensure_session()
####    await self._client.send.Page.reload(session_id=session_id)

####async def get_element(self, backend_node_id: int) -> 'Element':
####    """Get an element by its backend node ID."""
####    session_id = await self._ensure_session()

####    from .element import Element as Element_

####    return Element_(self._browser_session, backend_node_id, session_id)

####async def evaluate(self, page_function: str, *args) -> str:
####    """Execute JavaScript in the target.

####Args:
####    page_function: JavaScript code that MUST start with (...args) => format
####    *args: Arguments to pass to the function

####Returns:
####    String representation of the JavaScript execution result.
####    Objects and arrays are JSON-stringified.
####    """
####    session_id = await self._ensure_session()

####    # Clean and fix common JavaScript string parsing issues
####    page_function = self._fix_javascript_string(page_function)

####    # Enforce arrow function format
####    if not (page_function.startswith('(') and '=>' in page_function):
####        raise ValueError(f'JavaScript code must start with (...args) => format. Got: {page_function[:50]}...')

####    # Build the expression - call the arrow function with provided args
####    if args:
####        # Convert args to JSON representation for safe passing
####        import json

####        arg_strs = [json.dumps(arg) for arg in args]
####        expression = f'({page_function})({", ".join(arg_strs)})'
####    else:
####        expression = f'({page_function})()'

```

```

    ## Debug: log the actual expression being evaluated
    logger.debug(f'Evaluating JavaScript: {repr(expression)}')

    params: 'EvaluateParameters' = {'expression': expression, 'returnByValue': True, 'awaitPromise': True}
    result = await self._client.send.Runtime.evaluate(
        ## params,
        ## session_id=session_id,
        ## )

    if 'exceptionDetails' in result:
        raise RuntimeError(f'JavaScript evaluation failed: {result["exceptionDetails"]}')

    value = result.get('result', {}).get('value')

    ## Always return string representation
    if value is None:
        return ''
    elif isinstance(value, str):
        return value
    else:
        ## Convert objects, numbers, booleans to string
        import json

    try:
        return json.dumps(value) if isinstance(value, (dict, list)) else str(value)
    except (TypeError, ValueError):
        return str(value)

    def _fix_javascript_string(self, js_code: str) -> str:
        """Fix common JavaScript string parsing issues when written as Python string."""

        ## Just do minimal, safe cleaning
        js_code = js_code.strip()

        ## Only fix the most common and safe issues:

        ## 1. Remove obvious Python string wrapper quotes if they exist
        if (js_code.startswith('"') and js_code.endswith('"')) or (js_code.startswith("'") and js_code.endswith("'")):
            ## Check if it's a wrapped string (not part of JS syntax)
            inner = js_code[1:-1]
            if inner.count('"') + inner.count("'") == 0 or '()' ==>' in inner:
                js_code = inner

        ## 2. Only fix clearly escaped quotes that shouldn't be
        ## But be very conservative - only if we're sure it's a Python string artifact
        if '\\\\' in js_code and js_code.count('\\\\') > js_code.count(''):
            js_code = js_code.replace('\\\\', '\\')
        if "\\\"" in js_code and js_code.count("\\\"") > js_code.count(''):
            js_code = js_code.replace("\\\"", "\"")

        ## 3. Basic whitespace normalization only
        js_code = js_code.strip()

        ## Final validation - ensure it's not empty
        if not js_code:
            raise ValueError('JavaScript code is empty after cleaning')

    return js_code

    async def screenshot(self, format: str = 'png', quality: int | None = None) -> str:
        """Take a screenshot and return base64 encoded image.

        Args:
            format: Image format ('jpeg', 'png', 'webp')
            quality: Quality 0-100 for JPEG format

        Returns:
            Base64-encoded image data
        """
        session_id = await self._ensure_session()

    params: 'CaptureScreenshotParameters' = {'format': format}

```

```

    if quality is not None and format.lower() == 'jpeg':
        params['quality'] = quality

    result = await self._client.send.Page.captureScreenshot(params, session_id=session_id)

    return result['data']

    async def press(self, key: str) -> None:
        """Press a key on the page (sends keyboard input to the focused element or page)."""
        session_id = await self._ensure_session()

        # Handle key combinations like "Control+A"
        if '+' in key:
            parts = key.split('+')
            modifiers = parts[:-1]
            main_key = parts[-1]

            # Calculate modifier bitmask
            modifier_value = 0
            modifier_map = {'Alt': 1, 'Control': 2, 'Meta': 4, 'Shift': 8}
            for mod in modifiers:
                modifier_value |= modifier_map.get(mod, 0)

            # Press modifier keys
            for mod in modifiers:
                code, vk_code = get_key_info(mod)
                params: 'DispatchKeyEventParameters' = {'type': 'keyDown', 'key': mod, 'code': code}
                if vk_code is not None:
                    params['windowsVirtualKeyCode'] = vk_code
                await self._client.send.Input.dispatchKeyEvent(params, session_id=session_id)

            # Press main key with modifiers bitmask
            main_code, main_vk_code = get_key_info(main_key)
            main_down_params: 'DispatchKeyEventParameters' = {
                'type': 'keyDown',
                'key': main_key,
                'code': main_code,
                'modifiers': modifier_value,
            }
            if main_vk_code is not None:
                main_down_params['windowsVirtualKeyCode'] = main_vk_code
            await self._client.send.Input.dispatchKeyEvent(main_down_params, session_id=session_id)

            main_up_params: 'DispatchKeyEventParameters' = {
                'type': 'keyUp',
                'key': main_key,
                'code': main_code,
                'modifiers': modifier_value,
            }
            if main_vk_code is not None:
                main_up_params['windowsVirtualKeyCode'] = main_vk_code
            await self._client.send.Input.dispatchKeyEvent(main_up_params, session_id=session_id)

            # Release modifier keys
            for mod in reversed(modifiers):
                code, vk_code = get_key_info(mod)
                release_params: 'DispatchKeyEventParameters' = {'type': 'keyUp', 'key': mod, 'code': code}
                if vk_code is not None:
                    release_params['windowsVirtualKeyCode'] = vk_code
                await self._client.send.Input.dispatchKeyEvent(release_params, session_id=session_id)
            else:
                # Simple key press
                code, vk_code = get_key_info(key)
                key_down_params: 'DispatchKeyEventParameters' = {'type': 'keyDown', 'key': key, 'code': code}
                if vk_code is not None:
                    key_down_params['windowsVirtualKeyCode'] = vk_code
                await self._client.send.Input.dispatchKeyEvent(key_down_params, session_id=session_id)

                key_up_params: 'DispatchKeyEventParameters' = {'type': 'keyUp', 'key': key, 'code': code}
                if vk_code is not None:
                    key_up_params['windowsVirtualKeyCode'] = vk_code
                await self._client.send.Input.dispatchKeyEvent(key_up_params, session_id=session_id)

```



```

■async def set_viewport_size(self, width: int, height: int) -> None:
■    """Set the viewport size."""
■    session_id = await self._ensure_session()

■    params: 'SetDeviceMetricsOverrideParameters' = {
■        'width': width,
■        'height': height,
■        'deviceScaleFactor': 1.0,
■        'mobile': False,
■    }
■    await self._client.send.Emulation.setDeviceMetricsOverride(
■        params,
■        session_id=session_id,
■    )

■    # Target properties (from CDP getTargetInfo)
■    async def get_target_info(self) -> 'TargetInfo':
■        """Get target information."""
■        params: 'GetTargetInfoParameters' = {'targetId': self._target_id}
■        result = await self._client.send.Target.getTargetInfo(params)
■        return result['targetInfo']

■    async def get_url(self) -> str:
■        """Get the current URL."""
■        info = await self.get_target_info()
■        return info.get('url', '')

■    async def get_title(self) -> str:
■        """Get the current title."""
■        info = await self.get_target_info()
■        return info.get('title', '')

■    async def goto(self, url: str) -> None:
■        """Navigate this target to a URL."""
■        session_id = await self._ensure_session()

■        params: 'NavigateParameters' = {'url': url}
■        await self._client.send.Page.navigate(params, session_id=session_id)

■    async def navigate(self, url: str) -> None:
■        """Alias for goto."""
■        await self.goto(url)

■    async def go_back(self) -> None:
■        """Navigate back in history."""
■        session_id = await self._ensure_session()

■        try:
■            # Get navigation history
■            history = await self._client.send.Page.getNavigationHistory(session_id=session_id)
■            current_index = history['currentIndex']
■            entries = history['entries']

■            # Check if we can go back
■            if current_index <= 0:
■                raise RuntimeError('Cannot go back - no previous entry in history')

■            # Navigate to the previous entry
■            previous_entry_id = entries[current_index - 1]['id']
■            params: 'NavigateToHistoryEntryParameters' = {'entryId': previous_entry_id}
■            await self._client.send.Page.navigateToHistoryEntry(params, session_id=session_id)

■        except Exception as e:
■            raise RuntimeError(f'Failed to navigate back: {e}')

■    async def go_forward(self) -> None:
■        """Navigate forward in history."""
■        session_id = await self._ensure_session()

■        try:
■            # Get navigation history
■            history = await self._client.send.Page.getNavigationHistory(session_id=session_id)

```

```

    current_index = history['currentIndex']
    entries = history['entries']

    # Check if we can go forward
    if current_index >= len(entries) - 1:
        raise RuntimeError('Cannot go forward - no next entry in history')

    # Navigate to the next entry
    next_entry_id = entries[current_index + 1]['id']
    params: 'NavigateToHistoryEntryParameters' = {'entryId': next_entry_id}
    await self._client.send.Page.navigateToHistoryEntry(params, session_id=session_id)

    except Exception as e:
        raise RuntimeError(f'Failed to navigate forward: {e}')

    # Element finding methods (these would need to be implemented based on DOM queries)
    async def get_elements_by_css_selector(self, selector: str) -> list['Element']:
        """Get elements by CSS selector."""
        session_id = await self._ensure_session()

        # Get document first
        doc_result = await self._client.send.DOM.getDocument(session_id=session_id)
        document_node_id = doc_result['root']['nodeId']

        # Query selector all
        query_params: 'QuerySelectorAllParameters' = {'nodeId': document_node_id, 'selector': selector}
        result = await self._client.send.DOM.querySelectorAll(query_params, session_id=session_id)

        elements = []
        from .element import Element as Element_

        # Convert node IDs to backend node IDs
        for node_id in result['nodeIds']:
            # Get backend node ID
            describe_params: 'DescribeNodeParameters' = {'nodeId': node_id}
            node_result = await self._client.send.DOM.describeNode(describe_params, session_id=session_id)
            backend_node_id = node_result['node']['backendNodeId']
            elements.append(Element_(self._browser_session, backend_node_id, session_id))

        return elements

    # AI METHODS

    @property
    def dom_service(self) -> 'DomService':
        """Get the DOM service for this target."""
        return DomService(self._browser_session)

    async def get_element_by_prompt(self, prompt: str, llm: 'BaseChatModel | None' = None) -> 'Element | None':
        """Get an element by a prompt."""
        await self._ensure_session()
        llm = llm or self._llm

        if not llm:
            raise ValueError('LLM not provided')

        dom_service = self.dom_service

        # Lazy fetch all_frames inside get_dom_tree if needed (for cross-origin iframes)
        enhanced_dom_tree, _ = await dom_service.get_dom_tree(target_id=self._target_id, all_frames=None)

        session_id = self._browser_session.id
        serialized_dom_state, _ = DOMTreeSerializer(
            enhanced_dom_tree, None, paint_order_filtering=True, session_id=session_id
        ).serialize_accessible_elements()

        llm_representation = serialized_dom_state.llm_representation()

        system_message = SystemMessage(
            content="""You are an AI created to find an element on a page by a prompt.

<browser_state>

```

Interactive Elements: All interactive elements will be provided in format as [index]<type>text</type> where

- index: Numeric identifier for interaction
- type: HTML element type (button, input, etc.)
- text: Element description

Examples:

```
[33]<div>User form</div>
```

```
[35]<button aria-label='Submit form'>Submit</button>
```

Note that:

- Only elements with numeric indexes in [] are interactive
- (stacked) indentation (with \t) is important and means that the element is a (html) child of the element
- Pure text elements without [] are not interactive.

```
</browser_state>
```

Your task is to find an element index (if any) that matches the prompt (written in <prompt> tag).

If none of the elements matches the, return None.

Before you return the element index, reason about the state and elements for a sentence or two.""""
■■)

```
■■state_message = UserMessage(  
■■■content=f"""  
■■■<browser_state>  
■■■{llm_representation}  
■■■</browser_state>
```

```
■■■<prompt>  
■■■{prompt}  
■■■</prompt>  
■■■"""  
■■)
```

```
■■class ElementResponse(BaseModel):  
■■■# thinking: str  
■■■element_highlight_index: int | None
```

```
■■llm_response = await llm.ainvoke(  
■■■[  
■■■system_message,  
■■■state_message,  
■■■],  
■■■output_format=ElementResponse,  
■■)
```

```
■■element_highlight_index = llm_response.completion.element_highlight_index
```

```
■■if element_highlight_index is None or element_highlight_index not in serialized_dom_state.selector_map:  
■■■return None
```

```
■■element = serialized_dom_state.selector_map[element_highlight_index]
```

```
■■from .element import Element as Element_
```

```
■■return Element_(self._browser_session, element.backend_node_id, self._session_id)
```

```
■■async def must_get_element_by_prompt(self, prompt: str, llm: 'BaseChatModel | None' = None) -> 'ElementResponse':  
■■■"""Get an element by a prompt.
```

```
■■■@dev LLM can still return None, this just raises an error if the element is not found.  
■■■"""
```

```
■■■element = await self.get_element_by_prompt(prompt, llm)
```

```
■■■if element is None:
```

```
■■■raise ValueError(f'No element found for prompt: {prompt}')
```

```
■■■return element
```

```
■■async def extract_content(self, prompt: str, structured_output: type[T], llm: 'BaseChatModel | None' = None):  
■■■"""Extract structured content from the current page using LLM.
```

```
■■■Extracts clean markdown from the page and sends it to LLM for structured data extraction.
```

```

    Args:
        prompt: Description of what content to extract
        structured_output: Pydantic BaseModel class defining the expected output structure
        llm: Language model to use for extraction

    Returns:
        The structured BaseModel instance with extracted content
    """
    llm = llm or self._llm

    if not llm:
        raise ValueError('LLM not provided')

    # Extract clean markdown using the same method as in tools/service.py
    try:
        content, content_stats = await self._extract_clean_markdown()
    except Exception as e:
        raise RuntimeError(f'Could not extract clean markdown: {type(e).__name__}')

    # System prompt for structured extraction
    system_prompt = """
You are an expert at extracting structured data from the markdown of a webpage.

<input>
You will be given a query and the markdown of a webpage that has been filtered to remove noise and adver
</input>

<instructions>
- You are tasked to extract information from the webpage that is relevant to the query.
- You should ONLY use the information available in the webpage to answer the query. Do not make up infor
- If the information relevant to the query is not available in the page, your response should mention th
- If the query asks for all items, products, etc., make sure to directly list all of them.
- Return the extracted content in the exact structured format specified.
</instructions>

<output>
- Your output should present ALL the information relevant to the query in the specified structured forma
- Do not answer in conversational format - directly output the relevant information in the structured fo
</output>
""".strip()

    # Build prompt with just query and content
    prompt_content = f'<query>\n{prompt}\n</query>\n\n<webpage_content>\n{content}\n</webpage_content>'
```

```

    # Send to LLM with structured output
    import asyncio

    try:
        response = await asyncio.wait_for(
            llm.ainvoke(
                [SystemMessage(content=system_prompt), UserMessage(content=prompt_content)], output_format=structu
            ),
            timeout=120.0,
        )

    # Return the structured output BaseModel instance
    return response.completion
    except Exception as e:
        raise RuntimeError(str(e))

    async def _extract_clean_markdown(self, extract_links: bool = False) -> tuple[str, dict]:
        """Extract clean markdown from the current page using enhanced DOM tree.

        Uses the shared markdown extractor for consistency with tools/service.py.
        """
        from system.dom.markdown_extractor import extract_clean_markdown

        dom_service = self.dom_service
        return await extract_clean_markdown(dom_service=dom_service, target_id=self._target_id, extract_links=

```

.lsystemactorutils.py

```
"""Utility functions for actor operations."""

class Utils:
    """Utility functions for actor operations."""

    @staticmethod
    def get_key_info(key: str) -> tuple[str, int | None]:
        """Get the code and windowsVirtualKeyCode for a key.

        Args:
            key: Key name (e.g., 'Enter', 'ArrowUp', 'a', 'A')

        Returns:
            Tuple of (code, windowsVirtualKeyCode)

        Reference: Windows Virtual Key Codes
        https://docs.microsoft.com/en-us/windows/win32/inputdev/virtual-key-codes
        """
        # Complete mapping of key names to (code, virtualKeyCode)
        # Based on standard Windows Virtual Key Codes
        key_map = {
            # Navigation keys
            'Backspace': ('Backspace', 8),
            'Tab': ('Tab', 9),
            'Enter': ('Enter', 13),
            'Escape': ('Escape', 27),
            'Space': ('Space', 32),
            ' ': ('Space', 32),
            'PageUp': ('PageUp', 33),
            'PageDown': ('PageDown', 34),
            'End': ('End', 35),
            'Home': ('Home', 36),
            'ArrowLeft': ('ArrowLeft', 37),
            'ArrowUp': ('ArrowUp', 38),
            'ArrowRight': ('ArrowRight', 39),
            'ArrowDown': ('ArrowDown', 40),
            'Insert': ('Insert', 45),
            'Delete': ('Delete', 46),
            # Modifier keys
            'Shift': ('ShiftLeft', 16),
            'ShiftLeft': ('ShiftLeft', 16),
            'ShiftRight': ('ShiftRight', 16),
            'Control': ('ControlLeft', 17),
            'ControlLeft': ('ControlLeft', 17),
            'ControlRight': ('ControlRight', 17),
            'Alt': ('AltLeft', 18),
            'AltLeft': ('AltLeft', 18),
            'AltRight': ('AltRight', 18),
            'Meta': ('MetaLeft', 91),
            'MetaLeft': ('MetaLeft', 91),
            'MetaRight': ('MetaRight', 92),
            # Function keys F1-F24
            'F1': ('F1', 112),
            'F2': ('F2', 113),
            'F3': ('F3', 114),
            'F4': ('F4', 115),
            'F5': ('F5', 116),
            'F6': ('F6', 117),
            'F7': ('F7', 118),
            'F8': ('F8', 119),
            'F9': ('F9', 120),
            'F10': ('F10', 121),
            'F11': ('F11', 122),
            'F12': ('F12', 123),
            'F13': ('F13', 124),
            'F14': ('F14', 125),
            'F15': ('F15', 126),
```

```

■■■■'F16': ('F16', 127),
■■■■'F17': ('F17', 128),
■■■■'F18': ('F18', 129),
■■■■'F19': ('F19', 130),
■■■■'F20': ('F20', 131),
■■■■'F21': ('F21', 132),
■■■■'F22': ('F22', 133),
■■■■'F23': ('F23', 134),
■■■■'F24': ('F24', 135),
■■■■# Numpad keys
■■■■'NumLock': ('NumLock', 144),
■■■■'Numpad0': ('Numpad0', 96),
■■■■'Numpad1': ('Numpad1', 97),
■■■■'Numpad2': ('Numpad2', 98),
■■■■'Numpad3': ('Numpad3', 99),
■■■■'Numpad4': ('Numpad4', 100),
■■■■'Numpad5': ('Numpad5', 101),
■■■■'Numpad6': ('Numpad6', 102),
■■■■'Numpad7': ('Numpad7', 103),
■■■■'Numpad8': ('Numpad8', 104),
■■■■'Numpad9': ('Numpad9', 105),
■■■■'NumpadMultiply': ('NumpadMultiply', 106),
■■■■'NumpadAdd': ('NumpadAdd', 107),
■■■■'NumpadSubtract': ('NumpadSubtract', 109),
■■■■'NumpadDecimal': ('NumpadDecimal', 110),
■■■■'NumpadDivide': ('NumpadDivide', 111),
■■■■# Lock keys
■■■■'CapsLock': ('CapsLock', 20),
■■■■'ScrollLock': ('ScrollLock', 145),
■■■■# OEM/Punctuation keys (US keyboard layout)
■■■■'Semicolon': ('Semicolon', 186),
■■■■';': ('Semicolon', 186),
■■■■'Equal': ('Equal', 187),
■■■■'=': ('Equal', 187),
■■■■'Comma': ('Comma', 188),
■■■■',': ('Comma', 188),
■■■■'Minus': ('Minus', 189),
■■■■'-': ('Minus', 189),
■■■■'Period': ('Period', 190),
■■■■'.': ('Period', 190),
■■■■'Slash': ('Slash', 191),
■■■■'/' : ('Slash', 191),
■■■■'Backquote': ('Backquote', 192),
■■■■'`': ('Backquote', 192),
■■■■'BracketLeft': ('BracketLeft', 219),
■■■■ '[': ('BracketLeft', 219),
■■■■'Backslash': ('Backslash', 220),
■■■■ '\\': ('Backslash', 220),
■■■■'BracketRight': ('BracketRight', 221),
■■■■ ']': ('BracketRight', 221),
■■■■'Quote': ('Quote', 222),
■■■■ '"': ('Quote', 222),
■■■■# Media/Browser keys
■■■■'AudioVolumeMute': ('AudioVolumeMute', 173),
■■■■'AudioVolumeDown': ('AudioVolumeDown', 174),
■■■■'AudioVolumeUp': ('AudioVolumeUp', 175),
■■■■'MediaTrackNext': ('MediaTrackNext', 176),
■■■■'MediaTrackPrevious': ('MediaTrackPrevious', 177),
■■■■'MediaStop': ('MediaStop', 178),
■■■■'MediaPlayPause': ('MediaPlayPause', 179),
■■■■'BrowserBack': ('BrowserBack', 166),
■■■■'BrowserForward': ('BrowserForward', 167),
■■■■'BrowserRefresh': ('BrowserRefresh', 168),
■■■■'BrowserStop': ('BrowserStop', 169),
■■■■'BrowserSearch': ('BrowserSearch', 170),
■■■■'BrowserFavorites': ('BrowserFavorites', 171),
■■■■'BrowserHome': ('BrowserHome', 172),
■■■■# Additional common keys
■■■■'Clear': ('Clear', 12),
■■■■'Pause': ('Pause', 19),
■■■■'Select': ('Select', 41),
■■■■'Print': ('Print', 42),

```

```

■■■■'Execute': ('Execute', 43),
■■■■'PrintScreen': ('PrintScreen', 44),
■■■■'Help': ('Help', 47),
■■■■'ContextMenu': ('ContextMenu', 93),
■■■}

■■■if key in key_map:
■■■return key_map[key]

■■■# Handle alphanumeric keys dynamically
■■■if len(key) == 1:
■■■■if key.isalpha():
■■■■■# Letter keys: A-Z have VK codes 65-90
■■■■return (f'Key{key.upper()}', ord(key.upper()))
■■■■elif key.isdigit():
■■■■■# Digit keys: 0-9 have VK codes 48-57 (same as ASCII)
■■■■return (f'Digit{key}', ord(key))

■■■# Fallback: use the key name as code, no virtual key code
■■■return (key, None)

# Backward compatibility: provide standalone function
def get_key_info(key: str) -> tuple[str, int | None]:
■■■"""Get the code and windowsVirtualKeyCode for a key.

■■■Args:
■■■key: Key name (e.g., 'Enter', 'ArrowUp', 'a', 'A')

■■■Returns:
■■■Tuple of (code, windowsVirtualKeyCode)

■■■Reference: Windows Virtual Key Codes
■■■https://docs.microsoft.com/en-us/windows/win32/inputdev/virtual-key-codes
■■■"""
■■■return Utils.get_key_info(key)

```

.\systemlactor__init__.py

```

■■■■CDP-Use High-Level Library

A Playwright-like library built on top of CDP (Chrome DevTools Protocol).
■■■■

from .element import Element
from .mouse import Mouse
from .page import Page
from .utils import Utils

__all__ = ['Page', 'Element', 'Mouse', 'Utils']

```

.\systemlactor\playground\flights.py

```

import asyncio

from system import Agent, Browser, ChatOpenAI

llm = ChatOpenAI('gpt-4.1-mini')

async def main():
■■■■
■■■Main function demonstrating mixed automation with Browser-Use and Playwright.
■■■■

```

```

■print('■ Mixed Automation with Browser-Use and Actor API')

■browser = Browser(keep_alive=True)
■await browser.start()

■page = await browser.get_current_page() or await browser.new_page()

■# Go to apple wikipedia page
■await page.goto('https://www.google.com/travel/flights')

■await asyncio.sleep(1)

■round_trip_button = await page.must_get_element_by_prompt('round trip button', llm)
■await round_trip_button.click()

■one_way_button = await page.must_get_element_by_prompt('one way button', llm)
■await one_way_button.click()

■await asyncio.sleep(1)

■agent = Agent(task='Find the cheapest flight from London to Paris on 2025-10-15', llm=llm, browser_session=browser)
■await agent.run()

■input('Press Enter to continue...')

■await browser.stop()

if __name__ == '__main__':
    ■asyncio.run(main())

```

.\system\actor\playground\mixed_automation.py

```

import asyncio

from pydantic import BaseModel

from system import Browser, ChatOpenAI

TASK = """
On the current wikipedia page, find the latest huge edit and tell me what it was about.
"""

class LatestEditFinder(BaseModel):
    """Find the latest huge edit on the current wikipedia page."""

    ■latest_edit: str
    ■edit_time: str
    ■edit_author: str
    ■edit_summary: str
    ■edit_url: str

llm = ChatOpenAI('gpt-4.1-mini')

async def main():
    """
    ■Main function demonstrating mixed automation with Browser-Use and Playwright.
    """
    ■print('■ Mixed Automation with Browser-Use and Actor API')

    ■browser = Browser(keep_alive=True)
    ■await browser.start()

    ■page = await browser.get_current_page() or await browser.new_page()
    ■# Go to apple wikipedia page

```



```

■await page.goto('https://browser-use.github.io/stress-tests/challenges/angularjs-form.html')

■await asyncio.sleep(1)

■element = await page.get_element_by_prompt('zip code input', llm)

■print('Element found', element)

■if element:
■■await element.click()
■else:
■■print('No element found')

■await browser.stop()

if __name__ == '__main__':
■asyncio.run(main())

```

.\system\actor\playground\playground.py

```

#!/usr/bin/env python3
"""
Playground script to test the browser-use actor API.

This script demonstrates:
- Starting a browser session
- Using the actor API to navigate and interact
- Finding elements, clicking, scrolling, JavaScript evaluation
- Testing most of the available methods
"""

import asyncio
import json
import logging

from system import Browser

# Configure logging to see what's happening
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def main():
    ■■■Main playground function.■■■
    ■logger.info('■ Starting browser actor playground')

    ■# Create browser session
    ■browser = Browser()

    ■try:
    ■■# Start the browser
    ■■await browser.start()
    ■■logger.info('■ Browser session started')

    ■■# Navigate to Wikipedia using integrated methods
    ■■logger.info('■ Navigating to Wikipedia...')
    ■■page = await browser.new_page('https://en.wikipedia.org')

    ■■# Get basic page info
    ■■url = await page.get_url()
    ■■title = await page.get_title()
    ■■logger.info(f'■ Page loaded: {title} ({url})')

    ■■# Take a screenshot
    ■■logger.info('■ Taking initial screenshot...')
    ■■screenshot_b64 = await page.screenshot()
    ■■logger.info(f'■ Screenshot captured: {len(screenshot_b64)} bytes')

```

```

■■# Set viewport size
■■logger.info('■■ Setting viewport to 1920x1080...')
■■await page.set_viewport_size(1920, 1080)

■■# Execute some JavaScript to count links
■■logger.info('■ Counting article links using JavaScript...')
■■js_code = """() => {
■■■// Find all article links on the page
■■■const links = Array.from(document.querySelectorAll('a[href*="/wiki/"]:not([href*=":"])'))
■■■.filter(link => !link.href.includes('Main_Page') && !link.href.includes('Special:'));
■■■
■■■return {
■■■■total: links.length,
■■■■sample: links.slice(0, 3).map(link => ({
■■■■■href: link.href,
■■■■■text: link.textContent.trim()
■■■■})))
■■■};
■■}"""

■■link_info = json.loads(await page.evaluate(js_code))
■■logger.info(f'■ Found {link_info["total"]} article links')
■■# Try to find and interact with links using CSS selector
■■try:
■■■# Find article links on the page
■■■links = await page.get_elements_by_css_selector('a[href*="/wiki/"]:not([href*=":"])')

■■■if links:
■■■■logger.info(f'■ Found {len(links)} wiki links via CSS selector')

■■■■# Pick the first link
■■■■link_element = links[0]

■■■■# Get link info using available methods
■■■■basic_info = await link_element.get_basic_info()
■■■■link_href = await link_element.get_attribute('href')

■■■■logger.info(f'■ Selected element: <{basic_info["nodeName"]}>')
■■■■logger.info(f'■ Link href: {link_href}')

■■■■if basic_info['boundingBox']:
■■■■■bbox = basic_info['boundingBox']
■■■■■logger.info(f'■ Position: ({bbox["x"]}, {bbox["y"]}) Size: {bbox["width"]}x{bbox["height"]}')

■■■■# Test element interactions with robust implementations
■■■■logger.info('■ Hovering over the element...')
■■■■await link_element.hover()
■■■■await asyncio.sleep(1)

■■■■logger.info('■ Focusing the element...')
■■■■await link_element.focus()
■■■■await asyncio.sleep(0.5)

■■■■# Click the link using robust click method
■■■■logger.info('■■ Clicking the link with robust fallbacks...')
■■■■await link_element.click()

■■■■# Wait for navigation
■■■■await asyncio.sleep(3)

■■■■# Get new page info
■■■■new_url = await page.get_url()
■■■■new_title = await page.get_title()
■■■■logger.info(f'■ Navigated to: {new_title}')
■■■■logger.info(f'■ New URL: {new_url}')
■■■■else:
■■■■logger.warning('■ No links found to interact with')

■■except Exception as e:
■■■logger.warning(f'■■ Link interaction failed: {e}')

■■# Scroll down the page

```

```

■■■logger.info('■ Scrolling down the page...')
■■■mouse = await page.mouse
■■■await mouse.scroll(x=0, y=100, delta_y=500)
■■■await asyncio.sleep(1)

■■■# Test mouse operations
■■■logger.info('■■■ Testing mouse operations...')
■■■await mouse.move(x=100, y=200)
■■■await mouse.click(x=150, y=250)

■■■# Execute more JavaScript examples
■■■logger.info('■ Testing JavaScript evaluation...')

■■■# Simple expressions
■■■page_height = await page.evaluate('() => document.body.scrollHeight')
■■■current_scroll = await page.evaluate('() => window.pageYOffset')
■■■logger.info(f'■ Page height: {page_height}px, current scroll: {current_scroll}px')

■■■# JavaScript with arguments
■■■result = await page.evaluate('(x) => x * 2', 21)
■■■logger.info(f'■ JavaScript with args: 21 * 2 = {result}')

■■■# More complex JavaScript
■■■page_stats = json.loads(
■■■await page.evaluate('"""() => {
■■■return {
■■■url: window.location.href,
■■■title: document.title,
■■■links: document.querySelectorAll('a').length,
■■■images: document.querySelectorAll('img').length,
■■■scrollTop: window.pageYOffset,
■■■viewportHeight: window.innerHeight
■■■}"""')
■■■)
■■■logger.info(f'■ Page stats: {page_stats}')

■■■# Get page title using different methods
■■■title_via_js = await page.evaluate('() => document.title')
■■■title_via_api = await page.get_title()
■■■logger.info(f'■ Title via JS: "{title_via_js}"')
■■■logger.info(f'■ Title via API: "{title_via_api}"')

■■■# Take a final screenshot
■■■logger.info('■ Taking final screenshot...')
■■■final_screenshot = await page.screenshot()
■■■logger.info(f'■ Final screenshot: {len(final_screenshot)} bytes')

■■■# Test browser navigation with error handling
■■■logger.info('■■■ Testing browser back navigation...')
■■■try:
■■■await page.go_back()
■■■await asyncio.sleep(2)

■■■back_url = await page.get_url()
■■■back_title = await page.get_title()
■■■logger.info(f'■ After going back: {back_title}')
■■■logger.info(f'■ Back URL: {back_url}')
■■■except RuntimeError as e:
■■■logger.info(f'■■■ Navigation back failed as expected: {e}')

■■■# Test creating new page
■■■logger.info('■ Creating new blank page...')
■■■new_page = await browser.new_page()
■■■new_page_url = await new_page.get_url()
■■■logger.info(f'■ New page created with URL: {new_page_url}')

■■■# Get all pages
■■■all_pages = await browser.get_pages()
■■■logger.info(f'■ Total pages: {len(all_pages)}')

■■■# Test form interaction if we can find a form

```

```

    try:
        ## Look for search input on the page
        search_inputs = await page.get_elements_by_css_selector('input[type="search"], input[name*="search"]')

        if search_inputs:
            search_input = search_inputs[0]
            logger.info('■ Found search input, testing form interaction...')

            await search_input.focus()
            await search_input.fill('test search query')
            await page.press('Enter')

            logger.info('■ Form interaction test completed')
        else:
            logger.info('■ No search inputs found for form testing')

    except Exception as e:
        logger.info(f'■ Form interaction test skipped: {e}')

    ## wait 2 seconds before closing the new page
    logger.info('■ Waiting 2 seconds before closing the new page...')
    await asyncio.sleep(2)
    logger.info('■ Closing new page...')
    await browser.close_page(new_page)

    logger.info('■ Playground completed successfully!')

    input('Press Enter to continue...')

    except Exception as e:
        logger.error(f'■ Error in playground: {e}', exc_info=True)

    finally:
        ## Clean up
        logger.info('■ Cleaning up...')
        try:
            await browser.stop()
            logger.info('■ Browser session stopped')
        except Exception as e:
            logger.error(f'■ Error stopping browser: {e}')

if __name__ == '__main__':
    asyncio.run(main())

```

.\system\agent\cloud_events.py

```

import base64
import os
from datetime import datetime, timezone
from pathlib import Path

import anyio
from bubus import BaseEvent
from pydantic import Field, field_validator
from uuid_extensions import uuid7str

MAX_STRING_LENGTH = 500000 # 100K chars ~ 25k tokens should be enough
MAX_URL_LENGTH = 100000
MAX_TASK_LENGTH = 100000
MAX_COMMENT_LENGTH = 2000
MAX_FILE_CONTENT_SIZE = 50 * 1024 * 1024 # 50MB

class UpdateAgentTaskEvent(BaseEvent):
    ## Required fields for identification
    id: str # The task ID to update
    user_id: str = Field(max_length=255) # For authorization

```

```

device_id: str | None = Field(None, max_length=255) # Device ID for auth lookup

## Optional fields that can be updated
stopped: bool | None = None
paused: bool | None = None
done_output: str | None = Field(None, max_length=MAX_STRING_LENGTH)
finished_at: datetime | None = None
agent_state: dict | None = None
user_feedback_type: str | None = Field(None, max_length=10) # UserFeedbackType enum value as string
user_comment: str | None = Field(None, max_length=MAX_COMMENT_LENGTH)
gif_url: str | None = Field(None, max_length=MAX_URL_LENGTH)

@classmethod
def from_agent(cls, agent) -> 'UpdateAgentTaskEvent':
    """Create an UpdateAgentTaskEvent from an Agent instance"""
    ## The _task_start_time is not directly used in UpdateAgentTaskEvent,
    ## but the original code checked for its existence.
    ## We'll keep the check but make it non-blocking as per the instruction's intent
    ## to make it "safe if _task_start_time doesn't exist yet".
    ## The provided snippet for `task_start_time` and `start_time_iso` seems
    ## to be a partial copy-paste from `CreateAgentTaskEvent` and is not directly
    ## relevant or syntactically correct for `UpdateAgentTaskEvent`'s fields.
    ## We will only remove the `ValueError` and allow the method to proceed.
    ## If `_task_start_time` was intended for a field in this event, it's not present.

    done_output = agent.history.final_result() if agent.history else None
    if done_output and len(done_output) > MAX_STRING_LENGTH:
        done_output = done_output[:MAX_STRING_LENGTH]
    return cls(
        id=str(agent.task_id),
        user_id='', # To be filled by cloud handler
        device_id=agent.cloud_sync.auth_client.device_id
        if hasattr(agent, 'cloud_sync') and agent.cloud_sync and agent.cloud_sync.auth_client
        else None,
        stopped=agent.state.stopped if hasattr(agent.state, 'stopped') else False,
        paused=agent.state.paused if hasattr(agent.state, 'paused') else False,
        done_output=done_output,
        finished_at=datetime.now(timezone.utc) if agent.history and agent.history.is_done() else None,
        agent_state=agent.state.model_dump() if hasattr(agent.state, 'model_dump') else {},
        user_feedback_type=None,
        user_comment=None,
        gif_url=None,
        ## user_feedback_type and user_comment would be set by the API/frontend
        ## gif_url would be set after GIF generation if needed
    )

class CreateAgentOutputFileEvent(BaseEvent):
    ## Model fields
    id: str = Field(default_factory=uuid7str)
    user_id: str = Field(max_length=255)
    device_id: str | None = Field(None, max_length=255) # Device ID for auth lookup
    task_id: str
    file_name: str = Field(max_length=255)
    file_content: str | None = None # Base64 encoded file content
    content_type: str | None = Field(None, max_length=100) # MIME type for file uploads
    created_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

    @field_validator('file_content')
    @classmethod
    def validate_file_size(cls, v: str | None) -> str | None:
        """Validate base64 file content size."""
        if v is None:
            return v
        ## Remove data URL prefix if present
        if ',' in v:
            v = v.split(',')[1]
        ## Estimate decoded size (base64 is ~33% larger)
        estimated_size = len(v) * 3 / 4
        if estimated_size > MAX_FILE_CONTENT_SIZE:
            raise ValueError(f'File content exceeds maximum size of {MAX_FILE_CONTENT_SIZE / 1024 / 1024}MB')
        return v

```

```

    @classmethod
    async def from_agent_and_file(cls, agent, output_path: str) -> 'CreateAgentOutputFileEvent':
        """Create a CreateAgentOutputFileEvent from a file path"""

        gif_path = Path(output_path)
        if not gif_path.exists():
            raise FileNotFoundError(f'File not found: {output_path}')

        gif_size = os.path.getsize(gif_path)

        ## Read GIF content for base64 encoding if needed
        gif_content = None
        if gif_size < 50 * 1024 * 1024: # Only read if < 50MB
            async with await anyio.open_file(gif_path, 'rb') as f:
                gif_bytes = await f.read()
                gif_content = base64.b64encode(gif_bytes).decode('utf-8')

        return cls(
            user_id='', # To be filled by cloud handler
            device_id=agent.cloud_sync.auth_client.device_id
            if hasattr(agent, 'cloud_sync') and agent.cloud_sync and agent.cloud_sync.auth_client
            else None,
            task_id=str(agent.task_id),
            file_name=gif_path.name,
            file_content=gif_content, # Base64 encoded
            content_type='image/gif',
        )

class CreateAgentStepEvent(BaseEvent):
    ## Model fields
    id: str = Field(default_factory=uuid7str)
    user_id: str = Field(max_length=255) # Added for authorization checks
    device_id: str | None = Field(None, max_length=255) # Device ID for auth lookup
    created_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
    agent_task_id: str
    step: int
    evaluation_previous_goal: str = Field(max_length=MAX_STRING_LENGTH)
    memory: str = Field(max_length=MAX_STRING_LENGTH)
    next_goal: str = Field(max_length=MAX_STRING_LENGTH)
    actions: list[dict]
    screenshot_url: str | None = Field(None, max_length=MAX_FILE_CONTENT_SIZE) # ~50MB for base64 images
    url: str = Field(default='', max_length=MAX_URL_LENGTH)

    @field_validator('screenshot_url')
    @classmethod
    def validate_screenshot_size(cls, v: str | None) -> str | None:
        """Validate screenshot URL or base64 content size."""
        if v is None or not v.startswith('data:'):
            return v
        ## It's base64 data, check size
        if ',' in v:
            base64_part = v.split(',')[1]
            estimated_size = len(base64_part) * 3 / 4
            if estimated_size > MAX_FILE_CONTENT_SIZE:
                raise ValueError(f'Screenshot content exceeds maximum size of {MAX_FILE_CONTENT_SIZE / 1024 / 1024}')
        return v

    @classmethod
    def from_agent_step(
        cls, agent, model_output, result: list, actions_data: list[dict], browser_state_summary
    ) -> 'CreateAgentStepEvent':
        """Create a CreateAgentStepEvent from agent step data"""
        ## Get first action details if available
        first_action = model_output.action[0] if model_output.action else None

        ## Extract current state from model output
        current_state = model_output.current_state if hasattr(model_output, 'current_state') else None

        ## Capture screenshot as base64 data URL if available
        screenshot_url = None
        if browser_state_summary.screenshot:

```

```

■■■■screenshot_url = f'data:image/png;base64,{browser_state_summary.screenshot}'
■■■■import logging

■■■■logger = logging.getLogger(__name__)
■■■■logger.debug(f'■■ Including screenshot in CreateAgentStepEvent, length: {len(browser_state_summary.screenshot)}')
■■■■else:
■■■■import logging

■■■■logger = logging.getLogger(__name__)
■■■■logger.debug('■■ No screenshot in browser_state_summary for CreateAgentStepEvent')

■■■■return cls(
■■■■user_id='', # To be filled by cloud handler
■■■■device_id=agent.cloud_sync.auth_client.device_id
■■■■if hasattr(agent, 'cloud_sync') and agent.cloud_sync and agent.cloud_sync.auth_client
■■■■else None,
■■■■agent_task_id=str(agent.task_id),
■■■■step=agent.state.n_steps,
■■■■evaluation_previous_goal=current_state.evaluation_previous_goal if current_state else '',
■■■■memory=current_state.memory if current_state else '',
■■■■next_goal=current_state.next_goal if current_state else '',
■■■■actions=actions_data, # List of action dicts
■■■■url=browser_state_summary.url,
■■■■screenshot_url=screenshot_url,
■■■■)

class CreateAgentTaskEvent(BaseEvent):
    ■■ # Model fields
    ■■ id: str = Field(default_factory=uuid7str)
    ■■ user_id: str = Field(max_length=255) # Added for authorization checks
    ■■ device_id: str | None = Field(None, max_length=255) # Device ID for auth lookup
    ■■ agent_session_id: str
    ■■ llm_model: str = Field(max_length=200) # LLMModel enum value as string
    ■■ stopped: bool = False
    ■■ paused: bool = False
    ■■ task: str = Field(max_length=MAX_TASK_LENGTH)
    ■■ done_output: str | None = Field(None, max_length=MAX_STRING_LENGTH)
    ■■ scheduled_task_id: str | None = None
    ■■ started_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
    ■■ finished_at: datetime | None = None
    ■■ agent_state: dict = Field(default_factory=dict)
    ■■ user_feedback_type: str | None = Field(None, max_length=10) # UserFeedbackType enum value as string
    ■■ user_comment: str | None = Field(None, max_length=MAX_COMMENT_LENGTH)
    ■■ gif_url: str | None = Field(None, max_length=MAX_URL_LENGTH)

    ■■ @classmethod
    ■■ def from_agent(cls, agent) -> 'CreateAgentTaskEvent':
    ■■ """Create a CreateAgentTaskEvent from an Agent instance"""
    ■■ return cls(
    ■■■■ id=str(agent.task_id),
    ■■■■ user_id='', # To be filled by cloud handler
    ■■■■ device_id=agent.cloud_sync.auth_client.device_id
    ■■■■ if hasattr(agent, 'cloud_sync') and agent.cloud_sync and agent.cloud_sync.auth_client
    ■■■■ else None,
    ■■■■ agent_session_id=str(agent.session_id),
    ■■■■ task=agent.task,
    ■■■■ llm_model=agent.llm.model_name,
    ■■■■ agent_state=agent.state.model_dump() if hasattr(agent.state, 'model_dump') else {},
    ■■■■ stopped=False,
    ■■■■ paused=False,
    ■■■■ done_output=None,
    ■■■■ started_at=datetime.fromtimestamp(agent._task_start_time, tz=timezone.utc),
    ■■■■ finished_at=None,
    ■■■■ user_feedback_type=None,
    ■■■■ user_comment=None,
    ■■■■ gif_url=None,
    ■■■■ )

class CreateAgentSessionEvent(BaseEvent):
    ■■ # Model fields

```

```

■id: str = Field(default_factory=uuid7str)
■user_id: str = Field(max_length=255)
■device_id: str | None = Field(None, max_length=255) # Device ID for auth lookup
■browser_session_id: str = Field(max_length=255)
■browser_session_live_url: str = Field(max_length=MAX_URL_LENGTH)
■browser_session_cdp_url: str = Field(max_length=MAX_URL_LENGTH)
■browser_session_stopped: bool = False
■browser_session_stopped_at: datetime | None = None
■is_source_api: bool | None = None
■browser_state: dict = Field(default_factory=dict)
■browser_session_data: dict | None = None

■@classmethod
■def from_agent(cls, agent) -> 'CreateAgentSessionEvent':
■    """Create a CreateAgentSessionEvent from an Agent instance"""
■    return cls(
■        ■id=str(agent.session_id),
■        ■user_id='', # To be filled by cloud handler
■        ■device_id=agent.cloud_sync.auth_client.device_id
■        ■if hasattr(agent, 'cloud_sync') and agent.cloud_sync and agent.cloud_sync.auth_client
■        ■else None,
■        ■browser_session_id=agent.browser_session.id,
■        ■browser_session_live_url='', # To be filled by cloud handler
■        ■browser_session_cdp_url='', # To be filled by cloud handler
■        ■browser_state={
■            ■'viewport': agent.browser_profile.viewport if agent.browser_profile else {'width': 1280, 'height':
■            ■'user_agent': agent.browser_profile.user_agent if agent.browser_profile else None,
■            ■'headless': agent.browser_profile.headless if agent.browser_profile else True,
■            ■'initial_url': None, # Will be updated during execution
■            ■'final_url': None, # Will be updated during execution
■            ■'total_pages_visited': 0, # Will be updated during execution
■            ■'session_duration_seconds': 0, # Will be updated during execution
■        },
■        ■browser_session_data={
■            ■'cookies': [],
■            ■'secrets': {},
■            ■# TODO: send secrets safely so tasks can be replayed on cloud seamlessly
■            ■# 'secrets': dict(agent.sensitive_data) if agent.sensitive_data else {},
■            ■'allowed_domains': agent.browser_profile.allowed_domains if agent.browser_profile else [],
■        },
■    )

class UpdateAgentSessionEvent(BaseEvent):
■    """Event to update an existing agent session"""

■    # Model fields
■    id: str # Session ID to update
■    user_id: str = Field(max_length=255)
■    device_id: str | None = Field(None, max_length=255)
■    browser_session_stopped: bool | None = None
■    browser_session_stopped_at: datetime | None = None
■    end_reason: str | None = Field(None, max_length=100) # Why the session ended

```

.lsystem\agent\gif.py

```

from __future__ import annotations

import base64
import io
import logging
import os
import platform
from typing import TYPE_CHECKING

from system.agent.views import AgentHistoryList
from system.browser.views import PLACEHOLDER_4PX_SCREENSHOT
from system.config import CONFIG

```



```

if TYPE_CHECKING:
    from PIL import Image, ImageFont

logger = logging.getLogger(__name__)

def decode_unicode_escapes_to_utf8(text: str) -> str:
    """Handle decoding any unicode escape sequences embedded in a string (needed to render non-ASCII language)"""
    if r'\u' not in text:
        """# doesn't have any escape sequences that need to be decoded"""
        return text

    try:
        """# Try to decode Unicode escape sequences"""
        return text.encode('latin1').decode('unicode_escape')
    except (UnicodeEncodeError, UnicodeDecodeError):
        """# logger.debug(f"Failed to decode unicode escape sequences while generating gif text: {text}")"""
        return text

def create_history_gif(
    task: str,
    history: AgentHistoryList,
    """#
    output_path: str = 'agent_history.gif',
    duration: int = 3000,
    show_goals: bool = True,
    show_task: bool = True,
    show_logo: bool = False,
    font_size: int = 40,
    title_font_size: int = 56,
    goal_font_size: int = 44,
    margin: int = 40,
    line_spacing: float = 1.5,
) -> None:
    """Create a GIF from the agent's history with overlaid task and goal text."""
    if not history.history:
        """logger.warning('No history to create GIF from')"""
        return

    from PIL import Image, ImageFont

    images = []

    """# if history is empty, we can't create a gif"""
    if not history.history:
        """logger.warning('No history to create GIF from')"""
        return

    """# Get all screenshots from history (including None placeholders)"""
    screenshots = history.screenshots(return_none_if_not_screenshot=True)

    if not screenshots:
        """logger.warning('No screenshots found in history')"""
        return

    """# Find the first non-placeholder screenshot"""
    """# A screenshot is considered a placeholder if:
    """# 1. It's the exact 4px placeholder for about:blank pages, OR
    """# 2. It comes from a new tab page (chrome://newtab/, about:blank, etc.)"""
    first_real_screenshot = None
    for screenshot in screenshots:
        """if screenshot and screenshot != PLACEHOLDER_4PX_SCREENSHOT:
            first_real_screenshot = screenshot
            break"""

    if not first_real_screenshot:
        """logger.warning('No valid screenshots found (all are placeholders or from new tab pages)')"""
        return

    """# Try to load nicer fonts"""

```

```

try:
    ## Try different font options in order of preference
    ## ArialUni is a font that comes with Office and can render most non-alphabet characters
    font_options = [
        'PingFang',
        'STHeiti Medium',
        'Microsoft YaHei', # 
        'SimHei', # 
        'SimSun', # 
        'Noto Sans CJK SC', # 
        'WenQuanYi Micro Hei', # 
        'Helvetica',
        'Arial',
        'DejaVuSans',
        'Verdana',
    ]
    font_loaded = False

    for font_name in font_options:
        try:
            if platform.system() == 'Windows':
                ## Need to specify the abs font path on Windows
                font_name = os.path.join(CONFIG.WIN_FONT_DIR, font_name + '.ttf')
            regular_font = ImageFont.truetype(font_name, font_size)
            title_font = ImageFont.truetype(font_name, title_font_size)
            goal_font = ImageFont.truetype(font_name, goal_font_size)
            font_loaded = True
            break
        except OSError:
            continue

    if not font_loaded:
        raise OSError('No preferred fonts found')

    except OSError:
        regular_font = ImageFont.load_default()
        title_font = ImageFont.load_default()

    goal_font = regular_font

    ## Load logo if requested
    logo = None
    if show_logo:
        try:
            logo = Image.open('./static/browser-use.png')
            ## Resize logo to be small (e.g., 40px height)
            logo_height = 150
            aspect_ratio = logo.width / logo.height
            logo_width = int(logo_height * aspect_ratio)
            logo = logo.resize((logo_width, logo_height), Image.Resampling.LANCZOS)
        except Exception as e:
            logger.warning(f'Could not load logo: {e}')

    ## Create task frame if requested
    if show_task and task:
        ## Find the first non-placeholder screenshot for the task frame
        first_real_screenshot = None
        for item in history.history:
            screenshot_b64 = item.state.get_screenshot()
            if screenshot_b64 and screenshot_b64 != PLACEHOLDER_4PX_SCREENSHOT:
                first_real_screenshot = screenshot_b64
                break

    if first_real_screenshot:
        task_frame = _create_task_frame(
            task,
            first_real_screenshot,
            title_font, # type: ignore
            regular_font, # type: ignore
            logo,
            line_spacing,
        )

```

```

■■■images.append(task_frame)
■■■else:
■■■logger.warning('No real screenshots found for task frame, skipping task frame')

■■# Process each history item with its corresponding screenshot
■■for i, (item, screenshot) in enumerate(zip(history.history, screenshots), 1):
■■if not screenshot:
■■■continue

■■■# Skip placeholder screenshots from about:blank pages
■■■# These are 4x4 white PNGs encoded as a specific base64 string
■■■if screenshot == PLACEHOLDER_4PX_SCREENSHOT:
■■■logger.debug(f'Skipping placeholder screenshot from about:blank page at step {i}')
■■■continue

■■■# Skip screenshots from new tab pages
■■■from system.utils import is_new_tab_page

■■■if is_new_tab_page(item.state.url):
■■■logger.debug(f'Skipping screenshot from new tab page ({item.state.url}) at step {i}')
■■■continue

■■■# Convert base64 screenshot to PIL Image
■■■img_data = base64.b64decode(screenshot)
■■■image = Image.open(io.BytesIO(img_data))

■■■if show_goals and item.model_output:
■■■image = _add_overlay_to_image(
■■■■image=image,
■■■■step_number=i,
■■■■goal_text=item.model_output.current_state.next_goal,
■■■■regular_font=regular_font, # type: ignore
■■■■title_font=title_font, # type: ignore
■■■■margin=margin,
■■■■logo=logo,
■■■)

■■images.append(image)

■■if images:
■■■# Save the GIF
■■■images[0].save(
■■■■output_path,
■■■■save_all=True,
■■■■append_images=images[1:],
■■■■duration=duration,
■■■■loop=0,
■■■■optimize=False,
■■■)
■■logger.info(f'Created GIF at {output_path}')
■■else:
■■logger.warning('No images found in history to create GIF')

def _create_task_frame(
    ■task: str,
    ■first_screenshot: str,
    ■title_font: ImageFont.FreeTypeFont,
    ■regular_font: ImageFont.FreeTypeFont,
    ■logo: Image.Image | None = None,
    ■line_spacing: float = 1.5,
    ) -> Image.Image:
    ■"""Create initial frame showing the task."""
    ■from PIL import Image, ImageDraw, ImageFont

    ■img_data = base64.b64decode(first_screenshot)
    ■template = Image.open(io.BytesIO(img_data))
    ■image = Image.new('RGB', template.size, (0, 0, 0))
    ■draw = ImageDraw.Draw(image)

    ■# Calculate vertical center of image
    ■center_y = image.height // 2

```

```

■# Draw task text with dynamic font size based on task length
■margin = 140 # Increased margin
■max_width = image.width - (2 * margin)

■# Dynamic font size calculation based on task length
■# Start with base font size (regular + 16)
■base_font_size = regular_font.size + 16
■min_font_size = max(regular_font.size - 10, 16) # Don't go below 16pt
■max_font_size = base_font_size # Cap at the base font size

■# Calculate dynamic font size based on text length and complexity
■# Longer texts get progressively smaller fonts
■text_length = len(task)
■if text_length > 200:
■# For very long text, reduce font size logarithmically
■font_size = max(base_font_size - int(10 * (text_length / 200)), min_font_size)
■else:
■font_size = base_font_size

■# Try to create a larger font, but fall back to regular font if it fails
■try:
■# larger_font = ImageFont.truetype(regular_font.path, font_size) # type: ignore
■except (OSError, AttributeError):
■# Fall back to regular font if .path is not available or font loading fails
■larger_font = regular_font

■# Generate wrapped text with the calculated font size
■wrapped_text = _wrap_text(task, larger_font, max_width)

■# Calculate line height with spacing
■line_height = larger_font.size * line_spacing

■# Split text into lines and draw with custom spacing
■lines = wrapped_text.split('\n')
■total_height = line_height * len(lines)

■# Start position for first line
■text_y = center_y - (total_height / 2) + 50 # Shifted down slightly

■for line in lines:
■# Get line width for centering
■line_bbox = draw.textbbox((0, 0), line, font=larger_font)
■text_x = (image.width - (line_bbox[2] - line_bbox[0])) // 2

■draw.text(
■# (text_x, text_y),
■# line,
■# font=larger_font,
■# fill=(255, 255, 255),
■# )
■text_y += line_height

■# Add logo if provided (top right corner)
■if logo:
■logo_margin = 20
■logo_x = image.width - logo.width - logo_margin
■image.paste(logo, (logo_x, logo_margin), logo if logo.mode == 'RGBA' else None)

■return image

def _add_overlay_to_image(
■image: Image.Image,
■step_number: int,
■goal_text: str,
■regular_font: ImageFont.FreeTypeFont,
■title_font: ImageFont.FreeTypeFont,
■margin: int,
■logo: Image.Image | None = None,
■display_step: bool = True,
■text_color: tuple[int, int, int, int] = (255, 255, 255, 255),
■text_box_color: tuple[int, int, int, int] = (0, 0, 0, 255),

```

```

) -> Image.Image:
■""""Add step number and goal overlay to an image."""

■from PIL import Image, ImageDraw

■goal_text = decode_unicode_escapes_to_utf8(goal_text)
■image = image.convert('RGBA')
■txt_layer = Image.new('RGBA', image.size, (0, 0, 0, 0))
■draw = ImageDraw.Draw(txt_layer)
■if display_step:
■    ■# Add step number (bottom left)
■    ■step_text = str(step_number)
■    ■step_bbox = draw.textbbox((0, 0), step_text, font=title_font)
■    ■step_width = step_bbox[2] - step_bbox[0]
■    ■step_height = step_bbox[3] - step_bbox[1]

■    ■# Position step number in bottom left
■    ■x_step = margin + 10 # Slight additional offset from edge
■    ■y_step = image.height - margin - step_height - 10 # Slight offset from bottom

■    ■# Draw rounded rectangle background for step number
■    ■padding = 20 # Increased padding
■    ■step_bg_bbox = (
■        ■x_step - padding,
■        ■y_step - padding,
■        ■x_step + step_width + padding,
■        ■y_step + step_height + padding,
■    ■)
■    ■draw.rounded_rectangle(
■        ■step_bg_bbox,
■        ■radius=15, # Add rounded corners
■        ■fill=text_box_color,
■    ■)

■    ■# Draw step number
■    ■draw.text(
■        ■(x_step, y_step),
■        ■step_text,
■        ■font=title_font,
■        ■fill=text_color,
■    ■)

■    ■# Draw goal text (centered, bottom)
■    ■max_width = image.width - (4 * margin)
■    ■wrapped_goal = _wrap_text(goal_text, title_font, max_width)
■    ■goal_bbox = draw.multiline_textbbox((0, 0), wrapped_goal, font=title_font)
■    ■goal_width = goal_bbox[2] - goal_bbox[0]
■    ■goal_height = goal_bbox[3] - goal_bbox[1]

■    ■# Center goal text horizontally, place above step number
■    ■x_goal = (image.width - goal_width) // 2
■    ■y_goal = y_step - goal_height - padding * 4 # More space between step and goal

■    ■# Draw rounded rectangle background for goal
■    ■padding_goal = 25 # Increased padding for goal
■    ■goal_bg_bbox = (
■        ■x_goal - padding_goal, # Remove extra space for logo
■        ■y_goal - padding_goal,
■        ■x_goal + goal_width + padding_goal,
■        ■y_goal + goal_height + padding_goal,
■    ■)
■    ■draw.rounded_rectangle(
■        ■goal_bg_bbox,
■        ■radius=15, # Add rounded corners
■        ■fill=text_box_color,
■    ■)

■    ■# Draw goal text
■    ■draw.multiline_text(
■        ■(x_goal, y_goal),
■        ■wrapped_goal,
■        ■font=title_font,

```

```

    fill=text_color,
    align='center',
)

# Add logo if provided (top right corner)
if logo:
    logo_layer = Image.new('RGBA', image.size, (0, 0, 0, 0))
    logo_margin = 20
    logo_x = image.width - logo.width - logo_margin
    logo_layer.paste(logo, (logo_x, logo_margin), logo if logo.mode == 'RGBA' else None)
    txt_layer = Image.alpha_composite(logo_layer, txt_layer)

# Composite and convert
result = Image.alpha_composite(image, txt_layer)
return result.convert('RGB')

def _wrap_text(text: str, font: ImageFont.FreeTypeFont, max_width: int) -> str:
    """
    Wrap text to fit within a given width.

    Args:
        text: Text to wrap
        font: Font to use for text
        max_width: Maximum width in pixels

    Returns:
        Wrapped text with newlines
    """
    text = decode_unicode_escapes_to_utf8(text)
    words = text.split()
    lines = []
    current_line = []

    for word in words:
        current_line.append(word)
        line = ' '.join(current_line)
        bbox = font.getbbox(line)
        if bbox[2] > max_width:
            if len(current_line) == 1:
                lines.append(current_line.pop())
            else:
                current_line.pop()
                lines.append(' '.join(current_line))
                current_line = [word]

    if current_line:
        lines.append(' '.join(current_line))

    return '\n'.join(lines)

```

.system\agent\judge.py

```

"""Judge system for evaluating browser-use agent execution traces."""

import base64
import logging
from pathlib import Path
from typing import Literal

from system.llm.messages import (
    BaseMessage,
    ContentPartImageParam,
    ContentPartTextParam,
    ImageURL,
    SystemMessage,
    UserMessage,
)

```

```

logger = logging.getLogger(__name__)

def _encode_image(image_path: str) -> str | None:
    """Encode image to base64 string."""
    try:
        path = Path(image_path)
        if not path.exists():
            return None
        with open(path, 'rb') as f:
            return base64.b64encode(f.read()).decode('utf-8')
    except Exception as e:
        logger.warning(f'Failed to encode image {image_path}: {e}')
    return None

def _truncate_text(text: str, max_length: int, from_beginning: bool = False) -> str:
    """Truncate text to maximum length with eval system indicator."""
    if len(text) <= max_length:
        return text
    if from_beginning:
        return '...[text truncated]' + text[-max_length + 23 :]
    else:
        return text[: max_length - 23] + '...[text truncated]...'

def construct_judge_messages(
    task: str,
    final_result: str,
    agent_steps: list[str],
    screenshot_paths: list[str],
    max_images: int = 10,
    ground_truth: str | None = None,
    use_vision: bool | Literal['auto'] = True,
) -> list[BaseMessage]:
    """
    Construct messages for judge evaluation of agent trace.

    Args:
        task: The original task description
        final_result: The final result returned to the user
        agent_steps: List of formatted agent step descriptions
        screenshot_paths: List of screenshot file paths
        max_images: Maximum number of screenshots to include
        ground_truth: Optional ground truth answer or criteria that must be satisfied for success

    Returns:
        List of messages for LLM judge evaluation
    """
    task_truncated = _truncate_text(task, 40000)
    final_result_truncated = _truncate_text(final_result, 40000)
    steps_text = '\n'.join(agent_steps)
    steps_text_truncated = _truncate_text(steps_text, 40000)

    # Only include screenshots if use_vision is not False
    encoded_images: list[ContentPartImageParam] = []
    if use_vision is not False:
        # Select last N screenshots
        selected_screenshots = screenshot_paths[-max_images:] if len(screenshot_paths) > max_images else screenshot_paths

        # Encode screenshots
        for img_path in selected_screenshots:
            encoded = _encode_image(img_path)
            if encoded:
                encoded_images.append(
                    ContentPartImageParam(
                        image_url=ImageURL(
                            url=f'data:image/png;base64,{encoded}',
                            media_type='image/png',
                        )
                    )
                )

    # Construct messages
    messages = [
        SystemMessage(
            content=[
                Text(
                    task_truncated,
                ),
                Text(
                    final_result_truncated,
                ),
                Text(
                    steps_text_truncated,
                ),
            ],
        ),
        HumanMessage(
            content=[
                Text(
                    ground_truth,
                ),
            ],
        ),
    ]

    return messages

```

```

■# System prompt for judge - conditionally add ground truth section
■ground_truth_section = ''
■if ground_truth:
■    ground_truth_section = ""
■    **GROUND TRUTH VALIDATION (HIGHEST PRIORITY):**
The <ground_truth> section contains verified correct information for this task. This can be:
- **Evaluation criteria**: Specific conditions that must be met (e.g., "The success popup should show up
- **Factual answers**: The correct answer to a question or information retrieval task (e.g. "10/11/24",
- **Expected outcomes**: What should happen after task completion (e.g., "Google Doc must be created", "

The ground truth takes ABSOLUTE precedence over all other evaluation criteria. If the ground truth is no
""

■system_prompt = f""You are an expert judge evaluating browser automation agent performance.

<evaluation_framework>
{ground_truth_section}
**PRIMARY EVALUATION CRITERIA (in order of importance):**
1. **Task Satisfaction (Most Important)**: Did the agent accomplish what the user asked for? Break down
2. **Output Quality**: Is the final result in the correct format and complete? Does it match exactly wha
3. **Tool Effectiveness**: Did the browser interactions work as expected? Were tools used appropriately?
4. **Agent Reasoning**: Quality of decision-making, planning, and problem-solving throughout the trajec
5. **Browser Handling**: Navigation stability, error recovery, and technical execution. If the browser c

**VERDICT GUIDELINES:**
- true: Task completed as requested, human-like execution, all of the users criteria were met and the ag
- false: Task not completed, or only partially completed.

**Examples of task completion verdict:**
- If task asks for 10 items and agent finds 4 items correctly: false
- If task completed to full user requirements but with some errors to improve in the trajectory: true
- If task impossible due to captcha/login requirements: false
- If the trajectory is ideal and the output is perfect: true
- If the task asks to search all headphones in amazon under $100 but the agent searches all headphones a
- If the task asks to research a property and create a google doc with the result but the agents only re
- If the task asks to complete an action on the page, and the agent reports that the action is completed
- If the task asks to use a certain tool or site to complete the task but the agent completes the task w
- If the task asks to look for a section of a page that does not exist: false
- If the agent concludes the task is impossible but it is not: false
- If the agent concludes the task is impossible and it truly is impossible: false
- If the agent is unable to complete the task because no login information was provided and it is truly

**FAILURE CONDITIONS (automatically set verdict to false):**
- Blocked by captcha or missing authentication
- Output format completely wrong or missing
- Infinite loops or severe technical failures
- Critical user requirements ignored
- Page not loaded
- Browser crashed
- Agent could not interact with required UI elements
- The agent moved on from a important step in the task without completing it
- The agent made up content that is not in the screenshot or the page state
- The agent calls done action before completing all key points of the task

**IMPOSSIBLE TASK DETECTION:**
Set `impossible_task` to true when the task fundamentally could not be completed due to:
- Vague or ambiguous task instructions that cannot be reasonably interpreted
- Website genuinely broken or non-functional (be conservative - temporary issues don't count)
- Required links/pages truly inaccessible (404, 403, etc.)
- Task requires authentication/login but no credentials were provided
- Task asks for functionality that doesn't exist on the target site
- Other insurmountable external obstacles beyond the agent's control

Do NOT mark as impossible if:
- Agent made poor decisions but task was achievable
- Temporary page loading issues that could be retried
- Agent didn't try the right approach
- Website works but agent struggled with it

**CAPTCHA DETECTION:**
Set `reached_captcha` to true if:
- Screenshots show captcha challenges (reCAPTCHA, hCaptcha, etc.)

```


- Agent reports being blocked by bot detection
- Error messages indicate captcha/verification requirements
- Any evidence the agent encountered anti-bot measures during execution

****IMPORTANT EVALUATION NOTES:****

- ****evaluate for action**** - For each key step of the trace, double check whether the action that the agent took was correct
- ****screenshot is not entire content**** - The agent has the entire DOM content, but the screenshot is only a portion of it
- ****Penalize poor tool usage**** - Wrong tools, inefficient approaches, ignoring available information.
- ****ignore unexpected dates and times**** - These agent traces are from varying dates, you can assume the date is the date of the trace
- ****IMPORTANT****: be very picky about the user's request - Have very high standard for the agent completion
- ****IMPORTANT****: be initially doubtful of the agent's self reported success, be sure to verify that its actions were successful

</evaluation_framework>

<response_format>

Respond with EXACTLY this JSON structure (no additional text before or after):

```
{
  "reasoning": "Breakdown of user task into key points. Detailed analysis covering: what went well, what went wrong, and what the agent learned from the execution.",
  "verdict": true or false,
  "failure_reason": "Max 5 sentences explanation of why the task was not completed successfully in case of failure.",
  "impossible_task": true or false,
  "reached_captcha": true or false
}
```

Build user prompt with conditional ground truth section

ground_truth_prompt = ''

if ground_truth:

ground_truth_prompt = f"""

<ground_truth>

{ground_truth}

</ground_truth>

"""

user_prompt = f"""

<task>

{task_truncated or 'No task provided'}

</task>

{ground_truth_prompt}

<agent_trajectory>

{steps_text_truncated or 'No agent trajectory provided'}

</agent_trajectory>

<final_result>

{final_result_truncated or 'No final result provided'}

</final_result>

{len(encoded_images)} screenshots from execution are attached.

Evaluate this agent execution given the criteria and respond with the exact JSON structure requested."""

Build messages with screenshots

content_parts: list[ContentPartTextParam | ContentPartImageParam] = [ContentPartTextParam(text=user_prompt)]

content_parts.extend(encoded_images)

return [

SystemMessage(content=system_prompt),

UserMessage(content=content_parts),

]

.\system\agent\prompts.py

```
import importlib.resources
from datetime import datetime
from typing import TYPE_CHECKING, Literal, Optional
from system.dom.views import NodeType, SimplifiedNode
```

```

from system.llm.messages import ContentPartImageParam, ContentPartTextParam, ImageURL, SystemMessage, US
from system.observability import observe_debug
from system.utils import is_new_tab_page, sanitize_surrogates

```

```

if TYPE_CHECKING:
    from system.agent.views import AgentStepInfo
    from system.browser.views import BrowserStateSummary
    from system.filesystem.file_system import FileSystem

```

```

def _is_anthropic_4_5_model(model_name: str | None) -> bool:
    """Check if the model is Claude Opus 4.5 or Haiku 4.5 (requires 4096+ token prompts for caching)."""
    if not model_name:
        return False
    model_lower = model_name.lower()
    # Check for Opus 4.5 or Haiku 4.5 variants
    is_opus_4_5 = 'opus' in model_lower and ('4.5' in model_lower or '4-5' in model_lower)
    is_haiku_4_5 = 'haiku' in model_lower and ('4.5' in model_lower or '4-5' in model_lower)
    return is_opus_4_5 or is_haiku_4_5

```

```

class SystemPrompt:
    def __init__(
        self,
        max_actions_per_step: int = 3,
        override_system_message: str | None = None,
        extend_system_message: str | None = None,
        use_thinking: bool = True,
        flash_mode: bool = False,
        is_anthropic: bool = False,
        is_browser_use_model: bool = False,
        model_name: str | None = None,
    ):
        self.max_actions_per_step = max_actions_per_step
        self.use_thinking = use_thinking
        self.flash_mode = flash_mode
        self.is_anthropic = is_anthropic
        self.is_browser_use_model = is_browser_use_model
        self.model_name = model_name
        # Check if this is an Anthropic 4.5 model that needs longer prompts for caching
        self.is_anthropic_4_5 = _is_anthropic_4_5_model(model_name)
        self.prompt = ''
        if override_system_message is not None:
            self.prompt = override_system_message
        else:
            self._load_prompt_template()
            self.prompt = self.prompt_template.format(max_actions=self.max_actions_per_step)

        if extend_system_message:
            self.prompt += f'\n{extend_system_message}'

        self.system_message = SystemMessage(content=self.prompt, cache=True)

    def _load_prompt_template(self) -> None:
        """Load the prompt template from the markdown file."""
        try:
            # Choose the appropriate template based on model type and mode
            # Browser-use models use simplified prompts optimized for fine-tuned models
            if self.is_browser_use_model:
                if self.flash_mode:
                    template_filename = 'system_prompt_browser_use_flash.md'
                elif self.use_thinking:
                    template_filename = 'system_prompt_browser_use.md'
                else:
                    template_filename = 'system_prompt_browser_use_no_thinking.md'
            # Anthropic 4.5 models (Opus 4.5, Haiku 4.5) need 4096+ token prompts for caching
            elif self.is_anthropic_4_5 and self.flash_mode:
                template_filename = 'system_prompt_anthropic_flash.md'
            elif self.flash_mode and self.is_anthropic:
                template_filename = 'system_prompt_flash_anthropic.md'
            elif self.flash_mode:
                template_filename = 'system_prompt_flash.md'

```

```

    elif self.use_thinking:
        template_filename = 'system_prompt.md'
    else:
        template_filename = 'system_prompt_no_thinking.md'

    # This works both in development and when installed as a package
    with (
        importlib.resources.files('system.agent.system_prompts')
        .joinpath(template_filename)
        .open('r', encoding='utf-8') as f
    ):
        self.prompt_template = f.read()
    except Exception as e:
        raise RuntimeError(f'Failed to load system prompt template: {e}')

    def get_system_message(self) -> SystemMessage:
        """
        Get the system prompt for the agent.

        Returns:
            SystemMessage: Formatted system prompt
        """
        return self.system_message

class AgentMessagePrompt:
    vision_detail_level: Literal['auto', 'low', 'high']

    def __init__(
        self,
        browser_state_summary: 'BrowserStateSummary',
        file_system: 'FileSystem',
        agent_history_description: str | None = None,
        read_state_description: str | None = None,
        task: str | None = None,
        include_attributes: list[str] | None = None,
        step_info: Optional['AgentStepInfo'] = None,
        page_filtered_actions: str | None = None,
        max_clickable_elements_length: int = 40000,
        sensitive_data: str | None = None,
        available_file_paths: list[str] | None = None,
        screenshots: list[str] | None = None,
        vision_detail_level: Literal['auto', 'low', 'high'] = 'auto',
        include_recent_events: bool = False,
        sample_images: list[ContentPartTextParam | ContentPartImageParam] | None = None,
        read_state_images: list[dict] | None = None,
        llm_screenshot_size: tuple[int, int] | None = None,
        unavailable_skills_info: str | None = None,
        plan_description: str | None = None,
    ):
        self.browser_state: 'BrowserStateSummary' = browser_state_summary
        self.file_system: 'FileSystem | None' = file_system
        self.agent_history_description: str | None = agent_history_description
        self.read_state_description: str | None = read_state_description
        self.task: str | None = task
        self.include_attributes = include_attributes
        self.step_info = step_info
        self.page_filtered_actions: str | None = page_filtered_actions
        self.max_clickable_elements_length: int = max_clickable_elements_length
        self.sensitive_data: str | None = sensitive_data
        self.available_file_paths: list[str] | None = available_file_paths
        self.screenshots = screenshots or []
        self.vision_detail_level = vision_detail_level
        self.include_recent_events = include_recent_events
        self.sample_images = sample_images or []
        self.read_state_images = read_state_images or []
        self.unavailable_skills_info: str | None = unavailable_skills_info
        self.plan_description: str | None = plan_description
        self.llm_screenshot_size = llm_screenshot_size
        assert self.browser_state

    def _extract_page_statistics(self) -> dict[str, int]:

```

```

"""Extract high-level page statistics from DOM tree for LLM context"""
stats = {
    'links': 0,
    'iframes': 0,
    'shadow_open': 0,
    'shadow_closed': 0,
    'scroll_containers': 0,
    'images': 0,
    'interactive_elements': 0,
    'total_elements': 0,
}

if not self.browser_state.dom_state or not self.browser_state.dom_state._root:
    return stats

def traverse_node(node: SimplifiedNode) -> None:
    """Recursively traverse simplified DOM tree to count elements"""
    if not node or not node.original_node:
        return

    original = node.original_node
    stats['total_elements'] += 1

    # Count by node type and tag
    if original.node_type == NodeType.ELEMENT_NODE:
        tag = original.tag_name.lower() if original.tag_name else ''

        if tag == 'a':
            stats['links'] += 1
        elif tag in ('iframe', 'frame'):
            stats['iframes'] += 1
        elif tag == 'img':
            stats['images'] += 1

        # Check if scrollable
        if original.is_actually_scrollable:
            stats['scroll_containers'] += 1

        # Check if interactive
        if node.is_interactive:
            stats['interactive_elements'] += 1

        # Check if this element hosts shadow DOM
        if node.is_shadow_host:
            # Check if any shadow children are closed
            has_closed_shadow = any(
                child.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE
                and child.original_node.shadow_root_type
                and child.original_node.shadow_root_type.lower() == 'closed'
                for child in node.children
            )
            if has_closed_shadow:
                stats['shadow_closed'] += 1
            else:
                stats['shadow_open'] += 1

        elif original.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
            # Shadow DOM fragment - these are the actual shadow roots
            # But don't double-count since we count them at the host level above
            pass

    # Traverse children
    for child in node.children:
        traverse_node(child)

traverse_node(self.browser_state.dom_state._root)
return stats

@observe_debug(ignore_input=True, ignore_output=True, name='_get_browser_state_description')
def _get_browser_state_description(self) -> str:
    # Extract page statistics first
    page_stats = self._extract_page_statistics()

```

```

    ## Format statistics
    stats_text = '<page_stats>'
    if page_stats['total_elements'] < 10:
        stats_text += 'Page appears empty (SPA not loaded?) - '
        stats_text += f'{page_stats["links"]} links, {page_stats["interactive_elements"]} interactive, '
        stats_text += f'{page_stats["iframes"]} iframes'
        if page_stats['shadow_open'] > 0 or page_stats['shadow_closed'] > 0:
            stats_text += f', {page_stats["shadow_open"]} shadow(open), {page_stats["shadow_closed"]} shadow(close)'
        if page_stats['images'] > 0:
            stats_text += f', {page_stats["images"]} images'
        stats_text += f', {page_stats["total_elements"]} total elements'
        stats_text += '</page_stats>\n'

    elements_text = self.browser_state.dom_state.llm_representation(include_attributes=self.include_attributes)

    if len(elements_text) > self.max_clickable_elements_length:
        elements_text = elements_text[: self.max_clickable_elements_length]
        truncated_text = f' (truncated to {self.max_clickable_elements_length} characters)'
    else:
        truncated_text = ''

    has_content_above = False
    has_content_below = False
    ## Enhanced page information for the model
    page_info_text = ''
    if self.browser_state.page_info:
        pi = self.browser_state.page_info
        ## Compute page statistics dynamically
        pages_above = pi.pixels_above / pi.viewport_height if pi.viewport_height > 0 else 0
        pages_below = pi.pixels_below / pi.viewport_height if pi.viewport_height > 0 else 0
        has_content_above = pages_above > 0
        has_content_below = pages_below > 0
        total_pages = pi.page_height / pi.viewport_height if pi.viewport_height > 0 else 0
        current_page_position = pi.scroll_y / max(pi.page_height - pi.viewport_height, 1)
        page_info_text = '<page_info>'
        page_info_text += f'{pages_above:.1f} above, '
        page_info_text += f'{pages_below:.1f} below '

    page_info_text += '</page_info>\n'
    ## , at {current_page_position:.0%} of page
    if elements_text != '':
        if not has_content_above:
            elements_text = f'[Start of page]\n{elements_text}'
        if not has_content_below:
            elements_text = f'{elements_text}\n[End of page]'
    else:
        elements_text = 'empty page'

    tabs_text = ''
    current_tab_candidates = []

    ## Find tabs that match both URL and title to identify current tab more reliably
    for tab in self.browser_state.tabs:
        if tab.url == self.browser_state.url and tab.title == self.browser_state.title:
            current_tab_candidates.append(tab.target_id)

    ## If we have exactly one match, mark it as current
    ## Otherwise, don't mark any tab as current to avoid confusion
    current_target_id = current_tab_candidates[0] if len(current_tab_candidates) == 1 else None

    for tab in self.browser_state.tabs:
        tabs_text += f'Tab {tab.target_id[-4:]}: {tab.url} - {tab.title[:30]}\n'

    current_tab_text = f'Current tab: {current_target_id[-4:]}' if current_target_id is not None else ''

    ## Check if current page is a PDF viewer and add appropriate message
    pdf_message = ''
    if self.browser_state.is_pdf_viewer:
        pdf_message = (
            'PDF viewer cannot be rendered. In this page, DO NOT use the extract action as PDF content cannot be rendered.'
        )
    pdf_message += (

```

```

#####'Use the read_file action on the downloaded PDF in available_file_paths to read the full text content
#####)

##### Add recent events if available and requested
#####recent_events_text = ''
#####if self.include_recent_events and self.browser_state.recent_events:
#####recent_events_text = f'Recent browser events: {self.browser_state.recent_events}\n'

##### Add closed popup messages if any
#####closed_popups_text = ''
#####if self.browser_state.closed_popup_messages:
#####closed_popups_text = 'Auto-closed JavaScript dialogs:\n'
#####for popup_msg in self.browser_state.closed_popup_messages:
#####closed_popups_text += f' - {popup_msg}\n'
#####closed_popups_text += '\n'

#####browser_state = f""#{stats_text}{current_tab_text}
Available tabs:
{tabs_text}
{page_info_text}
{recent_events_text}{closed_popups_text}{pdf_message}Interactive elements{truncated_text}:
{elements_text}
""
#####return browser_state

#####def _get_agent_state_description(self) -> str:
#####if self.step_info:
#####step_info_description = f'Step{self.step_info.step_number + 1} maximum:{self.step_info.max_steps}\n'
#####else:
#####step_info_description = ''

#####time_str = datetime.now().strftime('%Y-%m-%d')
#####step_info_description += f'Today:{time_str}'

#####_todo_contents = self.file_system.get_todo_contents() if self.file_system else ''
#####if not len(_todo_contents):
#####_todo_contents = '[empty todo.md, fill it when applicable]'

#####agent_state = f""
<user_request>
{self.task}
</user_request>
<file_system>
{self.file_system.describe() if self.file_system else 'No file system available'}
</file_system>
<todo_contents>
{_todo_contents}
</todo_contents>
""
#####if self.plan_description:
#####agent_state += f'<plan>\n{self.plan_description}\n</plan>\n'

#####if self.sensitive_data:
#####agent_state += f'<sensitive_data>{self.sensitive_data}</sensitive_data>\n'

#####agent_state += f'<step_info>{step_info_description}</step_info>\n'
#####if self.available_file_paths:
#####available_file_paths_text = '\n'.join(self.available_file_paths)
#####agent_state += f'<available_file_paths>{available_file_paths_text}\nUse with absolute paths</available_file_paths>\n'
#####return agent_state

#####def _resize_screenshot(self, screenshot_b64: str) -> str:
#####"""Resize screenshot to llm_screenshot_size if configured."""
#####if not self.llm_screenshot_size:
#####return screenshot_b64

#####try:
#####import base64
#####import logging
#####from io import BytesIO

#####from PIL import Image

```

```

#####img = Image.open(BytesIO(base64.b64decode(screenshot_b64)))
#####if img.size == self.llm_screenshot_size:
#####return screenshot_b64

#####logging.getLogger(__name__).info(
#####f'■ Resizing screenshot from {img.size[0]}x{img.size[1]} to {self.llm_screenshot_size[0]}x{self.llm_screenshot_size[1]}')
#####)

#####img_resized = img.resize(self.llm_screenshot_size, Image.Resampling.LANCZOS)
#####buffer = BytesIO()
#####img_resized.save(buffer, format='PNG')
#####return base64.b64encode(buffer.getvalue()).decode('utf-8')
#####except Exception as e:
#####logging.getLogger(__name__).warning(f'Failed to resize screenshot: {e}, using original')
#####return screenshot_b64

#####@observe_debug(ignore_input=True, ignore_output=True, name='get_user_message')
#####def get_user_message(self, use_vision: bool = True) -> UserMessage:
#####    """Get complete state as a single cached message"""
#####    # Don't pass screenshot to model if page is a new tab page, step is 0, and there's only one tab
#####    if (
#####        is_new_tab_page(self.browser_state.url)
#####        and self.step_info is not None
#####        and self.step_info.step_number == 0
#####        and len(self.browser_state.tabs) == 1
#####    ):
#####        use_vision = False

#####    # Build complete state description
#####    state_description = (
#####        '<agent_history>\n'
#####        + (self.agent_history_description.strip('\n') if self.agent_history_description else '')
#####        + '\n</agent_history>\n\n'
#####    )
#####    state_description += '<agent_state>\n' + self._get_agent_state_description().strip('\n') + '\n</agent_state>\n'
#####    state_description += '<browser_state>\n' + self._get_browser_state_description().strip('\n') + '\n</browser_state>\n'
#####    # Only add read_state if it has content
#####    read_state_description = self.read_state_description.strip('\n').strip() if self.read_state_description else ''
#####    if read_state_description:
#####        state_description += '<read_state>\n' + read_state_description + '\n</read_state>\n'

#####    if self.page_filtered_actions:
#####        state_description += '<page_specific_actions>\n'
#####        state_description += self.page_filtered_actions + '\n'
#####        state_description += '</page_specific_actions>\n'

#####    # Add unavailable skills information if any
#####    if self.unavailable_skills_info:
#####        state_description += '\n' + self.unavailable_skills_info + '\n'

#####    # Sanitize surrogates from all text content
#####    state_description = sanitize_surrogates(state_description)

#####    # Check if we have images to include (from read_file action)
#####    has_images = bool(self.read_state_images)

#####    if (use_vision is True and self.screenshots) or has_images:
#####        # Start with text description
#####        content_parts: list[ContentPartTextParam | ContentPartImageParam] = [ContentPartTextParam(text=state_description)]

#####        # Add sample images
#####        content_parts.extend(self.sample_images)

#####        # Add screenshots with labels
#####        for i, screenshot in enumerate(self.screenshots):
#####            if i == len(self.screenshots) - 1:
#####                label = 'Current screenshot:'
#####            else:
#####                # Use simple, accurate labeling since we don't have actual step timing info
#####                label = 'Previous screenshot:'

#####        # Add label as text content

```

```

#####content_parts.append(ContentPartTextParam(text=label))

###### Resize screenshot if llm_screenshot_size is configured
#####processed_screenshot = self._resize_screenshot(screenshot)

###### Add the screenshot
#####content_parts.append(
#####ContentPartImageParam(
#####image_url=ImageURL(
#####url=f'data:image/png;base64,{processed_screenshot}',
#####media_type='image/png',
#####detail=self.vision_detail_level,
#####),
#####)
#####)

###### Add read_state images (from read_file action) before screenshots
#####for img_data in self.read_state_images:
#####img_name = img_data.get('name', 'unknown')
#####img_base64 = img_data.get('data', '')

#####if not img_base64:
#####continue

###### Detect image format from name
#####if img_name.lower().endswith('.png'):
#####media_type = 'image/png'
#####else:
#####media_type = 'image/jpeg'

###### Add label
#####content_parts.append(ContentPartTextParam(text=f'Image from file: {img_name}'))

###### Add the image
#####content_parts.append(
#####ContentPartImageParam(
#####image_url=ImageURL(
#####url=f'data:{media_type};base64,{img_base64}',
#####media_type=media_type,
#####detail=self.vision_detail_level,
#####),
#####)
#####)

#####return UserMessage(content=content_parts, cache=True)

#####return UserMessage(content=state_description, cache=True)

def get_rerun_summary_prompt(original_task: str, total_steps: int, success_count: int, error_count: int):
#####return f'''You are analyzing the completion of a rerun task. Based on the screenshot and execution info

Original task: {original_task}

Execution statistics:
- Total steps: {total_steps}
- Successful steps: {success_count}
- Failed steps: {error_count}

Analyze the screenshot to determine:
1. Whether the task completed successfully
2. What the final state shows
3. Overall completion status (complete/partial/failed)

Respond with:
- summary: A clear, concise summary of what happened during the rerun
- success: Whether the task completed successfully (true/false)
- completion_status: One of "complete", "partial", or "failed"'''

def get_rerun_summary_message(prompt: str, screenshot_b64: str | None = None) -> UserMessage:
#####'''

```


■Build a UserMessage for rerun summary generation.

■Args:

■prompt: The prompt text

■screenshot_b64: Optional base64-encoded screenshot

■Returns:

■UserMessage with prompt and optional screenshot

■"""

■if screenshot_b64:

■# With screenshot: use multi-part content

■content_parts: list[ContentPartTextParam | ContentPartImageParam] = [

■ContentPartTextParam(type='text', text=prompt),

■ContentPartImageParam(

■type='image_url',

■image_url=ImageURL(url=f'data:image/png;base64,{screenshot_b64}'),

■),

■]

■return UserMessage(content=content_parts)

■else:

■# Without screenshot: use simple string content

■return UserMessage(content=prompt)

def get_ai_step_system_prompt() -> str:

■"""

■Get system prompt for AI step action used during rerun.

■Returns:

■System prompt string for AI step

■"""

■return """

You are an expert at extracting data from webpages.

<input>

You will be given:

1. A query describing what to extract

2. The markdown of the webpage (filtered to remove noise)

3. Optionally, a screenshot of the current page state

</input>

<instructions>

- Extract information from the webpage that is relevant to the query

- ONLY use the information available in the webpage - do not make up information

- If the information is not available, mention that clearly

- If the query asks for all items, list all of them

</instructions>

<output>

- Present ALL relevant information in a concise way

- Do not use conversational format - directly output the relevant information

- If information is unavailable, state that clearly

</output>

""".strip()

def get_ai_step_user_prompt(query: str, stats_summary: str, content: str) -> str:

■"""

■Build user prompt for AI step action.

■Args:

■query: What to extract or analyze

■stats_summary: Content statistics summary

■content: Page markdown content

■Returns:

■Formatted prompt string

■"""

■return f'<query>\n{query}\n</query>\n\n<content_stats>\n{stats_summary}\n</content_stats>\n\n<webpage_c

.lsystemagent\service.py

```
import asyncio
import gc
import inspect
import json
import logging
import re
import tempfile
import time
from collections.abc import Awaitable, Callable
from pathlib import Path
from typing import TYPE_CHECKING, Any, Generic, Literal, TypeVar, cast
from urllib.parse import urlparse

if TYPE_CHECKING:
    from system.skills.views import Skill

from dotenv import load_dotenv

from system.agent.cloud_events import (
    CreateAgentOutputFileEvent,
    CreateAgentSessionEvent,
    CreateAgentStepEvent,
    CreateAgentTaskEvent,
    UpdateAgentTaskEvent,
)
from system.agent.message_manager.utils import save_conversation
from system.llm.base import BaseChatModel
from system.llm.exceptions import ModelProviderError, ModelRateLimitError
from system.llm.messages import BaseMessage, ContentPartImageParam, ContentPartTextParam, UserMessage
from system.tokens.service import TokenCost

load_dotenv()

from bubus import EventBus
from pydantic import BaseModel, ValidationError
from uuid_extensions import uuid7str

from system import Browser, BrowserProfile, BrowserSession
from system.agent.judge import construct_judge_messages

# Lazy import for gif to avoid heavy agent.views import at startup
# from system.agent.gif import create_history_gif
from system.agent.message_manager.service import (
    MessageManager,
)
from system.agent.prompts import SystemPrompt
from system.agent.views import (
    ActionLoopDetector,
    ActionResult,
    AgentError,
    AgentHistory,
    AgentHistoryList,
    AgentOutput,
    AgentSettings,
    AgentState,
    AgentStepInfo,
    AgentStructuredOutput,
    BrowserStateHistory,
    DetectedVariable,
    JudgementResult,
    MessageCompactionSettings,
    PageFingerprint,
    PlanItem,
    StepMetadata,
)
from system.agent.workflow import WorkflowRecorder, WorkflowTemplate, WorkflowExecutor
from system.browser.events import _get_timeout
from system.browser.session import DEFAULT_BROWSER_PROFILE
```

```

from system.browser.views import BrowserStateSummary
from system.config import CONFIG
from system.dom.views import DOMInteractedElement, MatchLevel
from system.filesystem.file_system import FileSystem
from system.observability import observe, observe_debug
from system.telemetry.service import ProductTelemetry
from system.telemetry.views import AgentTelemetryEvent
from system.tools.registry.views import ActionModel
from system.tools.service import Tools
from system.utils import (
    ■URL_PATTERN,
    ■_log_pretty_path,
    ■check_latest_browser_use_version,
    ■get_browser_use_version,
    ■time_execution_async,
    ■time_execution_sync,
)

logger = logging.getLogger(__name__)

def log_response(response: AgentOutput, registry=None, logger=None) -> None:
    ■"""Utility function to log the model's response."""

    ■# Use module logger if no logger provided
    ■if logger is None:
        ■■logger = logging.getLogger(__name__)

    ■# Only log thinking if it's present
    ■if response.current_state.thinking:
        ■■logger.debug(f'■ Thinking:\n{response.current_state.thinking}')

```

```

■■browser_profile: BrowserProfile | None = None,
■■browser_session: BrowserSession | None = None,
■■browser: Browser | None = None, # Alias for browser_session
■■tools: Tools[Context] | None = None,
■■controller: Tools[Context] | None = None, # Alias for tools
■■# Skills integration
■■skill_ids: list[str | Literal['*']] | None = None,
■■skills: list[str | Literal['*']] | None = None, # Alias for skill_ids
■■skill_service: Any | None = None,
■■# Initial agent run parameters
■■sensitive_data: dict[str, str | dict[str, str]] | None = None,
■■initial_actions: list[dict[str, dict[str, Any]]] | None = None,
■■# Cloud Callbacks
■■register_new_step_callback: (
■■■■Callable[['BrowserStateSummary', 'AgentOutput', int], None] # Sync callback
■■■■| Callable[['BrowserStateSummary', 'AgentOutput', int], Awaitable[None]] # Async callback
■■■■) = None,
■■register_done_callback: (
■■■■Callable[['AgentHistoryList'], Awaitable[None]] # Async Callback
■■■■| Callable[['AgentHistoryList'], None] # Sync Callback
■■■■) = None,
■■register_external_agent_status_raise_error_callback: Callable[[], Awaitable[bool]] | None = None,
■■register_should_stop_callback: Callable[[], Awaitable[bool]] | None = None,
■■# Agent settings
■■output_model_schema: type[AgentStructuredOutput] | None = None,
■■extraction_schema: dict | None = None,
■■use_vision: bool | Literal['auto'] = True,
■■save_conversation_path: str | Path | None = None,
■■save_conversation_path_encoding: str | None = 'utf-8',
■■max_failures: int = 5,
■■override_system_message: str | None = None,
■■extend_system_message: str | None = None,
■■generate_gif: bool | str = False,
■■available_file_paths: list[str] | None = None,
■■include_attributes: list[str] | None = None,
■■max_actions_per_step: int = 5,
■■use_thinking: bool = True,
■■flash_mode: bool = False,
■■demo_mode: bool | None = None,
■■max_history_items: int | None = None,
■■page_extraction_llm: BaseChatModel | None = None,
■■fallback_llm: BaseChatModel | None = None,
■■use_judge: bool = True,
■■ground_truth: str | None = None,
■■judge_llm: BaseChatModel | None = None,
■■injected_agent_state: AgentState | None = None,
■■source: str | None = None,
■■file_system_path: str | None = None,
■■task_id: str | None = None,
■■calculate_cost: bool = False,
■■display_files_in_done_text: bool = True,
■■include_tool_call_examples: bool = False,
■■vision_detail_level: Literal['auto', 'low', 'high'] = 'auto',
■■llm_timeout: int | None = None,
■■step_timeout: int = 180,
■■directly_open_url: bool = True,
■■include_recent_events: bool = False,
■■sample_images: list[ContentPartTextParam | ContentPartImageParam] | None = None,
■■final_response_after_failure: bool = True,
■■enable_planning: bool = True,
■■planning_replan_on_stall: int = 3,
■■planning_exploration_limit: int = 5,
■■loop_detection_window: int = 20,
■■loop_detection_enabled: bool = True,
■■llm_screenshot_size: tuple[int, int] | None = None,
■■message_compaction: MessageCompactionSettings | bool | None = True,
■■max_clickable_elements_length: int = 40000,
■■url_shortening_limit: int = 25,
■■workflow_template: WorkflowTemplate | None = None,
■■**kwargs,

```

```

    ):
    ## Validate llm_screenshot_size
    if llm_screenshot_size is not None:
        if not isinstance(llm_screenshot_size, tuple) or len(llm_screenshot_size) != 2:
            raise ValueError('llm_screenshot_size must be a tuple of (width, height)')
        width, height = llm_screenshot_size
        if not isinstance(width, int) or not isinstance(height, int):
            raise ValueError('llm_screenshot_size dimensions must be integers')
        if width < 100 or height < 100:
            raise ValueError('llm_screenshot_size dimensions must be at least 100 pixels')
        self.logger.info(f'LLM screenshot resizing enabled: {width}x{height}')
    if llm is None:
        default_llm_name = CONFIG.DEFAULT_LLM
        if default_llm_name:
            from system.llm.models import get_llm_by_name

        llm = get_llm_by_name(default_llm_name)
    else:
        ## No default LLM specified, use the original default
        from system import ChatBrowserUse

        llm = ChatBrowserUse()

    ## set flashmode = True if llm is ChatBrowserUse
    if llm.provider == 'browser-use':
        flash_mode = True

    ## Flash mode strips plan fields from the output schema, so planning is structurally impossible
    if flash_mode:
        enable_planning = False

    ## Auto-configure llm_screenshot_size for Claude Sonnet models
    if llm_screenshot_size is None:
        model_name = getattr(llm, 'model', '')
        if isinstance(model_name, str) and model_name.startswith('claude-sonnet'):
            llm_screenshot_size = (1400, 850)
            logger.info('Auto-configured LLM screenshot size for Claude Sonnet: 1400x850')

    if page_extraction_llm is None:
        page_extraction_llm = llm
    if judge_llm is None:
        judge_llm = llm
    if available_file_paths is None:
        available_file_paths = []

    ## Set timeout based on model name if not explicitly provided
    if llm_timeout is None:

        def _get_model_timeout(llm_model: BaseChatModel) -> int:
            """Determine timeout based on model name"""
            model_name = getattr(llm_model, 'model', '').lower()
            if 'gemini' in model_name:
                if '3-pro' in model_name:
                    return 90
                return 75
            elif 'groq' in model_name:
                return 30
            elif 'o3' in model_name or 'claude' in model_name or 'sonnet' in model_name or 'deepseek' in model_name:
                return 90
            else:
                return 75 # Default timeout

        llm_timeout = _get_model_timeout(llm)

    self.id = task_id or uuid7str()
    self.task_id: str = self.id
    self.session_id: str = uuid7str()

    base_profile = browser_profile or DEFAULT_BROWSER_PROFILE
    if base_profile is DEFAULT_BROWSER_PROFILE:
        base_profile = base_profile.model_copy()
    if demo_mode is not None and base_profile.demo_mode != demo_mode:

```

```

    self.base_profile = base_profile.model_copy(update={'demo_mode': demo_mode})
    self.browser_profile = base_profile

    ## Handle browser vs browser_session parameter (browser takes precedence)
    if browser and browser_session:
        raise ValueError('Cannot specify both "browser" and "browser_session" parameters. Use "browser" for browser or browser_session')
    self.browser_session = browser or browser_session

    self.injected_browser_session = (browser_session is not None)

    if browser_session is not None and demo_mode is not None and browser_session.browser_profile.demo_mode:
        self.browser_session.browser_profile = browser_session.browser_profile.model_copy(update={'demo_mode': demo_mode})

    self.browser_session = browser_session or BrowserSession(
        browser_profile=self.browser_profile,
        id=uuid7str()[-4:] + self.id[-4:], # re-use the same 4-char suffix so they show up together in logs
    )

    self._demo_mode_enabled: bool = bool(self.browser_profile.demo_mode) if self.browser_session else False
    if self._demo_mode_enabled and getattr(self.browser_profile, 'headless', False):
        self.logger.warning(
            'Demo mode is enabled but the browser is headless=True; set headless=False to view the in-browser page'
        )

    ## Initialize available file paths as direct attribute
    self.available_file_paths = available_file_paths

    ## Set up tools first (needed to detect output_model_schema)
    if tools is not None:
        self.tools = tools
    elif controller is not None:
        self.tools = controller
    else:
        ## Exclude screenshot tool when use_vision is not auto
        exclude_actions = ['screenshot'] if use_vision != 'auto' else []
        self.tools = Tools(exclude_actions=exclude_actions, display_files_in_done_text=display_files_in_done_text)

    ## Enforce screenshot exclusion when use_vision != 'auto', even if user passed custom tools
    if use_vision != 'auto':
        self.tools.exclude_action('screenshot')

    ## Enable coordinate clicking for models that support it
    model_name = getattr(llm, 'model', '').lower()
    supports_coordinate_clicking = any(
        pattern in model_name for pattern in ['claude-sonnet-4', 'claude-opus-4', 'gemini-3-pro', 'browser-use']
    )
    if supports_coordinate_clicking:
        self.tools.set_coordinate_clicking(True)

    ## Handle skills vs skill_ids parameter (skills takes precedence)
    if skills and skill_ids:
        raise ValueError('Cannot specify both "skills" and "skill_ids" parameters. Use "skills" for the clear list of skills')
    self.skill_ids = skills or skill_ids

    ## Skills integration - use injected service or create from skill_ids
    self.skill_service = None
    self._skills_registered = False
    if skill_service is not None:
        self.skill_service = skill_service
    elif skill_ids:
        from system.skills import SkillService

        self.skill_service = SkillService(skill_ids=skill_ids)

    ## Structured output - use explicit param or detect from tools
    tools_output_model = self.tools.get_output_model()
    if output_model_schema is not None and tools_output_model is not None:
        ## Both provided - warn if they differ
        if output_model_schema is not tools_output_model:
            self.logger.warning(
                f'output_model_schema ({output_model_schema.__name__}) differs from Tools output_model '
                f'({tools_output_model.__name__}). Using Agent output_model_schema.'
            )

```

```

#####)
#####elif output_model_schema is None and tools_output_model is not None:
#####    # Only tools has it - use that (cast is safe: both are BaseModel subclasses)
#####    output_model_schema = cast(type[AgentStructuredOutput], tools_output_model)
#####    self.output_model_schema = output_model_schema
#####    if self.output_model_schema is not None:
#####        self.tools.use_structured_output_action(self.output_model_schema)

#####    # Extraction schema: explicit param takes priority, otherwise auto-bridge from output_model_schema
#####    self.extraction_schema = extraction_schema
#####    if self.extraction_schema is None and self.output_model_schema is not None:
#####        self.extraction_schema = self.output_model_schema.model_json_schema()

#####    # Core components - task enhancement now has access to output_model_schema from tools
#####    self.task = self._enhance_task_with_schema(task, output_model_schema)
#####    self.llm = llm
#####    self.judge_llm = judge_llm

#####    # Fallback LLM configuration
#####    self._fallback_llm: BaseChatModel | None = fallback_llm
#####    self._using_fallback_llm: bool = False
#####    self._original_llm: BaseChatModel = llm # Store original for reference
#####    self.directly_open_url = directly_open_url
#####    self.include_recent_events = include_recent_events
#####    self._url_shortening_limit = _url_shortening_limit

#####    self.sensitive_data = sensitive_data

#####    self.sample_images = sample_images

#####    if isinstance(message_compaction, bool):
#####        message_compaction = MessageCompactionSettings(enabled=message_compaction)

#####    self.settings = AgentSettings(
#####        use_vision=use_vision,
#####        vision_detail_level=vision_detail_level,
#####        save_conversation_path=save_conversation_path,
#####        save_conversation_path_encoding=save_conversation_path_encoding,
#####        max_failures=max_failures,
#####        override_system_message=override_system_message,
#####        extend_system_message=extend_system_message,
#####        generate_gif=generate_gif,
#####        include_attributes=include_attributes,
#####        max_actions_per_step=max_actions_per_step,
#####        use_thinking=use_thinking,
#####        flash_mode=flash_mode,
#####        max_history_items=max_history_items,
#####        page_extraction_llm=page_extraction_llm,
#####        calculate_cost=calculate_cost,
#####        include_tool_call_examples=include_tool_call_examples,
#####        llm_timeout=llm_timeout,
#####        step_timeout=step_timeout,
#####        final_response_after_failure=final_response_after_failure,
#####        use_judge=use_judge,
#####        ground_truth=ground_truth,
#####        enable_planning=enable_planning,
#####        planning_replan_on_stall=planning_replan_on_stall,
#####        planning_exploration_limit=planning_exploration_limit,
#####        loop_detection_window=loop_detection_window,
#####        loop_detection_enabled=loop_detection_enabled,
#####        message_compaction=message_compaction,
#####        max_clickable_elements_length=max_clickable_elements_length,
#####    )

#####    # Token cost service
#####    self.token_cost_service = TokenCost(include_cost=calculate_cost)
#####    self.token_cost_service.register_llm(llm)
#####    self.token_cost_service.register_llm(page_extraction_llm)
#####    self.token_cost_service.register_llm(judge_llm)
#####    if self.settings.message_compaction and self.settings.message_compaction.compaction_llm:
#####        self.token_cost_service.register_llm(self.settings.message_compaction.compaction_llm)
#####    # Initialize state

```

```

self.state = injected_agent_state or AgentState()

## Configure loop detector window size from settings
self.state.loop_detector.window_size = self.settings.loop_detection_window

## Initialize history
self.history = AgentHistoryList(history=[], usage=None)

## Initialize agent directory
import time

timestamp = int(time.time())
base_tmp = Path(tempfile.gettempdir())
self.agent_directory = base_tmp / f'browser_use_agent_{self.id}_{timestamp}'

## Initialize file system and screenshot service
self._set_file_system(file_system_path)
self._set_screenshot_service()

## Execute workflow deterministically tracking
self.workflow_executor = WorkflowExecutor(workflow_template) if workflow_template else None

## Action setup
self._setup_action_models()
self._set_browser_use_version_and_source(source)

initial_url = None

## only load url if no initial actions are provided
if self.directly_open_url and not self.state.follow_up_task and not initial_actions:
    initial_url = self._extract_start_url(self.task)
    if initial_url:
        self.logger.info(f'Found URL in task: {initial_url}, adding as initial action...')
        initial_actions = [{'navigate': {'url': initial_url, 'new_tab': False}}]

self.initial_url = initial_url

self.initial_actions = self._convert_initial_actions(initial_actions) if initial_actions else None
## Verify we can connect to the model
self._verify_and_setup_llm()

## TODO: move this logic to the LLMs
## Handle users trying to use use_vision=True with DeepSeek models
if 'deepseek' in self.llm.model.lower():
    self.logger.warning('DeepSeek models do not support use_vision=True yet. Setting use_vision=False')
    self.settings.use_vision = False

## Handle users trying to use use_vision=True with XAI models that don't support it
## grok-3 variants and grok-code don't support vision; grok-2 and grok-4 do
model_lower = self.llm.model.lower()
if 'grok-3' in model_lower or 'grok-code' in model_lower:
    self.logger.warning('This XAI model does not support use_vision=True yet. Setting use_vision=False')
    self.settings.use_vision = False

self.logger.debug(
    f'{" +vision" if self.settings.use_vision else ""}'
    f' extraction_model={self.settings.page_extraction_llm.model if self.settings.page_extraction_llm else ""}'
    f'{" +file_system" if self.file_system else ""}'
)

## Store llm_screenshot_size in browser_session so tools can access it
self.browser_session.llm_screenshot_size = llm_screenshot_size

## Check if LLM is ChatAnthropic instance
is_anthropic = False

## Check if model is a browser-use fine-tuned model (uses simplified prompts)
is_browser_use_model = 'browser-use/' in self.llm.model.lower()

## Initialize message manager with state
## Initial system prompt with all actions - will be updated during each step
self._message_manager = MessageManager(

```



```

        task=self.task,
        system_message=SystemPrompt(
            max_actions_per_step=self.settings.max_actions_per_step,
            override_system_message=override_system_message,
            extend_system_message=extend_system_message,
            use_thinking=self.settings.use_thinking,
            flash_mode=self.settings.flash_mode,
            is_anthropic=is_anthropic,
            is_browser_use_model=is_browser_use_model,
            model_name=self.llm.model,
        ).get_system_message(),
        file_system=self.file_system,
        state=self.state.message_manager_state,
        use_thinking=self.settings.use_thinking,
        # Settings that were previously in MessageManagerSettings
        include_attributes=self.settings.include_attributes,
        sensitive_data=sensitive_data,
        max_history_items=self.settings.max_history_items,
        vision_detail_level=self.settings.vision_detail_level,
        include_tool_call_examples=self.settings.include_tool_call_examples,
        include_recent_events=self.include_recent_events,
        sample_images=self.sample_images,
        llm_screenshot_size=llm_screenshot_size,
        max_clickable_elements_length=self.settings.max_clickable_elements_length,
    )

    if self.sensitive_data:
        # Check if sensitive_data has domain-specific credentials
        has_domain_specific_credentials = any(isinstance(v, dict) for v in self.sensitive_data.values())

        # If no allowed_domains are configured, show a security warning
        if not self.browser_profile.allowed_domains:
            self.logger.warning(
                'Agent(sensitive_data=.....) was provided but Browser(allowed_domains=[...]) is not locked'
                '\n    If the agent visits a malicious website and encounters a prompt-injection attack, y'
            )

        # If we're using domain-specific credentials, validate domain patterns
        elif has_domain_specific_credentials:
            # For domain-specific format, ensure all domain patterns are included in allowed_domains
            domain_patterns = [k for k, v in self.sensitive_data.items() if isinstance(v, dict)]

            # Validate each domain pattern against allowed_domains
            for domain_pattern in domain_patterns:
                is_allowed = False
                for allowed_domain in self.browser_profile.allowed_domains:
                    # Special cases that don't require URL matching
                    if domain_pattern == allowed_domain or allowed_domain == '*':
                        is_allowed = True
                        break

            # Need to create example URLs to compare the patterns
            # Extract the domain parts, ignoring scheme
            pattern_domain = domain_pattern.split('://')[-1] if '://' in domain_pattern else domain_pattern
            allowed_domain_part = allowed_domain.split('://')[-1] if '://' in allowed_domain else allowed_domain

            # Check if pattern is covered by an allowed domain
            # Example: "google.com" is covered by "*.google.com"
            if pattern_domain == allowed_domain_part or (
                allowed_domain_part.startswith('.') and
                (
                    pattern_domain == allowed_domain_part[2:]
                    or pattern_domain.endswith('.') + allowed_domain_part[2:]
                )
            ):
                is_allowed = True
                break

            if not is_allowed:
                self.logger.warning(
                    f'Domain pattern "{domain_pattern}" in sensitive_data is not covered by any pattern in allow

```

```

#####f' This may be a security risk as credentials could be used on unintended domains.'
#####)

##### Callbacks
#####self.register_new_step_callback = register_new_step_callback
#####self.register_done_callback = register_done_callback
#####self.register_should_stop_callback = register_should_stop_callback
#####self.register_external_agent_status_raise_error_callback = register_external_agent_status_raise_error

##### Telemetry
#####self.telemetry = ProductTelemetry()

##### Event bus with WAL persistence
##### Default to ~/.config/browseruse/events/{agent_session_id}.jsonl
##### wal_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'events' / f'{self.session_id}.jsonl'
#####self.eventbus = EventBus(name=f'Agent_{str(self.id)[-4:]}')

#####if self.settings.save_conversation_path:
#####self.settings.save_conversation_path = Path(self.settings.save_conversation_path).expanduser().resolve()
#####self.logger.info(f'■ Saving conversation to {_log_pretty_path(self.settings.save_conversation_path)}')

##### Initialize download tracking
#####assert self.browser_session is not None, 'BrowserSession is not set up'
#####self.has_downloads_path = self.browser_session.browser_profile.downloads_path is not None
#####if self.has_downloads_path:
#####self._last_known_downloads: list[str] = []
#####self.logger.debug('■ Initialized download tracking for agent')

##### Event-based pause control (kept out of AgentState for serialization)
#####self._external_pause_event = asyncio.Event()
#####self._external_pause_event.set()

#####def _enhance_task_with_schema(self, task: str, output_model_schema: type[AgentStructuredOutput] | None):
#####    """Enhance task description with output schema information if provided."""
#####    if output_model_schema is None:
#####        return task

#####    try:
#####        schema = output_model_schema.model_json_schema()
#####        import json

#####        schema_json = json.dumps(schema, indent=2)

#####        enhancement = f'\nExpected output format: {output_model_schema.__name__}\n{schema_json}'
#####        return task + enhancement
#####    except Exception as e:
#####        self.logger.debug(f'Could not parse output schema: {e}')

#####    return task

#####@property
#####def logger(self) -> logging.Logger:
#####    """Get instance-specific logger with task ID in the name"""
#####    # logger may be called in __init__ so we don't assume self.* attributes have been initialized
#####    _task_id = task_id[-4:] if (task_id := getattr(self, 'task_id', None)) else '----'
#####    _browser_session_id = browser_session.id[-4:] if (browser_session := getattr(self, 'browser_session', None)) else '-'
#####    _current_target_id = (
#####        browser_session.agent_focus_target_id[-2:]
#####        if (browser_session := getattr(self, 'browser_session', None)) and browser_session.agent_focus_target_id
#####        else '-'
#####    )
#####    return logging.getLogger(f'system.Agent_{_task_id}_{_browser_session_id}_{_current_target_id}')

#####@property
#####def browser_profile(self) -> BrowserProfile:
#####    assert self.browser_session is not None, 'BrowserSession is not set up'
#####    return self.browser_session.browser_profile

#####@property
#####def is_using_fallback_llm(self) -> bool:
#####    """Check if the agent is currently using the fallback LLM."""
#####    return self._using_fallback_llm

```

```

    @property
    def current_llm_model(self) -> str:
        """Get the model name of the currently active LLM."""
        return self.llm.model if hasattr(self.llm, 'model') else 'unknown'

    async def _check_and_update_downloads(self, context: str = '') -> None:
        """Check for new downloads and update available file paths."""
        if not self.has_downloads_path:
            return

        assert self.browser_session is not None, 'BrowserSession is not set up'

        try:
            current_downloads = self.browser_session.downloaded_files
            if current_downloads != self._last_known_downloads:
                self._update_available_file_paths(current_downloads)
                self._last_known_downloads = current_downloads
            if context:
                self.logger.debug(f'Updated available files')
        except Exception as e:
            error_context = f'{context}' if context else ''
            self.logger.debug(f'Failed to check for downloads{error_context}: {type(e).__name__}: {e}')

    def _update_available_file_paths(self, downloads: list[str]) -> None:
        """Update available_file_paths with downloaded files."""
        if not self.has_downloads_path:
            return

        current_files = set(self.available_file_paths or [])
        new_files = set(downloads) - current_files

        if new_files:
            self.available_file_paths = list(current_files | new_files)

            self.logger.info(
                f'Added {len(new_files)} downloaded files to available_file_paths (total: {len(self.available_file_paths)})'
            )
            for file_path in new_files:
                self.logger.info(f'New file available: {file_path}')
            else:
                self.logger.debug(f'No new downloads detected (tracking {len(current_files)} files)')

    def _set_file_system(self, file_system_path: str | None = None) -> None:
        """Check for conflicting parameters
        if self.state.file_system_state and file_system_path:
            raise ValueError(
                'Cannot provide both file_system_state (from agent state) and file_system_path. '
                'Either restore from existing state or create new file system at specified path, not both.'
            )

        """
        """Check if we should restore from existing state first
        if self.state.file_system_state:
            try:
                """Restore file system from state at the exact same location
                self.file_system = FileSystem.from_state(self.state.file_system_state)
                """The parent directory of base_dir is the original file_system_path
                self.file_system_path = str(self.file_system.base_dir)
                self.logger.debug(f'File system restored from state to: {self.file_system_path}')
                return
            except Exception as e:
                self.logger.error(f'Failed to restore file system from state: {e}')
                raise e

        """Initialize new file system
        try:
            if file_system_path:
                self.file_system = FileSystem(file_system_path)
                self.file_system_path = file_system_path
            else:
                """Use the agent directory for file system
                self.file_system = FileSystem(self.agent_directory)
                self.file_system_path = str(self.agent_directory)

```

```

    except Exception as e:
        self.logger.error(f'Failed to initialize file system: {e}.')
        raise e

    # Save file system state to agent state
    self.state.file_system_state = self.file_system.get_state()

    self.logger.debug(f'File system path: {self.file_system_path}')

    def _set_screenshot_service(self) -> None:
        """Initialize screenshot service using agent directory"""
        try:
            from system.screenshots.service import ScreenshotService

            self.screenshot_service = ScreenshotService(self.agent_directory)
            self.logger.debug(f'Screenshot service initialized in: {self.agent_directory}/screenshots')
        except Exception as e:
            self.logger.error(f'Failed to initialize screenshot service: {e}.')
            raise e

    def save_file_system_state(self) -> None:
        """Save current file system state to agent state"""
        if self.file_system:
            self.state.file_system_state = self.file_system.get_state()
        else:
            self.logger.error('File system is not set up. Cannot save state.')
            raise ValueError('File system is not set up. Cannot save state.')

    def _set_browser_use_version_and_source(self, source_override: str | None = None) -> None:
        """Get the version from pyproject.toml and determine the source of the browser-use package"""
        # Use the helper function for version detection
        version = get_browser_use_version()

        # Determine source
        try:
            package_root = Path(__file__).parent.parent.parent
            repo_files = ['.git', 'README.md', 'docs', 'examples']
            if all(Path(package_root / file).exists() for file in repo_files):
                source = 'git'
            else:
                source = 'pip'
        except Exception as e:
            self.logger.debug(f'Error determining source: {e}')
            source = 'unknown'

        if source_override is not None:
            source = source_override
            self.logger.debug(f'Version: {version}, Source: {source}') # moved later to _log_agent_run so that
            self.version = version
            self.source = source

    def _setup_action_models(self) -> None:
        """Setup dynamic action models from tools registry"""
        # Initially only include actions with no filters
        self.ActionModel = self.tools.registry.create_action_model()
        # Create output model with the dynamic actions
        if self.settings.flash_mode:
            self.AgentOutput = AgentOutput.type_with_custom_actions_flash_mode(self.ActionModel)
        elif self.settings.use_thinking:
            self.AgentOutput = AgentOutput.type_with_custom_actions(self.ActionModel)
        else:
            self.AgentOutput = AgentOutput.type_with_custom_actions_no_thinking(self.ActionModel)

        # used to force the done action when max_steps is reached
        self.DoneActionModel = self.tools.registry.create_action_model(include_actions=['done'])
        if self.settings.flash_mode:
            self.DoneAgentOutput = AgentOutput.type_with_custom_actions_flash_mode(self.DoneActionModel)
        elif self.settings.use_thinking:
            self.DoneAgentOutput = AgentOutput.type_with_custom_actions(self.DoneActionModel)
        else:
            self.DoneAgentOutput = AgentOutput.type_with_custom_actions_no_thinking(self.DoneActionModel)
    def _get_skill_slug(self, skill: 'Skill', all_skills: list['Skill']) -> str:

```

```

    """Generate a clean slug from skill title for action names

    Converts title to lowercase, removes special characters, replaces spaces with underscores.
    Adds UUID suffix if there are duplicate slugs.

    Args:
        skill: The skill to get slug for
        all_skills: List of all skills to check for duplicates

    Returns:
        Slug like "cloned_github_stars_tracker" or "get_weather_data_a1b2" if duplicate

    Examples:
        "[Cloned] Github Stars Tracker" -> "cloned_github_stars_tracker"
        "Get Weather Data" -> "get_weather_data"
    """
    import re

    # Remove special characters and convert to lowercase
    slug = re.sub(r'^\w\s', '', skill.title.lower())
    # Replace whitespace and hyphens with underscores
    slug = re.sub(r'\s\-', '_', slug)
    # Remove leading/trailing underscores
    slug = slug.strip('_')

    # Check for duplicates and add UUID suffix if needed
    same_slug_count = sum(
        1 for s in all_skills if re.sub(r'\s\-', '_', re.sub(r'^\w\s', '', s.title.lower()).strip('_'))
    )
    if same_slug_count > 1:
        return f'{slug}_{skill.id[:4]}'
    else:
        return slug

    async def _register_skills_as_actions(self) -> None:
        """Register each skill as a separate action using slug as action name"""
        if not self.skill_service or self._skills_registered:
            return

        self.logger.info('■ Registering skill actions...')

        # Fetch all skills (auto-initializes if needed)
        skills = await self.skill_service.get_all_skills()

        if not skills:
            self.logger.warning('No skills loaded from SkillService')
            return

        # Register each skill as its own action
        for skill in skills:
            slug = self._get_skill_slug(skill, skills)
            param_model = skill.parameters_pydantic(exclude_cookies=True)

            # Create description with skill title in quotes
            description = f'{skill.description} (Skill: "{skill.title}")'

            # Create handler for this specific skill
            def make_skill_handler(skill_id: str):
                async def skill_handler(params: BaseModel) -> ActionResult:
                    """Execute a specific skill"""
                    assert self.skill_service is not None, 'SkillService not initialized'

                    # Convert parameters to dict
                    if isinstance(params, BaseModel):
                        skill_params = params.model_dump()
                    elif isinstance(params, dict):
                        skill_params = params
                    else:
                        return ActionResult(extracted_content=None, error=f'Invalid parameters type: {type(params)}')

                # Get cookies from browser
                cookies = await self.browser_session.cookies()

```

```

        try:
            result = await self.skill_service.execute_skill(
                skill_id=skill_id, parameters=skill_params, cookies=_cookies
            )

            if result.success:
                return ActionResult(
                    extracted_content=str(result.result) if result.result else None,
                    error=None,
                )
            else:
                return ActionResult(extracted_content=None, error=result.error or 'Skill execution failed')
            except Exception as e:
                # Check if it's a MissingCookieException
                if type(e).__name__ == 'MissingCookieException':
                    # Format: "Missing cookies (name): description"
                    cookie_name = getattr(e, 'cookie_name', 'unknown')
                    cookie_description = getattr(e, 'cookie_description', str(e))
                    error_msg = f'Missing cookies ({cookie_name}): {cookie_description}'
                    return ActionResult(extracted_content=None, error=error_msg)
                return ActionResult(extracted_content=None, error=f'Skill execution error: {type(e).__name__}: {e}')

    return skill_handler

    # Create the handler for this skill
    handler = make_skill_handler(skill.id)
    handler.__name__ = slug

    # Register the action with the slug as the action name
    self.tools.registry.action(description=description, param_model=param_model)(handler)

    # Mark as registered
    self._skills_registered = True

    # Rebuild action models to include the new skill actions
    self._setup_action_models()

    # Reconvert initial actions with the new ActionModel type if they exist
    if self.initial_actions:
        # Convert back to dict form first
        initial_actions_dict = []
        for action in self.initial_actions:
            action_dump = action.model_dump(exclude_unset=True)
            initial_actions_dict.append(action_dump)
        # Reconvert using new ActionModel
        self.initial_actions = self._convert_initial_actions(initial_actions_dict)

    self.logger.info(f'✓ Registered {len(self.skills)} skill actions')

    async def _get_unavailable_skills_info(self) -> str:
        """Get information about skills that are unavailable due to missing cookies

        Returns:
            Formatted string describing unavailable skills and how to make them available
        """
        if not self.skill_service:
            return ''

        try:
            # Get all skills
            skills = await self.skill_service.get_all_skills()
            if not skills:
                return ''

            # Get current cookies
            current_cookies = await self.browser_session.cookies()
            cookie_dict = {cookie['name']: cookie['value'] for cookie in current_cookies}

            # Check each skill for missing required cookies
            unavailable_skills: list[dict[str, Any]] = []

            for skill in skills:

```

```

##### Get cookie parameters for this skill
##### cookie_params = [p for p in skill.parameters if p.type == 'cookie']

##### if not cookie_params:
##### # No cookies needed, skip
##### continue

##### Check for missing required cookies
##### missing_cookies: list[dict[str, str]] = []
##### for cookie_param in cookie_params:
##### is_required = cookie_param.required if cookie_param.required is not None else True

##### if is_required and cookie_param.name not in cookie_dict:
##### missing_cookies.append(
##### { 'name': cookie_param.name, 'description': cookie_param.description or 'No description provided'
##### })

##### if missing_cookies:
##### unavailable_skills.append(
##### {
##### 'id': skill.id,
##### 'title': skill.title,
##### 'description': skill.description,
##### 'missing_cookies': missing_cookies,
##### })

##### if not unavailable_skills:
##### return ''

##### # Format the unavailable skills info with slugs
##### lines = ['Unavailable Skills (missing required cookies):']
##### for skill_info in unavailable_skills:
##### # Get the full skill object to use the slug helper
##### skill_obj = next((s for s in skills if s.id == skill_info['id']), None)
##### slug = self._get_skill_slug(skill_obj, skills) if skill_obj else skill_info['title']
##### title = skill_info['title']

##### lines.append(f'\n • {slug} ("{title}")')
##### lines.append(f'    Description: {skill_info["description"]}')
##### lines.append(f'    Missing cookies:')
##### for cookie in skill_info['missing_cookies']:
##### lines.append(f'        - {cookie["name"]}: {cookie["description"]}')

##### return '\n'.join(lines)

##### except Exception as e:
##### self.logger.error(f'Error getting unavailable skills info: {type(e).__name__}: {e}')
##### return ''

##### def add_new_task(self, new_task: str) -> None:
##### """Add a new task to the agent, keeping the same task_id as tasks are continuous"""
##### # Simply delegate to message manager - no need for new task_id or events
##### # The task continues with new instructions, it doesn't end and start a new one
##### self.task = new_task
##### self._message_manager.add_new_task(new_task)
##### # Mark as follow-up task and recreate eventbus (gets shut down after each run)
##### self.state.follow_up_task = True
##### # Reset control flags so agent can continue
##### self.state.stopped = False
##### self.state.paused = False
##### agent_id_suffix = str(self.id)[-4:].replace('-', '_')
##### if agent_id_suffix and agent_id_suffix[0].isdigit():
##### agent_id_suffix = 'a' + agent_id_suffix
##### self.eventbus = EventBus(name=f'Agent_{agent_id_suffix}')

##### async def _check_stop_or_pause(self) -> None:
##### """Check if the agent should stop or pause, and handle accordingly."""

##### # Check new should_stop_callback - sets stopped state cleanly without raising
##### if self.register_should_stop_callback:
##### if await self.register_should_stop_callback():

```

```

        self.logger.info('External callback requested stop')
        self.state.stopped = True
        raise InterruptedError

    if self.register_external_agent_status_raise_error_callback:
        if await self.register_external_agent_status_raise_error_callback():
            raise InterruptedError

    if self.state.stopped:
        raise InterruptedError

    if self.state.paused:
        raise InterruptedError

    @observe(name='agent.step', ignore_output=True, ignore_input=True)
    @time_execution_async('--step')
    async def step(self, step_info: AgentStepInfo | None = None) -> None:
        """Execute one step of the task"""
        # Initialize timing first, before any exceptions can occur

        self.step_start_time = time.time()

        browser_state_summary = None

        try:
            if self.browser_session:
                try:
                    captcha_wait = await self.browser_session.wait_if_captcha_solving()
                    if captcha_wait and captcha_wait.waited:
                        # Reset step timing to exclude the captcha wait from step duration metrics
                        self.step_start_time = time.time()
                        duration_s = captcha_wait.duration_ms / 1000
                        outcome = captcha_wait.result # 'success' | 'failed' | 'timeout'
                        msg = f'Waited {duration_s:.1f}s for {captcha_wait.vendor} CAPTCHA to be solved. Result: {outcome}'
                        self.logger.info(f'█ {msg}')
                        # Inject the outcome so the LLM sees what happened
                        captcha_result = ActionResult(long_term_memory=msg)
                        if self.state.last_result:
                            self.state.last_result.append(captcha_result)
                        else:
                            self.state.last_result = [captcha_result]
                except Exception as e:
                    self.logger.warning(f'Phase 0 captcha wait failed (non-fatal): {e}')

            # Phase 1: Prepare context and timing
            browser_state_summary = await self._prepare_context(step_info)

            # Phase 2: Get model output and execute actions
            await self._get_next_action(browser_state_summary)
            await self._execute_actions()

            # Phase 3: Post-processing
            await self._post_process()

        except Exception as e:
            # Handle ALL exceptions in one place
            await self._handle_step_error(e)

        finally:
            await self._finalize(browser_state_summary)

    async def _prepare_context(self, step_info: AgentStepInfo | None = None) -> BrowserStateSummary:
        """Prepare the context for the step: browser state, action models, page actions"""
        # step_start_time is now set in step() method

        assert self.browser_session is not None, 'BrowserSession is not set up'

        self.logger.debug(f'█ Step {self.state.n_steps}: Getting browser state...')
        # Always take screenshots for all steps
        self.logger.debug('█ Requesting browser state with include_screenshot=True')
        browser_state_summary = await self.browser_session.get_browser_state_summary(
            include_screenshot=True, # always capture even if use_vision=False so that cloud sync is useful (it

```



```

    include_recent_events=self.include_recent_events,
    )
    if browser_state_summary.screenshot:
        self.logger.debug(f'Got browser state WITH screenshot, length: {len(browser_state_summary.screenshot)}')
    else:
        self.logger.debug(f'Got browser state WITHOUT screenshot')

    # Check for new downloads after getting browser state (catches PDF auto-downloads and previous step downloads)
    await self._check_and_update_downloads(f'Step {self.state.n_steps}: after getting browser state')

    self._log_step_context(browser_state_summary)
    await self._check_stop_or_pause()

    # Update action models with page-specific actions
    self.logger.debug(f'Step {self.state.n_steps}: Updating action models...')
    await self._update_action_models_for_page(browser_state_summary.url)

    # Get page-specific filtered actions
    page_filtered_actions = self.tools.registry.get_prompt_description(browser_state_summary.url)

    # Page-specific actions will be included directly in the browser_state message
    self.logger.debug(f'Step {self.state.n_steps}: Creating state messages for context...')

    # Get unavailable skills info if skills service is enabled
    unavailable_skills_info = None
    if self.skill_service is not None:
        unavailable_skills_info = await self._get_unavailable_skills_info()

    # Render plan description for injection into agent context
    plan_description = self._render_plan_description()

    self._message_manager.prepare_step_state(
        browser_state_summary=browser_state_summary,
        model_output=self.state.last_model_output,
        result=self.state.last_result,
        step_info=step_info,
        sensitive_data=self.sensitive_data,
    )

    await self._maybe_compact_messages(step_info)

    self._message_manager.create_state_messages(
        browser_state_summary=browser_state_summary,
        model_output=self.state.last_model_output,
        result=self.state.last_result,
        step_info=step_info,
        use_vision=self.settings.use_vision,
        page_filtered_actions=page_filtered_actions if page_filtered_actions else None,
        sensitive_data=self.sensitive_data,
        available_file_paths=self.available_file_paths, # Always pass current available_file_paths
        unavailable_skills_info=unavailable_skills_info,
        plan_description=plan_description,
        skip_state_update=True,
    )

    await self._inject_budget_warning(step_info)
    self._inject_replan_nudge()
    self._inject_exploration_nudge()
    self._update_loop_detector_page_state(browser_state_summary)
    self._inject_loop_detection_nudge()
    await self._force_done_after_last_step(step_info)
    await self._force_done_after_failure()
    return browser_state_summary

    async def _maybe_compact_messages(self, step_info: AgentStepInfo | None = None) -> None:
        """Optionally compact message history to keep prompts small."""
        settings = self.settings.message_compaction
        if not settings or not settings.enabled:
            return

        compaction_llm = settings.compaction_llm or self.settings.page_extraction_llm or self.llm
        await self._message_manager.maybe_compact_messages(

```

```

    self.llm=compaction_llm,
    self.settings=settings,
    self.step_info=step_info,
)

@observe_debug(ignore_input=True, name='get_next_action')
async def _get_next_action(self, browser_state_summary: BrowserStateSummary) -> None:
    """Execute LLM interaction with retry logic and handle callbacks"""
    input_messages = self._message_manager.get_messages()

    # =====
    # PHASE 4.X: DETERMINISTIC WORKFLOW MACRO EXECUTION
    # If we have an active macro, bypass the LLM entirely and execute locally
    if self.workflow_executor and self.workflow_executor.mode == "macro":
        try:
            self.logger.debug(f"Step {self.state.n_steps}: Executing macro deterministically via WorkflowExecutor")
            macro_output = self.workflow_executor.get_next_action(
                browser_state_summary.dom_state,
                self.AgentOutput
            )
            if macro_output is not None:
                self.state.last_model_output = macro_output
                await self._handle_post_llm_processing(browser_state_summary, input_messages)
            return
        except Exception as e:
            self.logger.warning(f"[WorkflowExecutor] Engine crashed: {e}. Falling back to default LLM logic")
    # =====

    self.logger.debug(
        f"Step {self.state.n_steps}: Calling LLM with {len(input_messages)} messages (model: {self.llm.model})"
    )

    try:
        model_output = await asyncio.wait_for(
            self._get_model_output_with_retry(input_messages), timeout=self.settings.llm_timeout
        )
    except TimeoutError:

        @observe(name='_llm_call_timed_out_with_input')
        async def _log_model_input_to_lmnr(input_messages: list[BaseMessage]) -> None:
            """Log the model input"""
            pass

        await _log_model_input_to_lmnr(input_messages)

        raise TimeoutError(
            f"LLM call timed out after {self.settings.llm_timeout} seconds. Keep your thinking and output short"
        )

    self.state.last_model_output = model_output

    # Check again for paused/stopped state after getting model output
    await self._check_stop_or_pause()

    # Handle callbacks and conversation saving
    await self._handle_post_llm_processing(browser_state_summary, input_messages)

    # check again if Ctrl+C was pressed before we commit the output to history
    await self._check_stop_or_pause()

    async def _execute_actions(self) -> None:
        """Execute the actions from model output"""
        if self.state.last_model_output is None:
            raise ValueError('No model output to execute actions from')

    result = await self.multi_act(self.state.last_model_output.action)
    self.state.last_result = result

    async def _post_process(self) -> None:
        """Handle post-action processing like download tracking and result logging"""
        assert self.browser_session is not None, 'BrowserSession is not set up'
        # Check for new downloads after executing actions

```

```

await self._check_and_update_downloads('after executing actions')

## Update plan state from model output
if self.state.last_model_output is not None:
    self._update_plan_from_model_output(self.state.last_model_output)

## Record executed actions for loop detection
self._update_loop_detector_actions()

## check for action errors - only count single-action steps toward consecutive failures;
## multi-action steps with errors are handled by loop detection and replan nudges instead
if self.state.last_result and len(self.state.last_result) == 1 and self.state.last_result[-1].error:
    self.state.consecutive_failures += 1
    self.logger.debug(f'■ Step {self.state.n_steps}: Consecutive failures: {self.state.consecutive_failures}')
    return

if self.state.consecutive_failures > 0:
    self.state.consecutive_failures = 0
    self.logger.debug(f'■ Step {self.state.n_steps}: Consecutive failures reset to: {self.state.consecutive_failures}')

## Log completion results
if self.state.last_result and len(self.state.last_result) > 0 and self.state.last_result[-1].is_done:
    success = self.state.last_result[-1].success
    if success:
        ## Green color for success
        self.logger.info(f'\n■ \033[32m Final Result:\033[0m \n{self.state.last_result[-1].extracted_content}')
    else:
        ## Red color for failure
        self.logger.info(f'\n■ \033[31m Final Result:\033[0m \n{self.state.last_result[-1].extracted_content}')
    if self.state.last_result[-1].attachments:
        total_attachments = len(self.state.last_result[-1].attachments)
        for i, file_path in enumerate(self.state.last_result[-1].attachments):
            self.logger.info(f'■ Attachment {i + 1 if total_attachments > 1 else ""}: {file_path}')

async def _handle_step_error(self, error: Exception) -> None:
    """Handle all types of errors that can occur during a step"""

    ## Handle InterruptedError specially
    if isinstance(error, InterruptedError):
        error_msg = 'The agent was interrupted mid-step' + (f' - {str(error)}' if str(error) else '')
        ## NOTE: This is not an error, it's a normal part of the execution when the user interrupts the agent
        self.logger.warning(f'{error_msg}')
        return

    ## Handle browser closed/disconnected errors
    if self._is_connection_like_error(error):
        ## If reconnection is in progress, wait for it instead of stopping
        if self.browser_session.is_reconnecting:
            wait_timeout = self.browser_session.RECONNECT_WAIT_TIMEOUT
            self.logger.warning(
                f'■ Connection error during reconnection, waiting up to {wait_timeout}s for reconnect: {error}'
            )
            try:
                await asyncio.wait_for(self.browser_session._reconnect_event.wait(), timeout=wait_timeout)
            except TimeoutError:
                pass

        ## Check if reconnection succeeded
        if self.browser_session.is_cdp_connected:
            self.logger.info('■ Reconnection succeeded, retrying step...')
            self.state.last_result = [ActionResult(error=f'Connection lost and recovered: {error}')]
            return

        ## Not reconnecting or reconnection failed – check if truly terminal
        if self._is_browser_closed_error(error):
            self.logger.warning(f'■ Browser closed or disconnected: {error}')
            self.state.stopped = True
            self._external_pause_event.set()
            return

    ## Handle all other exceptions
    include_trace = self.logger.isEnabledFor(logging.DEBUG)

```

```

    error_msg = AgentError.format_error(error, include_trace=include_trace)
    max_total_failures = self.settings.max_failures + int(self.settings.final_response_after_failure)
    prefix = f'■ Result failed {self.state.consecutive_failures + 1}/{max_total_failures} times: '
    self.state.consecutive_failures += 1

    ## Use WARNING for partial failures, ERROR only when max failures reached
    is_final_failure = self.state.consecutive_failures >= max_total_failures
    log_level = logging.ERROR if is_final_failure else logging.WARNING

    if 'Could not parse response' in error_msg or 'tool_use_failed' in error_msg:
        ## give model a hint how output should look like
        self.logger.log(log_level, f'Model: {self.llm.model} failed')
        self.logger.log(log_level, f'{prefix}{error_msg}')
    else:
        self.logger.log(log_level, f'{prefix}{error_msg}')

    await self._demo_mode_log(f'Step error: {error_msg}', 'error', {'step': self.state.n_steps})
    self.state.last_result = [ActionResult(error=error_msg)]
    return None

def _is_connection_like_error(self, error: Exception) -> bool:
    """Check if the error looks like a CDP/WebSocket connection failure.

    Unlike _is_browser_closed_error(), this does NOT check if the CDP client is None
    or if reconnection is in progress – it purely looks at the error signature.
    """
    error_str = str(error).lower()
    return (
        isinstance(error, ConnectionError)
        or 'websocket connection closed' in error_str
        or 'connection closed' in error_str
        or 'browser has been closed' in error_str
        or 'browser closed' in error_str
        or 'no browser' in error_str
    )

def _is_browser_closed_error(self, error: Exception) -> bool:
    """Check if the browser has been closed or disconnected.

    Only returns True when the error itself is a CDP/WebSocket connection failure
    AND the CDP client is gone AND we're not actively reconnecting.
    Avoids false positives on unrelated errors (element not found, timeouts,
    parse errors) that happen to coincide with a transient None state during
    reconnects or resets.
    """
    ## During reconnection, don't treat connection errors as terminal
    if self.browser_session.is_reconnecting:
        return False

    error_str = str(error).lower()
    is_connection_error = (
        isinstance(error, ConnectionError)
        or 'websocket connection closed' in error_str
        or 'connection closed' in error_str
        or 'browser has been closed' in error_str
        or 'browser closed' in error_str
        or 'no browser' in error_str
    )
    return is_connection_error and self.browser_session.cdp_client_root is None

async def _finalize(self, browser_state_summary: BrowserStateSummary | None) -> None:
    """Finalize the step with history, logging, and events"""
    step_end_time = time.time()
    if not self.state.last_result:
        return

    if browser_state_summary:
        step_interval = None
        if len(self.history.history) > 0:
            last_history_item = self.history.history[-1]

            if last_history_item.metadata:

```

```

#####previous_end_time = last_history_item.metadata.step_end_time
#####previous_start_time = last_history_item.metadata.step_start_time
#####step_interval = max(0, previous_end_time - previous_start_time)
#####metadata = StepMetadata(
#####step_number=self.state.n_steps,
#####step_start_time=self.step_start_time,
#####step_end_time=step_end_time,
#####step_interval=step_interval,
#####)

###### Use _make_history_item like main branch
#####await self._make_history_item(
#####self.state.last_model_output,
#####browser_state_summary,
#####self.state.last_result,
#####metadata,
#####state_message=self._message_manager.last_state_message_text,
#####)

###### Log step completion summary
#####summary_message = self._log_step_completion_summary(self.step_start_time, self.state.last_result)
#####if summary_message:
#####await self._demo_mode_log(summary_message, 'info', {'step': self.state.n_steps})

###### Save file system state after step completion
#####self.save_file_system_state()

###### Emit both step created and executed events
#####if browser_state_summary and self.state.last_model_output:
###### Extract key step data for the event
#####actions_data = []
#####if self.state.last_model_output.action:
#####for action in self.state.last_model_output.action:
#####    action_dict = action.model_dump() if hasattr(action, 'model_dump') else {}
#####    actions_data.append(action_dict)

###### Emit CreateAgentStepEvent
#####step_event = CreateAgentStepEvent.from_agent_step(
#####self,
#####self.state.last_model_output,
#####self.state.last_result,
#####actions_data,
#####browser_state_summary,
#####)
#####self.eventbus.dispatch(step_event)

###### MOCK FRONTEND EVENT BUS: Emitting "Billing & Usage"
#####try:
#####current_usage = await self.token_cost_service.get_usage_summary()
#####self._emit_mock_frontend_event({
#####    "event_type": "telemetry_billing",
#####    "model": self.llm.model if hasattr(self.llm, 'model') else "unknown",
#####    "tokens": {
#####        "prompt": current_usage.total_prompt_tokens,
#####        "completion": current_usage.total_completion_tokens,
#####        "cached": current_usage.total_prompt_cached_tokens
#####    },
#####    "step": self.state.n_steps,
#####    "status": "success"
#####})
#####except Exception as e:
#####self.logger.debug(f'Failed to emit billing event: {e}')

###### Increment step counter after step is fully completed
#####self.state.n_steps += 1
#####
#####def _emit_mock_frontend_event(self, data: dict) -> None:
#####    """Hack to mock frontend socket emission for Phase 4.
#####    This isolates the output explicitly so the user can see exactly what the UI state expects.
#####    """
#####    stream_file = Path("C:/Users/maham/tmp/mock_frontend_stream.jsonl")
#####    stream_file.parent.mkdir(parents=True, exist_ok=True)

```

```

    try:
        with open(stream_file, "a") as f:
            f.write(json.dumps(data) + "\n")
    except Exception as e:
        self.logger.warning(f"Mock SSE Stream write failed: {e}")

    def _update_plan_from_model_output(self, model_output: AgentOutput) -> None:
        """Update the plan state from model output fields (current_plan_item, plan_update)."""
        if not self.settings.enable_planning:
            return

        # If model provided a new plan via plan_update, replace the current plan
        if model_output.plan_update is not None:
            self.state.plan = [PlanItem(text=step_text) for step_text in model_output.plan_update]
            self.state.current_plan_item_index = 0
            self.state.plan_generation_step = self.state.n_steps
            if self.state.plan:
                self.state.plan[0].status = 'current'
                self.logger.info(
                    f"Plan {'updated' if self.state.plan_generation_step else 'created'} with {len(self.state.plan)}"
                )
            return

        # If model provided a step index update, advance the plan
        if model_output.current_plan_item is not None and self.state.plan is not None:
            new_idx = model_output.current_plan_item
            # Clamp to valid range
            new_idx = max(0, min(new_idx, len(self.state.plan) - 1))
            old_idx = self.state.current_plan_item_index

            # Mark steps between old and new as done
            for i in range(old_idx, new_idx):
                if i < len(self.state.plan) and self.state.plan[i].status in ('current', 'pending'):
                    self.state.plan[i].status = 'done'

            # Mark the new step as current
            if new_idx < len(self.state.plan):
                self.state.plan[new_idx].status = 'current'

            self.state.current_plan_item_index = new_idx

    def _render_plan_description(self) -> str | None:
        """Render the current plan as a text description for injection into agent context."""
        if not self.settings.enable_planning or self.state.plan is None:
            return None

        markers = {'done': '[x]', 'current': '[>]', 'pending': '[ ]', 'skipped': '[-]'}
        lines = []
        for i, step in enumerate(self.state.plan):
            marker = markers.get(step.status, '[ ]')
            lines.append(f'{marker} {i}: {step.text}')
        return '\n'.join(lines)

    def _inject_replan_nudge(self) -> None:
        """Inject a replan nudge when stall detection threshold is met."""
        if not self.settings.enable_planning or self.state.plan is None:
            return
        if self.settings.planning_replan_on_stall <= 0:
            return
        if self.state.consecutive_failures >= self.settings.planning_replan_on_stall:
            msg = (
                'REPLAN SUGGESTED: You have failed '
                f'{self.state.consecutive_failures} consecutive times. '
                'Your current plan may need revision. '
                'Output a new `plan_update` with revised steps to recover.'
            )
            self.logger.info(f"Replan nudge injected after {self.state.consecutive_failures} consecutive failures")
            self._message_manager._add_context_message(UserMessage(content=msg))

    def _inject_exploration_nudge(self) -> None:
        """Nudge the agent to create a plan (or call done) after exploring without one."""
        if not self.settings.enable_planning or self.state.plan is not None:

```

```

    return
    if self.settings.planning_exploration_limit <= 0:
        return
    if self.state.n_steps >= self.settings.planning_exploration_limit:
        msg = (
            'PLANNING NUDGE: You have taken '
            f'{self.state.n_steps} steps without creating a plan. '
            'If the task is complex, output a `plan_update` with clear todo items now. '
            'If the task is already done or nearly done, call `done` instead.'
        )
        self.logger.info(f'Exploration nudge injected after {self.state.n_steps} steps without a plan')
        self._message_manager._add_context_message(UserMessage(content=msg))

    def _inject_loop_detection_nudge(self) -> None:
        """Inject an escalating nudge when behavioral loops are detected."""
        if not self.settings.loop_detection_enabled:
            return
        nudge = self.state.loop_detector.get_nudge_message()
        if nudge:
            self.logger.info(
                f'Loop detection nudge injected (repetition={self.state.loop_detector.max_repetition_count}, '
                f'stagnation={self.state.loop_detector.consecutive_stagnant_pages})'
            )
            self._message_manager._add_context_message(UserMessage(content=nudge))

    def _update_loop_detector_actions(self) -> None:
        """Record the actions from the latest step into the loop detector."""
        if not self.settings.loop_detection_enabled:
            return
        if self.state.last_model_output is None:
            return
        # Actions to exclude: wait always hashes identically (instant false positive),
        # done is terminal, go_back is navigation recovery
        _LOOP_EXEMPT_ACTIONS = {'wait', 'done', 'go_back'}
        for action in self.state.last_model_output.action:
            action_data = action.model_dump(exclude_unset=True)
            action_name = next(iter(action_data.keys()), 'unknown')
            if action_name in _LOOP_EXEMPT_ACTIONS:
                continue
            params = action_data.get(action_name, {})
            if not isinstance(params, dict):
                params = {}
            self.state.loop_detector.record_action(action_name, params)

    def _update_loop_detector_page_state(self, browser_state_summary: BrowserStateSummary) -> None:
        """Record the current page state for stagnation detection."""
        if not self.settings.loop_detection_enabled:
            return
        url = browser_state_summary.url or ''
        element_count = len(browser_state_summary.dom_state.selector_map) if browser_state_summary.dom_state else 0
        # Use the DOM text representation for fingerprinting
        dom_text = ''
        if browser_state_summary.dom_state:
            try:
                dom_text = browser_state_summary.dom_state.llm_representation()
            except Exception:
                dom_text = ''
        self.state.loop_detector.record_page_state(url, dom_text, element_count)

    async def _inject_budget_warning(self, step_info: AgentStepInfo | None = None) -> None:
        """Inject a prominent budget warning when the agent has used >= 75% of its step budget.

        This gives the LLM advance notice to wrap up, save partial results, and call done
        rather than exhausting all steps with nothing saved.
        """
        if step_info is None:
            return

        steps_used = step_info.step_number + 1 # Convert 0-indexed to 1-indexed
        budget_ratio = steps_used / step_info.max_steps

        if budget_ratio >= 0.75 and not step_info.is_last_step():

```

```

    steps_remaining = step_info.max_steps - steps_used
    pct = int(budget_ratio * 100)
    msg = (
        f'BUDGET WARNING: You have used {steps_used}/{step_info.max_steps} steps '
        f'({pct}%). {steps_remaining} steps remaining. '
        f'If the task cannot be completed in the remaining steps, prioritize: '
        f'(1) consolidate your results (save to files if the file system is in use), '
        f'(2) call done with what you have. '
        f'Partial results are far more valuable than exhausting all steps with nothing saved.'
    )
    self.logger.info(f'Step budget warning: {steps_used}/{step_info.max_steps} ({pct}%)')
    self._message_manager._add_context_message(UserMessage(content=msg))

    async def _force_done_after_last_step(self, step_info: AgentStepInfo | None = None) -> None:
        """Handle special processing for the last step"""
        if step_info and step_info.is_last_step():
            # Add last step warning if needed
            msg = 'You reached max_steps - this is your last step. Your only tool available is the "done" tool.'
            msg += '\nIf the task is not yet fully finished as requested by the user, set success in "done" to f'
            msg += '\nInclude everything you found out for the ultimate task in the done text.'
            self.logger.debug('Last step finishing up')
            self._message_manager._add_context_message(UserMessage(content=msg))
            self.AgentOutput = self.DoneAgentOutput

    async def _force_done_after_failure(self) -> None:
        """Force done after failure"""
        # Create recovery message
        if self.state.consecutive_failures >= self.settings.max_failures and self.settings.final_response_after_max_failures:
            msg = f'You failed {self.settings.max_failures} times. Therefore we terminate the agent.'
            msg += '\nYour only tool available is the "done" tool. No other tool is available. All other tools w'
            msg += '\nIf the task is not yet fully finished as requested by the user, set success in "done" to f'
            msg += '\nInclude everything you found out for the ultimate task in the done text.'

            self.logger.debug('Force done action, because we reached max_failures.')
            self._message_manager._add_context_message(UserMessage(content=msg))
            self.AgentOutput = self.DoneAgentOutput

    @observe(ignore_input=True, ignore_output=False)
    async def _judge_trace(self) -> JudgementResult | None:
        """Judge the trace of the agent"""
        task = self.task
        final_result = self.history.final_result() or ''
        agent_steps = self.history.agent_steps()
        screenshot_paths = [p for p in self.history.screenshot_paths() if p is not None]

        # Construct input messages for judge evaluation
        input_messages = construct_judge_messages(
            task=task,
            final_result=final_result,
            agent_steps=agent_steps,
            screenshot_paths=screenshot_paths,
            max_images=10,
            ground_truth=self.settings.ground_truth,
            use_vision=self.settings.use_vision,
        )

        # Call LLM with JudgementResult as output format
        kwargs: dict = {'output_format': JudgementResult}

        # Only pass request_type for ChatBrowserUse (other providers don't support it)
        if self.judge_llm.provider == 'browser-use':
            kwargs['request_type'] = 'judge'

        try:
            response = await self.judge_llm.ainvoke(input_messages, **kwargs)
            judgement: JudgementResult = response.completion # type: ignore[assignment]
            return judgement
        except Exception as e:
            self.logger.error(f'Judge trace failed: {e}')
            # Return a default judgement on failure
            return None
    async def _judge_and_log(self) -> None:

```



```

    """Run judge evaluation and log the verdict.

    The judge verdict is attached to the action result but does NOT override
    last_result.success – that stays as the agent's self-report. Telemetry
    sends both values so the eval platform can compare agent vs judge.
    """
    judgement = await self._judge_trace()

    ## Attach judgement to last action result
    if self.history.history[-1].result[-1].is_done:
        last_result = self.history.history[-1].result[-1]
        last_result.judgement = judgement

    ## Get self-reported success
    self_reported_success = last_result.success

    ## Log the verdict based on self-reported success and judge verdict
    if judgement:
        ## If both self-reported and judge agree on success, don't log
        if self_reported_success is True and judgement.verdict is True:
            return

        judgement_log = '\n'
        ## If agent reported success but judge thinks it failed, show warning
        if self_reported_success is True and judgement.verdict is False:
            judgement_log += '  \033[33mAgent reported success but judge thinks task failed\033[0m\n'

        ## Otherwise, show full judge result
        verdict_color = '\033[32m' if judgement.verdict else '\033[31m'
        verdict_text = '  PASS' if judgement.verdict else '  FAIL'
        judgement_log += f'  {verdict_color}Judge Verdict: {verdict_text}\033[0m\n'
        if judgement.failure_reason:
            judgement_log += f'    Failure Reason: {judgement.failure_reason}\n'
        if judgement.reached_captcha:
            judgement_log += '    Captcha Detected: Agent encountered captcha challenges\n'
            judgement_log += '    Use Browser Use Cloud for the most stealth browser infra: https://docs.brows
            judgement_log += f'    {judgement.reasoning}\n'
        self.logger.info(judgement_log)

    async def _get_model_output_with_retry(self, input_messages: list[BaseMessage]) -> AgentOutput:
        """Get model output with retry logic for empty actions"""
        model_output = await self.get_model_output(input_messages)
        self.logger.debug(
            f'  Step {self.state.n_steps}: Got LLM response with {len(model_output.action)} if model_output.action
        )

        if (
            not model_output.action
            or not isinstance(model_output.action, list)
            or all(action.model_dump() == {} for action in model_output.action)
        ):
            self.logger.warning('Model returned empty action. Retrying...')

            clarification_message = UserMessage(
                content='You forgot to return an action. Please respond with a valid JSON action according to the e
            )

            retry_messages = input_messages + [clarification_message]
            model_output = await self.get_model_output(retry_messages)

            if not model_output.action or all(action.model_dump() == {} for action in model_output.action):
                self.logger.warning('Model still returned empty after retry. Inserting safe noop action.')
                action_instance = self.ActionModel()
                setattr(
                    action_instance,
                    'done',
                    {
                        'success': False,
                        'text': 'No next action returned by LLM!',
                    },
                )
                model_output.action = [action_instance]

```

```

    return model_output

    async def _handle_post_llm_processing(
        self,
        browser_state_summary: BrowserStateSummary,
        input_messages: list[BaseMessage],
    ) -> None:
        """Handle callbacks and conversation saving after LLM interaction"""
        if self.register_new_step_callback and self.state.last_model_output:
            if inspect.iscoroutinefunction(self.register_new_step_callback):
                await self.register_new_step_callback(
                    browser_state_summary,
                    self.state.last_model_output,
                    self.state.n_steps,
                )
            else:
                self.register_new_step_callback(
                    browser_state_summary,
                    self.state.last_model_output,
                    self.state.n_steps,
                )

        if self.settings.save_conversation_path and self.state.last_model_output:
            # Treat save_conversation_path as a directory (consistent with other recording paths)
            conversation_dir = Path(self.settings.save_conversation_path)
            conversation_filename = f'conversation_{self.id}_{self.state.n_steps}.txt'
            target = conversation_dir / conversation_filename
            await save_conversation(
                input_messages,
                self.state.last_model_output,
                target,
                self.settings.save_conversation_path_encoding,
            )

    async def _make_history_item(
        self,
        model_output: AgentOutput | None,
        browser_state_summary: BrowserStateSummary,
        result: list[ActionResult],
        metadata: StepMetadata | None = None,
        state_message: str | None = None,
    ) -> None:
        """Create and store history item"""

        if model_output:
            interacted_elements = AgentHistory.get_interacted_element(model_output, browser_state_summary.dom_st)
        else:
            interacted_elements = [None]

        # Store screenshot and get path
        screenshot_path = None
        if browser_state_summary.screenshot:
            self.logger.debug(
                f'■ Storing screenshot for step {self.state.n_steps}, screenshot length: {len(browser_state_summary.screenshot)}'
            )
            screenshot_path = await self.screenshot_service.store_screenshot(browser_state_summary.screenshot, screenshot_path)
            self.logger.debug(f'■ Screenshot stored at: {screenshot_path}')
        else:
            self.logger.debug(f'■ No screenshot in browser_state_summary for step {self.state.n_steps}')

        state_history = BrowserStateHistory(
            url=browser_state_summary.url,
            title=browser_state_summary.title,
            tabs=browser_state_summary.tabs,
            interacted_element=interacted_elements,
            screenshot_path=screenshot_path,
        )

        history_item = AgentHistory(
            model_output=model_output,
            result=result,
            state=state_history,

```

```

    metadata=metadata,
    state_message=state_message,
)

self.history.add_item(history_item)

def _remove_think_tags(self, text: str) -> str:
    THINK_TAGS = re.compile(r'<think>.*?</think>', re.DOTALL)
    STRAY_CLOSE_TAG = re.compile(r'.*?</think>', re.DOTALL)
    # Step 1: Remove well-formed <think>...</think>
    text = re.sub(THINK_TAGS, '', text)
    # Step 2: If there's an unmatched closing tag </think>,
    #         remove everything up to and including that.
    text = re.sub(STRAY_CLOSE_TAG, '', text)
    return text.strip()

# region - URL replacement
def _replace_urls_in_text(self, text: str) -> tuple[str, dict[str, str]]:
    """Replace URLs in a text string"""

    replaced_urls: dict[str, str] = {}

    def replace_url(match: re.Match) -> str:
        """Url can only have 1 query and 1 fragment"""
        import hashlib

        original_url = match.group(0)

        # Find where the query/fragment starts
        query_start = original_url.find('?')
        fragment_start = original_url.find('#')

        # Find the earliest position of query or fragment
        after_path_start = len(original_url) # Default: no query/fragment
        if query_start != -1:
            after_path_start = min(after_path_start, query_start)
        if fragment_start != -1:
            after_path_start = min(after_path_start, fragment_start)

        # Split URL into base (up to path) and after_path (query + fragment)
        base_url = original_url[:after_path_start]
        after_path = original_url[after_path_start:]

        # If after_path is within the limit, don't shorten
        if len(after_path) <= self._url_shortening_limit:
            return original_url

        # If after_path is too long, truncate and add hash
        if after_path:
            truncated_after_path = after_path[: self._url_shortening_limit]
            # Create a short hash of the full after_path content
            hash_obj = hashlib.md5(after_path.encode('utf-8'))
            short_hash = hash_obj.hexdigest()[:7]
            # Create shortened URL
            shortened = f'{base_url}{truncated_after_path}...{short_hash}'
            # Only use shortened URL if it's actually shorter than the original
            if len(shortened) < len(original_url):
                replaced_urls[shortened] = original_url
            return shortened

        return original_url

    return URL_PATTERN.sub(replace_url, text), replaced_urls

def _process_messages_and_replace_long_urls_shorter_ones(self, input_messages: list[BaseMessage]) -> c
    """Replace long URLs with shorter ones
    @dev edits input_messages in place

    returns:
    tuple[filtered_input_messages, urls we replaced {shorter_url: original_url}]
    """
    from system.llm.messages import AssistantMessage, UserMessage

```

```

    urls_replaced: dict[str, str] = {}

    # Process each message, in place
    for message in input_messages:
        # no need to process SystemMessage, we have control over that anyway
        if isinstance(message, (UserMessage, AssistantMessage)):
            if isinstance(message.content, str):
                # Simple string content
                message.content, replaced_urls = self._replace_urls_in_text(message.content)
                urls_replaced.update(replaced_urls)

            elif isinstance(message.content, list):
                # List of content parts
                for part in message.content:
                    if isinstance(part, ContentPartTextParam):
                        part.text, replaced_urls = self._replace_urls_in_text(part.text)
                        urls_replaced.update(replaced_urls)

    return urls_replaced

    @staticmethod
    def _recursive_process_all_strings_inside_pydantic_model(model: BaseModel, url_replacements: dict[str, str], field_name: str, field_value: str):
        """Recursively process all strings inside a Pydantic model, replacing shortened URLs with originals in place"""
        for field_name, field_value in model.__dict__.items():
            if isinstance(field_value, str):
                # Replace shortened URLs with original URLs in string
                processed_string = Agent._replace_shortened_urls_in_string(field_value, url_replacements)
                setattr(model, field_name, processed_string)
            elif isinstance(field_value, BaseModel):
                # Recursively process nested Pydantic models
                Agent._recursive_process_all_strings_inside_pydantic_model(field_value, url_replacements)
            elif isinstance(field_value, dict):
                # Process dictionary values in place
                Agent._recursive_process_dict(field_value, url_replacements)
            elif isinstance(field_value, (list, tuple)):
                Agent._recursive_process_list_or_tuple(field_value, url_replacements)
            setattr(model, field_name, processed_value)

    @staticmethod
    def _recursive_process_dict(dictionary: dict, url_replacements: dict[str, str]) -> None:
        """Helper method to process dictionaries."""
        for k, v in dictionary.items():
            if isinstance(v, str):
                dictionary[k] = Agent._replace_shortened_urls_in_string(v, url_replacements)
            elif isinstance(v, BaseModel):
                Agent._recursive_process_all_strings_inside_pydantic_model(v, url_replacements)
            elif isinstance(v, dict):
                Agent._recursive_process_dict(v, url_replacements)
            elif isinstance(v, (list, tuple)):
                Agent._recursive_process_list_or_tuple(v, url_replacements)

    @staticmethod
    def _recursive_process_list_or_tuple(container: list | tuple, url_replacements: dict[str, str]) -> list | tuple:
        """Helper method to process lists and tuples."""
        if isinstance(container, tuple):
            # For tuples, create a new tuple with processed items
            processed_items = []
            for item in container:
                if isinstance(item, str):
                    processed_items.append(Agent._replace_shortened_urls_in_string(item, url_replacements))
                elif isinstance(item, BaseModel):
                    Agent._recursive_process_all_strings_inside_pydantic_model(item, url_replacements)
                    processed_items.append(item)
                elif isinstance(item, dict):
                    Agent._recursive_process_dict(item, url_replacements)
                    processed_items.append(item)
                elif isinstance(item, (list, tuple)):
                    processed_items.append(Agent._recursive_process_list_or_tuple(item, url_replacements))
            else:
                processed_items.append(item)
            return tuple(processed_items)
        else:

```

```

##### For lists, modify in place
##### for i, item in enumerate(container):
##### if isinstance(item, str):
##### container[i] = Agent._replace_shortened_urls_in_string(item, url_replacements)
##### elif isinstance(item, BaseModel):
##### Agent._recursive_process_all_strings_inside_pydantic_model(item, url_replacements)
##### elif isinstance(item, dict):
##### Agent._recursive_process_dict(item, url_replacements)
##### elif isinstance(item, (list, tuple)):
##### container[i] = Agent._recursive_process_list_or_tuple(item, url_replacements)
##### return container

##### @staticmethod
##### def _replace_shortened_urls_in_string(text: str, url_replacements: dict[str, str]) -> str:
##### """Replace all shortened URLs in a string with their original URLs."""
##### result = text
##### for shortened_url, original_url in url_replacements.items():
##### result = result.replace(shortened_url, original_url)
##### return result

##### # endregion - URL replacement

##### @time_execution_async('--get_next_action')
##### @observe_debug(ignore_input=True, ignore_output=True, name='get_model_output')
##### async def get_model_output(self, input_messages: list[BaseMessage]) -> AgentOutput:
##### """Get next action from LLM based on current state"""

##### urls_replaced = self._process_messages_and_replace_long_urls_shorter_ones(input_messages)

##### # Build kwargs for ainvoke
##### # Note: ChatBrowserUse will automatically generate action descriptions from output_format schema
##### kwargs: dict = {'output_format': self.AgentOutput, 'session_id': self.session_id}

##### try:
##### response = await self.llm.ainvoke(input_messages, **kwargs)
##### parsed: AgentOutput = response.completion # type: ignore[assignment]

##### # Replace any shortened URLs in the LLM response back to original URLs
##### if urls_replaced:
##### self._recursive_process_all_strings_inside_pydantic_model(parsed, urls_replaced)

##### # cut the number of actions to max_actions_per_step if needed
##### if len(parsed.action) > self.settings.max_actions_per_step:
##### parsed.action = parsed.action[: self.settings.max_actions_per_step]

##### if not (hasattr(self.state, 'paused') and (self.state.paused or self.state.stopped)):
##### log_response(parsed, self.tools.registry.registry, self.logger)
##### await self._broadcast_model_state(parsed)
#####
##### # MOCK FRONTEND EVENT BUS: Emitting "Thought"
##### if parsed.current_state and parsed.current_state.thinking:
##### self._emit_mock_frontend_event({
##### "event_type": "agent_thought",
##### "text": parsed.current_state.thinking,
##### "step": self.state.n_steps
##### })

##### self._log_next_action_summary(parsed)
##### return parsed
##### except ValidationError:
##### # Just re-raise - Pydantic's validation errors are already descriptive
##### raise
##### except (ModelRateLimitError, ModelProviderError) as e:
##### # Check if we can switch to a fallback LLM
##### if not self._try_switch_to_fallback_llm(e):
##### # No fallback available, re-raise the original error
##### raise
##### # Retry with the fallback LLM
##### return await self.get_model_output(input_messages)

##### def _try_switch_to_fallback_llm(self, error: ModelRateLimitError | ModelProviderError) -> bool:
##### """

```

```

    Attempt to switch to a fallback LLM after a rate limit or provider error.

    Returns True if successfully switched to a fallback, False if no fallback available.
    Once switched, the agent will use the fallback LLM for the rest of the run.
    """
    # Already using fallback - can't switch again
    if self._using_fallback_llm:
        self.logger.warning(
            f'Fallback LLM also failed ({type(error).__name__}: {error.message}), no more fallbacks available')
        return False

    # Check if error is retryable (rate limit, auth errors, or server errors)
    # 401: API key invalid/expired - fallback to different provider
    # 402: Insufficient credits/payment required - fallback to different provider
    # 429: Rate limit exceeded
    # 500, 502, 503, 504: Server errors
    retryable_status_codes = {401, 402, 429, 500, 502, 503, 504}
    is_retryable = isinstance(error, ModelRateLimitError) or (
        hasattr(error, 'status_code') and error.status_code in retryable_status_codes
    )

    if not is_retryable:
        return False

    # Check if we have a fallback LLM configured
    if self._fallback_llm is None:
        self.logger.warning(f'LLM error ({type(error).__name__}: {error.message}) but no fallback_llm configured')
        return False

    self._log_fallback_switch(error, self._fallback_llm)

    # Switch to the fallback LLM
    self.llm = self._fallback_llm
    self._using_fallback_llm = True

    # Register the fallback LLM for token cost tracking
    self.token_cost_service.register_llm(self._fallback_llm)

    return True

    def _log_fallback_switch(self, error: ModelRateLimitError | ModelProviderError, fallback: BaseChatModel):
        """Log when switching to a fallback LLM."""
        original_model = self._original_llm.model if hasattr(self._original_llm, 'model') else 'unknown'
        fallback_model = fallback.model if hasattr(fallback, 'model') else 'unknown'
        error_type = type(error).__name__
        status_code = getattr(error, 'status_code', 'N/A')

        self.logger.warning(
            f'Primary LLM ({original_model}) failed with {error_type} (status={status_code}), '
            f'switching to fallback LLM ({fallback_model})')

    async def _log_agent_run(self) -> None:
        """Log the agent run"""
        # Blue color for task
        self.logger.info(f'\033[34m Task: {self.task}\033[0m')

        self.logger.debug(f'Browser-Use Library Version {self.version} ({self.source})')

        # Check for latest version and log upgrade message if needed
        if CONFIG.BROWSER_USE_VERSION_CHECK:
            latest_version = await check_latest_browser_use_version()
            if latest_version and latest_version != self.version:
                self.logger.info(
                    f'Newer version available: {latest_version} (current: {self.version}). Upgrade with: uv add browser-use')

    def _log_first_step_startup(self) -> None:
        """Log startup message only on the first step"""
        if len(self.history.history) == 0:
            self.logger.info(

```

```

#####f'Starting a browser-use agent with version {self.version}, with provider={self.llm.provider} and m
#####)

def _log_step_context(self, browser_state_summary: BrowserStateSummary) -> None:
    """Log step context information"""
    url = browser_state_summary.url if browser_state_summary else ''
    url_short = url[:50] + '...' if len(url) > 50 else url
    interactive_count = len(browser_state_summary.dom_state.selector_map) if browser_state_summary else 0
    self.logger.info('\n')
    self.logger.info(f'■ Step {self.state.n_steps}:')
    self.logger.debug(f'Evaluating page with {interactive_count} interactive elements on: {url_short}')

def _log_next_action_summary(self, parsed: 'AgentOutput') -> None:
    """Log a comprehensive summary of the next action(s)"""
    if not (self.logger.isEnabledFor(logging.DEBUG) and parsed.action):
        return

    action_count = len(parsed.action)

    ## Collect action details
    action_details = []
    for i, action in enumerate(parsed.action):
        action_data = action.model_dump(exclude_unset=True)
        action_name = next(iter(action_data.keys())) if action_data else 'unknown'
        action_params = action_data.get(action_name, {}) if action_data else {}

        ## Format key parameters concisely
        param_summary = []
        if isinstance(action_params, dict):
            for key, value in action_params.items():
                if key == 'index':
                    param_summary.append(f'#{value}')
                elif key == 'text' and isinstance(value, str):
                    text_preview = value[:30] + '...' if len(value) > 30 else value
                    param_summary.append(f'text="{text_preview}"')
                elif key == 'url':
                    param_summary.append(f'url="{value}"')
                elif key == 'success':
                    param_summary.append(f'success={value}')
                elif isinstance(value, (str, int, bool)):
                    val_str = str(value)[:30] + '...' if len(str(value)) > 30 else str(value)
                    param_summary.append(f'{key}={val_str}')

        param_str = f'({", ".join(param_summary)})' if param_summary else ''
        action_details.append(f'{action_name}{param_str}')

def _prepare_demo_message(self, message: str, limit: int = 600) -> str:
    ## Previously truncated long entries; keep full text for better context in demo panel
    return message.strip()

async def _demo_mode_log(self, message: str, level: str = 'info', metadata: dict[str, Any] | None = None):
    if not self._demo_mode_enabled or not message or self.browser_session is None:
        return
    try:
        await self.browser_session.send_demo_mode_log(
            message=self._prepare_demo_message(message),
            level=level,
            metadata=metadata or {},
        )
    except Exception as exc:
        self.logger.debug(f'[DemoMode] Failed to send overlay log: {exc}')

async def _broadcast_model_state(self, parsed: 'AgentOutput') -> None:
    if not self._demo_mode_enabled:
        return

    state = parsed.current_state
    step_meta = {'step': self.state.n_steps}

    if state.thinking:
        await self._demo_mode_log(state.thinking, 'thought', step_meta)
    if state.evaluation_previous_goal:

```

```

    eval_text = state.evaluation_previous_goal
    level = 'success' if 'success' in eval_text.lower() else 'warning' if 'failure' in eval_text.lower()
    await self._demo_mode_log(eval_text, level, step_meta)

    if state.memory:
        await self._demo_mode_log(f'Memory: {state.memory}', 'info', step_meta)

    if state.next_goal:
        await self._demo_mode_log(f'Next goal: {state.next_goal}', 'info', step_meta)

    def _log_step_completion_summary(self, step_start_time: float, result: list[ActionResult]) -> str | None:
        """Log step completion summary with action count, timing, and success/failure stats"""
        if not result:
            return None

        step_duration = time.time() - step_start_time
        action_count = len(result)

        # Count success and failures
        success_count = sum(1 for r in result if not r.error)
        failure_count = action_count - success_count

        # Format success/failure indicators
        success_indicator = f'█ {success_count}' if success_count > 0 else ''
        failure_indicator = f'█ {failure_count}' if failure_count > 0 else ''
        status_parts = [part for part in [success_indicator, failure_indicator] if part]
        status_str = ' | '.join(status_parts) if status_parts else '█ 0'

        message = (
            f'█ Step {self.state.n_steps}: Ran {action_count} action{" " if action_count == 1 else "s"} '
            f'in {step_duration:.2f}s: {status_str}'
        )
        self.logger.debug(message)
        return message

    def _log_final_outcome_messages(self) -> None:
        """Log helpful messages to user based on agent run outcome"""
        # Check if agent failed
        is_successful = self.history.is_successful()

        if is_successful is False or is_successful is None:
            # Get final result to check for specific failure reasons
            final_result = self.history.final_result()
            final_result_str = str(final_result).lower() if final_result else ''

            # Check for captcha/cloudflare related failures
            captcha_keywords = ['captcha', 'cloudflare', 'recaptcha', 'challenge', 'bot detection', 'access denied']
            has_captcha_issue = any(keyword in final_result_str for keyword in captcha_keywords)

            if has_captcha_issue:
                # Suggest use_cloud=True for captcha/cloudflare issues
                task_preview = self.task[:10] if len(self.task) > 10 else self.task
                self.logger.info('')
                self.logger.info('Failed because of CAPTCHA? For better browser stealth, try:')
                self.logger.info(f'    agent = Agent(task="{task_preview}...", browser=Browser(use_cloud=True))')

            # General failure message
            self.logger.info('')
            self.logger.info('Did the Agent not work as expected? Let us fix this!')
            self.logger.info('    Open a short issue on GitHub: https://github.com/browser-use/browser-use/issues')

    def _log_agent_event(self, max_steps: int, agent_run_error: str | None = None) -> None:
        """Sent the agent event for this run to telemetry"""

    token_summary = self.token_cost_service.get_usage_tokens_for_model(self.llm.model)

    # Prepare action_history data correctly
    action_history_data = []
    for item in self.history.history:
        if item.model_output and item.model_output.action:
            # Convert each ActionModel in the step to its dictionary representation
            step_actions = [

```



```

#####action.model_dump(exclude_unset=True)
#####for action in item.model_output.action
#####if action # Ensure action is not None if list allows it
#####]
#####action_history_data.append(step_actions)
#####else:
###### Append None or [] if a step had no actions or no model output
#####action_history_data.append(None)

#####final_res = self.history.final_result()
#####final_result_str = json.dumps(final_res) if final_res is not None else None

###### Extract judgement data if available
#####judgement_data = self.history.judgement()
#####judge_verdict = judgement_data.get('verdict') if judgement_data else None
#####judge_reasoning = judgement_data.get('reasoning') if judgement_data else None
#####judge_failure_reason = judgement_data.get('failure_reason') if judgement_data else None
#####judge_reached_captcha = judgement_data.get('reached_captcha') if judgement_data else None
#####judge_impossible_task = judgement_data.get('impossible_task') if judgement_data else None

#####self.telemetry.capture(
#####AgentTelemetryEvent(
#####task=self.task,
#####model=self.llm.model,
#####model_provider=self.llm.provider,
#####max_steps=max_steps,
#####max_actions_per_step=self.settings.max_actions_per_step,
#####use_vision=self.settings.use_vision,
#####version=self.version,
#####source=self.source,
#####cdp_url=urlparse(self.browser_session.cdp_url).hostname
#####if self.browser_session and self.browser_session.cdp_url
#####else None,
#####agent_type=None, # Regular Agent (not code-use)
#####action_errors=self.history.errors(),
#####action_history=action_history_data,
#####urls_visited=self.history.urls(),
#####steps=self.state.n_steps,
#####total_input_tokens=token_summary.prompt_tokens,
#####total_output_tokens=token_summary.completion_tokens,
#####prompt_cached_tokens=token_summary.prompt_cached_tokens,
#####total_tokens=token_summary.total_tokens,
#####total_duration_seconds=self.history.total_duration_seconds(),
#####success=self.history.is_successful(),
#####final_result_response=final_result_str,
#####error_message=agent_run_error,
#####judge_verdict=judge_verdict,
#####judge_reasoning=judge_reasoning,
#####judge_failure_reason=judge_failure_reason,
#####judge_reached_captcha=judge_reached_captcha,
#####judge_impossible_task=judge_impossible_task,
#####)
#####)

#####async def take_step(self, step_info: AgentStepInfo | None = None) -> tuple[bool, bool]:
#####"""Take a step

#####Returns:
#####    Tuple[bool, bool]: (is_done, is_valid)
#####"""
#####if step_info is not None and step_info.step_number == 0:
###### First step
#####self._log_first_step_startup()
###### Normally there was no try catch here but the callback can raise an InterruptedError which we skip
#####try:
#####    await self._execute_initial_actions()
#####except InterruptedError:
#####    pass
#####except Exception as e:
#####    raise e

#####await self.step(step_info)

```

```

    if self.history.is_done():
        await self.log_completion()

    # Run full judge before done callback if enabled
    if self.settings.use_judge:
        await self._judge_and_log()

    if self.register_done_callback:
        if inspect.iscoroutinefunction(self.register_done_callback):
            await self.register_done_callback(self.history)
        else:
            self.register_done_callback(self.history)
    return True, True

    return False, False

def _extract_start_url(self, task: str) -> str | None:
    """Extract URL from task string using naive pattern matching."""

    import re

    # Remove email addresses from task before looking for URLs
    task_without_emails = re.sub(r'\b[A-Za-z0-9._%+-]@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', '', task)

    # Look for common URL patterns
    patterns = [
        r'https?://[^\s<>"\']+', # Full URLs with http/https
        r'(?:(www\.)?[a-zA-Z0-9-]+(?:\.[a-zA-Z0-9-]+)*\.[a-zA-Z]{2,}){1,}(:/[\s<>"\']*)?', # Domain names with
    ]

    # File extensions that should be excluded from URL detection
    # These are likely files rather than web pages to navigate to
    excluded_extensions = {
        # Documents
        'pdf',
        'doc',
        'docx',
        'xls',
        'xlsx',
        'ppt',
        'pptx',
        'odt',
        'ods',
        'odp',
        # Text files
        'txt',
        'md',
        'csv',
        'json',
        'xml',
        'yaml',
        'yml',
        # Archives
        'zip',
        'rar',
        '7z',
        'tar',
        'gz',
        'bz2',
        'xz',
        # Images
        'jpg',
        'jpeg',
        'png',
        'gif',
        'bmp',
        'svg',
        'webp',
        'ico',
        # Audio/Video
        'mp3',
        'mp4',
    }

```

```

    'avi',
    'mkv',
    'mov',
    'wav',
    'flac',
    'ogg',
    # Code/Data
    'py',
    'js',
    'css',
    'java',
    'cpp',
    # Academic/Research
    'bib',
    'bibtex',
    'tex',
    'latex',
    'cls',
    'sty',
    # Other common file types
    'exe',
    'msi',
    'dmg',
    'pkg',
    'deb',
    'rpm',
    'iso',
    # GitHub/Project paths
    'polynomial',
}

excluded_words = {
    'never',
    'dont',
    'not',
    "don't",
}

found_urls = []
for pattern in patterns:
    matches = re.finditer(pattern, task_without_emails)
    for match in matches:
        url = match.group(0)
        original_position = match.start() # Store original position before URL modification

        # Remove trailing punctuation that's not part of URLs
        url = re.sub(r'[.,;:!?()\[\]]+$', '', url)

        # Check if URL ends with a file extension that should be excluded
        url_lower = url.lower()
        should_exclude = False
        for ext in excluded_extensions:
            if f'.{ext}' in url_lower:
                should_exclude = True
                break

        if should_exclude:
            self.logger.debug(f'Excluding URL with file extension from auto-navigation: {url}')
            continue

        # If in the 20 characters before the url position is a word in excluded_words skip to avoid "Never
        context_start = max(0, original_position - 20)
        context_text = task_without_emails[context_start:original_position]
        if any(word.lower() in context_text.lower() for word in excluded_words):
            self.logger.debug(
                f'Excluding URL with word in excluded words from auto-navigation: {url} (context: "{context_text}")
            )
            continue

        # Add https:// if missing (after excluded words check to avoid position calculation issues)
        if not url.startswith(('http://', 'https://')):
            url = 'https://' + url

```

```

        found_urls.append(url)

    unique_urls = list(set(found_urls))
    ## If multiple URLs found, skip directly_open_urling
    if len(unique_urls) > 1:
        self.logger.debug(f'Multiple URLs found ({len(found_urls)}), skipping directly_open_url to avoid amb
        return None

    ## If exactly one URL found, return it
    if len(unique_urls) == 1:
        return unique_urls[0]

    return None

    async def _execute_step(
        self,
        step: int,
        max_steps: int,
        step_info: AgentStepInfo,
        on_step_start: AgentHookFunc | None = None,
        on_step_end: AgentHookFunc | None = None,
    ) -> bool:
        """
        Execute a single step with timeout.

        Returns:
        bool: True if task is done, False otherwise
        """
        if on_step_start is not None:
            await on_step_start(self)

        await self._demo_mode_log(
            f'Starting step {step + 1}/{max_steps}',
            'info',
            {'step': step + 1, 'total_steps': max_steps},
        )

        self.logger.debug(f'■ Starting step {step + 1}/{max_steps}...')

        try:
            await asyncio.wait_for(
                self.step(step_info),
                timeout=self.settings.step_timeout,
            )
            self.logger.debug(f'■ Completed step {step + 1}/{max_steps}')
        except TimeoutError:
            ## Handle step timeout gracefully
            error_msg = f'Step {step + 1} timed out after {self.settings.step_timeout} seconds'
            self.logger.error(f'■ {error_msg}')
            await self._demo_mode_log(error_msg, 'error', {'step': step + 1})
            self.state.consecutive_failures += 1
            self.state.last_result = [ActionResult(error=error_msg)]

        if on_step_end is not None:
            await on_step_end(self)

        if self.history.is_done():
            await self.log_completion()

        ## Run full judge before done callback if enabled
        if self.settings.use_judge:
            await self._judge_and_log()

        if self.register_done_callback:
            if inspect.iscoroutinefunction(self.register_done_callback):
                await self.register_done_callback(self.history)
            else:
                self.register_done_callback(self.history)

        return True

    return False

```

```

@observe(name='agent.run', ignore_input=True, ignore_output=True)
@time_execution_async('--run')
async def run(
    self,
    max_steps: int = 500,
    on_step_start: AgentHookFunc | None = None,
    on_step_end: AgentHookFunc | None = None,
) -> AgentHistoryList[AgentStructuredOutput]:
    """Execute the task with maximum number of steps"""

    loop = asyncio.get_event_loop()
    agent_run_error: str | None = None # Initialize error tracking variable
    self._force_exit_telemetry_logged = False # ADDED: Flag for custom telemetry on force exit
    should_delay_close = False

    ## Set up the signal handler with callbacks specific to this agent
    from system.utils import SignalHandler

    ## Define the custom exit callback function for second CTRL+C
    def on_force_exit_log_telemetry():
        self._log_agent_event(max_steps=max_steps, agent_run_error='SIGINT: Cancelled by user')
        ## NEW: Call the flush method on the telemetry instance
        if hasattr(self, 'telemetry') and self.telemetry:
            self.telemetry.flush()
        self._force_exit_telemetry_logged = True # Set the flag

    signal_handler = SignalHandler(
        loop=loop,
        pause_callback=self.pause,
        resume_callback=self.resume,
        custom_exit_callback=on_force_exit_log_telemetry, # Pass the new telemetrycallback
        exit_on_second_int=True,
    )
    signal_handler.register()

    try:
        await self._log_agent_run()

        self.logger.debug(
            f'■ Agent setup: Agent Session ID {self.session_id[-4:]}, Task ID {self.task_id[-4:]}, Browser Sess
        )

        ## Initialize timing for session and task
        self._session_start_time = time.time()
        self._task_start_time = self._session_start_time # Initialize task start time

        ## Only dispatch session events if this is the first run
        if not self.state.session_initialized:
            self.logger.debug('■ Dispatching CreateAgentSessionEvent...')
            ## Emit CreateAgentSessionEvent at the START of run()
            self.eventbus.dispatch(CreateAgentSessionEvent.from_agent(self))

            self.state.session_initialized = True

            self.logger.debug('■ Dispatching CreateAgentTaskEvent...')
            ## Emit CreateAgentTaskEvent at the START of run()
            self.eventbus.dispatch(CreateAgentTaskEvent.from_agent(self))

            ## Log startup message on first step (only if we haven't already done steps)
            self._log_first_step_startup()
            ## Start browser session and attach watchdogs
            await self.browser_session.start()
            if self._demo_mode_enabled:
                await self._demo_mode_log(f'Started task: {self.task}', 'info', {'tag': 'task'})
                await self._demo_mode_log(
                    'Demo mode active - follow the side panel for live thoughts and actions.',
                    'info',
                    {'tag': 'status'},
                )

            ## Register skills as actions if SkillService is configured
            await self._register_skills_as_actions()

```

```

##### Normally there was no try catch here but the callback can raise an InterruptedError
##### try:
##### await self._execute_initial_actions()
##### except InterruptedError:
##### pass
##### except Exception as e:
##### raise e

##### self.logger.debug(
##### f'■ Starting main execution loop with max {max_steps} steps (currently at step {self.state.n_steps})
##### )
##### while self.state.n_steps <= max_steps:
##### current_step = self.state.n_steps - 1 # Convert to 0-indexed for step_info

##### # Use the consolidated pause state management
##### if self.state.paused:
##### self.logger.debug(f'■ Step {self.state.n_steps}: Agent paused, waiting to resume...')
##### await self._external_pause_event.wait()
##### signal_handler.reset()

##### # Check if we should stop due to too many failures, if final_response_after_failure is True, we try
##### if (self.state.consecutive_failures) >= self.settings.max_failures + int(
##### self.settings.final_response_after_failure
##### ):
##### self.logger.error(f'■ Stopping due to {self.settings.max_failures} consecutive failures')
##### agent_run_error = f'Stopped due to {self.settings.max_failures} consecutive failures'
##### break

##### # Check control flags before each step
##### if self.state.stopped:
##### self.logger.info('■ Agent stopped')
##### agent_run_error = 'Agent stopped programmatically'
##### break

##### step_info = AgentStepInfo(step_number=current_step, max_steps=max_steps)
##### is_done = await self._execute_step(current_step, max_steps, step_info, on_step_start, on_step_end)

##### if is_done:
##### # Agent has marked the task as done
##### if self._demo_mode_enabled and self.history.history:
##### final_result_text = self.history.final_result() or 'Task completed'
##### await self._demo_mode_log(f'Final Result: {final_result_text}', 'success', {'tag': 'task'})

##### should_delay_close = True
##### break
##### else:
##### agent_run_error = 'Failed to complete task in maximum steps'

##### self.history.add_item(
##### AgentHistory(
##### model_output=None,
##### result=[ActionResult(error=agent_run_error, include_in_memory=True)],
##### state=BrowserStateHistory(
##### url='',
##### title='',
##### tabs=[],
##### interacted_element=[],
##### screenshot_path=None,
##### ),
##### metadata=None,
##### )
##### )

##### self.logger.info(f'■ {agent_run_error}')

##### self.history.usage = await self.token_cost_service.get_usage_summary()

##### # NEW: Workflow Macro Recording
##### # Only record if the agent completed successfully (no errors) and wasn't executing an injected macro
##### if agent_run_error is None and self.history.is_successful() and not self.workflow_executor:
##### try:

```

```

#####from system.agent.workflow import WorkflowRecorder
#####import os
#####
#####workflow_template = WorkflowRecorder.extract_workflow(self.history, original_task=self.task)
#####
###### Save to a dedicated file for the mock frontend
#####workflow_json = workflow_template.to_json()
#####
###### Ensure output directory exists
#####output_dir = Path("C:/Users/maham/tmp")
#####output_dir.mkdir(parents=True, exist_ok=True)
#####
#####workflow_path = output_dir / "mock_macro_workflow.json"
#####with open(workflow_path, "w", encoding="utf-8") as f:
#####    f.write(workflow_json)
#####    self.logger.info(f'##### Workflow macro successfully recorded to {workflow_path}')
#####
###### Also emit a lightweight event via the mock stream
#####await self._emit_mock_frontend_event({
#####    "event_type": "workflow_macro_recorded",
#####    "workflow_id": workflow_template.workflow_id,
#####    "target_url": workflow_template.target_url,
#####    "step_count": len(workflow_template.steps)
#####})
#####except Exception as macro_err:
#####    self.logger.warning(f'##### Failed to compile workflow macro: {macro_err}')

###### set the model output schema and call it on the fly
#####if self.history._output_model_schema is None and self.output_model_schema is not None:
#####    self.history._output_model_schema = self.output_model_schema

#####return self.history

#####except KeyboardInterrupt:
#####    # Already handled by our signal handler, but catch any direct KeyboardInterrupt as well
#####    self.logger.debug('Got KeyboardInterrupt during execution, returning current history')
#####    agent_run_error = 'KeyboardInterrupt'

#####self.history.usage = await self.token_cost_service.get_usage_summary()

#####return self.history

#####except Exception as e:
#####    self.logger.error(f'Agent run failed with exception: {e}', exc_info=True)
#####    agent_run_error = str(e)
#####    raise e

#####finally:
#####    if should_delay_close and self._demo_mode_enabled and agent_run_error is None:
#####        await asyncio.sleep(30)
#####    if agent_run_error:
#####        await self._demo_mode_log(f'Agent stopped: {agent_run_error}', 'error', {'tag': 'run'})
#####    # Log token usage summary
#####    await self.token_cost_service.log_usage_summary()

###### Unregister signal handlers before cleanup
#####signal_handler.unregister()

#####if not self._force_exit_telemetry_logged: # MODIFIED: Check the flag
#####    try:
#####        self._log_agent_event(max_steps=max_steps, agent_run_error=agent_run_error)
#####    except Exception as log_e: # Catch potential errors during logging itself
#####        self.logger.error(f'Failed to log telemetry event: {log_e}', exc_info=True)
#####    else:
#####        # ADDED: Info message when custom telemetry for SIGINT was already logged
#####        self.logger.debug('Telemetry for force exit (SIGINT) was logged by custom exit callback.')

###### NOTE: CreateAgentSessionEvent and CreateAgentTaskEvent are now emitted at the START of run()
###### to match backend requirements for CREATE events to be fired when entities are created,
###### not when they are completed

###### Emit UpdateAgentTaskEvent at the END of run() with final task state

```

```

self.eventbus.dispatch(UpdateAgentTaskEvent.from_agent(self))

# Generate GIF if needed before stopping event bus
if self.settings.generate_gif:
    output_path: str = 'agent_history.gif'
    if isinstance(self.settings.generate_gif, str):
        output_path = self.settings.generate_gif

# Lazy import gif module to avoid heavy startup cost
from system.agent.gif import create_history_gif

create_history_gif(task=self.task, history=self.history, output_path=output_path)

# Only emit output file event if GIF was actually created
if Path(output_path).exists():
    output_event = await CreateAgentOutputFileEvent.from_agent_and_file(self, output_path)
    self.eventbus.dispatch(output_event)

# Log final messages to user based on outcome
self._log_final_outcome_messages()

# Stop the event bus gracefully, waiting for all events to be processed
# Configurable via TIMEOUT_AgentEventBusStop env var (default: 3.0s)
await self.eventbus.stop(clear=True, timeout=_get_timeout('TIMEOUT_AgentEventBusStop', 3.0))

await self.close()

@observe_debug(ignore_input=True, ignore_output=True)
@time_execution_async('--multi_act')
async def multi_act(self, actions: list[ActionModel]) -> list[ActionResult]:
    """Execute multiple actions with page-change guards.

    Two layers of protection prevent executing actions against stale DOM:
    1. Static flag: actions tagged with terminates_sequence=True (navigate, search, go_back, switch)
       automatically abort remaining queued actions.
    2. Runtime detection: after every action, the current URL and focused target are compared
       to pre-action values. Any change aborts the remaining queue.
    """
    results: list[ActionResult] = []
    time_elapsed = 0
    total_actions = len(actions)

    assert self.browser_session is not None, 'BrowserSession is not set up'
    try:
        if (
            self.browser_session._cached_browser_state_summary is not None
            and self.browser_session._cached_browser_state_summary.dom_state is not None
        ):
            cached_selector_map = dict(self.browser_session._cached_browser_state_summary.dom_state.selector_map)
            cached_element_hashes = {e.parent_branch_hash() for e in cached_selector_map.values()}
        else:
            cached_selector_map = {}
            cached_element_hashes = set()
        except Exception as e:
            self.logger.error(f'Error getting cached selector map: {e}')
            cached_selector_map = {}
            cached_element_hashes = set()

        for i, action in enumerate(actions):
            # Get action name from the action model BEFORE try block to ensure it's always available in except
            action_data = action.model_dump(exclude_unset=True)
            action_name = next(iter(action_data.keys())) if action_data else 'unknown'

            if i > 0:
                # ONLY ALLOW TO CALL `done` IF IT IS A SINGLE ACTION
                if action_data.get('done') is not None:
                    msg = f'Done action is allowed only as a single action - stopped after action {i} / {total_actions}'
                    self.logger.debug(msg)
                    break

            # wait between actions (only after first action)
            if i > 0:

```



```

#####self.logger.debug(f'Waiting {self.browser_profile.wait_between_actions} seconds between actions')
#####await asyncio.sleep(self.browser_profile.wait_between_actions)

#####try:
#####await self._check_stop_or_pause()

###### Log action before execution
#####await self._log_action(action, action_name, i + 1, total_actions)

###### Capture pre-action state for runtime page-change detection
#####pre_action_url = await self.browser_session.get_current_page_url()
#####pre_action_focus = self.browser_session.agent_focus_target_id

#####time_start = time.time()

#####result = await self.tools.act(
#####    action=action,
#####    browser_session=self.browser_session,
#####    file_system=self.file_system,
#####    page_extraction_llm=self.settings.page_extraction_llm,
#####    sensitive_data=self.sensitive_data,
#####    available_file_paths=self.available_file_paths,
#####    extraction_schema=self.extraction_schema,
#####)

#####time_end = time.time()
#####time_elapsed = time_end - time_start

#####if result.error:
#####    await self._demo_mode_log(
#####        f'Action "{action_name}" failed: {result.error}',
#####        'error',
#####        {'action': action_name, 'step': self.state.n_steps},
#####    )
#####elif result.is_done:
#####    completion_text = result.long_term_memory or result.extracted_content or 'Task marked as done.'
#####    level = 'success' if result.success is not False else 'warning'
#####    await self._demo_mode_log(
#####        completion_text,
#####        level,
#####        {'action': action_name, 'step': self.state.n_steps},
#####    )

#####results.append(result)

#####if results[-1].is_done or results[-1].error or i == total_actions - 1:
#####    break

###### --- Page-change guards (only when more actions remain) ---

###### Layer 1: Static flag – action metadata declares it changes the page
#####registered_action = self.tools.registry.registry.actions.get(action_name)
#####if registered_action and registered_action.terminates_sequence:
#####    self.logger.info(
#####        f'Action "{action_name}" terminates sequence – skipping {total_actions - i - 1} remaining action'
#####    )
#####    break

###### Layer 2: Runtime detection – URL or focus target changed
#####post_action_url = await self.browser_session.get_current_page_url()
#####post_action_focus = self.browser_session.agent_focus_target_id

#####if post_action_url != pre_action_url or post_action_focus != pre_action_focus:
#####    self.logger.info(f'Page changed after "{action_name}" – skipping {total_actions - i - 1} remaining'
#####    break

#####except Exception as e:
#####    # Handle any exceptions during action execution
#####    self.logger.error(f'Executing action {i + 1} failed -> {type(e).__name__}: {e}')
#####    await self._demo_mode_log(
#####        f'Action "{action_name}" raised {type(e).__name__}: {e}',
#####        'error',

```

```

        {'action': action_name, 'step': self.state.n_steps},
        )
        raise e

    return results

    async def _log_action(self, action, action_name: str, action_num: int, total_actions: int) -> None:
        """Log the action before execution with colored formatting"""
        ## Color definitions
        blue = '\033[34m' # Action name
        magenta = '\033[35m' # Parameter names
        reset = '\033[0m'

        ## Format action number and name
        if total_actions > 1:
            action_header = f'## [{action_num}/{total_actions}] {blue}{action_name}{reset}:'
            plain_header = f'## [{action_num}/{total_actions}] {action_name}:'
        else:
            action_header = f'## {blue}{action_name}{reset}:'
            plain_header = f'## {action_name}:'

        ## Get action parameters
        action_data = action.model_dump(exclude_unset=True)
        params = action_data.get(action_name, {})

        ## Build parameter parts with colored formatting
        param_parts = []
        plain_param_parts = []

        if params and isinstance(params, dict):
            for param_name, value in params.items():
                ## Truncate long values for readability
                if isinstance(value, str) and len(value) > 150:
                    display_value = value[:150] + '...'
                elif isinstance(value, list) and len(str(value)) > 200:
                    display_value = str(value)[:200] + '...'
                else:
                    display_value = value

                param_parts.append(f'{magenta}{param_name}{reset}: {display_value}')
                plain_param_parts.append(f'{param_name}: {display_value}')

        ## Join all parts
        if param_parts:
            params_string = ', '.join(param_parts)
            self.logger.info(f' {action_header} {params_string}')
        else:
            self.logger.info(f' {action_header}')

        if self._demo_mode_enabled:
            panel_message = plain_header
            if plain_param_parts:
                panel_message = f'{panel_message} {"", ".join(plain_param_parts)}'
            await self._demo_mode_log(panel_message.strip(), 'action', {'action': action_name, 'step': self.state.n_steps})

    async def log_completion(self) -> None:
        """Log the completion of the task"""
        ## self._task_end_time = time.time()
        ## self._task_duration = self._task_end_time - self._task_start_time TODO: this is not working when using
        if self.history.is_successful():
            self.logger.info(' Task completed successfully')
            await self._demo_mode_log('Task completed successfully', 'success', {'tag': 'task'})

    async def _generate_rerun_summary(
        self, original_task: str, results: list[ActionResult], summary_llm: BaseChatModel | None = None
    ) -> ActionResult:
        """Generate AI summary of rerun completion using screenshot and last step info"""
        from system.agent.views import RerunSummaryAction

        ## Get current screenshot
        screenshot_b64 = None
        try:

```

```

■■■screenshot = await self.browser_session.take_screenshot(full_page=False)
■■■if screenshot:
■■■import base64

■■■screenshot_b64 = base64.b64encode(screenshot).decode('utf-8')
■■■except Exception as e:
■■■self.logger.warning(f'Failed to capture screenshot for rerun summary: {e}')

■■■# Build summary prompt and message
■■■error_count = sum(1 for r in results if r.error)
■■■success_count = len(results) - error_count

■■■from system.agent.prompts import get_rerun_summary_message, get_rerun_summary_prompt

■■■prompt = get_rerun_summary_prompt(
■■■original_task=original_task,
■■■total_steps=len(results),
■■■success_count=success_count,
■■■error_count=error_count,
■■■)

■■■# Use provided LLM, agent's LLM, or fall back to OpenAI with structured output
■■■try:
■■■# Determine which LLM to use
■■■if summary_llm is None:
■■■# Try to use the agent's LLM first
■■■summary_llm = self.llm
■■■self.logger.debug('Using agent LLM for rerun summary')
■■■else:
■■■self.logger.debug(f'Using provided LLM for rerun summary: {summary_llm.model}')

■■■# Build message with prompt and optional screenshot
■■■from system.llm.messages import BaseMessage

■■■message = get_rerun_summary_message(prompt, screenshot_b64)
■■■messages: list[BaseMessage] = [message] # type: ignore[list-item]

■■■# Try calling with structured output first
■■■self.logger.debug(f'Calling LLM for rerun summary with {len(messages)} message(s)')
■■■try:
■■■kwargs: dict = {'output_format': RerunSummaryAction}
■■■response = await summary_llm.ainvoke(messages, **kwargs)
■■■summary: RerunSummaryAction = response.completion # type: ignore[assignment]
■■■self.logger.debug(f'LLM response type: {type(summary)}')
■■■self.logger.debug(f'LLM response: {summary}')
■■■except Exception as structured_error:
■■■# If structured output fails (e.g., Browser-Use LLM doesn't support it for this type),
■■■# fall back to text response without parsing
■■■self.logger.debug(f'Structured output failed: {structured_error}, falling back to text response')

■■■response = await summary_llm.ainvoke(messages, None)
■■■response_text = response.completion
■■■self.logger.debug(f'LLM text response: {response_text}')

■■■# Use the text response directly as the summary
■■■summary = RerunSummaryAction(
■■■summary=response_text if isinstance(response_text, str) else str(response_text),
■■■success=error_count == 0,
■■■completion_status='complete' if error_count == 0 else ('partial' if success_count > 0 else 'failed')
■■■)

■■■self.logger.info(f'■ Rerun Summary: {summary.summary}')
■■■self.logger.info(f'■ Status: {summary.completion_status} (success={summary.success})')

■■■return ActionResult(
■■■is_done=True,
■■■success=summary.success,
■■■extracted_content=summary.summary,
■■■long_term_memory=f'Rerun completed with status: {summary.completion_status}. {summary.summary[:100]}'
■■■)

■■■except Exception as e:

```

```

    self.logger.warning(f'Failed to generate AI summary: {e.__class__.__name__}: {e}')
    self.logger.debug('Full error traceback:', exc_info=True)
    # Fallback to simple summary
    return ActionResult(
        is_done=True,
        success=error_count == 0,
        extracted_content=f'Rerun completed: {success_count}/{len(results)} steps succeeded',
        long_term_memory=f'Rerun completed: {success_count} steps succeeded, {error_count} errors',
    )

    async def _execute_ai_step(
        self,
        query: str,
        include_screenshot: bool = False,
        extract_links: bool = False,
        ai_step_llm: BaseChatModel | None = None,
    ) -> ActionResult:
        """
        Execute an AI step during rerun to re-evaluate extract actions.
        Analyzes full page DOM/markdown + optional screenshot.

        Args:
            query: What to analyze or extract from the current page
            include_screenshot: Whether to include screenshot in analysis
            extract_links: Whether to include links in markdown extraction
            ai_step_llm: Optional LLM to use. If not provided, uses agent's LLM

        Returns:
            ActionResult with extracted content
        """
        from system.agent.prompts import get_ai_step_system_prompt, get_ai_step_user_prompt, get_rerun_summary
        from system.llm.messages import SystemMessage, UserMessage
        from system.utils import sanitize_surrogates

        # Use provided LLM or agent's LLM
        llm = ai_step_llm or self.llm
        self.logger.debug(f'Using LLM for AI step: {llm.model}')

        # Extract clean markdown
        try:
            from system.dom.markdown_extractor import extract_clean_markdown

            content, content_stats = await extract_clean_markdown(
                browser_session=self.browser_session, extract_links=extract_links
            )
        except Exception as e:
            return ActionResult(error=f'Could not extract clean markdown: {type(e).__name__}: {e}')

        # Get screenshot if requested
        screenshot_b64 = None
        if include_screenshot:
            try:
                screenshot = await self.browser_session.take_screenshot(full_page=False)
            except Exception as e:
                self.logger.warning(f'Failed to capture screenshot for ai_step: {e}')

        # Build prompt with content stats
        original_html_length = content_stats['original_html_chars']
        initial_markdown_length = content_stats['initial_markdown_chars']
        final_filtered_length = content_stats['final_filtered_chars']
        chars_filtered = content_stats['filtered_chars_removed']

        stats_summary = f'Content processed: {original_html_length:,} HTML chars → {initial_markdown_length:,} '
        if chars_filtered > 0:
            stats_summary += f' (filtered {chars_filtered:,} chars of noise)'

        # Sanitize content
        content = sanitize_surrogates(content)

```

```

    query = sanitize_surrogates(query)

    ## Get prompts from prompts.py
    system_prompt = get_ai_step_system_prompt()
    prompt_text = get_ai_step_user_prompt(query, stats_summary, content)

    ## Build user message with optional screenshot
    if screenshot_b64:
        user_message = get_rerun_summary_message(prompt_text, screenshot_b64)
    else:
        user_message = UserMessage(content=prompt_text)

    try:
        import asyncio

        response = await asyncio.wait_for(llm.ainvoke([SystemMessage(content=system_prompt), user_message]),

        current_url = await self.browser_session.get_current_page_url()
        extracted_content = (
            f'<url>\n{current_url}\n</url>\n<query>\n{query}\n</query>\n<result>\n{response.completion}\n</result>'
        )

        ## Simple memory handling
        MAX_MEMORY_LENGTH = 1000
        if len(extracted_content) < MAX_MEMORY_LENGTH:
            memory = extracted_content
            include_extracted_content_only_once = False
        else:
            file_name = await self.file_system.save_extracted_content(extracted_content)
            memory = f'Query: {query}\nContent in {file_name} and once in <read_state>.'
            include_extracted_content_only_once = True

        self.logger.info(f'AI Step: {memory}')
        return ActionResult(
            extracted_content=extracted_content,
            include_extracted_content_only_once=include_extracted_content_only_once,
            long_term_memory=memory,
        )
    except Exception as e:
        self.logger.warning(f'Failed to execute AI step: {e.__class__.__name__}: {e}')
        self.logger.debug('Full error traceback:', exc_info=True)
        return ActionResult(error=f'AI step failed: {e}')

    async def rerun_history(
        self,
        history: AgentHistoryList,
        max_retries: int = 3,
        skip_failures: bool = False,
        delay_between_actions: float = 2.0,
        max_step_interval: float = 45.0,
        summary_llm: BaseChatModel | None = None,
        ai_step_llm: BaseChatModel | None = None,
        wait_for_elements: bool = False,
    ) -> list[ActionResult]:
        """
        Rerun a saved history of actions with error handling and retry logic.

        Args:
            history: The history to replay
            max_retries: Maximum number of retries per action
            skip_failures: Whether to skip failed actions or stop execution. When True, also skip
                steps that had errors in the original run (e.g., modal close buttons that
                auto-dismissed, or elements that became non-interactable)
            delay_between_actions: Delay between actions in seconds (used when no saved interval)
            max_step_interval: Maximum delay from saved step_interval (caps LLM time from original)
            summary_llm: Optional LLM to use for generating the final summary. If not provided, use
            ai_step_llm: Optional LLM to use for AI steps (extract actions). If not provided, use
            wait_for_elements: If True, wait for minimum number of elements before attempting element
                matching. Useful for SPA pages where shadow DOM content loads dynamically.
                Default is False.

        Returns:

```

```

        List of action results (including AI summary as the final result)
        """
        # Skip cloud sync session events for rerunning (we're replaying, not starting new)
        self.state.session_initialized = True

        # Initialize browser session
        await self.browser_session.start()

        results = []

        # Track previous step for redundant retry detection
        previous_item: AgentHistory | None = None
        previous_step_succeeded: bool = False

        try:
            for i, history_item in enumerate(history.history):
                goal = history_item.model_output.current_state.next_goal if history_item.model_output else ''
                step_num = history_item.metadata.step_number if history_item.metadata else i
                step_name = 'Initial actions' if step_num == 0 else f'Step {step_num}'

                # Determine step delay
                if history_item.metadata and history_item.metadata.step_interval is not None:
                    # Cap the saved interval to max_step_interval (saved interval includes LLM time)
                    step_delay = min(history_item.metadata.step_interval, max_step_interval)
                    # Format delay nicely - show ms for values < 1s, otherwise show seconds
                    if step_delay < 1.0:
                        delay_str = f'{step_delay * 1000:.0f}ms'
                    else:
                        delay_str = f'{step_delay:.1f}s'
                    if history_item.metadata.step_interval > max_step_interval:
                        delay_source = f'capped to {delay_str} (saved was {history_item.metadata.step_interval:.1f}s)'
                    else:
                        delay_source = f'using saved step_interval={delay_str}'
                else:
                    step_delay = delay_between_actions
                    if step_delay < 1.0:
                        delay_str = f'{step_delay * 1000:.0f}ms'
                    else:
                        delay_str = f'{step_delay:.1f}s'
                    delay_source = f'using default delay={delay_str}'

                self.logger.info(f'Replaying {step_name} ({i + 1}/{len(history.history)}) [{delay_source}]: {goal}')

                if (
                    not history_item.model_output
                    or not history_item.model_output.action
                    or history_item.model_output.action == [None]
                ):
                    self.logger.warning(f'{step_name}: No action to replay, skipping')
                    results.append(ActionResult(error='No action to replay'))
                    continue

                # Check if the original step had errors - skip if skip_failures is enabled
                original_had_error = any(r.error for r in history_item.result if r.error)
                if original_had_error and skip_failures:
                    error_msgs = [r.error for r in history_item.result if r.error]
                    self.logger.warning(
                        f'{step_name}: Original step had error(s), skipping (skip_failures=True): {error_msgs[0][:100] if error_msgs else ""}'
                    )
                    results.append(
                        ActionResult(
                            error=f'Skipped - original step had error: {error_msgs[0][:100] if error_msgs else "unknown"}'
                        )
                    )
                    continue

                # Check if this step is a redundant retry of the previous step
                # This handles cases where original run needed to click same element multiple times
                # due to slow page response, but during replay the first click already worked
                if self._is_redundant_retry_step(history_item, previous_item, previous_step_succeeded):
                    self.logger.info(f'{step_name}: Skipping redundant retry (previous step already succeeded with same action)')
                    results.append(

```

```

#####ActionResult(
#####extracted_content='Skipped - redundant retry of previous step',
#####include_in_memory=False,
#####)
#####)
###### Don't update previous_item/previous_step_succeeded - keep tracking the original step
#####continue

#####retry_count = 0
#####step_succeeded = False
#####menu_reopened = False # Track if we've already tried reopening the menu
###### Exponential backoff: 5s base, doubling each retry, capped at 30s
#####base_retry_delay = 5.0
#####max_retry_delay = 30.0
#####while retry_count < max_retries:
#####try:
#####result = await self._execute_history_step(history_item, step_delay, ai_step_llm, wait_for_element)
#####results.extend(result)
#####step_succeeded = True
#####break

#####except Exception as e:
#####error_str = str(e)
#####retry_count += 1

###### Check if this is a "Could not find matching element" error for a menu item
###### If so, try to re-open the dropdown from the previous step before retrying
#####if (
#####not menu_reopened
#####and 'Could not find matching element' in error_str
#####and previous_item is not None
#####and self._is_menu_opener_step(previous_item)
#####):
###### Check if current step targets a menu item element
#####curr_elements = history_item.state.interacted_element if history_item.state else []
#####curr_elem = curr_elements[0] if curr_elements else None
#####if self._is_menu_item_element(curr_elem):
#####self.logger.info(
#####'■ Dropdown may have closed. Attempting to re-open by re-executing previous step...'
#####)
#####reopened = await self._reexecute_menu_opener(previous_item, ai_step_llm)
#####if reopened:
#####menu_reopened = True
###### Don't increment retry_count for the menu reopen attempt
###### Retry immediately with minimal delay
#####retry_count -= 1
#####step_delay = 0.5 # Use short delay after reopening
#####self.logger.info('■ Dropdown re-opened, retrying element match...')
#####continue

#####if retry_count == max_retries:
#####error_msg = f'{step_name} failed after {max_retries} attempts: {error_str}'
#####self.logger.error(error_msg)
###### Always record the error in results so AI summary counts it correctly
#####results.append(ActionResult(error=error_msg))
#####if not skip_failures:
#####raise RuntimeError(error_msg)
###### With skip_failures=True, continue to next step
#####else:
###### Exponential backoff: 5s, 10s, 20s, ... capped at 30s
#####retry_delay = min(base_retry_delay * (2 ** (retry_count - 1)), max_retry_delay)
#####self.logger.warning(
#####f'{step_name} failed (attempt {retry_count}/{max_retries}), retrying in {retry_delay}s...'
#####)
#####await asyncio.sleep(retry_delay)

###### Update tracking for redundant retry detection
#####previous_item = history_item
#####previous_step_succeeded = step_succeeded

###### Generate AI summary of rerun completion
#####self.logger.info('■ Generating AI summary of rerun completion...')

```

```

    summary_result = await self._generate_rerun_summary(self.task, results, summary_llm)
    results.append(summary_result)

    return results
    finally:
        # Always close resources, even on failure
        await self.close()

    async def _execute_initial_actions(self) -> None:
        # Execute initial actions if provided
        if self.initial_actions and not self.state.follow_up_task:
            self.logger.debug(f'■ Executing {len(self.initial_actions)} initial actions...')
            result = await self.multi_act(self.initial_actions)
            # update result 1 to mention that its was automatically loaded
            if result and self.initial_url and result[0].long_term_memory:
                result[0].long_term_memory = f'Found initial url and automatically loaded it. {result[0].long_term_memory}'
            self.state.last_result = result

        # Save initial actions to history as step 0 for rerun capability
        # Skip browser state capture for initial actions (usually just URL navigation)
        if self.settings.flash_mode:
            model_output = self.AgentOutput(
                evaluation_previous_goal=None,
                memory='Initial navigation',
                next_goal=None,
                action=self.initial_actions,
            )
        else:
            model_output = self.AgentOutput(
                evaluation_previous_goal='Start',
                memory=None,
                next_goal='Initial navigation',
                action=self.initial_actions,
            )

        metadata = StepMetadata(step_number=0, step_start_time=time.time(), step_end_time=time.time(), step_

        # Create minimal browser state history for initial actions
        state_history = BrowserStateHistory(
            url=self.initial_url or '',
            title='Initial Actions',
            tabs=[],
            interacted_element=[None] * len(self.initial_actions), # No DOM elements needed
            screenshot_path=None,
        )

        history_item = AgentHistory(
            model_output=model_output,
            result=result,
            state=state_history,
            metadata=metadata,
        )

        self.history.add_item(history_item)
        self.logger.debug('■ Saved initial actions to history as step 0')
        self.logger.debug('Initial actions completed')

    async def _wait_for_minimum_elements(
        self,
        min_elements: int,
        timeout: float = 30.0,
        poll_interval: float = 1.0,
    ) -> BrowserStateSummary | None:
        """Wait for the page to have at least min_elements interactive elements.

        This helps handle SPA pages where shadow DOM and dynamic content
        may not be immediately available even when document.readyState is 'complete'.

        Args:
            min_elements: Minimum number of interactive elements to wait for
            timeout: Maximum time to wait in seconds
            poll_interval: Time between polling attempts in seconds

```



```

    Returns:
    BrowserStateSummary if minimum elements found, None if timeout
    """
    assert self.browser_session is not None, 'BrowserSession is not set up'

    start_time = time.time()
    last_count = 0

    while (time.time() - start_time) < timeout:
        state = await self.browser_session.get_browser_state_summary(include_screenshot=False)
        if state and state.dom_state.selector_map:
            current_count = len(state.dom_state.selector_map)
            if current_count >= min_elements:
                self.logger.debug(f'Page has {current_count} elements (needed {min_elements}), proceeding with')
                return state
            if current_count != last_count:
                self.logger.debug(
                    f'Waiting for elements: {current_count}/{min_elements} '
                    f'(timeout in {timeout - (time.time() - start_time):.1f}s)'
                )
            last_count = current_count
            await asyncio.sleep(poll_interval)

    # Return last state even if we didn't reach min_elements
    self.logger.warning(f'Timeout waiting for {min_elements} elements, proceeding with {last_count} elements')
    return await self.browser_session.get_browser_state_summary(include_screenshot=False)

    def _count_expected_elements_from_history(self, history_item: AgentHistory) -> int:
        """Estimate the minimum number of elements expected based on history.

        Uses the action indices from the history to determine the minimum
        number of elements the page should have. If an action targets index N,
        the page needs at least N+1 elements in the selector_map.
        """
        if not history_item.model_output or not history_item.model_output.action:
            return 0

        max_index = -1 # Use -1 to indicate no index found yet
        for action in history_item.model_output.action:
            # Get the element index this action targets
            index = action.get_index()
            if index is not None:
                max_index = max(max_index, index)

        # Need at least max_index + 1 elements (indices are 0-based)
        # Cap at 50 to avoid waiting forever for very high indices
        # max_index >= 0 means we found at least one action with an index
        return min(max_index + 1, 50) if max_index >= 0 else 0

    async def _execute_history_step(
        self,
        history_item: AgentHistory,
        delay: float,
        ai_step_llm: BaseChatModel | None = None,
        wait_for_elements: bool = False,
    ) -> list[ActionResult]:
        """Execute a single step from history with element validation.

        For extract actions, uses AI to re-evaluate the content since page content may have changed.

        Args:
            history_item: The history step to execute
            delay: Delay before executing the step
            ai_step_llm: Optional LLM to use for AI steps
            wait_for_elements: If True, wait for minimum elements before element matching
        """
        assert self.browser_session is not None, 'BrowserSession is not set up'

        await asyncio.sleep(delay)

        # Optionally wait for minimum elements before element matching (useful for SPAs)
        if wait_for_elements:

```

```

##### Determine if we need to wait for elements (actions that interact with DOM elements)
#####needs_element_matching = False
#####if history_item.model_output:
#####for i, action in enumerate(history_item.model_output.action):
#####    action_data = action.model_dump(exclude_unset=True)
#####    action_name = next(iter(action_data.keys()), None)
#####    # Actions that need element matching
#####    if action_name in ('click', 'input', 'hover', 'select_option', 'drag_and_drop'):
#####        historical_elem = (
#####            history_item.state.interacted_element[i] if i < len(history_item.state.interacted_element) else
#####            )
#####        if historical_elem is not None:
#####            needs_element_matching = True
#####            break

##### If we need element matching, wait for minimum elements before proceeding
#####if needs_element_matching:
#####    min_elements = self._count_expected_elements_from_history(history_item)
#####    if min_elements > 0:
#####        state = await self._wait_for_minimum_elements(min_elements, timeout=15.0, poll_interval=1.0)
#####    else:
#####        state = await self.browser_session.get_browser_state_summary(include_screenshot=False)
#####    else:
#####        state = await self.browser_session.get_browser_state_summary(include_screenshot=False)
#####    else:
#####        state = await self.browser_session.get_browser_state_summary(include_screenshot=False)
#####    if not state or not history_item.model_output:
#####        raise ValueError('Invalid state or model output')

#####results = []
#####pending_actions = []

#####for i, action in enumerate(history_item.model_output.action):
#####    # Check if this is an extract action - use AI step instead
#####    action_data = action.model_dump(exclude_unset=True)
#####    action_name = next(iter(action_data.keys()), None)

#####    if action_name == 'extract':
#####        # Execute any pending actions first to maintain correct order
#####        # (e.g., if step is [click, extract], click must happen before extract)
#####        if pending_actions:
#####            batch_results = await self.multi_act(pending_actions)
#####            results.extend(batch_results)
#####            pending_actions = []

#####        # Now execute AI step for extract action
#####        extract_params = action_data['extract']
#####        query = extract_params.get('query', '')
#####        extract_links = extract_params.get('extract_links', False)

#####        self.logger.info(f'##### Using AI step for extract action: {query[:50]}...')
#####        ai_result = await self._execute_ai_step(
#####            query=query,
#####            include_screenshot=False, # Match original extract behavior
#####            extract_links=extract_links,
#####            ai_step_llm=ai_step_llm,
#####        )
#####        results.append(ai_result)
#####    else:
#####        # For non-extract actions, update indices and collect for batch execution
#####        historical_elem = history_item.state.interacted_element[i]
#####        updated_action = await self._update_action_indices(
#####            historical_elem,
#####            action,
#####            state,
#####        )
#####        if updated_action is None:
#####            # Build informative error message with diagnostic info
#####            elem_info = self._format_element_for_error(historical_elem)
#####            selector_map = state.dom_state.selector_map or {}
#####            selector_count = len(selector_map)
#####            # Find elements with same node_name for diagnostics

```

```

#####hist_node = historical_elem.node_name.lower() if historical_elem else ''
#####similar_elements = []
#####if historical_elem and historical_elem.attributes:
#####hist_aria = historical_elem.attributes.get('aria-label', '')
#####for idx, elem in selector_map.items():
#####if elem.node_name.lower() == hist_node and elem.attributes:
#####elem_aria = elem.attributes.get('aria-label', '')
#####if elem_aria:
#####similar_elements.append(f'{idx}:{elem_aria[:30]}')
#####if len(similar_elements) >= 5:
#####break

#####diagnostic = ''
#####if similar_elements:
#####diagnostic = f'\n Available <{hist_node.upper()}> with aria-label: {similar_elements}'
#####elif hist_node:
#####same_node_count = sum(1 for e in selector_map.values() if e.node_name.lower() == hist_node)
#####diagnostic = (
#####f'\n Found {same_node_count} <{hist_node.upper()}> elements (none with matching identifiers)'
#####)

#####raise ValueError(
#####f'Could not find matching element for action {i} in current page.\n'
#####f' Looking for: {elem_info}\n'
#####f' Page has {selector_count} interactive elements.{diagnostic}\n'
#####f' Tried: EXACT hash → STABLE hash → XPATH → AX_NAME → ATTRIBUTE matching'
#####)
#####pending_actions.append(updated_action)

###### Execute any remaining pending actions
#####if pending_actions:
#####batch_results = await self.multi_act(pending_actions)
#####results.extend(batch_results)

#####return results

#####async def _update_action_indices(
#####self,
#####historical_element: DOMInteractedElement | None,
#####action: ActionModel, # Type this properly based on your action model
#####browser_state_summary: BrowserStateSummary,
#####) -> ActionModel | None:
#####"""
#####Update action indices based on current page state.
#####Returns updated action or None if element cannot be found.

#####Cascading matching strategy (tries each level in order):
#####1. EXACT: Full element_hash match (includes all attributes + ax_name)
#####2. STABLE: Hash with dynamic CSS classes filtered out (focus, hover, animation, etc.)
#####3. XPATH: XPath string match (structural position in DOM)
#####4. AX_NAME: Accessible name match from accessibility tree (robust for dynamic menus)
#####5. ATTRIBUTE: Unique attribute match (name, id, aria-label) for old history files
#####"""
#####if not historical_element or not browser_state_summary.dom_state.selector_map:
#####return action

#####selector_map = browser_state_summary.dom_state.selector_map
#####highlight_index: int | None = None
#####match_level: MatchLevel | None = None

###### Debug: log what we're looking for and what's available
#####self.logger.info(
#####f'■ Searching for element: <{historical_element.node_name}> '
#####f'hash={historical_element.element_hash} stable_hash={historical_element.stable_hash}'
#####)
###### Log what elements are in selector_map for debugging
#####if historical_element.node_name:
#####hist_name = historical_element.node_name.lower()
#####matching_nodes = [
#####(idx, elem.node_name, elem.attributes.get('name') if elem.attributes else None)
#####for idx, elem in selector_map.items()
#####if elem.node_name.lower() == hist_name

```

```

    ]
    self.logger.info(
        f'Selector map has {len(selector_map)} elements, '
        f'{len(matching_nodes)} are <{hist_name.upper()}>: {matching_nodes}'
    )

    ## Level 1: EXACT hash match
    for idx, elem in selector_map.items():
        if elem.element_hash == historical_element.element_hash:
            highlight_index = idx
            match_level = MatchLevel.EXACT
            break

    if highlight_index is None:
        self.logger.debug(f'EXACT hash match failed (checked {len(selector_map)} elements)')

    ## Level 2: STABLE hash match (dynamic classes filtered)
    ## Use stored stable_hash (computed at save time from EnhancedDOMTreeNode - single source of truth)
    if highlight_index is None and historical_element.stable_hash is not None:
        for idx, elem in selector_map.items():
            if elem.compute_stable_hash() == historical_element.stable_hash:
                highlight_index = idx
                match_level = MatchLevel.STABLE
                self.logger.info('Element matched at STABLE level (dynamic classes filtered)')
                break
        if highlight_index is None:
            self.logger.debug('STABLE hash match failed')
        elif highlight_index is None:
            self.logger.debug('STABLE hash match skipped (no stable_hash in history)')

    ## Level 3: XPATH match
    if highlight_index is None and historical_element.x_path:
        for idx, elem in selector_map.items():
            if elem.xpath == historical_element.x_path:
                highlight_index = idx
                match_level = MatchLevel.XPATH
                self.logger.info(f'Element matched at XPATH level: {historical_element.x_path}')
                break
        if highlight_index is None:
            self.logger.debug(f'XPATH match failed for: {historical_element.x_path[-60:]}')

    ## Level 4: ax_name (accessible name) match - robust for dynamic SPAs with menus
    ## This uses the accessible name from the accessibility tree which is stable
    ## even when DOM structure changes (e.g., dynamically generated menu items)
    if highlight_index is None and historical_element.ax_name:
        hist_name = historical_element.node_name.lower()
        hist_ax_name = historical_element.ax_name
        for idx, elem in selector_map.items():
            ## Match by node type and accessible name
            elem_ax_name = elem.ax_node.name if elem.ax_node else None
            if elem.node_name.lower() == hist_name and elem_ax_name == hist_ax_name:
                highlight_index = idx
                match_level = MatchLevel.AX_NAME
                self.logger.info(f'Element matched at AX_NAME level: "{hist_ax_name}"')
                break
        if highlight_index is None:
            ## Log available ax_names for debugging
            same_type_ax_names = [
                (idx, elem.ax_node.name if elem.ax_node else None)
                for idx, elem in selector_map.items()
            ]
            if elem.node_name.lower() == hist_name and elem.ax_node and elem.ax_node.name
            self.logger.debug(
                f'AX_NAME match failed for <{hist_name.upper()}> ax_name="{hist_ax_name}". '
                f'Page has {len(same_type_ax_names)} <{hist_name.upper()}> with ax_names: '
                f'{same_type_ax_names[:5]}{"..." if len(same_type_ax_names) > 5 else ""}'
            )

    ## Level 5: Unique attribute fallback (for old history files without stable_hash)
    if highlight_index is None and historical_element.attributes:
        hist_attrs = historical_element.attributes
        hist_name = historical_element.node_name.lower()

```

```

##### Try matching by unique identifiers: name, id, or aria-label
##### for attr_key in ['name', 'id', 'aria-label']:
##### if attr_key in hist_attrs and hist_attrs[attr_key]:
##### for idx, elem in selector_map.items():
##### if (
##### elem.node_name.lower() == hist_name
##### and elem.attributes
##### and elem.attributes.get(attr_key) == hist_attrs[attr_key]
##### ):
##### highlight_index = idx
##### match_level = MatchLevel.ATTRIBUTE
##### self.logger.info(f'Element matched via {attr_key} attribute: {hist_attrs[attr_key]}')
##### break
##### if highlight_index is not None:
##### break

##### if highlight_index is None:
##### tried_attrs = [k for k in ['name', 'id', 'aria-label'] if k in hist_attrs and hist_attrs[k]]
##### # Log what was tried and what's available on the page for debugging
##### same_node_elements = [
##### (idx, elem.attributes.get('aria-label') or elem.attributes.get('id') or elem.attributes.get('name'))
##### for idx, elem in selector_map.items()
##### if elem.node_name.lower() == hist_name and elem.attributes
##### ]
##### self.logger.info(
##### f'ATTRIBUTE match failed for <{hist_name.upper()}> '
##### f'(tried: {tried_attrs}, looking for: {[hist_attrs.get(k) for k in tried_attrs]}). '
##### f'Page has {len(same_node_elements)} <{hist_name.upper()}> elements with identifiers: '
##### f'{same_node_elements[:5]}{"..." if len(same_node_elements) > 5 else ""}'
##### )

##### if highlight_index is None:
##### return None

##### old_index = action.get_index()
##### if old_index != highlight_index:
##### action.set_index(highlight_index)
##### level_name = match_level.name if match_level else 'UNKNOWN'
##### self.logger.info(f'Element index updated {old_index} → {highlight_index} (matched at {level_name})')

##### return action

##### def _format_element_for_error(self, elem: DOMInteractedElement | None) -> str:
##### """Format element info for error messages during history rerun."""
##### if elem is None:
##### return '<no element recorded>'

##### parts = [f'<{elem.node_name}>']

##### # Add key identifying attributes
##### if elem.attributes:
##### for key in ['name', 'id', 'aria-label', 'type']:
##### if key in elem.attributes and elem.attributes[key]:
##### parts.append(f'{key}="{elem.attributes[key]}"')

##### # Add hash info
##### parts.append(f'hash={elem.element_hash}')
##### if elem.stable_hash:
##### parts.append(f'stable_hash={elem.stable_hash}')

##### # Add xpath (truncated)
##### if elem.x_path:
##### xpath_short = elem.x_path if len(elem.x_path) <= 60 else f'...{elem.x_path[-57:]}'
##### parts.append(f'xpath="{xpath_short}"')

##### return ' '.join(parts)

##### def _is_redundant_retry_step(
##### self,
##### current_item: AgentHistory,
##### previous_item: AgentHistory | None,
##### previous_step_succeeded: bool,

```

```

    ■) -> bool:
    ■"""
    ■Detect if current step is a redundant retry of the previous step.

    ■This handles cases where the original run needed to click the same element multiple
    ■times due to slow page response, but during replay the first click already succeeded.
    ■When the page has already navigated, subsequent retry clicks on the same element
    ■would fail because that element no longer exists.

    ■Returns True if:
    ■- Previous step succeeded
    ■- Both steps target the same element (by element_hash, stable_hash, or xpath)
    ■- Both steps perform the same action type (e.g., both are clicks)
    ■"""
    ■if not previous_item or not previous_step_succeeded:
    ■    return False

    ■# Get interacted elements from both steps (first action in each)
    ■curr_elements = current_item.state.interacted_element
    ■prev_elements = previous_item.state.interacted_element

    ■if not curr_elements or not prev_elements:
    ■    return False

    ■curr_elem = curr_elements[0] if curr_elements else None
    ■prev_elem = prev_elements[0] if prev_elements else None

    ■if not curr_elem or not prev_elem:
    ■    return False

    ■# Check if same element by various matching strategies
    ■same_by_hash = curr_elem.element_hash == prev_elem.element_hash
    ■same_by_stable_hash = (
    ■    curr_elem.stable_hash is not None
    ■    and prev_elem.stable_hash is not None
    ■    and curr_elem.stable_hash == prev_elem.stable_hash
    ■)
    ■same_by_xpath = curr_elem.x_path == prev_elem.x_path

    ■if not (same_by_hash or same_by_stable_hash or same_by_xpath):
    ■    return False

    ■# Check if same action type
    ■curr_actions = current_item.model_output.action if current_item.model_output else []
    ■prev_actions = previous_item.model_output.action if previous_item.model_output else []

    ■if not curr_actions or not prev_actions:
    ■    return False

    ■# Get the action type (first key in the action dict)
    ■curr_action_data = curr_actions[0].model_dump(exclude_unset=True)
    ■prev_action_data = prev_actions[0].model_dump(exclude_unset=True)

    ■curr_action_type = next(iter(curr_action_data.keys()), None)
    ■prev_action_type = next(iter(prev_action_data.keys()), None)

    ■if curr_action_type != prev_action_type:
    ■    return False

    ■self.logger.debug(
    ■    f'■ Detected redundant retry: both steps target same element '
    ■    f'■<{curr_elem.node_name}> with action "{curr_action_type}"'
    ■)

    ■return True

    ■def _is_menu_opener_step(self, history_item: AgentHistory | None) -> bool:
    ■    """
    ■    Detect if a step opens a dropdown/menu.

    ■    Checks for common patterns indicating a menu opener:
    ■    - Element has aria-haspopup attribute

```

```

    ■■- Element has data-gw-click="toggleSubMenu" (Guidewire pattern)
    ■■- Element has expand-button in class name
    ■■- Element role is "menuitem" with aria-expanded

    ■■Returns True if the step appears to open a dropdown/submenu.
    ■■"""
    ■■if not history_item or not history_item.state or not history_item.state.interacted_element:
    ■■■return False

    ■■elem = history_item.state.interacted_element[0] if history_item.state.interacted_element else None
    ■■if not elem:
    ■■■return False

    ■■attrs = elem.attributes or {}

    ■■# Check for common menu opener indicators
    ■■if attrs.get('aria-haspopup') in ('true', 'menu', 'listbox'):
    ■■■return True
    ■■if attrs.get('data-gw-click') == 'toggleSubMenu':
    ■■■return True
    ■■if 'expand-button' in attrs.get('class', ''):
    ■■■return True
    ■■if attrs.get('role') == 'menuitem' and attrs.get('aria-expanded') in ('false', 'true'):
    ■■■return True
    ■■if attrs.get('role') == 'button' and attrs.get('aria-expanded') in ('false', 'true'):
    ■■■return True

    ■■return False

    ■■def _is_menu_item_element(self, elem: 'DOMInteractedElement | None') -> bool:
    ■■"""
    ■■Detect if an element is a menu item that appears inside a dropdown/menu.

    ■■Checks for:
    ■■- role="menuitem", "option", "menuitemcheckbox", "menuitemradio"
    ■■- Element is inside a menu structure (has menu-related parent indicators)
    ■■- ax_name is set (menu items typically have accessible names)

    ■■Returns True if the element appears to be a menu item.
    ■■"""
    ■■if not elem:
    ■■■return False

    ■■attrs = elem.attributes or {}

    ■■# Check for menu item roles
    ■■role = attrs.get('role', '')
    ■■if role in ('menuitem', 'option', 'menuitemcheckbox', 'menuitemradio', 'treeitem'):
    ■■■return True

    ■■# Elements in Guidewire menus have these patterns
    ■■if 'gw-action--inner' in attrs.get('class', ''):
    ■■■return True
    ■■if 'menuitem' in attrs.get('class', '').lower():
    ■■■return True

    ■■# If element has an ax_name and looks like it could be in a menu
    ■■# This is a softer check - only used if the previous step was a menu opener
    ■■if elem.ax_name and elem.ax_name not in ('', None):
    ■■# Common menu container classes
    ■■elem_class = attrs.get('class', '').lower()
    ■■if any(x in elem_class for x in ['dropdown', 'popup', 'menu', 'submenu', 'action']):
    ■■■return True

    ■■return False

    ■■async def _reexecute_menu_opener(
    ■■self,
    ■■opener_item: AgentHistory,
    ■■ai_step_llm: 'BaseChatModel | None' = None,
    ■■) -> bool:
    ■■"""

```

```

    ■■Re-execute a menu opener step to re-open a closed dropdown.

    ■■This is used when a menu item can't be found because the dropdown
    ■■closed during the wait between steps.

    ■■Returns True if re-execution succeeded, False otherwise.
    ■■"""
    ■■try:
    ■■■self.logger.info('■ Re-opening dropdown/menu by re-executing previous step...')
    ■■■# Use a minimal delay - we want to quickly re-open the menu
    ■■■await self._execute_history_step(opener_item, delay=0.5, ai_step_llm=ai_step_llm, wait_for_elements=
    ■■■# Small delay to let the menu render
    ■■■await asyncio.sleep(0.3)
    ■■■return True
    ■■except Exception as e:
    ■■■self.logger.warning(f'Failed to re-open dropdown: {e}')
    ■■■return False

    ■■async def load_and_rerun(
    ■■self,
    ■■history_file: str | Path | None = None,
    ■■variables: dict[str, str] | None = None,
    ■■**kwargs,
    ■■) -> list[ActionResult]:
    ■■"""
    ■■Load history from file and rerun it, optionally substituting variables.

    ■■Args:
    ■■■history_file: Path to the history file
    ■■■variables: Optional dict mapping variable names to new values (e.g. {'email': 'new@example.com'})
    ■■■**kwargs: Additional arguments passed to rerun_history:
    ■■■- max_retries: Maximum retries per action (default: 3)
    ■■■- skip_failures: Continue on failure (default: True)
    ■■■- delay_between_actions: Delay when no saved interval (default: 2.0s)
    ■■■- max_step_interval: Cap on saved step_interval (default: 45.0s)
    ■■■- summary_llm: Custom LLM for final summary
    ■■■- ai_step_llm: Custom LLM for extract re-evaluation
    ■■"""
    ■■if not history_file:
    ■■■history_file = 'AgentHistory.json'
    ■■history = AgentHistoryList.load_from_file(history_file, self.AgentOutput)

    ■■# Substitute variables if provided
    ■■if variables:
    ■■■history = self._substitute_variables_in_history(history, variables)

    ■■return await self.rerun_history(history, **kwargs)

    ■■def save_history(self, file_path: str | Path | None = None) -> None:
    ■■"""Save the history to a file with sensitive data filtering"""
    ■■if not file_path:
    ■■■file_path = 'AgentHistory.json'
    ■■self.history.save_to_file(file_path, sensitive_data=self.sensitive_data)

    ■■def pause(self) -> None:
    ■■"""Pause the agent before the next step"""
    ■■print('\n\n■■ Paused the agent and left the browser open.\n\tPress [Enter] to resume or [Ctrl+C] again')
    ■■self.state.paused = True
    ■■self._external_pause_event.clear()

    ■■def resume(self) -> None:
    ■■"""Resume the agent"""
    ■■# TODO: Locally the browser got closed
    ■■print('-----')
    ■■print('■■ Resuming agent execution where it left off...\n')
    ■■self.state.paused = False
    ■■self._external_pause_event.set()

    ■■def stop(self) -> None:
    ■■"""Stop the agent"""
    ■■self.logger.info('■■ Agent stopping')
    ■■self.state.stopped = True

```



```

    ## Signal pause event to unblock any waiting code so it can check the stopped state
    self._external_pause_event.set()

    ## Task stopped

    def _convert_initial_actions(self, actions: list[dict[str, dict[str, Any]]]) -> list[ActionModel]:
        """Convert dictionary-based actions to ActionModel instances"""
        converted_actions = []
        action_model = self.ActionModel
        for action_dict in actions:
            ## Each action_dict should have a single key-value pair
            action_name = next(iter(action_dict))
            params = action_dict[action_name]

            ## Get the parameter model for this action from registry
            action_info = self.tools.registry.actions[action_name]
            param_model = action_info.param_model

            ## Create validated parameters using the appropriate param model
            validated_params = param_model(**params)

            ## Create ActionModel instance with the validated parameters
            action_model = self.ActionModel(**{action_name: validated_params})
            converted_actions.append(action_model)

        return converted_actions

    def _verify_and_setup_llm(self):
        """
        Verify that the LLM API keys are setup and the LLM API is responding properly.
        Also handles tool calling method detection if in auto mode.
        """

        ## Skip verification if already done
        if getattr(self.llm, '_verified_api_keys', None) is True or CONFIG.SKIP_LLM_API_KEY_VERIFICATION:
            setattr(self.llm, '_verified_api_keys', True)
            return True

    @property
    def message_manager(self) -> MessageManager:
        return self._message_manager

    async def close(self):
        """Close all resources"""
        try:
            ## Only close browser if it wasn't injected and keep_alive is False (or not set)
            if self.browser_session is not None and not self.injected_browser_session:
                if not self.browser_session.browser_profile.keep_alive:
                    ## Kill the browser session - this dispatches BrowserStopEvent,
                    ## stops the EventBus with clear=True, and recreates a fresh EventBus
                    await self.browser_session.kill()
                else:
                    ## keep_alive=True sessions shouldn't keep the event loop alive after agent.run()
                    await self.browser_session.event_bus.stop(
                        clear=False,
                        timeout=_get_timeout('TIMEOUT_BrowserSessionEventBusStopOnAgentClose', 1.0),
                    )
                try:
                    self.browser_session.event_bus.event_queue = None
                    self.browser_session.event_bus._on_idle = None
                except Exception:
                    pass

            ## Close skill service if configured
            if self.skill_service is not None:
                await self.skill_service.close()

            ## Force garbage collection
            gc.collect()

            ## Debug: Log remaining threads and asyncio tasks
            import threading

```

```

    """threads = threading.enumerate()
    """self.logger.debug(f'■ Remaining threads ({len(threads)}): {[t.name for t in threads]}')

    """# Get all asyncio tasks
    """tasks = asyncio.all_tasks(asyncio.get_event_loop())
    """# Filter out the current task (this close() coroutine)
    """other_tasks = [t for t in tasks if t != asyncio.current_task()]
    """if other_tasks:
    """self.logger.debug(f'■ Remaining asyncio tasks ({len(other_tasks)}):')
    """for task in other_tasks[:10]: # Limit to first 10 to avoid spam
    """self.logger.debug(f' - {task.get_name(): {task}}')

    """except Exception as e:
    """self.logger.error(f'Error during cleanup: {e}')

    """async def _update_action_models_for_page(self, page_url: str) -> None:
    """"""Update action models with page-specific actions"""
    """# Create new action model with current page's filtered actions
    """self.ActionModel = self.tools.registry.create_action_model(page_url=page_url)
    """# Update output model with the new actions
    """if self.settings.flash_mode:
    """self.AgentOutput = AgentOutput.type_with_custom_actions_flash_mode(self.ActionModel)
    """elif self.settings.use_thinking:
    """self.AgentOutput = AgentOutput.type_with_custom_actions(self.ActionModel)
    """else:
    """self.AgentOutput = AgentOutput.type_with_custom_actions_no_thinking(self.ActionModel)

    """# Update done action model too
    """self.DoneActionModel = self.tools.registry.create_action_model(include_actions=['done'], page_url=page_url)
    """if self.settings.flash_mode:
    """self.DoneAgentOutput = AgentOutput.type_with_custom_actions_flash_mode(self.DoneActionModel)
    """elif self.settings.use_thinking:
    """self.DoneAgentOutput = AgentOutput.type_with_custom_actions(self.DoneActionModel)
    """else:
    """self.DoneAgentOutput = AgentOutput.type_with_custom_actions_no_thinking(self.DoneActionModel)

    """async def authenticate_cloud_sync(self, show_instructions: bool = True) -> bool:
    """"""
    """Authenticate with cloud service for future runs.

    """This is useful when users want to authenticate after a task has completed
    """so that future runs will sync to the cloud.

    """Args:
    """show_instructions: Whether to show authentication instructions to user

    """Returns:
    """bool: True if authentication was successful
    """"""
    """self.logger.warning('Cloud sync has been removed and is no longer available')
    """return False

    """def run_sync(
    """self,
    """max_steps: int = 500,
    """on_step_start: AgentHookFunc | None = None,
    """on_step_end: AgentHookFunc | None = None,
    """) -> AgentHistoryList[AgentStructuredOutput]:
    """"""Synchronous wrapper around the async run method for easier usage without asyncio."""
    """import asyncio

    """return asyncio.run(self.run(max_steps=max_steps, on_step_start=on_step_start, on_step_end=on_step_end))

    """def detect_variables(self) -> dict[str, DetectedVariable]:
    """"""Detect reusable variables in agent history"""
    """from system.agent.variable_detector import detect_variables_in_history

    """return detect_variables_in_history(self.history)

    """def _substitute_variables_in_history(self, history: AgentHistoryList, variables: dict[str, str]) -> AgentHistoryList:
    """"""Substitute variables in history with new values for rerunning with different data"""
    """from system.agent.variable_detector import detect_variables_in_history

```

```

    ## Detect variables in the history
    detected_vars = detect_variables_in_history(history)

    ## Build a mapping of original values to new values
    value_replacements: dict[str, str] = {}
    for var_name, new_value in variables.items():
        if var_name in detected_vars:
            old_value = detected_vars[var_name].original_value
            value_replacements[old_value] = new_value
        else:
            self.logger.warning(f'Variable "{var_name}" not found in history, skipping substitution')

    if not value_replacements:
        self.logger.info('No variables to substitute')
        return history

    ## Create a deep copy of history to avoid modifying the original
    import copy

    modified_history = copy.deepcopy(history)

    ## Substitute values in all actions
    substitution_count = 0
    for history_item in modified_history.history:
        if not history_item.model_output or not history_item.model_output.action:
            continue

        for action in history_item.model_output.action:
            ## Handle both Pydantic models and dicts
            if hasattr(action, 'model_dump'):
                action_dict = action.model_dump()
            elif isinstance(action, dict):
                action_dict = action
            else:
                action_dict = vars(action) if hasattr(action, '__dict__') else {}

            ## Substitute in all string fields
            substitution_count += self._substitute_in_dict(action_dict, value_replacements)

            ## Update the action with modified values
            if hasattr(action, 'model_dump'):
                ## For Pydantic RootModel, we need to recreate from the modified dict
                if hasattr(action, 'root'):
                    ## This is a RootModel - recreate it from the modified dict
                    new_action = type(action).model_validate(action_dict)
                    ## Replace the root field in-place using object.__setattr__ to bypass Pydantic's immutability
                    object.__setattr__(action, 'root', getattr(new_action, 'root'))
                else:
                    ## Regular Pydantic model - update fields in-place
                    for key, val in action_dict.items():
                        if hasattr(action, key):
                            setattr(action, key, val)
                        elif isinstance(action, dict):
                            action.update(action_dict)

            self.logger.info(f'Substituted {substitution_count} value(s) in {len(value_replacements)} variable type')
        return modified_history

    def _substitute_in_dict(self, data: dict, replacements: dict[str, str]) -> int:
        """Recursively substitute values in a dictionary, returns count of substitutions made"""
        count = 0
        for key, value in data.items():
            if isinstance(value, str):
                ## Replace if exact match
                if value in replacements:
                    data[key] = replacements[value]
                    count += 1
            elif isinstance(value, dict):
                ## Recurse into nested dicts
                count += self._substitute_in_dict(value, replacements)
            elif isinstance(value, list):
                ## Handle lists

```

```

        for i, item in enumerate(value):
            if isinstance(item, str) and item in replacements:
                value[i] = replacements[item]
                count += 1
            elif isinstance(item, dict):
                count += self._substitute_in_dict(item, replacements)
        return count

```

.\system\agent\variable_detector.py

```

"""Detect variables in agent history for reuse"""

import re

from system.agent.views import AgentHistoryList, DetectedVariable
from system.dom.views import DOMInteractedElement

def detect_variables_in_history(history: AgentHistoryList) -> dict[str, DetectedVariable]:
    """
    ■Analyze agent history and detect reusable variables.

    ■Uses two strategies:
    ■1. Element attributes (id, name, type, placeholder, aria-label) - most reliable
    ■2. Value pattern matching (email, phone, date formats) - fallback

    ■Returns:
    ■Dictionary mapping variable names to DetectedVariable objects
    """
    detected: dict[str, DetectedVariable] = {}
    detected_values: set[str] = set() # Track which values we've already detected

    for step_idx, history_item in enumerate(history.history):
        if not history_item.model_output:
            continue

        for action_idx, action in enumerate(history_item.model_output.action):
            # Convert action to dict - handle both Pydantic models and dict-like objects
            if hasattr(action, 'model_dump'):
                action_dict = action.model_dump()
            elif isinstance(action, dict):
                action_dict = action
            else:
                # For SimpleNamespace or similar objects
                action_dict = vars(action)

            # Get the interacted element for this action (if available)
            element = None
            if history_item.state and history_item.state.interacted_element:
                if len(history_item.state.interacted_element) > action_idx:
                    element = history_item.state.interacted_element[action_idx]

            # Detect variables in this action
            _detect_in_action(action_dict, element, detected, detected_values)

    return detected

def _detect_in_action(
    action_dict: dict,
    element: DOMInteractedElement | None,
    detected: dict[str, DetectedVariable],
    detected_values: set[str],
) -> None:
    """Detect variables in a single action using element context"""

    # Extract action type and parameters
    for action_type, params in action_dict.items():

```

```

    if not isinstance(params, dict):
        continue

    # Check fields that commonly contain variables
    fields_to_check = ['text', 'query']

    for field in fields_to_check:
        if field not in params:
            continue

        value = params[field]
        if not isinstance(value, str) or not value.strip():
            continue

        # Skip if we already detected this exact value
        if value in detected_values:
            continue

        # Try to detect variable type (with element context)
        var_info = _detect_variable_type(value, element)
        if not var_info:
            continue

        var_name, var_format = var_info

        # Ensure unique variable name
        var_name = _ensure_unique_name(var_name, detected)

        # Add detected variable
        detected[var_name] = DetectedVariable(
            name=var_name,
            original_value=value,
            type='string',
            format=var_format,
        )

        detected_values.add(value)

def _detect_variable_type(
    value: str,
    element: DOMInteractedElement | None = None,
) -> tuple[str, str | None] | None:
    """
    Detect if a value looks like a variable, using element context when available.

    Priority:
    1. Element attributes (id, name, type, placeholder, aria-label) - most reliable
    2. Value pattern matching (email, phone, date formats) - fallback

    Returns:
    (variable_name, format) or None if not detected
    """

    # STRATEGY 1: Use element attributes (most reliable)
    if element and element.attributes:
        attr_detection = _detect_from_attributes(element.attributes)
        if attr_detection:
            return attr_detection

    # STRATEGY 2: Pattern matching on value (fallback)
    return _detect_from_value_pattern(value)

def _detect_from_attributes(attributes: dict[str, str]) -> tuple[str, str | None] | None:
    """
    Detect variable from element attributes.

    Check attributes in priority order:
    1. type attribute (HTML5 input types - most specific)
    2. id, name, placeholder, aria-label (semantic hints)
    """

```

```

■# Check 'type' attribute first (HTML5 input types)
■input_type = attributes.get('type', '').lower()
■if input_type == 'email':
■    return ('email', 'email')
■elif input_type == 'tel':
■    return ('phone', 'phone')
■elif input_type == 'date':
■    return ('date', 'date')
■elif input_type == 'number':
■    return ('number', 'number')
■elif input_type == 'url':
■    return ('url', 'url')

■# Combine semantic attributes for keyword matching
■semantic_attrs = [
■    attributes.get('id', ''),
■    attributes.get('name', ''),
■    attributes.get('placeholder', ''),
■    attributes.get('aria-label', ''),
■]

■combined_text = ' '.join(semantic_attrs).lower()

■# Address detection
■if any(keyword in combined_text for keyword in ['address', 'street', 'addr']):
■    if 'billing' in combined_text:
■        return ('billing_address', None)
■    elif 'shipping' in combined_text:
■        return ('shipping_address', None)
■    else:
■        return ('address', None)

■# Comment/Note detection
■if any(keyword in combined_text for keyword in ['comment', 'note', 'message', 'description']):
■    return ('comment', None)

■# Email detection
■if 'email' in combined_text or 'e-mail' in combined_text:
■    return ('email', 'email')

■# Phone detection
■if any(keyword in combined_text for keyword in ['phone', 'tel', 'mobile', 'cell']):
■    return ('phone', 'phone')

■# Name detection (order matters - check specific before general)
■if 'first' in combined_text and 'name' in combined_text:
■    return ('first_name', None)
■elif 'last' in combined_text and 'name' in combined_text:
■    return ('last_name', None)
■elif 'full' in combined_text and 'name' in combined_text:
■    return ('full_name', None)
■elif 'name' in combined_text:
■    return ('name', None)

■# Date detection
■if any(keyword in combined_text for keyword in ['date', 'dob', 'birth']):
■    return ('date', 'date')

■# City detection
■if 'city' in combined_text:
■    return ('city', None)

■# State/Province detection
■if 'state' in combined_text or 'province' in combined_text:
■    return ('state', None)

■# Country detection
■if 'country' in combined_text:
■    return ('country', None)

■# Zip code detection
■if any(keyword in combined_text for keyword in ['zip', 'postal', 'postcode']):

```

```

    return ('zip_code', 'postal_code')

    # Company detection
    if 'company' in combined_text or 'organization' in combined_text:
        return ('company', None)

    return None

def _detect_from_value_pattern(value: str) -> tuple[str, str | None] | None:
    """
    Detect variable type from value pattern (fallback when no element context).

    Patterns:
    - Email: contains @ and . with valid format
    - Phone: digits with separators, 10+ chars
    - Date: YYYY-MM-DD format
    - Name: Capitalized word(s), 2-30 chars, letters only
    - Number: Pure digits, 1-9 chars
    """

    # Email detection - most specific first
    if '@' in value and '.' in value:
        # Basic email validation
        if re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', value):
            return ('email', 'email')

    # Phone detection (digits with separators, 10+ chars)
    if re.match(r'^[\d\s\-\(\)\+]+$ ', value):
        # Remove separators and check length
        digits_only = re.sub(r'[\s\-\(\)\+]', '', value)
        if len(digits_only) >= 10:
            return ('phone', 'phone')

    # Date detection (YYYY-MM-DD or similar)
    if re.match(r'^\d{4}-\d{2}-\d{2}$', value):
        return ('date', 'date')

    # Name detection (capitalized, only letters/spaces, 2-30 chars)
    if value and value[0].isupper() and value.replace(' ', '').replace('-', '').isalpha() and 2 <= len(value):
        words = value.split()
        if len(words) == 1:
            return ('first_name', None)
        elif len(words) == 2:
            return ('full_name', None)
        else:
            return ('name', None)

    # Number detection (pure digits, not phone length)
    if value.isdigit() and 1 <= len(value) <= 9:
        return ('number', 'number')

    return None

def _ensure_unique_name(base_name: str, existing: dict[str, DetectedVariable]) -> str:
    """
    Ensure variable name is unique by adding suffix if needed.

    Examples:
    first_name -> first_name
    first_name (exists) -> first_name_2
    first_name_2 (exists) -> first_name_3
    """
    if base_name not in existing:
        return base_name

    # Add numeric suffix
    counter = 2
    while f'{base_name}_{counter}' in existing:
        counter += 1
    return f'{base_name}_{counter}'

```

.lsystemagentviews.py

```
from __future__ import annotations

import hashlib
import json
import logging
import re
import traceback
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Generic, Literal

from pydantic import BaseModel, ConfigDict, Field, ValidationError, create_model, model_validator
from typing_extensions import import TypeVar
from uuid_extensions import uuid7str

from system.agent.message_manager.views import MessageManagerState
from system.browser.views import BrowserStateHistory
from system.dom.views import DEFAULT_INCLUDE_ATTRIBUTES, DOMInteractedElement, DOMSelectorMap

# from system.dom.history_tree_processor.service import (
#     # DOMElementNode,
#     # DOMHistoryElement,
#     # HistoryTreeProcessor,
# )
# from system.dom.views import SelectorMap
from system.filesystem.file_system import FileSystemState
from system.llm.base import BaseChatModel
from system.tokens.views import UsageSummary
from system.tools.registry.views import ActionModel

logger = logging.getLogger(__name__)

class MessageCompactionSettings(BaseModel):
    """Summarizes older history into a compact memory block to reduce prompt size."""

    enabled: bool = True
    compact_every_n_steps: int = 15
    trigger_char_count: int | None = None # Min char floor; set via trigger_token_count if preferred
    trigger_token_count: int | None = None # Alternative to trigger_char_count (~4 chars/token)
    chars_per_token: float = 4.0
    keep_last_items: int = 6
    summary_max_chars: int = 6000
    include_read_state: bool = False
    compaction_llm: BaseChatModel | None = None

    @model_validator(mode='after')
    def _resolve_trigger_threshold(self) -> MessageCompactionSettings:
        if self.trigger_char_count is not None and self.trigger_token_count is not None:
            raise ValueError('Set trigger_char_count or trigger_token_count, not both.')
        if self.trigger_token_count is not None:
            self.trigger_char_count = int(self.trigger_token_count * self.chars_per_token)
        elif self.trigger_char_count is None:
            self.trigger_char_count = 40000 # ~10k tokens
        return self

class AgentSettings(BaseModel):
    """Configuration options for the Agent"""

    use_vision: bool | Literal['auto'] = True
    vision_detail_level: Literal['auto', 'low', 'high'] = 'auto'
    save_conversation_path: str | Path | None = None
    save_conversation_path_encoding: str | None = 'utf-8'
    max_failures: int = 5
```



```

■generate_gif: bool | str = False
■override_system_message: str | None = None
■extend_system_message: str | None = None
■include_attributes: list[str] | None = DEFAULT_INCLUDE_ATTRIBUTES
■max_actions_per_step: int = 5
■use_thinking: bool = True
■flash_mode: bool = False # If enabled, disables evaluation_previous_goal and next_goal, and sets use_t
■use_judge: bool = True
■ground_truth: str | None = None # Ground truth answer or criteria for judge validation
■max_history_items: int | None = None
■message_compaction: MessageCompactionSettings | None = None
■enable_planning: bool = True
■planning_replan_on_stall: int = 3 # consecutive failures before replan nudge; 0 = disabled
■planning_exploration_limit: int = 5 # steps without a plan before nudge; 0 = disabled

■page_extraction_llm: BaseChatModel | None = None
■calculate_cost: bool = False
■include_tool_call_examples: bool = False
■llm_timeout: int = 60 # Timeout in seconds for LLM calls (auto-detected: 30s for gemini, 90s for o3, 6
■step_timeout: int = 180 # Timeout in seconds for each step
■final_response_after_failure: bool = True # If True, attempt one final recovery call after max_failure

■# Loop detection settings
■loop_detection_window: int = 20 # Rolling window size for action similarity tracking
■loop_detection_enabled: bool = True # Whether to enable loop detection nudges
■max_clickable_elements_length: int = 40000 # Max characters for clickable elements in prompt

class PageFingerprint(BaseModel):
    ■"""Lightweight fingerprint of the browser page state."""

    ■model_config = ConfigDict(frozen=True)

    ■url: str
    ■element_count: int
    ■text_hash: str # First 16 chars of SHA-256 of the DOM text representation

    ■@staticmethod
    ■def from_browser_state(url: str, dom_text: str, element_count: int) -> PageFingerprint:
        ■text_hash = hashlib.sha256(dom_text.encode('utf-8', errors='replace')).hexdigest()[:16]
        ■return PageFingerprint(url=url, element_count=element_count, text_hash=text_hash)

    def _normalize_action_for_hash(action_name: str, params: dict[str, Any]) -> str:
        ■"""Normalize action parameters for similarity hashing.

        ■For search actions: strip minor keyword variations by sorting tokens.
        ■For click actions: hash by element type + rough text content, ignoring index.
        ■For navigate: hash by URL domain only.
        ■For others: hash by action_name + sorted params.
        ■"""
        ■if action_name == 'search':
            ■query = str(params.get('query', ''))
            ■# Normalize search: lowercase, sort tokens, collapse whitespace
            ■tokens = sorted(set(re.sub(r'[^\w\s]', ' ', query.lower()).split()))
            ■engine = params.get('engine', 'google')
            ■return f'search|{engine}|{"|".join(tokens)}'

        ■if action_name in ('click', 'input'):
            ■# For element-interaction actions, we only use the index (element identity).
            ■# Two clicks on the same element index are the same action.
            ■index = params.get('index')
            ■if action_name == 'input':
                ■text = str(params.get('text', ''))
                ■# Normalize input text: lowercase, strip whitespace
                ■return f'input|{index}|{text.strip().lower()}'
            ■return f'click|{index}'

        ■if action_name == 'navigate':
            ■url = str(params.get('url', ''))
            ■# Hash by full URL – navigating to different paths is genuine exploration,
            ■# only repeated navigation to the exact same URL is a loop signal.

```

```

    return f'navigate|{url}'

    if action_name == 'scroll':
        direction = 'down' if params.get('down', True) else 'up'
        index = params.get('index')
        return f'scroll|{direction}|{index}'

    # Default: hash by action name + sorted params (excluding None values)
    filtered = {k: v for k, v in sorted(params.items()) if v is not None}
    return f'{action_name}|{json.dumps(filtered, sort_keys=True, default=str)}'

def compute_action_hash(action_name: str, params: dict[str, Any]) -> str:
    """Compute a stable hash string for an action based on type + normalized parameters."""
    normalized = _normalize_action_for_hash(action_name, params)
    return hashlib.sha256(normalized.encode('utf-8')).hexdigest()[:12]

class ActionLoopDetector(BaseModel):
    """Tracks action repetition and page stagnation to detect behavioral loops.

    This is a soft detection system – it generates context messages for the LLM
    but never blocks actions. The agent can still repeat if it wants to.
    """

    model_config = ConfigDict(arbitrary_types_allowed=True)

    # Rolling window of recent action hashes
    window_size: int = 20
    recent_action_hashes: list[str] = Field(default_factory=list)

    # Page fingerprint tracking for stagnation detection
    recent_page_fingerprints: list[PageFingerprint] = Field(default_factory=list)

    # Current repetition state
    max_repetition_count: int = 0 # Highest count of any single hash in the window
    most_repeated_hash: str | None = None
    consecutive_stagnant_pages: int = 0 # How many consecutive steps had the same page fingerprint

    def record_action(self, action_name: str, params: dict[str, Any]) -> None:
        """Record an action and update repetition statistics."""
        h = compute_action_hash(action_name, params)
        self.recent_action_hashes.append(h)
        # Trim to window size
        if len(self.recent_action_hashes) > self.window_size:
            self.recent_action_hashes = self.recent_action_hashes[-self.window_size :]
        self._update_repetition_stats()

    def record_page_state(self, url: str, dom_text: str, element_count: int) -> None:
        """Record the current page fingerprint and update stagnation count."""
        fp = PageFingerprint.from_browser_state(url, dom_text, element_count)
        if self.recent_page_fingerprints and self.recent_page_fingerprints[-1] == fp:
            self.consecutive_stagnant_pages += 1
        else:
            self.consecutive_stagnant_pages = 0
            self.recent_page_fingerprints.append(fp)
        # Keep only last few fingerprints (no need for a large window)
        if len(self.recent_page_fingerprints) > 5:
            self.recent_page_fingerprints = self.recent_page_fingerprints[-5:]

    def _update_repetition_stats(self) -> None:
        """Recompute max_repetition_count from the current window."""
        if not self.recent_action_hashes:
            self.max_repetition_count = 0
            self.most_repeated_hash = None
            return
        counts: dict[str, int] = {}
        for h in self.recent_action_hashes:
            counts[h] = counts.get(h, 0) + 1
        self.most_repeated_hash = max(counts, key=lambda k: counts[k])
        self.max_repetition_count = counts[self.most_repeated_hash]
    def get_nudge_message(self) -> str | None:

```

```

    """Return an escalating awareness nudge based on repetition severity, or None if no loop detected."""
    messages: list[str] = []

    ## Action repetition nudges (escalating at 5, 8, 12)
    if self.max_repetition_count >= 12:
        messages.append(
            f'Heads up: you have repeated a similar action {self.max_repetition_count} times '
            f'in the last {len(self.recent_action_hashes)} actions. '
            'If you are making progress with each repetition, keep going. '
            'If not, a different approach might get you there faster.'
        )
    elif self.max_repetition_count >= 8:
        messages.append(
            f'Heads up: you have repeated a similar action {self.max_repetition_count} times '
            f'in the last {len(self.recent_action_hashes)} actions. '
            'Are you still making progress with each attempt? '
            'If so, carry on. Otherwise, it might be worth trying a different approach.'
        )
    elif self.max_repetition_count >= 5:
        messages.append(
            f'Heads up: you have repeated a similar action {self.max_repetition_count} times '
            f'in the last {len(self.recent_action_hashes)} actions. '
            'If this is intentional and making progress, carry on. '
            'If not, it might be worth reconsidering your approach.'
        )

    ## Page stagnation nudge
    if self.consecutive_stagnant_pages >= 5:
        messages.append(
            f'The page content has not changed across {self.consecutive_stagnant_pages} consecutive actions. '
            'Your actions might not be having the intended effect. '
            'It could be worth trying a different element or approach.'
        )

    if messages:
        return '\n\n'.join(messages)
    return None

class AgentState(BaseModel):
    """Holds all state information for an Agent"""

    model_config = ConfigDict(arbitrary_types_allowed=True)

    agent_id: str = Field(default_factory=uuid7str)
    n_steps: int = 1
    consecutive_failures: int = 0
    last_result: list[ActionResult] | None = None
    plan: list[PlanItem] | None = None
    current_plan_item_index: int = 0
    plan_generation_step: int | None = None
    last_model_output: AgentOutput | None = None

    ## Pause/resume state (kept serialisable for checkpointing)
    paused: bool = False
    stopped: bool = False
    session_initialized: bool = False # Track if session events have been dispatched
    follow_up_task: bool = False # Track if the agent is a follow-up task

    message_manager_state: MessageManagerState = Field(default_factory=MessageManagerState)
    file_system_state: FileSystemState | None = None

    ## Loop detection state
    loop_detector: ActionLoopDetector = Field(default_factory=ActionLoopDetector)

    @dataclass
    class AgentStepInfo:
        step_number: int
        max_steps: int

        def is_last_step(self) -> bool:

```

```

    """Check if this is the last step"""
    return self.step_number >= self.max_steps - 1

class JudgementResult(BaseModel):
    """LLM judgement of agent trace"""

    reasoning: str | None = Field(default=None, description='Explanation of the judgement')
    verdict: bool = Field(description='Whether the trace was successful or not')
    failure_reason: str | None = Field(
        default=None,
        description='Max 5 sentences explanation of why the task was not completed successfully in case of failure'
    )
    impossible_task: bool = Field(
        default=False,
        description='True if the task was impossible to complete due to vague instructions, broken website, etc.'
    )
    reached_captcha: bool = Field(
        default=False,
        description='True if the agent encountered captcha challenges during task execution',
    )

class ActionResult(BaseModel):
    """Result of executing an action"""

    ## For done action
    is_done: bool | None = False
    success: bool | None = None

    ## For trace judgement
    judgement: JudgementResult | None = None

    ## Error handling - always include in long term memory
    error: str | None = None

    ## Files
    attachments: list[str] | None = None # Files to display in the done message

    ## Images (base64 encoded) - separate from text content for efficient handling
    images: list[dict[str, Any]] | None = None # [{"name": "file.jpg", "data": "base64_string"}]

    ## Always include in long term memory
    long_term_memory: str | None = None # Memory of this action

    ## if update_only_read_state is True we add the extracted_content to the agent context only once for the rerun
    ## if update_only_read_state is False we add the extracted_content to the agent long term memory if not already present
    extracted_content: str | None = None
    include_extracted_content_only_once: bool = False # Whether the extracted content should be used to update the agent context only once

    ## Metadata for observability (e.g., click coordinates)
    metadata: dict | None = None

    ## Deprecated
    include_in_memory: bool = False # whether to include in extracted_content inside long_term_memory

    @model_validator(mode='after')
    def validate_success_requires_done(self):
        """Ensure success=True can only be set when is_done=True"""
        if self.success is True and self.is_done is not True:
            raise ValueError(
                'success=True can only be set when is_done=True. '
                'For regular actions that succeed, leave success as None. '
                'Use success=False only for actions that fail.'
            )
        return self

class RerunSummaryAction(BaseModel):
    """AI-generated summary for rerun completion"""

    summary: str = Field(description='Summary of what happened during the rerun')

```

```

■ success: bool = Field(description='Whether the rerun completed successfully based on visual inspection')
■ completion_status: Literal['complete', 'partial', 'failed'] = Field(
■ description='Status of rerun completion: complete (all steps succeeded), partial (some steps succeeded)'
■ )

```

```

class StepMetadata(BaseModel):
■ """Metadata for a single step including timing and token information"""

```

```

■ step_start_time: float
■ step_end_time: float
■ step_number: int
■ step_interval: float | None = None

■ @property
■ def duration_seconds(self) -> float:
■ """Calculate step duration in seconds"""
■ return self.step_end_time - self.step_start_time

```

```

class PlanItem(BaseModel):
■ text: str
■ status: Literal['pending', 'current', 'done', 'skipped'] = 'pending'

```

```

class AgentBrain(BaseModel):
■ thinking: str | None = None
■ evaluation_previous_goal: str
■ memory: str
■ next_goal: str

```

```

class AgentOutput(BaseModel):
■ model_config = ConfigDict(arbitrary_types_allowed=True, extra='forbid')

■ thinking: str | None = None
■ evaluation_previous_goal: str | None = None
■ memory: str | None = None
■ next_goal: str | None = None
■ current_plan_item: int | None = None
■ plan_update: list[str] | None = None
■ action: list[ActionModel] = Field(
■ ...,
■ json_schema_extra={'min_items': 1}, # Ensure at least one action is provided
■ )

```

```

■ @classmethod
■ def model_json_schema(cls, **kwargs):
■ schema = super().model_json_schema(**kwargs)
■ schema['required'] = ['evaluation_previous_goal', 'memory', 'next_goal', 'action']
■ return schema

```

```

■ @property
■ def current_state(self) -> AgentBrain:
■ """For backward compatibility - returns an AgentBrain with the flattened properties"""
■ return AgentBrain(
■ thinking=self.thinking,
■ evaluation_previous_goal=self.evaluation_previous_goal if self.evaluation_previous_goal else '',
■ memory=self.memory if self.memory else '',
■ next_goal=self.next_goal if self.next_goal else '',
■ )

```

```

■ @staticmethod
■ def type_with_custom_actions(custom_actions: type[ActionModel]) -> type[AgentOutput]:
■ """Extend actions with custom actions"""

```

```

■ model_ = create_model(
■ 'AgentOutput',
■ __base__=AgentOutput,
■ action=(
■ list[custom_actions], # type: ignore
■ Field(..., description='List of actions to execute', json_schema_extra={'min_items': 1}),

```

```

    ),
    __module__=AgentOutput.__module__,
    )
    return model_

    @staticmethod
    def type_with_custom_actions_no_thinking(custom_actions: type[ActionModel]) -> type[AgentOutput]:
        """Extend actions with custom actions and exclude thinking field"""

        class AgentOutputNoThinking(AgentOutput):
            @classmethod
            def model_json_schema(cls, **kwargs):
                schema = super().model_json_schema(**kwargs)
                del schema['properties']['thinking']
                schema['required'] = ['evaluation_previous_goal', 'memory', 'next_goal', 'action']
                return schema

        model = create_model(
            'AgentOutput',
            __base__=AgentOutputNoThinking,
            action=(
                list[custom_actions], # type: ignore
                Field(..., json_schema_extra={'min_items': 1}),
            ),
            __module__=AgentOutputNoThinking.__module__,
        )

        return model

    @staticmethod
    def type_with_custom_actions_flash_mode(custom_actions: type[ActionModel]) -> type[AgentOutput]:
        """Extend actions with custom actions for flash mode - memory and action fields only"""

        class AgentOutputFlashMode(AgentOutput):
            @classmethod
            def model_json_schema(cls, **kwargs):
                schema = super().model_json_schema(**kwargs)
                # Remove thinking, evaluation_previous_goal, next_goal, and plan fields
                del schema['properties']['thinking']
                del schema['properties']['evaluation_previous_goal']
                del schema['properties']['next_goal']
                schema['properties'].pop('current_plan_item', None)
                schema['properties'].pop('plan_update', None)
                # Update required fields to only include remaining properties
                schema['required'] = ['memory', 'action']
                return schema

        model = create_model(
            'AgentOutput',
            __base__=AgentOutputFlashMode,
            action=(
                list[custom_actions], # type: ignore
                Field(..., json_schema_extra={'min_items': 1}),
            ),
            __module__=AgentOutputFlashMode.__module__,
        )

        return model

class AgentHistory(BaseModel):
    """History item for agent actions"""

    model_output: AgentOutput | None
    result: list[ActionResult]
    state: BrowserStateHistory
    metadata: StepMetadata | None = None
    state_message: str | None = None

    model_config = ConfigDict(arbitrary_types_allowed=True, protected_namespaces=())

    @staticmethod

```

```

def get_interacted_element(model_output: AgentOutput, selector_map: DOMSelectorMap) -> list[DOMInteractedElement]:
    elements = []
    for action in model_output.action:
        index = action.get_index()
        if index is not None and index in selector_map:
            el = selector_map[index]
            elements.append(DOMInteractedElement.load_from_enhanced_dom_tree(el))
        else:
            elements.append(None)
    return elements

def _filter_sensitive_data_from_string(self, value: str, sensitive_data: dict[str, str] | dict[str, str] | None) -> str:
    """Filter out sensitive data from a string value"""
    if not sensitive_data:
        return value

    ## Collect all sensitive values, immediately converting old format to new format
    sensitive_values: dict[str, str] = {}

    ## Process all sensitive data entries
    for key_or_domain, content in sensitive_data.items():
        if isinstance(content, dict):
            ## Already in new format: {domain: {key: value}}
            for key, val in content.items():
                if val: # Skip empty values
                    sensitive_values[key] = val
            elif content: # Old format: {key: value} - convert to new format internally
                ## We treat this as if it was {'http*://*': {key_or_domain: content}}
                sensitive_values[key_or_domain] = content

    ## If there are no valid sensitive data entries, just return the original value
    if not sensitive_values:
        return value

    ## Replace all valid sensitive data values with their placeholder tags
    for key, val in sensitive_values.items():
        value = value.replace(val, f'<secret>{key}</secret>')

    return value

def _filter_sensitive_data_from_dict(
    self, data: dict[str, Any], sensitive_data: dict[str, str] | dict[str, str] | None
) -> dict[str, Any]:
    """Recursively filter sensitive data from a dictionary"""
    if not sensitive_data:
        return data

    filtered_data = {}
    for key, value in data.items():
        if isinstance(value, str):
            filtered_data[key] = self._filter_sensitive_data_from_string(value, sensitive_data)
        elif isinstance(value, dict):
            filtered_data[key] = self._filter_sensitive_data_from_dict(value, sensitive_data)
        elif isinstance(value, list):
            filtered_data[key] = [
                self._filter_sensitive_data_from_string(item, sensitive_data)
                if isinstance(item, str)
                else self._filter_sensitive_data_from_dict(item, sensitive_data)
                if isinstance(item, dict)
                else item
            ]
        else:
            filtered_data[key] = value
    return filtered_data

def model_dump(self, sensitive_data: dict[str, str] | dict[str, str] | None = None, **kwargs) -> dict[str, Any]:
    """Custom serialization handling circular references and filtering sensitive data"""

    ## Handle action serialization
    model_output_dump = None
    if self.model_output:

```

```

    action_dump = [action.model_dump(exclude_none=True, mode='json') for action in self.model_output.actions]

    # Filter sensitive data only from input action parameters if sensitive_data is provided
    if sensitive_data:
        action_dump = [
            self._filter_sensitive_data_from_dict(action, sensitive_data) if 'input' in action else action
            for action in action_dump
        ]

    model_output_dump = {
        'evaluation_previous_goal': self.model_output.evaluation_previous_goal,
        'memory': self.model_output.memory,
        'next_goal': self.model_output.next_goal,
        'action': action_dump, # This preserves the actual action data
    }

    # Only include thinking if it's present
    if self.model_output.thinking is not None:
        model_output_dump['thinking'] = self.model_output.thinking
    if self.model_output.current_plan_item is not None:
        model_output_dump['current_plan_item'] = self.model_output.current_plan_item
    if self.model_output.plan_update is not None:
        model_output_dump['plan_update'] = self.model_output.plan_update

    # Handle result serialization - don't filter ActionResult data
    # as it should contain meaningful information for the agent
    result_dump = [r.model_dump(exclude_none=True, mode='json') for r in self.result]

    return {
        'model_output': model_output_dump,
        'result': result_dump,
        'state': self.state.to_dict(),
        'metadata': self.metadata.model_dump() if self.metadata else None,
        'state_message': self.state_message,
    }

```

```

AgentStructuredOutput = TypeVar('AgentStructuredOutput', bound=BaseModel)

```

```

class AgentHistoryList(BaseModel, Generic[AgentStructuredOutput]):
    """List of AgentHistory messages, i.e. the history of the agent's actions and thoughts."""

    history: list[AgentHistory]
    usage: UsageSummary | None = None

    _output_model_schema: type[AgentStructuredOutput] | None = None

    def total_duration_seconds(self) -> float:
        """Get total duration of all steps in seconds"""
        total = 0.0
        for h in self.history:
            if h.metadata:
                total += h.metadata.duration_seconds
        return total

    def __len__(self) -> int:
        """Return the number of history items"""
        return len(self.history)

    def __str__(self) -> str:
        """Representation of the AgentHistoryList object"""
        return f'AgentHistoryList(all_results={self.action_results()}, all_model_outputs={self.model_actions()})'

    def add_item(self, history_item: AgentHistory) -> None:
        """Add a history item to the list"""
        self.history.append(history_item)

    def __repr__(self) -> str:
        """Representation of the AgentHistoryList object"""
        return self.__str__()

    def save_to_file(self, filepath: str | Path, sensitive_data: dict[str, str | dict[str, str]] | None = None) -> None:

```



```

    """Save history to JSON file with proper serialization and optional sensitive data filtering"""
    try:
        Path(filepath).parent.mkdir(parents=True, exist_ok=True)
        data = self.model_dump(sensitive_data=sensitive_data)
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2)
    except Exception as e:
        raise e

    def save_as_playwright_script(
        self,
        output_path: str | Path,
        sensitive_data_keys: list[str] | None = None,
        browser_config: BrowserConfig | None = None,
        context_config: BrowserContextConfig | None = None,
    ) -> None:
        """
        Generates a Playwright script based on the agent's history and saves it to a file.
        Args:
            output_path: The path where the generated Python script will be saved.
            sensitive_data_keys: A list of keys used as placeholders for sensitive data
                               (e.g., ['username_placeholder', 'password_placeholder']).
                               These will be loaded from environment variables in the
                               generated script.
            browser_config: Configuration of the original Browser instance.
            context_config: Configuration of the original BrowserContext instance.
        """
        from system.agent.playwright_script_generator import PlaywrightScriptGenerator

    try:
        serialized_history = self.model_dump()['history']
        generator = PlaywrightScriptGenerator(serialized_history, sensitive_data_keys, browser_config, context_config)

        script_content = generator.generate_script_content()
        path_obj = Path(output_path)
        path_obj.parent.mkdir(parents=True, exist_ok=True)
        with open(path_obj, 'w', encoding='utf-8') as f:
            f.write(script_content)
    except Exception as e:
        raise e

    def model_dump(self, **kwargs) -> dict[str, Any]:
        """Custom serialization that properly uses AgentHistory's model_dump"""
        return {
            'history': [h.model_dump(**kwargs) for h in self.history],
        }

    @classmethod
    def load_from_dict(cls, data: dict[str, Any], output_model: type[AgentOutput]) -> AgentHistoryList:
        """Load history from dict and validate output_model actions to enrich with custom actions"""
        for h in data['history']:
            if h['model_output']:
                if isinstance(h['model_output'], dict):
                    h['model_output'] = output_model.model_validate(h['model_output'])
                else:
                    h['model_output'] = None
            if 'interacted_element' not in h['state']:
                h['state']['interacted_element'] = None

        history = cls.model_validate(data)
        return history

    @classmethod
    def load_from_file(cls, filepath: str | Path, output_model: type[AgentOutput]) -> AgentHistoryList:
        """Load history from JSON file"""
        with open(filepath, encoding='utf-8') as f:
            data = json.load(f)
        return cls.load_from_dict(data, output_model)

    def last_action(self) -> None | dict:
        """Last action in history"""
        if self.history and self.history[-1].model_output:

```

```

    return self.history[-1].model_output.action[-1].model_dump(exclude_none=True, mode='json')
    return None

    def errors(self) -> list[str | None]:
        """Get all errors from history, with None for steps without errors"""
        errors = []
        for h in self.history:
            step_errors = [r.error for r in h.result if r.error]

            # each step can have only one error
            errors.append(step_errors[0] if step_errors else None)
        return errors

    def final_result(self) -> None | str:
        """Final result from history"""
        if self.history and self.history[-1].result[-1].extracted_content:
            return self.history[-1].result[-1].extracted_content
        return None

    def is_done(self) -> bool:
        """Check if the agent is done"""
        if self.history and len(self.history[-1].result) > 0:
            last_result = self.history[-1].result[-1]
            return last_result.is_done is True
        return False

    def is_successful(self) -> bool | None:
        """Check if the agent completed successfully - the agent decides in the last step if it was successful"""
        if self.history and len(self.history[-1].result) > 0:
            last_result = self.history[-1].result[-1]
            if last_result.is_done is True:
                return last_result.success
        return None

    def has_errors(self) -> bool:
        """Check if the agent has any non-None errors"""
        return any(error is not None for error in self.errors())

    def judgement(self) -> dict | None:
        """Get the judgement result as a dictionary if it exists"""
        if self.history and len(self.history[-1].result) > 0:
            last_result = self.history[-1].result[-1]
            if last_result.judgement:
                return last_result.judgement.model_dump()
        return None

    def is_judged(self) -> bool:
        """Check if the agent trace has been judged"""
        if self.history and len(self.history[-1].result) > 0:
            last_result = self.history[-1].result[-1]
            return last_result.judgement is not None
        return False

    def is_validated(self) -> bool | None:
        """Check if the judge validated the agent execution (verdict is True). Returns None if not judged yet"""
        if self.history and len(self.history[-1].result) > 0:
            last_result = self.history[-1].result[-1]
            if last_result.judgement:
                return last_result.judgement.verdict
        return None

    def urls(self) -> list[str | None]:
        """Get all unique URLs from history"""
        return [h.state.url if h.state.url is not None else None for h in self.history]

    def screenshot_paths(self, n_last: int | None = None, return_none_if_not_screenshot: bool = True) -> list[str | None]:
        """Get all screenshot paths from history"""
        if n_last == 0:
            return []
        if n_last is None:
            if return_none_if_not_screenshot:
                return [h.state.screenshot_path if h.state.screenshot_path is not None else None for h in self.history]

```

```

    else:
        return [h.state.screenshot_path for h in self.history if h.state.screenshot_path is not None]
    else:
        if return_none_if_not_screenshot:
            return [h.state.screenshot_path if h.state.screenshot_path is not None else None for h in self.history]
        else:
            return [h.state.screenshot_path for h in self.history[-n_last:]] if h.state.screenshot_path is not None

def screenshots(self, n_last: int | None = None, return_none_if_not_screenshot: bool = True) -> list[str]:
    """Get all screenshots from history as base64 strings"""
    if n_last == 0:
        return []

    history_items = self.history if n_last is None else self.history[-n_last:]
    screenshots = []

    for item in history_items:
        screenshot_b64 = item.state.get_screenshot()
        if screenshot_b64:
            screenshots.append(screenshot_b64)
        else:
            if return_none_if_not_screenshot:
                screenshots.append(None)
            # If return_none_if_not_screenshot is False, we skip None values

    return screenshots

def action_names(self) -> list[str]:
    """Get all action names from history"""
    action_names = []
    for action in self.model_actions():
        actions = list(action.keys())
        if actions:
            action_names.append(actions[0])
    return action_names

def model_thoughts(self) -> list[AgentBrain]:
    """Get all thoughts from history"""
    return [h.model_output.current_state for h in self.history if h.model_output]

def model_outputs(self) -> list[AgentOutput]:
    """Get all model outputs from history"""
    return [h.model_output for h in self.history if h.model_output]

# get all actions with params
def model_actions(self) -> list[dict]:
    """Get all actions from history"""
    outputs = []

    for h in self.history:
        if h.model_output:
            # Guard against None interacted_element before zipping
            interacted_elements = h.state.interacted_element or [None] * len(h.model_output.action)
            for action, interacted_element in zip(h.model_output.action, interacted_elements):
                output = action.model_dump(exclude_none=True, mode='json')
                output['interacted_element'] = interacted_element
                outputs.append(output)
    return outputs

def action_history(self) -> list[list[dict]]:
    """Get truncated action history with only essential fields"""
    step_outputs = []

    for h in self.history:
        step_actions = []
        if h.model_output:
            # Guard against None interacted_element before zipping
            interacted_elements = h.state.interacted_element or [None] * len(h.model_output.action)
            # Zip actions with interacted elements and results
            for action, interacted_element, result in zip(h.model_output.action, interacted_elements, h.result):
                action_output = action.model_dump(exclude_none=True, mode='json')
                action_output['interacted_element'] = interacted_element

```

```

##### Only keep long_term_memory from result
##### action_output['result'] = result.long_term_memory if result and result.long_term_memory else None
##### step_actions.append(action_output)
##### step_outputs.append(step_actions)

##### return step_outputs

def action_results(self) -> list[ActionResult]:
    """Get all results from history"""
    results = []
    for h in self.history:
        results.extend([r for r in h.result if r])
    return results

def extracted_content(self) -> list[str]:
    """Get all extracted content from history"""
    content = []
    for h in self.history:
        content.extend([r.extracted_content for r in h.result if r.extracted_content])
    return content

def model_actions_filtered(self, include: list[str] | None = None) -> list[dict]:
    """Get all model actions from history as JSON"""
    if include is None:
        include = []
    outputs = self.model_actions()
    result = []
    for o in outputs:
        for i in include:
            if i == list(o.keys())[0]:
                result.append(o)
    return result

def number_of_steps(self) -> int:
    """Get the number of steps in the history"""
    return len(self.history)

def agent_steps(self) -> list[str]:
    """Format agent history as readable step descriptions for judge evaluation."""
    steps = []

    ## Iterate through history items (each is an AgentHistory)
    for i, h in enumerate(self.history):
        step_text = f'Step {i + 1}: \n'

        ## Get actions from model_output
        if h.model_output and h.model_output.action:
            ## Use model_dump with mode='json' to serialize enums properly
            actions_list = [action.model_dump(exclude_none=True, mode='json') for action in h.model_output.action]
            action_json = json.dumps(actions_list, indent=1)
            step_text += f'Actions: {action_json} \n'

        ## Get results (already a list[ActionResult] in h.result)
        if h.result:
            for j, result in enumerate(h.result):
                if result.extracted_content:
                    content = str(result.extracted_content)
                    step_text += f'Result {j + 1}: {content} \n'

                if result.error:
                    error = str(result.error)
                    step_text += f'Error {j + 1}: {error} \n'

        steps.append(step_text)

    return steps

@property
def structured_output(self) -> AgentStructuredOutput | None:
    """Get the structured output from the history

    Returns:

```

```

    """The structured output if both final_result and _output_model_schema are available,
    otherwise None
    """
    final_result = self.final_result()
    if final_result is not None and self._output_model_schema is not None:
        return self._output_model_schema.model_validate_json(final_result)

    return None

    def get_structured_output(self, output_model: type[AgentStructuredOutput]) -> AgentStructuredOutput | None:
        """Get the structured output from history, parsing with the provided schema.

        Use this method when accessing structured output from sandbox execution,
        since the _output_model_schema private attribute is not preserved during serialization.

        Args:
            output_model: The Pydantic model class to parse the output with

        Returns:
            The parsed structured output, or None if no final result exists
            """
        final_result = self.final_result()
        if final_result is not None:
            return output_model.model_validate_json(final_result)
        return None

class AgentError:
    """Container for agent error handling"""

    VALIDATION_ERROR = 'Invalid model output format. Please follow the correct schema.'
    RATE_LIMIT_ERROR = 'Rate limit reached. Waiting before retry.'
    NO_VALID_ACTION = 'No valid action found'

    @staticmethod
    def format_error(error: Exception, include_trace: bool = False) -> str:
        """Format error message based on error type and optionally include trace"""
        message = ''
        if isinstance(error, ValidationError):
            return f'{AgentError.VALIDATION_ERROR}\nDetails: {str(error)}'
        # Lazy import to avoid loading openai SDK (~800ms) at module level
        from openai import RateLimitError

        if isinstance(error, RateLimitError):
            return AgentError.RATE_LIMIT_ERROR

        # Handle LLM response validation errors from llm_use
        error_str = str(error)
        if 'LLM response missing required fields' in error_str or 'Expected format: AgentOutput' in error_str:
            # Extract the main error message without the huge stacktrace
            lines = error_str.split('\n')
            main_error = lines[0] if lines else error_str

        # Provide a clearer error message
        helpful_msg = f'{main_error}\n\nThe previous response had an invalid output structure. Please stick

        if include_trace:
            helpful_msg += f'\n\nFull stacktrace:\n{traceback.format_exc()}'

        return helpful_msg

        if include_trace:
            return f'{str(error)}\nStacktrace:\n{traceback.format_exc()}'
        return f'{str(error)}'

class DetectedVariable(BaseModel):
    """A detected variable in agent history"""

    name: str
    original_value: str
    type: str = 'string'

```

```
■format: str | None = None
```

```
class VariableMetadata(BaseModel):  
    ■"""Metadata about detected variables in history"""  
  
    ■detected_variables: dict[str, DetectedVariable] = Field(default_factory=dict)
```

.\system\agent\workflow.py

```
import json  
import logging  
from typing import Any, Dict, List, Optional  
from uuid import uuid4  
  
from pydantic import BaseModel, Field  
  
from system.agent.views import AgentHistoryList  
from system.dom.views import DOMInteractedElement  
  
logger = logging.getLogger(__name__)  
  
class SemanticSelector(BaseModel):  
    """  
    Abstracted selector that doesn't rely solely on fragile DOM paths.  
    It captures the semantic meaning of the element at the time of interaction.  
    """  
    role: Optional[str] = None  
    name: Optional[str] = None  
    tag_name: Optional[str] = None  
    xpath: Optional[str] = None  
    attributes: Optional[Dict[str, str]] = None  
    text_content: Optional[str] = None  
  
class ActionCommand(BaseModel):  
    """The raw action commanded by the agent"""  
    action_type: str # e.g. "click", "input", "navigate", "extract", "done"  
    parameters: Dict[str, Any]  
  
class WorkflowStep(BaseModel):  
    """A single step in an abstracted action graph"""  
    step_index: int  
    command: ActionCommand  
    semantic_selector: Optional[SemanticSelector] = None  
    original_thought: Optional[str] = None  
  
class WorkflowTemplate(BaseModel):  
    """A fully serialized workflow that can be executed later without reasoning"""  
    workflow_id: str  
    target_url: Optional[str] = None  
    original_task: Optional[str] = None  
    steps: List[WorkflowStep]  
  
    def to_json(self) -> str:  
        return self.model_dump_json(indent=2)  
  
class WorkflowRecorder:  
    """  
    Extracts semantic action graphs from MIRA's AgentHistoryList.  
    This creates replayable UI automations that are robust to DOM changes.  
    """  
  
    @staticmethod
```

```

def extract_workflow(history_list: AgentHistoryList, original_task: Optional[str] = None, workflow_id: Optional[str] = None)
    """
    Parses the entire history list of a successful agent run and turns it into an Action Graph.
    """
    if workflow_id is None:
        workflow_id = str(uuid4())

    steps: List[WorkflowStep] = []
    target_url = None
    step_counter = 1

    for history_item in history_list.history:
        if not history_item.model_output:
            continue

        # Keep track of the first meaningful URL
        if not target_url and history_item.state and history_item.state.url != 'about:blank':
            target_url = history_item.state.url

        thought = history_item.model_output.thinking
        actions = history_item.model_output.action

        # Use the interacted elements captured during state processing
        interacted_elements = history_item.state.interacted_element or []

        for i, action_model in enumerate(actions):
            action_dump = action_model.model_dump(exclude_none=True)

            # MIRA Action models map {action_name: {parameters}}
            action_type = next(iter(action_dump.keys()))
            parameters = action_dump[action_type]

            command = ActionCommand(
                action_type=action_type,
                parameters=parameters
            )

            semantic_selector = None

            # If this action interacted with a DOM element, extract its semantic profile
            if i < len(interacted_elements) and interacted_elements[i]:
                elem: DOMInteractedElement = interacted_elements[i]

                attributes = elem.attributes or {}

                semantic_selector = SemanticSelector(
                    role=attributes.get('role'),
                    name=elem.ax_name or attributes.get('aria-label') or attributes.get('name'),
                    tag_name=elem.node_name,
                    xpath=elem.x_path,
                    attributes=attributes,
                    text_content=elem.node_value
                )

            # Only record meaningful steps for the macro
            if action_type in ['done', 'scroll', 'wait']:
                # We might skip recording these, or keep them if they are critical.
                # 'done' usually just signals completion to the user.
                if action_type == 'done':
                    continue

            steps.append(WorkflowStep(
                step_index=step_counter,
                command=command,
                semantic_selector=semantic_selector,
                original_thought=thought if i == 0 else None # Attach thought to first action of the macro
            ))
            step_counter += 1

    # Pruning logic: If the agent hit a failure and went back,
    # we could implement basic pruning here (e.g. removing steps between redundant navigations).
    # For phase v1, we serialize the exact successful path.

```

```

        return WorkflowTemplate(
            workflow_id=workflow_id,
            target_url=target_url,
            original_task=original_task,
            steps=steps
        )

class WorkflowExecutor:
    """
    Executes a WorkflowTemplate deterministically by finding elements semantically
    in the DOMState without relying on LLM generation.
    """
    def __init__(self, workflow_template: WorkflowTemplate):
        self.workflow = workflow_template
        self.current_step_index = 0
        self.mode = "macro"

    def get_next_action(self, dom_state: Any, agent_output_class: type[BaseModel]) -> Optional[Any]: # AgentOutput
        """
        Returns an AgentOutput with the exact ActionModel if it can perfectly
        resolve the next step, else returns None to fallback to LLM.
        """
        if self.mode == "autonomous":
            return None

        if self.current_step_index >= len(self.workflow.steps):
            self.mode = "autonomous"
            return None

        step = self.workflow.steps[self.current_step_index]

        # If no semantic selector is required (like wait or scroll), execute directly
        if not step.semantic_selector:
            self.current_step_index += 1
            return self._build_agent_output(step, agent_output_class)

        # Try to find the element
        best_index = self._find_element(step.semantic_selector, dom_state.selector_map)

        if best_index is None:
            logger.warning(
                f"■■■ [WorkflowExecutor] Failed to find element for step {step.step_index}. "
                f"Selector: {step.semantic_selector.model_dump_json(exclude_none=True)}. "
                f"Falling back to autonomous LLM recovery."
            )
            self.mode = "autonomous"
            return None

        # Found it! Update parameters with dynamic index
        logger.info(f"■■ [WorkflowExecutor] Semantically matched element for step {step.step_index} to index {best_index}")
        step.command.parameters['index'] = best_index
        self.current_step_index += 1

        return self._build_agent_output(step, agent_output_class)

    def _build_agent_output(self, step: WorkflowStep, agent_output_class: type[BaseModel]) -> Any: # AgentOutput
        # Convert dictionary command back to ActionModel schema
        action_payload = {step.command.action_type: step.command.parameters}

        try:
            # Create a mock thought so the frontend trace stream knows what happened
            thought = f"[MACRO EXECUTION] Executing step {step.step_index}: {step.command.action_type}"
            if step.original_thought:
                thought += f"\nOriginal Intent: {step.original_thought}"

            payload = {
                "evaluation_previous_goal": "Success",
                "memory": "Macro execution",
                "next_goal": "Continue macro",
                "thinking": thought,
            }

```



```

        "action": [action_payload]
    }

    return agent_output_class.model_validate(payload)
except Exception as e:
    logger.error(f"Failed to compile step action: {e}")
    self.mode = "autonomous"
    return None

def _find_element(self, selector: SemanticSelector, selector_map: dict) -> Optional[int]:
    """
    Tier 1: Exact Match
    Tier 2: Fuzzy Semantic Match
    """
    best_match_idx = None
    highest_score = 0

    for idx, node in selector_map.items():
        score = 0

        # Extract current node details
        node_tag = node.node_name if hasattr(node, 'node_name') else ""
        node_role = node.attributes.get('role') if hasattr(node, 'attributes') else None
        node_ax_name = node.attributes.get('aria-label') or node.attributes.get('name') or (node.ax_

        # Exact Tag Match (+2)
        if selector.tag_name and node_tag == selector.tag_name:
            score += 2

        # Exact Role Match (+3)
        if selector.role and node_role == selector.role:
            score += 3

        # Exact Name/Label Match (+5)
        if selector.name and node_ax_name and selector.name.lower() == node_ax_name.lower():
            score += 5
        # Fuzzy Name Match (+3)
        elif selector.name and node_ax_name and selector.name.lower() in node_ax_name.lower():
            score += 3

        # Exact Text Match (+4)
        if selector.text_content:
            node_text = node.get_all_children_text() if hasattr(node, 'get_all_children_text') else
            if selector.text_content.strip() == node_text.strip():
                score += 4

        if score > highest_score and score >= 4: # Minimum threshold to prevent false clicks
            highest_score = score
            best_match_idx = int(idx)

    return best_match_idx

```

.lsystem\agent\message_manager\service.py

```

from __future__ import annotations

import logging
from typing import Literal

from system.agent.message_manager.views import (
    ■HistoryItem,
)
from system.agent.prompts import AgentMessagePrompt
from system.agent.views import (
    ■ActionResult,
    ■AgentOutput,
    ■AgentStepInfo,
    ■MessageCompactionSettings,

```



```

    lines.append(prefix + content)

return lines
except Exception as e:
    logger.warning(f'Failed to format message line for logging: {e}')
    # Return a simple fallback line
    return ['[ ?]: [Error formatting message]']

# ===== End of Logging Helper Functions =====

class MessageManager:
    vision_detail_level: Literal['auto', 'low', 'high']

    def __init__(
        self,
        task: str,
        system_message: SystemMessage,
        file_system: FileSystem,
        state: MessageManagerState = MessageManagerState(),
        use_thinking: bool = True,
        include_attributes: list[str] | None = None,
        sensitive_data: dict[str, str | dict[str, str]] | None = None,
        max_history_items: int | None = None,
        vision_detail_level: Literal['auto', 'low', 'high'] = 'auto',
        include_tool_call_examples: bool = False,
        include_recent_events: bool = False,
        sample_images: list[ContentPartTextParam | ContentPartImageParam] | None = None,
        llm_screenshot_size: tuple[int, int] | None = None,
        max_clickable_elements_length: int = 40000,
    ):
        self.task = task
        self.state = state
        self.system_prompt = system_message
        self.file_system = file_system
        self.sensitive_data_description = ''
        self.use_thinking = use_thinking
        self.max_history_items = max_history_items
        self.vision_detail_level = vision_detail_level
        self.include_tool_call_examples = include_tool_call_examples
        self.include_recent_events = include_recent_events
        self.sample_images = sample_images
        self.llm_screenshot_size = llm_screenshot_size
        self.max_clickable_elements_length = max_clickable_elements_length

        assert max_history_items is None or max_history_items > 5, 'max_history_items must be None or greater

    # Store settings as direct attributes instead of in a settings object
    self.include_attributes = include_attributes or []
    self.sensitive_data = sensitive_data
    self.last_input_messages = []
    self.last_state_message_text: str | None = None
    # Only initialize messages if state is empty
    if len(self.state.history.get_messages()) == 0:
        self._set_message_with_type(self.system_prompt, 'system')

    @property
    def agent_history_description(self) -> str:
        """Build agent history description from list of items, respecting max_history_items limit"""
        compacted_prefix = ''
        if self.state.compacted_memory:
            compacted_prefix = f'<compacted_memory>\n{self.state.compacted_memory}\n</compacted_memory>\n'

        if self.max_history_items is None:
            # Include all items
            return compacted_prefix + '\n'.join(item.to_string() for item in self.state.agent_history_items)

        total_items = len(self.state.agent_history_items)

        # If we have fewer items than the limit, just return all items
        if total_items <= self.max_history_items:

```

```

    return compacted_prefix + '\n'.join(item.to_string() for item in self.state.agent_history_items)

    ## We have more items than the limit, so we need to omit some
    omitted_count = total_items - self.max_history_items

    ## Show first item + omitted message + most recent (max_history_items - 1) items
    ## The omitted message doesn't count against the limit, only real history items do
    recent_items_count = self.max_history_items - 1 # -1 for first item

    items_to_include = [
        self.state.agent_history_items[0].to_string(), # Keep first item (initialization)
        f'<sys>[... {omitted_count} previous steps omitted...]</sys>',
    ]
    ## Add most recent items
    items_to_include.extend([item.to_string() for item in self.state.agent_history_items[-recent_items_count:]])

    return compacted_prefix + '\n'.join(items_to_include)

    def add_new_task(self, new_task: str) -> None:
        new_task = '<follow_up_user_request>' + new_task.strip() + ' </follow_up_user_request>'
        if '<initial_user_request>' not in self.task:
            self.task = '<initial_user_request>' + self.task + '</initial_user_request>'
        self.task += '\n' + new_task
        task_update_item = HistoryItem(system_message=new_task)
        self.state.agent_history_items.append(task_update_item)

    def prepare_step_state(
        self,
        browser_state_summary: BrowserStateSummary,
        model_output: AgentOutput | None = None,
        result: list[ActionResult] | None = None,
        step_info: AgentStepInfo | None = None,
        sensitive_data=None,
    ) -> None:
        """Prepare state for the next LLM call without building the final state message."""
        self.state.history.context_messages.clear()
        self._update_agent_history_description(model_output, result, step_info)

        effective_sensitive_data = sensitive_data if sensitive_data is not None else self.sensitive_data
        if effective_sensitive_data is not None:
            self.sensitive_data = effective_sensitive_data
            self.sensitive_data_description = self._get_sensitive_data_description(browser_state_summary.url)

    async def maybe_compact_messages(
        self,
        llm: BaseChatModel | None,
        settings: MessageCompactionSettings | None,
        step_info: AgentStepInfo | None = None,
    ) -> bool:
        """Summarize older history into a compact memory block.

        Step interval is the primary trigger; char count is a minimum floor.
        """
        if not settings or not settings.enabled:
            return False
        if llm is None:
            return False
        if step_info is None:
            return False

        ## Step cadence gate
        steps_since = step_info.step_number - (self.state.last_compaction_step or 0)
        if steps_since < settings.compact_every_n_steps:
            return False

        ## Char floor gate
        history_items = self.state.agent_history_items
        full_history_text = '\n'.join(item.to_string() for item in history_items).strip()
        trigger_char_count = settings.trigger_char_count or 40000
        if len(full_history_text) < trigger_char_count:
            return False
        logger.debug(f'Compacting message history (items={len(history_items)}, chars={len(full_history_text)})')

```

```

    ## Build compaction input
    compaction_sections = []
    if self.state.compacted_memory:
        compaction_sections.append(
            f'<previous_compacted_memory>\n{self.state.compacted_memory}\n</previous_compacted_memory>'
        )
    compaction_sections.append(f'<agent_history>\n{full_history_text}\n</agent_history>')
    if settings.include_read_state and self.state.read_state_description:
        compaction_sections.append(f'<read_state>\n{self.state.read_state_description}\n</read_state>')
    compaction_input = '\n\n'.join(compaction_sections)

    if self.sensitive_data:
        filtered = self._filter_sensitive_data(UserMessage(content=compaction_input))
        compaction_input = filtered.text

    system_prompt = (
        'You are summarizing an agent run for prompt compaction.\n'
        'Capture task requirements, key facts, decisions, partial progress, errors, and next steps.\n'
        'Preserve important entities, values, URLs, and file paths.\n'
        'Return plain text only. Do not include tool calls or JSON.'
    )
    if settings.summary_max_chars:
        system_prompt += f' Keep under {settings.summary_max_chars} characters if possible.'

    messages = [SystemMessage(content=system_prompt), UserMessage(content=compaction_input)]
    try:
        response = await llm.ainvoke(messages)
        summary = (response.completion or '').strip()
    except Exception as e:
        logger.warning(f'Failed to compact messages: {e}')
        return False

    if not summary:
        return False

    if settings.summary_max_chars and len(summary) > settings.summary_max_chars:
        summary = summary[: settings.summary_max_chars].rstrip() + '...'

    self.state.compacted_memory = summary
    self.state.compaction_count += 1
    self.state.last_compaction_step = step_info.step_number

    ## Keep first item + most recent items
    keep_last = max(0, settings.keep_last_items)
    if len(history_items) > keep_last + 1:
        if keep_last == 0:
            self.state.agent_history_items = [history_items[0]]
        else:
            self.state.agent_history_items = [history_items[0]] + history_items[-keep_last:]

    logger.debug(f'Compaction complete (summary_chars={len(summary)}, history_items={len(self.state.agent_history_items)})')

    return True

def _update_agent_history_description(
    self,
    model_output: AgentOutput | None = None,
    result: list[ActionResult] | None = None,
    step_info: AgentStepInfo | None = None,
) -> None:
    """Update the agent history description"""

    if result is None:
        result = []
        step_number = step_info.step_number if step_info else None

    self.state.read_state_description = ''
    self.state.read_state_images = [] # Clear images from previous step

    action_results = ''
    result_len = len(result)
    read_state_idx = 0

```

```

    for idx, action_result in enumerate(result):
        if action_result.include_extracted_content_only_once and action_result.extracted_content:
            self.state.read_state_description += (
                f'<read_state_{read_state_idx}>\n{action_result.extracted_content}\n</read_state_{read_state_idx}>'
            )
            read_state_idx += 1
            logger.debug(f'Added extracted_content to read_state_description: {action_result.extracted_content}')

        # Store images for one-time inclusion in the next message
        if action_result.images:
            self.state.read_state_images.extend(action_result.images)
            logger.debug(f'Added {len(action_result.images)} image(s) to read_state_images')

        if action_result.long_term_memory:
            action_results += f'{action_result.long_term_memory}\n'
            logger.debug(f'Added long_term_memory to action_results: {action_result.long_term_memory}')
            elif action_result.extracted_content and not action_result.include_extracted_content_only_once:
                action_results += f'{action_result.extracted_content}\n'
                logger.debug(f'Added extracted_content to action_results: {action_result.extracted_content}')

        if action_result.error:
            if len(action_result.error) > 200:
                error_text = action_result.error[:100] + '.....' + action_result.error[-100:]
            else:
                error_text = action_result.error
            action_results += f'{error_text}\n'
            logger.debug(f'Added error to action_results: {error_text}')

        # Simple 60k character limit for read_state_description
        MAX_CONTENT_SIZE = 60000
        if len(self.state.read_state_description) > MAX_CONTENT_SIZE:
            self.state.read_state_description = (
                self.state.read_state_description[:MAX_CONTENT_SIZE] + '\n... [Content truncated at 60k characters]'
            )
            logger.debug(f'Truncated read_state_description to {MAX_CONTENT_SIZE} characters')

        self.state.read_state_description = self.state.read_state_description.strip('\n')

        if action_results:
            action_results = f'Result\n{action_results}'
            action_results = action_results.strip('\n') if action_results else None

        # Simple 60k character limit for action_results
        if action_results and len(action_results) > MAX_CONTENT_SIZE:
            action_results = action_results[:MAX_CONTENT_SIZE] + '\n... [Content truncated at 60k characters]'
            logger.debug(f'Truncated action_results to {MAX_CONTENT_SIZE} characters')

        # Build the history item
        if model_output is None:
            # Add history item for initial actions (step 0) or errors (step > 0)
            if step_number is not None:
                if step_number == 0 and action_results:
                    # Step 0 with initial action results
                    history_item = HistoryItem(step_number=step_number, action_results=action_results)
                    self.state.agent_history_items.append(history_item)
                elif step_number > 0:
                    # Error case for steps > 0
                    history_item = HistoryItem(step_number=step_number, error='Agent failed to output in the right format')
                    self.state.agent_history_items.append(history_item)
                else:
                    history_item = HistoryItem(
                        step_number=step_number,
                        evaluation_previous_goal=model_output.current_state.evaluation_previous_goal,
                        memory=model_output.current_state.memory,
                        next_goal=model_output.current_state.next_goal,
                        action_results=action_results,
                    )
                    self.state.agent_history_items.append(history_item)

    def _get_sensitive_data_description(self, current_page_url) -> str:
        sensitive_data = self.sensitive_data
        if not sensitive_data:

```

```

    return ''

    # Collect placeholders for sensitive data
    placeholders: set[str] = set()

    for key, value in sensitive_data.items():
        if isinstance(value, dict):
            # New format: {domain: {key: value}}
            if current_page_url and match_url_with_domain_pattern(current_page_url, key, True):
                placeholders.update(value.keys())
            else:
                # Old format: {key: value}
                placeholders.add(key)

    if placeholders:
        placeholder_list = sorted(list(placeholders))
        # Format as bullet points for clarity
        formatted_placeholders = '\n'.join(f' - {p}' for p in placeholder_list)

        info = 'SENSITIVE DATA - Use these placeholders for secure input:\n'
        info += f'{formatted_placeholders}\n\n'
        info += 'IMPORTANT: When entering sensitive values, you MUST wrap the placeholder name in <secret> t
        info += f'Example: To enter the value for "{placeholder_list[0]}"', use: <secret>{placeholder_list[0]}
        info += 'The system will automatically replace these tags with the actual secret values.'
        return info

    return ''

@observe_debug(ignore_input=True, ignore_output=True, name='create_state_messages')
@time_execution_sync('--create_state_messages')
def create_state_messages(
    self,
    browser_state_summary: BrowserStateSummary,
    model_output: AgentOutput | None = None,
    result: list[ActionResult] | None = None,
    step_info: AgentStepInfo | None = None,
    use_vision: bool | Literal['auto'] = True,
    page_filtered_actions: str | None = None,
    sensitive_data=None,
    available_file_paths: list[str] | None = None, # Always pass current available_file_paths
    unavailable_skills_info: str | None = None, # Information about skills that cannot be used yet
    plan_description: str | None = None, # Rendered plan for injection into agent state
    skip_state_update: bool = False,
) -> None:
    """Create single state message with all content"""

    if not skip_state_update:
        self.prepare_step_state(
            browser_state_summary=browser_state_summary,
            model_output=model_output,
            result=result,
            step_info=step_info,
            sensitive_data=sensitive_data,
        )

    # Use only the current screenshot, but check if action results request screenshot inclusion
    screenshots = []
    include_screenshot_requested = False

    # Check if any action results request screenshot inclusion
    if result:
        for action_result in result:
            if action_result.metadata and action_result.metadata.get('include_screenshot'):
                include_screenshot_requested = True
                logger.debug('Screenshot inclusion requested by action result')
                break

    # Handle different use_vision modes:
    # - "auto": Only include screenshot if explicitly requested by action (e.g., screenshot)
    # - True: Always include screenshot
    # - False: Never include screenshot
    include_screenshot = False

```

```

    if use_vision is True:
        # Always include screenshot when use_vision=True
        include_screenshot = True
    elif use_vision == 'auto':
        # Only include screenshot if explicitly requested by action when use_vision="auto"
        include_screenshot = include_screenshot_requested
    # else: use_vision is False, never include screenshot (include_screenshot stays False)

    if include_screenshot and browser_state_summary.screenshot:
        screenshots.append(browser_state_summary.screenshot)

    # Use vision in the user message if screenshots are included
    effective_use_vision = len(screenshots) > 0

    # Create single state message with all content
    assert browser_state_summary
    state_message = AgentMessagePrompt(
        browser_state_summary=browser_state_summary,
        file_system=self.file_system,
        agent_history_description=self.agent_history_description,
        read_state_description=self.state.read_state_description,
        task=self.task,
        include_attributes=self.include_attributes,
        step_info=step_info,
        page_filtered_actions=page_filtered_actions,
        max_clickable_elements_length=self.max_clickable_elements_length,
        sensitive_data=self.sensitive_data_description,
        available_file_paths=available_file_paths,
        screenshots=screenshots,
        vision_detail_level=self.vision_detail_level,
        include_recent_events=self.include_recent_events,
        sample_images=self.sample_images,
        read_state_images=self.state.read_state_images,
        llm_screenshot_size=self.llm_screenshot_size,
        unavailable_skills_info=unavailable_skills_info,
        plan_description=plan_description,
    ).get_user_message(effective_use_vision)

    # Store state message text for history
    self.last_state_message_text = state_message.text

    # Set the state message with caching enabled
    self._set_message_with_type(state_message, 'state')

    def _log_history_lines(self) -> str:
        """Generate a formatted log string of message history for debugging / printing to terminal"""
        # TODO: fix logging

    # try:
    #     total_input_tokens = 0
    #     message_lines = []
    #     terminal_width = shutil.get_terminal_size((80, 20)).columns

    #     for i, m in enumerate(self.state.history.messages):
    #         try:
    #             total_input_tokens += m.metadata.tokens
    #             is_last_message = i == len(self.state.history.messages) - 1

    #             # Extract content for logging
    #             content = _log_extract_message_content(m.message, is_last_message, m.metadata)

    #             # Format the message line(s)
    #             lines = _log_format_message_line(m, content, is_last_message, terminal_width)
    #             message_lines.extend(lines)
    #         except Exception as e:
    #             logger.warning(f'Failed to format message {i} for logging: {e}')
    #             # Add a fallback line for this message
    #             message_lines.append('█[  ?]: [Error formatting this message]')

    #     # Build final log message
    #     return (
    #         f'█ LLM Message history ({len(self.state.history.messages)} messages, {total_input_tokens} tokens'

```



```

    """# """+ '\n'.join(message_lines)
    """# """
    """# except Exception as e:
    """# """logger.warning(f'Failed to generate history log: {e}')
    """# """# Return a minimal fallback message
    """# """# Return f'""" LLM Message history (error generating log: {e})'

    """return ''

    @time_execution_sync('--get_messages')
    def get_messages(self) -> list[BaseMessage]:
        """"""Get current message list, potentially trimmed to max tokens"""

        """# Log message history for debugging
        """logger.debug(self._log_history_lines())
        """self.last_input_messages = self.state.history.get_messages()
        """return self.last_input_messages

    def _set_message_with_type(self, message: BaseMessage, message_type: Literal['system', 'state']) -> None:
        """"""Replace a specific state message slot with a new message"""
        """# System messages don't need filtering - they only contain instructions/placeholders
        """# State messages need filtering - they include agent_history_description which contains
        """# action results with real sensitive values (after placeholder replacement during execution)
        """if message_type == 'system':
        """self.state.history.system_message = message
        """elif message_type == 'state':
        """if self.sensitive_data:
        """message = self._filter_sensitive_data(message)
        """self.state.history.state_message = message
        """else:
        """raise ValueError(f'Invalid state message type: {message_type}')

    def _add_context_message(self, message: BaseMessage) -> None:
        """"""Add a contextual message specific to this step (e.g., validation errors, retry instructions, timeouts)
        """# Context messages typically contain error messages and validation info, not action results
        """# with sensitive data, so filtering is not needed here
        """self.state.history.context_messages.append(message)

    @time_execution_sync('--filter_sensitive_data')
    def _filter_sensitive_data(self, message: BaseMessage) -> BaseMessage:
        """"""Filter out sensitive data from the message"""

    def replace_sensitive(value: str) -> str:
        """if not self.sensitive_data:
        """return value

    """# Collect all sensitive values, immediately converting old format to new format
    """sensitive_values: dict[str, str] = {}

    """# Process all sensitive data entries
    """for key_or_domain, content in self.sensitive_data.items():
    """if isinstance(content, dict):
    """# Already in new format: {domain: {key: value}}
    """for key, val in content.items():
    """if val: # Skip empty values
    """sensitive_values[key] = val
    """elif content: # Old format: {key: value} - convert to new format internally
    """# We treat this as if it was {'http://*': {key_or_domain: content}}
    """sensitive_values[key_or_domain] = content

    """# If there are no valid sensitive data entries, just return the original value
    """if not sensitive_values:
    """logger.warning('No valid entries found in sensitive_data dictionary')
    """return value

    """# Replace all valid sensitive data values with their placeholder tags
    """for key, val in sensitive_values.items():
    """value = value.replace(val, f'<secret>{key}</secret>')

    """return value

    """if isinstance(message.content, str):

```

```

        message.content = replace_sensitive(message.content)
    elif isinstance(message.content, list):
        for i, item in enumerate(message.content):
            if isinstance(item, ContentPartTextParam):
                item.text = replace_sensitive(item.text)
            message.content[i] = item
    return message

```

`.\system\agent\message_manager\utils.py`

```

from __future__ import annotations

import json
import logging
from pathlib import Path
from typing import Any

import anyio

from system.llm.messages import BaseMessage

logger = logging.getLogger(__name__)

async def save_conversation(
    input_messages: list[BaseMessage],
    response: Any,
    target: str | Path,
    encoding: str | None = None,
) -> None:
    """Save conversation history to file asynchronously."""
    target_path = Path(target)
    # create folders if not exists
    if target_path.parent:
        await anyio.Path(target_path.parent).mkdir(parents=True, exist_ok=True)

    await anyio.Path(target_path).write_text(
        await _format_conversation(input_messages, response),
        encoding=encoding or 'utf-8',
    )

async def _format_conversation(messages: list[BaseMessage], response: Any) -> str:
    """Format the conversation including messages and response."""
    lines = []

    # Format messages
    for message in messages:
        lines.append(f' {message.role} ')

        lines.append(message.text)
        lines.append('') # Empty line after each message

    # Format response
    lines.append(json.dumps(json.loads(response.model_dump_json(exclude_unset=True)), indent=2, ensure_ascii=False))

    return '\n'.join(lines)

# Note: _write_messages_to_file and _write_response_to_file have been merged into _format_conversation
# This is more efficient for async operations and reduces file I/O

```

`.\system\agent\message_manager\views.py`

```

from __future__ import annotations

from typing import TYPE_CHECKING, Any

from pydantic import BaseModel, ConfigDict, Field

from system.llm.messages import (
    BaseMessage,
)

if TYPE_CHECKING:
    pass

class HistoryItem(BaseModel):
    """Represents a single agent history item with its data and string representation"""

    step_number: int | None = None
    evaluation_previous_goal: str | None = None
    memory: str | None = None
    next_goal: str | None = None
    action_results: str | None = None
    error: str | None = None
    system_message: str | None = None

    model_config = ConfigDict(arbitrary_types_allowed=True)

    def model_post_init(self, __context) -> None:
        """Validate that error and system_message are not both provided"""
        if self.error is not None and self.system_message is not None:
            raise ValueError('Cannot have both error and system_message at the same time')

    def to_string(self) -> str:
        """Get string representation of the history item"""
        step_str = 'step' if self.step_number is not None else 'step_unknown'

        if self.error:
            return f"""<{step_str}>
{self.error}"""
        elif self.system_message:
            return self.system_message
        else:
            content_parts = []

            # Only include evaluation_previous_goal if it's not None/empty
            if self.evaluation_previous_goal:
                content_parts.append(f'{self.evaluation_previous_goal}')

            # Always include memory
            if self.memory:
                content_parts.append(f'{self.memory}')

            # Only include next_goal if it's not None/empty
            if self.next_goal:
                content_parts.append(f'{self.next_goal}')

            if self.action_results:
                content_parts.append(self.action_results)

            content = '\n'.join(content_parts)

            return f"""<{step_str}>
{content}"""

class MessageHistory(BaseModel):
    """History of messages"""

    system_message: BaseMessage | None = None
    state_message: BaseMessage | None = None
    context_messages: list[BaseMessage] = Field(default_factory=list)
    model_config = ConfigDict(arbitrary_types_allowed=True)

```

```

def get_messages(self) -> list[BaseMessage]:
    """Get all messages in the correct order: system -> state -> contextual"""
    messages = []
    if self.system_message:
        messages.append(self.system_message)
    if self.state_message:
        messages.append(self.state_message)
    messages.extend(self.context_messages)

    return messages

class MessageManagerState(BaseModel):
    """Holds the state for MessageManager"""

    history: MessageHistory = Field(default_factory=MessageHistory)
    tool_id: int = 1
    agent_history_items: list[HistoryItem] = Field(
        default_factory=lambda: [HistoryItem(step_number=0, system_message='Agent initialized')]
    )
    read_state_description: str = ''
    # Images to include in the next state message (cleared after each step)
    read_state_images: list[dict[str, Any]] = Field(default_factory=list)
    compacted_memory: str | None = None
    compaction_count: int = 0
    last_compaction_step: int | None = None

    model_config = ConfigDict(arbitrary_types_allowed=True)

```

.lsystem\agent\system_prompts__init__.py

```
# System prompt templates for browser-use agent
```

.lsystem\browser\demo_mode.py

```

"""Demo mode helper for injecting and updating the in-browser log panel."""

from __future__ import annotations

import asyncio
import json
import logging
from datetime import datetime, timezone
from typing import Any

from system.browser.session import BrowserSession

# Embedded JavaScript for demo panel (injected into browser pages)
_DEMO_PANEL_SCRIPT = r"""(function () {
    // SESSION_ID_PLACEHOLDER will be replaced by DemoMode with actual session ID
    const SESSION_ID = '__BROWSER_USE_SESSION_ID_PLACEHOLDER__';
    const EXCLUDE_ATTR = 'data-browser-use-exclude-' + SESSION_ID;
    const PANEL_ID = 'browser-use-demo-panel';
    const STYLE_ID = 'browser-use-demo-panel-style';
    const STORAGE_KEY = '__browserUseDemoLogs__';
    const STORAGE_HTML_KEY = '__browserUseDemoLogsHTML__';
    const PANEL_STATE_KEY = '__browserUseDemoPanelState__';
    const TOGGLE_BUTTON_ID = 'browser-use-demo-toggle';
    const MAX_MESSAGES = 100;
    const EXPANDED_IDS_KEY = '__browserUseExpandedEntries__';
    const LEVEL_ICONS = {
        info: '■',
        action: '■',
        thought: '■',
    }
}
)"""

```

```

    success: '■',
    warning: '■■',
    error: '■■■',
  };
  const LEVEL_LABELS = {
    info: 'info',
    action: 'action',
    thought: 'thought',
    success: 'success',
    warning: 'warning',
    error: 'error',
  };

  if (window.__browserUseDemoPanelLoaded) {
    const existingPanel = document.getElementById(PANEL_ID);
    if (!existingPanel) {
      initializePanel();
    }
    return;
  }
  window.__browserUseDemoPanelLoaded = true;

  const state = {
    panel: null,
    list: null,
    messages: [],
    isOpen: true,
    toggleButton: null,
  };
  state.messages = restoreMessages();

  function initializePanel() {
    console.log('Browser-use demo panel initialized with session ID:', SESSION_ID);
    addStyles();
    state.isOpen = loadPanelState();
    state.panel = buildPanel();
    state.list = state.panel.querySelector('[data-role="log-list"]');
    appendToHost(state.panel);
    state.toggleButton = buildToggleButton();
    appendToHost(state.toggleButton);
    const savedWidth = loadPanelWidth();
    if (savedWidth) {
      document.documentElement.style.setProperty('--browser-use-demo-panel-width', `${savedWidth}px`);
    }

    if (!hydrateFromStoredMarkup()) {
      state.messages.forEach((entry) => appendEntry(entry, false));
    }
    attachCloseHandler();
    if (state.isOpen) {
      openPanel(false);
    } else {
      closePanel(false);
    }
    adjustLayout();
    window.addEventListener('resize', debounce(adjustLayout, 150));
  }

  function appendToHost(node) {
    if (!node) {
      return;
    }

    const host = document.body || document.documentElement;
    if (!host.contains(node)) {
      host.appendChild(node);
    }

    if (!document.body) {
      document.addEventListener(
        'DOMContentLoaded',
        () => {

```

```

        if (document.body && node.parentNode !== document.body) {
            document.body.appendChild(node);
        }
    },
    { once: true }
);
}
}

function addStyles() {
    if (document.getElementById(STYLE_ID)) {
        return;
    }
    const style = document.createElement('style');
    style.id = STYLE_ID;
    style.setAttribute(EXCLUDE_ATTR, 'true');
    style.textContent = `
        #${PANEL_ID} {
            position: fixed;
            top: 0;
            right: 0;
            width: var(--browser-use-demo-panel-width, 340px);
            max-width: calc(100vw - 64px);
            height: 100vh;
            display: flex;
            flex-direction: column;
            background: #05070d;
            color: #f8f9ff;
            font-family: 'JetBrains Mono', 'Fira Code', 'Monaco', 'Menlo', monospace;
            font-size: 13px;
            line-height: 1.4;
            box-shadow: -6px 0 25px rgba(0, 0, 0, 0.35);
            z-index: 2147480000;
            border-left: 1px solid rgba(255, 255, 255, 0.14);
            backdrop-filter: blur(10px);
            pointer-events: auto;
            transform: translateX(0);
            opacity: 1;
            transition: transform 0.25s ease, opacity 0.25s ease;
        }

        #${PANEL_ID}[data-open="false"] {
            transform: translateX(110%);
            opacity: 0;
            pointer-events: none;
        }

        #${PANEL_ID} .browser-use-demo-header {
            padding: 16px 18px 12px;
            border-bottom: 1px solid rgba(255, 255, 255, 0.14);
            display: flex;
            align-items: baseline;
            justify-content: space-between;
            gap: 8px;
            flex-wrap: wrap;
        }

        #${PANEL_ID} .browser-use-demo-header h1 {
            font-size: 15px;
            text-transform: uppercase;
            letter-spacing: 0.12em;
            margin: 0;
            color: #f8f9ff;
        }

        #${PANEL_ID} .browser-use-badge {
            font-size: 11px;
            padding: 2px 10px;
            border-radius: 999px;
            border: 1px solid rgba(255, 255, 255, 0.4);
            text-transform: uppercase;
            letter-spacing: 0.08em;
    `;
}

```

```

    color: #f8f9ff;
}

#{$PANEL_ID} .browser-use-logo img {
  height: 36px;
}

#{$PANEL_ID} .browser-use-header-actions {
  margin-left: auto;
  display: flex;
  align-items: center;
  gap: 8px;
}

#{$PANEL_ID} .browser-use-close-btn {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.05);
  color: #f8f9ff;
  cursor: pointer;
  font-size: 16px;
  line-height: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: background 0.2s ease, border 0.2s ease;
}

#{$PANEL_ID} .browser-use-close-btn:hover {
  background: rgba(255, 255, 255, 0.15);
  border-color: rgba(255, 255, 255, 0.35);
}

#{$PANEL_ID} .browser-use-demo-body {
  flex: 1;
  overflow-y: auto;
  scrollbar-width: thin;
  scrollbar-color: rgba(255, 255, 255, 0.3) transparent;
  padding: 8px 0 12px;
}

#{$PANEL_ID} .browser-use-demo-body::-webkit-scrollbar {
  width: 8px;
}

#{$PANEL_ID} .browser-use-demo-body::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.25);
  border-radius: 999px;
}

.browser-use-demo-entry {
  display: flex;
  gap: 12px;
  padding: 10px 18px;
  border-left: 2px solid transparent;
  border-bottom: 1px solid rgba(255, 255, 255, 0.04);
  animation: browser-use-fade-in 0.25s ease;
  background: #000000;
}

.browser-use-demo-entry:last-child {
  border-bottom-color: transparent;
}

.browser-use-entry-icon {
  font-size: 16px;
  line-height: 1.2;
  width: 20px;
}

.browser-use-entry-content {

```

```

    flex: 1;
    min-width: 0;
}

.browser-use-entry-meta {
    font-size: 11px;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    color: white;
    margin-bottom: 4px;
    display: flex;
    justify-content: space-between;
    gap: 12px;
}

.browser-use-entry-message {
    margin: 0;
    word-break: break-word;
    font-size: 12px;
    color: #f8f9ff;
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.browser-use-markdown-content {
    margin: 0;
    line-height: 1.5;
}

.browser-use-markdown-content p {
    margin: 0 0 8px 0;
}

.browser-use-markdown-content p:last-child {
    margin-bottom: 0;
}

.browser-use-markdown-content h1,
.browser-use-markdown-content h2,
.browser-use-markdown-content h3 {
    margin: 8px 0 4px 0;
    font-weight: 600;
    color: #f8f9ff;
}

.browser-use-markdown-content h1 {
    font-size: 16px;
}

.browser-use-markdown-content h2 {
    font-size: 14px;
}

.browser-use-markdown-content h3 {
    font-size: 13px;
}

.browser-use-markdown-content code {
    background: rgba(255, 255, 255, 0.1);
    padding: 2px 6px;
    border-radius: 3px;
    font-family: 'JetBrains Mono', 'Fira Code', 'Monaco', 'Menlo', monospace;
    font-size: 11px;
    color: #60a5fa;
}

.browser-use-markdown-content pre {
    background: rgba(0, 0, 0, 0.3);
    padding: 8px 12px;
    border-radius: 4px;
    overflow-x: auto;
}

```



```

    margin: 8px 0;
    border: 1px solid rgba(255, 255, 255, 0.1);
}

.browser-use-markdown-content pre code {
    background: transparent;
    padding: 0;
    color: #f8f9ff;
    font-size: 11px;
    white-space: pre;
}

.browser-use-markdown-content ul,
.browser-use-markdown-content ol {
    margin: 4px 0 4px 16px;
    padding: 0;
}

.browser-use-markdown-content li {
    margin: 2px 0;
}

.browser-use-markdown-content a {
    color: #60a5fa;
    text-decoration: underline;
}

.browser-use-markdown-content a:hover {
    color: #93c5fd;
}

.browser-use-markdown-content strong {
    font-weight: 600;
    color: #f8f9ff;
}

.browser-use-markdown-content em {
    font-style: italic;
}

.browser-use-demo-entry:not(.expanded) .browser-use-markdown-content {
    max-height: 120px;
    overflow: hidden;
    mask-image: linear-gradient(to bottom, rgba(0,0,0,1), rgba(0,0,0,0));
}

.browser-use-entry-toggle {
    align-self: flex-start;
    background: rgba(255, 255, 255, 0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);
    color: #f8f9ff;
    padding: 2px 10px;
    font-size: 11px;
    border-radius: 999px;
    cursor: pointer;
}

.browser-use-demo-entry.level-info { border-left-color: #60a5fa; }
.browser-use-demo-entry.level-action { border-left-color: #34d399; }
.browser-use-demo-entry.level-thought { border-left-color: #f97316; }
.browser-use-demo-entry.level-warning { border-left-color: #fbbf24; }
.browser-use-demo-entry.level-success { border-left-color: #22c55e; }
.browser-use-demo-entry.level-error { border-left-color: #f87171; }

@keyframes browser-use-fade-in {
    from { opacity: 0; transform: translateY(6px); }
    to { opacity: 1; transform: translateY(0); }
}

@media (max-width: 1024px) {
    #${PANEL_ID} {
        font-size: 12px;
    }
}

```

```

    }
    #${PANEL_ID} .browser-use-demo-header {
        padding: 12px 16px 10px;
    }
}

#${TOGGLE_BUTTON_ID} {
    position: fixed;
    top: 20px;
    right: 20px;
    width: 44px;
    height: 44px;
    border-radius: 50%;
    border: 1px solid rgba(255, 255, 255, 0.2);
    background: rgba(5, 7, 13, 0.92);
    color: #f8f9ff;
    font-size: 18px;
    display: none;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    z-index: 2147480001;
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.4);
    transition: transform 0.2s ease, background 0.2s ease;
}

#${TOGGLE_BUTTON_ID}:hover {
    transform: scale(1.05);
    background: rgba(5, 7, 13, 0.98);
}

#${TOGGLE_BUTTON_ID} img {
    display: block;
    width: 24px;
    height: auto;
    max-width: 100%;
    max-height: 100%;
    object-fit: contain;
    pointer-events: none;
    user-select: none;
}
`;
document.head.appendChild(style);
}

function buildPanel() {
    const panel = document.createElement('section');
    panel.id = PANEL_ID;
    panel.setAttribute('role', 'complementary');
    panel.setAttribute('aria-label', 'Browser-use demo panel');
    panel.setAttribute(EXCLUDE_ATTR, 'true');

    const header = document.createElement('header');
    header.className = 'browser-use-demo-header';
    const title = document.createElement('div');
    title.className = 'browser-use-logo';
    const logo = document.createElement('img');
    logo.src = 'https://raw.githubusercontent.com/browser-use/browser-use/main/static/browser-use-dark.png';
    logo.alt = 'Browser-use';
    logo.loading = 'lazy';
    title.appendChild(logo);
    const actions = document.createElement('div');
    actions.className = 'browser-use-header-actions';
    const closeBtn = document.createElement('button');
    closeBtn.type = 'button';
    closeBtn.className = 'browser-use-close-btn';
    closeBtn.setAttribute(EXCLUDE_ATTR, 'true');
    closeBtn.setAttribute('aria-label', 'Hide demo panel');
    closeBtn.dataset.role = 'close-toggle';
    closeBtn.innerHTML = '&times;';
    actions.appendChild(closeBtn);
    header.appendChild(title);

```

```

    header.appendChild(actions);

    const body = document.createElement('div');
    body.className = 'browser-use-demo-body';
    body.setAttribute('data-role', 'log-list');

    panel.appendChild(header);
    panel.appendChild(body);
    panel.setAttribute('data-open', 'true');
    return panel;
}

function buildToggleButton() {
    const button = document.createElement('button');
    button.id = TOGGLE_BUTTON_ID;
    button.type = 'button';
    button.setAttribute(EXCLUDE_ATTR, 'true');
    button.setAttribute('aria-label', 'Open demo panel');
    const img = document.createElement('img');
    img.alt = 'Browser-use';
    img.loading = 'eager';
    // Use embedded SVG as data URI to avoid CSP issues
    const logoSvg = '<svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" fill="none" viewBox=
img.src = 'data:image/svg+xml; charset=utf-8,' + encodeURIComponent(logoSvg);
    button.appendChild(img);
    button.addEventListener('click', () => openPanel(true));
    return button;
}

function attachCloseHandler() {
    const closeBtn = state.panel?.querySelector('[data-role="close-toggle"]');
    if (closeBtn) {
        closeBtn.addEventListener('click', () => closePanel(true));
    }
}

function openPanel(saveState = true) {
    state.isOpen = true;
    if (state.panel) {
        state.panel.setAttribute('data-open', 'true');
    }
    if (state.toggleButton) {
        state.toggleButton.style.display = 'none';
    }
    adjustLayout();
    if (saveState) {
        persistPanelState();
    }
}

function closePanel(saveState = true) {
    state.isOpen = false;
    if (state.panel) {
        state.panel.setAttribute('data-open', 'false');
    }
    document.body.style.marginRight = '';
    if (state.toggleButton) {
        state.toggleButton.style.display = 'flex';
    }
    if (saveState) {
        persistPanelState();
    }
}

function persistPanelState() {
    try {
        sessionStorage.setItem(PANEL_STATE_KEY, state.isOpen ? 'open' : 'closed');
    } catch (err) {
        // Ignore storage errors
    }
}

function loadPanelState() {

```

```

    try {
      const stored = sessionStorage.getItem(PANEL_STATE_KEY);
      if (!stored) return true;
      return stored === 'open';
    } catch (err) {
      return true;
    }
  }

function adjustLayout() {
  const width = computePanelWidth();
  document.documentElement.style.setProperty('--browser-use-demo-panel-width', `${width}px`);
  if (state.isOpen) {
    document.body.style.marginRight = `${width + 16}px`;
    if (state.toggleButton) {
      state.toggleButton.style.display = 'none';
    }
  } else {
    document.body.style.marginRight = '';
    if (state.toggleButton) {
      state.toggleButton.style.display = 'flex';
    }
  }
}

function computePanelWidth() {
  const viewport = Math.max(window.innerWidth, 320);
  const maxAvailable = Math.max(220, viewport - 240);
  const target = Math.min(380, Math.max(260, viewport * 0.3));
  const width = Math.max(220, Math.min(target, maxAvailable));
  try {
    sessionStorage.setItem('__browserUsePanelWidth__', String(width));
  } catch {
    // fallback
  }
  return width;
}

function loadPanelWidth() {
  try {
    const saved = sessionStorage.getItem('__browserUsePanelWidth__');
    return saved ? Number(saved) : null;
  } catch {
    return null;
  }
}

function restoreMessages() {
  try {
    const raw = sessionStorage.getItem(STORAGE_KEY);
    if (!raw) return [];
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch (err) {
    return [];
  }
}

function persistMessages() {
  try {
    sessionStorage.setItem(STORAGE_KEY, JSON.stringify(state.messages.slice(-MAX_MESSAGES)));
    if (state.list) {
      sessionStorage.setItem(STORAGE_HTML_KEY, state.list.innerHTML);
    }
  } catch (err) {
    // Ignore sessionStorage errors (private mode, etc.)
  }
}

function hydrateFromStoredMarkup() {
  if (!state.list) return false;
  try {

```

```

const html = sessionStorage.getItem(STORAGE_HTML_KEY);
if (html) {
  state.list.innerHTML = html;
  for (const entryNode of state.list.querySelectorAll('.browser-use-demo-entry')) {
    const toggle = entryNode.querySelector('.browser-use-entry-toggle');
    if (toggle) {
      toggle.addEventListener('click', () =>
        toggleEntryExpansion(entryNode, toggle, entryNode.getAttribute('data-id'))
      );
    }
    applyPersistedExpansion(entryNode);
  }
  state.list.scrollTop = state.list.scrollHeight;
  return true;
}
} catch (err) {
  // ignore hydration failures
}
return false;
}

function normalizeEntry(detail) {
  if (!detail) return null;
  const entry = typeof detail === 'string' ? { message: detail } : { ...detail };
  entry.message = typeof entry.message === 'string' ? entry.message : JSON.stringify(entry.message ??);
  entry.level = (entry.level || 'info').toLowerCase();
  if (!LEVEL_ICONS[entry.level]) {
    entry.level = 'info';
  }

  if (!entry.metadata || typeof entry.metadata !== 'object') {
    entry.metadata = {};
  }

  entry.timestamp = entry.timestamp || new Date().toISOString();
  entry.id = entry.id || `${Date.now()}-${Math.random().toString(16).slice(2)}`;
  return entry;
}

function appendEntry(entry, shouldPersist = true) {
  if (shouldPersist) {
    state.messages.push(entry);
    if (state.messages.length > MAX_MESSAGES) {
      state.messages = state.messages.slice(-MAX_MESSAGES);
    }
    persistMessages();
  }

  if (!state.list) {
    return;
  }

  const node = createEntryNode(entry);
  applyPersistedExpansion(node);
  state.list.appendChild(node);
  state.list.scrollTop = state.list.scrollHeight;
}

function createEntryNode(entry) {
  const row = document.createElement('article');
  row.className = `browser-use-demo-entry level-${entry.level}`;
  row.setAttribute('data-id', entry.id);

  const icon = document.createElement('span');
  icon.className = 'browser-use-entry-icon';
  icon.textContent = LEVEL_ICONS[entry.level] || LEVEL_ICONS.info;

  const content = document.createElement('div');
  content.className = 'browser-use-entry-content';

  const meta = document.createElement('div');
  meta.className = 'browser-use-entry-meta';

```

```

const time = formatTime(entry.timestamp);
const label = LEVEL_LABELS[entry.level] || entry.level;
meta.innerHTML = `<span>${time}</span><span>${label}</span>`;

const messageWrapper = document.createElement('div');
messageWrapper.className = 'browser-use-entry-message';
const messageText = entry.message.trim();
const messageHtml = messageText;
const message = document.createElement('div');
message.className = 'browser-use-markdown-content';
message.textContent = messageHtml;
messageWrapper.appendChild(message);

if (messageText.length > 160) {
  const toggle = document.createElement('button');
  toggle.type = 'button';
  toggle.className = 'browser-use-entry-toggle';
  toggle.setAttribute(EXCLUDE_ATTR, 'true');
  toggle.textContent = 'Expand';
  toggle.addEventListener('click', () => toggleEntryExpansion(row, toggle, entry.id));
  messageWrapper.appendChild(toggle);
} else {
  row.classList.add('expanded');
}

content.appendChild(meta);
content.appendChild(messageWrapper);
row.appendChild(icon);
row.appendChild(content);
return row;
}

function applyPersistedExpansion(node) {
  if (!node) return;
  try {
    const expanded = new Set(JSON.parse(sessionStorage.getItem(EXPANDED_IDS_KEY) || '[]'));
    const id = node.getAttribute('data-id');
    if (id && expanded.has(id)) {
      node.classList.add('expanded');
      const toggle = node.querySelector('.browser-use-entry-toggle');
      if (toggle) {
        toggle.textContent = 'Collapse';
      }
    }
  } catch {
    // ignore
  }
}

function toggleEntryExpansion(row, toggle, entryId) {
  if (!row) return;
  const isExpanded = row.classList.toggle('expanded');
  if (toggle) {
    toggle.textContent = isExpanded ? 'Collapse' : 'Expand';
  }
  try {
    const expanded = new Set(JSON.parse(sessionStorage.getItem(EXPANDED_IDS_KEY) || '[]'));
    if (isExpanded) {
      expanded.add(entryId);
    } else {
      expanded.delete(entryId);
    }
    sessionStorage.setItem(EXPANDED_IDS_KEY, JSON.stringify(Array.from(expanded)));
  } catch {
    // ignore persistence issues
  }
}

function formatTime(value) {
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) {
    return new Date().toLocaleTimeString();
  }
}

```

```

    }
    return date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });
}

function debounce(fn, delay) {
    let frame;
    return (...args) => {
        cancelAnimationFrame(frame);
        frame = requestAnimationFrame(() => fn.apply(null, args));
    };
}

function handleLogEvent(event) {
    const entry = normalizeEntry(event?.detail);
    if (!entry) return;
    appendEntry(entry, true);
}

const boot = () => {
    if (window.__browserUseDemoPanelBootstrapped) {
        return;
    }

    const start = () => {
        if (window.__browserUseDemoPanelBootstrapped) {
            return;
        }
        if (!document.body) {
            requestAnimationFrame(start);
            return;
        }
        window.__browserUseDemoPanelBootstrapped = true;
        initializePanel();
    };

    start();
};

if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', boot, { once: true });
} else {
    boot();
}
window.addEventListener('browser-use-log', handleLogEvent);
})();
"""

```

```

class DemoMode:

```

```

    """Encapsulates browser overlay injection and log broadcasting for demo mode."""

```

```

    VALID_LEVELS = {'info', 'action', 'thought', 'error', 'success', 'warning'}

```

```

    def __init__(self, session: BrowserSession):

```

```

        self.session = session

```

```

        self.logger = logging.getLogger(f'{{__name__}}.DemoMode')

```

```

        self._script_identifier: str | None = None

```

```

        self._script_source: str | None = None

```

```

        self._panel_ready = False

```

```

        self._lock = asyncio.Lock()

```

```

    def reset(self) -> None:

```

```

        self._script_identifier = None

```

```

        self._panel_ready = False

```

```

    def _load_script(self) -> str:

```

```

        if self._script_source is None:

```

```

            self._script_source = _DEMO_PANEL_SCRIPT

```

```

        # Replace placeholder with actual session ID

```

```

        session_id = self.session.id

```

```

        script_with_session_id = self._script_source.replace('__BROWSER_USE_SESSION_ID_PLACEHOLDER__', session_id)

```

```

    self.logger.debug(f'Injecting session ID {session_id} into demo panel script')
    return script_with_session_id

    async def ensure_ready(self) -> None:
        """Add init script and inject overlay into currently open pages."""
        if not self.session.browser_profile.demo_mode:
            return
        if self.session._cdp_client_root is None:
            raise RuntimeError('Root CDP client not initialized')

        async with self._lock:
            script = self._load_script()

            if self._script_identifier is None:
                self._script_identifier = await self.session._cdp_add_init_script(script)
                self.logger.debug('Added auto-injection script for demo overlay')

            await self._inject_into_open_pages(script)
            self._panel_ready = True
            self.logger.debug('Demo overlay injected successfully')

        async def send_log(self, message: str, level: str = 'info', metadata: dict[str, Any] | None = None) -> None:
            """Send a log entry to the in-browser panel."""
            if not message or not self.session.browser_profile.demo_mode:
                return

            try:
                await self.ensure_ready()
            except Exception as exc:
                self.logger.warning(f'Failed to ensure demo mode is ready: {exc}')
                return

            if self.session.agent_focus_target_id is None:
                self.logger.debug('Cannot send demo log: no active target')
                return

            level_value = level.lower()
            if level_value not in self.VALID_LEVELS:
                level_value = 'info'

            payload = {
                'message': message,
                'level': level_value,
                'metadata': metadata or {},
                'timestamp': datetime.now(timezone.utc).isoformat(),
            }

            # --- TTS Transcript Capture ---
            try:
                import os
                # We save to transcript.jsonl in the current working directory.
                # This is an append-only log of every agent action/thought for TTS later.
                transcript_path = os.path.join(os.getcwd(), 'transcript.jsonl')
                with open(transcript_path, 'a', encoding='utf-8') as tf:
                    tf.write(json.dumps(payload, ensure_ascii=False) + '\n')
            except Exception as e:
                self.logger.debug(f'Failed to append to transcript: {e}')
            # -----

            script = self._build_event_expression(json.dumps(payload, ensure_ascii=False))

            try:
                session = await self.session.get_or_create_cdp_session(target_id=None, focus=False)
            except Exception as exc:
                self.logger.debug(f'Cannot acquire CDP session for demo log: {exc}')
                return

            try:
                await session.cdp_client.send.Runtime.evaluate(
                    params={'expression': script, 'awaitPromise': False}, session_id=session.session_id
                )
            except Exception as exc:

```



```

■■■self.logger.debug(f'Failed to send demo log: {exc}')

■def _build_event_expression(self, payload: str) -> str:
■    return f"""
■    (()) => {{
■    ■const detail = {{payload}};
■    ■const event = new CustomEvent('browser-use-log', {{ detail }});
■    ■window.dispatchEvent(event);
■    }}();
■    """.strip()

■async def _inject_into_open_pages(self, script: str) -> None:
■    targets = await self.session._cdp_get_all_pages( # - intentional private access
■■■include_http=True,
■■■include_about=True,
■■■include_pages=True,
■■■include_iframes=False,
■■■include_workers=False,
■■■include_chrome=False,
■■■include_chrome_extensions=False,
■■■include_chrome_error=False,
■■■)

■    target_ids = [t['targetId'] for t in targets]
■    if not target_ids and self.session.agent_focus_target_id:
■■■target_ids = [self.session.agent_focus_target_id]

■    for target_id in target_ids:
■■■try:
■■■■await self._inject_into_target(target_id, script)
■■■except Exception as exc:
■■■■self.logger.debug(f'Failed to inject demo overlay into {target_id}: {exc}')

■async def _inject_into_target(self, target_id: str, script: str) -> None:
■    session = await self.session.get_or_create_cdp_session(target_id=target_id, focus=False)
■    await session.cdp_client.send.Runtime.evaluate(
■■■params={'expression': script, 'awaitPromise': False},
■■■session_id=session.session_id,
■■■)

```

.\system\browser\events.py

```

"""Event definitions for browser communication."""

import inspect
import os
from typing import Any, Literal

from bubus import BaseEvent
from bubus.models import T_EventResultType
from cdp_use.cdp.target import TargetID
from pydantic import BaseModel, Field, field_validator

from system.browser.views import BrowserStateSummary
from system.dom.views import EnhancedDOMTreeNode

def _get_timeout(env_var: str, default: float) -> float | None:
    """
    ■Safely parse environment variable timeout values with robust error handling.

    ■Args:
    ■env_var: Environment variable name (e.g. 'TIMEOUT_NavigateToUrlEvent')
    ■default: Default timeout value as float (e.g. 15.0)

    ■Returns:
    ■Parsed float value or the default if parsing fails
    ■Raises:
    """

```

```

ValueError: Only if both env_var and default are invalid (should not happen with valid defaults)
"""
# Try environment variable first
env_value = os.getenv(env_var)
if env_value:
    try:
        parsed = float(env_value)
        if parsed < 0:
            print(f'Warning: {env_var}={env_value} is negative, using default {default}')
            return default
        return parsed
    except (ValueError, TypeError):
        print(f'Warning: {env_var}={env_value} is not a valid number, using default {default}')

# Fall back to default
return default

```

```

# =====
# Agent/Tools -> BrowserSession Events (High-level browser actions)
# =====

```

```

class ElementSelectedEvent(BaseEvent[T_EventResultType]):

```

```

    """An element was selected."""

```

```

    node: EnhancedDOMTreeNode

```

```

    @field_validator('node', mode='before')

```

```

    @classmethod

```

```

    def serialize_node(cls, data: EnhancedDOMTreeNode | None) -> EnhancedDOMTreeNode | None:

```

```

        if data is None:

```

```

            return None

```

```

        return EnhancedDOMTreeNode(

```

```

            node_id=data.node_id,

```

```

            backend_node_id=data.backend_node_id,

```

```

            session_id=data.session_id,

```

```

            frame_id=data.frame_id,

```

```

            target_id=data.target_id,

```

```

            node_type=data.node_type,

```

```

            node_name=data.node_name,

```

```

            node_value=data.node_value,

```

```

            attributes=data.attributes,

```

```

            is_scrollable=data.is_scrollable,

```

```

            is_visible=data.is_visible,

```

```

            absolute_position=data.absolute_position,

```

```

            # override the circular reference fields in EnhancedDOMTreeNode as they cant be serialized and aren't

```

```

            # only used internally by the DOM service during DOM tree building process, not intended public API

```

```

            content_document=None,

```

```

            shadow_root_type=None,

```

```

            shadow_roots=[],

```

```

            parent_node=None,

```

```

            children_nodes=[],

```

```

            ax_node=None,

```

```

            snapshot_node=None,

```

```

        )

```

```

# TODO: add page handle to events

```

```

# class PageHandle(share a base with browser.session.CDPSession?):

```

```

#     url: str

```

```

#     target_id: TargetID

```

```

#     @classmethod

```

```

#     def from_target_id(cls, target_id: TargetID) -> Self:

```

```

#         return cls(target_id=target_id)

```

```

#     @classmethod

```

```

#     def from_target_id(cls, target_id: TargetID) -> Self:

```

```

#         return cls(target_id=target_id)

```

```

#     @classmethod

```

```

#     def from_url(cls, url: str) -> Self:

```

```

#     @property

```

```

#     def root_frame_id(self) -> str:

```

```

#     return self.target_id
# @property
# def session_id(self) -> str:
#     return browser_session.get_or_create_cdp_session(self.target_id).session_id

# class PageSelectedEvent(BaseEvent[T_EventResultType]):
#     """An event like SwitchToTabEvent(page=PageHandle) or CloseTabEvent(page=PageHandle)"""
#     page: PageHandle

class NavigateToUrlEvent(BaseEvent[None]):
    """Navigate to a specific URL."""

    url: str
    wait_until: Literal['load', 'domcontentloaded', 'networkidle', 'commit'] = 'load'
    timeout_ms: int | None = None
    new_tab: bool = Field(
        default=False, description='Set True to leave the current tab alone and open a new tab in the foreground'
    )
    existing_tab: PageHandle | None = None # TODO

    # time limits enforced by bubus, not exposed to LLM:
    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_NavigateToUrlEvent', 10.0))

class ClickElementEvent(ElementSelectedEvent[dict[str, Any] | None]):
    """Click an element."""

    node: 'EnhancedDOMTreeNode'
    button: Literal['left', 'right', 'middle'] = 'left'
    click_count: int = 1 # TODO
    expect_download: bool = False # moved to downloads_watchdog.py

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_ClickElementEvent', 10.0))

class ClickCoordinateEvent(BaseEvent[dict]):
    """Click at specific coordinates."""

    coordinate_x: int
    coordinate_y: int
    button: Literal['left', 'right', 'middle'] = 'left'
    force: bool = False # If True, skip safety checks (file input, print, select)

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_ClickCoordinateEvent', 10.0))

class TypeTextEvent(ElementSelectedEvent[dict | None]):
    """Type text into an element."""

    node: 'EnhancedDOMTreeNode'
    text: str
    clear: bool = True
    is_sensitive: bool = False # Flag to indicate if text contains sensitive data
    sensitive_key_name: str | None = None # Name of the sensitive key being typed (e.g., 'username', 'password')

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_TypeTextEvent', 60.0))

class ScrollEvent(ElementSelectedEvent[None]):
    """Scroll the page or element."""

    direction: Literal['up', 'down', 'left', 'right']
    amount: int # pixels
    node: 'EnhancedDOMTreeNode | None' = None # None means scroll page

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_ScrollEvent', 8.0))

class SwitchTabEvent(BaseEvent[TargetID]):
    """Switch to a different tab."""
    target_id: TargetID | None = Field(default=None, description='None means switch to the most recently opened tab')

```

```

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_SwitchTabEvent', 10.0))

class CloseTabEvent(BaseEvent[None]):
    """Close a tab."""

    target_id: TargetID

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_CloseTabEvent', 10.0))

class ScreenshotEvent(BaseEvent[str]):
    """Request to take a screenshot."""

    full_page: bool = False
    clip: dict[str, float] | None = None # {x, y, width, height}

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_ScreenshotEvent', 15.0))

class BrowserStateRequestEvent(BaseEvent[BrowserStateSummary]):
    """Request current browser state."""

    include_dom: bool = True
    include_screenshot: bool = True
    include_recent_events: bool = False

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserStateRequestEvent', 10.0))

# class WaitForConditionEvent(BaseEvent):
#     """Wait for a condition."""

#     condition: Literal['navigation', 'selector', 'timeout', 'load_state']
#     timeout: float = 30000
#     selector: str | None = None
#     state: Literal['attached', 'detached', 'visible', 'hidden'] | None = None

class GoBackEvent(BaseEvent[None]):
    """Navigate back in browser history."""

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_GoBackEvent', 15.0))

class GoForwardEvent(BaseEvent[None]):
    """Navigate forward in browser history."""

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_GoForwardEvent', 15.0))

class RefreshEvent(BaseEvent[None]):
    """Refresh/reload the current page."""

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_RefreshEvent', 15.0))

class WaitEvent(BaseEvent[None]):
    """Wait for a specified number of seconds."""

    seconds: float = 3.0
    max_seconds: float = 10.0 # Safety cap

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_WaitEvent', 60.0)) #

class SendKeysEvent(BaseEvent[None]):
    """Send keyboard keys/shortcuts."""

    keys: str # e.g., "ctrl+a", "cmd+c", "Enter"

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_SendKeysEvent', 60.0))

```

```

class UploadFileEvent(ElementSelectedEvent[None]):
    """Upload a file to an element."""

    node: 'EnhancedDOMTreeNode'
    file_path: str

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_UploadFileEvent', 30.0))

class GetDropdownOptionsEvent(ElementSelectedEvent[dict[str, str]]):
    """Get all options from any dropdown (native <select>, ARIA menus, or custom dropdowns).

    Returns a dict containing dropdown type, options list, and element metadata."""

    node: 'EnhancedDOMTreeNode'

    event_timeout: float | None = Field(
        default_factory=lambda: _get_timeout('TIMEOUT_GetDropdownOptionsEvent', 15.0)
    ) # some dropdowns lazy-load the list of options on first interaction, so we need to wait for them to

class SelectDropdownOptionEvent(ElementSelectedEvent[dict[str, str]]):
    """Select a dropdown option by exact text from any dropdown type.

    Returns a dict containing success status and selection details."""

    node: 'EnhancedDOMTreeNode'
    text: str # The option text to select

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_SelectDropdownOptionEvent', 15.0))

class ScrollToTextEvent(BaseEvent[None]):
    """Scroll to specific text on the page. Raises exception if text not found."""

    text: str
    direction: Literal['up', 'down'] = 'down'

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_ScrollToTextEvent', 15.0))

# =====

class BrowserStartEvent(BaseEvent):
    """Start/connect to browser."""

    cdp_url: str | None = None
    launch_options: dict[str, Any] = Field(default_factory=dict)

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserStartEvent', 30.0))

class BrowserStopEvent(BaseEvent):
    """Stop/disconnect from browser."""

    force: bool = False

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserStopEvent', 45.0))

class BrowserLaunchResult(BaseModel):
    """Result of launching a browser."""

    # TODO: add browser executable_path, pid, version, latency, user_data_dir, X11 $DISPLAY, host IP address, etc.
    cdp_url: str

class BrowserLaunchEvent(BaseEvent[BrowserLaunchResult]):
    """Launch a local browser process."""

    # TODO: add executable_path, proxy settings, preferences, extra launch args, etc.

```

```

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserLaunchEvent', 30.0))

class BrowserKillEvent(BaseEvent):
    ■"""Kill local browser subprocess."""

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserKillEvent', 30.0))

# TODO: replace all Runtime.evaluate() calls with this event
# class ExecuteJavaScriptEvent(BaseEvent):
#     ■"""Execute JavaScript in page context."""

#     ■target_id: TargetID
#     ■expression: str
#     ■await_promise: bool = True

#     ■event_timeout: float | None = 60.0 # seconds

# TODO: add this and use the old BrowserProfile.viewport options to set it
# class SetViewportEvent(BaseEvent):
#     ■"""Set the viewport size."""

#     ■width: int
#     ■height: int
#     ■device_scale_factor: float = 1.0

#     ■event_timeout: float | None = 15.0 # seconds

# Moved to storage state
# class SetCookiesEvent(BaseEvent):
#     ■"""Set browser cookies."""

#     ■cookies: list[dict[str, Any]]

#     ■event_timeout: float | None = (
#     ■■30.0 # only long to support the edge case of restoring a big localStorage / on many origins (has to be long)
#     ■)

# class GetCookiesEvent(BaseEvent):
#     ■"""Get browser cookies."""

#     ■urls: list[str] | None = None

#     ■event_timeout: float | None = 30.0 # seconds

# =====
# DOM-related Events
# =====

class BrowserConnectedEvent(BaseEvent):
    ■"""Browser has started/connected."""

    ■cdp_url: str

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserConnectedEvent', 30.0))

class BrowserStoppedEvent(BaseEvent):
    ■"""Browser has stopped/disconnected."""

    ■reason: str | None = None

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserStoppedEvent', 30.0))

class TabCreatedEvent(BaseEvent):
    ■"""A new tab was created."""

```

```

    ■target_id: TargetID
    ■url: str

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_TabCreatedEvent', 30))

class TabClosedEvent(BaseEvent):
    ■"""A tab was closed."""

    ■target_id: TargetID

    ■# TODO:
    ■# new_focus_target_id: int | None = None
    ■# new_focus_url: str | None = None

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_TabClosedEvent', 3.0))

    # TODO: emit this when DOM changes significantly, inner frame navigates, form submits, history.pushState
    # class TabUpdatedEvent(BaseEvent):
    #     ■"""Tab information updated (URL changed, etc.)."""

    #     ■target_id: TargetID
    #     ■url: str

class AgentFocusChangedEvent(BaseEvent):
    ■"""Agent focus changed to a different tab."""

    ■target_id: TargetID
    ■url: str

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_AgentFocusChangedEvent', 3.0))

class TargetCrashedEvent(BaseEvent):
    ■"""A target has crashed."""

    ■target_id: TargetID
    ■error: str

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_TargetCrashedEvent', 3.0))

class NavigationStartedEvent(BaseEvent):
    ■"""Navigation started."""

    ■target_id: TargetID
    ■url: str

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_NavigationStartedEvent', 3.0))

class NavigationCompleteEvent(BaseEvent):
    ■"""Navigation completed."""

    ■target_id: TargetID
    ■url: str
    ■status: int | None = None
    ■error_message: str | None = None # Error/timeout message if navigation had issues
    ■loading_status: str | None = None # Detailed loading status (e.g., network timeout info)

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_NavigationCompleteEvent', 3.0))

# =====
# Error Events
# =====

class BrowserErrorEvent(BaseEvent):
    ■"""An error occurred in the browser layer."""

```

```

    ■error_type: str
    ■message: str
    ■details: dict[str, Any] = Field(default_factory=dict)

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserErrorEvent', 3))

class BrowserReconnectingEvent(BaseEvent):
    ■"""WebSocket reconnection attempt is starting."""

    ■cdp_url: str
    ■attempt: int
    ■max_attempts: int

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserReconnectingEvent', 3))

class BrowserReconnectedEvent(BaseEvent):
    ■"""WebSocket reconnection succeeded."""

    ■cdp_url: str
    ■attempt: int
    ■downtime_seconds: float

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_BrowserReconnectedEvent', 3))

# =====
# Storage State Events
# =====

class SaveStorageStateEvent(BaseEvent):
    ■"""Request to save browser storage state."""

    ■path: str | None = None # Optional path, uses profile default if not provided

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_SaveStorageStateEvent', 3))

class StorageStateSavedEvent(BaseEvent):
    ■"""Notification that storage state was saved."""

    ■path: str
    ■cookies_count: int
    ■origins_count: int

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_StorageStateSavedEvent', 3))

class LoadStorageStateEvent(BaseEvent):
    ■"""Request to load browser storage state."""

    ■path: str | None = None # Optional path, uses profile default if not provided

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_LoadStorageStateEvent', 3))

# TODO: refactor this to:
# - on_BrowserConnectedEvent() -> dispatch(LoadStorageStateEvent()) -> _copy_storage_state_from_json_to_profile
# - on_BrowserStopEvent() -> dispatch(SaveStorageStateEvent()) -> _copy_storage_state_from_browser_to_json
# and get rid of StorageStateSavedEvent and StorageStateLoadedEvent, have the original events + provide
class StorageStateLoadedEvent(BaseEvent):
    ■"""Notification that storage state was loaded."""

    ■path: str
    ■cookies_count: int
    ■origins_count: int

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_StorageStateLoadedEvent', 3))
# =====
# File Download Events

```



```

# =====

class DownloadStartedEvent(BaseEvent):
    """A file download has started (CDP downloadWillBegin received)."""

    guid: str # CDP download GUID to correlate with FileDownloadedEvent
    url: str
    suggested_filename: str
    auto_download: bool = False # Whether this was triggered automatically

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_DownloadStartedEvent'))

class DownloadProgressEvent(BaseEvent):
    """A file download progress update (CDP downloadProgress received)."""

    guid: str # CDP download GUID to correlate with other download events
    received_bytes: int
    total_bytes: int # 0 if unknown
    state: str # 'inProgress', 'completed', or 'canceled'

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_DownloadProgressEvent'))

class FileDownloadedEvent(BaseEvent):
    """A file has been downloaded."""

    guid: str | None = None # CDP download GUID to correlate with DownloadStartedEvent
    url: str
    path: str
    file_name: str
    file_size: int
    file_type: str | None = None # e.g., 'pdf', 'zip', 'docx', etc.
    mime_type: str | None = None # e.g., 'application/pdf'
    from_cache: bool = False
    auto_download: bool = False # Whether this was an automatic download (e.g., PDF auto-download)

    event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_FileDownloadedEvent'))

class AboutBlankDVDScreensaverShownEvent(BaseEvent):
    """AboutBlankWatchdog has shown DVD screensaver animation on an about:blank tab."""

    target_id: TargetID
    error: str | None = None

class DialogOpenedEvent(BaseEvent):
    """Event dispatched when a JavaScript dialog is opened and handled."""

    dialog_type: str # 'alert', 'confirm', 'prompt', or 'beforeunload'
    message: str
    url: str
    frame_id: str | None = None # Can be None when frameId is not provided by CDP
    # target_id: TargetID # TODO: add this to avoid needing target_id_from_frame() later

# =====
# Captcha Solver Events
# =====

class CaptchaSolverStartedEvent(BaseEvent):
    """Captcha solving started by the browser proxy.

    Emitted when the browser proxy detects a CAPTCHA and begins solving it.
    The agent should wait for a corresponding CaptchaSolverFinishedEvent before proceeding.
    """

    target_id: TargetID
    vendor: str # e.g. 'cloudflare', 'recaptcha', 'hcaptcha', 'datadome', 'perimeterx', 'geetest'

```

```

    ■url: str
    ■started_at: int # Unix millis

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_CaptchaSolverStarted'))

class CaptchaSolverFinishedEvent(BaseEvent):
    ■"""Captcha solving finished by the browser proxy.

    ■Emitted when the browser proxy finishes solving a CAPTCHA (successfully or not).
    ■"""

    ■target_id: TargetID
    ■vendor: str
    ■url: str
    ■duration_ms: int
    ■finished_at: int # Unix millis
    ■success: bool # Whether the captcha was solved successfully

    ■event_timeout: float | None = Field(default_factory=lambda: _get_timeout('TIMEOUT_CaptchaSolverFinished'))

# Note: Model rebuilding for forward references is handled in the importing modules
# Events with 'EnhancedDOMTreeNode' forward references (ClickElementEvent, TypeTextEvent,
# ScrollEvent, UploadFileEvent) need model_rebuild() called after imports are complete

def _check_event_names_dont_overlap():
    ■"""
    ■check that event names defined in this file are valid and non-overlapping
    ■(naively n^2 so it's pretty slow but ok for now, optimize when >20 events)
    ■"""
    ■event_names = {
    ■■name.split('[')[0]
    ■■for name in globals().keys()
    ■■if not name.startswith('_')
    ■■and inspect.isclass(globals()[name])
    ■■and issubclass(globals()[name], BaseEvent)
    ■■and name != 'BaseEvent'
    ■}
    ■for name_a in event_names:
    ■■assert name_a.endswith('Event'), f'Event with name {name_a} does not end with "Event"'
    ■■for name_b in event_names:
    ■■■if name_a != name_b: # Skip self-comparison
    ■■■■assert name_a not in name_b, (
    ■■■■f'Event with name {name_a} is a substring of {name_b}, all events must be completely unique to avoid
    ■■■■)

# overlapping event names are a nightmare to trace and rename later, dont do it!
# e.g. prevent ClickEvent and FailedClickEvent are terrible names because one is a substring of the other
# must be ClickEvent and ClickFailedEvent to preserve the usefulness of codebase grep/sed/awk as refactor
# at import time, we do a quick check that all event names defined above are valid and non-overlapping.
# this is hand written in blood by a human! not LLM slop. feel free to optimize but do not remove it with
_check_event_names_dont_overlap()

```

`.system\browser\profile.py`

```

import os
import sys
import tempfile
from collections.abc import Iterable
from enum import Enum
from functools import cache
from pathlib import Path
from typing import Annotated, Any, Literal, Self
from urllib.parse import urlparse
from pydantic import AfterValidator, AliasChoices, BaseModel, ConfigDict, Field, field_validator, model_validator

```

```

from system.browser.cloud.views import CloudBrowserParams
from system.config import CONFIG
from system.utils import _log_pretty_path, logger

```

```

def _get_enable_default_extensions_default() -> bool:
    """Get the default value for enable_default_extensions from env var or True."""
    env_val = os.getenv('BROWSER_USE_DISABLE_EXTENSIONS')
    if env_val is not None:
        ## If DISABLE_EXTENSIONS is truthy, return False (extensions disabled)
        return env_val.lower() in ('0', 'false', 'no', 'off', '')
    return True

```

```

CHROME_DEBUG_PORT = 9242 # use a non-default port to avoid conflicts with other tools / devs using 9222
DOMAIN_OPTIMIZATION_THRESHOLD = 100 # Convert domain lists to sets for O(1) lookup when >= this size
CHROME_DISABLED_COMPONENTS = [
    ## Playwright defaults: https://github.com/microsoft/playwright/blob/41008eeddd020e2dee1c540f7c0cdfa337e
    ## AcceptCHFrame,AutoExpandDetailsElement,AvoidUnnecessaryBeforeUnloadCheckSync,CertificateTransparency
    ## See https://github.com/microsoft/playwright/pull/10380
    'AcceptCHFrame',
    ## See https://github.com/microsoft/playwright/pull/10679
    'AutoExpandDetailsElement',
    ## See https://github.com/microsoft/playwright/issues/14047
    'AvoidUnnecessaryBeforeUnloadCheckSync',
    ## See https://github.com/microsoft/playwright/pull/12992
    'CertificateTransparencyComponentUpdater',
    'DestroyProfileOnBrowserClose',
    ## See https://github.com/microsoft/playwright/pull/13854
    'DialMediaRouteProvider',
    ## Chromium is disabling manifest version 2. Allow testing it as long as Chromium can actually run it.
    ## Disabled in https://chromium-review.googlesource.com/c/chromium/src/+/6265903.
    'ExtensionManifestV2Disabled',
    'GlobalMediaControls',
    ## See https://github.com/microsoft/playwright/pull/27605
    'HttpsUpgrades',
    'ImprovedCookieControls',
    'LazyFrameLoading',
    ## Hides the Lens feature in the URL address bar. Its not working in unofficial builds.
    'Lensoverlay',
    ## See https://github.com/microsoft/playwright/pull/8162
    'MediaRouter',
    ## See https://github.com/microsoft/playwright/issues/28023
    'PaintHolding',
    ## See https://github.com/microsoft/playwright/issues/32230
    'ThirdPartyStoragePartitioning',
    ## See https://github.com/microsoft/playwright/issues/16126
    'Translate',
    ## 3
    ## Added by us:
    'AutomationControlled',
    'BackForwardCache',
    'OptimizationHints',
    'ProcessPerSiteUpToMainFrameThreshold',
    'InterestFeedContentSuggestions',
    'CalculateNativeWinOcclusion', # chrome normally stops rendering tabs if they are not visible (occlude
    ## 'BackForwardCache', # agent does actually use back/forward navigation, but we can disable if we ever
    'HeavyAdPrivacyMitigations',
    'PrivacySandboxSettings4',
    'AutofillServerCommunication',
    'CrashReporting',
    'OverscrollHistoryNavigation',
    'InfiniteSessionRestore',
    'ExtensionDisableUnsupportedDeveloper',
    'ExtensionManifestV2Unsupported',
]

CHROME_HEADLESS_ARGS = [
    '--headless=new',
]

CHROME_DOCKER_ARGS = [

```

```

    ■# '--disable-gpu',      # GPU is actually supported in headless docker mode now, but sometimes useful to
    ■'--no-sandbox',
    ■'--disable-gpu-sandbox',
    ■'--disable-setuid-sandbox',
    ■'--disable-dev-shm-usage',
    ■'--no-xshm',
    ■'--no-zygote',
    ■# '--single-process', # might be the cause of "Target page, context or browser has been closed" errors
    ■'--disable-site-isolation-trials', # lowers RAM use by 10-16% in docker, but could lead to easier bot
]

```

```

CHROME_DISABLE_SECURITY_ARGS = [
    ■'--disable-site-isolation-trials',
    ■'--disable-web-security',
    ■'--disable-features=IsolateOrigins,site-per-process',
    ■'--allow-running-insecure-content',
    ■'--ignore-certificate-errors',
    ■'--ignore-ssl-errors',
    ■'--ignore-certificate-errors-spki-list',
]

```

```

CHROME_DETERMINISTIC_RENDERING_ARGS = [
    ■'--deterministic-mode',
    ■'--js-flags=--random-seed=1157259159',
    ■'--force-device-scale-factor=2',
    ■'--enable-webgl',
    ■# '--disable-skia-runtime-opts',
    ■# '--disable-2d-canvas-clip-aa',
    ■'--font-render-hinting=none',
    ■'--force-color-profile=srgb',
]

```

```

CHROME_DEFAULT_ARGS = [
    ■# # provided by playwright by default: https://github.com/microsoft/playwright/blob/41008eeddd020e2dee1
    ■'--disable-field-trial-config', # https://source.chromium.org/chromium/chromium/src/+main:testing/var
    ■'--disable-background-networking',
    ■'--disable-background-timer-throttling', # agents might be working on background pages if the human sw
    ■'--disable-background-occluded-windows', # same deal, agents are often working on backgrounded brow
    ■'--disable-back-forward-cache', # Avoids surprises like main request not being intercepted during page
    ■'--disable-breakpad',
    ■'--disable-client-side-phishing-detection',
    ■'--disable-component-extensions-with-background-pages',
    ■'--disable-component-update', # Avoids unneeded network activity after startup.
    ■'--no-default-browser-check',
    ■# '--disable-default-apps',
    ■'--disable-dev-shm-usage', # crucial for docker support, harmless in non-docker environments
    ■# '--disable-extensions',
    ■# '--disable-features=' + disabledFeatures(assistantMode).join(','),
    ■# '--allow-pre-commit-input', # duplicate removed
    ■'--disable-hang-monitor',
    ■'--disable-ipc-flooding-protection', # important to be able to make lots of CDP calls in a tight loop
    ■'--disable-popup-blocking',
    ■'--disable-prompt-on-repost',
    ■'--disable-renderer-backgrounding',
    ■# '--force-color-profile=srgb', # moved to CHROME_DETERMINISTIC_RENDERING_ARGS
    ■'--metrics-recording-only',
    ■'--no-first-run',
    ■# // See https://chromium-review.googlesource.com/c/chromium/src/+2436773
    ■'--no-service-autorun',
    ■'--export-tagged-pdf',
    ■# // https://chromium-review.googlesource.com/c/chromium/src/+4853540
    ■'--disable-search-engine-choice-screen',
    ■# // https://issues.chromium.org/41491762
    ■'--unsafely-disable-devtools-self-xss-warnings',
    ■# added by us:
    ■'--enable-features=NetworkService,NetworkServiceInProcess',
    ■'--enable-network-information-downlink-max',
    ■'--test-type=gpu',
    ■'--disable-sync',
    ■'--allow-legacy-extension-manifests',
    ■'--allow-pre-commit-input',

```

```

■'--disable-blink-features=AutomationControlled',
■'--install-autogenerated-theme=0,0,0',
■# '--hide-scrollbar', # leave them visible! the agent uses them to know when it ne
■'--log-level=2',
■# '--enable-logging=stderr',
■'--disable-focus-on-load',
■'--disable-window-activation',
■'--generate-pdf-document-outline',
■'--no-pings',
■'--ash-no-nudges',
■'--disable-infobars',
■'--simulate-outdated-no-au="Tue, 31 Dec 2099 23:59:59 GMT"',
■'--hide-crash-restore-bubble',
■'--suppress-message-center-popups',
■'--disable-domain-reliability',
■'--disable-datasaver-prompt',
■'--disable-speech-synthesis-api',
■'--disable-speech-api',
■'--disable-print-preview',
■'--safebrowsing-disable-auto-update',
■'--disable-external-intent-requests',
■'--disable-desktop-notifications',
■'--noerrdialogs',
■'--silent-debugger-extension-api',
■# Extension welcome tab suppression for automation
■'--disable-extensions-http-throttling',
■'--extensions-on-chrome-urls',
■'--disable-default-apps',
■f'--disable-features={','.join(CHROME_DISABLED_COMPONENTS)}',
]

```

```

class ViewportSize(BaseModel):
    ■width: int = Field(ge=0)
    ■height: int = Field(ge=0)

    ■def __getitem__(self, key: str) -> int:
    ■    return dict(self)[key]

    ■def __setitem__(self, key: str, value: int) -> None:
    ■    setattr(self, key, value)

@cache
def get_display_size() -> ViewportSize | None:
    ■# macOS
    ■try:
    ■    from AppKit import NSScreen # type: ignore[import]

    ■    screen = NSScreen.mainScreen().frame()
    ■    size = ViewportSize(width=int(screen.size.width), height=int(screen.size.height))
    ■    logger.debug(f'Display size: {size}')
    ■    return size
    ■except Exception:
    ■    pass

    ■# Windows & Linux
    ■try:
    ■    from screeninfo import get_monitors

    ■    monitors = get_monitors()
    ■    monitor = monitors[0]
    ■    size = ViewportSize(width=int(monitor.width), height=int(monitor.height))
    ■    logger.debug(f'Display size: {size}')
    ■    return size
    ■except Exception:
    ■    pass

    ■logger.debug('No display size found')
    ■return None
def get_window_adjustments() -> tuple[int, int]:
    ■"""Returns recommended x, y offsets for window positioning"""

```

```

    if sys.platform == 'darwin': # macOS
        return -4, 24 # macOS has a small title bar, no border
    elif sys.platform == 'win32': # Windows
        return -8, 0 # Windows has a border on the left
    else: # Linux
        return 0, 0

def validate_url(url: str, schemes: Iterable[str] = ()) -> str:
    """Validate URL format and optionally check for specific schemes."""
    parsed_url = urlparse(url)
    if not parsed_url.netloc:
        raise ValueError(f'Invalid URL format: {url}')
    if schemes and parsed_url.scheme and parsed_url.scheme.lower() not in schemes:
        raise ValueError(f'URL has invalid scheme: {url} (expected one of {schemes})')
    return url

def validate_float_range(value: float, min_val: float, max_val: float) -> float:
    """Validate that float is within specified range."""
    if not min_val <= value <= max_val:
        raise ValueError(f'Value {value} outside of range {min_val}-{max_val}')
    return value

def validate_cli_arg(arg: str) -> str:
    """Validate that arg is a valid CLI argument."""
    if not arg.startswith('--'):
        raise ValueError(f'Invalid CLI argument: {arg} (should start with --, e.g. --some-key="some value here")')
    return arg

# ===== Enum definitions =====

class RecordHarContent(str, Enum):
    OMIT = 'omit'
    EMBED = 'embed'
    ATTACH = 'attach'

class RecordHarMode(str, Enum):
    FULL = 'full'
    MINIMAL = 'minimal'

class BrowserChannel(str, Enum):
    CHROMIUM = 'chromium'
    CHROME = 'chrome'
    CHROME_BETA = 'chrome-beta'
    CHROME_DEV = 'chrome-dev'
    CHROME_CANARY = 'chrome-canary'
    MSEDGE = 'msedge'
    MSEDGE_BETA = 'msedge-beta'
    MSEDGE_DEV = 'msedge-dev'
    MSEDGE_CANARY = 'msedge-canary'

# Using constants from central location in system.config
BROWSERUSE_DEFAULT_CHANNEL = BrowserChannel.CHROMIUM

# ===== Type definitions with validators =====

UrlStr = Annotated[str, AfterValidator(validate_url)]
NonNegativeFloat = Annotated[float, AfterValidator(lambda x: validate_float_range(x, 0, float('inf')))]
CliArgStr = Annotated[str, AfterValidator(validate_cli_arg)]

# ===== Base Models =====
class BrowserContextArgs(BaseModel):
    """

```

```

■Base model for common browser context parameters used by
■both BrowserType.new_context() and BrowserType.launch_persistent_context().

■https://playwright.dev/python/docs/api/class-browser#browser-new-context
■"""

■model_config = ConfigDict(extra='ignore', validate_assignment=False, revalidate_instances='always', popo

■# Browser context parameters
■accept_downloads: bool = True

■# Security options
■# proxy: ProxySettings | None = None
■permissions: list[str] = Field(
■# default_factory=lambda: ['clipboardReadWrite', 'notifications'],
■# description='Browser permissions to grant (CDP Browser.grantPermissions).',
■# # clipboardReadWrite is for google sheets and pyperclip automations
■# # notifications are to avoid browser fingerprinting
■)
■# client_certificates: list[ClientCertificate] = Field(default_factory=list)
■# http_credentials: HttpCredentials | None = None

■# Viewport options
■user_agent: str | None = None
■screen: ViewportSize | None = None
■viewport: ViewportSize | None = Field(default=None)
■no_viewport: bool | None = None
■device_scale_factor: NonNegativeFloat | None = None
■# geolocation: Geolocation | None = None

■# Recording Options
■record_har_content: RecordHarContent = RecordHarContent.EMBED
■record_har_mode: RecordHarMode = RecordHarMode.FULL
■record_har_path: str | Path | None = Field(default=None, validation_alias=AliasChoices('save_har_path',
■record_video_dir: str | Path | None = Field(
■# default=None, validation_alias=AliasChoices('save_recording_path', 'record_video_dir')
■)

class BrowserConnectArgs(BaseModel):
    """
    ■Base model for common browser connect parameters used by
    ■both connect_over_cdp() and connect_over_ws().

    ■https://playwright.dev/python/docs/api/class-browsertype#browser-type-connect
    ■https://playwright.dev/python/docs/api/class-browsertype#browser-type-connect-over-cdp
    ■"""

    ■model_config = ConfigDict(extra='ignore', validate_assignment=True, revalidate_instances='always', popo

    ■headers: dict[str, str] | None = Field(default=None, description='Additional HTTP headers to be sent w

class BrowserLaunchArgs(BaseModel):
    """
    ■Base model for common browser launch parameters used by
    ■both launch() and launch_persistent_context().

    ■https://playwright.dev/python/docs/api/class-browsertype#browser-type-launch
    ■"""

    ■model_config = ConfigDict(
    ■# extra='ignore',
    ■# validate_assignment=True,
    ■# revalidate_instances='always',
    ■# from_attributes=True,
    ■# validate_by_name=True,
    ■# validate_by_alias=True,
    ■# populate_by_name=True,
    ■)

    ■env: dict[str, str | float | bool] | None = Field(

```

```

    default=None,
    description='Extra environment variables to set when launching the browser. If None, inherits from the
    ')
    executable_path: str | Path | None = Field(
        default=None,
        validation_alias=AliasChoices('browser_binary_path', 'chrome_binary_path'),
        description='Path to the chromium-based browser executable to use.',
    )
    headless: bool | None = Field(default=None, description='Whether to run the browser in headless or windowed mode')
    args: list[CliArgStr] = Field(
        default_factory=list, description='List of *extra* CLI args to pass to the browser when launching.'
    )
    ignore_default_args: list[CliArgStr] | Literal[True] = Field(
        default_factory=lambda: [
            '--enable-automation', # we mask the automation fingerprint via JS and other flags
            '--disable-extensions', # allow browser extensions
            '--hide-scrollbar', # always show scrollbars in screenshots so agent knows there is more content below
            '--disable-features=AcceptCHFrame,AutoExpandDetailsElement,AvoidUnnecessaryBeforeUnloadCheckSync,CertificateTransparencyEnforcement',
        ],
        description='List of default CLI args to stop playwright from applying (see https://github.com/microsoft/playwright/blob/main/docs/faq.md#how-to-run-chromium-headless-shell)'
    )
    channel: BrowserChannel | None = None # https://playwright.dev/docs/browsers#chromium-headless-shell
    chromium_sandbox: bool = Field(
        default=not CONFIG.IN_DOCKER, description='Whether to enable Chromium sandboxing (recommended unless running in Docker)'
    )
    devtools: bool = Field(
        default=False, description='Whether to open DevTools panel automatically for every page, only works with Chromium'
    )

    # proxy: ProxySettings | None = Field(default=None, description='Proxy settings to use to connect to the browser')
    downloads_path: str | Path | None = Field(
        default=None,
        description='Directory to save downloads to.',
        validation_alias=AliasChoices('downloads_dir', 'save_downloads_path'),
    )
    traces_dir: str | Path | None = Field(
        default=None,
        description='Directory for saving playwright trace.zip files (playwright actions, screenshots, DOM snapshots, etc.)',
        validation_alias=AliasChoices('trace_path', 'traces_dir'),
    )

    # firefox_user_prefs: dict[str, str | float | bool] = Field(default_factory=dict)

    @model_validator(mode='after')
    def validate_devtools_headless(self) -> Self:
        """Cannot open devtools when headless is True"""
        assert not (self.headless and self.devtools), 'headless=True and devtools=True cannot both be set at the same time'
        return self

    @model_validator(mode='after')
    def set_default_downloads_path(self) -> Self:
        """Set a unique default downloads path if none is provided."""
        if self.downloads_path is None:
            import uuid

            # Create unique directory in /tmp for downloads
            unique_id = str(uuid.uuid4())[:8] # 8 characters
            downloads_path = Path(f'/tmp/browser-use-downloads-{unique_id}')

            # Ensure path doesn't already exist (extremely unlikely but possible)
            while downloads_path.exists():
                unique_id = str(uuid.uuid4())[:8]
                downloads_path = Path(f'/tmp/browser-use-downloads-{unique_id}')

            self.downloads_path = downloads_path
            self.downloads_path.mkdir(parents=True, exist_ok=True)
            return self

    @staticmethod
    def args_as_dict(args: list[str]) -> dict[str, str]:
        """Return the extra launch CLI args as a dictionary."""
        args_dict = {}

```



```

    for arg in args:
        key, value, *_ = [*arg.split('=', 1), '', '', '']
        args_dict[key.strip().lstrip('-')] = value.strip()
    return args_dict

    @staticmethod
    def args_as_list(args: dict[str, str]) -> list[str]:
        """Return the extra launch CLI args as a list of strings."""
        return [f'--{key.lstrip("-")}={value}' if value else f'--{key.lstrip("-)}' for key, value in args.items()]

# ===== API-specific Models =====

class BrowserNewContextArgs(BrowserContextArgs):
    """
    Pydantic model for new_context() arguments.
    Extends BaseContextParams with storage_state parameter.

    https://playwright.dev/python/docs/api/class-browser#browser-new-context
    """

    model_config = ConfigDict(extra='ignore', validate_assignment=False, revalidate_instances='always', populate_by_name=True)

    # storage_state is not supported in launch_persistent_context()
    storage_state: str | Path | dict[str, Any] | None = None
    # TODO: use StorageState type instead of dict[str, Any]

    # to apply this to existing contexts (incl cookies, localStorage, IndexedDB), see:
    # - https://github.com/microsoft/playwright/pull/34591/files
    # - playwright-core/src/server/storageScript.ts restore() function
    # - https://github.com/Skn0tt/playwright/blob/c446bc44bac4fbfd52439ba434f92192459be4e/packages/playwright-core/src/server/storageScript.ts

    @field_validator('storage_state', mode='after')
    def load_storage_state_from_file(self) -> Self:
        """Load storage_state from file if it's a path."""
        if isinstance(self.storage_state, (str, Path)):
            storage_state_file = Path(self.storage_state)
            try:
                parsed_storage_state = json.loads(storage_state_file.read_text())
                validated_storage_state = StorageState(**parsed_storage_state)
                self.storage_state = validated_storage_state
            except Exception as e:
                raise ValueError(f'Failed to load storage state file {self.storage_state}: {e}') from e
        return self
    pass

class BrowserLaunchPersistentContextArgs(BrowserLaunchArgs, BrowserContextArgs):
    """
    Pydantic model for launch_persistent_context() arguments.
    Combines browser launch parameters and context parameters,
    plus adds the user_data_dir parameter.

    https://playwright.dev/python/docs/api/class-browser#browser-type-launch-persistent-context
    """

    model_config = ConfigDict(extra='ignore', validate_assignment=False, revalidate_instances='always')

    # Required parameter specific to launch_persistent_context, but can be None to use incognito temp dir
    user_data_dir: str | Path | None = None

    @field_validator('user_data_dir', mode='after')
    @classmethod
    def validate_user_data_dir(cls, v: str | Path | None) -> str | Path:
        """Validate user data dir is set to a non-default path."""
        if v is None:
            return tempfile.mkdtemp(prefix='browser-use-user-data-dir-')
        return Path(v).expanduser().resolve()

class ProxySettings(BaseModel):

```

```

    """Typed proxy settings for Chromium traffic.

    - server: Full proxy URL, e.g. "http://host:8080" or "socks5://host:1080"
    - bypass: Comma-separated hosts to bypass (e.g. "localhost,127.0.0.1,*.internal")
    - username/password: Optional credentials for authenticated proxies
    """

    server: str | None = Field(default=None, description='Proxy URL, e.g. http://host:8080 or socks5://host:1080')
    bypass: str | None = Field(default=None, description='Comma-separated hosts to bypass, e.g. localhost,127.0.0.1,*.internal')
    username: str | None = Field(default=None, description='Proxy auth username')
    password: str | None = Field(default=None, description='Proxy auth password')

    def __getitem__(self, key: str) -> str | None:
        return getattr(self, key)

class BrowserProfile(BrowserConnectArgs, BrowserLaunchPersistentContextArgs, BrowserLaunchArgs, BrowserNewContextArgs):
    """
    A BrowserProfile is a static template collection of kwargs that can be passed to:
    - BrowserType.launch(**BrowserLaunchArgs)
    - BrowserType.connect(**BrowserConnectArgs)
    - BrowserType.connect_over_cdp(**BrowserConnectArgs)
    - BrowserType.launch_persistent_context(**BrowserLaunchPersistentContextArgs)
    - BrowserContext.new_context(**BrowserNewContextArgs)
    - BrowserSession(**BrowserProfile)
    """

    model_config = ConfigDict(
        extra='ignore',
        validate_assignment=True,
        revalidate_instances='always',
        from_attributes=True,
        validate_by_name=True,
        validate_by_alias=True,
    )

    # ... extends options defined in:
    # BrowserLaunchPersistentContextArgs, BrowserLaunchArgs, BrowserNewContextArgs, BrowserConnectArgs

    # Session/connection configuration
    cdp_url: str | None = Field(default=None, description='CDP URL for connecting to existing browser instance')
    is_local: bool = Field(default=False, description='Whether this is a local browser instance')
    use_cloud: bool = Field(
        default=False,
        description='Use browser-use cloud browser service instead of local browser',
    )

    @property
    def cloud_browser(self) -> bool:
        """Alias for use_cloud field for compatibility."""
        return self.use_cloud

    cloud_browser_params: CloudBrowserParams | None = Field(
        default=None, description='Parameters for creating a cloud browser instance'
    )

    # custom options we provide that aren't native playwright kwargs
    disable_security: bool = Field(default=False, description='Disable browser security features.')
    deterministic_rendering: bool = Field(default=False, description='Enable deterministic rendering flags.')
    allowed_domains: list[str] | set[str] | None = Field(
        default=None,
        description='List of allowed domains for navigation e.g. ["*.google.com", "https://example.com", "chrome://"]'
    )
    prohibited_domains: list[str] | set[str] | None = Field(
        default=None,
        description='List of prohibited domains for navigation e.g. ["*.google.com", "https://example.com", "chrome://"]'
    )
    block_ip_addresses: bool = Field(
        default=False,
        description='Block navigation to URLs containing IP addresses (both IPv4 and IPv6). When True, blocks navigation to any URL containing an IP address'
    )
    keep_alive: bool | None = Field(default=None, description='Keep browser alive after agent run.')

```

```

## --- Proxy settings ---
## New consolidated proxy config (typed)
proxy: ProxySettings | None = Field(
    default=None,
    description='Proxy settings. Use system.browser.profile.ProxySettings(server, bypass, username, password, ...)',
)
enable_default_extensions: bool = Field(
    default_factory=get_enable_default_extensions_default,
    description='Enable automation-optimized extensions: ad blocking (uBlock Origin), cookie handling (Istio)',
)
captcha_solver: bool = Field(
    default=True,
    description='Enable the captcha solver watchdog that listens for captcha events from the browser proxy.',
)
demo_mode: bool = Field(
    default=False,
    description='Enable demo mode side panel that streams agent logs directly inside the browser window (Istio)',
)
cookie_whitelist_domains: list[str] = Field(
    default_factory=lambda: ['nature.com', 'qatarairways.com'],
    description='List of domains to whitelist in the "I still don\'t care about cookies" extension, preventing them from blocking cookies.',
)

window_size: ViewportSize | None = Field(
    default=None,
    description='Browser window size to use when headless=False.',
)
window_height: int | None = Field(default=None, description='DEPRECATED, use window_size["height"] instead')
window_width: int | None = Field(default=None, description='DEPRECATED, use window_size["width"] instead')
window_position: ViewportSize | None = Field(
    default=ViewportSize(width=0, height=0),
    description='Window position to use for the browser x,y from the top left when headless=False.',
)
cross_origin_iframes: bool = Field(
    default=True,
    description='Enable cross-origin iframe support (OOPIF/Out-of-Process iframes). When False, only same-origin iframes are supported.',
)
max_iframes: int = Field(
    default=100,
    description='Maximum number of iframe documents to process to prevent crashes.',
)
max_iframe_depth: int = Field(
    ge=0,
    default=5,
    description='Maximum depth for cross-origin iframe recursion (default: 5 levels deep).',
)

## --- Page load/wait timings ---

minimum_wait_page_load_time: float = Field(default=0.25, description='Minimum time to wait before capturing page content')
wait_for_network_idle_page_load_time: float = Field(default=0.5, description='Time to wait for network to be idle')

wait_between_actions: float = Field(default=0.1, description='Time to wait between actions.')

## --- UI/viewport/DOM ---
highlight_elements: bool = Field(default=True, description='Highlight interactive elements on the page.')
dom_highlight_elements: bool = Field(
    default=False,
    description='Highlight interactive elements in the DOM (only for debugging purposes).',
)
filter_highlight_ids: bool = Field(
    default=True,
    description='Only show element IDs in highlights if llm_representation is less than 10 characters.',
)
paint_order_filtering: bool = Field(default=True, description='Enable paint order filtering. Slightly slower but more accurate.')
interaction_highlight_color: str = Field(
    default='rgb(255, 127, 39)',
    description='Color to use for highlighting elements during interactions (CSS color string).',
)
interaction_highlight_duration: float = Field(default=1.0, description='Duration in seconds to show interaction highlights.')

## --- Downloads ---
auto_download_pdfs: bool = Field(default=True, description='Automatically download PDFs when navigating to a PDF file.')
profile_directory: str = 'Default' # e.g. 'Profile 1', 'Profile 2', 'Custom Profile', etc.

```

```

    ## these can be found in BrowserLaunchArgs, BrowserLaunchPersistentContextArgs, BrowserNewContextArgs, ...
    ## save_recording_path: alias of record_video_dir
    ## save_har_path: alias of record_har_path
    ## trace_path: alias of traces_dir

    ## these shadow the old playwright args on BrowserContextArgs, but it's ok
    ## because we handle them ourselves in a watchdog and we no longer use playwright, so they should live ...
    ## record_video_dir: Path | None = Field(
    ##     default=None,
    ##     description='Directory to save video recordings. If set, a video of the session will be recorded.',
    ##     validation_alias=AliasChoices('save_recording_path', 'record_video_dir'),
    ## )
    ## record_video_size: ViewportSize | None = Field(
    ##     default=None, description='Video frame size. If not set, it will use the viewport size.'
    ## )
    ## record_video_framerate: int = Field(default=30, description='The framerate to use for the video recording')

    ## TODO: finish implementing extension support in extensions.py
    ## extension_ids_to_preinstall: list[str] = Field(
    ##     default_factory=list, description='List of Chrome extension IDs to preinstall.'
    ## )
    ## extensions_dir: Path = Field(
    ##     default_factory=lambda: Path('~/.config/browseruse/cache/extensions').expanduser(),
    ##     description='Directory containing .crx extension files.',
    ## )

    def __repr__(self) -> str:
        ## short_dir = _log_pretty_path(self.user_data_dir) if self.user_data_dir else '<incognito>'
        ## return f'BrowserProfile(user_data_dir= {short_dir}, headless={self.headless})'

    def __str__(self) -> str:
        ## return 'BrowserProfile'

    @field_validator('allowed_domains', 'prohibited_domains', mode='after')
    @classmethod
    def optimize_large_domain_lists(cls, v: list[str] | set[str] | None) -> list[str] | set[str] | None:
        """Convert large domain lists (>=100 items) to sets for O(1) lookup performance."""
        ## if v is None or isinstance(v, set):
        ##     return v

        ## if len(v) >= DOMAIN_OPTIMIZATION_THRESHOLD:
        ##     logger.warning(
        ##         f'Optimizing domain list with {len(v)} items to set for O(1) lookup. '
        ##         f'Note: Pattern matching (*.domain.com, etc.) is not supported for lists >= {DOMAIN_OPTIMIZATION_THRESHOLD} '
        ##         f'Use exact domains only or keep list size < {DOMAIN_OPTIMIZATION_THRESHOLD} for pattern support.'
        ##     )
        ##     return set(v)

        ## return v

    @model_validator(mode='after')
    def copy_old_config_names_to_new(self) -> Self:
        """Copy old config window_width & window_height to window_size."""
        ## if self.window_width or self.window_height:
        ##     logger.warning(
        ##         f'BrowserProfile(window_width=..., window_height=...) are deprecated, use BrowserProfile(window_size=...) '
        ##     )
        ##     window_size = self.window_size or ViewportSize(width=0, height=0)
        ##     window_size['width'] = self.window_width or self.window_size['width'] or 1920
        ##     window_size['height'] = self.window_height or self.window_size['height'] or 1080
        ##     self.window_size = window_size

        ## return self

    @model_validator(mode='after')
    def warn_storage_state_user_data_dir_conflict(self) -> Self:
        """Warn when both storage_state and user_data_dir are set, as this can cause conflicts."""
        ## has_storage_state = self.storage_state is not None
        ## has_user_data_dir = (self.user_data_dir is not None) and ('tmp' not in str(self.user_data_dir).lower())

        ## if has_storage_state and has_user_data_dir:
        ##     logger.warning(

```

```

        """If BrowserSession(...) was passed both storage_state AND user_data_dir. storage_state={self.storage_state}
        and user_data_dir={self.user_data_dir}. If cookies/localStorage/sessionStorage in user_data_dir={self.user_data_dir}. '
        For multiple browsers in parallel, use only storage_state with user_data_dir=None, '
        or use a separate user_data_dir for each browser and set storage_state=None.'
        """
    def __init__(self, *args, **kwargs):
        return self

    @model_validator(mode='after')
    def warn_user_data_dir_non_default_version(self) -> Self:
        """
        If user is using default profile dir with a non-default channel, force-change it
        to avoid corrupting the default data dir created with a different channel.
        """

        is_not_using_default_chromium = self.executable_path or self.channel not in (BROWSERUSE_DEFAULT_CHROMIUM, BROWSERUSE_DEFAULT_CHROMIUM)
        if self.user_data_dir == CONFIG.BROWSER_USE_DEFAULT_USER_DATA_DIR and is_not_using_default_chromium:
            alternate_name = (
                Path(self.executable_path).name.lower().replace(' ', '-')
            )
            if self.executable_path:
                alternate_name = self.channel.name.lower()
            elif self.channel:
                alternate_name = self.channel
            else:
                alternate_name = 'None'
            logger.warning(
                f'{self} Changing user_data_dir= {self.log_pretty_path(self.user_data_dir)} -> .../default-{alternate_name}'
            )
            self.user_data_dir = CONFIG.BROWSER_USE_DEFAULT_USER_DATA_DIR.parent / f'default-{alternate_name}'
        return self

    @model_validator(mode='after')
    def warn_deterministic_rendering_weirdness(self) -> Self:
        if self.deterministic_rendering:
            logger.warning(
                'BrowserSession(deterministic_rendering=True) is NOT RECOMMENDED. It breaks many sites and increments the JS random seed and forces browsers across Linux/Mac/Windows to use the same font.'
            )
        return self

    @model_validator(mode='after')
    def validate_proxy_settings(self) -> Self:
        """Ensure proxy configuration is consistent."""
        if self.proxy and (self.proxy.bypass and not self.proxy.server):
            logger.warning('BrowserProfile.proxy.bypass provided but proxy has no server; bypass will be ignored')
        return self

    @model_validator(mode='after')
    def validate_highlight_elements_conflict(self) -> Self:
        """Ensure highlight_elements and dom_highlight_elements are not both enabled, with dom_highlight_elements taking priority"""
        if self.highlight_elements and self.dom_highlight_elements:
            logger.warning(
                'Both highlight_elements and dom_highlight_elements are enabled. '
                'dom_highlight_elements takes priority. Setting highlight_elements=False.'
            )
            self.highlight_elements = False
        return self

    def model_post_init(self, __context: Any) -> None:
        """Called after model initialization to set up display configuration."""
        self.detect_display_configuration()
        self._copy_profile()

    def _copy_profile(self) -> None:
        """Copy profile to temp directory if user_data_dir is not None and not already a temp dir."""
        if self.user_data_dir is None:
            return

        user_data_str = str(self.user_data_dir)
        if 'browser-use-user-data-dir-' in user_data_str.lower():
            # Already using a temp directory, no need to copy
            return

        is_chrome = (

```

```

    if 'chrome' in user_data_str.lower():
    or ('chrome' in str(self.executable_path).lower())
    or self.channel
    in (BrowserChannel.CHROME, BrowserChannel.CHROME_BETA, BrowserChannel.CHROME_DEV, BrowserChannel.CHROME_STABLE)

    if not is_chrome:
    return

    temp_dir = tempfile.mkdtemp(prefix='browser-use-user-data-dir-')
    path_original_user_data = Path(self.user_data_dir)
    path_original_profile = path_original_user_data / self.profile_directory
    path_temp_profile = Path(temp_dir) / self.profile_directory

    if path_original_profile.exists():
    import shutil

    shutil.copytree(path_original_profile, path_temp_profile)
    local_state_src = path_original_user_data / 'Local State'
    local_state_dst = Path(temp_dir) / 'Local State'
    if local_state_src.exists():
    shutil.copy(local_state_src, local_state_dst)
    logger.info(f'Copied profile ({self.profile_directory}) and Local State to temp directory: {temp_dir}')

    else:
    Path(temp_dir).mkdir(parents=True, exist_ok=True)
    path_temp_profile.mkdir(parents=True, exist_ok=True)
    logger.info(f'Created new profile ({self.profile_directory}) in temp directory: {temp_dir}')

    self.user_data_dir = temp_dir

    def get_args(self) -> list[str]:
    """Get the list of all Chrome CLI launch args for this profile (compiled from defaults, user-provided)

    if isinstance(self.ignore_default_args, list):
    default_args = set(CHROME_DEFAULT_ARGS) - set(self.ignore_default_args)
    elif self.ignore_default_args is True:
    default_args = []
    elif not self.ignore_default_args:
    default_args = CHROME_DEFAULT_ARGS

    assert self.user_data_dir is not None, 'user_data_dir must be set to a non-default path'

    # Capture args before conversion for logging
    pre_conversion_args = [
    *default_args,
    *self.args,
    f'--user-data-dir={self.user_data_dir}',
    f'--profile-directory={self.profile_directory}',
    *(CHROME_DOCKER_ARGS if (CONFIG.IN_DOCKER or not self.chromium_sandbox) else []),
    *(CHROME_HEADLESS_ARGS if self.headless else []),
    *(CHROME_DISABLE_SECURITY_ARGS if self.disable_security else []),
    *(CHROME_DETERMINISTIC_RENDERING_ARGS if self.deterministic_rendering else []),
    *(
    [f'--window-size={self.window_size["width"]},{self.window_size["height"]}']
    if self.window_size
    else ['--start-maximized'] if not self.headless else []
    ),
    *(
    [f'--window-position={self.window_position["width"]},{self.window_position["height"]}']
    if self.window_position
    else []
    ),
    *(self._get_extension_args() if self.enable_default_extensions else []),
    ]

    # Proxy flags
    proxy_server = self.proxy.server if self.proxy else None
    proxy_bypass = self.proxy.bypass if self.proxy else None

    if proxy_server:
    pre_conversion_args.append(f'--proxy-server={proxy_server}')

```

```

    if proxy_bypass:
        pre_conversion_args.append(f'--proxy-bypass-list={proxy_bypass}')

    # User agent flag
    if self.user_agent:
        pre_conversion_args.append(f'--user-agent={self.user_agent}')

    # Special handling for --disable-features to merge values instead of overwriting
    # This prevents disable_security=True from breaking extensions by ensuring
    # both default features (including extension-related) and security features are preserved
    disable_features_values = []
    non_disable_features_args = []

    # Extract and merge all --disable-features values
    for arg in pre_conversion_args:
        if arg.startswith('--disable-features='):
            features = arg.split('=', 1)[1]
            disable_features_values.extend(features.split(','))
        else:
            non_disable_features_args.append(arg)

    # Remove duplicates while preserving order
    if disable_features_values:
        unique_features = []
        seen = set()
        for feature in disable_features_values:
            feature = feature.strip()
            if feature and feature not in seen:
                unique_features.append(feature)
                seen.add(feature)

    # Add merged disable-features back
    non_disable_features_args.append(f'--disable-features={" ".join(unique_features)}')

    # convert to dict and back to dedupe and merge other duplicate args
    final_args_list = BrowserLaunchArgs.args_as_list(BrowserLaunchArgs.args_as_dict(non_disable_features_values))

    return final_args_list

    def _get_extension_args(self) -> list[str]:
        """Get Chrome args for enabling default extensions (ad blocker and cookie handler)."""
        extension_paths = self._ensure_default_extensions_downloaded()

        args = [
            '--enable-extensions',
            '--disable-extensions-file-access-check',
            '--disable-extensions-http-throttling',
            '--enable-extension-activity-logging',
        ]

        if extension_paths:
            args.append(f'--load-extension={" ".join(extension_paths)}')

        return args

    def _ensure_default_extensions_downloaded(self) -> list[str]:
        """
        Ensure default extensions are downloaded and cached locally.
        Returns list of paths to extension directories.
        """

    # Extension definitions - optimized for automation and content extraction
    # Combines uBlock Origin (ad blocking) + "I still don't care about cookies" (cookie banner handling)
    extensions = [
        {
            'name': 'uBlock Origin',
            'id': 'cjpalhdlnbpafiamejdnhcphjbkeiagm',
            'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=133&acceptfor',
        },
        {
            'name': "I still don't care about cookies",
            'id': 'edibdbjcnidpcccejdjdjppcpchdlm',
        },
    ]

```

```

##### 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=133&acceptfor
##### },
##### {
##### 'name': 'ClearURLs',
##### 'id': 'lckanjgmijmafbedllaakclkaicjfmnk',
##### 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=133&acceptfor
##### },
##### {
##### 'name': 'Force Background Tab',
##### 'id': 'gidlfommbibbmegmgajdbikelkdcmcl',
##### 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=133&acceptfor
##### },
##### {
##### # 'name': 'Captcha Solver: Auto captcha solving service',
##### # 'id': 'pgojnojmmhpofjgdmaebadhboahppod',
##### # 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=130&acceptf
##### # },
##### # Consent-O-Matic disabled - using uBlock Origin's cookie lists instead for simplicity
##### # {
##### # 'name': 'Consent-O-Matic',
##### # 'id': 'mdjildafknihdffpkfmmnpoiadjfnjd',
##### # 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=130&acceptf
##### # },
##### # {
##### # 'name': 'Privacy | Protect Your Payments',
##### # 'id': 'hmgpakheknboplhmlicfkkgjipfabmhp',
##### # 'url': 'https://clients2.google.com/service/update2/crx?response=redirect&prodversion=130&acceptf
##### # },
##### # },
##### # }
##### # }

##### # Create extensions cache directory
##### cache_dir = CONFIG.BROWSER_USE_EXTENSIONS_DIR
##### cache_dir.mkdir(parents=True, exist_ok=True)
##### # logger.debug(f'##### Extensions cache directory: {_log_pretty_path(cache_dir)}')

##### extension_paths = []
##### loaded_extension_names = []

##### for ext in extensions:
##### ext_dir = cache_dir / ext['id']
##### crx_file = cache_dir / f'{ext["id"]}.crx'

##### # Check if extension is already extracted
##### if ext_dir.exists() and (ext_dir / 'manifest.json').exists():
##### # logger.debug(f'##### Using cached {ext["name"]} extension from {_log_pretty_path(ext_dir)}')
##### extension_paths.append(str(ext_dir))
##### loaded_extension_names.append(ext['name'])
##### continue

##### try:
##### # Download extension if not cached
##### if not crx_file.exists():
##### # logger.info(f'##### Downloading {ext["name"]} extension...')
##### # self._download_extension(ext['url'], crx_file)
##### else:
##### # logger.debug(f'##### Found cached {ext["name"]} .crx file')

##### # Extract extension
##### logger.info(f'##### Extracting {ext["name"]} extension...')
##### self._extract_extension(crx_file, ext_dir)

##### extension_paths.append(str(ext_dir))
##### loaded_extension_names.append(ext['name'])

##### except Exception as e:
##### logger.warning(f'##### Failed to setup {ext["name"]} extension: {e}')
##### continue

##### # Apply minimal patch to cookie extension with configurable whitelist
##### for i, path in enumerate(extension_paths):
##### if loaded_extension_names[i] == "I still don't care about cookies":
##### self._apply_minimal_extension_patch(Path(path), self.cookie_whitelist_domains)

```



```

    if extension_paths:
        logger.debug(f'[BrowserProfile] ■ Extensions loaded ({len(extension_paths)}): [{"", ".join(loaded_ext
    else:
        logger.warning('[BrowserProfile] ■ No default extensions could be loaded')

    return extension_paths

    def _apply_minimal_extension_patch(self, ext_dir: Path, whitelist_domains: list[str]) -> None:
        """Minimal patch: pre-populate chrome.storage.local with configurable domain whitelist."""
        try:
            bg_path = ext_dir / 'data' / 'background.js'
            if not bg_path.exists():
                return

            with open(bg_path, encoding='utf-8') as f:
                content = f.read()

            # Create the whitelisted domains object for JavaScript with proper indentation
            whitelist_entries = [f'        "{domain}": true' for domain in whitelist_domains]
            whitelist_js = '{\n' + ',\n'.join(whitelist_entries) + '\n        }'

            # Find the initialize() function and inject storage setup before updateSettings()
            # The actual function uses 2-space indentation, not tabs
            old_init = """async function initialize(checkInitialized, magic) {
                if (checkInitialized && initialized) {
                    return;
                }
                loadCachedRules();
                await updateSettings();
                await recreateTabList(magic);
                initialized = true;
            }"""

            # New function with configurable whitelist initialization
            new_init = f"""// Pre-populate storage with configurable domain whitelist if empty
            async function ensureWhitelistStorage() {{
                const result = await chrome.storage.local.get({{ settings: null }});
                if (!result.settings) {{
                    const defaultSettings = {{
                        statusIndicators: true,
                        whitelistedDomains: {whitelist_js}
                    }};
                    await chrome.storage.local.set({{ settings: defaultSettings }});
                }}
            }}

            async function initialize(checkInitialized, magic) {{
                if (checkInitialized && initialized) {{
                    return;
                }}
                loadCachedRules();
                await ensureWhitelistStorage(); // Add storage initialization
                await updateSettings();
                await recreateTabList(magic);
                initialized = true;
            }}"""

            if old_init in content:
                content = content.replace(old_init, new_init)

            with open(bg_path, 'w', encoding='utf-8') as f:
                f.write(content)

            domain_list = ', '.join(whitelist_domains)
            logger.info(f'[BrowserProfile] ■ Cookie extension: {domain_list} pre-populated in storage')
        else:
            logger.debug(f'[BrowserProfile] Initialize function not found for patching')

    except Exception as e:
        logger.debug(f'[BrowserProfile] Could not patch extension storage: {e}')

    def _download_extension(self, url: str, output_path: Path) -> None:

```

```

    """Download extension .crx file."""
    import urllib.request

    try:
        with urllib.request.urlopen(url) as response:
            with open(output_path, 'wb') as f:
                f.write(response.read())
    except Exception as e:
        raise Exception(f'Failed to download extension: {e}')

    def _extract_extension(self, crx_path: Path, extract_dir: Path) -> None:
        """Extract .crx file to directory."""
        import os
        import zipfile

        # Remove existing directory
        if extract_dir.exists():
            import shutil

            shutil.rmtree(extract_dir)

        extract_dir.mkdir(parents=True, exist_ok=True)

        try:
            # CRX files are ZIP files with a header, try to extract as ZIP
            with zipfile.ZipFile(crx_path, 'r') as zip_ref:
                zip_ref.extractall(extract_dir)

            # Verify manifest exists
            if not (extract_dir / 'manifest.json').exists():
                raise Exception('No manifest.json found in extension')

        except zipfile.BadZipFile:
            # CRX files have a header before the ZIP data
            # Skip the CRX header and extract the ZIP part
            with open(crx_path, 'rb') as f:
                # Read CRX header to find ZIP start
                magic = f.read(4)
                if magic != b'Cr24':
                    raise Exception('Invalid CRX file format')

                version = int.from_bytes(f.read(4), 'little')
                if version == 2:
                    pubkey_len = int.from_bytes(f.read(4), 'little')
                    sig_len = int.from_bytes(f.read(4), 'little')
                    f.seek(16 + pubkey_len + sig_len) # Skip to ZIP data
                elif version == 3:
                    header_len = int.from_bytes(f.read(4), 'little')
                    f.seek(12 + header_len) # Skip to ZIP data

            # Extract ZIP data
            zip_data = f.read()

            # Write ZIP data to temp file and extract
            import tempfile

            with tempfile.NamedTemporaryFile(suffix='.zip', delete=False) as temp_zip:
                temp_zip.write(zip_data)
                temp_zip.flush()

            with zipfile.ZipFile(temp_zip.name, 'r') as zip_ref:
                zip_ref.extractall(extract_dir)

            os.unlink(temp_zip.name)

    def detect_display_configuration(self) -> None:
        """
        Detect the system display size and initialize the display-related config defaults:
        screen, window_size, window_position, viewport, no_viewport, device_scale_factor
        """

        display_size = get_display_size()

```

```

■■■has_screen_available = bool(display_size)
■■■self.screen = self.screen or display_size or ViewportSize(width=1920, height=1080)

■■■# if no headless preference specified, prefer headful if there is a display available
■■■if self.headless is None:
■■■self.headless = not has_screen_available

■■■# Determine viewport behavior based on mode and user preferences
■■■user_provided_viewport = self.viewport is not None

■■■if self.headless:
■■■# Headless mode: always use viewport for content size control
■■■self.viewport = self.viewport or self.window_size or self.screen
■■■self.window_position = None
■■■self.window_size = None
■■■self.no_viewport = False
■■■else:
■■■# Headful mode: respect user's viewport preference
■■■self.window_size = self.window_size or self.screen

■■■if user_provided_viewport:
■■■# User explicitly set viewport - enable viewport mode
■■■self.no_viewport = False
■■■else:
■■■# Default headful: content fits to window (no viewport)
■■■self.no_viewport = True if self.no_viewport is None else self.no_viewport

■■■# Handle special requirements (device_scale_factor forces viewport mode)
■■■if self.device_scale_factor and self.no_viewport is None:
■■■self.no_viewport = False

■■■# Finalize configuration
■■■if self.no_viewport:
■■■# No viewport mode: content adapts to window
■■■self.viewport = None
■■■self.device_scale_factor = None
■■■self.screen = None
■■■assert self.viewport is None
■■■assert self.no_viewport is True
■■■else:
■■■# Viewport mode: ensure viewport is set
■■■self.viewport = self.viewport or self.screen
■■■self.device_scale_factor = self.device_scale_factor or 1.0
■■■assert self.viewport is not None
■■■assert self.no_viewport is False

■■■assert not (self.headless and self.no_viewport), 'headless=True and no_viewport=True cannot both be s

```

.\system\browser\python_highlights.py

```

"""Python-based highlighting system for drawing bounding boxes on screenshots.

This module replaces JavaScript-based highlighting with fast Python image processing
to draw bounding boxes around interactive elements directly on screenshots.
"""

import asyncio
import base64
import io
import logging
import os

from PIL import Image, ImageDraw, ImageFont

from system.dom.views import DOMSelectorMap, EnhancedDOMTreeNode
from system.observability import observe_debug
from system.utils import time_execution_async
logger = logging.getLogger(__name__)

```

```

# Font cache to prevent repeated font loading and reduce memory usage
_FONT_CACHE: dict[tuple[str, int], ImageFont.FreeTypeFont | None] = {}

# Cross-platform font paths
_FONT_PATHS = [
    ■'/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf', # Linux (Debian/Ubuntu)
    ■'/usr/share/fonts/TTF/DejaVuSans-Bold.ttf', # Linux (Arch/Fedora)
    ■'/System/Library/Fonts/Arial.ttf', # macOS
    ■'C:\\Windows\\Fonts\\arial.ttf', # Windows
    ■'arial.ttf', # Windows (system path)
    ■'Arial Bold.ttf', # macOS alternative
    ■'/usr/share/fonts/truetype/liberation/LiberationSans-Bold.ttf', # Linux alternative
]

def get_cross_platform_font(font_size: int) -> ImageFont.FreeTypeFont | None:
    ■"""Get a cross-platform compatible font with caching to prevent memory leaks.

    ■Args:
    ■    font_size: Size of the font to load

    ■Returns:
    ■    ImageFont object or None if no system fonts are available
    ■"""
    ■# Use cache key based on font size
    ■cache_key = ('system_font', font_size)

    ■# Return cached font if available
    ■if cache_key in _FONT_CACHE:
    ■    ■return _FONT_CACHE[cache_key]

    ■# Try to load a system font
    ■font = None
    ■for font_path in _FONT_PATHS:
    ■    ■try:
    ■        ■font = ImageFont.truetype(font_path, font_size)
    ■        ■break
    ■    ■except OSError:
    ■        ■continue

    ■# Cache the result (even if None) to avoid repeated attempts
    ■_FONT_CACHE[cache_key] = font
    ■return font

def cleanup_font_cache() -> None:
    ■"""Clean up the font cache to prevent memory leaks in long-running applications."""
    ■global _FONT_CACHE
    ■_FONT_CACHE.clear()

# Color scheme for different element types
ELEMENT_COLORS = {
    ■'button': '#FF6B6B', # Red for buttons
    ■'input': '#4ECDC4', # Teal for inputs
    ■'select': '#45B7D1', # Blue for dropdowns
    ■'a': '#96CEB4', # Green for links
    ■'textarea': '#FF8C42', # Orange for text areas (was yellow, now more visible)
    ■'default': '#DDA0DD', # Light purple for other interactive elements
}

# Element type mappings
ELEMENT_TYPE_MAP = {
    ■'button': 'button',
    ■'input': 'input',
    ■'select': 'select',
    ■'a': 'a',
    ■'textarea': 'textarea',
}

def get_element_color(tag_name: str, element_type: str | None = None) -> str:

```

```

"""Get color for element based on tag name and type."""
# Check input type first
if tag_name == 'input' and element_type:
    if element_type in ['button', 'submit']:
        return ELEMENT_COLORS['button']

# Use tag-based color
return ELEMENT_COLORS.get(tag_name.lower(), ELEMENT_COLORS['default'])

def should_show_index_overlay(backend_node_id: int | None) -> bool:
    """Determine if index overlay should be shown."""
    return backend_node_id is not None

def draw_enhanced_bounding_box_with_text(
    draw, # ImageDraw.Draw - avoiding type annotation due to PIL typing issues
    bbox: tuple[int, int, int, int],
    color: str,
    text: str | None = None,
    font: ImageFont.FreeTypeFont | None = None,
    element_type: str = 'div',
    image_size: tuple[int, int] = (2000, 1500),
    device_pixel_ratio: float = 1.0,
) -> None:
    """Draw an enhanced bounding box with much bigger index containers and dashed borders."""
    x1, y1, x2, y2 = bbox

    # Draw dashed bounding box with pattern: 1 line, 2 spaces, 1 line, 2 spaces...
    dash_length = 4
    gap_length = 8
    line_width = 2

    # Helper function to draw dashed line
    def draw_dashed_line(start_x, start_y, end_x, end_y):
        if start_x == end_x: # Vertical line
            y = start_y
            while y < end_y:
                dash_end = min(y + dash_length, end_y)
                draw.line([(start_x, y), (start_x, dash_end)], fill=color, width=line_width)
                y += dash_length + gap_length
            else: # Horizontal line
                x = start_x
                while x < end_x:
                    dash_end = min(x + dash_length, end_x)
                    draw.line([(x, start_y), (dash_end, start_y)], fill=color, width=line_width)
                    x += dash_length + gap_length

    # Draw dashed rectangle
    draw_dashed_line(x1, y1, x2, y1) # Top
    draw_dashed_line(x2, y1, x2, y2) # Right
    draw_dashed_line(x2, y2, x1, y2) # Bottom
    draw_dashed_line(x1, y2, x1, y1) # Left

    # Draw much bigger index overlay if we have index text
    if text:
        try:
            # Scale font size for appropriate sizing across different resolutions
            img_width, img_height = image_size

            css_width = img_width # / device_pixel_ratio
            # Much smaller scaling - 1% of CSS viewport width, max 16px to prevent huge highlights
            base_font_size = max(10, min(20, int(css_width * 0.01)))
            # Use shared font loading function with caching
            big_font = get_cross_platform_font(base_font_size)
            if big_font is None:
                big_font = font # Fallback to original font if no system fonts found

            # Get text size with bigger font
            if big_font:
                bbox_text = draw.textbbox((0, 0), text, font=big_font)
                text_width = bbox_text[2] - bbox_text[0]

```

```

#####text_height = bbox_text[3] - bbox_text[1]
#####else:
###### Fallback for default font
#####bbox_text = draw.textbbox((0, 0), text)
#####text_width = bbox_text[2] - bbox_text[0]
#####text_height = bbox_text[3] - bbox_text[1]

###### Scale padding appropriately for different resolutions
#####padding = max(4, min(10, int(css_width * 0.005))) # 0.3% of CSS width, max 4px
#####element_width = x2 - x1
#####element_height = y2 - y1

###### Container dimensions
#####container_width = text_width + padding * 2
#####container_height = text_height + padding * 2

###### Position in top center - for small elements, place further up to avoid blocking content
###### Center horizontally within the element
#####bg_x1 = x1 + (element_width - container_width) // 2

###### Simple rule: if element is small, place index further up to avoid blocking icons
#####if element_width < 60 or element_height < 30:
###### Small element: place well above to avoid blocking content
#####bg_y1 = max(0, y1 - container_height - 5)
#####else:
###### Regular element: place inside with small offset
#####bg_y1 = y1 + 2

#####bg_x2 = bg_x1 + container_width
#####bg_y2 = bg_y1 + container_height

###### Center the number within the index box with proper baseline handling
#####text_x = bg_x1 + (container_width - text_width) // 2
###### Add extra vertical space to prevent clipping
#####text_y = bg_y1 + (container_height - text_height) // 2 - bbox_text[1] # Subtract top offset

###### Ensure container stays within image bounds
#####img_width, img_height = image_size
#####if bg_x1 < 0:
#####offset = -bg_x1
#####bg_x1 += offset
#####bg_x2 += offset
#####text_x += offset
#####if bg_y1 < 0:
#####offset = -bg_y1
#####bg_y1 += offset
#####bg_y2 += offset
#####text_y += offset
#####if bg_x2 > img_width:
#####offset = bg_x2 - img_width
#####bg_x1 -= offset
#####bg_x2 -= offset
#####text_x -= offset
#####if bg_y2 > img_height:
#####offset = bg_y2 - img_height
#####bg_y1 -= offset
#####bg_y2 -= offset
#####text_y -= offset

###### Draw bigger background rectangle with thicker border
#####draw.rectangle([bg_x1, bg_y1, bg_x2, bg_y2], fill=color, outline='white', width=2)

###### Draw white text centered in the index box
#####draw.text((text_x, text_y), text, fill='white', font=big_font or font)

#####except Exception as e:
#####logger.debug(f'Failed to draw enhanced text overlay: {e}')

def draw_bounding_box_with_text(
    draw, # ImageDraw.Draw - avoiding type annotation due to PIL typing issues
    bbox: tuple[int, int, int, int],

```

```

■color: str,
■text: str | None = None,
■font: ImageFont.FreeTypeFont | None = None,
) -> None:
■"""Draw a bounding box with optional text overlay."""
■x1, y1, x2, y2 = bbox

■# Draw dashed bounding box
■dash_length = 2
■gap_length = 6

■# Top edge
■x = x1
■while x < x2:
■    end_x = min(x + dash_length, x2)
■    draw.line([(x, y1), (end_x, y1)], fill=color, width=2)
■    draw.line([(x, y1 + 1), (end_x, y1 + 1)], fill=color, width=2)
■    x += dash_length + gap_length

■# Bottom edge
■x = x1
■while x < x2:
■    end_x = min(x + dash_length, x2)
■    draw.line([(x, y2), (end_x, y2)], fill=color, width=2)
■    draw.line([(x, y2 - 1), (end_x, y2 - 1)], fill=color, width=2)
■    x += dash_length + gap_length

■# Left edge
■y = y1
■while y < y2:
■    end_y = min(y + dash_length, y2)
■    draw.line([(x1, y), (x1, end_y)], fill=color, width=2)
■    draw.line([(x1 + 1, y), (x1 + 1, end_y)], fill=color, width=2)
■    y += dash_length + gap_length

■# Right edge
■y = y1
■while y < y2:
■    end_y = min(y + dash_length, y2)
■    draw.line([(x2, y), (x2, end_y)], fill=color, width=2)
■    draw.line([(x2 - 1, y), (x2 - 1, end_y)], fill=color, width=2)
■    y += dash_length + gap_length

■# Draw index overlay if we have index text
■if text:
■    try:
■        # Get text size
■        if font:
■            bbox_text = draw.textbbox((0, 0), text, font=font)
■            text_width = bbox_text[2] - bbox_text[0]
■            text_height = bbox_text[3] - bbox_text[1]
■        else:
■            # Fallback for default font
■            bbox_text = draw.textbbox((0, 0), text)
■            text_width = bbox_text[2] - bbox_text[0]
■            text_height = bbox_text[3] - bbox_text[1]

■        # Smart positioning based on element size
■        padding = 5
■        element_width = x2 - x1
■        element_height = y2 - y1
■        element_area = element_width * element_height
■        index_box_area = (text_width + padding * 2) * (text_height + padding * 2)

■        # Calculate size ratio to determine positioning strategy
■        size_ratio = element_area / max(index_box_area, 1)

■        if size_ratio < 4:
■            # Very small elements: place outside in bottom-right corner
■            text_x = x2 + padding
■            text_y = y2 - text_height
■            # Ensure it doesn't go off screen

```

```

#####text_x = min(text_x, 1200 - text_width - padding)
#####text_y = max(text_y, 0)
#####elif size_ratio < 16:
###### Medium elements: place in bottom-right corner inside
#####text_x = x2 - text_width - padding
#####text_y = y2 - text_height - padding
#####else:
###### Large elements: place in center
#####text_x = x1 + (element_width - text_width) // 2
#####text_y = y1 + (element_height - text_height) // 2

###### Ensure text stays within bounds
#####text_x = max(0, min(text_x, 1200 - text_width))
#####text_y = max(0, min(text_y, 800 - text_height))

###### Draw background rectangle for maximum contrast
#####bg_x1 = text_x - padding
#####bg_y1 = text_y - padding
#####bg_x2 = text_x + text_width + padding
#####bg_y2 = text_y + text_height + padding

###### Use white background with thick black border for maximum visibility
#####draw.rectangle([bg_x1, bg_y1, bg_x2, bg_y2], fill='white', outline='black', width=2)

###### Draw bold dark text on light background for best contrast
#####draw.text((text_x, text_y), text, fill='black', font=font)

#####except Exception as e:
#####logger.debug(f'Failed to draw text overlay: {e}')

def process_element_highlight(
    #element_id: int,
    #element: EnhancedDOMTreeNode,
    #draw,
    #device_pixel_ratio: float,
    #font,
    #filter_highlight_ids: bool,
    #image_size: tuple[int, int],
) -> None:
    """Process a single element for highlighting."""
    #try:
    ###### Use absolute_position coordinates directly
    #####if not element.absolute_position:
    #####return

    #####bounds = element.absolute_position

    ###### Scale coordinates from CSS pixels to device pixels for screenshot
    ###### The screenshot is captured at device pixel resolution, but coordinates are in CSS pixels
    #####x1 = int(bounds.x * device_pixel_ratio)
    #####y1 = int(bounds.y * device_pixel_ratio)
    #####x2 = int((bounds.x + bounds.width) * device_pixel_ratio)
    #####y2 = int((bounds.y + bounds.height) * device_pixel_ratio)

    ###### Ensure coordinates are within image bounds
    #####img_width, img_height = image_size
    #####x1 = max(0, min(x1, img_width))
    #####y1 = max(0, min(y1, img_height))
    #####x2 = max(x1, min(x2, img_width))
    #####y2 = max(y1, min(y2, img_height))

    ###### Skip if bounding box is too small or invalid
    #####if x2 - x1 < 2 or y2 - y1 < 2:
    #####return

    ###### Get element color based on type
    #####tag_name = element.tag_name if hasattr(element, 'tag_name') else 'div'
    #####element_type = None
    #####if hasattr(element, 'attributes') and element.attributes:
    #####element_type = element.attributes.get('type')
    #####color = get_element_color(tag_name, element_type)

```



```

    ■■■# Get element index for overlay and apply filtering
    ■■■backend_node_id = getattr(element, 'backend_node_id', None)
    ■■■index_text = None

    ■■■if backend_node_id is not None:
    ■■■# if filter_highlight_ids:
    ■■■# Use the meaningful text that matches what the LLM sees
    ■■■meaningful_text = element.get_meaningful_text_for_llm()
    ■■■# Show ID only if meaningful text is less than 5 characters
    ■■■if len(meaningful_text) < 3:
    ■■■# index_text = str(backend_node_id)
    ■■■else:
    ■■■# Always show ID when filter is disabled
    ■■■index_text = str(backend_node_id)

    ■■■# Draw enhanced bounding box with bigger index
    ■■■draw_enhanced_bounding_box_with_text(
    ■■■draw, (x1, y1, x2, y2), color, index_text, font, tag_name, image_size, device_pixel_ratio
    ■■■)

    ■■■except Exception as e:
    ■■■logger.debug(f'Failed to draw highlight for element {element_id}: {e}')

@observe_debug(ignore_input=True, ignore_output=True, name='create_highlighted_screenshot')
@time_execution_async('create_highlighted_screenshot')
async def create_highlighted_screenshot(
    ■■■screenshot_b64: str,
    ■■■selector_map: DOMSelectorMap,
    ■■■device_pixel_ratio: float = 1.0,
    ■■■viewport_offset_x: int = 0,
    ■■■viewport_offset_y: int = 0,
    ■■■filter_highlight_ids: bool = True,
) -> str:
    ■■■"""Create a highlighted screenshot with bounding boxes around interactive elements.

    ■■■Args:
    ■■■    screenshot_b64: Base64 encoded screenshot
    ■■■    selector_map: Map of interactive elements with their positions
    ■■■    device_pixel_ratio: Device pixel ratio for scaling coordinates
    ■■■    viewport_offset_x: X offset for viewport positioning
    ■■■    viewport_offset_y: Y offset for viewport positioning

    ■■■Returns:
    ■■■    Base64 encoded highlighted screenshot
    ■■■    """
    ■■■try:
    ■■■# Decode screenshot
    ■■■screenshot_data = base64.b64decode(screenshot_b64)
    ■■■image = Image.open(io.BytesIO(screenshot_data)).convert('RGBA')

    ■■■# Create drawing context
    ■■■draw = ImageDraw.Draw(image)

    ■■■# Load font using shared function with caching
    ■■■font = get_cross_platform_font(12)
    ■■■# If no system fonts found, font remains None and will use default font

    ■■■# Process elements sequentially to avoid ImageDraw thread safety issues
    ■■■# PIL ImageDraw is not thread-safe, so we process elements one by one
    ■■■for element_id, element in selector_map.items():
    ■■■# process_element_highlight(element_id, element, draw, device_pixel_ratio, font, filter_highlight_ids,

    ■■■# Convert back to base64
    ■■■output_buffer = io.BytesIO()
    ■■■try:
    ■■■# image.save(output_buffer, format='PNG')
    ■■■# output_buffer.seek(0)
    ■■■# highlighted_b64 = base64.b64encode(output_buffer.getvalue()).decode('utf-8')

    ■■■# logger.debug(f'Successfully created highlighted screenshot with {len(selector_map)} elements')
    ■■■return highlighted_b64

```

```

    finally:
        # Explicit cleanup to prevent memory leaks
        output_buffer.close()
        if 'image' in locals():
            image.close()

    except Exception as e:
        logger.error(f'Failed to create highlighted screenshot: {e}')
        # Clean up on error as well
        if 'image' in locals():
            image.close()
        # Return original screenshot on error
        return screenshot_b64

async def get_viewport_info_from_cdp(cdp_session) -> tuple[float, int, int]:
    """Get viewport information from CDP session.

    Returns:
        Tuple of (device_pixel_ratio, scroll_x, scroll_y)
    """
    try:
        # Get layout metrics which includes viewport info and device pixel ratio
        metrics = await cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=cdp_session.session_id)

        # Extract viewport information
        visual_viewport = metrics.get('visualViewport', {})
        css_visual_viewport = metrics.get('cssVisualViewport', {})
        css_layout_viewport = metrics.get('cssLayoutViewport', {})

        # Calculate device pixel ratio
        css_width = css_visual_viewport.get('clientWidth', css_layout_viewport.get('clientWidth', 1280.0))
        device_width = visual_viewport.get('clientWidth', css_width)
        device_pixel_ratio = device_width / css_width if css_width > 0 else 1.0

        # Get scroll position in CSS pixels
        scroll_x = int(css_visual_viewport.get('pageX', 0))
        scroll_y = int(css_visual_viewport.get('pageY', 0))

        return float(device_pixel_ratio), scroll_x, scroll_y

    except Exception as e:
        logger.debug(f'Failed to get viewport info from CDP: {e}')
        return 1.0, 0, 0

@time_execution_async('create_highlighted_screenshot_async')
async def create_highlighted_screenshot_async(
    screenshot_b64: str, selector_map: DOMSelectorMap, cdp_session=None, filter_highlight_ids: bool = True
) -> str:
    """Async wrapper for creating highlighted screenshots.

    Args:
        screenshot_b64: Base64 encoded screenshot
        selector_map: Map of interactive elements
        cdp_session: CDP session for getting viewport info
        filter_highlight_ids: Whether to filter element IDs based on meaningful text

    Returns:
        Base64 encoded highlighted screenshot
    """
    # Get viewport information if CDP session is available
    device_pixel_ratio = 1.0
    viewport_offset_x = 0
    viewport_offset_y = 0

    if cdp_session:
        try:
            device_pixel_ratio, viewport_offset_x, viewport_offset_y = await get_viewport_info_from_cdp(cdp_session)
        except Exception as e:
            logger.debug(f'Failed to get viewport info from CDP: {e}')
        # Create highlighted screenshot with async processing

```

```

    final_screenshot = await create_highlighted_screenshot(
        screenshot_b64, selector_map, device_pixel_ratio, viewport_offset_x, viewport_offset_y, filter_highlighted)

    filename = os.getenv('BROWSER_USE_SCREENSHOT_FILE')
    if filename:

        def _write_screenshot():
            try:
                with open(filename, 'wb') as f:
                    f.write(base64.b64decode(final_screenshot))
                logger.debug('Saved screenshot to ' + str(filename))
            except Exception as e:
                logger.warning(f'Failed to save screenshot to {filename}: {e}')

        await asyncio.to_thread(_write_screenshot)
    return final_screenshot

# Export the cleanup function for external use in long-running applications
__all__ = ['create_highlighted_screenshot', 'create_highlighted_screenshot_async', 'cleanup_font_cache']

```

.system\browser\session.py

```

"""Event-driven browser session with backwards compatibility."""

import asyncio
import logging
import time
from functools import cached_property
from pathlib import Path
from typing import TYPE_CHECKING, Any, Literal, Self, Union, cast, overload
from urllib.parse import urlparse, urlunparse
from uuid import UUID

import httpx
from bubus import EventBus
from cdp_use import CDPClient
from cdp_use.cdp.fetch import AuthRequiredEvent, RequestPausedEvent
from cdp_use.cdp.network import Cookie
from cdp_use.cdp.target import SessionID, TargetID
from pydantic import BaseModel, ConfigDict, Field, PrivateAttr
from uuid_extensions import uuid7str

from system.browser.cloud.cloud import CloudBrowserAuthError, CloudBrowserClient, CloudBrowserError

# CDP logging is now handled by setup_logging() in logging_config.py
# It automatically sets CDP logs to the same level as system logs
from system.browser.cloud.views import CloudBrowserParams, CreateBrowserRequest, ProxyCountryCode
from system.browser.events import (
    AgentFocusChangedEvent,
    BrowserConnectedEvent,
    BrowserErrorEvent,
    BrowserLaunchEvent,
    BrowserLaunchResult,
    BrowserReconnectedEvent,
    BrowserReconnectingEvent,
    BrowserStartEvent,
    BrowserStateRequestEvent,
    BrowserStopEvent,
    BrowserStoppedEvent,
    CloseTabEvent,
    FileDownloadedEvent,
    NavigateToUrlEvent,
    NavigationCompleteEvent,
    NavigationStartedEvent,
    SwitchTabEvent,
    TabClosedEvent,

```

```

    TabCreatedEvent,
)
from system.browser.profile import BrowserProfile, ProxySettings
from system.browser.views import BrowserStateSummary, TabInfo
from system.dom.views import DOMRect, EnhancedDOMTreeNode, TargetInfo
from system.observability import observe_debug
from system.utils import _log_pretty_url, create_task_with_error_handling, is_new_tab_page

if TYPE_CHECKING:
    from system.actor.page import Page
    from system.browser.demo_mode import DemoMode
    from system.browser.watchdogs.captcha_watchdog import CaptchaWaitResult

DEFAULT_BROWSER_PROFILE = BrowserProfile()

_LOGGED_UNIQUE_SESSION_IDS = set() # track unique session IDs that have been logged to make sure we alw
red = '\033[91m'
reset = '\033[0m'

class Target(BaseModel):
    """Browser target (page, iframe, worker) - the actual entity being controlled.

    A target represents a browsing context with its own URL, title, and type.
    Multiple CDP sessions can attach to the same target for communication.
    """
    model_config = ConfigDict(arbitrary_types_allowed=True, revalidate_instances='never')

    target_id: TargetID
    target_type: str # 'page', 'iframe', 'worker', etc.
    url: str = 'about:blank'
    title: str = 'Unknown title'

class CDPSession(BaseModel):
    """CDP communication channel to a target.

    A session is a connection that allows sending CDP commands to a specific target.
    Multiple sessions can attach to the same target.
    """
    model_config = ConfigDict(arbitrary_types_allowed=True, revalidate_instances='never')

    cdp_client: CDPClient
    target_id: TargetID
    session_id: SessionID

    # Lifecycle monitoring (populated by SessionManager)
    _lifecycle_events: Any = PrivateAttr(default=None)
    _lifecycle_lock: Any = PrivateAttr(default=None)

class BrowserSession(BaseModel):
    """Event-driven browser session with backwards compatibility.

    This class provides a 2-layer architecture:
    - High-level event handling for agents/tools
    - Direct CDP/Playwright calls for browser operations

    Supports both event-driven and imperative calling styles.

    Browser configuration is stored in the browser_profile, session identity in direct fields:
    ```python
 # Direct settings (recommended for most users)
 session = BrowserSession(headless=True, user_data_dir='./profile')

 # Or use a profile (for advanced use cases)
 session = BrowserSession(browser_profile=BrowserProfile(...))

 # Access session fields directly, browser settings via profile or property
 print(session.id) # Session field

```

```

 """

 model_config = ConfigDict(
 arbitrary_types_allowed=True,
 validate_assignment=True,
 extra='forbid',
 revalidate_instances='never', # resets private attrs on every model rebuild
)

 ## Overload 1: Cloud browser mode (use cloud-specific params)
 @overload
 def __init__(
 self,
 *,
 ## Cloud browser params - use these for cloud mode
 cloud_profile_id: UUID | str | None = None,
 cloud_proxy_country_code: ProxyCountryCode | None = None,
 cloud_timeout: int | None = None,
 ## Backward compatibility aliases
 profile_id: UUID | str | None = None,
 proxy_country_code: ProxyCountryCode | None = None,
 timeout: int | None = None,
 use_cloud: bool | None = None,
 cloud_browser: bool | None = None, # Backward compatibility alias
 cloud_browser_params: CloudBrowserParams | None = None,
 ## Common params that work with cloud
 id: str | None = None,
 headers: dict[str, str] | None = None,
 allowed_domains: list[str] | None = None,
 prohibited_domains: list[str] | None = None,
 keep_alive: bool | None = None,
 minimum_wait_page_load_time: float | None = None,
 wait_for_network_idle_page_load_time: float | None = None,
 wait_between_actions: float | None = None,
 captcha_solver: bool | None = None,
 auto_download_pdfs: bool | None = None,
 cookie_whitelist_domains: list[str] | None = None,
 cross_origin_iframes: bool | None = None,
 highlight_elements: bool | None = None,
 dom_highlight_elements: bool | None = None,
 paint_order_filtering: bool | None = None,
 max_iframes: int | None = None,
 max_iframe_depth: int | None = None,
) -> None: ...

 ## Overload 2: Local browser mode (use local browser params)
 @overload
 def __init__(
 self,
 *,
 ## Core configuration for local
 id: str | None = None,
 cdp_url: str | None = None,
 browser_profile: BrowserProfile | None = None,
 ## Local browser launch params
 executable_path: str | Path | None = None,
 headless: bool | None = None,
 user_data_dir: str | Path | None = None,
 args: list[str] | None = None,
 downloads_path: str | Path | None = None,
 ## Common params
 headers: dict[str, str] | None = None,
 allowed_domains: list[str] | None = None,
 prohibited_domains: list[str] | None = None,
 keep_alive: bool | None = None,
 minimum_wait_page_load_time: float | None = None,
 wait_for_network_idle_page_load_time: float | None = None,
 wait_between_actions: float | None = None,
 auto_download_pdfs: bool | None = None,
 cookie_whitelist_domains: list[str] | None = None,
 cross_origin_iframes: bool | None = None,

```

```

 highlight_elements: bool | None = None,
 dom_highlight_elements: bool | None = None,
 paint_order_filtering: bool | None = None,
 max_iframes: int | None = None,
 max_iframe_depth: int | None = None,
 ## All other local params
 env: dict[str, str | float | bool] | None = None,
 ignore_default_args: list[str] | Literal[True] | None = None,
 channel: str | None = None,
 chromium_sandbox: bool | None = None,
 devtools: bool | None = None,
 traces_dir: str | Path | None = None,
 accept_downloads: bool | None = None,
 permissions: list[str] | None = None,
 user_agent: str | None = None,
 screen: dict | None = None,
 viewport: dict | None = None,
 no_viewport: bool | None = None,
 device_scale_factor: float | None = None,
 record_har_content: str | None = None,
 record_har_mode: str | None = None,
 record_har_path: str | Path | None = None,
 record_video_dir: str | Path | None = None,
 record_video_framerate: int | None = None,
 record_video_size: dict | None = None,
 storage_state: str | Path | dict[str, Any] | None = None,
 disable_security: bool | None = None,
 deterministic_rendering: bool | None = None,
 proxy: ProxySettings | None = None,
 enable_default_extensions: bool | None = None,
 captcha_solver: bool | None = None,
 window_size: dict | None = None,
 window_position: dict | None = None,
 filter_highlight_ids: bool | None = None,
 profile_directory: str | None = None,
) -> None: ...

```

```

def __init__(
 self,
 ## Core configuration
 id: str | None = None,
 cdp_url: str | None = None,
 is_local: bool = False,
 browser_profile: BrowserProfile | None = None,
 ## Cloud browser params (don't mix with local browser params)
 cloud_profile_id: UUID | str | None = None,
 cloud_proxy_country_code: ProxyCountryCode | None = None,
 cloud_timeout: int | None = None,
 ## Backward compatibility aliases for cloud params
 profile_id: UUID | str | None = None,
 proxy_country_code: ProxyCountryCode | None = None,
 timeout: int | None = None,
 ## BrowserProfile fields that can be passed directly
 ## From BrowserConnectArgs
 headers: dict[str, str] | None = None,
 ## From BrowserLaunchArgs
 env: dict[str, str | float | bool] | None = None,
 executable_path: str | Path | None = None,
 headless: bool | None = None,
 args: list[str] | None = None,
 ignore_default_args: list[str] | Literal[True] | None = None,
 channel: str | None = None,
 chromium_sandbox: bool | None = None,
 devtools: bool | None = None,
 downloads_path: str | Path | None = None,
 traces_dir: str | Path | None = None,
 ## From BrowserContextArgs
 accept_downloads: bool | None = None,
 permissions: list[str] | None = None,
 user_agent: str | None = None,
 screen: dict | None = None,
 viewport: dict | None = None,

```

```

no_viewport: bool | None = None,
device_scale_factor: float | None = None,
record_har_content: str | None = None,
record_har_mode: str | None = None,
record_har_path: str | Path | None = None,
record_video_dir: str | Path | None = None,
record_video_framerate: int | None = None,
record_video_size: dict | None = None,
From BrowserLaunchPersistentContextArgs
user_data_dir: str | Path | None = None,
From BrowserNewContextArgs
storage_state: str | Path | dict[str, Any] | None = None,
BrowserProfile specific fields
Cloud Browser Fields
use_cloud: bool | None = None,
cloud_browser: bool | None = None, # Backward compatibility alias
cloud_browser_params: CloudBrowserParams | None = None,
Other params
disable_security: bool | None = None,
deterministic_rendering: bool | None = None,
allowed_domains: list[str] | None = None,
prohibited_domains: list[str] | None = None,
keep_alive: bool | None = None,
proxy: ProxySettings | None = None,
enable_default_extensions: bool | None = None,
captcha_solver: bool | None = None,
window_size: dict | None = None,
window_position: dict | None = None,
minimum_wait_page_load_time: float | None = None,
wait_for_network_idle_page_load_time: float | None = None,
wait_between_actions: float | None = None,
filter_highlight_ids: bool | None = None,
auto_download_pdfs: bool | None = None,
profile_directory: str | None = None,
cookie_whitelist_domains: list[str] | None = None,
DOM extraction layer configuration
cross_origin_iframes: bool | None = None,
highlight_elements: bool | None = None,
dom_highlight_elements: bool | None = None,
paint_order_filtering: bool | None = None,
Iframe processing limits
max_iframes: int | None = None,
max_iframe_depth: int | None = None,
):
Following the same pattern as AgentSettings in service.py
Only pass non-None values to avoid validation errors
profile_kwargs = {
 k: v
 for k, v in locals().items()
 if k
 not in [
 'self',
 'browser_profile',
 'id',
 'cloud_profile_id',
 'cloud_proxy_country_code',
 'cloud_timeout',
 'profile_id',
 'proxy_country_code',
 'timeout',
]
 and v is not None
}

Handle backward compatibility: prefer cloud_* params over old names
final_profile_id = cloud_profile_id if cloud_profile_id is not None else profile_id
final_proxy_country_code = cloud_proxy_country_code if cloud_proxy_country_code is not None else proxy_country_code
final_timeout = cloud_timeout if cloud_timeout is not None else timeout

If any cloud params are provided, create cloud_browser_params
if final_profile_id is not None or final_proxy_country_code is not None or final_timeout is not None:
 cloud_params = CreateBrowserRequest(

```

```

#####cloud_profile_id=final_profile_id,
#####cloud_proxy_country_code=final_proxy_country_code,
#####cloud_timeout=final_timeout,
#####)
#####profile_kwargs['cloud_browser_params'] = cloud_params
#####profile_kwargs['use_cloud'] = True

Handle backward compatibility: map cloud_browser to use_cloud
#####if 'cloud_browser' in profile_kwargs:
#####profile_kwargs['use_cloud'] = profile_kwargs.pop('cloud_browser')

If cloud_browser_params is set, force use_cloud=True
#####if cloud_browser_params is not None:
#####profile_kwargs['use_cloud'] = True

if is_local is False but executable_path is provided, set is_local to True
#####if is_local is False and executable_path is not None:
#####profile_kwargs['is_local'] = True
Only set is_local=True when cdp_url is missing if we're not using cloud browser
(cloud browser will provide cdp_url later)
#####use_cloud = profile_kwargs.get('use_cloud') or profile_kwargs.get('cloud_browser')
#####if not cdp_url and not use_cloud:
#####profile_kwargs['is_local'] = True

Create browser profile from direct parameters or use provided one
#####if browser_profile is not None:
Merge any direct kwargs into the provided browser_profile (direct kwargs take precedence)
#####merged_kwargs = {**browser_profile.model_dump(exclude_unset=True), **profile_kwargs}
#####resolved_browser_profile = BrowserProfile(**merged_kwargs)
#####else:
#####resolved_browser_profile = BrowserProfile(**profile_kwargs)

Initialize the Pydantic model
#####super().__init__(
#####id=id or str(uuid7str()),
#####browser_profile=resolved_browser_profile,
#####)

Session configuration (session identity only)
#####id: str = Field(default_factory=lambda: str(uuid7str()), description='Unique identifier for this browser')

Browser configuration (reusable profile)
#####browser_profile: BrowserProfile = Field(
#####default_factory=lambda: DEFAULT_BROWSER_PROFILE,
#####description='BrowserProfile() options to use for the session, otherwise a default profile will be used')

LLM screenshot resizing configuration
#####llm_screenshot_size: tuple[int, int] | None = Field(
#####default=None,
#####description='Target size (width, height) to resize screenshots before sending to LLM. Coordinates from top-left')

Cache of original viewport size for coordinate conversion (set when browser state is captured)
#####_original_viewport_size: tuple[int, int] | None = PrivateAttr(default=None)

#####@classmethod
#####def from_system_chrome(cls, profile_directory: str | None = None, **kwargs: Any) -> Self:
"""Create a BrowserSession using system's Chrome installation and profile"""
from system.skill_cli.utils import find_chrome_executable, get_chrome_profile_path, list_chrome_profiles

#####executable_path = find_chrome_executable()
#####if executable_path is None:
#####raise RuntimeError(
#####'Chrome not found. Please install Chrome or use Browser() with explicit executable_path.\n'
#####'Expected locations:\n'
#####' macOS: /Applications/Google Chrome.app/Contents/MacOS/Google Chrome\n'
#####' Linux: /usr/bin/google-chrome or /usr/bin/chromium\n'
#####' Windows: C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe')

#####user_data_dir = get_chrome_profile_path(None)

```



```

 if user_data_dir is None:
 raise RuntimeError(
 'Could not detect Chrome profile directory for your platform.\n'
 'Expected locations:\n'
 ' macOS: ~/Library/Application Support/Google/Chrome\n'
 ' Linux: ~/.config/google-chrome\n'
 ' Windows: %LocalAppData%\Google\Chrome\User Data'
)

 # Auto-select profile if not specified
 profiles = list_chrome_profiles()
 if profile_directory is None:
 if profiles:
 # Use first available profile
 profile_directory = profiles[0]['directory']
 logging.getLogger('system').info(
 f'Auto-selected Chrome profile: {profiles[0]["name"]} ({profile_directory})'
)
 else:
 profile_directory = 'Default'

 return cls(
 executable_path=executable_path,
 user_data_dir=user_data_dir,
 profile_directory=profile_directory,
 **kwargs,
)

 @classmethod
 def list_chrome_profiles(cls) -> list[dict[str, str]]:
 """List available Chrome profiles on the system"""
 from system.skill_cli.utils import list_chrome_profiles

 return list_chrome_profiles()

 # Convenience properties for common browser settings
 @property
 def cdp_url(self) -> str | None:
 """CDP URL from browser profile."""
 return self.browser_profile.cdp_url

 @property
 def is_local(self) -> bool:
 """Whether this is a local browser instance from browser profile."""
 return self.browser_profile.is_local

 @property
 def is_cdp_connected(self) -> bool:
 """Check if the CDP WebSocket connection is alive and usable.

 Returns True only if the root CDP client exists and its WebSocket is in OPEN state.
 A dead/closing/closed WebSocket returns False, preventing handlers from dispatching
 CDP commands that would hang until timeout on a broken connection.
 """
 if self._cdp_client_root is None or self._cdp_client_root.ws is None:
 return False
 try:
 from websockets.protocol import State

 return self._cdp_client_root.ws.state is State.OPEN
 except Exception:
 return False

 async def wait_if_captcha_solving(self, timeout: float | None = None) -> 'CaptchaWaitResult | None':
 """Wait if a captcha is currently being solved by the browser proxy.

 Returns:
 - A CaptchaWaitResult if we had to wait, or None if no captcha was in progress.
 """
 if self._captcha_watchdog is not None:
 return await self._captcha_watchdog.wait_if_captcha_solving(timeout=timeout)
 return None

```

```

 @property
 def is_reconnecting(self) -> bool:
 """Whether a WebSocket reconnection attempt is currently in progress."""
 return self._reconnecting

 @property
 def cloud_browser(self) -> bool:
 """Whether to use cloud browser service from browser profile."""
 return self.browser_profile.use_cloud

 @property
 def demo_mode(self) -> 'DemoMode | None':
 """Lazy init demo mode helper when enabled."""
 if not self.browser_profile.demo_mode:
 return None
 if self._demo_mode is None:
 from system.browser.demo_mode import DemoMode

 self._demo_mode = DemoMode(self)
 return self._demo_mode

 # Main shared event bus for all browser session + all watchdogs
 event_bus: EventBus = Field(default_factory=EventBus)

 # Mutable public state - which target has agent focus
 agent_focus_target_id: TargetID | None = None

 # Mutable private state shared between watchdogs
 _cdp_client_root: CDPClient | None = PrivateAttr(default=None)
 _connection_lock: Any = PrivateAttr(default=None) # asyncio.Lock for preventing concurrent connections

 # PUBLIC: SessionManager instance (OWNS all targets and sessions)
 session_manager: Any = Field(default=None, exclude=True) # SessionManager

 _cached_browser_state_summary: Any = PrivateAttr(default=None)
 _cached_selector_map: dict[int, EnhancedDOMTreeNode] = PrivateAttr(default_factory=dict)
 _downloaded_files: list[str] = PrivateAttr(default_factory=list) # Track files downloaded during this
 _closed_popup_messages: list[str] = PrivateAttr(default_factory=list) # Store messages from auto-close

 # Watchdogs
 _crash_watchdog: Any | None = PrivateAttr(default=None)
 _downloads_watchdog: Any | None = PrivateAttr(default=None)
 _aboutblank_watchdog: Any | None = PrivateAttr(default=None)
 _security_watchdog: Any | None = PrivateAttr(default=None)
 _storage_state_watchdog: Any | None = PrivateAttr(default=None)
 _local_browser_watchdog: Any | None = PrivateAttr(default=None)
 _default_action_watchdog: Any | None = PrivateAttr(default=None)
 _dom_watchdog: Any | None = PrivateAttr(default=None)
 _screenshot_watchdog: Any | None = PrivateAttr(default=None)
 _permissions_watchdog: Any | None = PrivateAttr(default=None)
 _recording_watchdog: Any | None = PrivateAttr(default=None)
 _captcha_watchdog: Any | None = PrivateAttr(default=None)
 _watchdogs_attached: bool = PrivateAttr(default=False)

 _cloud_browser_client: CloudBrowserClient = PrivateAttr(default_factory=lambda: CloudBrowserClient())
 _demo_mode: 'DemoMode | None' = PrivateAttr(default=None)

 # WebSocket reconnection state
 # Max wait = attempts * timeout_per_attempt + sum(delays) + small buffer
 # Default: 3 * 15s + (1+2+4)s + 2s = 54s
 RECONNECT_WAIT_TIMEOUT: float = 54.0
 _reconnecting: bool = PrivateAttr(default=False)
 _reconnect_event: asyncio.Event = PrivateAttr(default_factory=asyncio.Event)
 _reconnect_lock: asyncio.Lock = PrivateAttr(default_factory=asyncio.Lock)
 _reconnect_task: asyncio.Task | None = PrivateAttr(default=None)
 _intentional_stop: bool = PrivateAttr(default=False)

 _logger: Any = PrivateAttr(default=None)

 @property
 def logger(self) -> Any:
 """Get instance-specific logger with session ID in the name"""

```

```

 ## **regenerate it every time** because our id and str(self) can change as browser connection state changes
 ## if self._logger is None or not self._cdp_client_root:
 ## self._logger = logging.getLogger(f'system.{self}')
 ## return logging.getLogger(f'system.{self}')

 @cached_property
 def _id_for_logs(self) -> str:
 """Get human-friendly semi-unique identifier for differentiating different BrowserSession instances in logs
 str_id = self.id[-4:] # default to last 4 chars of truly random uuid, less helpful than cdp port but more unique
 port_number = (self.cdp_url or 'no-cdp').rsplit(':', 1)[-1].split('/', 1)[0].strip()
 port_is_random = not port_number.startswith('922')
 port_is_unique_enough = port_number not in _LOGGED_UNIQUE_SESSION_IDS
 if port_number and port_number.isdigit() and port_is_random and port_is_unique_enough:
 # if cdp port is random/unique enough to identify this session, use it as our id in logs
 _LOGGED_UNIQUE_SESSION_IDS.add(port_number)
 str_id = port_number
 return str_id

 @property
 def _tab_id_for_logs(self) -> str:
 return self.agent_focus_target_id[-2:] if self.agent_focus_target_id else f'{red}--{reset}'

 def __repr__(self) -> str:
 return f'BrowserSession {self._id_for_logs} {self._tab_id_for_logs} (cdp_url={self.cdp_url}, profile={self.profile})'

 def __str__(self) -> str:
 return f'BrowserSession {self._id_for_logs} {self._tab_id_for_logs}'

 async def reset(self) -> None:
 """Clear all cached CDP sessions with proper cleanup."""

 ## Suppress auto-reconnect callback during teardown
 self._intentional_stop = True
 ## Cancel any in-flight reconnection task
 if self._reconnect_task and not self._reconnect_task.done():
 self._reconnect_task.cancel()
 self._reconnect_task = None
 self._reconnecting = False
 self._reconnect_event.set() # unblock any waiters

 cdp_status = 'connected' if self._cdp_client_root else 'not connected'
 session_mgr_status = 'exists' if self.session_manager else 'None'
 self.logger.debug(
 f'Resetting browser session (CDP: {cdp_status}, SessionManager: {session_mgr_status}, '
 f'focus: {self.agent_focus_target_id[-4:] if self.agent_focus_target_id else "None"})'
)

 ## Clear session manager (which owns _targets, _sessions, _target_sessions)
 if self.session_manager:
 await self.session_manager.clear()
 self.session_manager = None

 ## Close CDP WebSocket before clearing to prevent stale event handlers
 if self._cdp_client_root:
 try:
 await self._cdp_client_root.stop()
 self.logger.debug('Closed CDP client WebSocket during reset')
 except Exception as e:
 self.logger.debug(f'Error closing CDP client during reset: {e}')

 self._cdp_client_root = None # type: ignore
 self._cached_browser_state_summary = None
 self._cached_selector_map.clear()
 self._downloaded_files.clear()

 self.agent_focus_target_id = None
 if self.is_local:
 self.browser_profile.cdp_url = None

 self._crash_watchdog = None
 self._downloads_watchdog = None
 self._aboutblank_watchdog = None

```

```

 self._security_watchdog = None
 self._storage_state_watchdog = None
 self._local_browser_watchdog = None
 self._default_action_watchdog = None
 self._dom_watchdog = None
 self._screenshot_watchdog = None
 self._permissions_watchdog = None
 self._recording_watchdog = None
 self._captcha_watchdog = None
 self._watchdogs_attached = False
 if self._demo_mode:
 self._demo_mode.reset()
 self._demo_mode = None

 self._intentional_stop = False
 self.logger.info('■ Browser session reset complete')

 def model_post_init(self, __context) -> None:
 """Register event handlers after model initialization."""
 self._connection_lock = asyncio.Lock()
 ## Initialize reconnect event as set (no reconnection pending)
 self._reconnect_event = asyncio.Event()
 self._reconnect_event.set()

 ## Check if handlers are already registered to prevent duplicates
 from system.browser.watchdog_base import BaseWatchdog

 start_handlers = self.event_bus.handlers.get('BrowserStartEvent', [])
 start_handler_names = [getattr(h, '__name__', str(h)) for h in start_handlers]

 if any('on_BrowserStartEvent' in name for name in start_handler_names):
 raise RuntimeError(
 '[BrowserSession] Duplicate handler registration attempted! '
 'on_BrowserStartEvent is already registered. '
 'This likely means BrowserSession was initialized multiple times with the same EventBus.'
)

 BaseWatchdog.attach_handler_to_session(self, BrowserStartEvent, self.on_BrowserStartEvent)
 BaseWatchdog.attach_handler_to_session(self, BrowserStopEvent, self.on_BrowserStopEvent)
 BaseWatchdog.attach_handler_to_session(self, NavigateToUrlEvent, self.on_NavigateToUrlEvent)
 BaseWatchdog.attach_handler_to_session(self, SwitchTabEvent, self.on_SwitchTabEvent)
 BaseWatchdog.attach_handler_to_session(self, TabCreatedEvent, self.on_TabCreatedEvent)
 BaseWatchdog.attach_handler_to_session(self, TabClosedEvent, self.on_TabClosedEvent)
 BaseWatchdog.attach_handler_to_session(self, AgentFocusChangedEvent, self.on_AgentFocusChangedEvent)
 BaseWatchdog.attach_handler_to_session(self, FileDownloadedEvent, self.on_FileDownloadedEvent)
 BaseWatchdog.attach_handler_to_session(self, CloseTabEvent, self.on_CloseTabEvent)

 @observe_debug(ignore_input=True, ignore_output=True, name='browser_session_start')
 async def start(self) -> None:
 """Start the browser session."""
 start_event = self.event_bus.dispatch(BrowserStartEvent())
 await start_event
 ## Ensure any exceptions from the event handler are propagated
 await start_event.event_result(raise_if_any=True, raise_if_none=False)

 async def kill(self) -> None:
 """Kill the browser session and reset all state."""
 self._intentional_stop = True
 self.logger.debug('■ kill() called - stopping browser with force=True and resetting state')

 ## First save storage state while CDP is still connected
 from system.browser.events import SaveStorageStateEvent

 save_event = self.event_bus.dispatch(SaveStorageStateEvent())
 await save_event

 ## Dispatch stop event to kill the browser
 await self.event_bus.dispatch(BrowserStopEvent(force=True))
 ## Stop the event bus
 await self.event_bus.stop(clear=True, timeout=5)
 ## Reset all state
 await self.reset()

```

```

 ## Create fresh event bus
 self.event_bus = EventBus()

 async def stop(self) -> None:
 """Stop the browser session without killing the browser process.

 This clears event buses and cached state but keeps the browser alive.
 Useful when you want to clean up resources but plan to reconnect later.
 """
 self._intentional_stop = True
 self.logger.debug('stop() called - stopping browser gracefully (force=False) and resetting state')

 ## First save storage state while CDP is still connected
 from system.browser.events import SaveStorageStateEvent

 save_event = self.event_bus.dispatch(SaveStorageStateEvent())
 await save_event

 ## Now dispatch BrowserStopEvent to notify watchdogs
 await self.event_bus.dispatch(BrowserStopEvent(force=False))

 ## Stop the event bus
 await self.event_bus.stop(clear=True, timeout=5)
 ## Reset all state
 await self.reset()
 ## Create fresh event bus
 self.event_bus = EventBus()

 @observe_debug(ignore_input=True, ignore_output=True, name='browser_start_event_handler')
 async def on_BrowserStartEvent(self, event: BrowserStartEvent) -> dict[str, str]:
 """Handle browser start request.

 Returns:
 Dict with 'cdp_url' key containing the CDP URL

 Note: This method is idempotent - calling start() multiple times is safe.
 - If already connected, it skips reconnection
 - If you need to reset state, call stop() or kill() first
 """

 ## Initialize and attach all watchdogs FIRST so LocalBrowserWatchdog can handle BrowserLaunchEvent
 await self.attach_all_watchdogs()

 try:
 ## If no CDP URL, launch local browser or cloud browser
 if not self.cdp_url:
 ## If self.browser_profile.use_cloud or self.browser_profile.cloud_browser_params is not None:
 ## Use cloud browser service
 try:
 ## Use cloud_browser_params if provided, otherwise create empty request
 cloud_params = self.browser_profile.cloud_browser_params or CreateBrowserRequest()
 cloud_browser_response = await self._cloud_browser_client.create_browser(cloud_params)
 self.browser_profile.cdp_url = cloud_browser_response.cdpUrl
 self.browser_profile.is_local = False
 self.logger.info('Successfully connected to cloud browser service')
 except CloudBrowserAuthError:
 raise CloudBrowserAuthError(
 'Authentication failed for cloud browser service. Set BROWSER_USE_API_KEY environment variable.'
)
 except CloudBrowserError as e:
 raise CloudBrowserError(f'Failed to create cloud browser: {e}')
 elif self.is_local:
 ## Launch local browser using event-driven approach
 launch_event = self.event_bus.dispatch(BrowserLaunchEvent())
 await launch_event

 ## Get the CDP URL from LocalBrowserWatchdog handler result
 launch_result: BrowserLaunchResult = cast(
 BrowserLaunchResult, await launch_event.event_result(raise_if_none=True, raise_if_any=True)
)
 self.browser_profile.cdp_url = launch_result.cdp_url
 else:

```

```

 raise ValueError('Got BrowserSession(is_local=False) but no cdp_url was provided to connect to!')

 assert self.cdp_url and '://' in self.cdp_url

 # Use lock to prevent concurrent connection attempts (race condition protection)
 async with self._connection_lock:
 # Only connect if not already connected
 if self._cdp_client_root is None:
 # Setup browser via CDP (for both local and remote cases)
 # Global timeout prevents connect() from hanging indefinitely on
 # slow/broken WebSocket connections (common on Lambda → remote browser)
 try:
 await asyncio.wait_for(self.connect(cdp_url=self.cdp_url), timeout=15.0)
 except TimeoutError:
 # Timeout cancels connect() via CancelledError, which bypasses
 # connect()'s `except Exception` cleanup (CancelledError is BaseException).
 # Clean up the partially-initialized client so future start attempts
 # don't skip reconnection due to _cdp_client_root being non-None.
 cdp_client = cast(CDPClient | None, self._cdp_client_root)
 if cdp_client is not None:
 try:
 await cdp_client.stop()
 except Exception:
 pass
 self._cdp_client_root = None
 manager = self.session_manager
 if manager is not None:
 try:
 await manager.clear()
 except Exception:
 pass
 self.session_manager = None
 self.agent_focus_target_id = None
 raise RuntimeError(
 f'connect() timed out after 15s – CDP connection to {self.cdp_url} is too slow or unresponsive'
)
 assert self.cdp_client is not None

 # Notify that browser is connected (single place)
 # Ensure BrowserConnected handlers (storage_state restore) complete before
 # start() returns so cookies/storage are applied before navigation.
 await self.event_bus.dispatch(BrowserConnectedEvent(cdp_url=self.cdp_url))

 if self.browser_profile.demo_mode:
 try:
 demo = self.demo_mode
 if demo:
 await demo.ensure_ready()
 except Exception as exc:
 self.logger.warning(f'[DemoMode] Failed to inject demo overlay: {exc}')
 else:
 self.logger.debug('Already connected to CDP, skipping reconnection')
 if self.browser_profile.demo_mode:
 try:
 demo = self.demo_mode
 if demo:
 await demo.ensure_ready()
 except Exception as exc:
 self.logger.warning(f'[DemoMode] Failed to inject demo overlay: {exc}')

 # Return the CDP URL for other components
 return {'cdp_url': self.cdp_url}

 except Exception as e:
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='BrowserStartEventError',
 message=f'Failed to start browser: {type(e).__name__} {e}',
 details={'cdp_url': self.cdp_url, 'is_local': self.is_local},
)
)
 raise

```

```

 async def on_NavigateToUrlEvent(self, event: NavigateToUrlEvent) -> None:
 """Handle navigation requests - core browser functionality."""
 self.logger.debug(f'[on_NavigateToUrlEvent] Received NavigateToUrlEvent: url={event.url}, new_tab={event.new_tab}')
 if not self.agent_focus_target_id:
 self.logger.warning('Cannot navigate - browser not connected')
 return

 target_id = None
 current_target_id = self.agent_focus_target_id

 ## If new_tab=True but we're already in a new tab, set new_tab=False
 current_target = self.session_manager.get_target(current_target_id)
 if event.new_tab and is_new_tab_page(current_target.url):
 self.logger.debug(f'[on_NavigateToUrlEvent] Already on blank tab ({current_target.url}), reusing')
 event.new_tab = False

 try:
 ## Find or create target for navigation
 self.logger.debug(f'[on_NavigateToUrlEvent] Processing new_tab={event.new_tab}')

 if event.new_tab:
 page_targets = self.session_manager.get_all_page_targets()
 self.logger.debug(f'[on_NavigateToUrlEvent] Found {len(page_targets)} existing tabs')

 ## Look for existing about:blank tab that's not the current one
 for idx, target in enumerate(page_targets):
 self.logger.debug(f'[on_NavigateToUrlEvent] Tab {idx}: url={target.url}, targetId={target.target_id}')
 if target.url == 'about:blank' and target.target_id != current_target_id:
 target_id = target.target_id
 self.logger.debug(f'Reusing existing about:blank tab #{target_id[-4:]}')
 break

 ## Create new tab if no reusable one found
 if not target_id:
 self.logger.debug(f'[on_NavigateToUrlEvent] No reusable about:blank tab found, creating new tab...')
 try:
 target_id = await self._cdp_create_new_page('about:blank')
 self.logger.debug(f'Created new tab #{target_id[-4:]}')
 ## Dispatch TabCreatedEvent for new tab
 await self.event_bus.dispatch(TabCreatedEvent(target_id=target_id, url='about:blank'))
 except Exception as e:
 self.logger.error(f'[on_NavigateToUrlEvent] Failed to create new tab: {type(e).__name__}: {e}')
 ## Fall back to using current tab
 target_id = current_target_id
 self.logger.warning(f'[on_NavigateToUrlEvent] Falling back to current tab #{target_id[-4:]}')
 else:
 ## Use current tab
 target_id = target_id or current_target_id

 ## Switch to target tab if needed (for both new_tab=True and new_tab=False)
 if self.agent_focus_target_id is None or self.agent_focus_target_id != target_id:
 self.logger.debug(
 f'[on_NavigateToUrlEvent] Switching to target tab {target_id[-4:]} (current: {self.agent_focus_target_id[-4:]})'
)
 ## Activate target (bring to foreground)
 await self.event_bus.dispatch(SwitchTabEvent(target_id=target_id))
 else:
 self.logger.debug(f'[on_NavigateToUrlEvent] Already on target tab {target_id[-4:]}, skipping SwitchTabEvent')

 assert self.agent_focus_target_id is not None and self.agent_focus_target_id == target_id, (
 'Agent focus not updated to new target_id after SwitchTabEvent should have switched to it'
)

 ## Dispatch navigation started
 await self.event_bus.dispatch(NavigationStartedEvent(target_id=target_id, url=event.url))

 ## Navigate to URL with proper lifecycle waiting
 await self._navigate_and_wait(event.url, target_id, wait_until=event.wait_until)

 ## Close any extension options pages that might have opened
 await self._close_extension_options_pages()
 ## Dispatch navigation complete

```

```

self.logger.debug(f'Dispatching NavigationCompleteEvent for {event.url} (tab #{target_id[-4:]})')
await self.event_bus.dispatch(
 NavigationCompleteEvent(
 target_id=target_id,
 url=event.url,
 status=None, # CDP doesn't provide status directly
)
)
await self.event_bus.dispatch(AgentFocusChangedEvent(target_id=target_id, url=event.url))

Note: These should be handled by dedicated watchdogs:
- Security checks (security_watchdog)
- Page health checks (crash_watchdog)
- Dialog handling (dialog_watchdog)
- Download handling (downloads_watchdog)
- DOM rebuilding (dom_watchdog)

except Exception as e:
 self.logger.error(f'Navigation failed: {type(e).__name__}: {e}')
 # target_id might be unbound if exception happens early
 if 'target_id' in locals() and target_id:
 await self.event_bus.dispatch(
 NavigationCompleteEvent(
 target_id=target_id,
 url=event.url,
 error_message=f'{type(e).__name__}: {e}',
)
)
 await self.event_bus.dispatch(AgentFocusChangedEvent(target_id=target_id, url=event.url))
 raise

async def _navigate_and_wait(
 self,
 url: str,
 target_id: str,
 timeout: float | None = None,
 wait_until: str = 'load',
) -> None:
 """Navigate to URL and wait for page readiness using CDP lifecycle events.

 Polls stored lifecycle events (registered once per session in SessionManager).
 wait_until controls the minimum acceptable signal: 'commit', 'domcontentloaded', 'load', 'networkidle'
 """
 cdp_session = await self.get_or_create_cdp_session(target_id, focus=False)

 if timeout is None:
 target = self.session_manager.get_target(target_id)
 current_url = target.url
 same_domain = (
 url.split('/')[2] == current_url.split('/')[2]
)
 if url.startswith('http') and current_url.startswith('http'):
 else False
)
 timeout = 3.0 if same_domain else 8.0

 nav_start_time = asyncio.get_event_loop().time()

 # Wrap Page.navigate() with timeout – heavy sites can block here for 10s+
 nav_timeout = 20.0
 try:
 nav_result = await asyncio.wait_for(
 cdp_session.cdp_client.send.Page.navigate(
 params={'url': url, 'transitionType': 'address_bar'},
 session_id=cdp_session.session_id,
),
 timeout=nav_timeout,
)
 except TimeoutError:
 duration_ms = (asyncio.get_event_loop().time() - nav_start_time) * 1000
 raise RuntimeError(f'Page.navigate() timed out after {nav_timeout}s ({duration_ms:.0f}ms) for {url}')

 if nav_result.get('errorText'):

```



```

 raise RuntimeError(f'Navigation failed: {nav_result["errorText"]}')

 if wait_until == 'commit':
 duration_ms = (asyncio.get_event_loop().time() - nav_start_time) * 1000
 self.logger.debug(f'Page ready for {url} (commit, {duration_ms:.0f}ms)')
 return

 navigation_id = nav_result.get('loaderId')
 start_time = asyncio.get_event_loop().time()
 seen_events = []

 if not hasattr(cdp_session, '_lifecycle_events'):
 raise RuntimeError(
 f'Lifecycle monitoring not enabled for {cdp_session.target_id[:8]}! '
 f'This is a bug - SessionManager should have initialized it. '
 f'Session: {cdp_session}'
)

 # Acceptable events by readiness level (higher is always acceptable)
 acceptable_events: set[str] = {'networkIdle'}
 if wait_until in ('load', 'domcontentloaded'):
 acceptable_events.add('load')
 if wait_until == 'domcontentloaded':
 acceptable_events.add('DOMContentLoaded')

 poll_interval = 0.05
 while (asyncio.get_event_loop().time() - start_time) < timeout:
 try:
 for event_data in list(cdp_session._lifecycle_events):
 event_name = event_data.get('name')
 event_loader_id = event_data.get('loaderId')

 event_str = f'{event_name}(loader={event_loader_id[:8] if event_loader_id else "none"})'
 if event_str not in seen_events:
 seen_events.append(event_str)

 if event_loader_id and navigation_id and event_loader_id != navigation_id:
 continue

 if event_name in acceptable_events:
 duration_ms = (asyncio.get_event_loop().time() - nav_start_time) * 1000
 self.logger.debug(f'Page ready for {url} ({event_name}, {duration_ms:.0f}ms)')
 return

 except Exception as e:
 self.logger.debug(f'Error polling lifecycle events: {e}')

 await asyncio.sleep(poll_interval)

 duration_ms = (asyncio.get_event_loop().time() - nav_start_time) * 1000
 if not seen_events:
 self.logger.error(
 f'No lifecycle events received for {url} after {duration_ms:.0f}ms! '
 f'Monitoring may have failed. Target: {cdp_session.target_id[:8]}'
)
 else:
 self.logger.warning(f'Page readiness timeout ({timeout}s, {duration_ms:.0f}ms) for {url}')

 async def on_SwitchTabEvent(self, event: SwitchTabEvent) -> TargetID:
 """Handle tab switching - core browser functionality."""
 if not self.agent_focus_target_id:
 raise RuntimeError('Cannot switch tabs - browser not connected')

 # Get all page targets
 page_targets = self.session_manager.get_all_page_targets()
 if event.target_id is None:
 # Most recently opened page
 if page_targets:
 # Update the target id to be the id of the most recently opened page, then proceed to switch to it
 event.target_id = page_targets[-1].target_id
 else:
 # No pages open at all, create a new one (handles switching to it automatically)

```

```

#####assert self._cdp_client_root is not None, 'CDP client root not initialized - browser may not be cor
#####new_target = await self._cdp_client_root.send.Target.createTarget(params={'url': 'about:blank'})
#####target_id = new_target['targetId']
Don't await, these may circularly trigger SwitchTabEvent and could deadlock, dispatch to enqueue
#####self.event_bus.dispatch(TabCreatedEvent(url='about:blank', target_id=target_id))
#####self.event_bus.dispatch(AgentFocusChangedEvent(target_id=target_id, url='about:blank'))
#####return target_id

Switch to the target
#####assert event.target_id is not None, 'target_id must be set at this point'
Ensure session exists and update agent focus (only for page/tab targets)
#####cdp_session = await self.get_or_create_cdp_session(target_id=event.target_id, focus=True)

Visually switch to the tab in the browser
The Force Background Tab extension prevents Chrome from auto-switching when links create new tabs,
but we still want the agent to be able to explicitly switch tabs when needed
#####await cdp_session.cdp_client.send.Target.activateTarget(params={'targetId': event.target_id})

Get target to access url
#####target = self.session_manager.get_target(event.target_id)

dispatch focus changed event
#####await self.event_bus.dispatch(
AgentFocusChangedEvent(
target_id=target.target_id,
url=target.url,
)
#####)
#####return target.target_id

#####async def on_CloseTabEvent(self, event: CloseTabEvent) -> None:
"""Handle tab closure - update focus if needed."""
try:
Dispatch tab closed event
await self.event_bus.dispatch(TabClosedEvent(target_id=event.target_id))

Try to close the target, but don't fail if it's already closed
try:
cdp_session = await self.get_or_create_cdp_session(target_id=None, focus=False)
await cdp_session.cdp_client.send.Target.closeTarget(params={'targetId': event.target_id})
except Exception as e:
self.logger.debug(f'Target may already be closed: {e}')
except Exception as e:
self.logger.warning(f'Error during tab close cleanup: {e}')

#####async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
"""Handle tab creation - apply viewport settings to new tab."""
Note: Tab switching prevention is handled by the Force Background Tab extension
The extension automatically keeps focus on the current tab when new tabs are created

Apply viewport settings if configured
if self.browser_profile.viewport and not self.browser_profile.no_viewport:
try:
viewport_width = self.browser_profile.viewport.width
viewport_height = self.browser_profile.viewport.height
device_scale_factor = self.browser_profile.device_scale_factor or 1.0

self.logger.info(
f'Setting viewport to {viewport_width}x{viewport_height} with device scale factor {device_scale_fa
)
Use the helper method with the new tab's target_id
await self._cdp_set_viewport(viewport_width, viewport_height, device_scale_factor, target_id=event.

self.logger.debug(f'Applied viewport {viewport_width}x{viewport_height} to tab {event.target_id[-8:
except Exception as e:
self.logger.warning(f'Failed to set viewport for new tab {event.target_id[-8:]}: {e}')

#####async def on_TabClosedEvent(self, event: TabClosedEvent) -> None:
"""Handle tab closure - update focus if needed."""
if not self.agent_focus_target_id:
return
Get current tab index

```

```

 self.current_target_id = self.agent_focus_target_id

 ## If the closed tab was the current one, find a new target
 if current_target_id == event.target_id:
 await self.event_bus.dispatch(SwitchTabEvent(target_id=None))

 async def on_AgentFocusChangedEvent(self, event: AgentFocusChangedEvent) -> None:
 """Handle agent focus change - update focus and clear cache."""
 self.logger.debug(f'AgentFocusChangedEvent received: target_id={event.target_id[-4:]} url={event.url}')

 ## Clear cached DOM state since focus changed
 if self._dom_watchdog:
 self._dom_watchdog.clear_cache()

 ## Clear cached browser state
 self._cached_browser_state_summary = None
 self._cached_selector_map.clear()
 self.logger.debug('Cached browser state cleared')

 ## Update agent focus if a specific target_id is provided (only for page/tab targets)
 if event.target_id:
 ## Ensure session exists and update agent focus (validates target_type internally)
 await self.get_or_create_cdp_session(target_id=event.target_id, focus=True)

 ## Apply viewport settings to the newly focused tab
 if self.browser_profile.viewport and not self.browser_profile.no_viewport:
 try:
 viewport_width = self.browser_profile.viewport.width
 viewport_height = self.browser_profile.viewport.height
 device_scale_factor = self.browser_profile.device_scale_factor or 1.0

 ## Use the helper method with the current tab's target_id
 await self._cdp_set_viewport(viewport_width, viewport_height, device_scale_factor, target_id=event.target_id)

 self.logger.debug(f'Applied viewport {viewport_width}x{viewport_height} to tab {event.target_id[-8:]}')
 except Exception as e:
 self.logger.warning(f'Failed to set viewport for tab {event.target_id[-8:]}: {e}')
 else:
 raise RuntimeError('AgentFocusChangedEvent received with no target_id for newly focused tab')

 async def on_FileDownloadedEvent(self, event: FileDownloadedEvent) -> None:
 """Track downloaded files during this session."""
 self.logger.debug(f'FileDownloadedEvent received: {event.file_name} at {event.path}')
 if event.path and event.path not in self._downloaded_files:
 self._downloaded_files.append(event.path)
 self.logger.info(f'Tracked download: {event.file_name} ({len(self._downloaded_files)} total downloads)')
 else:
 if not event.path:
 self.logger.warning(f'FileDownloadedEvent has no path: {event}')
 else:
 self.logger.debug(f'File already tracked: {event.path}')

 async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 """Handle browser stop request."""

 try:
 ## Check if we should keep the browser alive
 if self.browser_profile.keep_alive and not event.force:
 self.event_bus.dispatch(BrowserStoppedEvent(reason='Kept alive due to keep_alive=True'))
 return

 ## Clean up cloud browser session if using cloud browser
 if self.browser_profile.use_cloud:
 try:
 await self._cloud_browser_client.stop_browser()
 self.logger.info('Cloud browser session cleaned up')
 except Exception as e:
 self.logger.debug(f'Failed to cleanup cloud browser session: {e}')

 ## Clear CDP session cache before stopping
 self.logger.info(f'on_BrowserStopEvent - Calling reset() (force={event.force}, keep_alive={self.browser_profile.keep_alive})')

```

```

)
 await self.reset()

 # Reset state
 if self.is_local:
 self.browser_profile.cdp_url = None

 # Notify stop and wait for all handlers to complete
 # LocalBrowserWatchdog listens for BrowserStopEvent and dispatches BrowserKillEvent
 stop_event = self.event_bus.dispatch(BrowserStoppedEvent(reason='Stopped by request'))
 await stop_event

 except Exception as e:
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='BrowserStopEventError',
 message=f'Failed to stop browser: {type(e).__name__} {e}',
 details={'cdp_url': self.cdp_url, 'is_local': self.is_local},
)
)

 # region - ===== CDP-based replacements for browser_context operations =====
 @property
 def cdp_client(self) -> CDPClient:
 """Get the cached root CDP cdp_session.cdp_client. The client is created and started in self.connect()
 assert self._cdp_client_root is not None, 'CDP client not initialized - browser may not be connected'
 return self._cdp_client_root

 async def new_page(self, url: str | None = None) -> 'Page':
 """Create a new page (tab)."""
 from cdp_use.cdp.target.commands import CreateTargetParameters

 params: CreateTargetParameters = {'url': url or 'about:blank'}
 result = await self.cdp_client.send.Target.createTarget(params)

 target_id = result['targetId']

 # Import here to avoid circular import
 from system.actor.page import Page as Target

 return Target(self, target_id)

 async def get_current_page(self) -> 'Page | None':
 """Get the current page as an actor Page."""
 target_info = await self.get_current_target_info()

 if not target_info:
 return None

 from system.actor.page import Page as Target

 return Target(self, target_info['targetId'])

 async def must_get_current_page(self) -> 'Page':
 """Get the current page as an actor Page."""
 page = await self.get_current_page()
 if not page:
 raise RuntimeError('No current target found')

 return page

 async def get_pages(self) -> list['Page']:
 """Get all available pages using SessionManager (source of truth)."""
 # Import here to avoid circular import
 from system.actor.page import Page as PageActor

 page_targets = self.session_manager.get_all_page_targets() if self.session_manager else []

 targets = []
 for target in page_targets:
 targets.append(PageActor(self, target.target_id))
 return targets

```

```

def get_focused_target(self) -> 'Target | None':
 """Get the target that currently has agent focus.

 Returns:
 Target object if agent has focus, None otherwise.
 """
 if not self.session_manager:
 return None
 return self.session_manager.get_focused_target()

def get_page_targets(self) -> list['Target']:
 """Get all page/tab targets (excludes iframes, workers, etc.).

 Returns:
 List of Target objects for all page/tab targets.
 """
 if not self.session_manager:
 return []
 return self.session_manager.get_all_page_targets()

async def close_page(self, page: 'Union[Page, str]') -> None:
 """Close a page by Page object or target ID."""
 from cdp_use.cdp.target.commands import CloseTargetParameters

 # Import here to avoid circular import
 from system.actor.page import Page as Target

 if isinstance(page, Target):
 target_id = page._target_id
 else:
 target_id = str(page)

 params: CloseTargetParameters = {'targetId': target_id}
 await self.cdp_client.send.Target.closeTarget(params)

async def cookies(self) -> list['Cookie']:
 """Get cookies, optionally filtered by URLs."""

 result = await self.cdp_client.send.Storage.getCookies()
 return result['cookies']

async def clear_cookies(self) -> None:
 """Clear all cookies."""
 await self.cdp_client.send.Network.clearBrowserCookies()

async def export_storage_state(self, output_path: str | Path | None = None) -> dict[str, Any]:
 """Export all browser cookies and storage to storage_state format.

 Extracts decrypted cookies via CDP, bypassing keychain encryption.

 Args:
 output_path: Optional path to save storage_state.json. If None, returns dict only.

 Returns:
 Storage state dict with cookies in Playwright format.
 """
 from pathlib import Path

 # Get all cookies using Storage.getCookies (returns decrypted cookies from all domains)
 cookies = await self._cdp_get_cookies()

 # Convert CDP cookie format to Playwright storage_state format
 storage_state = {
 'cookies': [
 {
 'name': c['name'],
 'value': c['value'],
 'domain': c['domain'],
 'path': c['path'],
 'expires': c.get('expires', -1),
 'httpOnly': c.get('httpOnly', False),
 }
]
 }

```

```

 'secure': c.get('secure', False),
 'sameSite': c.get('sameSite', 'Lax'),
 }
 for c in cookies:
],
 'origins': [], # Could add localStorage/sessionStorage extraction if needed
}

if output_path:
 import json

 output_file = Path(output_path).expanduser().resolve()
 output_file.parent.mkdir(parents=True, exist_ok=True)
 output_file.write_text(json.dumps(storage_state, indent=2))
 self.logger.info(f'Exported {len(cookies)} cookies to {output_file}')

return storage_state

async def get_or_create_cdp_session(self, target_id: TargetID | None = None, focus: bool = True) -> CDP
 """Get CDP session for a target from the event-driven pool.

 With autoAttach=True, sessions are created automatically by Chrome and added
 to the pool via Target.attachedToTarget events. This method retrieves them.

 Args:
 target_id: Target ID to get session for. If None, uses current agent focus.
 focus: If True, switches agent focus to this target (page targets only).

 Returns:
 CDPSession for the specified target.

 Raises:
 ValueError: If target doesn't exist or session is not available.
 """
 assert self._cdp_client_root is not None, 'Root CDP client not initialized'
 assert self.session_manager is not None, 'SessionManager not initialized'

 # If no target_id specified, ensure current agent focus is valid and wait for recovery if needed
 if target_id is None:
 # Validate and wait for focus recovery if stale (centralized protection)
 focus_valid = await self.session_manager.ensure_valid_focus(timeout=5.0)
 if not focus_valid:
 raise ValueError(
 'No valid agent focus available - target may have detached and recovery failed. '
 'This indicates browser is in an unstable state.'
)

 assert self.agent_focus_target_id is not None, 'Focus validation passed but agent_focus_target_id is None'
 target_id = self.agent_focus_target_id

 session = self.session_manager._get_session_for_target(target_id)

 if not session:
 # Session not in pool yet - wait for attach event
 self.logger.debug(f'[SessionManager] Waiting for target {target_id[:8]}... to attach...')

 # Wait up to 2 seconds for the attach event
 for attempt in range(20):
 await asyncio.sleep(0.1)
 session = self.session_manager._get_session_for_target(target_id)
 if session:
 self.logger.debug(f'[SessionManager] Target appeared after {attempt * 100}ms')
 break

 if not session:
 # Timeout - target doesn't exist
 raise ValueError(f'Target {target_id} not found - may have detached or never existed')

 # Validate session is still active
 is_valid = await self.session_manager.validate_session(target_id)
 if not is_valid:
 raise ValueError(f'Target {target_id} has detached - no active sessions')

```

```

 ## Update focus if requested
 ## CRITICAL: Only allow focus change to 'page' type targets, not iframes/workers
 ## if focus and self.agent_focus_target_id != target_id:
 ## ## Get target type from SessionManager
 ## target = self.session_manager.get_target(target_id)
 ## target_type = target.target_type if target else 'unknown'

 ## if target_type == 'page':
 ## ## Format current focus safely (could be None after detach)
 ## current_focus = self.agent_focus_target_id[:8] if self.agent_focus_target_id else 'None'
 ## self.logger.debug(f'[SessionManager] Switching focus: {current_focus}... → {target_id[:8]}...')
 ## self.agent_focus_target_id = target_id
 ## else:
 ## ## Ignore focus request for non-page targets (iframes, workers, etc.)
 ## ## These can detach at any time, causing agent_focus to point to dead target
 ## current_focus = self.agent_focus_target_id[:8] if self.agent_focus_target_id else 'None'
 ## self.logger.debug(
 ## f'[SessionManager] Ignoring focus request for {target_type} target {target_id[:8]}... '
 ## f'f(agent_focus stays on {current_focus}...)'
 ##)

 ## Resume if waiting for debugger (non-essential, don't let it block connect)
 ## if focus:
 ## try:
 ## await asyncio.wait_for(
 ## session.cdp_client.send.Runtime.runIfWaitingForDebugger(session_id=session.session_id),
 ## timeout=3.0,
 ##)
 ## except Exception:
 ## pass # May fail if not waiting, or timeout – either is fine

 ## return session

 async def set_extra_headers(self, headers: dict[str, str], target_id: TargetID | None = None) -> None:
 """Set extra HTTP headers using CDP Network.setExtraHTTPHeaders.

 These headers will be sent with every HTTP request made by the target.
 Network domain must be enabled first (done automatically for page targets
 in SessionManager._enable_page_monitoring).

 Args:
 headers: Dictionary of header name -> value pairs to inject into every request.
 target_id: Target to set headers on. Defaults to the current agent focus target.
 """
 ## if target_id is None:
 ## if not self.agent_focus_target_id:
 ## return
 ## target_id = self.agent_focus_target_id

 ## cdp_session = await self.get_or_create_cdp_session(target_id, focus=False)
 ## # Ensure Network domain is enabled (idempotent - safe to call multiple times)
 ## await cdp_session.cdp_client.send.Network.enable(session_id=cdp_session.session_id)
 ## await cdp_session.cdp_client.send.Network.setExtraHTTPHeaders(
 ## params={'headers': cast(Any, headers)}, session_id=cdp_session.session_id
 ##)

 ## # endregion - ===== CDP-based ... =====

 ## # region - ===== Helper Methods =====
 @observe_debug(ignore_input=True, ignore_output=True, name='get_browser_state_summary')
 async def get_browser_state_summary(
 self,
 include_screenshot: bool = True,
 cached: bool = False,
 include_recent_events: bool = False,
) -> BrowserStateSummary:
 ## if cached and self._cached_browser_state_summary is not None and self._cached_browser_state_summary.d
 ## ## Don't use cached state if it has 0 interactive elements
 ## selector_map = self._cached_browser_state_summary.dom_state.selector_map

 ## ## Don't use cached state if we need a screenshot but the cached state doesn't have one
 ## if include_screenshot and not self._cached_browser_state_summary.screenshot:

```

```

#####self.logger.debug('### Cached browser state has no screenshot, fetching fresh state with screenshot
Fall through to fetch fresh state with screenshot
#####elif selector_map and len(selector_map) > 0:
#####self.logger.debug('### Using pre-cached browser state summary for open tab')
#####return self._cached_browser_state_summary
#####else:
#####self.logger.debug('### Cached browser state has 0 interactive elements, fetching fresh state')
Fall through to fetch fresh state

Dispatch the event and wait for result
#####event: BrowserStateRequestEvent = cast(
#####BrowserStateRequestEvent,
#####self.event_bus.dispatch(
#####BrowserStateRequestEvent(
#####include_dom=True,
#####include_screenshot=include_screenshot,
#####include_recent_events=include_recent_events,
#####)
#####),
#####)

The handler returns the BrowserStateSummary directly
#####result = await event.event_result(raise_if_none=True, raise_if_any=True)
#####assert result is not None and result.dom_state is not None
#####return result

#####async def get_state_as_text(self) -> str:
#####"""Get the browser state as text."""
#####state = await self.get_browser_state_summary()
#####assert state.dom_state is not None
#####dom_state = state.dom_state
#####return dom_state.llm_representation()

#####async def attach_all_watchdogs(self) -> None:
#####"""Initialize and attach all watchdogs with explicit handler registration."""
Prevent duplicate watchdog attachment
#####if self._watchdogs_attached:
#####self.logger.debug('Watchdogs already attached, skipping duplicate attachment')
#####return

#####from system.browser.watchdogs.aboutblank_watchdog import AboutBlankWatchdog
#####from system.browser.watchdogs.captcha_watchdog import CaptchaWatchdog

from system.browser.crash_watchdog import CrashWatchdog
#####from system.browser.watchdogs.default_action_watchdog import DefaultActionWatchdog
#####from system.browser.watchdogs.dom_watchdog import DOMWatchdog
#####from system.browser.watchdogs.downloads_watchdog import DownloadsWatchdog
#####from system.browser.watchdogs.har_recording_watchdog import HarRecordingWatchdog
#####from system.browser.watchdogs.local_browser_watchdog import LocalBrowserWatchdog
#####from system.browser.watchdogs.permissions_watchdog import PermissionsWatchdog
#####from system.browser.watchdogs.popups_watchdog import PopupsWatchdog
#####from system.browser.watchdogs.recording_watchdog import RecordingWatchdog
#####from system.browser.watchdogs.screenshot_watchdog import ScreenshotWatchdog
#####from system.browser.watchdogs.security_watchdog import SecurityWatchdog
#####from system.browser.watchdogs.storage_state_watchdog import StorageStateWatchdog

Initialize CrashWatchdog
CrashWatchdog.model_rebuild()
self._crash_watchdog = CrashWatchdog(event_bus=self.event_bus, browser_session=self)
self.event_bus.on(BrowserConnectedEvent, self._crash_watchdog.on_BrowserConnectedEvent)
self.event_bus.on(BrowserStoppedEvent, self._crash_watchdog.on_BrowserStoppedEvent)
self._crash_watchdog.attach_to_session()

Initialize DownloadsWatchdog
DownloadsWatchdog.model_rebuild()
self._downloads_watchdog = DownloadsWatchdog(event_bus=self.event_bus, browser_session=self)
self.event_bus.on(BrowserLaunchEvent, self._downloads_watchdog.on_BrowserLaunchEvent)
self.event_bus.on(TabCreatedEvent, self._downloads_watchdog.on_TabCreatedEvent)
self.event_bus.on(TabClosedEvent, self._downloads_watchdog.on_TabClosedEvent)
self.event_bus.on(BrowserStoppedEvent, self._downloads_watchdog.on_BrowserStoppedEvent)
self.event_bus.on(NavigationCompleteEvent, self._downloads_watchdog.on_NavigationCompleteEvent)
self._downloads_watchdog.attach_to_session()

```



```

 if self.browser_profile.auto_download_pdfs:
 self.logger.debug('■ PDF auto-download enabled for this session')

 ## Initialize StorageStateWatchdog conditionally
 ## Enable when user provides either storage_state or user_data_dir (indicating they want persistence)
 should_enable_storage_state = (
 self.browser_profile.storage_state is not None or self.browser_profile.user_data_dir is not None
)

 if should_enable_storage_state:
 StorageStateWatchdog.model_rebuild()
 self._storage_state_watchdog = StorageStateWatchdog(
 event_bus=self.event_bus,
 browser_session=self,
 ## More conservative defaults when auto-enabled
 auto_save_interval=60.0, # 1 minute instead of 30 seconds
 save_on_change=False, # Only save on shutdown by default
)
 self._storage_state_watchdog.attach_to_session()
 self.logger.debug(
 f'■ StorageStateWatchdog enabled (storage_state: {bool(self.browser_profile.storage_state)}, user_data_dir: {bool(self.browser_profile.user_data_dir)})'
)
 else:
 self.logger.debug('■ StorageStateWatchdog disabled (no storage_state or user_data_dir configured)')

 ## Initialize LocalBrowserWatchdog
 LocalBrowserWatchdog.model_rebuild()
 self._local_browser_watchdog = LocalBrowserWatchdog(event_bus=self.event_bus, browser_session=self)
 self.event_bus.on(BrowserLaunchEvent, self._local_browser_watchdog.on_BrowserLaunchEvent)
 self.event_bus.on(BrowserKillEvent, self._local_browser_watchdog.on_BrowserKillEvent)
 self.event_bus.on(BrowserStopEvent, self._local_browser_watchdog.on_BrowserStopEvent)
 self._local_browser_watchdog.attach_to_session()

 ## Initialize SecurityWatchdog (hooks NavigationWatchdog and implements allowed_domains restriction)
 SecurityWatchdog.model_rebuild()
 self._security_watchdog = SecurityWatchdog(event_bus=self.event_bus, browser_session=self)
 ## Core navigation is now handled in BrowserSession directly
 ## SecurityWatchdog only handles security policy enforcement
 self._security_watchdog.attach_to_session()

 ## Initialize AboutBlankWatchdog (handles about:blank pages and DVD loading animation on first load)
 AboutBlankWatchdog.model_rebuild()
 self._aboutblank_watchdog = AboutBlankWatchdog(event_bus=self.event_bus, browser_session=self)
 self.event_bus.on(BrowserStopEvent, self._aboutblank_watchdog.on_BrowserStopEvent)
 self.event_bus.on(BrowserStoppedEvent, self._aboutblank_watchdog.on_BrowserStoppedEvent)
 self.event_bus.on(TabCreatedEvent, self._aboutblank_watchdog.on_TabCreatedEvent)
 self.event_bus.on(TabClosedEvent, self._aboutblank_watchdog.on_TabClosedEvent)
 self._aboutblank_watchdog.attach_to_session()

 ## Initialize PopupsWatchdog (handles accepting and dismissing JS dialogs, alerts, confirm, onbeforeunload)
 PopupsWatchdog.model_rebuild()
 self._popups_watchdog = PopupsWatchdog(event_bus=self.event_bus, browser_session=self)
 self.event_bus.on(TabCreatedEvent, self._popups_watchdog.on_TabCreatedEvent)
 self.event_bus.on(DialogCloseEvent, self._popups_watchdog.on_DialogCloseEvent)
 self._popups_watchdog.attach_to_session()

 ## Initialize PermissionsWatchdog (handles granting and revoking browser permissions like clipboard, microphone, etc.)
 PermissionsWatchdog.model_rebuild()
 self._permissions_watchdog = PermissionsWatchdog(event_bus=self.event_bus, browser_session=self)
 self.event_bus.on(BrowserConnectedEvent, self._permissions_watchdog.on_BrowserConnectedEvent)
 self._permissions_watchdog.attach_to_session()

 ## Initialize DefaultActionWatchdog (handles all default actions like click, type, scroll, go back, go forward, etc.)
 DefaultActionWatchdog.model_rebuild()
 self._default_action_watchdog = DefaultActionWatchdog(event_bus=self.event_bus, browser_session=self)
 self.event_bus.on(ClickElementEvent, self._default_action_watchdog.on_ClickElementEvent)
 self.event_bus.on(TypeTextEvent, self._default_action_watchdog.on_TypeTextEvent)
 self.event_bus.on(ScrollEvent, self._default_action_watchdog.on_ScrollEvent)
 self.event_bus.on(GoBackEvent, self._default_action_watchdog.on_GoBackEvent)
 self.event_bus.on(GoForwardEvent, self._default_action_watchdog.on_GoForwardEvent)
 self.event_bus.on(RefreshEvent, self._default_action_watchdog.on_RefreshEvent)
 self.event_bus.on(WaitEvent, self._default_action_watchdog.on_WaitEvent)

```

```

 ## self.event_bus.on(SendKeysEvent, self._default_action_watchdog.on_SendKeysEvent)
 ## self.event_bus.on(UploadFileEvent, self._default_action_watchdog.on_UploadFileEvent)
 ## self.event_bus.on(ScrollToTextEvent, self._default_action_watchdog.on_ScrollToTextEvent)
 ##self._default_action_watchdog.attach_to_session()

 ## Initialize ScreenshotWatchdog (handles taking screenshots of the browser)
 ##ScreenshotWatchdog.model_rebuild()
 ##self._screenshot_watchdog = ScreenshotWatchdog(event_bus=self.event_bus, browser_session=self)
 ## self.event_bus.on(BrowserStartEvent, self._screenshot_watchdog.on_BrowserStartEvent)
 ## self.event_bus.on(BrowserStoppedEvent, self._screenshot_watchdog.on_BrowserStoppedEvent)
 ## self.event_bus.on(ScreenshotEvent, self._screenshot_watchdog.on_ScreenshotEvent)
 ##self._screenshot_watchdog.attach_to_session()

 ## Initialize DOMWatchdog (handles building the DOM tree and detecting interactive elements, depends on
 ##DOMWatchdog.model_rebuild()
 ##self._dom_watchdog = DOMWatchdog(event_bus=self.event_bus, browser_session=self)
 ## self.event_bus.on(TabCreatedEvent, self._dom_watchdog.on_TabCreatedEvent)
 ## self.event_bus.on(BrowserStateRequestEvent, self._dom_watchdog.on_BrowserStateRequestEvent)
 ##self._dom_watchdog.attach_to_session()

 ## Initialize RecordingWatchdog (handles video recording)
 ##RecordingWatchdog.model_rebuild()
 ##self._recording_watchdog = RecordingWatchdog(event_bus=self.event_bus, browser_session=self)
 ##self._recording_watchdog.attach_to_session()

 ## Initialize HarRecordingWatchdog if record_har_path is configured (handles HTTPS HAR capture)
 ##if self.browser_profile.record_har_path:
 ## ##HarRecordingWatchdog.model_rebuild()
 ## ##self._har_recording_watchdog = HarRecordingWatchdog(event_bus=self.event_bus, browser_session=self)
 ## ##self._har_recording_watchdog.attach_to_session()

 ## Initialize CaptchaWatchdog (listens for captcha solver events from the browser proxy)
 ##if self.browser_profile.captcha_solver:
 ## ##CaptchaWatchdog.model_rebuild()
 ## ##self._captcha_watchdog = CaptchaWatchdog(event_bus=self.event_bus, browser_session=self)
 ## ##self._captcha_watchdog.attach_to_session()

 ## Mark watchdogs as attached to prevent duplicate attachment
 ##self._watchdogs_attached = True

 ##async def connect(self, cdp_url: str | None = None) -> Self:
 ## """Connect to a remote chromium-based browser via CDP using cdp-use.

 ## This MUST succeed or the browser is unusable. Fails hard on any error.
 ## """

 ##self.browser_profile.cdp_url = cdp_url or self.cdp_url
 ##if not self.cdp_url:
 ## ##raise RuntimeError('Cannot setup CDP connection without CDP URL')

 ## Prevent duplicate connections - clean up existing connection first
 ##if self._cdp_client_root is not None:
 ## ##self.logger.warning(
 ## ##'connect() called but CDP client already exists! Cleaning up old connection before creating new
 ## ##)
 ## ##try:
 ## ##await self._cdp_client_root.stop()
 ## ##except Exception as e:
 ## ##self.logger.debug(f'Error stopping old CDP client: {e}')
 ## ##self._cdp_client_root = None

 ##if not self.cdp_url.startswith('ws'):
 ## ## If it's an HTTP URL, fetch the WebSocket URL from /json/version endpoint
 ## ##parsed_url = urlparse(self.cdp_url)
 ## ##path = parsed_url.path.rstrip('/')

 ## ##if not path.endswith('/json/version'):
 ## ##path = path + '/json/version'

 ## ##url = urlunparse(
 ## ##(parsed_url.scheme, parsed_url.netloc, path, parsed_url.params, parsed_url.query, parsed_url.fragment)
 ## ##)

```

```

Run a tiny HTTP client to query for the WebSocket URL from the /json/version endpoint
Default httpx timeout is 5s which can race the global wait_for(connect(), 15s).
Use 30s as a safety net for direct connect() callers; the wait_for is the real deadline.
async with httpx.AsyncClient(timeout=httpx.Timeout(30.0)) as client:
headers = self.browser_profile.headers or {}
version_info = await client.get(url, headers=headers)
self.logger.debug(f'Raw version info: {str(version_info)}')
self.browser_profile.cdp_url = version_info.json()['websocketDebuggerUrl']

assert self.cdp_url is not None, 'CDP URL is None.'

browser_location = 'local browser' if self.is_local else 'remote browser'
self.logger.debug(f'##### Connecting to existing chromium-based browser via CDP: {self.cdp_url} -> ({browser_location})')

try:
Create and store the CDP client for direct CDP communication
headers = getattr(self.browser_profile, 'headers', None)
self._cdp_client_root = CDPCClient(
self.cdp_url,
additional_headers=headers,
max_ws_frame_size=200 * 1024 * 1024, # Use 200MB limit to handle pages with very large DOMs
)
assert self._cdp_client_root is not None
await self._cdp_client_root.start()

Initialize event-driven session manager FIRST (before enabling autoAttach)
SessionManager will:
1. Register attach/detach event handlers
2. Discover and attach to all existing targets
3. Initialize sessions and enable lifecycle monitoring
4. Enable autoAttach for future targets
from system.browser.session_manager import SessionManager

self.session_manager = SessionManager(self)
await self.session_manager.start_monitoring()
self.logger.debug('Event-driven session manager started')

Enable auto-attach so Chrome automatically notifies us when NEW targets attach/detach
This is the foundation of event-driven session management
await self._cdp_client_root.send.Target.setAutoAttach(
params={'autoAttach': True, 'waitForDebuggerOnStart': False, 'flatten': True}
)
self.logger.debug('CDP client connected with auto-attach enabled')

Get browser targets from SessionManager (source of truth)
SessionManager has already discovered all targets via start_monitoring()
page_targets_from_manager = self.session_manager.get_all_page_targets()

Check for chrome://newtab pages and redirect them to about:blank (in parallel)
from system.utils import is_new_tab_page

async def _redirect_newtab(target):
target_url = target.url
target_id = target.target_id
self.logger.debug(f'##### Redirecting {target_url} to about:blank for target {target_id}')
try:
session = await self.get_or_create_cdp_session(target_id, focus=False)
await session.cdp_client.send.Page.navigate(params={'url': 'about:blank'}, session_id=session.session_id)
target.url = 'about:blank'
except Exception as e:
self.logger.warning(f'Failed to redirect {target_url}: {e}')

redirect_tasks = [
_redirect_newtab(target)
for target in page_targets_from_manager
if is_new_tab_page(target.url) and target.url != 'about:blank'
]
if redirect_tasks:
await asyncio.gather(*redirect_tasks, return_exceptions=True)

Ensure we have at least one page
if not page_targets_from_manager:

```

```

new_target = await self._cdp_client_root.send.Target.createTarget(params={'url': 'about:blank'})
target_id = new_target['targetId']
self.logger.debug(f'Created new blank page: {target_id}')
else:
 target_id = page_targets_from_manager[0].target_id
self.logger.debug(f'Using existing page: {target_id}')

Set up initial focus using the public API
Note: get_or_create_cdp_session() will wait for attach event and set focus
try:
 await self.get_or_create_cdp_session(target_id, focus=True)
 # agent_focus_target_id is now set by get_or_create_cdp_session
 self.logger.debug(f'Agent focus set to {target_id[:8]}...')
except ValueError as e:
 raise RuntimeError(f'Failed to get session for initial target {target_id}: {e}') from e

Note: Lifecycle monitoring is enabled automatically in SessionManager._handle_target_attached()
when targets attach, so no manual enablement needed!

Enable proxy authentication handling if configured
await self._setup_proxy_auth()

Attach WS drop detection callback for auto-reconnection
self._intentional_stop = False
self._attach_ws_drop_callback()

Verify the target is working
if self.agent_focus_target_id:
 target = self.session_manager.get_target(self.agent_focus_target_id)
 if target.title == 'Unknown title':
 self.logger.warning('Target created but title is unknown (may be normal for about:blank)')

Dispatch TabCreatedEvent for all initial tabs (so watchdogs can initialize)
for idx, target in enumerate(page_targets_from_manager):
 target_url = target.url
 self.logger.debug(f'Dispatching TabCreatedEvent for initial tab {idx}: {target_url}')
 self.event_bus.dispatch(TabCreatedEvent(url=target_url, target_id=target.target_id))

Dispatch initial focus event
if page_targets_from_manager:
 initial_url = page_targets_from_manager[0].url
 self.event_bus.dispatch(AgentFocusChangedEvent(target_id=page_targets_from_manager[0].target_id, url=initial_url))
 self.logger.debug(f'Initial agent focus set to tab 0: {initial_url}')

except Exception as e:
 # Fatal error - browser is not usable without CDP connection
 self.logger.error(f'FATAL: Failed to setup CDP connection: {e}')
 self.logger.error('Browser cannot continue without CDP connection')

Clear SessionManager state
if self.session_manager:
 try:
 await self.session_manager.clear()
 self.logger.debug('Cleared SessionManager state after initialization failure')
 except Exception as cleanup_error:
 self.logger.debug(f'Error clearing SessionManager: {cleanup_error}')

Close CDP client WebSocket and unregister handlers
if self._cdp_client_root:
 try:
 await self._cdp_client_root.stop() # Close WebSocket and unregister handlers
 self.logger.debug('Closed CDP client WebSocket after initialization failure')
 except Exception as cleanup_error:
 self.logger.debug(f'Error closing CDP client: {cleanup_error}')

self.session_manager = None
self._cdp_client_root = None
self.agent_focus_target_id = None
Re-raise as a fatal error
raise RuntimeError(f'Failed to establish CDP connection to browser: {e}') from e

return self

```

```

 async def _setup_proxy_auth(self) -> None:
 """Enable CDP Fetch auth handling for authenticated proxy, if credentials provided.

 Handles HTTP proxy authentication challenges (Basic/Proxy) by providing
 configured credentials from BrowserProfile.
 """

 assert self._cdp_client_root

 try:
 proxy_cfg = self.browser_profile.proxy
 username = proxy_cfg.username if proxy_cfg else None
 password = proxy_cfg.password if proxy_cfg else None
 if not username or not password:
 self.logger.debug('Proxy credentials not provided; skipping proxy auth setup')
 return

 # Enable Fetch domain with auth handling (do not pause all requests)
 try:
 await self._cdp_client_root.send.Fetch.enable(params={'handleAuthRequests': True})
 self.logger.debug('Fetch.enable(handleAuthRequests=True) enabled on root client')
 except Exception as e:
 self.logger.debug(f'Fetch.enable on root failed: {type(e).__name__}: {e}')

 # Also enable on the focused target's session if available to ensure events are delivered
 try:
 if self.agent_focus_target_id:
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 await cdp_session.cdp_client.send.Fetch.enable(
 params={'handleAuthRequests': True},
 session_id=cdp_session.session_id,
)
 self.logger.debug('Fetch.enable(handleAuthRequests=True) enabled on focused session')
 except Exception as e:
 self.logger.debug(f'Fetch.enable on focused session failed: {type(e).__name__}: {e}')

 def _on_auth_required(event: AuthRequiredEvent, session_id: SessionID | None = None):
 # event keys may be snake_case or camelCase depending on generator; handle both
 request_id = event.get('requestId') or event.get('request_id')
 if not request_id:
 return

 challenge = event.get('authChallenge') or event.get('auth_challenge') or {}
 source = (challenge.get('source') or '').lower()
 # Only respond to proxy challenges
 if source == 'proxy' and request_id:

 async def _respond():
 assert self._cdp_client_root
 try:
 await self._cdp_client_root.send.Fetch.continueWithAuth(
 params={
 'requestId': request_id,
 'authChallengeResponse': {
 'response': 'ProvideCredentials',
 'username': username,
 'password': password,
 },
 },
 session_id=session_id,
)
 except Exception as e:
 self.logger.debug(f'Proxy auth respond failed: {type(e).__name__}: {e}')

 # schedule
 create_task_with_error_handling(
 _respond(), name='auth_respond', logger_instance=self.logger, suppress_exceptions=True
)
 else:
 # Default behaviour for non-proxy challenges: let browser handle
 async def _default():
 assert self._cdp_client_root

```

```

 try:
 await self._cdp_client_root.send.Fetch.continueWithAuth(
 params={'requestId': request_id, 'authChallengeResponse': {'response': 'Default'}},
 session_id=session_id,
)
 except Exception as e:
 self.logger.debug(f'Default auth respond failed: {type(e).__name__}: {e}')

 if request_id:
 create_task_with_error_handling(
 _default(), name='auth_default', logger_instance=self.logger, suppress_exceptions=True
)

 def _on_request_paused(event: RequestPausedEvent, session_id: SessionID | None = None):
 # Continue all paused requests to avoid stalling the network
 request_id = event.get('requestId') or event.get('request_id')
 if not request_id:
 return

 async def _continue():
 assert self._cdp_client_root
 try:
 await self._cdp_client_root.send.Fetch.continueRequest(
 params={'requestId': request_id},
 session_id=session_id,
)
 except Exception:
 pass

 create_task_with_error_handling(
 _continue(), name='request_continue', logger_instance=self.logger, suppress_exceptions=True
)

 # Register event handler on root client
 try:
 self._cdp_client_root.register.Fetch.authRequired(_on_auth_required)
 self._cdp_client_root.register.Fetch.requestPaused(_on_request_paused)
 if self.agent_focus_target_id:
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 cdp_session.cdp_client.register.Fetch.authRequired(_on_auth_required)
 cdp_session.cdp_client.register.Fetch.requestPaused(_on_request_paused)
 self.logger.debug('Registered Fetch.authRequired handlers')
 except Exception as e:
 self.logger.debug(f'Failed to register authRequired handlers: {type(e).__name__}: {e}')

 # Ensure Fetch is enabled for the current focused target's session, too
 try:
 if self.agent_focus_target_id:
 # Use safe API with focus=False to avoid changing focus
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 await cdp_session.cdp_client.send.Fetch.enable(
 params={'handleAuthRequests': True, 'patterns': [{'urlPattern': '*'}]},
 session_id=cdp_session.session_id,
)
 except Exception as e:
 self.logger.debug(f'Fetch.enable on focused session failed: {type(e).__name__}: {e}')
 except Exception as e:
 self.logger.debug(f'Skipping proxy auth setup: {type(e).__name__}: {e}')

 async def reconnect(self) -> None:
 """Re-establish the CDP WebSocket connection to an already-running browser.

 This is a lightweight reconnection that:
 1. Stops the old CDPClient (WS already dead, just clean state)
 2. Clears SessionManager (all CDP sessions are invalid post-disconnect)
 3. Creates a new CDPClient with the same cdp_url
 4. Re-initializes SessionManager and re-enables autoAttach
 5. Re-discovers page targets and restores agent focus
 6. Re-enables proxy auth if configured
 """
 assert self.cdp_url, 'Cannot reconnect without a CDP URL'
 old_focus_target_id = self.agent_focus_target_id

```

```

 ## 1. Stop old CDPClient (WS is already dead, this just cleans internal state)
 if self._cdp_client_root:
 try:
 await self._cdp_client_root.stop()
 except Exception as e:
 self.logger.debug(f'Error stopping old CDP client during reconnect: {e}')
 self._cdp_client_root = None

 ## 2. Clear SessionManager (all sessions are stale)
 if self.session_manager:
 try:
 await self.session_manager.clear()
 except Exception as e:
 self.logger.debug(f'Error clearing SessionManager during reconnect: {e}')
 self.session_manager = None

 self.agent_focus_target_id = None

 ## 3. Create new CDPClient with the same cdp_url
 headers = getattr(self.browser_profile, 'headers', None)
 self._cdp_client_root = CDPClient(
 self.cdp_url,
 additional_headers=headers,
 max_ws_frame_size=200 * 1024 * 1024,
)
 await self._cdp_client_root.start()

 ## 4. Re-initialize SessionManager
 from system.browser.session_manager import SessionManager

 self.session_manager = SessionManager(self)
 await self.session_manager.start_monitoring()

 ## 5. Re-enable autoAttach
 await self._cdp_client_root.send.Target.setAutoAttach(
 params={'autoAttach': True, 'waitForDebuggerOnStart': False, 'flatten': True}
)

 ## 6. Re-discover page targets and restore focus
 page_targets = self.session_manager.get_all_page_targets()

 ## Prefer the old focus target if it still exists
 restored = False
 if old_focus_target_id:
 for target in page_targets:
 if target.target_id == old_focus_target_id:
 await self.get_or_create_cdp_session(old_focus_target_id, focus=True)
 restored = True
 self.logger.debug(f'Restored agent focus to previous target {old_focus_target_id[:8]}...')
 break

 if not restored:
 if page_targets:
 fallback_id = page_targets[0].target_id
 await self.get_or_create_cdp_session(fallback_id, focus=True)
 self.logger.debug(f'Agent focus set to fallback target {fallback_id[:8]}...')
 else:
 ## No pages exist – create one
 new_target = await self._cdp_client_root.send.Target.createTarget(params={'url': 'about:blank'})
 target_id = new_target['targetId']
 await self.get_or_create_cdp_session(target_id, focus=True)
 self.logger.debug(f'Created new blank page during reconnect: {target_id[:8]}...')

 ## 7. Re-enable proxy auth if configured
 await self._setup_proxy_auth()

 ## 8. Attach the WS drop detection callback to the new client
 self._attach_ws_drop_callback()

 async def _auto_reconnect(self, max_attempts: int = 3) -> None:
 """Attempt to reconnect with exponential backoff.
 Dispatches BrowserReconnectingEvent before each attempt and

```

```

 BrowserReconnectedEvent on success.
 """
 async with self._reconnect_lock:
 if self._reconnecting:
 return # already in progress from another caller
 self._reconnecting = True
 self._reconnect_event.clear()

 start_time = time.time()
 delays = [1.0, 2.0, 4.0]

 try:
 for attempt in range(1, max_attempts + 1):
 self.event_bus.dispatch(
 BrowserReconnectingEvent(
 cdp_url=self.cdp_url or '',
 attempt=attempt,
 max_attempts=max_attempts,
)
)
 self.logger.warning(f'WebSocket reconnection attempt {attempt}/{max_attempts}...')

 try:
 await asyncio.wait_for(self.reconnect(), timeout=15.0)
 # Success
 downtime = time.time() - start_time
 self.event_bus.dispatch(
 BrowserReconnectedEvent(
 cdp_url=self.cdp_url or '',
 attempt=attempt,
 downtime_seconds=downtime,
)
)
 self.logger.info(f'WebSocket reconnected after {downtime:.1f}s (attempt {attempt})')
 return
 except Exception as e:
 self.logger.warning(f'Reconnection attempt {attempt} failed: {type(e).__name__}: {e}')
 if attempt < max_attempts:
 delay = delays[attempt - 1] if attempt - 1 < len(delays) else delays[-1]
 await asyncio.sleep(delay)

 # All attempts exhausted
 self.logger.error(f'All {max_attempts} reconnection attempts failed')
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='ReconnectionFailed',
 message=f'Failed to reconnect after {max_attempts} attempts ({time.time() - start_time:.1f}s)',
 details={'cdp_url': self.cdp_url or '', 'max_attempts': max_attempts},
)
)
 finally:
 self._reconnecting = False
 self._reconnect_event.set() # wake up all waiters regardless of outcome

 def _attach_ws_drop_callback(self) -> None:
 """Attach a done callback to the CDPClient's message handler task to detect WS drops."""
 if not self._cdp_client_root or not hasattr(self._cdp_client_root, '_message_handler_task'):
 return

 task = self._cdp_client_root._message_handler_task
 if task is None or task.done():
 return

 def _on_message_handler_done(fut: asyncio.Future) -> None:
 # Guard: skip if intentionally stopped, already reconnecting, or no cdp_url
 if self._intentional_stop or self._reconnecting or not self.cdp_url:
 return

 # The message handler task exiting means the WS connection dropped
 exc = fut.exception() if not fut.cancelled() else None
 self.logger.warning(
 f'CDP WebSocket message handler exited unexpectedly'
)

```



```

#####f'{f": {type(exc).__name__}: {exc}" if exc else " (connection closed)"}'
#####)

Fire auto-reconnect as an asyncio task
#####try:
#####loop = asyncio.get_running_loop()
#####self._reconnect_task = loop.create_task(self._auto_reconnect())
#####except RuntimeError:
No running event loop - can't reconnect
#####self.logger.error('■ No event loop available for auto-reconnect')

#####task.add_done_callback(_on_message_handler_done)

#####async def get_tabs(self) -> list[TabInfo]:
#####"""Get information about all open tabs using cached target data."""
#####tabs = []

Safety check - return empty list if browser not connected yet
#####if not self.session_manager:
#####return tabs

Get all page targets from SessionManager
#####page_targets = self.session_manager.get_all_page_targets()

#####for i, target in enumerate(page_targets):
#####target_id = target.target_id
#####url = target.url
#####title = target.title

#####try:
Skip JS execution for chrome:// pages and new tab pages
#####if is_new_tab_page(url) or url.startswith('chrome://'):
Use URL as title for chrome pages, or mark new tabs as unusable
#####if is_new_tab_page(url):
#####title = ''
#####elif not title:
For chrome:// pages without a title, use the URL itself
#####title = url

Special handling for PDF pages without titles
#####if (not title or title == '') and (url.endswith('.pdf') or 'pdf' in url):
PDF pages might not have a title, use URL filename
#####try:
#####from urllib.parse import urlparse

#####filename = urlparse(url).path.split('/')[-1]
#####if filename:
#####title = filename
#####except Exception:
#####pass

#####except Exception as e:
Fallback to basic title handling
#####self.logger.debug(f'■ Failed to get target info for tab #{i}: {_log_pretty_url(url)} - {type(e).__name__}')

#####if is_new_tab_page(url):
#####title = ''
#####elif url.startswith('chrome://'):
#####title = url
#####else:
#####title = ''

#####tab_info = TabInfo(
#####target_id=target_id,
#####url=url,
#####title=title,
#####parent_target_id=None,
#####)
#####tabs.append(tab_info)

#####return tabs
endregion - ===== Helper Methods =====

```

```

region - ===== ID Lookup Methods =====
async def get_current_target_info(self) -> TargetInfo | None:
 """Get info about the current active target using cached session data."""
 if not self.agent_focus_target_id:
 return None

 target = self.session_manager.get_target(self.agent_focus_target_id)

 return {
 'targetId': target.target_id,
 'url': target.url,
 'title': target.title,
 'type': target.target_type,
 'attached': True,
 'canAccessOpener': False,
 }

async def get_current_page_url(self) -> str:
 """Get the URL of the current page."""
 if self.agent_focus_target_id:
 target = self.session_manager.get_target(self.agent_focus_target_id)
 return target.url
 return 'about:blank'

async def get_current_page_title(self) -> str:
 """Get the title of the current page."""
 if self.agent_focus_target_id:
 target = self.session_manager.get_target(self.agent_focus_target_id)
 return target.title
 return 'Unknown page title'

async def navigate_to(self, url: str, new_tab: bool = False) -> None:
 """Navigate to a URL using the standard event system.

 Args:
 url: URL to navigate to
 new_tab: Whether to open in a new tab
 """
 from system.browser.events import NavigateToUrlEvent

 event = self.event_bus.dispatch(NavigateToUrlEvent(url=url, new_tab=new_tab))
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)

endregion - ===== ID Lookup Methods =====

region - ===== DOM Helper Methods =====

async def get_dom_element_by_index(self, index: int) -> EnhancedDOMTreeNode | None:
 """Get DOM element by index.

 Get element from cached selector map.

 Args:
 index: The element index from the serialized DOM

 Returns:
 EnhancedDOMTreeNode or None if index not found
 """
 # Check cached selector map
 if self._cached_selector_map and index in self._cached_selector_map:
 return self._cached_selector_map[index]

 return None

def update_cached_selector_map(self, selector_map: dict[int, EnhancedDOMTreeNode]) -> None:
 """Update the cached selector map with new DOM state.

 This should be called by the DOM watchdog after rebuilding the DOM.

 Args:
 selector_map: The new selector map from DOM serialization

```

```

 """
 self._cached_selector_map = selector_map

 # Alias for backwards compatibility
 async def get_element_by_index(self, index: int) -> EnhancedDOMTreeNode | None:
 """Alias for get_dom_element_by_index for backwards compatibility."""
 return await self.get_dom_element_by_index(index)

 async def get_dom_element_at_coordinates(self, x: int, y: int) -> EnhancedDOMTreeNode | None:
 """Get DOM element at coordinates as EnhancedDOMTreeNode.

 First checks the cached selector_map for a matching element, then falls back
 to CDP DOM.describeNode if not found. This ensures safety checks (e.g., for
 <select> elements and file inputs) work correctly.

 Args:
 x: X coordinate relative to viewport
 y: Y coordinate relative to viewport

 Returns:
 EnhancedDOMTreeNode at the coordinates, or None if no element found
 """
 from system.dom.views import NodeType

 # Get current page to access CDP session
 page = await self.get_current_page()
 if page is None:
 raise RuntimeError('No active page found')

 # Get session ID for CDP call
 session_id = await page._ensure_session()

 try:
 # Call CDP DOM.getNodeForLocation to get backend_node_id
 result = await self.cdp_client.send.DOM.getNodeForLocation(
 params={
 'x': x,
 'y': y,
 'includeUserAgentShadowDOM': False,
 'ignorePointerEventsNone': False,
 },
 session_id=session_id,
)

 backend_node_id = result.get('backendNodeId')
 if backend_node_id is None:
 self.logger.debug(f'No element found at coordinates ({x}, {y})')
 return None

 # Try to find element in cached selector_map (avoids extra CDP call)
 if self._cached_selector_map:
 for node in self._cached_selector_map.values():
 if node.backend_node_id == backend_node_id:
 self.logger.debug(f'Found element at ({x}, {y}) in cached selector_map')
 return node

 # Not in cache - fall back to CDP DOM.describeNode to get actual node info
 try:
 describe_result = await self.cdp_client.send.DOM.describeNode(
 params={'backendNodeId': backend_node_id},
 session_id=session_id,
)
 node_info = describe_result.get('node', {})
 node_name = node_info.get('nodeName', '')

 # Parse attributes from flat list [key1, val1, key2, val2, ...] to dict
 attrs_list = node_info.get('attributes', [])
 attributes = {attrs_list[i]: attrs_list[i + 1] for i in range(0, len(attrs_list), 2)}

 return EnhancedDOMTreeNode(
 node_id=result.get('nodeId', 0),
 backend_node_id=backend_node_id,

```

```

#####node_type=NodeType(node_info.get('nodeType', NodeType.ELEMENT_NODE.value)),
#####node_name=node_name,
#####node_value=node_info.get('nodeValue', '') or '',
#####attributes=attributes,
#####is_scrollable=None,
#####frame_id=result.get('frameId'),
#####session_id=session_id,
#####target_id=self.agent_focus_target_id or '',
#####content_document=None,
#####shadow_root_type=None,
#####shadow_roots=None,
#####parent_node=None,
#####children_nodes=None,
#####ax_node=None,
#####snapshot_node=None,
#####is_visible=None,
#####absolute_position=None,
#####)
#####except Exception as e:
#####self.logger.debug(f'DOM.describeNode failed for backend_node_id={backend_node_id}: {e}')
Fall back to minimal node if describeNode fails
#####return EnhancedDOMTreeNode(
#####node_id=result.get('nodeId', 0),
#####backend_node_id=backend_node_id,
#####node_type=NodeType.ELEMENT_NODE,
#####node_name='',
#####node_value='',
#####attributes={},
#####is_scrollable=None,
#####frame_id=result.get('frameId'),
#####session_id=session_id,
#####target_id=self.agent_focus_target_id or '',
#####content_document=None,
#####shadow_root_type=None,
#####shadow_roots=None,
#####parent_node=None,
#####children_nodes=None,
#####ax_node=None,
#####snapshot_node=None,
#####is_visible=None,
#####absolute_position=None,
#####)

#####except Exception as e:
#####self.logger.warning(f'Failed to get DOM element at coordinates ({x}, {y}): {e}')
#####return None

#####async def get_target_id_from_tab_id(self, tab_id: str) -> TargetID:
#####"""Get the full-length TargetID from the truncated 4-char tab_id using SessionManager."""
#####if not self.session_manager:
#####raise RuntimeError('SessionManager not initialized')

#####for full_target_id in self.session_manager.get_all_target_ids():
#####if full_target_id.endswith(tab_id):
#####if await self.session_manager.is_target_valid(full_target_id):
#####return full_target_id
Stale target - Chrome should have sent detach event
If we're here, event listener will clean it up
#####self.logger.debug(f'Found stale target {full_target_id}, skipping')

#####raise ValueError(f'No TargetID found ending in tab_id=...{tab_id}')

#####async def get_target_id_from_url(self, url: str) -> TargetID:
#####"""Get the TargetID from a URL using SessionManager (source of truth)."""
#####if not self.session_manager:
#####raise RuntimeError('SessionManager not initialized')

Search in SessionManager targets (exact match first)
#####for target_id, target in self.session_manager.get_all_targets().items():
#####if target.target_type in ('page', 'tab') and target.url == url:
#####return target_id
Still not found, try substring match as fallback

```

```

 for target_id, target in self.session_manager.get_all_targets().items():
 if target.target_type in ('page', 'tab') and url in target.url:
 return target_id

 raise ValueError(f'No TargetID found for url={url}')

 async def get_most_recently_opened_target_id(self) -> TargetID:
 """Get the most recently opened target ID using SessionManager."""
 # Get all page targets from SessionManager
 page_targets = self.session_manager.get_all_page_targets()
 if not page_targets:
 raise RuntimeError('No page targets available')
 return page_targets[-1].target_id

 def is_file_input(self, element: Any) -> bool:
 """Check if element is a file input.

 Args:
 element: The DOM element to check

 Returns:
 True if element is a file input, False otherwise
 """
 if self._dom_watchdog:
 return self._dom_watchdog.is_file_input(element)
 # Fallback if watchdog not available
 return (
 hasattr(element, 'node_name')
 and element.node_name.upper() == 'INPUT'
 and hasattr(element, 'attributes')
 and element.attributes.get('type', '').lower() == 'file'
)

 async def get_selector_map(self) -> dict[int, EnhancedDOMTreeNode]:
 """Get the current selector map from cached state or DOM watchdog.

 Returns:
 Dictionary mapping element indices to EnhancedDOMTreeNode objects
 """
 # First try cached selector map
 if self._cached_selector_map:
 return self._cached_selector_map

 # Try to get from DOM watchdog
 if self._dom_watchdog and hasattr(self._dom_watchdog, 'selector_map'):
 return self._dom_watchdog.selector_map or {}

 # Return empty dict if nothing available
 return {}

 async def get_index_by_id(self, element_id: str) -> int | None:
 """Find element index by its id attribute.

 Args:
 element_id: The id attribute value to search for

 Returns:
 Index of the element, or None if not found
 """
 selector_map = await self.get_selector_map()
 for idx, element in selector_map.items():
 if element.attributes.get('id') == element_id:
 return idx
 return None

 async def get_index_by_class(self, class_name: str) -> int | None:
 """Find element index by its class attribute (matches if class contains the given name).

 Args:
 class_name: The class name to search for

 Returns:

```

```

 """Index of the first matching element, or None if not found
 """
 selector_map = await self.get_selector_map()
 for idx, element in selector_map.items():
 if element.attributes:
 element_class = element.attributes.get('class', '')
 if class_name in element_class.split():
 return idx
 return None

 async def remove_highlights(self) -> None:
 """Remove highlights from the page using CDP."""
 if not self.browser_profile.highlight_elements:
 return

 try:
 # Get cached session
 cdp_session = await self.get_or_create_cdp_session()

 # Remove highlights via JavaScript - be thorough
 script = """
 (function() {
 // Remove all browser-use highlight elements
 const highlights = document.querySelectorAll('[data-browser-use-highlight]');
 console.log('Removing', highlights.length, 'browser-use highlight elements');
 highlights.forEach(el => el.remove());

 // Also remove by ID in case selector missed anything
 const highlightContainer = document.getElementById('browser-use-debug-highlights');
 if (highlightContainer) {
 console.log('Removing highlight container by ID');
 highlightContainer.remove();
 }

 // Final cleanup - remove any orphaned tooltips
 const orphanedTooltips = document.querySelectorAll('[data-browser-use-highlight="tooltip"]');
 orphanedTooltips.forEach(el => el.remove());

 return { removed: highlights.length };
 })();
 """
 result = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': script, 'returnByValue': True}, session_id=cdp_session.session_id
)

 # Log the result for debugging
 if result and 'result' in result and 'value' in result['result']:
 removed_count = result['result']['value'].get('removed', 0)
 self.logger.debug(f'Successfully removed {removed_count} highlight elements')
 else:
 self.logger.debug('Highlight removal completed')

 except Exception as e:
 self.logger.warning(f'Failed to remove highlights: {e}')

 @observe_debug(ignore_input=True, ignore_output=True, name='get_element_coordinates')
 async def get_element_coordinates(self, backend_node_id: int, cdp_session: CDPSession) -> DOMRect | None:
 """Get element coordinates for a backend node ID using multiple methods.

 This method tries DOM.getContentQuads first, then falls back to DOM.getBoxModel,
 and finally uses JavaScript getBoundingClientRect as a last resort.

 Args:
 backend_node_id: The backend node ID to get coordinates for
 cdp_session: The CDP session to use

 Returns:
 DOMRect with coordinates or None if element not found/no bounds
 """
 session_id = cdp_session.session_id
 quads = []
 # Method 1: Try DOM.getContentQuads first (best for inline elements and complex layouts)

```



```

self.logger.debug(f'JavaScript getBoundingClientRect failed: {e}')

Convert quads to bounding rectangle if we have them
if quads:
 # Use the first quad (most relevant for the element)
 quad = quads[0]
 if len(quad) >= 8:
 # Calculate bounding rect from quad points
 x_coords = [quad[i] for i in range(0, 8, 2)]
 y_coords = [quad[i] for i in range(1, 8, 2)]

 min_x = min(x_coords)
 min_y = min(y_coords)
 max_x = max(x_coords)
 max_y = max(y_coords)

 width = max_x - min_x
 height = max_y - min_y

 if width > 0 and height > 0:
 return DOMRect(x=min_x, y=min_y, width=width, height=height)

return None

async def highlight_interaction_element(self, node: 'EnhancedDOMTreeNode') -> None:
 """Temporarily highlight an element during interaction for user visibility.

 This creates a visual highlight on the browser that shows the user which element
 is being interacted with. The highlight automatically fades after the configured duration.

 Args:
 node: The DOM node to highlight with backend_node_id for coordinate lookup
 """
 if not self.browser_profile.highlight_elements:
 return

 try:
 import json

 cdp_session = await self.get_or_create_cdp_session()

 # Get current coordinates
 rect = await self.get_element_coordinates(node.backend_node_id, cdp_session)

 color = self.browser_profile.interaction_highlight_color
 duration_ms = int(self.browser_profile.interaction_highlight_duration * 1000)

 if not rect:
 self.logger.debug(f'No coordinates found for backend node {node.backend_node_id}')
 return

 # Create animated corner brackets that start offset and animate inward
 script = f"""
 (function() {{
 const rect = {json.dumps({'x': rect.x, 'y': rect.y, 'width': rect.width, 'height': rect.height})};
 const color = {json.dumps(color)};
 const duration = {duration_ms};

 // Scale corner size based on element dimensions to ensure gaps between corners
 const maxCornerSize = 20;
 const minCornerSize = 8;
 const cornerSize = Math.max(
 minCornerSize,
 Math.min(maxCornerSize, Math.min(rect.width, rect.height) * 0.35)
);
 const borderWidth = 3;
 const startOffset = 10; // Starting offset in pixels
 const finalOffset = -3; // Final position slightly outside the element

 // Get current scroll position
 const scrollX = window.pageXOffset || document.documentElement.scrollLeft || 0;
 const scrollY = window.pageYOffset || document.documentElement.scrollTop || 0;
 }})
 """
 self.logger.debug(f'Executing highlight script for node {node.backend_node_id}')
 self.execute_script(script, cdp_session)
 except Exception as e:
 self.logger.error(f'Error highlighting element: {e}')

```



```

#####// Create container for all corners
#####const container = document.createElement('div');
#####container.setAttribute('data-browser-use-interaction-highlight', 'true');
#####container.style.cssText = `
#####position: absolute;
#####left: ${rect.x + scrollX}px;
#####top: ${rect.y + scrollY}px;
#####width: ${rect.width}px;
#####height: ${rect.height}px;
#####pointer-events: none;
#####z-index: 2147483647;
#####`;

#####// Create 4 corner brackets
#####const corners = [
#####{{ pos: 'top-left', startX: -startOffset, startY: -startOffset, finalX: finalOffset, finalY: finalOffset}},
#####{{ pos: 'top-right', startX: startOffset, startY: -startOffset, finalX: -finalOffset, finalY: finalOffset}},
#####{{ pos: 'bottom-left', startX: -startOffset, startY: startOffset, finalX: finalOffset, finalY: -finalOffset}},
#####{{ pos: 'bottom-right', startX: startOffset, startY: startOffset, finalX: -finalOffset, finalY: -finalOffset}},
#####];

#####corners.forEach(corner => {{
#####const bracket = document.createElement('div');
#####bracket.style.cssText = `
#####position: absolute;
#####width: ${cornerSize}px;
#####height: ${cornerSize}px;
#####pointer-events: none;
#####transition: all 0.15s ease-out;
#####`;

#####// Position corners
#####if (corner.pos === 'top-left') {{
#####bracket.style.top = '0';
#####bracket.style.left = '0';
#####bracket.style.borderTop = `${borderWidth}px solid ${color}`;
#####bracket.style.borderLeft = `${borderWidth}px solid ${color}`;
#####bracket.style.transform = `translate(${corner.startX}px, ${corner.startY}px)`;
#####}} else if (corner.pos === 'top-right') {{
#####bracket.style.top = '0';
#####bracket.style.right = '0';
#####bracket.style.borderTop = `${borderWidth}px solid ${color}`;
#####bracket.style.borderRight = `${borderWidth}px solid ${color}`;
#####bracket.style.transform = `translate(${corner.startX}px, ${corner.startY}px)`;
#####}} else if (corner.pos === 'bottom-left') {{
#####bracket.style.bottom = '0';
#####bracket.style.left = '0';
#####bracket.style.borderBottom = `${borderWidth}px solid ${color}`;
#####bracket.style.borderLeft = `${borderWidth}px solid ${color}`;
#####bracket.style.transform = `translate(${corner.startX}px, ${corner.startY}px)`;
#####}} else if (corner.pos === 'bottom-right') {{
#####bracket.style.bottom = '0';
#####bracket.style.right = '0';
#####bracket.style.borderBottom = `${borderWidth}px solid ${color}`;
#####bracket.style.borderRight = `${borderWidth}px solid ${color}`;
#####bracket.style.transform = `translate(${corner.startX}px, ${corner.startY}px)`;
#####}}

#####container.appendChild(bracket);

#####// Animate to final position slightly outside the element
#####setTimeout(() => {{
#####bracket.style.transform = `translate(${corner.finalX}px, ${corner.finalY}px)`;
#####}}, 10);
#####}});

#####document.body.appendChild(container);

#####// Auto-remove after duration
#####setTimeout(() => {{
#####container.style.opacity = '0';
#####container.style.transition = 'opacity 0.3s ease-out';
#####}});

```

```

 setTimeout(() => container.remove(), 300);
 }}, duration);

 return {{ created: true }};
 }}());
 """

 # Fire and forget - don't wait for completion

 await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': script, 'returnByValue': True}, session_id=cdp_session.session_id
)

 except Exception as e:
 # Don't fail the action if highlighting fails
 self.logger.debug(f'Failed to highlight interaction element: {e}')

 async def highlight_coordinate_click(self, x: int, y: int) -> None:
 """Temporarily highlight a coordinate click position for user visibility.

 This creates a visual highlight at the specified coordinates showing where
 the click action occurred. The highlight automatically fades after the configured duration.

 Args:
 x: Horizontal coordinate relative to viewport left edge
 y: Vertical coordinate relative to viewport top edge
 """
 if not self.browser_profile.highlight_elements:
 return

 try:
 import json

 cdp_session = await self.get_or_create_cdp_session()

 color = self.browser_profile.interaction_highlight_color
 duration_ms = int(self.browser_profile.interaction_highlight_duration * 1000)

 # Create animated crosshair and circle at the click coordinates
 script = f"""
 (function() {{
 const x = {x};
 const y = {y};
 const color = {json.dumps(color)};
 const duration = {duration_ms};

 // Get current scroll position
 const scrollX = window.pageXOffset || document.documentElement.scrollLeft || 0;
 const scrollY = window.pageYOffset || document.documentElement.scrollTop || 0;

 // Create container
 const container = document.createElement('div');
 container.setAttribute('data-browser-use-coordinate-highlight', 'true');
 container.style.cssText = `
 position: absolute;
 left: ${x + scrollX}px;
 top: ${y + scrollY}px;
 width: 0;
 height: 0;
 pointer-events: none;
 z-index: 2147483647;
 `;

 // Create outer circle
 const outerCircle = document.createElement('div');
 outerCircle.style.cssText = `
 position: absolute;
 left: -15px;
 top: -15px;
 width: 30px;
 height: 30px;
 border: 3px solid ${color};
 """

```

```

 border-radius: 50%;
 opacity: 0;
 transform: scale(0.3);
 transition: all 0.2s ease-out;
 `;
 container.appendChild(outerCircle);

 // Create center dot
 const centerDot = document.createElement('div');
 centerDot.style.cssText = `
 position: absolute;
 left: -4px;
 top: -4px;
 width: 8px;
 height: 8px;
 background: ${color};
 border-radius: 50%;
 opacity: 0;
 transform: scale(0);
 transition: all 0.15s ease-out;
 `;
 container.appendChild(centerDot);

 document.body.appendChild(container);

 // Animate in
 setTimeout(() => {{
 outerCircle.style.opacity = '0.8';
 outerCircle.style.transform = 'scale(1)';
 centerDot.style.opacity = '1';
 centerDot.style.transform = 'scale(1)';
 }}, 10);

 // Animate out and remove
 setTimeout(() => {{
 outerCircle.style.opacity = '0';
 outerCircle.style.transform = 'scale(1.5)';
 centerDot.style.opacity = '0';
 setTimeout(() => container.remove(), 300);
 }}, duration);

 return {{ created: true }};
}}})();
"""

Fire and forget - don't wait for completion
await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': script, 'returnByValue': True}, session_id=cdp_session.session_id
)

except Exception as e:
 # Don't fail the action if highlighting fails
 self.logger.debug(f'Failed to highlight coordinate click: {e}')

async def add_highlights(self, selector_map: dict[int, 'EnhancedDOMTreeNode']) -> None:
 """Add visual highlights to the browser DOM for user visibility."""
 if not self.browser_profile.dom_highlight_elements or not selector_map:
 return

 try:
 import json

 # Convert selector_map to the format expected by the highlighting script
 elements_data = []
 for _, node in selector_map.items():
 # Get bounding box using absolute position (includes iframe translations) if available
 if node.absolute_position:
 # Use absolute position which includes iframe coordinate translations
 rect = node.absolute_position
 bbox = {'x': rect.x, 'y': rect.y, 'width': rect.width, 'height': rect.height}

 # Only include elements with valid bounding boxes

```

```

#####if bbox and bbox.get('width', 0) > 0 and bbox.get('height', 0) > 0:
#####element = {
'x': bbox['x'],
'y': bbox['y'],
'width': bbox['width'],
'height': bbox['height'],
'element_name': node.node_name,
'is_clickable': node.snapshot_node.is_clickable if node.snapshot_node else True,
'is_scrollable': getattr(node, 'is_scrollable', False),
'attributes': node.attributes or {},
'frame_id': getattr(node, 'frame_id', None),
'node_id': node.node_id,
'backend_node_id': node.backend_node_id,
'xpath': node.xpath,
'text_content': node.get_all_children_text()[:50]
if hasattr(node, 'get_all_children_text')
else node.node_value[:50],
#####}
#####elements_data.append(element)

###if not elements_data:
###self.logger.debug('### No valid elements to highlight')
###return

###self.logger.debug(f'### Creating highlights for {len(elements_data)} elements')

Always remove existing highlights first
###await self.remove_highlights()

Add a small delay to ensure removal completes
###import asyncio

###await asyncio.sleep(0.05)

Get CDP session
###cdp_session = await self.get_or_create_cdp_session()

Create the proven highlighting script from v0.6.0 with fixed positioning
###script = f"""
###(function() {{
#####// Interactive elements data
#####const interactiveElements = {json.dumps(elements_data)};

#####console.log('=== BROWSER-USE HIGHLIGHTING ===');
#####console.log('Highlighting', interactiveElements.length, 'interactive elements');

#####// Double-check: Remove any existing highlight container first
#####const existingContainer = document.getElementById('browser-use-debug-highlights');
#####if (existingContainer) {{
console.log('### Found existing highlight container, removing it first');
existingContainer.remove();
#####}}

#####// Also remove any stray highlight elements
#####const strayHighlights = document.querySelectorAll('[data-browser-use-highlight]');
#####if (strayHighlights.length > 0) {{
console.log('### Found', strayHighlights.length, 'stray highlight elements, removing them');
strayHighlights.forEach(el => el.remove());
#####}}

#####// Use maximum z-index for visibility
#####const HIGHLIGHT_Z_INDEX = 2147483647;

#####// Create container for all highlights - use FIXED positioning (key insight from v0.6.0)
#####const container = document.createElement('div');
#####container.id = 'browser-use-debug-highlights';
#####container.setAttribute('data-browser-use-highlight', 'container');

#####container.style.cssText = `
#####position: absolute;
#####top: 0;
#####left: 0;

```

```

width: 100vw;
height: 100vh;
pointer-events: none;
z-index: ${HIGHLIGHT_Z_INDEX};
overflow: visible;
margin: 0;
padding: 0;
border: none;
outline: none;
box-shadow: none;
background: none;
font-family: inherit;
`;

// Helper function to create text elements safely
function createTextElement(tag, text, styles) {{
const element = document.createElement(tag);
element.textContent = text;
if (styles) element.style.cssText = styles;
return element;
}}

// Add highlights for each element
interactiveElements.forEach((element, index) => {{
const highlight = document.createElement('div');
highlight.setAttribute('data-browser-use-highlight', 'element');
highlight.setAttribute('data-element-id', element.backend_node_id);
highlight.style.cssText = `
position: absolute;
left: ${element.x}}px;
top: ${element.y}}px;
width: ${element.width}}px;
height: ${element.height}}px;
outline: 2px dashed #4a90e2;
outline-offset: -2px;
background: transparent;
pointer-events: none;
box-sizing: content-box;
transition: outline 0.2s ease;
margin: 0;
padding: 0;
border: none;
`;

// Enhanced label with backend node ID
const label = createTextElement('div', element.backend_node_id, `
position: absolute;
top: -20px;
left: 0;
background-color: #4a90e2;
color: white;
padding: 2px 6px;
font-size: 11px;
font-family: 'Monaco', 'Menlo', 'Ubuntu Mono', monospace;
font-weight: bold;
border-radius: 3px;
white-space: nowrap;
z-index: ${HIGHLIGHT_Z_INDEX + 1};
box-shadow: 0 2px 4px rgba(0,0,0,0.3);
border: none;
outline: none;
margin: 0;
line-height: 1.2;
`);

highlight.appendChild(label);
container.appendChild(highlight);
}});

// Add container to document
document.body.appendChild(container);
console.log('Highlighting complete - added', interactiveElements.length, 'highlights');

```

```

 return {{ added: interactiveElements.length }};
 }}));
 """

 # Execute the script
 result = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': script, 'returnByValue': True}, session_id=cdp_session.session_id
)

 # Log the result
 if result and 'result' in result and 'value' in result['result']:
 added_count = result['result']['value'].get('added', 0)
 self.logger.debug(f'Successfully added {added_count} highlight elements to browser DOM')
 else:
 self.logger.debug('Browser highlight injection completed')

 except Exception as e:
 self.logger.warning(f'Failed to add browser highlights: {e}')
 import traceback

 self.logger.debug(f'Browser highlight traceback: {traceback.format_exc()}')

 async def _close_extension_options_pages(self) -> None:
 """Close any extension options/welcome pages that have opened."""
 try:
 # Get all page targets from SessionManager
 page_targets = self.session_manager.get_all_page_targets()

 for target in page_targets:
 target_url = target.url
 target_id = target.target_id

 # Check if this is an extension options/welcome page
 if 'chrome-extension://' in target_url and (
 'options.html' in target_url or 'welcome.html' in target_url or 'onboarding.html' in target_url
):
 self.logger.info(f'[BrowserSession] Closing extension options page: {target_url}')
 try:
 await self._cdp_close_page(target_id)
 except Exception as e:
 self.logger.debug(f'[BrowserSession] Could not close extension page {target_id}: {e}')

 except Exception as e:
 self.logger.debug(f'[BrowserSession] Error closing extension options pages: {e}')

 async def send_demo_mode_log(self, message: str, level: str = 'info', metadata: dict[str, Any] | None = None):
 """Send a message to the in-browser demo panel if enabled."""
 if not self.browser_profile.demo_mode:
 return
 demo = self.demo_mode
 if not demo:
 return
 try:
 await demo.send_log(message=message, level=level, metadata=metadata or {})
 except Exception as exc:
 self.logger.debug(f'[DemoMode] Failed to send log: {exc}')

 @property
 def downloaded_files(self) -> list[str]:
 """Get list of files downloaded during this browser session.

 Returns:
 list[str]: List of absolute file paths to downloaded files in this session
 """
 return self._downloaded_files.copy()

 # endregion - ===== Helper Methods =====

 # region - ===== CDP-based replacements for browser_context operations =====

 async def _cdp_get_all_pages(
 self,

```

```

 include_http: bool = True,
 include_about: bool = True,
 include_pages: bool = True,
 include_iframes: bool = False,
 include_workers: bool = False,
 include_chrome: bool = False,
 include_chrome_extensions: bool = False,
 include_chrome_error: bool = False,
) -> list[TargetInfo]:
 """Get all browser pages/tabs using SessionManager (source of truth)."""
 # Safety check - return empty list if browser not connected yet
 if not self.session_manager:
 return []

 # Build TargetInfo dicts from SessionManager owned data (crystal clear ownership)
 result = []
 for target_id, target in self.session_manager.get_all_targets().items():
 # Create TargetInfo dict
 target_info: TargetInfo = {
 'targetId': target.target_id,
 'type': target.target_type,
 'title': target.title,
 'url': target.url,
 'attached': True,
 'canAccessOpener': False,
 }

 # Apply filters
 if self._is_valid_target(
 target_info,
 include_http=include_http,
 include_about=include_about,
 include_pages=include_pages,
 include_iframes=include_iframes,
 include_workers=include_workers,
 include_chrome=include_chrome,
 include_chrome_extensions=include_chrome_extensions,
 include_chrome_error=include_chrome_error,
):
 result.append(target_info)

 return result

 async def _cdp_create_new_page(self, url: str = 'about:blank', background: bool = False, new_window: bool = False):
 """Create a new page/tab using CDP Target.createTarget. Returns target ID."""
 # Use the root CDP client to create tabs at the browser level
 if self._cdp_client_root:
 result = await self._cdp_client_root.send.Target.createTarget(
 params={'url': url, 'newWindow': new_window, 'background': background}
)
 else:
 # Fallback to using cdp_client if root is not available
 result = await self.cdp_client.send.Target.createTarget(
 params={'url': url, 'newWindow': new_window, 'background': background}
)
 return result['targetId']

 async def _cdp_close_page(self, target_id: TargetID) -> None:
 """Close a page/tab using CDP Target.closeTarget."""
 await self.cdp_client.send.Target.closeTarget(params={'targetId': target_id})

 async def _cdp_get_cookies(self) -> list[Cookie]:
 """Get cookies using CDP Network.getCookies."""
 cdp_session = await self.get_or_create_cdp_session(target_id=None)
 result = await asyncio.wait_for(
 cdp_session.cdp_client.send.Storage.getCookies(session_id=cdp_session.session_id), timeout=8.0
)
 return result.get('cookies', [])

 async def _cdp_set_cookies(self, cookies: list[Cookie]) -> None:
 """Set cookies using CDP Storage.setCookies."""
 if not self.agent_focus_target_id or not cookies:

```

```

 return

 cdp_session = await self.get_or_create_cdp_session(target_id=None)
 # Storage.setCookies expects params dict with 'cookies' key
 await cdp_session.cdp_client.send.Storage.setCookies(
 params={'cookies': cookies}, # type: ignore[arg-type]
 session_id=cdp_session.session_id,
)

 async def _cdp_clear_cookies(self) -> None:
 """Clear all cookies using CDP Network.clearBrowserCookies."""
 cdp_session = await self.get_or_create_cdp_session()
 await cdp_session.cdp_client.send.Storage.clearCookies(session_id=cdp_session.session_id)

 async def _cdp_grant_permissions(self, permissions: list[str], origin: str | None = None) -> None:
 """Grant permissions using CDP Browser.grantPermissions."""
 params = {'permissions': permissions}
 # if origin:
 # params['origin'] = origin
 cdp_session = await self.get_or_create_cdp_session()
 # await cdp_session.cdp_client.send.Browser.grantPermissions(params=params, session_id=cdp_session.session_id)
 raise NotImplementedError('Not implemented yet')

 async def _cdp_set_geolocation(self, latitude: float, longitude: float, accuracy: float = 100) -> None:
 """Set geolocation using CDP Emulation.setGeolocationOverride."""
 await self.cdp_client.send.Emulation.setGeolocationOverride(
 params={'latitude': latitude, 'longitude': longitude, 'accuracy': accuracy}
)

 async def _cdp_clear_geolocation(self) -> None:
 """Clear geolocation override using CDP."""
 await self.cdp_client.send.Emulation.clearGeolocationOverride()

 async def _cdp_add_init_script(self, script: str) -> str:
 """Add script to evaluate on new document using CDP Page.addScriptToEvaluateOnNewDocument."""
 assert self._cdp_client_root is not None
 cdp_session = await self.get_or_create_cdp_session()

 result = await cdp_session.cdp_client.send.Page.addScriptToEvaluateOnNewDocument(
 params={'source': script, 'runImmediately': True}, session_id=cdp_session.session_id
)
 return result['identifier']

 async def _cdp_remove_init_script(self, identifier: str) -> None:
 """Remove script added with addScriptToEvaluateOnNewDocument."""
 cdp_session = await self.get_or_create_cdp_session(target_id=None)
 await cdp_session.cdp_client.send.Page.removeScriptToEvaluateOnNewDocument(
 params={'identifier': identifier}, session_id=cdp_session.session_id
)

 async def _cdp_set_viewport(
 self, width: int, height: int, device_scale_factor: float = 1.0, mobile: bool = False, target_id: str | None = None
) -> None:
 """Set viewport using CDP Emulation.setDeviceMetricsOverride.

 Args:
 width: Viewport width
 height: Viewport height
 device_scale_factor: Device scale factor (default 1.0)
 mobile: Whether to emulate mobile device (default False)
 target_id: Optional target ID to set viewport for. If not provided, uses agent_focus.
 """
 if target_id:
 # Set viewport for specific target
 cdp_session = await self.get_or_create_cdp_session(target_id, focus=False)
 elif self.agent_focus_target_id:
 # Use current focus - use safe API with focus=False to avoid changing focus
 try:
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 except ValueError:
 self.logger.warning('Cannot set viewport: focused target has no sessions')
 return

```



```

 else:
 self.logger.warning('Cannot set viewport: no target_id provided and agent_focus not initialized')
 return

 await cdp_session.cdp_client.send.Emulation.setDeviceMetricsOverride(
 params={'width': width, 'height': height, 'deviceScaleFactor': device_scale_factor, 'mobile': mobile},
 session_id=cdp_session.session_id,
)

 async def _cdp_get_origins(self) -> list[dict[str, Any]]:
 """Get origins with localStorage and sessionStorage using CDP."""
 origins = []
 cdp_session = await self.get_or_create_cdp_session(target_id=None)

 try:
 # Enable DOMStorage domain to track storage
 await cdp_session.cdp_client.send.DOMStorage.enable(session_id=cdp_session.session_id)

 try:
 # Get all frames to find unique origins
 frames_result = await cdp_session.cdp_client.send.Page.getFrameTree(session_id=cdp_session.session_id)

 # Extract unique origins from frames
 unique_origins = set()

 def _extract_origins(frame_tree):
 """Recursively extract origins from frame tree."""
 frame = frame_tree.get('frame', {})
 origin = frame.get('securityOrigin')
 if origin and origin != 'null':
 unique_origins.add(origin)

 # Process child frames
 for child in frame_tree.get('childFrames', []):
 _extract_origins(child)

 async def _get_storage_items(origin: str, is_local_storage: bool) -> list[dict[str, str]] | None:
 """Helper to get storage items for an origin."""
 storage_type = 'localStorage' if is_local_storage else 'sessionStorage'
 try:
 result = await cdp_session.cdp_client.send.DOMStorage.getDOMStorageItems(
 params={'storageId': {'securityOrigin': origin, 'isLocalStorage': is_local_storage}},
 session_id=cdp_session.session_id,
)

 items = []
 for item in result.get('entries', []):
 if len(item) == 2: # Each item is [key, value]
 items.append({'name': item[0], 'value': item[1]})

 return items if items else None
 except Exception as e:
 self.logger.debug(f'Failed to get {storage_type} for {origin}: {e}')
 return None

 _extract_origins(frames_result.get('frameTree', {}))

 # For each unique origin, get localStorage and sessionStorage
 for origin in unique_origins:
 origin_data = {'origin': origin}

 # Get localStorage
 local_storage = await _get_storage_items(origin, is_local_storage=True)
 if local_storage:
 origin_data['localStorage'] = local_storage

 # Get sessionStorage
 session_storage = await _get_storage_items(origin, is_local_storage=False)
 if session_storage:
 origin_data['sessionStorage'] = session_storage

 # Only add origin if it has storage data

```

```

 if 'localStorage' in origin_data or 'sessionStorage' in origin_data:
 origins.append(origin_data)

 finally:
 # Always disable DOMStorage tracking when done
 await cdp_session.cdp_client.send.DOMStorage.disable(session_id=cdp_session.session_id)

 except Exception as e:
 self.logger.warning(f'Failed to get origins: {e}')

 return origins

 async def _cdp_get_storage_state(self) -> dict:
 """Get storage state (cookies, localStorage, sessionStorage) using CDP."""
 # Use the _cdp_get_cookies helper which handles session attachment
 cookies = await self._cdp_get_cookies()

 # Get origins with localStorage/sessionStorage
 origins = await self._cdp_get_origins()

 return {
 'cookies': cookies,
 'origins': origins,
 }

 async def _cdp_navigate(self, url: str, target_id: TargetID | None = None) -> None:
 """Navigate to URL using CDP Page.navigate."""
 # Use provided target_id or fall back to agent_focus_target_id

 assert self._cdp_client_root is not None, 'CDP client not initialized - browser may not be connected'
 assert self.agent_focus_target_id is not None, 'Agent focus not initialized - browser may not be connected'

 target_id_to_use = target_id or self.agent_focus_target_id
 cdp_session = await self.get_or_create_cdp_session(target_id_to_use, focus=True)

 # Use helper to navigate on the target
 await cdp_session.cdp_client.send.Page.navigate(params={'url': url}, session_id=cdp_session.session_id)

 @staticmethod
 def _is_valid_target(
 target_info: TargetInfo,
 include_http: bool = True,
 include_chrome: bool = False,
 include_chrome_extensions: bool = False,
 include_chrome_error: bool = False,
 include_about: bool = True,
 include_iframes: bool = True,
 include_pages: bool = True,
 include_workers: bool = False,
) -> bool:
 """Check if a target should be processed.

 Args:
 target_info: Target info dict from CDP

 Returns:
 True if target should be processed, False if it should be skipped
 """
 target_type = target_info.get('type', '')
 url = target_info.get('url', '')

 url_allowed, type_allowed = False, False

 # Always allow new tab pages (chrome://new-tab-page/, chrome://newtab/, about:blank)
 # so they can be redirected to about:blank in connect()
 from system.utils import is_new_tab_page

 if is_new_tab_page(url):
 url_allowed = True

 if url.startswith('chrome-error://') and include_chrome_error:
 url_allowed = True

```

```

 if url.startswith('chrome://') and include_chrome:
 url_allowed = True

 if url.startswith('chrome-extension://') and include_chrome_extensions:
 url_allowed = True

 # dont allow about:srcdoc! there are also other rare about: pages that we want to avoid
 if url == 'about:blank' and include_about:
 url_allowed = True

 if (url.startswith('http://') or url.startswith('https://')) and include_http:
 url_allowed = True

 if target_type in ('service_worker', 'shared_worker', 'worker') and include_workers:
 type_allowed = True

 if target_type in ('page', 'tab') and include_pages:
 type_allowed = True

 if target_type in ('iframe', 'webview') and include_iframes:
 type_allowed = True

 return url_allowed and type_allowed

 async def get_all_frames(self) -> tuple[dict[str, dict], dict[str, str]]:
 """Get a complete frame hierarchy from all browser targets.

 Returns:
 Tuple of (all_frames, target_sessions) where:
 - all_frames: dict mapping frame_id -> frame info dict with all metadata
 - target_sessions: dict mapping target_id -> session_id for active sessions
 """
 all_frames = {} # frame_id -> FrameInfo dict
 target_sessions = {} # target_id -> session_id (keep sessions alive during collection)

 # Check if cross-origin iframe support is enabled
 include_cross_origin = self.browser_profile.cross_origin_iframes

 # Get all targets - only include iframes if cross-origin support is enabled
 targets = await self._cdp_get_all_pages(
 include_http=True,
 include_about=True,
 include_pages=True,
 include_iframes=include_cross_origin, # Only include iframe targets if flag is set
 include_workers=False,
 include_chrome=False,
 include_chrome_extensions=False,
 include_chrome_error=include_cross_origin, # Only include error pages if cross-origin is enabled
)
 all_targets = targets

 # First pass: collect frame trees from ALL targets
 for target in all_targets:
 target_id = target['targetId']

 # Skip iframe targets if cross-origin support is disabled
 if not include_cross_origin and target.get('type') == 'iframe':
 continue

 # When cross-origin support is disabled, only process the current target
 if not include_cross_origin:
 # Only process the current focus target
 if self.agent_focus_target_id and target_id != self.agent_focus_target_id:
 continue
 # Use the existing agent_focus target's session - use safe API with focus=False
 try:
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 except ValueError:
 continue # Skip if no session available
 else:
 # Get cached session for this target (don't change focus - iterating frames)
 cdp_session = await self.get_or_create_cdp_session(target_id, focus=False)

```

```

#####if cdp_session:
#####target_sessions[target_id] = cdp_session.session_id

#####try:
Try to get frame tree (not all target types support this)
#####frame_tree_result = await cdp_session.cdp_client.send.Page.getFrameTree(session_id=cdp_session.session_id)

Process the frame tree recursively
#####def process_frame_tree(node, parent_frame_id=None):
"""Recursively process frame tree and add to all_frames."""
frame = node.get('frame', {})
current_frame_id = frame.get('id')

if current_frame_id:
For iframe targets, check if the frame has a parentId field
This indicates it's an OOPIF with a parent in another target
actual_parent_id = frame.get('parentId') or parent_frame_id

Create frame info with all CDP response data plus our additions
frame_info = {
**frame, # Include all original frame data: id, url, parentId, etc.
'frameTargetId': target_id, # Target that can access this frame
'parentId': actual_parent_id, # Use parentId from frame if available
'childFrameIds': [], # Will be populated below
'isCrossOrigin': False, # Will be determined based on context
'isValidTarget': self._is_valid_target(
target,
include_http=True,
include_about=True,
include_pages=True,
include_iframes=True,
include_workers=False,
include_chrome=False, # chrome://newtab, chrome://settings, etc. are not valid frames we can
include_chrome_extensions=False, # chrome-extension://
include_chrome_error=False, # chrome-error:// (e.g. when iframes fail to load or are blocked)
),
}

Check if frame is cross-origin based on crossOriginIsolatedContextType
cross_origin_type = frame.get('crossOriginIsolatedContextType')
if cross_origin_type and cross_origin_type != 'NotIsolated':
frame_info['isCrossOrigin'] = True

For iframe targets, the frame itself is likely cross-origin
if target.get('type') == 'iframe':
frame_info['isCrossOrigin'] = True

Skip cross-origin frames if support is disabled
if not include_cross_origin and frame_info.get('isCrossOrigin'):
return # Skip this frame and its children

Add child frame IDs (note: OOPIFs won't appear here)
child_frames = node.get('childFrames', [])
for child in child_frames:
child_frame = child.get('frame', {})
child_frame_id = child_frame.get('id')
if child_frame_id:
frame_info['childFrameIds'].append(child_frame_id)

Store or merge frame info
if current_frame_id in all_frames:
Frame already seen from another target, merge info
existing = all_frames[current_frame_id]
If this is an iframe target, it has direct access to the frame
if target.get('type') == 'iframe':
existing['frameTargetId'] = target_id
existing['isCrossOrigin'] = True
else:
all_frames[current_frame_id] = frame_info

Process child frames recursively (only if we're not skipping this frame)
if include_cross_origin or not frame_info.get('isCrossOrigin'):

```

```

#####for child in child_frames:
#####process_frame_tree(child, current_frame_id)

Process the entire frame tree
#####process_frame_tree(frame_tree_result.get('frameTree', {}))

#####except Exception as e:
Target doesn't support Page domain or has no frames
#####self.logger.debug(f'Failed to get frame tree for target {target_id}: {e}')

Second pass: populate backend node IDs and parent target IDs
Only do this if cross-origin support is enabled
#####if include_cross_origin:
#####await self._populate_frame_metadata(all_frames, target_sessions)

#####return all_frames, target_sessions

#####async def _populate_frame_metadata(self, all_frames: dict[str, dict], target_sessions: dict[str, str])
#####"""Populate additional frame metadata like backend node IDs and parent target IDs.

#####Args:
#####all_frames: Frame hierarchy dict to populate
#####target_sessions: Active target sessions
#####"""
#####for frame_id_iter, frame_info in all_frames.items():
#####parent_frame_id = frame_info.get('parentFrameId')

#####if parent_frame_id and parent_frame_id in all_frames:
#####parent_frame_info = all_frames[parent_frame_id]
#####parent_target_id = parent_frame_info.get('frameTargetId')

Store parent target ID
#####frame_info['parentTargetId'] = parent_target_id

Try to get backend node ID from parent context
#####if parent_target_id in target_sessions:
#####assert parent_target_id is not None
#####parent_session_id = target_sessions[parent_target_id]
#####try:
Enable DOM domain
#####await self.cdp_client.send.DOM.enable(session_id=parent_session_id)

Get frame owner info to find backend node ID
#####frame_owner = await self.cdp_client.send.DOM.getFrameOwner(
#####params={'frameId': frame_id_iter}, session_id=parent_session_id
#####)

#####if frame_owner:
#####frame_info['backendNodeId'] = frame_owner.get('backendNodeId')
#####frame_info['nodeId'] = frame_owner.get('nodeId')

#####except Exception:
Frame owner not available (likely cross-origin)
#####pass

#####async def find_frame_target(self, frame_id: str, all_frames: dict[str, dict] | None = None) -> dict | None
#####"""Find the frame info for a specific frame ID.

#####Args:
#####frame_id: The frame ID to search for
#####all_frames: Optional pre-built frame hierarchy. If None, will call get_all_frames()

#####Returns:
#####Frame info dict if found, None otherwise
#####"""
#####if all_frames is None:
#####all_frames, _ = await self.get_all_frames()

#####return all_frames.get(frame_id)

#####async def cdp_client_for_target(self, target_id: TargetID) -> CDPSession:
#####return await self.get_or_create_cdp_session(target_id, focus=False)

```

```

 async def cdp_client_for_frame(self, frame_id: str) -> CDPSession:
 """Get a CDP client attached to the target containing the specified frame.

 Builds a unified frame hierarchy from all targets to find the correct target
 for any frame, including OOPiFs (Out-of-Process iframes).

 Args:
 frame_id: The frame ID to search for

 Returns:
 Tuple of (cdp_cdp_session, target_id) for the target containing the frame

 Raises:
 ValueError: If the frame is not found in any target
 """
 # If cross-origin iframes are disabled, just use the main session
 if not self.browser_profile.cross_origin_iframes:
 return await self.get_or_create_cdp_session()

 # Get complete frame hierarchy
 all_frames, target_sessions = await self.get_all_frames()

 # Find the requested frame
 frame_info = await self.find_frame_target(frame_id, all_frames)

 if frame_info:
 target_id = frame_info.get('frameTargetId')

 if target_id in target_sessions:
 assert target_id is not None
 # Use existing session
 session_id = target_sessions[target_id]
 # Return the client with session attached (don't change focus)
 return await self.get_or_create_cdp_session(target_id, focus=False)

 # Frame not found
 raise ValueError(f"Frame with ID '{frame_id}' not found in any target")

 async def cdp_client_for_node(self, node: EnhancedDOMTreeNode) -> CDPSession:
 """Get CDP client for a specific DOM node based on its frame.

 IMPORTANT: backend_node_id is only valid in the session where the DOM was captured.
 We trust the node's session_id/frame_id/target_id instead of searching all sessions.
 """

 # Strategy 1: If node has session_id, try to use that exact session (most specific)
 if node.session_id and self.session_manager:
 try:
 # Find the CDP session by session_id from SessionManager
 cdp_session = self.session_manager.get_session(node.session_id)
 if cdp_session:
 # Get target to log URL
 target = self.session_manager.get_target(cdp_session.target_id)
 self.logger.debug(f"Using session from node.session_id for node {node.backend_node_id}: {target.url}")
 return cdp_session
 except Exception as e:
 self.logger.debug(f'Failed to get session by session_id {node.session_id}: {e}')

 # Strategy 2: If node has frame_id, use that frame's session
 if node.frame_id:
 try:
 cdp_session = await self.cdp_client_for_frame(node.frame_id)
 target = self.session_manager.get_target(cdp_session.target_id)
 self.logger.debug(f"Using session from node.frame_id for node {node.backend_node_id}: {target.url}")
 return cdp_session
 except Exception as e:
 self.logger.debug(f'Failed to get session for frame {node.frame_id}: {e}')

 # Strategy 3: If node has target_id, use that target's session
 if node.target_id:
 try:
 cdp_session = await self.get_or_create_cdp_session(target_id=node.target_id, focus=False)

```

```

 target = self.session_manager.get_target(cdp_session.target_id)
 self.logger.debug(f'Using session from node.target_id for node {node.backend_node_id}: {target.url}')
 return cdp_session
 except Exception as e:
 self.logger.debug(f'Failed to get session for target {node.target_id}: {e}')

 ## Strategy 4: Fallback to agent_focus_target_id (the page where agent is currently working)
 if self.agent_focus_target_id:
 target = self.session_manager.get_target(self.agent_focus_target_id)
 try:
 ## Use safe API with focus=False to avoid changing focus
 cdp_session = await self.get_or_create_cdp_session(self.agent_focus_target_id, focus=False)
 if target:
 self.logger.warning(
 f'Node {node.backend_node_id} has no session/frame/target info. Using agent_focus session: {target.url}')
 return cdp_session
 except ValueError:
 pass # Fall through to last resort

 ## Last resort: use main session
 self.logger.error(f'No session info for node {node.backend_node_id} and no agent_focus available. Using main session')
 return await self.get_or_create_cdp_session()

@observe_debug(ignore_input=True, ignore_output=True, name='take_screenshot')
async def take_screenshot(
 self,
 path: str | None = None,
 full_page: bool = False,
 format: str = 'png',
 quality: int | None = None,
 clip: dict | None = None,
) -> bytes:
 """Take a screenshot using CDP.

 Args:
 path: Optional file path to save screenshot
 full_page: Capture entire scrollable page beyond viewport
 format: Image format ('png', 'jpeg', 'webp')
 quality: Quality 0-100 for JPEG format
 clip: Region to capture {'x': int, 'y': int, 'width': int, 'height': int}

 Returns:
 Screenshot data as bytes
 """
 import base64

 from cdp_use.cdp.page import CaptureScreenshotParameters

 cdp_session = await self.get_or_create_cdp_session()

 ## Build parameters dict explicitly to satisfy TypedDict expectations
 params: CaptureScreenshotParameters = {
 'format': format,
 'captureBeyondViewport': full_page,
 }

 if quality is not None and format == 'jpeg':
 params['quality'] = quality

 if clip:
 params['clip'] = {
 'x': clip['x'],
 'y': clip['y'],
 'width': clip['width'],
 'height': clip['height'],
 'scale': 1,
 }

 params = CaptureScreenshotParameters(**params)

 result = await cdp_session.cdp_client.send.Page.captureScreenshot(params=params, session_id=cdp_session.session_id)

```

```

 if not result or 'data' not in result:
 raise Exception('Screenshot failed - no data returned')

 screenshot_data = base64.b64decode(result['data'])

 if path:
 Path(path).write_bytes(screenshot_data)

 return screenshot_data

 async def screenshot_element(
 self,
 selector: str,
 path: str | None = None,
 format: str = 'png',
 quality: int | None = None,
) -> bytes:
 """Take a screenshot of a specific element.

 Args:
 selector: CSS selector for the element
 path: Optional file path to save screenshot
 format: Image format ('png', 'jpeg', 'webp')
 quality: Quality 0-100 for JPEG format

 Returns:
 Screenshot data as bytes
 """

 bounds = await self._get_element_bounds(selector)
 if not bounds:
 raise ValueError(f"Element '{selector}' not found or has no bounds")

 return await self.take_screenshot(
 path=path,
 format=format,
 quality=quality,
 clip=bounds,
)

 async def _get_element_bounds(self, selector: str) -> dict | None:
 """Get element bounding box using CDP."""

 cdp_session = await self.get_or_create_cdp_session()

 # Get document
 doc = await cdp_session.cdp_client.send.DOM.getDocument(params={'depth': 1}, session_id=cdp_session.session_id)

 # Query selector
 node_result = await cdp_session.cdp_client.send.DOM.querySelector(
 params={'nodeId': doc['root']['nodeId'], 'selector': selector}, session_id=cdp_session.session_id
)

 node_id = node_result.get('nodeId')
 if not node_id:
 return None

 # Get bounding box
 box_result = await cdp_session.cdp_client.send.DOM.getBoxModel(
 params={'nodeId': node_id}, session_id=cdp_session.session_id
)

 box_model = box_result.get('model')
 if not box_model:
 return None

 content = box_model['content']
 return {
 'x': min(content[0], content[2], content[4], content[6]),
 'y': min(content[1], content[3], content[5], content[7]),
 'width': max(content[0], content[2], content[4], content[6]) - min(content[0], content[2], content[4], content[6]),
 'height': max(content[1], content[3], content[5], content[7]) - min(content[1], content[3], content[5], content[7])
 }

```



```
 }
}
```

## **`.system\browser\session_manager.py`**

```
"""Event-driven CDP session management.

Manages CDP sessions by listening to Target.attachedToTarget and Target.detachedFromTarget
events, ensuring the session pool always reflects the current browser state.
"""

import asyncio
from typing import TYPE_CHECKING

from cdp_use.cdp.target import AttachedToTargetEvent, DetachedFromTargetEvent, SessionID, TargetID

from system.utils import create_task_with_error_handling

if TYPE_CHECKING:
 from system.browser.session import BrowserSession, CDPSession, Target

class SessionManager:
 """Event-driven CDP session manager.

 Automatically synchronizes the CDP session pool with browser state via CDP events.

 Key features:
 - Sessions added/removed automatically via Target attach/detach events
 - Multiple sessions can attach to the same target
 - Targets only removed when ALL sessions detach
 - No stale sessions - pool always reflects browser reality

 SessionManager is the SINGLE SOURCE OF TRUTH for all targets and sessions.
 """

 def __init__(self, browser_session: 'BrowserSession'):
 self.browser_session = browser_session
 self.logger = browser_session.logger

 ## All targets (entities: pages, iframes, workers)
 self._targets: dict[TargetID, 'Target'] = {}

 ## All sessions (communication channels)
 self._sessions: dict[SessionID, 'CDPSession'] = {}

 ## Mapping: target -> sessions attached to it
 self._target_sessions: dict[TargetID, set[SessionID]] = {}

 ## Reverse mapping: session -> target it belongs to
 self._session_to_target: dict[SessionID, TargetID] = {}

 self._lock = asyncio.Lock()
 self._recovery_lock = asyncio.Lock()

 ## Focus recovery coordination - event-driven instead of polling
 self._recovery_in_progress: bool = False
 self._recovery_complete_event: asyncio.Event | None = None
 self._recovery_task: asyncio.Task | None = None

 async def start_monitoring(self) -> None:
 """Start monitoring Target attach/detach events.

 Registers CDP event handlers to keep the session pool synchronized with browser state.
 Also discovers and initializes all existing targets on startup.
 """
 if not self.browser_session._cdp_client_root:
 raise RuntimeError('CDP client not initialized')
 ## Capture cdp_client_root in closure to avoid type errors
```

```

 cdp_client = self.browser_session._cdp_client_root

 ## Enable target discovery to receive targetInfoChanged events automatically
 ## This eliminates the need for getTargetInfo() polling calls
 await cdp_client.send(Target.setDiscoverTargets(
 params={'discover': True, 'filter': [{'type': 'page'}, {'type': 'iframe'}]}
))

 ## Register synchronous event handlers (CDP requirement)
 def on_attached(event: AttachedToTargetEvent, session_id: SessionID | None = None):
 ## _handle_target_attached() handles:
 ## - setAutoAttach for children
 ## - Create CDPSession
 ## - Enable monitoring (for pages/tabs)
 ## - Add to pool
 create_task_with_error_handling(
 self._handle_target_attached(event),
 name='handle_target_attached',
 logger_instance=self.logger,
 suppress_exceptions=True,
)

 def on_detached(event: DetachedFromTargetEvent, session_id: SessionID | None = None):
 create_task_with_error_handling(
 self._handle_target_detached(event),
 name='handle_target_detached',
 logger_instance=self.logger,
 suppress_exceptions=True,
)

 def on_target_info_changed(event, session_id: SessionID | None = None):
 ## Update session info from targetInfoChanged events (no polling needed!)
 create_task_with_error_handling(
 self._handle_target_info_changed(event),
 name='handle_target_info_changed',
 logger_instance=self.logger,
 suppress_exceptions=True,
)

 cdp_client.register.Target.attachedToTarget(on_attached)
 cdp_client.register.Target.detachedFromTarget(on_detached)
 cdp_client.register.Target.targetInfoChanged(on_target_info_changed)

 self.logger.debug('[SessionManager] Event monitoring started')

 ## Discover and initialize ALL existing targets
 await self._initialize_existing_targets()

 def _get_session_for_target(self, target_id: TargetID) -> 'CDPSession | None':
 """Internal: Get ANY valid session for a target (picks first available).

 INTERNAL API - Use browser_session.get_or_create_cdp_session() instead!
 This method has no validation, no focus management, no recovery.

 Args:
 target_id: Target ID to get session for

 Returns:
 CDPSession if exists, None if target has detached
 """
 session_ids = self._target_sessions.get(target_id, set())
 if not session_ids:
 ## Check if this is the focused target - indicates stale focus that needs cleanup
 if self.browser_session.agent_focus_target_id == target_id:
 self.logger.warning(
 f'[SessionManager] Attempted to get session for stale focused target {target_id[:8]}... '
 f'Clearing stale focus and triggering recovery.'
)

 ## Clear stale focus immediately (defense in depth)
 self.browser_session.agent_focus_target_id = None
 ## Trigger recovery if not already in progress

```

```

#####if not self._recovery_in_progress:
#####self.logger.warning('[SessionManager] Recovery was not in progress! Triggering now.')
#####self._recovery_task = create_task_with_error_handling(
#####self._recover_agent_focus(target_id),
#####name='recover_agent_focus_from_stale_get',
#####logger_instance=self.logger,
#####suppress_exceptions=False,
#####)
#####return None
#####return self._sessions.get(next(iter(session_ids)))

def get_all_page_targets(self) -> list:
#####Get all page/tab targets using owned data.

#####Returns:
#####List of Target objects for all page/tab targets
#####"""
#####page_targets = []
#####for target in self._targets.values():
#####if target.target_type in ('page', 'tab'):
#####page_targets.append(target)
#####return page_targets

#####async def validate_session(self, target_id: TargetID) -> bool:
#####"""Check if a target still has active sessions.

#####Args:
#####target_id: Target ID to validate

#####Returns:
#####True if target has active sessions, False if it should be removed
#####"""
#####if target_id not in self._target_sessions:
#####return False
#####return len(self._target_sessions[target_id]) > 0

#####async def clear(self) -> None:
#####"""Clear all owned data structures for cleanup."""
#####async with self._lock:
Clear owned data (single source of truth)
#####self._targets.clear()
#####self._sessions.clear()
#####self._target_sessions.clear()
#####self._session_to_target.clear()

#####self.logger.info('[SessionManager] Cleared all owned data (targets, sessions, mappings)')

#####async def is_target_valid(self, target_id: TargetID) -> bool:
#####"""Check if a target is still valid and has active sessions.

#####Args:
#####target_id: Target ID to validate

#####Returns:
#####True if target is valid and has active sessions, False otherwise
#####"""
#####if target_id not in self._target_sessions:
#####return False
#####return len(self._target_sessions[target_id]) > 0

def get_target_id_from_session_id(self, session_id: SessionID) -> TargetID | None:
#####Look up which target a session belongs to.

#####Args:
#####session_id: The session ID to look up

#####Returns:
#####Target ID if found, None otherwise
#####"""
#####return self._session_to_target.get(session_id)

def get_target(self, target_id: TargetID) -> 'Target | None':

```

```

 """Get target from owned data.

 Args:
 target_id: Target ID to get

 Returns:
 Target object if found, None otherwise
 """
 return self._targets.get(target_id)

 def get_all_targets(self) -> dict[TargetID, 'Target']:
 """Get all targets (read-only access to owned data).

 Returns:
 Dict mapping target_id to Target objects
 """
 return self._targets

 def get_all_target_ids(self) -> list[TargetID]:
 """Get all target IDs from owned data.

 Returns:
 List of all target IDs
 """
 return list(self._targets.keys())

 def get_all_sessions(self) -> dict[SessionID, 'CDPSession']:
 """Get all sessions (read-only access to owned data).

 Returns:
 Dict mapping session_id to CDPSession objects
 """
 return self._sessions

 def get_session(self, session_id: SessionID) -> 'CDPSession | None':
 """Get session from owned data.

 Args:
 session_id: Session ID to get

 Returns:
 CDPSession object if found, None otherwise
 """
 return self._sessions.get(session_id)

 def get_all_sessions_for_target(self, target_id: TargetID) -> list['CDPSession']:
 """Get ALL sessions attached to a target from owned data.

 Args:
 target_id: Target ID to get sessions for

 Returns:
 List of all CDPSession objects for this target
 """
 session_ids = self._target_sessions.get(target_id, set())
 return [self._sessions[sid] for sid in session_ids if sid in self._sessions]

 def get_target_sessions_mapping(self) -> dict[TargetID, set[SessionID]]:
 """Get target->sessions mapping (read-only access).

 Returns:
 Dict mapping target_id to set of session_ids
 """
 return self._target_sessions

 def get_focused_target(self) -> 'Target | None':
 """Get the target that currently has agent focus.

 Convenience method that uses browser_session.agent_focus_target_id.

 Returns:
 Target object if agent has focus, None otherwise

```

```

 """
 if not self.browser_session.agent_focus_target_id:
 return None
 return self.get_target(self.browser_session.agent_focus_target_id)

 async def ensure_valid_focus(self, timeout: float = 3.0) -> bool:
 """Ensure agent_focus_target_id points to a valid, attached CDP session.

 If the focus target is stale (detached), this method waits for automatic recovery.
 Uses event-driven coordination instead of polling for efficiency.

 Args:
 timeout: Maximum time to wait for recovery in seconds (default: 3.0)

 Returns:
 True if focus is valid or successfully recovered, False if no focus or recovery failed
 """
 if not self.browser_session.agent_focus_target_id:
 # No focus at all - might be initial state or complete failure
 if self._recovery_in_progress and self._recovery_complete_event:
 # Recovery is happening, wait for it
 try:
 await asyncio.wait_for(self._recovery_complete_event.wait(), timeout=timeout)
 # Check again after recovery - simple existence check
 focus_id = self.browser_session.agent_focus_target_id
 return bool(focus_id and self._get_session_for_target(focus_id))
 except TimeoutError:
 self.logger.error(f'[SessionManager] ■ Timed out waiting for recovery after {timeout}s')
 return False
 return False

 ## Simple existence check - does the focused target have a session?
 cdp_session = self._get_session_for_target(self.browser_session.agent_focus_target_id)
 if cdp_session:
 # Session exists - validate it's still active
 is_valid = await self.validate_session(self.browser_session.agent_focus_target_id)
 if is_valid:
 return True

 ## Focus is stale - wait for recovery using event instead of polling
 stale_target_id = self.browser_session.agent_focus_target_id
 self.logger.warning(
 f'[SessionManager] ■ Stale agent_focus detected (target {stale_target_id[:8]} if stale_target_id else...)'
 f'waiting for recovery...'
)

 ## Check if recovery is already in progress
 if not self._recovery_in_progress:
 self.logger.warning(
 f'[SessionManager] ■ Recovery not in progress for stale focus! '
 'This indicates a bug - recovery should have been triggered.'
)
 return False

 ## Wait for recovery complete event (event-driven, not polling!)
 if self._recovery_complete_event:
 try:
 start_time = asyncio.get_event_loop().time()
 await asyncio.wait_for(self._recovery_complete_event.wait(), timeout=timeout)
 elapsed = asyncio.get_event_loop().time() - start_time

 ## Verify recovery succeeded - simple existence check
 focus_id = self.browser_session.agent_focus_target_id
 if focus_id and self._get_session_for_target(focus_id):
 self.logger.info(
 f'[SessionManager] ■ Agent focus recovered to {self.browser_session.agent_focus_target_id[:8]}...'
 f'after {elapsed * 1000:.0f}ms'
)
 return True
 else:
 self.logger.error(
 f'[SessionManager] ■ Recovery completed but focus still invalid after {elapsed * 1000:.0f}ms'
)

```

```

 return False

 except TimeoutError:
 self.logger.error(
 f'[SessionManager] ■ Recovery timed out after {timeout}s '
 f'(was: {stale_target_id[:8]} if stale_target_id else "None")...', '
 f'now: {self.browser_session.agent_focus_target_id[:8]} if self.browser_session.agent_focus_target_id else "None"')
 return False
 else:
 self.logger.error('[SessionManager] ■ Recovery event not initialized')
 return False

 async def _handle_target_attached(self, event: AttachedToTargetEvent) -> None:
 """Handle Target.attachedToTarget event.

 Called automatically by Chrome when a new target/session is created.
 This is the ONLY place where sessions are added to the pool.
 """
 target_id = event['targetInfo']['targetId']
 session_id = event['sessionId']
 target_type = event['targetInfo']['type']
 target_info = event['targetInfo']
 waiting_for_debugger = event.get('waitingForDebugger', False)

 self.logger.debug(
 f'[SessionManager] Target attached: {target_id[:8]}... (session={session_id[:8]}... , '
 f'type={target_type}, waitingForDebugger={waiting_for_debugger})'
)

 ## Defensive check: browser may be shutting down and _cdp_client_root could be None
 if self.browser_session._cdp_client_root is None:
 self.logger.debug(
 f'[SessionManager] Skipping target attach for {target_id[:8]}... - browser shutting down (no CDP client)'
)
 return

 ## Enable auto-attach for this session's children (do this FIRST, outside lock)
 try:
 await self.browser_session._cdp_client_root.send.Target.setAutoAttach(
 params={'autoAttach': True, 'waitForDebuggerOnStart': False, 'flatten': True}, session_id=session_id)
 except Exception as e:
 error_str = str(e)
 ## Expected for short-lived targets (workers, temp iframes) that detach before this executes
 if '-32001' not in error_str and 'Session with given id not found' not in error_str:
 self.logger.debug(f'[SessionManager] Auto-attach failed for {target_type}: {e}')

 async with self._lock:
 ## Track this session for the target
 if target_id not in self._target_sessions:
 self._target_sessions[target_id] = set()

 self._target_sessions[target_id].add(session_id)
 self._session_to_target[session_id] = target_id

 ## Create or update Target (source of truth for url/title)
 if target_id not in self._targets:
 from system.browser.session import Target

 target = Target(
 target_id=target_id,
 target_type=target_type,
 url=target_info.get('url', 'about:blank'),
 title=target_info.get('title', 'Unknown title'),
)
 self._targets[target_id] = target
 self.logger.debug(f'[SessionManager] Created target {target_id[:8]}... (type={target_type})')
 else:
 ## Update existing target info
 existing_target = self._targets[target_id]

```

```

 existing_target.url = target_info.get('url', existing_target.url)
 existing_target.title = target_info.get('title', existing_target.title)

 ## Create CDPSession (communication channel)
 from system.browser.session import CDPSession

 assert self.browser_session._cdp_client_root is not None, 'Root CDP client required'

 cdp_session = CDPSession(
 cdp_client=self.browser_session._cdp_client_root,
 target_id=target_id,
 session_id=session_id,
)

 ## Add to sessions dict
 self._sessions[session_id] = cdp_session

 ## If proxy auth is configured, enable Fetch auth handling on this session
 ## Avoids overwriting Target.attachedToTarget handlers elsewhere
 try:
 proxy_cfg = self.browser_session.browser_profile.proxy
 username = proxy_cfg.username if proxy_cfg else None
 password = proxy_cfg.password if proxy_cfg else None
 if username and password:
 await cdp_session.cdp_client.send.Fetch.enable(
 params={'handleAuthRequests': True},
 session_id=cdp_session.session_id,
)
 self.logger.debug(f'[SessionManager] Fetch.enable(handleAuthRequests=True) on session {session_id[:8]}')
 except Exception as e:
 self.logger.debug(f'[SessionManager] Fetch.enable on attached session failed: {type(e).__name__}: {e}')

 self.logger.debug(
 f'[SessionManager] Created session {session_id[:8]}... for target {target_id[:8]}... '
 f'(total sessions: {len(self._sessions)})'
)

 ## Enable lifecycle events and network monitoring for page targets
 if target_type in ('page', 'tab'):
 await self._enable_page_monitoring(cdp_session)

 ## Resume execution if waiting for debugger
 if waiting_for_debugger:
 try:
 assert self.browser_session._cdp_client_root is not None
 await self.browser_session._cdp_client_root.send.Runtime.runIfWaitingForDebugger(session_id=session_id)
 except Exception as e:
 self.logger.warning(f'[SessionManager] Failed to resume execution: {e}')

 async def _handle_target_info_changed(self, event: dict) -> None:
 """Handle Target.targetInfoChanged event.

 Updates target title/URL without polling getTargetInfo().
 Chrome fires this automatically when title or URL changes.
 """
 target_info = event.get('targetInfo', {})
 target_id = target_info.get('targetId')

 if not target_id:
 return

 async with self._lock:
 ## Update target if it exists (source of truth for url/title)
 if target_id in self._targets:
 target = self._targets[target_id]

 target.title = target_info.get('title', target.title)
 target.url = target_info.get('url', target.url)

 async def _handle_target_detached(self, event: DetachedFromTargetEvent) -> None:
 """Handle Target.detachedFromTarget event.
 Called automatically by Chrome when a target/session is destroyed.

```

```

 """This is the ONLY place where sessions are removed from the pool.
 """
 session_id = event['sessionId']
 target_id = event.get('targetId') # May be empty

 ## If targetId not in event, look it up via session mapping
 if not target_id:
 ## async with self._lock:
 ## target_id = self._session_to_target.get(session_id)

 if not target_id:
 ## self.logger.warning(f'[SessionManager] Session detached but target unknown (session={session_id[:8]})')
 ## return

 agent_focus_lost = False
 target_fully_removed = False
 target_type = None

 ## async with self._lock:
 ## # Remove this session from target's session set
 ## if target_id in self._target_sessions:
 ## self._target_sessions[target_id].discard(session_id)

 ## remaining_sessions = len(self._target_sessions[target_id])

 ## self.logger.debug(
 ## f'[SessionManager] Session detached: target={target_id[:8]}... '
 ## f'session={session_id[:8]}... (remaining={remaining_sessions})'
 ##)

 ## # Only remove target when NO sessions remain
 ## if remaining_sessions == 0:
 ## self.logger.debug(f'[SessionManager] No sessions remain for target {target_id[:8]}..., removing target')

 ## target_fully_removed = True

 ## # Check if agent_focus points to this target
 ## agent_focus_lost = self.browser_session.agent_focus_target_id == target_id

 ## # Immediately clear stale focus to prevent operations on detached target
 ## if agent_focus_lost:
 ## self.logger.debug(
 ## f'[SessionManager] Clearing stale agent_focus_target_id {target_id[:8]}... '
 ## f'to prevent operations on detached target'
 ##)
 ## self.browser_session.agent_focus_target_id = None

 ## # Get target type before removing (needed for TabClosedEvent dispatch)
 ## target = self._targets.get(target_id)
 ## target_type = target.target_type if target else None

 ## # Remove target (entity) from owned data
 ## if target_id in self._targets:
 ## self._targets.pop(target_id)
 ## self.logger.debug(
 ## f'[SessionManager] Removed target {target_id[:8]}... (remaining targets: {len(self._targets)})'
 ##)

 ## # Clean up tracking
 ## del self._target_sessions[target_id]
 ## else:
 ## # Target not tracked - already removed or never attached
 ## self.logger.debug(
 ## f'[SessionManager] Session detached from untracked target: target={target_id[:8]}... '
 ## f'session={session_id[:8]}... (target was already removed or attach event was missed)'
 ##)

 ## # Remove session from owned sessions dict
 ## if session_id in self._sessions:
 ## self._sessions.pop(session_id)
 ## self.logger.debug(
 ## f'[SessionManager] Removed session {session_id[:8]}... (remaining sessions: {len(self._sessions)})'

```



```

)

 ## Remove from reverse mapping
 if session_id in self._session_to_target:
 del self._session_to_target[session_id]

 ## Dispatch TabClosedEvent only for page/tab targets that are fully removed (not iframes/workers or pa
 if target_fully_removed:
 if target_type in ('page', 'tab'):
 from system.browser.events import TabClosedEvent

 self.browser_session.event_bus.dispatch(TabClosedEvent(target_id=target_id))
 self.logger.debug(f'[SessionManager] Dispatched TabClosedEvent for page target {target_id[:8]}...')
 elif target_type:
 self.logger.debug(
 f'[SessionManager] Target {target_id[:8]}... fully removed (type={target_type}) - not dispatching
)

 ## Auto-recover agent_focus outside the lock to avoid blocking other operations
 if agent_focus_lost:
 ## Create recovery task instead of awaiting directly - allows concurrent operations to wait on same r
 if not self._recovery_in_progress:
 self._recovery_task = create_task_with_error_handling(
 self._recover_agent_focus(target_id),
 name='recover_agent_focus',
 logger_instance=self.logger,
 suppress_exceptions=False,
)

 async def _recover_agent_focus(self, crashed_target_id: TargetID) -> None:
 """Auto-recover agent_focus when the focused target crashes/detaches.

 Uses recovery lock to prevent concurrent recovery attempts from creating multiple emergency tabs.
 Coordinates with ensure_valid_focus() via events for efficient waiting.

 Args:
 crashed_target_id: The target ID that was lost
 """
 try:
 ## Prevent concurrent recovery attempts
 async with self._recovery_lock:
 ## Set recovery state INSIDE lock to prevent race conditions
 if self._recovery_in_progress:
 self.logger.debug('[SessionManager] Recovery already in progress, waiting for it to complete')
 ## Wait for ongoing recovery instead of starting a new one
 if self._recovery_complete_event:
 try:
 await asyncio.wait_for(self._recovery_complete_event.wait(), timeout=5.0)
 except TimeoutError:
 self.logger.error('[SessionManager] Timed out waiting for ongoing recovery')
 return

 ## Set recovery state
 self._recovery_in_progress = True
 self._recovery_complete_event = asyncio.Event()

 if self.browser_session.cdp_client_root is None:
 self.logger.debug('[SessionManager] Skipping focus recovery - browser shutting down (no CDP client)
 return

 ## Check if another recovery already fixed agent_focus
 if self.browser_session.agent_focus_target_id and self.browser_session.agent_focus_target_id != cra
 self.logger.debug(
 f'[SessionManager] Agent focus already recovered by concurrent operation '
 f'(now: {self.browser_session.agent_focus_target_id[:8]}...), skipping recovery'
)
 return

 ## Note: agent_focus_target_id may already be None (cleared in _handle_target_detached)
 current_focus_desc = (
 f'{self.browser_session.agent_focus_target_id[:8]}...'
)
 if self.browser_session.agent_focus_target_id

```

```

 elif 'None (already cleared)'
)

 self.logger.warning(
 f'[SessionManager] Agent focus target {crashed_target_id[:8]}... detached! '
 f'Current focus: {current_focus_desc}. Auto-recovering by switching to another target...'
)

 # Perform recovery (outside lock to allow concurrent operations)
 # Try to find another valid page target
 page_targets = self.get_all_page_targets()

 new_target_id = None
 is_existing_tab = False

 if page_targets:
 # Switch to most recent page that's not the crashed one
 new_target_id = page_targets[-1].target_id
 is_existing_tab = True
 self.logger.info(f'[SessionManager] Switching agent_focus to existing tab {new_target_id[:8]}...')
 else:
 # No pages exist - create a new one
 self.logger.warning('[SessionManager] No tabs remain! Creating new tab for agent...')
 new_target_id = await self.browser_session._cdp_create_new_page('about:blank')
 self.logger.info(f'[SessionManager] Created new tab {new_target_id[:8]}... for agent')

 # Dispatch TabCreatedEvent so watchdogs can initialize
 from system.browser.events import TabCreatedEvent

 self.browser_session.event_bus.dispatch(TabCreatedEvent(url='about:blank', target_id=new_target_id))

 # Wait for CDP attach event to create session
 # Note: This polling is necessary - waiting for external Chrome CDP event
 # _handle_target_attached will add session to pool when Chrome fires attachedToTarget
 new_session = None
 for attempt in range(20): # Wait up to 2 seconds
 await asyncio.sleep(0.1)
 new_session = self._get_session_for_target(new_target_id)
 if new_session:
 break

 if new_session:
 self.browser_session.agent_focus_target_id = new_target_id
 self.logger.info(f'[SessionManager] Agent focus recovered: {new_target_id[:8]}...')

 # Visually activate the tab in browser (only for existing tabs)
 if is_existing_tab:
 try:
 assert self.browser_session._cdp_client_root is not None
 await self.browser_session._cdp_client_root.send(Target.activateTarget(params={'targetId': new_target_id}))
 self.logger.debug(f'[SessionManager] Activated tab {new_target_id[:8]}... in browser UI')
 except Exception as e:
 self.logger.debug(f'[SessionManager] Failed to activate tab visually: {e}')

 # Get target to access url (from owned data)
 target = self.get_target(new_target_id)
 target_url = target.url if target else 'about:blank'

 # Dispatch focus changed event
 from system.browser.events import AgentFocusChangedEvent

 self.browser_session.event_bus.dispatch(AgentFocusChangedEvent(target_id=new_target_id, url=target_url))
 return

 # Recovery failed - create emergency fallback tab
 self.logger.error(
 f'[SessionManager] Failed to get session for {new_target_id[:8]}... after 2s, creating emergency'
)

 fallback_target_id = await self.browser_session._cdp_create_new_page('about:blank')
 self.logger.warning(f'[SessionManager] Created emergency fallback tab {fallback_target_id[:8]}...')
 # Try one more time with fallback

```

```

Note: This polling is necessary - waiting for external Chrome CDP event
#####for _ in range(20):
await asyncio.sleep(0.1)
fallback_session = self._get_session_for_target(fallback_target_id)
if fallback_session:
self.browser_session.agent_focus_target_id = fallback_target_id
self.logger.warning(f'[SessionManager] ■ Agent focus set to emergency fallback: {fallback_target_id}')

#####from system.browser.events import AgentFocusChangedEvent, TabCreatedEvent

#####self.browser_session.event_bus.dispatch(TabCreatedEvent(url='about:blank', target_id=fallback_target_id))
#####self.browser_session.event_bus.dispatch(
AgentFocusChangedEvent(target_id=fallback_target_id, url='about:blank')
#####)
#####return

Complete failure - this should never happen
#####self.logger.critical(
f'[SessionManager] ■ CRITICAL: Failed to recover agent_focus even with fallback! Agent may be in broken state')

#####except Exception as e:
self.logger.error(f'[SessionManager] ■ Error during agent_focus recovery: {type(e).__name__}: {e}')
finally:
Always signal completion and reset recovery state
This allows all waiting operations to proceed (success or failure)
if self._recovery_complete_event:
self._recovery_complete_event.set()
self._recovery_in_progress = False
self._recovery_task = None
self.logger.debug(f'[SessionManager] Recovery state reset')

#####async def _initialize_existing_targets(self) -> None:
"""Discover and initialize all existing targets at startup.

Attaches to each target and initializes it SYNCHRONOUSLY.
Chrome will also fire attachedToTarget events, but _handle_target_attached() is
idempotent (checks if target already in pool), so duplicate handling is safe.

This eliminates race conditions - monitoring is guaranteed ready before navigation.
"""
cdp_client = self.browser_session.cdp_client_root
assert cdp_client is not None

Get all existing targets
targets_result = await cdp_client.send.Target.getTargets()
existing_targets = targets_result.get('targetInfos', [])

self.logger.debug(f'[SessionManager] Discovered {len(existing_targets)} existing targets')

Track target IDs for verification
target_ids_to_wait_for = []

Just attach to ALL existing targets - Chrome fires attachedToTarget events
The on_attached handler (via create_task) does ALL the work
for target in existing_targets:
target_id = target['targetId']
target_type = target.get('type', 'unknown')

try:
Just attach - event handler does everything
await cdp_client.send.Target.attachToTarget(params={'targetId': target_id, 'flatten': True})
target_ids_to_wait_for.append(target_id)
except Exception as e:
self.logger.debug(
f'[SessionManager] Failed to attach to existing target {target_id[:8]}... (type={target_type}): {e}')

Wait for event handlers to complete their work (they run via create_task)
Use event-driven approach instead of polling for better performance
ready_event = asyncio.Event()
async def check_all_ready():

```

```

 """Check if all sessions are ready and signal completion."""
 while True:
 ready_count = 0
 for tid in target_ids_to_wait_for:
 session = self._get_session_for_target(tid)
 if session:
 target = self._targets.get(tid)
 target_type = target.target_type if target else 'unknown'
 # For pages, verify monitoring is enabled
 if target_type in ('page', 'tab'):
 if hasattr(session, '_lifecycle_events') and session._lifecycle_events is not None:
 ready_count += 1
 else:
 # Non-page targets don't need monitoring
 ready_count += 1

 if ready_count == len(target_ids_to_wait_for):
 ready_event.set()
 return

 await asyncio.sleep(0.05)

 # Start checking in background
 check_task = create_task_with_error_handling(
 check_all_ready(), name='check_all_targets_ready', logger_instance=self.logger
)

 try:
 # Wait for completion with timeout
 await asyncio.wait_for(ready_event.wait(), timeout=2.0)
 except TimeoutError:
 # Timeout - count what's ready
 ready_count = 0
 for tid in target_ids_to_wait_for:
 session = self._get_session_for_target(tid)
 if session:
 target = self._targets.get(tid)
 target_type = target.target_type if target else 'unknown'
 # For pages, verify monitoring is enabled
 if target_type in ('page', 'tab'):
 if hasattr(session, '_lifecycle_events') and session._lifecycle_events is not None:
 ready_count += 1
 else:
 # Non-page targets don't need monitoring
 ready_count += 1
 self.logger.warning(
 '[SessionManager] Initialization timeout after 2.0s: {ready_count}/{len(target_ids_to_wait_for)}'
)
 finally:
 check_task.cancel()
 try:
 await check_task
 except asyncio.CancelledError:
 pass

 async def _enable_page_monitoring(self, cdp_session: 'CDPSession') -> None:
 """Enable lifecycle events and network monitoring for a page target.

 This is called once per page when it's created, avoiding handler accumulation.
 Registers a SINGLE lifecycle handler per session that stores events for navigations to consume.

 Args:
 cdp_session: The CDP session to enable monitoring on
 """
 try:
 # Enable Page domain first (required for lifecycle events)
 await cdp_session.cdp_client.send.Page.enable(session_id=cdp_session.session_id)

 # Enable lifecycle events (load, DOMContentLoaded, networkIdle, etc.)
 await cdp_session.cdp_client.send.Page.setLifecycleEventsEnabled(
 params={'enabled': True}, session_id=cdp_session.session_id
)

```

```

Enable network monitoring for networkIdle detection
#####await cdp_session.cdp_client.send.Network.enable(session_id=cdp_session.session_id)

Initialize lifecycle event storage for this session (thread-safe)
#####from collections import deque

#####cdp_session._lifecycle_events = deque(maxlen=50) # Keep last 50 events
#####cdp_session._lifecycle_lock = asyncio.Lock()

Register ONE handler per session that stores events
#####def on_lifecycle_event(event, session_id=None):
event_name = event.get('name', 'unknown')
event_loader_id = event.get('loaderId', 'none')

Find which target this session belongs to
#####target_id_from_event = None
#####if session_id:
target_id_from_event = self.get_target_id_from_session_id(session_id)

Check if this event is for our target
#####if target_id_from_event == cdp_session.target_id:
Store event for navigations to consume
event_data = {
'name': event_name,
'loaderId': event_loader_id,
'timestamp': asyncio.get_event_loop().time(),
}
Append is atomic in CPython
try:
cdp_session._lifecycle_events.append(event_data)
except Exception as e:
Only log errors, not every event
self.logger.error(f'[SessionManager] Failed to store lifecycle event: {e}')

Register the handler ONCE (this is the only place we register)
#####cdp_session.cdp_client.register.Page.lifecycleEvent(on_lifecycle_event)

#####except Exception as e:
Don't fail - target might be short-lived or already detached
error_str = str(e)
if '-32001' in error_str or 'Session with given id not found' in error_str:
self.logger.debug(
f'[SessionManager] Target {cdp_session.target_id[:8]}... detached before monitoring could be enabled'
)
else:
self.logger.warning(
f'[SessionManager] Failed to enable monitoring for target {cdp_session.target_id[:8]}...: {e}'
)

```

## ***.\system\browser\video\_recorder.py***

```

"""Video Recording Service for Browser Use Sessions."""

import base64
import io
import logging
import math
from pathlib import Path
from typing import Optional

from system.browser.profile import ViewportSize

try:
 import imageio.v2 as iio # type: ignore[import-not-found]
 import numpy as np # type: ignore[import-not-found]
 from imageio.core.format import Format # type: ignore[import-not-found]
 from PIL import Image
 IMAGEIO_AVAILABLE = True

```

```

except ImportError:
 IMAGEIO_AVAILABLE = False

logger = logging.getLogger(__name__)

def _get_padded_size(size: ViewportSize, macro_block_size: int = 16) -> ViewportSize:
 """Calculates the dimensions padded to the nearest multiple of macro_block_size."""
 width = int(math.ceil(size['width'] / macro_block_size)) * macro_block_size
 height = int(math.ceil(size['height'] / macro_block_size)) * macro_block_size
 return ViewportSize(width=width, height=height)

class VideoRecorderService:
 """
 Handles the video encoding process for a browser session using imageio.

 This service captures individual frames from the CDP screencast, decodes them,
 and appends them to a video file using a pip-installable ffmpeg backend.
 It automatically resizes frames to match the target video dimensions.
 """

 def __init__(self, output_path: Path, size: ViewportSize, framerate: int):
 """
 Initializes the video recorder.

 Args:
 output_path: The full path where the video will be saved.
 size: A ViewportSize object specifying the width and height of the video.
 framerate: The desired framerate for the output video.
 """
 self.output_path = output_path
 self.size = size
 self.framerate = framerate
 self._writer: Optional['Format.Writer'] = None
 self._is_active = False
 self.padded_size = _get_padded_size(self.size)

 def start(self) -> None:
 """
 Prepares and starts the video writer.

 If the required optional dependencies are not installed, this method will
 log an error and do nothing.
 """
 if not IMAGEIO_AVAILABLE:
 logger.error(
 'MP4 recording requires optional dependencies. Please install them with: pip install "browser-use[v'
)
 return

 try:
 self.output_path.parent.mkdir(parents=True, exist_ok=True)
 # The macro_block_size is set to None because we handle padding ourselves
 self._writer = iio.get_writer(
 str(self.output_path),
 fps=self.framerate,
 codec='libx264',
 quality=8, # A good balance of quality and file size (1-10 scale)
 pixelformat='yuv420p', # Ensures compatibility with most players
 macro_block_size=None,
)
 self._is_active = True
 logger.debug(f'Video recorder started. Output will be saved to {self.output_path}')
 except Exception as e:
 logger.error(f'Failed to initialize video writer: {e}')
 self._is_active = False

 def add_frame(self, frame_data_b64: str) -> None:
 """
 Decodes a base64-encoded PNG frame, resizes it, pads it to be codec-compatible,
 and appends it to the video.

```

```

 Args:
 frame_data_b64: A base64-encoded string of the PNG frame data.
 """
 if not self._is_active or not self._writer:
 return

 try:
 frame_bytes = base64.b64decode(frame_data_b64)

 # Use PIL to handle image processing in memory - much faster than spawning ffmpeg subprocess per frame
 with Image.open(io.BytesIO(frame_bytes)) as img:
 # 1. Resize if needed to target viewport size
 if img.size != (self.size['width'], self.size['height']):
 # Use BICUBIC as it's faster than LANCZOS and good enough for screen recordings
 img = img.resize((self.size['width'], self.size['height']), Image.Resampling.BICUBIC)

 # 2. Handle Padding (Macro block alignment for codecs)
 # Check if padding is actually needed
 if self.padded_size['width'] != self.size['width'] or self.padded_size['height'] != self.size['height']:
 new_img = Image.new('RGB', (self.padded_size['width'], self.padded_size['height']), (0, 0, 0))
 # Center the image
 x_offset = (self.padded_size['width'] - self.size['width']) // 2
 y_offset = (self.padded_size['height'] - self.size['height']) // 2
 new_img.paste(img, (x_offset, y_offset))
 img = new_img

 # 3. Convert to numpy array for imageio
 img_array = np.array(img)

 self._writer.append_data(img_array)
 except Exception as e:
 logger.warning(f'Could not process and add video frame: {e}')

 def stop_and_save(self) -> None:
 """
 Finalizes the video file by closing the writer.

 This method should be called when the recording session is complete.
 """
 if not self._is_active or not self._writer:
 return

 try:
 self._writer.close()
 logger.info(f'Video recording saved successfully to: {self.output_path}')
 except Exception as e:
 logger.error(f'Failed to finalize and save video: {e}')
 finally:
 self._is_active = False
 self._writer = None

```

## ***.\system\browser\views.py***

```

from dataclasses import dataclass, field
from typing import Any

from bubus import BaseEvent
from cdp_use.cdp.target import TargetID
from pydantic import AliasChoices, BaseModel, ConfigDict, Field, field_serializer

from system.dom.views import DOMInteractedElement, SerializedDOMState

Known placeholder image data for about:blank pages - a 4x4 white PNG
PLACEHOLDER_4PX_SCREENSHOT = (
 'iVBORw0KGgoAAAANSUHEUgAAAAQAAAECAIAAAAmkwkPAAAAFE1EQVR4nGP8//8/AwwwMSAB3BwAlm4DBfIlvkvAAAAASUVORK5CYJ'
)
Pydantic
class TabInfo(BaseModel):

```

```

 """Represents information about a browser tab"""

 model_config = ConfigDict(
 extra='forbid',
 validate_by_name=True,
 validate_by_alias=True,
 populate_by_name=True,
)

 # Original fields
 url: str
 title: str
 target_id: TargetID = Field(serialization_alias='tab_id', validation_alias=AliasChoices('tab_id', 'target_id'))
 parent_target_id: TargetID | None = Field(
 default=None, serialization_alias='parent_tab_id', validation_alias=AliasChoices('parent_tab_id', 'parent_target_id'))
 # parent page that contains this popup or cross-origin iframe

 @field_serializer('target_id')
 def serialize_target_id(self, target_id: TargetID, _info: Any) -> str:
 return target_id[-4:]

 @field_serializer('parent_target_id')
 def serialize_parent_target_id(self, parent_target_id: TargetID | None, _info: Any) -> str | None:
 return parent_target_id[-4:] if parent_target_id else None

class PageInfo(BaseModel):
 """Comprehensive page size and scroll information"""

 # Current viewport dimensions
 viewport_width: int
 viewport_height: int

 # Total page dimensions
 page_width: int
 page_height: int

 # Current scroll position
 scroll_x: int
 scroll_y: int

 # Calculated scroll information
 pixels_above: int
 pixels_below: int
 pixels_left: int
 pixels_right: int

 # Page statistics are now computed dynamically instead of stored

 @dataclass
 class NetworkRequest:
 """Information about a pending network request"""

 url: str
 method: str = 'GET'
 loading_duration_ms: float = 0.0 # How long this request has been loading (ms since request started, not yet finished)
 resource_type: str | None = None # e.g., 'Document', 'Stylesheet', 'Image', 'Script', 'XHR', 'Fetch'

 @dataclass
 class PaginationButton:
 """Information about a pagination button detected on the page"""

 button_type: str # 'next', 'prev', 'first', 'last', 'page_number'
 backend_node_id: int # Backend node ID for clicking
 text: str # Button text/label
 selector: str # XPath or other selector to locate the element
 is_disabled: bool = False # Whether the button appears disabled

 @dataclass

```



```

class BrowserStateSummary:
 """The summary of the browser's current state designed for an LLM to process"""

 # provided by SerializedDOMState:
 dom_state: SerializedDOMState

 url: str
 title: str
 tabs: list[TabInfo]
 screenshot: str | None = field(default=None, repr=False)
 page_info: PageInfo | None = None # Enhanced page information

 # Keep legacy fields for backward compatibility
 pixels_above: int = 0
 pixels_below: int = 0
 browser_errors: list[str] = field(default_factory=list)
 is_pdf_viewer: bool = False # Whether the current page is a PDF viewer
 recent_events: str | None = None # Text summary of recent browser events
 pending_network_requests: list[NetworkRequest] = field(default_factory=list) # Currently loading network requests
 pagination_buttons: list[PaginationButton] = field(default_factory=list) # Detected pagination buttons
 closed_popup_messages: list[str] = field(default_factory=list) # Messages from auto-closed JavaScript popups

 @dataclass
 class BrowserStateHistory:
 """The summary of the browser's state at a past point in time to use in LLM message history"""

 url: str
 title: str
 tabs: list[TabInfo]
 interacted_element: list[DOMInteractedElement | None] | list[None]
 screenshot_path: str | None = None

 def get_screenshot(self) -> str | None:
 """Load screenshot from disk and return as base64 string"""
 if not self.screenshot_path:
 return None

 import base64
 from pathlib import Path

 path_obj = Path(self.screenshot_path)
 if not path_obj.exists():
 return None

 try:
 with open(path_obj, 'rb') as f:
 screenshot_data = f.read()
 return base64.b64encode(screenshot_data).decode('utf-8')
 except Exception:
 return None

 def to_dict(self) -> dict[str, Any]:
 data = {}
 data['tabs'] = [tab.model_dump() for tab in self.tabs]
 data['screenshot_path'] = self.screenshot_path
 data['interacted_element'] = [el.to_dict() if el else None for el in self.interacted_element]
 data['url'] = self.url
 data['title'] = self.title
 return data

 class BrowserError(Exception):
 """Browser error with structured memory for LLM context management.

 This exception class provides separate memory contexts for browser actions:
 - short_term_memory: Immediate context shown once to the LLM for the next action
 - long_term_memory: Persistent error information stored across steps
 """

 message: str
 short_term_memory: str | None = None

```

```

 ■long_term_memory: str | None = None
 ■details: dict[str, Any] | None = None
 ■while_handling_event: BaseEvent[Any] | None = None

 ■def __init__(
 ■■self,
 ■■message: str,
 ■■short_term_memory: str | None = None,
 ■■long_term_memory: str | None = None,
 ■■details: dict[str, Any] | None = None,
 ■■event: BaseEvent[Any] | None = None,
 ■■):
 ■■"""Initialize a BrowserError with structured memory contexts.

 ■■Args:
 ■■■message: Technical error message for logging and debugging
 ■■■short_term_memory: Context shown once to LLM (e.g., available actions, options)
 ■■■long_term_memory: Persistent error info stored in agent memory
 ■■■details: Additional metadata for debugging
 ■■■event: The browser event that triggered this error
 ■■"""
 ■■self.message = message
 ■■self.short_term_memory = short_term_memory
 ■■self.long_term_memory = long_term_memory
 ■■self.details = details
 ■■self.while_handling_event = event
 ■■super().__init__(message)

 ■def __str__(self) -> str:
 ■■if self.details:
 ■■■return f'{self.message} ({self.details}) during: {self.while_handling_event}'
 ■■elif self.while_handling_event:
 ■■■return f'{self.message} (while handling: {self.while_handling_event})'
 ■■else:
 ■■■return self.message

class URLNotAllowedError(BrowserError):
 ■"""Error raised when a URL is not allowed"""

```

## ***.\system\browser\watchdog\_base.py***

```

 """Base watchdog class for browser monitoring components."""

import asyncio
import inspect
import time
from collections.abc import Iterable
from typing import Any, ClassVar

from bubus import BaseEvent, EventBus
from pydantic import BaseModel, ConfigDict, Field

from system.browser.session import BrowserSession

class BaseWatchdog(BaseModel):
 ■"""Base class for all browser watchdogs.

 ■Watchdogs monitor browser state and emit events based on changes.
 ■They automatically register event handlers based on method names.

 ■Handler methods should be named: on_EventTypeName(self, event: EventTypeName)
 ■"""

 ■model_config = ConfigDict(
 ■■arbitrary_types_allowed=True, # allow non-serializable objects like EventBus/BrowserSession in fields
 ■■extra='forbid', # dont allow implicit class/instance state, everything must be a properly typed Field

```

```

 validate_assignment=False, # avoid re-triggering __init__ / validators on values on every assignment
 revalidate_instances='never', # avoid re-triggering __init__ / validators and erasing private attrs
)

Class variables to statically define the list of events relevant to each watchdog
(not enforced, just to make it easier to understand the code and debug watchdogs at runtime)
LISTENS_TO: ClassVar[list[type[BaseEvent[Any]]]] = [] # Events this watchdog listens to
EMITS: ClassVar[list[type[BaseEvent[Any]]]] = [] # Events this watchdog emits

Core dependencies
event_bus: EventBus = Field()
browser_session: BrowserSession = Field()

Shared state that other watchdogs might need to access should not be defined on BrowserSession, not here!
Shared helper methods needed by other watchdogs should be defined on BrowserSession, not here!
Alternatively, expose some events on the watchdog to allow access to state/helpers via event_bus system

Private state internal to the watchdog can be defined like this on BaseWatchdog subclasses:
_screenshot_cache: dict[str, bytes] = PrivateAttr(default_factory=dict)
_browser_crash_watcher_task: asyncio.Task | None = PrivateAttr(default=None)
_cdp_download_tasks: WeakSet[asyncio.Task] = PrivateAttr(default_factory=WeakSet)
...

@property
def logger(self):
 """Get the logger from the browser session."""
 return self.browser_session.logger

@staticmethod
def attach_handler_to_session(browser_session: 'BrowserSession', event_class: type[BaseEvent[Any]], handler: Callable):
 """Attach a single event handler to a browser session.

 Args:
 browser_session: The browser session to attach to
 event_class: The event class to listen for
 handler: The handler method (must start with 'on_' and end with event type)
 """
 event_bus = browser_session.event_bus

 ## Validate handler naming convention
 assert hasattr(handler, '__name__'), 'Handler must have a __name__ attribute'
 assert handler.__name__.startswith('on_'), f'Handler {handler.__name__} must start with "on_"'
 assert handler.__name__.endswith(event_class.__name__), (
 f'Handler {handler.__name__} must end with event type {event_class.__name__}'
)

 ## Get the watchdog instance if this is a bound method
 watchdog_instance = getattr(handler, '__self__', None)
 watchdog_class_name = watchdog_instance.__class__.__name__ if watchdog_instance else 'Unknown'

 ## Events that should always run even when CDP is disconnected (lifecycle management)
 LIFECYCLE_EVENT_NAMES = frozenset(
 {
 'BrowserStartEvent',
 'BrowserStopEvent',
 'BrowserStoppedEvent',
 'BrowserLaunchEvent',
 'BrowserErrorEvent',
 'BrowserKillEvent',
 'BrowserReconnectingEvent',
 'BrowserReconnectedEvent',
 }
)

 ## Create a wrapper function with unique name to avoid duplicate handler warnings
 ## Capture handler by value to avoid closure issues
 def make_unique_handler(actual_handler):
 async def unique_handler(event):
 ## Circuit breaker: skip handler if CDP WebSocket is dead
 ## (prevents handlers from hanging on broken connections until timeout)
 ## Lifecycle events are exempt – they manage browser start/stop
 if event.event_type not in LIFECYCLE_EVENT_NAMES and not browser_session.is_cdp_connected:

```

```

If reconnection is in progress, wait for it instead of silently skipping
if browser_session.is_reconnecting:
wait_timeout = browser_session.RECONNECT_WAIT_TIMEOUT
browser_session.logger.debug(
f'■ [{watchdog_class_name}]{actual_handler.__name__}] ■ Waiting for reconnection ({wait_timeout}s)'
)
try:
await asyncio.wait_for(browser_session._reconnect_event.wait(), timeout=wait_timeout)
except TimeoutError:
raise ConnectionError(
f'■ [{watchdog_class_name}]{actual_handler.__name__}] '
f'Reconnection wait timed out after {wait_timeout}s'
)
After wait: check if reconnection actually succeeded
if not browser_session.is_cdp_connected:
raise ConnectionError(
f'■ [{watchdog_class_name}]{actual_handler.__name__}] Reconnection failed – CDP still not connected'
)
Reconnection succeeded – fall through to execute handler normally
else:
Not reconnecting – intentional stop, backward compat silent skip
browser_session.logger.debug(
f'■ [{watchdog_class_name}]{actual_handler.__name__}] ■ Skipped – CDP not connected'
)
return None

just for debug logging, not used for anything else
parent_event = event_bus.event_history.get(event.event_parent_id) if event.event_parent_id else None
grandparent_event = (
event_bus.event_history.get(parent_event.event_parent_id)
if parent_event and parent_event.event_parent_id
else None
)
parent = (
f'■ triggered by on_{parent_event.event_type}#{parent_event.event_id[-4:]}'
if parent_event
else '■ by Agent'
)
grandparent = (
(
f'■ under {grandparent_event.event_type}#{grandparent_event.event_id[-4:]}'
if grandparent_event
else '■ by Agent'
)
if parent_event
else ''
)
event_str = f'#{event.event_id[-4:]}'
time_start = time.time()
watchdog_and_handler_str = f'■ [{watchdog_class_name}]{actual_handler.__name__}({event_str})'.ljust(100)
browser_session.logger.debug(f'■ {watchdog_and_handler_str} ■ Starting... {parent} {grandparent}')

try:
EXECUTE THE EVENT HANDLER FUNCTION
result = await actual_handler(event)

if isinstance(result, Exception):
raise result

just for debug logging, not used for anything else
time_end = time.time()
time_elapsed = time_end - time_start
result_summary = '' if result is None else f'➡■ <{type(result).__name__}>'
parents_summary = f' {parent}'.replace('■ triggered by ', '■ returned to ').replace(
'■ by Agent', '■ returned to Agent'
)
browser_session.logger.debug(
f'■ {watchdog_and_handler_str} Succeeded ({time_elapsed:.2f}s){result_summary}{parents_summary}'
)
return result
except Exception as e:
time_end = time.time()

```

```

time_elapsed = time_end - time_start
original_error = e
browser_session.logger.error(
 f'■ {watchdog_and_handler_str} ■ Failed ({time_elapsed:.2f}s): {type(e).__name__}: {e}'
)

attempt to repair potentially crashed CDP session
try:
 if browser_session.agent_focus_target_id:
 # With event-driven sessions, Chrome will send detach/attach events
 # SessionManager handles pool cleanup automatically
 target_id_to_restore = browser_session.agent_focus_target_id
 browser_session.logger.debug(
 f'■ {watchdog_and_handler_str} ■ Session error detected, waiting for CDP events to sync (target_id={target_id_to_restore})'
)

 # Wait for new attach event to restore the session
 # This will raise ValueError if target doesn't re-attach
 await browser_session.get_or_create_cdp_session(target_id=target_id_to_restore, focus=True)
 else:
 # Try to get any available session
 await browser_session.get_or_create_cdp_session(target_id=None, focus=True)
 except Exception as sub_error:
 if 'ConnectionClosedError' in str(type(sub_error)) or 'ConnectionError' in str(type(sub_error)):
 browser_session.logger.error(
 f'■ {watchdog_and_handler_str} ■ Browser closed or CDP Connection disconnected by remote. {type(sub_error).__name__}: {sub_error}'
)
 raise
 else:
 browser_session.logger.error(
 f'■ {watchdog_and_handler_str} ■ CDP connected but failed to re-create CDP session after error: {sub_error}'
)

Always re-raise the original error with its traceback preserved
raise

return unique_handler

unique_handler = make_unique_handler(handler)
unique_handler.__name__ = f'{watchdog_class_name}.{handler.__name__}'

Check if this handler is already registered - throw error if duplicate
existing_handlers = event_bus.handlers.get(event_class.__name__, [])
handler_names = [getattr(h, '__name__', str(h)) for h in existing_handlers]

if unique_handler.__name__ in handler_names:
 raise RuntimeError(
 f'[{watchdog_class_name}] Duplicate handler registration attempted! '
 f'Handler {unique_handler.__name__} is already registered for {event_class.__name__}. '
 f'This likely means attach_to_session() was called multiple times.'
)

event_bus.on(event_class, unique_handler)

@staticmethod
def detach_handler_from_session(browser_session: 'BrowserSession', event_class: type[BaseEvent[Any]], handler: Callable):
 """Detach a single event handler from a browser session."""
 event_bus = browser_session.event_bus

 # Get the watchdog instance if this is a bound method
 watchdog_instance = getattr(handler, '__self__', None)
 watchdog_class_name = watchdog_instance.__class__.__name__ if watchdog_instance else 'Unknown'

 # Find and remove the handler by its unique name pattern
 unique_handler_name = f'{watchdog_class_name}.{handler.__name__}'

 existing_handlers = event_bus.handlers.get(event_class.__name__, [])
 for existing_handler in existing_handlers[:]: # copy list to allow modification during iteration
 if getattr(existing_handler, '__name__', '') == unique_handler_name:
 existing_handlers.remove(existing_handler)
 break
 def attach_to_session(self) -> None:

```

```

 """Attach watchdog to its browser session and start monitoring.

 This method handles event listener registration. The watchdog is already
 bound to a browser session via self.browser_session from initialization.
 """
 # Register event handlers automatically based on method names
 assert self.browser_session is not None, 'Root CDP client not initialized - browser may not be connected'

 from system.browser import events

 event_classes = {}
 for name in dir(events):
 obj = getattr(events, name)
 if inspect.isclass(obj) and issubclass(obj, BaseEvent) and obj is not BaseEvent:
 event_classes[name] = obj

 # Find all handler methods (on_EventName)
 registered_events = set()
 for method_name in dir(self):
 if method_name.startswith('on_') and callable(getattr(self, method_name)):
 # Extract event name from method name (on_EventName -> EventName)
 event_name = method_name[3:] # Remove 'on_' prefix

 if event_name in event_classes:
 event_class = event_classes[event_name]

 # ASSERTION: If LISTENS_TO is defined, enforce it
 if self.LISTENS_TO:
 assert event_class in self.LISTENS_TO, (
 f'[{self.__class__.__name__}] Handler {method_name} listens to {event_name} '
 f'but {event_name} is not declared in LISTENS_TO: {[e.__name__ for e in self.LISTENS_TO]}'
)

 handler = getattr(self, method_name)

 # Use the static helper to attach the handler
 self.attach_handler_to_session(self.browser_session, event_class, handler)
 registered_events.add(event_class)

 # ASSERTION: If LISTENS_TO is defined, ensure all declared events have handlers
 if self.LISTENS_TO:
 missing_handlers = set(self.LISTENS_TO) - registered_events
 if missing_handlers:
 missing_names = [e.__name__ for e in missing_handlers]
 self.logger.warning(
 f'[{self.__class__.__name__}] LISTENS_TO declares {missing_names} '
 f'but no handlers found (missing on_{", ".join(missing_names)} methods)'
)

 def __del__(self) -> None:
 """Clean up any running tasks during garbage collection."""

 # A BIT OF MAGIC: Cancel any private attributes that look like asyncio tasks
 try:
 for attr_name in dir(self):
 # e.g. _browser_crash_watcher_task = asyncio.Task
 if attr_name.startswith('_') and attr_name.endswith('_task'):
 try:
 task = getattr(self, attr_name)
 if hasattr(task, 'cancel') and callable(task.cancel) and not task.done():
 task.cancel()
 # self.logger.debug(f'[{self.__class__.__name__}] Cancelled {attr_name} during cleanup')
 except Exception:
 pass # Ignore errors during cleanup

 # e.g. _cdp_download_tasks = WeakSet[asyncio.Task] or list[asyncio.Task]
 if attr_name.startswith('_') and attr_name.endswith('_tasks') and isinstance(getattr(self, attr_name), (WeakSet, list)):
 for task in getattr(self, attr_name):
 try:
 if hasattr(task, 'cancel') and callable(task.cancel) and not task.done():
 task.cancel()
 # self.logger.debug(f'[{self.__class__.__name__}] Cancelled {attr_name} during cleanup')

```

```

 except Exception:
 pass # Ignore errors during cleanup
 except Exception as e:
 from system.utils import logger

 logger.error(f'Error during BrowserSession {self.__class__.__name__} garbage collection __del__')

```

## ***.\system\browser\\_\_init\_\_.py***

```

from typing import TYPE_CHECKING

Type stubs for lazy imports
if TYPE_CHECKING:
 from .profile import BrowserProfile, ProxySettings
 from .session import BrowserSession

Lazy imports mapping for heavy browser components
_LAZY_IMPORTS = {
 'ProxySettings': ('.profile', 'ProxySettings'),
 'BrowserProfile': ('.profile', 'BrowserProfile'),
 'BrowserSession': ('.session', 'BrowserSession'),
}

def __getattr__(name: str):
 """Lazy import mechanism for heavy browser components."""
 if name in _LAZY_IMPORTS:
 module_path, attr_name = _LAZY_IMPORTS[name]
 try:
 from importlib import import_module

 # Use relative import for current package
 full_module_path = f'system.browser{module_path}'
 module = import_module(full_module_path)
 attr = getattr(module, attr_name)
 # Cache the imported attribute in the module's globals
 module.globals()[name] = attr
 return attr
 except ImportError as e:
 raise ImportError(f'Failed to import {name} from {full_module_path}: {e}') from e

 raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

__all__ = [
 'BrowserSession',
 'BrowserProfile',
 'ProxySettings',
]

```

## ***.\system\browser\cloud\cloud.py***

```

"""Cloud browser service integration for browser-use.

This module provides integration with the browser-use cloud browser service.
When cloud_browser=True, it automatically creates a cloud browser instance
and returns the CDP URL for connection.
"""

import logging
import os

import httpx

```

```

from system.browser.cloud.views import CloudBrowserAuthError, CloudBrowserError, CloudBrowserResponse, CloudBrowserRequest
from system.sync.auth import CloudAuthConfig

```

```

logger = logging.getLogger(__name__)

```

```

class CloudBrowserClient:

```

```

 """Client for browser-use cloud browser service."""

```

```

 def __init__(self, api_base_url: str = 'https://api.browser-use.com'):

```

```

 self.api_base_url = api_base_url

```

```

 self.client = httpx.AsyncClient(timeout=30.0)

```

```

 self.current_session_id: str | None = None

```

```

 async def create_browser(

```

```

 self, request: CreateBrowserRequest, extra_headers: dict[str, str] | None = None

```

```

) -> CloudBrowserResponse:

```

```

 """Create a new cloud browser instance. For full docs refer to https://docs.cloud.browser-use.com/api"""

```

```

 Args:

```

```

 request: CreateBrowserRequest object containing browser creation parameters

```

```

 Returns:

```

```

 CloudBrowserResponse: Contains CDP URL and other browser info

```

```

 """

```

```

 url = f'{self.api_base_url}/api/v2/browsers'

```

```

 # Try to get API key from environment variable first, then auth config

```

```

 api_token = os.getenv('BROWSER_USE_API_KEY')

```

```

 if not api_token:

```

```

 # Fallback to auth config file

```

```

 try:

```

```

 auth_config = CloudAuthConfig.load_from_file()

```

```

 api_token = auth_config.api_token

```

```

 except Exception:

```

```

 pass

```

```

 if not api_token:

```

```

 raise CloudBrowserAuthError(

```

```

 'No authentication token found. Please set BROWSER_USE_API_KEY environment variable to authenticate'

```

```

)

```

```

 headers = {'X-Browser-Use-API-Key': api_token, 'Content-Type': 'application/json', **(extra_headers or {})}

```

```

 # Convert request to dictionary and exclude unset fields

```

```

 request_body = request.model_dump(exclude_unset=True)

```

```

 try:

```

```

 logger.info('Creating cloud browser instance...')

```

```

 response = await self.client.post(url, headers=headers, json=request_body)

```

```

 if response.status_code == 401:

```

```

 raise CloudBrowserAuthError(

```

```

 'Authentication failed. Please make sure you have set BROWSER_USE_API_KEY environment variable to authenticate'

```

```

)

```

```

 elif response.status_code == 403:

```

```

 raise CloudBrowserAuthError('Access forbidden. Please check your browser-use cloud subscription status')

```

```

 elif not response.is_success:

```

```

 error_msg = f'Failed to create cloud browser: HTTP {response.status_code}'

```

```

 try:

```

```

 error_data = response.json()

```

```

 if 'detail' in error_data:

```

```

 error_msg += f' - {error_data["detail"]}'

```

```

 except Exception:

```

```

 pass

```

```

 raise CloudBrowserError(error_msg)

```

```

 browser_data = response.json()

```

```

 browser_response = CloudBrowserResponse(**browser_data)

```

```

 # Store session ID for cleanup

```



```

self.current_session_id = browser_response.id

logger.info(f'Cloud browser created successfully: {browser_response.id}')
logger.debug(f'CDP URL: {browser_response.cdpUrl}')
Cyan color for live URL
logger.info(f'\033[36mLive URL: {browser_response.liveUrl}\033[0m')

return browser_response

except httpx.TimeoutException:
 raise CloudBrowserError('Timeout while creating cloud browser. Please try again.')
except httpx.ConnectError:
 raise CloudBrowserError('Failed to connect to cloud browser service. Please check your internet connection.')
except Exception as e:
 if isinstance(e, (CloudBrowserError, CloudBrowserAuthError)):
 raise
 raise CloudBrowserError(f'Unexpected error creating cloud browser: {e}')

async def stop_browser(
 self, session_id: str | None = None, extra_headers: dict[str, str] | None = None
) -> CloudBrowserResponse:
 """Stop a cloud browser session.

 Args:
 session_id: Session ID to stop. If None, uses current session.

 Returns:
 CloudBrowserResponse: Updated browser info with stopped status

 Raises:
 CloudBrowserAuthError: If authentication fails
 CloudBrowserError: If stopping fails
 """
 if session_id is None:
 session_id = self.current_session_id

 if not session_id:
 raise CloudBrowserError('No session ID provided and no current session available')

 url = f'{self.api_base_url}/api/v2/browsers/{session_id}'

 # Try to get API key from environment variable first, then auth config
 api_token = os.getenv('BROWSER_USE_API_KEY')

 if not api_token:
 # Fallback to auth config file
 try:
 auth_config = CloudAuthConfig.load_from_file()
 api_token = auth_config.api_token
 except Exception:
 pass

 if not api_token:
 raise CloudBrowserAuthError(
 'No authentication token found. Please set BROWSER_USE_API_KEY environment variable to authenticate'
)

 headers = {'X-Browser-Use-API-Key': api_token, 'Content-Type': 'application/json', **(extra_headers or {})}
 request_body = {'action': 'stop'}

 try:
 logger.info(f'Stopping cloud browser session: {session_id}')

 response = await self.client.patch(url, headers=headers, json=request_body)

 if response.status_code == 401:
 raise CloudBrowserAuthError(
 'Authentication failed. Please make sure you have set the BROWSER_USE_API_KEY environment variable'
)
 elif response.status_code == 404:
 # Session already stopped or doesn't exist - treating as error and clearing session

```

```

 logger.debug(f'Cloud browser session {session_id} not found (already stopped)')
 # Clear current session if it was this one
 if session_id == self.current_session_id:
 self.current_session_id = None
 raise CloudBrowserError(f'Cloud browser session {session_id} not found')
 elif not response.is_success:
 error_msg = f'Failed to stop cloud browser: HTTP {response.status_code}'
 try:
 error_data = response.json()
 if 'detail' in error_data:
 error_msg += f' - {error_data["detail"]}'
 except Exception:
 pass
 raise CloudBrowserError(error_msg)

 browser_data = response.json()
 browser_response = CloudBrowserResponse(**browser_data)

 # Clear current session if it was this one
 if session_id == self.current_session_id:
 self.current_session_id = None

 logger.info(f'Cloud browser session stopped: {browser_response.id}')
 logger.debug(f'Status: {browser_response.status}')

 return browser_response

 except httpx.TimeoutException:
 raise CloudBrowserError('Timeout while stopping cloud browser. Please try again.')
 except httpx.ConnectError:
 raise CloudBrowserError('Failed to connect to cloud browser service. Please check your internet connection.')
 except Exception as e:
 if isinstance(e, (CloudBrowserError, CloudBrowserAuthError)):
 raise
 raise CloudBrowserError(f'Unexpected error stopping cloud browser: {e}')

 async def close(self):
 """Close the HTTP client and cleanup any active sessions."""
 # Try to stop current session if active
 if self.current_session_id:
 try:
 await self.stop_browser()
 except Exception as e:
 logger.debug(f'Failed to stop cloud browser session during cleanup: {e}')

 await self.client.aclose()

```

## **`.system\browser\cloud\views.py`**

```

from typing import Literal
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field

ProxyCountryCode = (
 Literal[
 'us', # United States
 'uk', # United Kingdom
 'fr', # France
 'it', # Italy
 'jp', # Japan
 'au', # Australia
 'de', # Germany
 'fi', # Finland
 'ca', # Canada
 'in', # India
]
 | str
)

```

```

)

Browser session timeout limits (in minutes)
MAX_FREE_USER_SESSION_TIMEOUT = 15 # Free users limited to 15 minutes
MAX_PAID_USER_SESSION_TIMEOUT = 240 # Paid users can go up to 4 hours

Requests
class CreateBrowserRequest(BaseModel):
 """Request to create a cloud browser instance.

 Args:
 cloud_profile_id: The ID of the profile to use for the session
 cloud_proxy_country_code: Country code for proxy location
 cloud_timeout: The timeout for the session in minutes
 """

 model_config = ConfigDict(extra='forbid', populate_by_name=True)

 profile_id: UUID | str | None = Field(
 default=None,
 alias='cloud_profile_id',
 description='The ID of the profile to use for the session. Can be a UUID or a string of UUID.',
 title='Cloud Profile ID',
)

 proxy_country_code: ProxyCountryCode | None = Field(
 default=None,
 alias='cloud_proxy_country_code',
 description='Country code for proxy location.',
 title='Cloud Proxy Country Code',
)

 timeout: int | None = Field(
 ge=1,
 le=MAX_PAID_USER_SESSION_TIMEOUT,
 default=None,
 alias='cloud_timeout',
 description=f'The timeout for the session in minutes. Free users are limited to {MAX_FREE_USER_SESSION_TIMEOUT} minutes.',
 title='Cloud Timeout',
)

 CloudBrowserParams = CreateBrowserRequest # alias for easier readability

Responses
class CloudBrowserResponse(BaseModel):
 """Response from cloud browser API."""

 id: str
 status: str
 liveUrl: str = Field(alias='liveUrl')
 cdpUrl: str = Field(alias='cdpUrl')
 timeoutAt: str = Field(alias='timeoutAt')
 startedAt: str = Field(alias='startedAt')
 finishedAt: str | None = Field(alias='finishedAt', default=None)

Errors
class CloudBrowserError(Exception):
 """Exception raised when cloud browser operations fail."""

 pass

class CloudBrowserAuthError(CloudBrowserError):
 """Exception raised when cloud browser authentication fails."""

 pass

```

## **.\system\browser\watchdogs\aboutblank\_watchdog.py**

```
"""About:blank watchdog for managing about:blank tabs with DVD screensaver."""

from typing import TYPE_CHECKING, ClassVar

from bubus import BaseEvent
from cdp_use.cdp.target import TargetID
from pydantic import PrivateAttr

from system.browser.events import (
 ■AboutBlankDVDScreensaverShownEvent,
 ■BrowserStopEvent,
 ■BrowserStoppedEvent,
 ■CloseTabEvent,
 ■NavigateToUrlEvent,
 ■TabClosedEvent,
 ■TabCreatedEvent,
)
from system.browser.watchdog_base import BaseWatchdog

if TYPE_CHECKING:
 ■pass

class AboutBlankWatchdog(BaseWatchdog):
 ■"""Ensures there's always exactly one about:blank tab with DVD screensaver."""

 ■# Event contracts
 ■LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 ■■BrowserStopEvent,
 ■■BrowserStoppedEvent,
 ■■TabCreatedEvent,
 ■■TabClosedEvent,
 ■]
 ■EMITS: ClassVar[list[type[BaseEvent]]] = [
 ■■NavigateToUrlEvent,
 ■■CloseTabEvent,
 ■■AboutBlankDVDScreensaverShownEvent,
 ■]

 ■_stopping: bool = PrivateAttr(default=False)

 ■async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 ■■"""Handle browser stop request - stop creating new tabs."""
 ■■# logger.info('[AboutBlankWatchdog] Browser stop requested, stopping tab creation')
 ■■self._stopping = True

 ■async def on_BrowserStoppedEvent(self, event: BrowserStoppedEvent) -> None:
 ■■"""Handle browser stopped event."""
 ■■# logger.info('[AboutBlankWatchdog] Browser stopped')
 ■■self._stopping = True

 ■async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 ■■"""Check tabs when a new tab is created."""
 ■■# logger.debug(f'[AboutBlankWatchdog] ■ New tab created: {event.url}')

 ■■# If an about:blank tab was created, show DVD screensaver on all about:blank tabs
 ■■if event.url == 'about:blank':
 ■■■await self._show_dvd_screensaver_on_about_blank_tabs()

 ■async def on_TabClosedEvent(self, event: TabClosedEvent) -> None:
 ■■"""Check tabs when a tab is closed and proactively create about:blank if needed."""
 ■■# Don't create new tabs if browser is shutting down
 ■■if self._stopping:
 ■■■return

 ■■# Don't attempt CDP operations if the WebSocket is dead – dispatching
 ■■# NavigateToUrlEvent on a broken connection will hang until timeout
 ■■if not self.browser_session.is_cdp_connected:
```

```

 self.logger.debug('[AboutBlankWatchdog] CDP not connected, skipping tab recovery')
 return

 ## Check if we're about to close the last tab (event happens BEFORE tab closes)
 ## Use _cdp_get_all_pages for quick check without fetching titles
 page_targets = await self.browser_session._cdp_get_all_pages()
 if len(page_targets) < 1:
 self.logger.debug(
 '[AboutBlankWatchdog] Last tab closing, creating new about:blank tab to avoid closing entire browser'
)
 ## Create the animation tab since no tabs should remain
 navigate_event = self.event_bus.dispatch(NavigateToUrlEvent(url='about:blank', new_tab=True))
 await navigate_event
 ## Show DVD screensaver on the new tab
 await self._show_dvd_screensaver_on_about_blank_tabs()
 else:
 ## Multiple tabs exist, check after close
 await self._check_and_ensure_about_blank_tab()

 async def attach_to_target(self, target_id: TargetID) -> None:
 """AboutBlankWatchdog doesn't monitor individual targets."""
 pass

 async def _check_and_ensure_about_blank_tab(self) -> None:
 """Check current tabs and ensure exactly one about:blank tab with animation exists."""
 try:
 if not self.browser_session.is_cdp_connected:
 return

 ## For quick checks, just get page targets without titles to reduce noise
 page_targets = await self.browser_session._cdp_get_all_pages()

 ## If no tabs exist at all, create one to keep browser alive
 if len(page_targets) == 0:
 ## Only create a new tab if there are no tabs at all
 self.logger.debug('[AboutBlankWatchdog] No tabs exist, creating new about:blank DVD screensaver tab')
 navigate_event = self.event_bus.dispatch(NavigateToUrlEvent(url='about:blank', new_tab=True))
 await navigate_event
 ## Show DVD screensaver on the new tab
 await self._show_dvd_screensaver_on_about_blank_tabs()
 ## Otherwise there are tabs, don't create new ones to avoid interfering

 except Exception as e:
 self.logger.error(f'[AboutBlankWatchdog] Error ensuring about:blank tab: {e}')

 async def _show_dvd_screensaver_on_about_blank_tabs(self) -> None:
 """Show DVD screensaver on all about:blank pages only."""
 try:
 ## Get just the page targets without expensive title fetching
 page_targets = await self.browser_session._cdp_get_all_pages()
 browser_session_label = str(self.browser_session.id)[-4:]

 for page_target in page_targets:
 target_id = page_target['targetId']
 url = page_target['url']

 ## Only target about:blank pages specifically
 if url == 'about:blank':
 await self._show_dvd_screensaver_loading_animation_cdp(target_id, browser_session_label)

 except Exception as e:
 self.logger.error(f'[AboutBlankWatchdog] Error showing DVD screensaver: {e}')

 async def _show_dvd_screensaver_loading_animation_cdp(self, target_id: TargetID, browser_session_label: str) -> None:
 """
 Injects a DVD screensaver-style bouncing logo loading animation overlay into the target using CDP.
 This is used to visually indicate that the browser is setting up or waiting.
 """
 try:
 ## Create temporary session for this target without switching focus
 temp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)
 ## Inject the DVD screensaver script (from main branch with idempotency added)

```

```

#####script = f"""
#####(function(browser_session_label) {{
#####// Idempotency check
#####if (window.__dvdAnimationRunning) {{
#####return; // Already running, don't add another
#####}}
#####window.__dvdAnimationRunning = true;
#####
#####// Ensure document.body exists before proceeding
#####if (!document.body) {{
#####// Try again after DOM is ready
#####window.__dvdAnimationRunning = false; // Reset flag to retry
#####if (document.readyState === 'loading') {{
#####document.addEventListener('DOMContentLoaded', () => arguments.callee(browser_session_label));
#####}}
#####return;
#####}}
#####
#####const animated_title = `Starting agent ${browser_session_label}...`;
#####if (document.title === animated_title) {{
#####return; // already run on this tab, dont run again
#####}}
#####document.title = animated_title;

#####// Create the main overlay
#####const loadingOverlay = document.createElement('div');
#####loadingOverlay.id = 'pretty-loading-animation';
#####loadingOverlay.style.position = 'fixed';
#####loadingOverlay.style.top = '0';
#####loadingOverlay.style.left = '0';
#####loadingOverlay.style.width = '100vw';
#####loadingOverlay.style.height = '100vh';
#####loadingOverlay.style.background = '#000';
#####loadingOverlay.style.zIndex = '99999';
#####loadingOverlay.style.overflow = 'hidden';

#####// Create the image element
#####const img = document.createElement('img');
#####img.src = 'https://cf.brower-use.com/logo.svg';
#####img.alt = 'Browser-Use';
#####img.style.width = '200px';
#####img.style.height = 'auto';
#####img.style.position = 'absolute';
#####img.style.left = '0px';
#####img.style.top = '0px';
#####img.style.zIndex = '2';
#####img.style.opacity = '0.8';

#####loadingOverlay.appendChild(img);
#####document.body.appendChild(loadingOverlay);

#####// DVD screensaver bounce logic
#####let x = Math.random() * (window.innerWidth - 300);
#####let y = Math.random() * (window.innerHeight - 300);
#####let dx = 1.2 + Math.random() * 0.4; // px per frame
#####let dy = 1.2 + Math.random() * 0.4;
#####// Randomize direction
#####if (Math.random() > 0.5) dx = -dx;
#####if (Math.random() > 0.5) dy = -dy;

#####function animate() {{
#####const imgWidth = img.offsetWidth || 300;
#####const imgHeight = img.offsetHeight || 300;
#####x += dx;
#####y += dy;

#####if (x <= 0) {{
#####x = 0;
#####dx = Math.abs(dx);
#####}} else if (x + imgWidth >= window.innerWidth) {{
#####x = window.innerWidth - imgWidth;
#####dx = -Math.abs(dx);
#####}}

#####if (y <= 0) {{
#####y = 0;
#####dy = Math.abs(dy);
#####}} else if (y + imgHeight >= window.innerHeight) {{
#####y = window.innerHeight - imgHeight;
#####dy = -Math.abs(dy);
#####}}

#####requestAnimationFrame(animate);
#####}}

#####)();
#####"""

```

```

 }
 if (y <= 0) {
 y = 0;
 dy = Math.abs(dy);
 } else if (y + imgHeight >= window.innerHeight) {
 y = window.innerHeight - imgHeight;
 dy = -Math.abs(dy);
 }

 img.style.left = `${x}px`;
 img.style.top = `${y}px`;

 requestAnimationFrame(animate);
 }
 animate();

 // Responsive: update bounds on resize
 window.addEventListener('resize', () => {
 x = Math.min(x, window.innerWidth - img.offsetWidth);
 y = Math.min(y, window.innerHeight - img.offsetHeight);
 });

 // Add a little CSS for smoothness
 const style = document.createElement('style');
 style.textContent = `
 #pretty-loading-animation {
 /*backdrop-filter: blur(2px) brightness(0.9);*/
 }
 #pretty-loading-animation img {
 user-select: none;
 pointer-events: none;
 }
 `;
 document.head.appendChild(style);
 })(`{browser_session_label}`);
 """

 await temp_session.cdp_client.send(Runtime.evaluate(params={'expression': script}, session_id=temp_s

 # No need to detach - session is cached

 # Dispatch event
 self.event_bus.dispatch(AboutBlankDVDScreensaverShownEvent(target_id=target_id))

 except Exception as e:
 self.logger.error(f'[AboutBlankWatchdog] Error injecting DVD screensaver: {e}')

```

## **`.\system\browser\watchdogs\captcha_watchdog.py`**

```

"""Captcha solver watchdog – monitors captcha events from the browser proxy.

Listens for BrowserUse.captchaSolverStarted/Finished CDP events and exposes a
wait_if_captcha_solving() method that the agent step loop uses to block until
a captcha is resolved (with a configurable timeout).

NOTE: Only a single captcha solve is tracked at a time. If multiple captchas
overlap (e.g. rapid successive navigations), only the latest one is tracked and
earlier in-flight waits may return prematurely.
"""

import asyncio
from dataclasses import dataclass
from typing import Any, ClassVar, Literal

from bubus import BaseEvent
from cdp_use.cdp.browseruse.events import CaptchaSolverFinishedEvent as CDPCaptchaSolverFinishedEvent
from cdp_use.cdp.browseruse.events import CaptchaSolverStartedEvent as CDPCaptchaSolverStartedEvent
from pydantic import PrivateAttr

```

```

from system.browser.events import (
 ■BrowserConnectedEvent,
 ■BrowserStoppedEvent,
 ■CaptchaSolverFinishedEvent,
 ■CaptchaSolverStartedEvent,
 ■_get_timeout,
)
from system.browser.watchdog_base import BaseWatchdog

CaptchaResultType = Literal['success', 'failed', 'timeout', 'unknown']

@dataclass
class CaptchaWaitResult:
 ■"""Result returned by wait_if_captcha_solving() when the agent had to wait."""

 ■waited: bool
 ■vendor: str
 ■url: str
 ■duration_ms: int
 ■result: CaptchaResultType

class CaptchaWatchdog(BaseWatchdog):
 ■"""Monitors captcha solver events from the browser proxy.

 ■When the proxy detects a CAPTCHA and starts solving it, a CDP event
 ■`BrowserUse.captchaSolverStarted` is sent over the WebSocket. This
 ■watchdog catches that event and blocks the agent's step loop (via
 ■`wait_if_captcha_solving`) until `BrowserUse.captchaSolverFinished`
 ■arrives or the configurable timeout expires.
 ■"""

 ■# Event contracts
 ■LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 ■ ■BrowserConnectedEvent,
 ■ ■BrowserStoppedEvent,
 ■]
 ■EMITS: ClassVar[list[type[BaseEvent]]] = [
 ■ ■CaptchaSolverStartedEvent,
 ■ ■CaptchaSolverFinishedEvent,
 ■]

 ■# --- private state ---
 ■_captcha_solving: bool = PrivateAttr(default=False)
 ■_captcha_solved_event: asyncio.Event = PrivateAttr(default_factory=asyncio.Event)
 ■_captcha_info: dict[str, Any] = PrivateAttr(default_factory=dict)
 ■_captcha_result: CaptchaResultType = PrivateAttr(default='unknown')
 ■_captcha_duration_ms: int = PrivateAttr(default=0)
 ■_cdp_handlers_registered: bool = PrivateAttr(default=False)

 ■def model_post_init(self, __context: Any) -> None:
 ■ ■# Start in "not blocked" state so callers never wait when there is no captcha.
 ■ ■self._captcha_solved_event.set()

 ■# -----
 ■# Event handlers
 ■# -----

 ■async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 ■ ■"""Register CDP event handlers for BrowserUse captcha solver events."""
 ■ ■if self._cdp_handlers_registered:
 ■ ■self.logger.debug('CaptchaWatchdog: CDP handlers already registered, skipping')
 ■ ■return

 ■ ■cdp_client = self.browser_session.cdp_client

 ■ ■def _on_captcha_started(event_data: CDPCaptchaSolverStartedEvent, session_id: str | None) -> None:
 ■ ■try:
 ■ ■self._captcha_solving = True
 ■ ■self._captcha_result = 'unknown'
 ■ ■self._captcha_duration_ms = 0

```



```

#####self._captcha_info = {
#####'vendor': event_data.get('vendor', 'unknown'),
#####'url': event_data.get('url', ''),
#####'targetId': event_data.get('targetId', ''),
#####'startedAt': event_data.get('startedAt', 0),
#####}
Block any waiter
#####self._captcha_solved_event.clear()

#####vendor = self._captcha_info['vendor']
#####url = self._captcha_info['url']
#####self.logger.info(f'■ Captcha solving started: {vendor} on {url}')

#####self.event_bus.dispatch(
CaptchaSolverStartedEvent(
target_id=event_data.get('targetId', ''),
vendor=vendor,
url=url,
started_at=event_data.get('startedAt', 0),
)
#####)
#####except Exception:
self.logger.exception('Error handling captchaSolverStarted CDP event')
Ensure consistent state: unblock any waiter
self._captcha_solving = False
self._captcha_solved_event.set()

#####def _on_captcha_finished(event_data: CDPCaptchaSolverFinishedEvent, session_id: str | None) -> None:
try:
success = event_data.get('success', False)
self._captcha_solving = False
self._captcha_duration_ms = event_data.get('durationMs', 0)
self._captcha_result = 'success' if success else 'failed'

vendor = event_data.get('vendor', self._captcha_info.get('vendor', 'unknown'))
url = event_data.get('url', self._captcha_info.get('url', ''))
duration_s = self._captcha_duration_ms / 1000

self.logger.info(f'■ Captcha solving finished: {self._captcha_result} – {vendor} on {url} ({duration_s}s)')

Unblock any waiter
self._captcha_solved_event.set()

self.event_bus.dispatch(
CaptchaSolverFinishedEvent(
target_id=event_data.get('targetId', ''),
vendor=vendor,
url=url,
duration_ms=self._captcha_duration_ms,
finished_at=event_data.get('finishedAt', 0),
success=success,
)
)
except Exception:
self.logger.exception('Error handling captchaSolverFinished CDP event')
Ensure consistent state: unblock any waiter
self._captcha_solving = False
self._captcha_solved_event.set()

#####cdp_client.register.BrowserUse.captchaSolverStarted(_on_captcha_started)
#####cdp_client.register.BrowserUse.captchaSolverFinished(_on_captcha_finished)
#####self._cdp_handlers_registered = True
#####self.logger.debug('■ CaptchaWatchdog: registered CDP event handlers for BrowserUse captcha events')

#####async def on_BrowserStoppedEvent(self, event: BrowserStoppedEvent) -> None:
"""Clear captcha state when the browser disconnects so nothing hangs."""
self._captcha_solving = False
self._captcha_result = 'unknown'
self._captcha_duration_ms = 0
self._captcha_info = {}
self._captcha_solved_event.set()
self._cdp_handlers_registered = False

```

```

■# -----
■# Public API
■# -----

■async def wait_if_captcha_solving(self, timeout: float | None = None) -> CaptchaWaitResult | None:
■ """Wait if a captcha is currently being solved.

■ Returns:
■ - `None` if no captcha was in progress.
■ - A `CaptchaWaitResult` with the outcome otherwise.
■ """
■ if not self._captcha_solving:
■ return None

■ if timeout is None:
■ timeout = _get_timeout('TIMEOUT_CaptchaSolverWait', 120.0)
■ assert timeout is not None
■ vendor = self._captcha_info.get('vendor', 'unknown')
■ url = self._captcha_info.get('url', '')
■ self.logger.info(f'■ Waiting for {vendor} captcha to be solved on {url} (timeout={timeout}s)...')

■ try:
■ await asyncio.wait_for(self._captcha_solved_event.wait(), timeout=timeout)
■ return CaptchaWaitResult(
■ waited=True,
■ vendor=vendor,
■ url=url,
■ duration_ms=self._captcha_duration_ms,
■ result=self._captcha_result,
■)
■ except TimeoutError:
■ # Timed out – unblock and report
■ self._captcha_solving = False
■ self._captcha_solved_event.set()
■ self.logger.warning(f'■ Captcha wait timed out after {timeout}s for {vendor} on {url}')
■ return CaptchaWaitResult(
■ waited=True,
■ vendor=vendor,
■ url=url,
■ duration_ms=int(timeout * 1000),
■ result='timeout',
■)

```

## **`.system\browser\watchdogs\crash_watchdog.py`**

```

"""Browser watchdog for monitoring crashes and network timeouts using CDP."""

import asyncio
import time
from typing import TYPE_CHECKING, ClassVar

import psutil
from bubus import BaseEvent
from cdp_use.cdp.target import SessionID, TargetID
from cdp_use.cdp.target.events import TargetCrashedEvent
from pydantic import Field, PrivateAttr

from system.browser.events import (
 ■BrowserConnectedEvent,
 ■BrowserErrorEvent,
 ■BrowserStoppedEvent,
 ■TabClosedEvent,
 ■TabCreatedEvent,
)
from system.browser.watchdog_base import BaseWatchdog
from system.utils import create_task_with_error_handling

if TYPE_CHECKING:

```

■pass

```
class NetworkRequestTracker:
 """Tracks ongoing network requests."""

 def __init__(self, request_id: str, start_time: float, url: str, method: str, resource_type: str | None):
 self.request_id = request_id
 self.start_time = start_time
 self.url = url
 self.method = method
 self.resource_type = resource_type

class CrashWatchdog(BaseWatchdog):
 """Monitors browser health for crashes and network timeouts using CDP."""

 # Event contracts
 LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 BrowserConnectedEvent,
 BrowserStoppedEvent,
 TabCreatedEvent,
 TabClosedEvent,
]
 EMITS: ClassVar[list[type[BaseEvent]]] = [BrowserErrorEvent]

 # Configuration
 network_timeout_seconds: float = Field(default=10.0)
 check_interval_seconds: float = Field(default=5.0) # Reduced frequency to reduce noise

 # Private state
 _active_requests: dict[str, NetworkRequestTracker] = PrivateAttr(default_factory=dict)
 _monitoring_task: asyncio.Task | None = PrivateAttr(default=None)
 _last_responsive_checks: dict[str, float] = PrivateAttr(default_factory=dict) # target_url -> timestamp
 _cdp_event_tasks: set[asyncio.Task] = PrivateAttr(default_factory=set) # Track CDP event handler tasks
 _targets_with_listeners: set[str] = PrivateAttr(default_factory=set) # Track targets that already have listeners

 async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 """Start monitoring when browser is connected."""
 # logger.debug('[CrashWatchdog] Browser connected event received, beginning monitoring')

 create_task_with_error_handling(
 self._start_monitoring(), name='start_crash_monitoring', logger_instance=self.logger, suppress_exceptions=True
)
 # logger.debug(f'[CrashWatchdog] Monitoring task started: {self._monitoring_task} and not self._monitoring_task')

 async def on_BrowserStoppedEvent(self, event: BrowserStoppedEvent) -> None:
 """Stop monitoring when browser stops."""
 # logger.debug('[CrashWatchdog] Browser stopped, ending monitoring')
 await self._stop_monitoring()

 async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 """Attach to new tab."""
 assert self.browser_session.agent_focus_target_id is not None, 'No current target ID'
 await self.attach_to_target(self.browser_session.agent_focus_target_id)

 async def on_TabClosedEvent(self, event: TabClosedEvent) -> None:
 """Clean up tracking when tab closes."""
 # Remove target from listener tracking to prevent memory leak
 if event.target_id in self._targets_with_listeners:
 self._targets_with_listeners.discard(event.target_id)
 self.logger.debug(f'[CrashWatchdog] Removed target {event.target_id[:8]}... from monitoring')

 async def attach_to_target(self, target_id: TargetID) -> None:
 """Set up crash monitoring for a specific target using CDP."""
 try:
 # Check if we already have listeners for this target
 if target_id in self._targets_with_listeners:
 self.logger.debug(f'[CrashWatchdog] Event listeners already exist for target: {target_id[:8]}...')
 return

 # Create temporary session for monitoring without switching focus
```

```

■■■cdp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)

■■■# Register crash event handler
■■■def on_target_crashed(event: TargetCrashedEvent, session_id: SessionID | None = None):
■■■ # Create and track the task
■■■ task = create_task_with_error_handling(
■■■ self._on_target_crash_cdp(target_id),
■■■ name='handle_target_crash',
■■■ logger_instance=self.logger,
■■■ suppress_exceptions=True,
■■■)
■■■ self._cdp_event_tasks.add(task)
■■■ # Remove from set when done
■■■ task.add_done_callback(lambda t: self._cdp_event_tasks.discard(t))

■■■cdp_session.cdp_client.register.Target.targetCrashed(on_target_crashed)

■■■# Track that we've added listeners to this target
■■■self._targets_with_listeners.add(target_id)

■■■target = self.browser_session.session_manager.get_target(target_id)
■■■if target:
■■■ self.logger.debug(f'[CrashWatchdog] Added target to monitoring: {target.url}')

■■■except Exception as e:
■■■ self.logger.warning(f'[CrashWatchdog] Failed to attach to target {target_id}: {e}')

■■■async def _on_request_cdp(self, event: dict) -> None:
■■■ """Track new network request from CDP event."""
■■■ request_id = event.get('requestId', '')
■■■ request = event.get('request', {})

■■■ self._active_requests[request_id] = NetworkRequestTracker(
■■■ request_id=request_id,
■■■ start_time=time.time(),
■■■ url=request.get('url', ''),
■■■ method=request.get('method', ''),
■■■ resource_type=event.get('type'),
■■■)
■■■ # logger.debug(f'[CrashWatchdog] Tracking request: {request.get("method", "")} {request.get("url", ""')}

■■■def _on_response_cdp(self, event: dict) -> None:
■■■ """Remove request from tracking on response."""
■■■ request_id = event.get('requestId', '')
■■■ if request_id in self._active_requests:
■■■ elapsed = time.time() - self._active_requests[request_id].start_time
■■■ response = event.get('response', {})
■■■ self.logger.debug(f'[CrashWatchdog] Request completed in {elapsed:.2f}s: {response.get("url", "")[:50]}')
■■■ # Don't remove yet - wait for loadingFinished

■■■def _on_request_failed_cdp(self, event: dict) -> None:
■■■ """Remove request from tracking on failure."""
■■■ request_id = event.get('requestId', '')
■■■ if request_id in self._active_requests:
■■■ elapsed = time.time() - self._active_requests[request_id].start_time
■■■ self.logger.debug(
■■■ f'[CrashWatchdog] Request failed after {elapsed:.2f}s: {self._active_requests[request_id].url[:50]}')
■■■ del self._active_requests[request_id]

■■■def _on_request_finished_cdp(self, event: dict) -> None:
■■■ """Remove request from tracking when loading is finished."""
■■■ request_id = event.get('requestId', '')
■■■ self._active_requests.pop(request_id, None)

■■■async def _on_target_crash_cdp(self, target_id: TargetID) -> None:
■■■ """Handle target crash detected via CDP."""
■■■ self.logger.debug(f'[CrashWatchdog] Target crashed: {target_id[:8]}..., waiting for detach event')

■■■ target = self.browser_session.session_manager.get_target(target_id)

■■■ is_agent_focus = (

```

```

 target
 and self.browser_session.agent_focus_target_id
 and target.target_id == self.browser_session.agent_focus_target_id
)

if is_agent_focus:
 self.logger.error(f'[CrashWatchdog] Agent focus tab crashed: {target.url} (SessionManager will aut

 # Emit browser error event
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='TargetCrash',
 message=f'Target crashed: {target_id}',
 details={
 'url': target.url if target else None,
 'target_id': target_id,
 'was_agent_focus': is_agent_focus,
 },
)
)

 async def _start_monitoring(self) -> None:
 """Start the monitoring loop."""
 assert self.browser_session.cdp_client is not None, 'Root CDP client not initialized - browser may no

 if self._monitoring_task and not self._monitoring_task.done():
 # logger.info('[CrashWatchdog] Monitoring already running')
 return

 self._monitoring_task = create_task_with_error_handling(
 self._monitoring_loop(), name='crash_monitoring_loop', logger_instance=self.logger, suppress_excepti
)
 # logger.debug('[CrashWatchdog] Monitoring loop created and started')

 async def _stop_monitoring(self) -> None:
 """Stop the monitoring loop and clean up all tracking."""
 if self._monitoring_task and not self._monitoring_task.done():
 self._monitoring_task.cancel()
 try:
 await self._monitoring_task
 except asyncio.CancelledError:
 pass
 self.logger.debug('[CrashWatchdog] Monitoring loop stopped')

 # Cancel all CDP event handler tasks
 for task in list(self._cdp_event_tasks):
 if not task.done():
 task.cancel()
 # Wait for all tasks to complete cancellation
 if self._cdp_event_tasks:
 await asyncio.gather(*self._cdp_event_tasks, return_exceptions=True)
 self._cdp_event_tasks.clear()

 # Clear all tracking
 self._active_requests.clear()
 self._targets_with_listeners.clear()
 self._last_responsive_checks.clear()

 async def _monitoring_loop(self) -> None:
 """Main monitoring loop."""
 await asyncio.sleep(10) # give browser time to start up and load the first page after first LLM call
 while True:
 try:
 await self._check_network_timeouts()
 await self._check_browser_health()
 await asyncio.sleep(self.check_interval_seconds)
 except asyncio.CancelledError:
 break
 except Exception as e:
 self.logger.error(f'[CrashWatchdog] Error in monitoring loop: {e}')

 async def _check_network_timeouts(self) -> None:

```

```

 """Check for network requests exceeding timeout."""
 current_time = time.time()
 timed_out_requests = []

 # Debug logging
 if self._active_requests:
 self.logger.debug(
 f'[CrashWatchdog] Checking {len(self._active_requests)} active requests for timeouts (threshold: {self.network_timeout_seconds}s)'
)

 for request_id, tracker in self._active_requests.items():
 elapsed = current_time - tracker.start_time
 self.logger.debug(
 f'[CrashWatchdog] Request {tracker.url[:30]}... elapsed: {elapsed:.1f}s, timeout: {self.network_timeout_seconds}s'
)
 if elapsed >= self.network_timeout_seconds:
 timed_out_requests.append((request_id, tracker))

 # Emit events for timed out requests
 for request_id, tracker in timed_out_requests:
 self.logger.warning(
 f'[CrashWatchdog] Network request timeout after {self.network_timeout_seconds}s: '
 f'{tracker.method} {tracker.url[:100]}...'
)

 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='NetworkTimeout',
 message=f'Network request timed out after {self.network_timeout_seconds}s',
 details={
 'url': tracker.url,
 'method': tracker.method,
 'resource_type': tracker.resource_type,
 'elapsed_seconds': current_time - tracker.start_time,
 },
)
)

 # Remove from tracking
 del self._active_requests[request_id]

 async def _check_browser_health(self) -> None:
 """Check if browser and targets are still responsive."""

 try:
 self.logger.debug(f'[CrashWatchdog] Checking browser health for target {self.browser_session.agent_focus_target_id}')
 cdp_session = await self.browser_session.get_or_create_cdp_session()

 for target in self.browser_session.session_manager.get_all_page_targets():
 if self.is_new_tab_page(target.url) and target.url != 'about:blank':
 self.logger.debug(f'[CrashWatchdog] Redirecting chrome://new-tab-page/ to about:blank {target.url}')
 cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=target.target_id)
 await cdp_session.cdp_client.send.Page.navigate(
 params={'url': 'about:blank'}, session_id=cdp_session.session_id
)

 # Quick ping to check if session is alive
 self.logger.debug(f'[CrashWatchdog] Attempting to run simple JS test expression in session {cdp_session.session_id}')
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Runtime.evaluate(params={'expression': '1+1'}, session_id=cdp_session.session_id),
 timeout=1.0,
)
 self.logger.debug(
 f'[CrashWatchdog] Browser health check passed for target {self.browser_session.agent_focus_target_id}'
)
 except Exception as e:
 self.logger.error(
 f'[CrashWatchdog] ■ Crashed/unresponsive session detected for target {self.browser_session.agent_focus_target_id}: {e} (Chrome will send detach event, SessionManager will auto-recover)'
)

 # Check browser process if we have PID

```

```

 if self.browser_session._local_browser_watchdog and (proc := self.browser_session._local_browser_watchdog):
 try:
 if proc.status() in (psutil.STATUS_ZOMBIE, psutil.STATUS_DEAD):
 self.logger.error(f'[CrashWatchdog] Browser process {proc.pid} has crashed')

 # Browser process crashed - SessionManager will clean up via detach events
 # Just dispatch error event and stop monitoring
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='BrowserProcessCrashed',
 message=f'Browser process {proc.pid} has crashed',
 details={'pid': proc.pid, 'status': proc.status()},
)
)

 self.logger.warning('[CrashWatchdog] Browser process dead - stopping health monitoring')
 await self._stop_monitoring()
 return
 except Exception:
 pass # psutil not available or process doesn't exist

 @staticmethod
 def _is_new_tab_page(url: str) -> bool:
 """Check if URL is a new tab page."""
 return url in ['about:blank', 'chrome://new-tab-page/', 'chrome://newtab/']

```

## **`.system\browser\watchdogs\default_action_watchdog.py`**

```

"""Default browser action handlers using CDP."""

import asyncio
import json
import os

from cdp_use.cdp.input.commands import DispatchKeyEventParameters

from system.actor.utils import get_key_info
from system.browser.events import (
 ClickCoordinateEvent,
 ClickElementEvent,
 GetDropdownOptionsEvent,
 GoBackEvent,
 GoForwardEvent,
 RefreshEvent,
 ScrollEvent,
 ScrollToTextEvent,
 SelectDropdownOptionEvent,
 SendKeysEvent,
 TypeTextEvent,
 UploadFileEvent,
 WaitEvent,
)
from system.browser.views import BrowserError, URLNotAllowedError
from system.browser.watchdog_base import BaseWatchdog
from system.dom.service import EnhancedDOMTreeNode
from system.observability import observe_debug

Import EnhancedDOMTreeNode and rebuild event models that have forward references to it
This must be done after all imports are complete
ClickCoordinateEvent.model_rebuild()
ClickElementEvent.model_rebuild()
GetDropdownOptionsEvent.model_rebuild()
SelectDropdownOptionEvent.model_rebuild()
TypeTextEvent.model_rebuild()
ScrollEvent.model_rebuild()
UploadFileEvent.model_rebuild()
class DefaultActionWatchdog(BaseWatchdog):
 """Handles default browser actions like click, type, and scroll using CDP."""

```

```

 async def _execute_click_with_download_detection(
 self,
 click_coro,
 download_start_timeout: float = 0.5,
 download_complete_timeout: float = 30.0,
) -> dict | None:
 """Execute a click operation and automatically wait for any triggered download

 Args:
 click_coro: Coroutine that performs the click (should return click_metadata dict or None)
 download_start_timeout: Time to wait for download to start after click (seconds)
 download_complete_timeout: Time to wait for download to complete once started (seconds)

 Returns:
 Click metadata dict, potentially with 'download' key containing download info.
 If a download times out but is still in progress, includes 'download_in_progress' with status.
 """
 import time

 download_started = asyncio.Event()
 download_completed = asyncio.Event()
 download_info: dict = {}
 progress_info: dict = {'last_update': 0.0, 'received_bytes': 0, 'total_bytes': 0, 'state': ''}

 def on_download_start(info: dict) -> None:
 """Direct callback when download starts (called from CDP handler)."""
 if info.get('auto_download'):
 return # ignore auto-downloads
 download_info['guid'] = info.get('guid', '')
 download_info['url'] = info.get('url', '')
 download_info['suggested_filename'] = info.get('suggested_filename', 'download')
 download_started.set()
 self.logger.debug(f'[ClickWithDownload] Download started: {download_info["suggested_filename"]}')

 def on_download_progress(info: dict) -> None:
 """Direct callback when download progress updates (called from CDP handler)."""
 # Match by guid if available
 if download_info.get('guid') and info.get('guid') != download_info['guid']:
 return # different download
 progress_info['last_update'] = time.time()
 progress_info['received_bytes'] = info.get('received_bytes', 0)
 progress_info['total_bytes'] = info.get('total_bytes', 0)
 progress_info['state'] = info.get('state', '')
 self.logger.debug(
 f'[ClickWithDownload] Progress: {progress_info["received_bytes"]}/{progress_info["total_bytes"]} by {info.get("state", "")}'
)

 def on_download_complete(info: dict) -> None:
 """Direct callback when download completes (called from CDP handler)."""
 if info.get('auto_download'):
 return # ignore auto-downloads
 # Match by guid if available, otherwise accept any non-auto download
 if download_info.get('guid') and info.get('guid') and info.get('guid') != download_info['guid']:
 return # different download
 download_info['path'] = info.get('path', '')
 download_info['file_name'] = info.get('file_name', '')
 download_info['file_size'] = info.get('file_size', 0)
 download_info['file_type'] = info.get('file_type', '')
 download_info['mime_type'] = info.get('mime_type', '')
 download_completed.set()
 self.logger.debug(f'[ClickWithDownload] Download completed: {download_info["file_name"]}')

 # Get the downloads watchdog and register direct callbacks
 downloads_watchdog = self.browser_session.downloads_watchdog
 self.logger.debug(f'[ClickWithDownload] downloads_watchdog={downloads_watchdog is not None}')
 if downloads_watchdog:
 self.logger.debug(f'[ClickWithDownload] Registering download callbacks...')
 downloads_watchdog.register_download_callbacks(
 on_start=on_download_start,
 on_progress=on_download_progress,
 on_complete=on_download_complete,
)

```



```

 else:
 self.logger.warning('[ClickWithDownload] No downloads_watchdog available!')

 try:
 # Perform the click
 click_metadata = await click_coro

 # Check for validation errors - return them immediately without waiting for downloads
 if isinstance(click_metadata, dict) and 'validation_error' in click_metadata:
 return click_metadata

 # Wait briefly to see if a download starts
 try:
 await asyncio.wait_for(download_started.wait(), timeout=download_start_timeout)

 # Download started!
 self.logger.info(f'■ Download started: {download_info.get("suggested_filename", "unknown")}')

 # Now wait for it to complete with longer timeout
 try:
 await asyncio.wait_for(download_completed.wait(), timeout=download_complete_timeout)

 # Download completed successfully
 msg = f'Downloaded file: {download_info["file_name"]} ({download_info["file_size"]} bytes) saved to {download_info["path"]}'
 self.logger.info(f'■ {msg}')

 # Merge download info into click_metadata
 if click_metadata is None:
 click_metadata = {}
 click_metadata['download'] = {
 'path': download_info['path'],
 'file_name': download_info['file_name'],
 'file_size': download_info['file_size'],
 'file_type': download_info.get('file_type'),
 'mime_type': download_info.get('mime_type'),
 }
 except TimeoutError:
 # Download timed out - check if it's still in progress
 if click_metadata is None:
 click_metadata = {}

 filename = download_info.get('suggested_filename', 'unknown')
 received = progress_info.get('received_bytes', 0)
 total = progress_info.get('total_bytes', 0)
 state = progress_info.get('state', 'unknown')
 last_update = progress_info.get('last_update', 0.0)
 time_since_update = time.time() - last_update if last_update > 0 else float('inf')

 # Check if download is still actively progressing (received update in last 5 seconds)
 is_still_active = time_since_update < 5.0 and state == 'InProgress'

 if is_still_active:
 # Download is still progressing - suggest waiting
 if total > 0:
 percent = (received / total) * 100
 progress_str = f'{percent:.1f}% ({received:,}/{total:,} bytes)'
 else:
 progress_str = f'{received:,} bytes downloaded (total size unknown)'

 msg = (
 f'Download timed out after {download_complete_timeout}s but is still in progress: '
 f'{filename} - {progress_str}. '
 f'The download appears to be progressing normally. Consider using the wait action '
 f'to allow more time for the download to complete.'
)
 self.logger.warning(f'■ {msg}')
 click_metadata['download_in_progress'] = {
 'file_name': filename,
 'received_bytes': received,
 'total_bytes': total,
 'state': state,
 'message': msg,
 }

```

```

 #####}
 #####else:
 ##### # Download may be stalled or completed
 ##### if received > 0:
 ##### msg = (
 ##### f'Download timed out after {download_complete_timeout}s: {filename}. '
 ##### f'Last progress: {received:,} bytes received. '
 ##### f'The download may have stalled or completed - check the downloads folder.'
 #####)
 ##### else:
 ##### msg = (
 ##### f'Download timed out after {download_complete_timeout}s: {filename}. '
 ##### f'No progress data received - the download may have failed to start properly.'
 #####)
 ##### self.logger.warning(f'### {msg}')
 ##### click_metadata['download_timeout'] = {
 ##### 'file_name': filename,
 ##### 'received_bytes': received,
 ##### 'total_bytes': total,
 ##### 'message': msg,
 ##### }
 #####except TimeoutError:
 ##### # No download started within grace period
 ##### pass

 #####return click_metadata if isinstance(click_metadata, dict) else None

 #####finally:
 ##### # Unregister download callbacks
 ##### if downloads_watchdog:
 ##### downloads_watchdog.unregister_download_callbacks(
 ##### on_start=on_download_start,
 ##### on_progress=on_download_progress,
 ##### on_complete=on_download_complete,
 #####)

 #####def _is_print_related_element(self, element_node: EnhancedDOMTreeNode) -> bool:
 ##### """Check if an element is related to printing (print buttons, print dialogs, etc.).

 ##### Primary check: onclick attribute (most reliable for print detection)
 ##### Fallback: button text/value (for cases without onclick)
 ##### """
 ##### # Primary: Check onclick attribute for print-related functions (most reliable)
 ##### onclick = element_node.attributes.get('onclick', '').lower() if element_node.attributes else ''
 ##### if onclick and 'print' in onclick:
 ##### # Matches: window.print(), PrintElem(), print(), etc.
 ##### return True

 ##### return False

 #####async def _handle_print_button_click(self, element_node: EnhancedDOMTreeNode) -> dict | None:
 ##### """Handle print button by directly generating PDF via CDP instead of opening dialog.

 ##### Returns:
 ##### Metadata dict with download path if successful, None otherwise
 ##### """
 ##### try:
 ##### import base64
 ##### import os
 ##### from pathlib import Path

 ##### # Get CDP session
 ##### cdp_session = await self.browser_session.get_or_create_cdp_session(focus=True)

 ##### # Generate PDF using CDP Page.printToPDF
 ##### result = await asyncio.wait_for(
 ##### cdp_session.cdp_client.send(Page.printToPDF(
 ##### params={
 ##### 'printBackground': True,
 ##### 'preferCSSPageSize': True,
 ##### },
 ##### session_id=cdp_session.session_id,

```

```

),
 timeout=15.0, # 15 second timeout for PDF generation
)

 pdf_data = result.get('data')
 if not pdf_data:
 self.logger.warning('PDF generation returned no data')
 return None

 # Decode base64 PDF data
 pdf_bytes = base64.b64decode(pdf_data)

 # Get downloads path
 downloads_path = self.browser_session.browser_profile.downloads_path
 if not downloads_path:
 self.logger.warning('No downloads path configured, cannot save PDF')
 return None

 # Generate filename from page title or URL
 try:
 page_title = await asyncio.wait_for(self.browser_session.get_current_page_title(), timeout=2.0)
 # Sanitize title for filename
 import re

 safe_title = re.sub(r'[^\w\s-]', '', page_title)[:50] # Max 50 chars
 filename = f'{safe_title}.pdf' if safe_title else 'print.pdf'
 except Exception:
 filename = 'print.pdf'

 # Ensure downloads directory exists
 downloads_dir = Path(downloads_path).expanduser().resolve()
 downloads_dir.mkdir(parents=True, exist_ok=True)

 # Generate unique filename if file exists
 final_path = downloads_dir / filename
 if final_path.exists():
 base, ext = os.path.splitext(filename)
 counter = 1
 while (downloads_dir / f'{base} ({counter}){ext}').exists():
 counter += 1
 final_path = downloads_dir / f'{base} ({counter}){ext}'

 # Write PDF to file
 import anyio

 async with await anyio.open_file(final_path, 'wb') as f:
 await f.write(pdf_bytes)

 file_size = final_path.stat().st_size
 self.logger.info(f'Generated PDF via CDP: {final_path} ({file_size:,} bytes)')

 # Dispatch FileDownloadedEvent
 from system.browser.events import FileDownloadedEvent

 page_url = await self.browser_session.get_current_page_url()
 self.browser_session.event_bus.dispatch(
 FileDownloadedEvent(
 url=page_url,
 path=str(final_path),
 file_name=final_path.name,
 file_size=file_size,
 file_type='pdf',
 mime_type='application/pdf',
 auto_download=False, # This was intentional (user clicked print)
)
)

 return {'pdf_generated': True, 'path': str(final_path)}

 except TimeoutError:
 self.logger.warning('PDF generation timed out')
 return None

```

```

 except Exception as e:
 self.logger.warning(f'Failed to generate PDF via CDP: {type(e).__name__}: {e}')
 return None

 @observe_debug(ignore_input=True, ignore_output=True, name='click_element_event')
 async def on_ClickElementEvent(self, event: ClickElementEvent) -> dict | None:
 """Handle click request with CDP. Automatically waits for file downloads if triggered."""
 try:
 # Check if session is alive before attempting any operations
 if not self.browser_session.agent_focus_target_id:
 error_msg = 'Cannot execute click: browser session is corrupted (target_id=None). Session may have'
 self.logger.error(f'{error_msg}')
 raise BrowserError(error_msg)

 # Use the provided node
 element_node = event.node
 index_for_logging = element_node.backend_node_id or 'unknown'

 # Check if element is a file input (should not be clicked)
 if self.browser_session.is_file_input(element_node):
 msg = f'Index {index_for_logging} - has an element which opens file upload dialog. To upload files'
 self.logger.info(f'{msg}')
 return {'validation_error': msg}

 # Detect print-related elements and handle them specially
 is_print_element = self._is_print_related_element(element_node)
 if is_print_element:
 self.logger.info(
 f'Detected print button (index {index_for_logging}), generating PDF directly instead of opening'
)
 click_metadata = await self._handle_print_button_click(element_node)
 if click_metadata and click_metadata.get('pdf_generated'):
 msg = f'Generated PDF: {click_metadata.get("path")}'
 self.logger.info(f'{msg}')
 return click_metadata
 else:
 self.logger.warning('PDF generation failed, falling back to regular click')

 # Execute click with automatic download detection
 click_metadata = await self._execute_click_with_download_detection(self._click_element_node_impl(element_node))

 # Check for validation errors
 if isinstance(click_metadata, dict) and 'validation_error' in click_metadata:
 self.logger.info(f'{click_metadata["validation_error"]}')
 return click_metadata

 # Build success message for non-download clicks
 if 'download' not in (click_metadata or {}):
 msg = f'Clicked button {element_node.node_name}: {element_node.get_all_children_text(max_depth=2)}'
 self.logger.debug(f'{msg}')
 self.logger.debug(f'Element xpath: {element_node.xpath}')

 return click_metadata

 except Exception:
 raise

 async def on_ClickCoordinateEvent(self, event: ClickCoordinateEvent) -> dict | None:
 """Handle click at coordinates with CDP. Automatically waits for file downloads if triggered."""
 try:
 # Check if session is alive before attempting any operations
 if not self.browser_session.agent_focus_target_id:
 error_msg = 'Cannot execute click: browser session is corrupted (target_id=None). Session may have'
 self.logger.error(f'{error_msg}')
 raise BrowserError(error_msg)

 # If force=True, skip safety checks and click directly (with download detection)
 if event.force:
 self.logger.debug(f'Force clicking at coordinates ({event.coordinate_x}, {event.coordinate_y})')
 return await self._execute_click_with_download_detection(
 self._click_on_coordinate(event.coordinate_x, event.coordinate_y, force=True)
)

```

```

Get element at coordinates for safety checks
#####element_node = await self.browser_session.get_dom_element_at_coordinates(event.coordinate_x, event.coordinate_y)
#####if element_node is None:
No element found, click directly (with download detection)
self.logger.debug(
f'No element found at coordinates ({event.coordinate_x}, {event.coordinate_y}), proceeding with click'
)
return await self._execute_click_with_download_detection(
self._click_on_coordinate(event.coordinate_x, event.coordinate_y, force=False)
)

Safety check: file input
#####if self.browser_session.is_file_input(element_node):
msg = f'Cannot click at ({event.coordinate_x}, {event.coordinate_y}) - element is a file input. To click, use the download button'
self.logger.info(f'{msg}')
return {'validation_error': msg}

Safety check: select element
#####tag_name = element_node.tag_name.lower() if element_node.tag_name else ''
#####if tag_name == 'select':
msg = f'Cannot click at ({event.coordinate_x}, {event.coordinate_y}) - element is a <select>. Use the dropdown arrow'
self.logger.info(f'{msg}')
return {'validation_error': msg}

Safety check: print-related elements
#####is_print_element = self._is_print_related_element(element_node)
#####if is_print_element:
self.logger.info(
f'Detected print button at ({event.coordinate_x}, {event.coordinate_y}), generating PDF directly'
)
click_metadata = await self._handle_print_button_click(element_node)
if click_metadata and click_metadata.get('pdf_generated'):
msg = f'Generated PDF: {click_metadata.get("path")}'
self.logger.info(f'{msg}')
return click_metadata
else:
self.logger.warning(f'PDF generation failed, falling back to regular click')

All safety checks passed, click at coordinates (with download detection)
#####return await self._execute_click_with_download_detection(
self._click_on_coordinate(event.coordinate_x, event.coordinate_y, force=False)
#####)

#####except Exception:
raise

#####async def on_TypeTextEvent(self, event: TypeTextEvent) -> dict | None:
"""Handle text input request with CDP."""
try:
Use the provided node
element_node = event.node
index_for_logging = element_node.backend_node_id or 'unknown'

Check if this is index 0 or a falsy index - type to the page (whatever has focus)
if not element_node.backend_node_id or element_node.backend_node_id == 0:
Type to the page without focusing any specific element
await self._type_to_page(event.text)
Log with sensitive data protection
if event.is_sensitive:
if event.sensitive_key_name:
self.logger.info(f'Typed <{event.sensitive_key_name}> to the page (current focus)')
else:
self.logger.info(f'Typed <sensitive> to the page (current focus)')
else:
self.logger.info(f'Typed "{event.text}" to the page (current focus)')
return None # No coordinates available for page typing
else:
try:
Try to type to the specific element
input_metadata = await self._input_text_element_node_impl(
element_node,
event.text,

```

```

#####clear=event.clear or (not event.text),
#####is_sensitive=event.is_sensitive,
#####)
Log with sensitive data protection
#####if event.is_sensitive:
#####if event.sensitive_key_name:
#####self.logger.info(f'### Typed <{event.sensitive_key_name}> into element with index {index_for_logging}')
#####else:
#####self.logger.info(f'### Typed <sensitive> into element with index {index_for_logging}')
#####else:
#####self.logger.info(f'### Typed "{event.text}" into element with index {index_for_logging}')
#####self.logger.debug(f'Element xpath: {element_node.xpath}')
#####return input_metadata # Return coordinates if available
#####except Exception as e:
Element not found or error - fall back to typing to the page
#####self.logger.warning(f'Failed to type to element {index_for_logging}: {e}. Falling back to page typing')
#####try:
#####await asyncio.wait_for(self._click_element_node_impl(element_node), timeout=10.0)
#####except Exception as e:
#####pass
#####await self._type_to_page(event.text)
Log with sensitive data protection
#####if event.is_sensitive:
#####if event.sensitive_key_name:
#####self.logger.info(f'### Typed <{event.sensitive_key_name}> to the page as fallback')
#####else:
#####self.logger.info(f'### Typed <sensitive> to the page as fallback')
#####else:
#####self.logger.info(f'### Typed "{event.text}" to the page as fallback')
#####return None # No coordinates available for fallback typing

Note: We don't clear cached state here - let multi_act handle DOM change detection
by explicitly rebuilding and comparing when needed
#####except Exception as e:
#####raise

#####async def on_ScrollEvent(self, event: ScrollEvent) -> None:
"""Handle scroll request with CDP."""
Check if we have a current target for scrolling
if not self.browser_session.agent_focus_target_id:
error_msg = 'No active target for scrolling'
raise BrowserError(error_msg)

try:
Convert direction and amount to pixels
Positive pixels = scroll down, negative = scroll up
pixels = event.amount if event.direction == 'down' else -event.amount

Element-specific scrolling if node is provided
if event.node is not None:
element_node = event.node
index_for_logging = element_node.backend_node_id or 'unknown'

Check if the element is an iframe
is_iframe = element_node.tag_name and element_node.tag_name.upper() == 'IFRAME'

Try to scroll the element's container
success = await self._scroll_element_container(element_node, pixels)
if success:
self.logger.debug(
f'### Scrolled element {index_for_logging} container {event.direction} by {event.amount} pixels'
)

For iframe scrolling, we need to force a full DOM refresh
because the iframe's content has changed position
if is_iframe:
self.logger.debug('### Forcing DOM refresh after iframe scroll')
Note: We don't clear cached state here - let multi_act handle DOM change detection
by explicitly rebuilding and comparing when needed

Wait a bit for the scroll to settle and DOM to update
await asyncio.sleep(0.2)

```

```

 return None

 # Perform target-level scroll
 await self._scroll_with_cdp_gesture(pixels)

 # Note: We don't clear cached state here - let multi_act handle DOM change detection
 # by explicitly rebuilding and comparing when needed

 # Log success
 self.logger.debug(f'Scrolled {event.direction} by {event.amount} pixels')
 return None
 except Exception as e:
 raise

===== Implementation Methods =====

async def _check_element_occlusion(self, backend_node_id: int, x: float, y: float, cdp_session) -> bool:
 """Check if an element is occluded by other elements at the given coordinates.

 Args:
 backend_node_id: The backend node ID of the target element
 x: X coordinate to check
 y: Y coordinate to check
 cdp_session: CDP session to use

 Returns:
 True if element is occluded, False if clickable
 """
 try:
 session_id = cdp_session.session_id

 # Get target element info for comparison
 target_result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id}, session_id=session_id
)

 if 'object' not in target_result:
 self.logger.debug('Could not resolve target element, assuming occluded')
 return True

 object_id = target_result['object']['objectId']

 # Get target element info
 target_info_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'objectId': object_id,
 'functionDeclaration': """
 function() {
 const getElementInfo = (el) => {
 return {
 tagName: el.tagName,
 id: el.id || '',
 className: el.className || '',
 textContent: (el.textContent || '').substring(0, 100)
 };
 };
 """
 })

 const elementAtPoint = document.elementFromPoint(arguments[0], arguments[1]);
 if (!elementAtPoint) {
 return { targetInfo: getElementInfo(this), isClickable: false };
 }

 // Simple containment-based clickability logic
 let isClickable = this === elementAtPoint ||
 this.contains(elementAtPoint) ||
 elementAtPoint.contains(this);

 // Check label-input associations when containment check fails
 if (!isClickable) {
 const target = this;

```

```

const atPoint = elementAtPoint;

// Case 1: target is <input>, atPoint is its associated <label> (or child of that label)
if (target.tagName === 'INPUT' && target.id) {
 const escapedId = CSS.escape(target.id);
 const assocLabel = document.querySelector('label[for="' + escapedId + '"]');
 if (assocLabel && (assocLabel === atPoint || assocLabel.contains(atPoint))) {
 isClickable = true;
 }
}

// Case 2: target is <input>, atPoint is inside a <label> ancestor that wraps the target
if (!isClickable && target.tagName === 'INPUT') {
 let ancestor = atPoint;
 for (let i = 0; i < 3 && ancestor; i++) {
 if (ancestor.tagName === 'LABEL' && ancestor.contains(target)) {
 isClickable = true;
 break;
 }
 ancestor = ancestor.parentElement;
 }
}

// Case 3: target is <label>, atPoint is the associated <input>
if (!isClickable && target.tagName === 'LABEL') {
 if (target.htmlFor && atPoint.tagName === 'INPUT' && atPoint.id === target.htmlFor) {
 isClickable = true;
 }
 // Also check if atPoint is an input inside the label
 if (!isClickable && atPoint.tagName === 'INPUT' && target.contains(atPoint)) {
 isClickable = true;
 }
}

return {
 targetInfo: getElementInfo(this),
 elementAtPointInfo: getElementInfo(elementAtPoint),
 isClickable: isClickable
};
},
"",
arguments: [{value: x}, {value: y}],
returnByValue: True,
},
session_id=session_id,
)

if 'result' not in target_info_result or 'value' not in target_info_result['result']:
 self.logger.debug('Could not get target element info, assuming occluded')
 return True

target_data = target_info_result['result']['value']
is_clickable = target_data.get('isClickable', False)

if is_clickable:
 self.logger.debug('Element is clickable (target, contained, or semantically related)')
 return False
else:
 target_info = target_data.get('targetInfo', {})
 element_at_point_info = target_data.get('elementAtPointInfo', {})
 self.logger.debug(
 f'Element is occluded. Target: {target_info.get("tagName", "unknown")} '
 f'(id={target_info.get("id", "none")}), '
 f'ElementAtPoint: {element_at_point_info.get("tagName", "unknown")} '
 f'(id={element_at_point_info.get("id", "none")})'
)
 return True

except Exception as e:
 self.logger.debug(f'Occlusion check failed: {e}, assuming not occluded')
 return False

```



```

 async def _click_element_node_impl(self, element_node) -> dict | None:
 """
 Click an element using pure CDP with multiple fallback methods for getting element geometry.

 Args:
 element_node: The DOM element to click
 """

 try:
 # Check if element is a file input or select dropdown - these should not be clicked
 tag_name = element_node.tag_name.lower() if element_node.tag_name else ''
 element_type = element_node.attributes.get('type', '').lower() if element_node.attributes else ''

 if tag_name == 'select':
 msg = f'Cannot click on <select> elements. Use dropdown_options(index={element_node.backend_node_id}). File uploads must use dropdown_options(index={element_node.backend_node_id}). File uploads must use dropdown_options(index={element_node.backend_node_id}).'
 # Return error dict instead of raising to avoid ERROR logs
 return {'validation_error': msg}

 if tag_name == 'input' and element_type == 'file':
 msg = f'Cannot click on file input element (index={element_node.backend_node_id}). File uploads must use dropdown_options(index={element_node.backend_node_id}). File uploads must use dropdown_options(index={element_node.backend_node_id}).'
 # Return error dict instead of raising to avoid ERROR logs
 return {'validation_error': msg}

 # Get CDP client
 cdp_session = await self.browser_session.cdp_client_for_node(element_node)

 # Get the correct session ID for the element's frame
 session_id = cdp_session.session_id

 # Get element bounds
 backend_node_id = element_node.backend_node_id

 # Get viewport dimensions for visibility checks
 layout_metrics = await cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=session_id)
 viewport_width = layout_metrics['layoutViewport']['clientWidth']
 viewport_height = layout_metrics['layoutViewport']['clientHeight']

 # Scroll element into view FIRST before getting coordinates
 try:
 await cdp_session.cdp_client.send.DOM.scrollIntoViewIfNeeded(
 params={'backendNodeId': backend_node_id, 'session_id': session_id}
)
 await asyncio.sleep(0.05) # Wait for scroll to complete
 self.logger.debug('Scrolled element into view before getting coordinates')
 except Exception as e:
 self.logger.debug(f'Failed to scroll element into view: {e}')

 # Get element coordinates using the unified method AFTER scrolling
 element_rect = await self.browser_session.get_element_coordinates(backend_node_id, cdp_session)

 # Convert rect to quads format if we got coordinates
 quads = []
 if element_rect:
 # Convert DOMRect to quad format
 x, y, w, h = element_rect.x, element_rect.y, element_rect.width, element_rect.height
 quads = [
 [
 x,
 y, # top-left
 x + w,
 y, # top-right
 x + w,
 y + h, # bottom-right
 x,
 y + h, # bottom-left
]
]
 self.logger.debug(
 f'Got coordinates from unified method: {element_rect.x}, {element_rect.y}, {element_rect.width}x{element_rect.height}'
)

 # If we still don't have quads, fall back to JS click

```

```

 if not quads:
 self.logger.warning('Could not get element geometry from any method, falling back to JavaScript cli
 try:
 result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id},
 session_id=session_id,
)
 assert 'object' in result and 'objectId' in result['object'], (
 'Failed to find DOM element based on backendNodeId, maybe page content changed?'
)
 object_id = result['object']['objectId']

 await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': 'function() { this.click(); }',
 'objectId': object_id,
 },
 session_id=session_id,
)
 await asyncio.sleep(0.05)
 # Navigation is handled by BrowserSession via events
 return None
 except Exception as js_e:
 self.logger.warning(f'CDP JavaScript click also failed: {js_e}')
 if 'No node with given id found' in str(js_e):
 raise Exception('Element with given id not found')
 else:
 raise Exception(f'Failed to click element: {js_e}')

 # Find the largest visible quad within the viewport
 best_quad = None
 best_area = 0

 for quad in quads:
 if len(quad) < 8:
 continue

 # Calculate quad bounds
 xs = [quad[i] for i in range(0, 8, 2)]
 ys = [quad[i] for i in range(1, 8, 2)]
 min_x, max_x = min(xs), max(xs)
 min_y, max_y = min(ys), max(ys)

 # Check if quad intersects with viewport
 if max_x < 0 or max_y < 0 or min_x > viewport_width or min_y > viewport_height:
 continue # Quad is completely outside viewport

 # Calculate visible area (intersection with viewport)
 visible_min_x = max(0, min_x)
 visible_max_x = min(viewport_width, max_x)
 visible_min_y = max(0, min_y)
 visible_max_y = min(viewport_height, max_y)

 visible_width = visible_max_x - visible_min_x
 visible_height = visible_max_y - visible_min_y
 visible_area = visible_width * visible_height

 if visible_area > best_area:
 best_area = visible_area
 best_quad = quad

 if not best_quad:
 # No visible quad found, use the first quad anyway
 best_quad = quads[0]
 self.logger.warning('No visible quad found, using first quad')

 # Calculate center point of the best quad
 center_x = sum(best_quad[i] for i in range(0, 8, 2)) / 4
 center_y = sum(best_quad[i] for i in range(1, 8, 2)) / 4

 # Ensure click point is within viewport bounds
 center_x = max(0, min(viewport_width - 1, center_x))

```

```

 center_y = max(0, min(viewport_height - 1, center_y))

 # Check for occlusion before attempting CDP click
 is_occluded = await self._check_element_occlusion(backend_node_id, center_x, center_y, cdp_session)

 if is_occluded:
 self.logger.debug('■ Element is occluded, falling back to JavaScript click')
 try:
 result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id,
 session_id=session_id,
 })
 assert 'object' in result and 'objectId' in result['object'], (
 'Failed to find DOM element based on backendNodeId'
)
 object_id = result['object']['objectId']

 await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': 'function() { this.click(); }',
 'objectId': object_id,
 },
 session_id=session_id,
)
 await asyncio.sleep(0.05)
 return None
 except Exception as js_e:
 self.logger.error(f'JavaScript click fallback failed: {js_e}')
 raise Exception(f'Failed to click occluded element: {js_e}')

 # Perform the click using CDP (element is not occluded)
 try:
 self.logger.debug(f'■ Dragging mouse over element before clicking x: {center_x}px y: {center_y}px')
 # Move mouse to element
 await cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mouseMoved',
 'x': center_x,
 'y': center_y,
 },
 session_id=session_id,
)
 await asyncio.sleep(0.05)

 # Mouse down
 self.logger.debug(f'■ Clicking x: {center_x}px y: {center_y}px ...')
 try:
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mousePressed',
 'x': center_x,
 'y': center_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=session_id,
),
 timeout=3.0, # 3 second timeout for mousePressed
)
 await asyncio.sleep(0.08)
 except TimeoutError:
 self.logger.debug('■ Mouse down timed out (likely due to dialog), continuing...')
 # Don't sleep if we timed out

 # Mouse up
 try:
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mouseReleased',
 'x': center_x,

```

```

 'y': center_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=session_id,
),
timeout=5.0, # 5 second timeout for mouseReleased
)
except TimeoutError:
 self.logger.debug('■ Mouse up timed out (possibly due to lag or dialog popup), continuing...')

 self.logger.debug('■ Clicked successfully using x,y coordinates')

 # Return coordinates as dict for metadata
 return {'click_x': center_x, 'click_y': center_y}

except Exception as e:
 self.logger.warning(f'CDP click failed: {type(e).__name__}: {e}')
 # Fall back to JavaScript click via CDP
 try:
 result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id},
 session_id=session_id,
)
 assert 'object' in result and 'objectId' in result['object'], (
 'Failed to find DOM element based on backendNodeId, maybe page content changed?'
)
 object_id = result['object']['objectId']

 await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': 'function() { this.click(); }',
 'objectId': object_id,
 },
 session_id=session_id,
)

 # Small delay for dialog dismissal
 await asyncio.sleep(0.1)

 return None
 except Exception as js_e:
 self.logger.warning(f'CDP JavaScript click also failed: {js_e}')
 raise Exception(f'Failed to click element: {e}')
 finally:
 # Always re-focus back to original top-level page session context in case click opened a new tab/po
 # Use timeout to prevent hanging if dialog is blocking
 try:
 cdp_session = await asyncio.wait_for(self.browser_session.get_or_create_cdp_session(focus=True), t
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Runtime.runIfWaitingForDebugger(session_id=cdp_session.session_id),
 timeout=2.0,
)
 except TimeoutError:
 self.logger.debug('■ Refocus after click timed out (page may be blocked by dialog). Continuing..')
 except Exception as e:
 self.logger.debug(f'■ Refocus error (non-critical): {type(e).__name__}: {e}')

 except URLNotAllowedError as e:
 raise e
 except BrowserError as e:
 raise e
 except Exception as e:
 # Extract key element info for error message
 element_info = f'<{element_node.tag_name or "unknown"}'
 if element_node.backend_node_id:
 element_info += f' index={element_node.backend_node_id}'
 element_info += '>'

 # Create helpful error message based on context
 error_detail = f'Failed to click element {element_info}. The element may not be interactable or visi
 # Add hint if element has index (common in code-use mode)

```

```

 if element_node.backed_node_id:
 error_detail += f' If the page changed after navigation/interaction, the index [{element_node.backed_node_id}] is out of range.'
 raise BrowserError(
 message=f'Failed to click element: {str(e)}',
 long_term_memory=error_detail,
)

 async def _click_on_coordinate(self, coordinate_x: int, coordinate_y: int, force: bool = False) -> dict:
 """
 Click directly at coordinates using CDP Input.dispatchEvent.

 Args:
 coordinate_x: X coordinate in viewport
 coordinate_y: Y coordinate in viewport
 force: If True, skip all safety checks (used when force=True in event)

 Returns:
 Dict with click coordinates or None
 """
 try:
 # Get CDP session
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 session_id = cdp_session.session_id

 self.logger.debug(f'Moving mouse to ({coordinate_x}, {coordinate_y})...')

 # Move mouse to coordinates
 await cdp_session.cdp_client.send.Input.dispatchEvent(
 params={
 'type': 'mouseMoved',
 'x': coordinate_x,
 'y': coordinate_y,
 },
 session_id=session_id,
)
 await asyncio.sleep(0.05)

 # Mouse down
 self.logger.debug(f'Clicking at ({coordinate_x}, {coordinate_y})...')
 try:
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Input.dispatchEvent(
 params={
 'type': 'mousePressed',
 'x': coordinate_x,
 'y': coordinate_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=session_id,
),
 timeout=3.0,
)
 await asyncio.sleep(0.05)
 except TimeoutError:
 self.logger.debug('Mouse down timed out (likely due to dialog), continuing...')

 # Mouse up
 try:
 await asyncio.wait_for(
 cdp_session.cdp_client.send.Input.dispatchEvent(
 params={
 'type': 'mouseReleased',
 'x': coordinate_x,
 'y': coordinate_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=session_id,
),
 timeout=5.0,
)

```

```

)
 except TimeoutError:
 self.logger.debug('■ Mouse up timed out (possibly due to lag or dialog popup), continuing...')

 self.logger.debug(f'■ Clicked successfully at ({coordinate_x}, {coordinate_y})')

 # Return coordinates as metadata
 return {'click_x': coordinate_x, 'click_y': coordinate_y}

 except Exception as e:
 self.logger.error(f'Failed to click at coordinates ({coordinate_x}, {coordinate_y}): {type(e).__name__}')
 raise BrowserError(
 message=f'Failed to click at coordinates: {e}',
 long_term_memory=f'Failed to click at coordinates ({coordinate_x}, {coordinate_y}). The coordinates'
)

 async def _type_to_page(self, text: str):
 """
 Type text to the page (whatever element currently has focus).
 This is used when index is 0 or when an element can't be found.
 """
 try:
 # Get CDP client and session
 cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=None, focus=True)

 # Type the text character by character to the focused element
 for char in text:
 # Handle newline characters as Enter key
 if char == '\n':
 # Send proper Enter key sequence
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyDown',
 'key': 'Enter',
 'code': 'Enter',
 'windowsVirtualKeyCode': 13,
 },
 session_id=cdp_session.session_id,
)
 # Send char event with carriage return
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'char',
 'text': '\r',
 },
 session_id=cdp_session.session_id,
)
 # Send keyup
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyUp',
 'key': 'Enter',
 'code': 'Enter',
 'windowsVirtualKeyCode': 13,
 },
 session_id=cdp_session.session_id,
)
 else:
 # Handle regular characters
 # Send keydown
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyDown',
 'key': char,
 },
 session_id=cdp_session.session_id,
)
 # Send char for actual text input
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'char',
 'text': char,

```

```

 },
 session_id=cdp_session.session_id,
)
 # Send keyup
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyUp',
 'key': char,
 },
 session_id=cdp_session.session_id,
)
 # Add 10ms delay between keystrokes
 await asyncio.sleep(0.010)
except Exception as e:
 raise Exception(f'Failed to type to page: {str(e)}')

def _get_char_modifiers_and_vk(self, char: str) -> tuple[int, int, str]:
 """Get modifiers, virtual key code, and base key for a character.

 Returns:
 (modifiers, windowsVirtualKeyCode, base_key)
 """
 # Characters that require Shift modifier
 shift_chars = {
 '!': ('1', 49),
 '@': ('2', 50),
 '#': ('3', 51),
 '$': ('4', 52),
 '%': ('5', 53),
 '^': ('6', 54),
 '&': ('7', 55),
 '*': ('8', 56),
 '(': ('9', 57),
 ')': ('0', 48),
 '_': ('-', 189),
 '+': ('=', 187),
 '{': ('[', 219),
 '}': (']', 221),
 '|': ('\\', 220),
 ':': (';', 186),
 '"': ('"', 222),
 '<': ('<', 188),
 '>': ('>', 190),
 '?': ('/', 191),
 '~': ('`', 192),
 }

 # Check if character requires Shift
 if char in shift_chars:
 base_key, vk_code = shift_chars[char]
 return (8, vk_code, base_key) # Shift=8

 # Uppercase letters require Shift
 if char.isupper():
 return (8, ord(char), char.lower()) # Shift=8

 # Lowercase letters
 if char.islower():
 return (0, ord(char.upper()), char)

 # Numbers
 if char.isdigit():
 return (0, ord(char), char)

 # Special characters without Shift
 no_shift_chars = {
 ' ': 32,
 '-': 189,
 '=': 187,
 '[': 219,
 ']': 221,
 '\\': 220,
 }

```

```

 '': 186,
 '"': 222,
 ',': 188,
 '.': 190,
 '/': 191,
 '`': 192,
}

if char in no_shift_chars:
 return (0, no_shift_chars[char], char)

Fallback
return (0, ord(char.upper()) if char.isalpha() else ord(char), char)

def _get_key_code_for_char(self, char: str) -> str:
 """Get the proper key code for a character (like Playwright does)."""
 # Key code mapping for common characters (using proper base keys + modifiers)
 key_codes = {
 ' ': 'Space',
 '.': 'Period',
 ',': 'Comma',
 '-': 'Minus',
 '_': 'Minus', # Underscore uses Minus with Shift
 '@': 'Digit2', # @ uses Digit2 with Shift
 '!': 'Digit1', # ! uses Digit1 with Shift (not 'Exclamation')
 '?': 'Slash', # ? uses Slash with Shift
 ':': 'Semicolon', # : uses Semicolon with Shift
 ';': 'Semicolon',
 '(': 'Digit9', # (uses Digit9 with Shift
 ')': 'Digit0', #) uses Digit0 with Shift
 '[': 'BracketLeft',
 ']': 'BracketRight',
 '{': 'BracketLeft', # { uses BracketLeft with Shift
 '}': 'BracketRight', # } uses BracketRight with Shift
 '/': 'Slash',
 '\\': 'Backslash',
 '=': 'Equal',
 '+': 'Equal', # + uses Equal with Shift
 '*': 'Digit8', # * uses Digit8 with Shift
 '&': 'Digit7', # & uses Digit7 with Shift
 '%': 'Digit5', # % uses Digit5 with Shift
 '$': 'Digit4', # $ uses Digit4 with Shift
 '#': 'Digit3', # # uses Digit3 with Shift
 '^': 'Digit6', # ^ uses Digit6 with Shift
 '~': 'Backquote', # ~ uses Backquote with Shift
 '`': 'Backquote',
 '"': 'Quote',
 "'": 'Quote', # " uses Quote with Shift
 }

 # Numbers
 if char.isdigit():
 return f'Digit{char}'

 # Letters
 if char.isalpha():
 return f'Key{char.upper()}'

 # Special characters
 if char in key_codes:
 return key_codes[char]

 # Fallback for unknown characters
 return f'Key{char.upper()}'

 async def _clear_text_field(self, object_id: str, cdp_session) -> bool:
 """Clear text field using multiple strategies, starting with the most reliable."""
 try:
 # Strategy 1: Direct JavaScript value/content setting (handles both inputs and contenteditable)
 self.logger.debug('Clearing text field using JavaScript value setting')

 clear_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(

```



```

#####params={
#####'functionDeclaration': ""
#####function() {
#####// Check if it's a contenteditable element
#####const hasContentEditable = this.getAttribute('contenteditable') === 'true' ||
#####this.getAttribute('contenteditable') === '' ||
#####this.isContentEditable === true;

#####if (hasContentEditable) {
#####// For contenteditable elements, clear all content
#####while (this.firstChild) {
#####this.removeChild(this.firstChild);
#####}
#####this.textContent = "";
#####this.innerHTML = "";

#####// Focus and position cursor at the beginning
#####this.focus();
#####const selection = window.getSelection();
#####const range = document.createRange();
#####range.setStart(this, 0);
#####range.setEnd(this, 0);
#####selection.removeAllRanges();
#####selection.addRange(range);

#####// Dispatch events
#####this.dispatchEvent(new Event("input", { bubbles: true }));
#####this.dispatchEvent(new Event("change", { bubbles: true }));

#####return {cleared: true, method: 'contenteditable', finalText: this.textContent};
#####} else if (this.value !== undefined) {
#####// For regular inputs with value property
#####try {
#####this.select();
#####} catch (e) {
#####// ignore
#####}
#####this.value = "";
#####this.dispatchEvent(new Event("input", { bubbles: true }));
#####this.dispatchEvent(new Event("change", { bubbles: true }));
#####return {cleared: true, method: 'value', finalText: this.value};
#####} else {
#####return {cleared: false, method: 'none', error: 'Not a supported input type'};
#####}
#####},
#####"",
#####'objectId': object_id,
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)

Check the clear result
#####clear_info = clear_result.get('result', {}).get('value', {})
#####self.logger.debug(f'Clear result: {clear_info}')

#####if clear_info.get('cleared'):
#####final_text = clear_info.get('finalText', '')
#####if not final_text or not final_text.strip():
#####self.logger.debug(f'■ Text field cleared successfully using {clear_info.get("method")}')
#####return True
#####else:
#####self.logger.debug(f'■ JavaScript clear partially failed, field still contains: "{final_text}"')
#####return False
#####else:
#####self.logger.debug(f'■ JavaScript clear failed: {clear_info.get("error", "Unknown error")}')
#####return False

#####except Exception as e:
#####self.logger.debug(f'JavaScript clear failed with exception: {e}')
#####return False
Strategy 2: Triple-click + Delete (fallback for stubborn fields)

```

```

 try:
 self.logger.debug('■ Fallback: Clearing using triple-click + Delete')

 # Get element center coordinates for triple-click
 bounds_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': 'function() { return this.getBoundingClientRect(); }',
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)

 if bounds_result.get('result', {}).get('value'):
 bounds = bounds_result['result']['value']
 center_x = bounds['x'] + bounds['width'] / 2
 center_y = bounds['y'] + bounds['height'] / 2

 # Triple-click to select all text
 await cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mousePressed',
 'x': center_x,
 'y': center_y,
 'button': 'left',
 'clickCount': 3,
 },
 session_id=cdp_session.session_id,
)
 await cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mouseReleased',
 'x': center_x,
 'y': center_y,
 'button': 'left',
 'clickCount': 3,
 },
 session_id=cdp_session.session_id,
)

 # Delete selected text
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyDown',
 'key': 'Delete',
 'code': 'Delete',
 },
 session_id=cdp_session.session_id,
)
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
 params={
 'type': 'keyUp',
 'key': 'Delete',
 'code': 'Delete',
 },
 session_id=cdp_session.session_id,
)

 self.logger.debug('■ Text field cleared using triple-click + Delete')
 return True

 except Exception as e:
 self.logger.debug(f'Triple-click clear failed: {e}')

 # Strategy 3: Keyboard shortcuts (last resort)
 try:
 import platform

 is_macos = platform.system() == 'Darwin'
 select_all_modifier = 4 if is_macos else 2 # Meta=4 (Cmd), Ctrl=2
 modifier_name = 'Cmd' if is_macos else 'Ctrl'
 self.logger.debug(f'■ Last resort: Clearing using {modifier_name}+A + Backspace')

```

```

Select all text (Ctrl/Cmd+A)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyDown',
#####'key': 'a',
#####'code': 'KeyA',
#####'modifiers': select_all_modifier,
#####},
#####session_id=cdp_session.session_id,
#####)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyUp',
#####'key': 'a',
#####'code': 'KeyA',
#####'modifiers': select_all_modifier,
#####},
#####session_id=cdp_session.session_id,
#####)

Delete selected text (Backspace)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyDown',
#####'key': 'Backspace',
#####'code': 'Backspace',
#####},
#####session_id=cdp_session.session_id,
#####)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyUp',
#####'key': 'Backspace',
#####'code': 'Backspace',
#####},
#####session_id=cdp_session.session_id,
#####)

#####self.logger.debug('■ Text field cleared using keyboard shortcuts')
#####return True

#####except Exception as e:
#####self.logger.debug(f'All clearing strategies failed: {e}')
#####return False

#####async def _focus_element_simple(
#####self, backend_node_id: int, object_id: str, cdp_session, input_coordinates: dict | None = None
#####) -> bool:
#####"""Simple focus strategy: CDP first, then click if failed."""

Strategy 1: Try CDP DOM.focus first
#####try:
#####result = await cdp_session.cdp_client.send.DOM.focus(
#####params={'backendNodeId': backend_node_id},
#####session_id=cdp_session.session_id,
#####)
#####self.logger.debug(f'Element focused using CDP DOM.focus (result: {result})')
#####return True

#####except Exception as e:
#####self.logger.debug(f'■ CDP DOM.focus threw exception: {type(e).__name__}: {e}')

Strategy 2: Try click to focus if CDP failed
#####if input_coordinates and 'input_x' in input_coordinates and 'input_y' in input_coordinates:
#####try:
#####click_x = input_coordinates['input_x']
#####click_y = input_coordinates['input_y']

#####self.logger.debug(f'■ Attempting click-to-focus at ({click_x:.1f}, {click_y:.1f})')

Click to focus
#####await cdp_session.cdp_client.send.Input.dispatchMouseEvent(

```

```

 params={
 'type': 'mousePressed',
 'x': click_x,
 'y': click_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=cdp_session.session_id,
)
 await cdp_session.cdp_client.send.Input.dispatchMouseEvent(
 params={
 'type': 'mouseReleased',
 'x': click_x,
 'y': click_y,
 'button': 'left',
 'clickCount': 1,
 },
 session_id=cdp_session.session_id,
)

 self.logger.debug('■ Element focused using click method')
 return True

 except Exception as e:
 self.logger.debug(f'Click focus failed: {e}')

 ## Both strategies failed
 self.logger.debug('Focus strategies failed, will attempt typing anyway')
 return False

def _requires_direct_value_assignment(self, element_node: EnhancedDOMTreeNode) -> bool:
 """
 Check if an element requires direct value assignment instead of character-by-character typing.

 Certain input types have compound components, custom plugins, or special requirements
 that make character-by-character typing unreliable. These need direct .value assignment:

 Native HTML5:
 - date, time, datetime-local: Have spinbutton components (ISO format required)
 - month, week: Similar compound structure
 - color: Expects hex format #RRGGBB
 - range: Needs numeric value within min/max

 jQuery/Bootstrap Datepickers:
 - Detected by class names or data attributes
 - Often expect specific date formats (MM/DD/YYYY, DD/MM/YYYY, etc.)

 Note: We use direct assignment because:
 1. Typing triggers intermediate validation that might reject partial values
 2. Compound components (like date spinbuttons) don't work with sequential typing
 3. It's much faster and more reliable
 4. We dispatch proper input/change events afterward to trigger listeners
 """
 if not element_node.tag_name or not element_node.attributes:
 return False

 tag_name = element_node.tag_name.lower()

 ## Check for native HTML5 inputs that need direct assignment
 if tag_name == 'input':
 input_type = element_node.attributes.get('type', '').lower()

 ## Native HTML5 inputs with compound components or strict formats
 if input_type in {'date', 'time', 'datetime-local', 'month', 'week', 'color', 'range'}:
 return True

 ## Detect jQuery/Bootstrap datepickers (text inputs with datepicker plugins)
 if input_type in {'text', ''}:
 ## Check for common datepicker indicators
 class_attr = element_node.attributes.get('class', '').lower()
 if any(
 indicator in class_attr
)

```

```

 for indicator in ['datepicker', 'daterangepicker', 'datetimepicker', 'bootstrap-datepicker']
):
 return True

 # Check for data attributes indicating datepickers
 if any(attr in element_node.attributes for attr in ['data-datepicker', 'data-date-format', 'data-pr
):
 return True

 return False

 async def _set_value_directly(self, element_node: EnhancedDOMTreeNode, text: str, object_id: str, cdp_s
 """
 Set element value directly using JavaScript for inputs that don't support typing.

 This is used for:
 - Date/time inputs where character-by-character typing doesn't work
 - jQuery datepickers that need direct value assignment
 - Color/range inputs that need specific formats
 - Any input with custom plugins that intercept typing

 After setting the value, we dispatch comprehensive events to ensure all frameworks
 and plugins recognize the change (React, Vue, Angular, jQuery, etc.)
 """
 try:
 # Set the value using JavaScript with comprehensive event dispatching
 # callFunctionOn expects a function body (not a self-invoking function)
 set_value_js = f"""
 function() {{
 // Store old value for comparison
 const oldValue = this.value;

 // REACT-COMPATIBLE VALUE SETTING:
 // React uses Object.getOwnPropertyDescriptor to track input changes
 // We need to use the native setter to bypass React's tracking and then trigger events
 const nativeInputValueSetter = Object.getOwnPropertyDescriptor(
 window.HTMLInputElement.prototype,
 'value'
).set;

 // Set the value using the native setter (bypasses React's control)
 nativeInputValueSetter.call(this, {json.dumps(text)});

 // Dispatch comprehensive events to ensure all frameworks detect the change
 // Order matters: focus -> input -> change -> blur (mimics user interaction)

 // 1. Focus event (in case element isn't focused)
 this.dispatchEvent(new FocusEvent('focus', {{ bubbles: true }}});

 // 2. Input event (CRITICAL for React onChange)
 // React listens to 'input' events on the document and checks for value changes
 const inputEvent = new Event('input', {{ bubbles: true, cancelable: true }});
 this.dispatchEvent(inputEvent);

 // 3. Change event (for form handling, traditional listeners)
 const changeEvent = new Event('change', {{ bubbles: true, cancelable: true }});
 this.dispatchEvent(changeEvent);

 // 4. Blur event (triggers final validation in some libraries)
 this.dispatchEvent(new FocusEvent('blur', {{ bubbles: true }}});

 // 5. jQuery-specific events (if jQuery is present)
 if (typeof jQuery !== 'undefined' && jQuery.fn) {{
 try {{
 jQuery(this).trigger('change');
 // Trigger datepicker-specific events if it's a datepicker
 if (jQuery(this).data('datepicker')) {{
 jQuery(this).datepicker('update');
 }}
 }} catch (e) {{
 // jQuery not available or error, continue anyway
 }}
 }}
 }}
 """

```

```

 """return this.value;
 """
 """

 result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'objectId': object_id,
 'functionDeclaration': set_value_js,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)

 # Verify the value was set correctly
 if 'result' in result and 'value' in result['result']:
 actual_value = result['result']['value']
 self.logger.debug(f'Value set directly to: "{actual_value}"')
 else:
 self.logger.warning('Could not verify value was set correctly')

 except Exception as e:
 self.logger.error(f'Failed to set value directly: {e}')
 raise

 async def _input_text_element_node_impl(
 self, element_node: EnhancedDOMTreeNode, text: str, clear: bool = True, is_sensitive: bool = False
) -> dict | None:
 """
 Input text into an element using pure CDP with improved focus fallbacks.

 For date/time inputs, uses direct value assignment instead of typing.
 """

 try:
 # Get CDP client
 cdp_client = self.browser_session.cdp_client

 # Get the correct session ID for the element's iframe
 session_id = await self._get_session_id_for_element(element_node)

 # cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=element_node.target_id)
 cdp_session = await self.browser_session.cdp_client_for_node(element_node)

 # Get element info
 backend_node_id = element_node.backend_node_id

 # Track coordinates for metadata
 input_coordinates = None

 # Scroll element into view
 try:
 await cdp_session.cdp_client.send.DOM.scrollIntoViewIfNeeded(
 params={'backendNodeId': backend_node_id}, session_id=cdp_session.session_id
)
 await asyncio.sleep(0.01)
 except Exception as e:
 # Node detached errors are common with shadow DOM and dynamic content
 # The element can still be interacted with even if scrolling fails
 error_str = str(e)
 if 'Node is detached from document' in error_str or 'detached from document' in error_str:
 self.logger.debug(
 f'Element node temporarily detached during scroll (common with shadow DOM), continuing: {element_node}'
)
 else:
 self.logger.debug(f'Failed to scroll element {element_node} into view before typing: {type(e).__name__}')

 # Get object ID for the element
 result = await cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id},
 session_id=cdp_session.session_id,
)
 assert 'object' in result and 'objectId' in result['object'], (

```

```

#####'Failed to find DOM element based on backendNodeId, maybe page content changed?'
#####)
#####object_id = result['object']['objectId']

Get current coordinates using unified method
#####coords = await self.browser_session.get_element_coordinates(backend_node_id, cdp_session)
#####if coords:
#####center_x = coords.x + coords.width / 2
#####center_y = coords.y + coords.height / 2

Check for occlusion before using coordinates for focus
#####is_occluded = await self._check_element_occlusion(backend_node_id, center_x, center_y, cdp_session)

#####if is_occluded:
#####self.logger.debug('■ Input element is occluded, skipping coordinate-based focus')
#####input_coordinates = None # Force fallback to CDP-only focus
#####else:
#####input_coordinates = {'input_x': center_x, 'input_y': center_y}
#####self.logger.debug(f'Using unified coordinates: x={center_x:.1f}, y={center_y:.1f}')
#####else:
#####input_coordinates = None
#####self.logger.debug('No coordinates found for element')

Ensure we have a valid object_id before proceeding
#####if not object_id:
#####raise ValueError('Could not get object_id for element')

Step 1: Focus the element using simple strategy
#####focused_successfully = await self._focus_element_simple(
#####backend_node_id=backend_node_id, object_id=object_id, cdp_session=cdp_session, input_coordinates=input_coordinates)

Step 2: Check if this element requires direct value assignment (date/time inputs)
#####requires_direct_assignment = self._requires_direct_value_assignment(element_node)

#####if requires_direct_assignment:
Date/time inputs: use direct value assignment instead of typing
#####self.logger.debug(
#####f'■ Element type={element_node.attributes.get("type")} requires direct value assignment, setting
#####)
#####await self._set_value_directly(element_node, text, object_id, cdp_session)

Return input coordinates for metadata
#####return input_coordinates

Step 3: Clear existing text if requested (only for regular inputs that support typing)
#####if clear:
#####cleared_successfully = await self._clear_text_field(object_id=object_id, cdp_session=cdp_session)
#####if not cleared_successfully:
#####self.logger.warning('■ Text field clearing failed, typing may append to existing text')

Step 4: Type the text character by character using proper human-like key events
This emulates exactly how a human would type, which modern websites expect
#####if is_sensitive:
Note: sensitive_key_name is not passed to this low-level method,
but we could extend the signature if needed for more granular logging
#####self.logger.debug('■ Typing <sensitive> character by character')
#####else:
#####self.logger.debug(f'■ Typing text character by character: "{text}"')

Detect contenteditable elements (may have leaf-start bug where first char is dropped)
#####_attrs = element_node.attributes or {}
#####_is_contenteditable = _attrs.get('contenteditable') in ('true', '') or (
#####_attrs.get('role') == 'textbox' and element_node.tag_name not in ('input', 'textarea'))
#####)

For contenteditable: after typing first char, check if dropped and retype if needed
#####_check_first_char = _is_contenteditable and len(text) > 0 and clear
#####_first_char = text[0] if _check_first_char else None

#####for i, char in enumerate(text):
Handle newline characters as Enter key

```

```

#####if char == '\n':
Send proper Enter key sequence
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'keyDown',
'key': 'Enter',
'code': 'Enter',
'windowsVirtualKeyCode': 13,
#####},
#####session_id=cdp_session.session_id,
#####)

Small delay to emulate human typing speed
#####await asyncio.sleep(0.001)

Send char event with carriage return
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'char',
'text': '\r',
'key': 'Enter',
#####},
#####session_id=cdp_session.session_id,
#####)

Send keyUp event
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'keyUp',
'key': 'Enter',
'code': 'Enter',
'windowsVirtualKeyCode': 13,
#####},
#####session_id=cdp_session.session_id,
#####)
#####else:
Handle regular characters
Get proper modifiers, VK code, and base key for the character
#####modifiers, vk_code, base_key = self._get_char_modifiers_and_vk(char)
#####key_code = self._get_key_code_for_char(base_key)

self.logger.debug(f'■ Typing character {i + 1}/{len(text)}: "{char}" (base_key: {base_key}, cod

Step 1: Send keyDown event (NO text parameter)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'keyDown',
'key': base_key,
'code': key_code,
'modifiers': modifiers,
'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)

Small delay to emulate human typing speed
#####await asyncio.sleep(0.005)

Step 2: Send char event (WITH text parameter) - this is crucial for text input
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'char',
'text': char,
'key': char,
#####},
#####session_id=cdp_session.session_id,
#####)

Step 3: Send keyUp event (NO text parameter)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
'type': 'keyUp',

```



```

#####'key': base_key,
#####'code': key_code,
#####'modifiers': modifiers,
#####'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)

After first char on contenteditable: check if dropped and retype if needed
#####if i == 0 and _check_first_char and _first_char:
#####check_result = await cdp_session.cdp_client.send.Runtime.evaluate(
#####params={'expression': 'document.activeElement.textContent'},
#####session_id=cdp_session.session_id,
#####)
#####content = check_result.get('result', {}).get('value', '')
#####if _first_char not in content:
#####self.logger.debug(f'First char "{_first_char}" was dropped (leaf-start bug), retyping')
Retype the first character - cursor now past leaf-start
#####modifiers, vk_code, base_key = self._get_char_modifiers_and_vk(_first_char)
#####key_code = self._get_key_code_for_char(base_key)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyDown',
#####'key': base_key,
#####'code': key_code,
#####'modifiers': modifiers,
#####'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)
#####await asyncio.sleep(0.005)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={'type': 'char', 'text': _first_char, 'key': _first_char},
#####session_id=cdp_session.session_id,
#####)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyUp',
#####'key': base_key,
#####'code': key_code,
#####'modifiers': modifiers,
#####'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)

Small delay between characters to look human (realistic typing speed)
#####await asyncio.sleep(0.001)

Step 4: Trigger framework-aware DOM events after typing completion
Modern JavaScript frameworks (React, Vue, Angular) rely on these events
to update their internal state and trigger re-renders
#####await self._trigger_framework_events(object_id=object_id, cdp_session=cdp_session)

Step 5: Read back actual value for verification (skip for sensitive data)
#####if not is_sensitive:
#####try:
#####await asyncio.sleep(0.05) # let autocomplete/formatter JS settle
#####readback_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'objectId': object_id,
#####'functionDeclaration': 'function() { return this.value !== undefined ? this.value : this.textContent }',
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)
#####actual_value = readback_result.get('result', {}).get('value')
#####if actual_value is not None:
#####if input_coordinates is None:
#####input_coordinates = {}
#####input_coordinates['actual_value'] = actual_value
#####except Exception as e:

```

```

#####self.logger.debug(f'Value readback failed (non-critical): {e}')

Step 6: Auto-retry on concatenation mismatch (only when clear was requested)
If we asked to clear but the readback value contains the typed text as a substring
yet is longer, the field had pre-existing text that wasn't cleared. Set directly.
if clear and not is_sensitive and input_coordinates and 'actual_value' in input_coordinates:
actual_value = input_coordinates['actual_value']
if (
isinstance(actual_value, str)
and actual_value != text
and len(actual_value) > len(text)
and (actual_value.endswith(text) or actual_value.startswith(text))
):
self.logger.info(f'Concatenation detected: got "{actual_value}", expected "{text}" - auto-retry')
try:
Clear + set value via native setter in one JS call (works with React/Vue)
retry_result = await cdp_session.cdp_client.send(Runtime.callFunctionOn(
params={
'objectId': object_id,
'functionDeclaration': """
function(newValue) {
if (this.value !== undefined) {
var desc = Object.getOwnPropertyDescriptor(
HTMLInputElement.prototype, 'value'
) || Object.getOwnPropertyDescriptor(
HTMLTextAreaElement.prototype, 'value'
);
if (desc && desc.set) {
desc.set.call(this, newValue);
} else {
this.value = newValue;
}
} else if (this.isContentEditable) {
this.textContent = newValue;
}
this.dispatchEvent(new Event('input', { bubbles: true }));
this.dispatchEvent(new Event('change', { bubbles: true }));
return this.value !== undefined ? this.value : this.textContent;
}
"""
'arguments': [{ 'value': text }],
'returnByValue': True,
},
session_id=cdp_session.session_id,
))
retry_value = retry_result.get('result', {}).get('value')
if retry_value is not None:
input_coordinates['actual_value'] = retry_value
if retry_value == text:
self.logger.info('Auto-retry fixed concatenation')
else:
self.logger.warning(f'Auto-retry value still differs: "{retry_value}"')
except Exception as e:
self.logger.debug(f'Auto-retry failed (non-critical): {e}')

Return coordinates metadata if available
return input_coordinates

except Exception as e:
self.logger.error(f'Failed to input text via CDP: {type(e).__name__}: {e}')
raise BrowserError(f'Failed to input text into element: {repr(element_node)}')

async def _trigger_framework_events(self, object_id: str, cdp_session) -> None:
"""
Trigger framework-aware DOM events after text input completion.

This is critical for modern JavaScript frameworks (React, Vue, Angular, etc.)
that rely on DOM events to update their internal state and trigger re-renders.

Args:
object_id: CDP object ID of the input element
cdp_session: CDP session for the element's context

```

```

 """
 try:
 # Execute JavaScript to trigger comprehensive event sequence
 framework_events_script = """
 function() {
 // Find the target element (available as 'this' when using objectId)
 const element = this;
 if (!element) return false;

 // Ensure element is focused
 element.focus();

 // Comprehensive event sequence for maximum framework compatibility
 const events = [
 // Input event - primary event for React controlled components
 { type: 'input', bubbles: true, cancelable: true },
 // Change event - important for form validation and Vue v-model
 { type: 'change', bubbles: true, cancelable: true },
 // Blur event - triggers validation in many frameworks
 { type: 'blur', bubbles: true, cancelable: true }
];

 let success = true;

 events.forEach(eventConfig => {
 try {
 const event = new Event(eventConfig.type, {
 bubbles: eventConfig.bubbles,
 cancelable: eventConfig.cancelable
 });

 // Special handling for InputEvent (more specific than Event)
 if (eventConfig.type === 'input') {
 const inputEvent = new InputEvent('input', {
 bubbles: true,
 cancelable: true,
 data: element.value,
 inputType: 'insertText'
 });
 element.dispatchEvent(inputEvent);
 } else {
 element.dispatchEvent(event);
 }
 } catch (e) {
 success = false;
 console.warn('Framework event dispatch failed:', eventConfig.type, e);
 }
 });

 // Special React synthetic event handling
 // React uses internal fiber properties for event system
 if (element._reactInternalFiber || element._reactInternalInstance || element.__reactInternalInstance) {
 try {
 // Trigger React's synthetic event system
 const syntheticInputEvent = new InputEvent('input', {
 bubbles: true,
 cancelable: true,
 data: element.value
 });

 // Force React to process this as a synthetic event
 Object.defineProperty(syntheticInputEvent, 'isTrusted', { value: true });
 element.dispatchEvent(syntheticInputEvent);
 } catch (e) {
 console.warn('React synthetic event failed:', e);
 }
 }

 // Special Vue reactivity trigger
 // Vue uses __vueParentComponent or __vue__ for component access
 if (element.__vue__ || element._vnode || element.__vueParentComponent) {
 try {

```

```

#####// Vue often needs explicit input event with proper timing
#####const vueEvent = new Event('input', { bubbles: true });
#####setTimeout(() => element.dispatchEvent(vueEvent), 0);
#####} catch (e) {
#####console.warn('Vue reactivity trigger failed:', e);
#####}
#####}

#####return success;
#####}
#####"""

Execute the framework events script
#####result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'objectId': object_id,
#####'functionDeclaration': framework_events_script,
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)

#####success = result.get('result', {}).get('value', False)
#####if success:
#####self.logger.debug('■ Framework events triggered successfully')
#####else:
#####self.logger.warning('■ Failed to trigger framework events')

#####except Exception as e:
#####self.logger.warning(f'■ Failed to trigger framework events: {type(e).__name__}: {e}')
Don't raise - framework events are a best-effort enhancement

#####async def _scroll_with_cdp_gesture(self, pixels: int) -> bool:
#####"""
#####Scroll using CDP Input.synthesizeScrollGesture to simulate realistic scroll gesture.

#####Args:
#####pixels: Number of pixels to scroll (positive = down, negative = up)

#####Returns:
#####True if successful, False if failed
#####"""
#####try:
Get focused CDP session using public API (validates and waits for recovery if needed)
#####cdp_session = await self.browser_session.get_or_create_cdp_session()
#####cdp_client = cdp_session.cdp_client
#####session_id = cdp_session.session_id

Get viewport dimensions from cached value if available
#####if self.browser_session._original_viewport_size:
#####viewport_width, viewport_height = self.browser_session._original_viewport_size
#####else:
Fallback: query layout metrics
#####layout_metrics = await cdp_client.send.Page.getLayoutMetrics(session_id=session_id)
#####viewport_width = layout_metrics['layoutViewport']['clientWidth']
#####viewport_height = layout_metrics['layoutViewport']['clientHeight']

Calculate center of viewport
#####center_x = viewport_width / 2
#####center_y = viewport_height / 2

For scroll gesture, positive yDistance scrolls up, negative scrolls down
(opposite of mouseWheel deltaY convention)
#####y_distance = -pixels

Synthesize scroll gesture - use very high speed for near-instant scrolling
#####await cdp_client.send.Input.synthesizeScrollGesture(
#####params={
#####'x': center_x,
#####'y': center_y,
#####'xDistance': 0,
#####'yDistance': y_distance,

```

```

 'speed': 50000, # pixels per second (high = near-instant scroll)
 },
 session_id=session_id,
)

 self.logger.debug(f'Scrolled via CDP gesture: {pixels}px')
 return True

except Exception as e:
 # Not critical - JavaScript fallback will handle scrolling
 self.logger.debug(f'CDP gesture scroll failed ({type(e).__name__}: {e}), falling back to JS')
 return False

async def _scroll_element_container(self, element_node, pixels: int) -> bool:
 """Try to scroll an element's container using CDP."""
 try:
 cdp_session = await self.browser_session.cdp_client_for_node(element_node)

 # Check if this is an iframe - if so, scroll its content directly
 if element_node.tag_name and element_node.tag_name.upper() == 'IFRAME':
 # For iframes, we need to scroll the content document, not the iframe element itself
 # Use JavaScript to directly scroll the iframe's content
 backend_node_id = element_node.backend_node_id

 # Resolve the node to get an object ID
 result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': backend_node_id},
 session_id=cdp_session.session_id,
)

 if 'object' in result and 'objectId' in result['object']:
 object_id = result['object']['objectId']

 # Scroll the iframe's content directly
 scroll_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': f"""
 function() {{
 try {{
 const doc = this.contentDocument || this.contentWindow.document;
 if (doc) {{
 const scrollElement = doc.documentElement || doc.body;
 if (scrollElement) {{
 const oldScrollTop = scrollElement.scrollTop;
 scrollElement.scrollTop += {pixels};
 const newScrollTop = scrollElement.scrollTop;
 return {{
 success: true,
 oldScrollTop: oldScrollTop,
 newScrollTop: newScrollTop,
 scrolled: newScrollTop - oldScrollTop
 }};
 }}
 }}
 }} catch (e) {{
 return {{success: false, error: e.toString()}};
 }}
 }}
 """
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)

 if scroll_result and 'result' in scroll_result and 'value' in scroll_result['result']:
 result_value = scroll_result['result']['value']
 if result_value.get('success'):
 self.logger.debug(f'Successfully scrolled iframe content by {result_value.get("scrolled", 0)}px')
 return True
 else:

```

```

#####self.logger.debug(f'Failed to scroll iframe: {result_value.get("error", "Unknown error")}')

For non-iframe elements, use the standard mouse wheel approach
Get element bounds to know where to scroll
####backend_node_id = element_node.backend_node_id
####box_model = await cdp_session.cdp_client.send.DOM.getBoxModel(
####params={'backendNodeId': backend_node_id}, session_id=cdp_session.session_id
####)
####content_quad = box_model['model']['content']

Calculate center point
####center_x = (content_quad[0] + content_quad[2] + content_quad[4] + content_quad[6]) / 4
####center_y = (content_quad[1] + content_quad[3] + content_quad[5] + content_quad[7]) / 4

Dispatch mouse wheel event at element location
####await cdp_session.cdp_client.send.Input.dispatchMouseEvent(
####params={
#####'type': 'mouseWheel',
#####'x': center_x,
#####'y': center_y,
#####'deltaX': 0,
#####'deltaY': pixels,
#####},
####session_id=cdp_session.session_id,
####)

####return True
####except Exception as e:
####self.logger.debug(f'Failed to scroll element container via CDP: {e}')
####return False

#####async def _get_session_id_for_element(self, element_node: EnhancedDOMTreeNode) -> str | None:
#####"""Get the appropriate CDP session ID for an element based on its frame."""
#####if element_node.frame_id:
Element is in an iframe, need to get session for that frame
#####try:
#####all_targets = self.browser_session.session_manager.get_all_targets()

Find the target for this frame
#####for target_id, target in all_targets.items():
#####if target.target_type == 'iframe' and element_node.frame_id in str(target_id):
Create temporary session for iframe target without switching focus
#####temp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)
#####return temp_session.session_id

If frame not found in targets, use main target session
#####self.logger.debug(f'Frame {element_node.frame_id} not found in targets, using main session')
#####except Exception as e:
#####self.logger.debug(f'Error getting frame session: {e}, using main session')

Use main target session - get_or_create_cdp_session validates focus automatically
#####cdp_session = await self.browser_session.get_or_create_cdp_session()
#####return cdp_session.session_id

#####async def on_GoBackEvent(self, event: GoBackEvent) -> None:
#####"""Handle navigate back request with CDP."""
#####cdp_session = await self.browser_session.get_or_create_cdp_session()
#####try:
Get CDP client and session

Get navigation history
#####history = await cdp_session.cdp_client.send.Page.getNavigationHistory(session_id=cdp_session.session_id)
#####current_index = history['currentIndex']
#####entries = history['entries']

Check if we can go back
#####if current_index <= 0:
#####self.logger.warning('Cannot go back - no previous entry in history')
#####return

Navigate to the previous entry
#####previous_entry_id = entries[current_index - 1]['id']

```

```

 await cdp_session.cdp_client.send.Page.navigateToHistoryEntry(
 params={'entryId': previous_entry_id}, session_id=cdp_session.session_id
)

 # Wait for navigation
 await asyncio.sleep(0.5)
 # Navigation is handled by BrowserSession via events

 self.logger.info(f'Navigated back to {entries[current_index - 1]["url"]}')
 except Exception as e:
 raise

 async def on_GoForwardEvent(self, event: GoForwardEvent) -> None:
 """Handle navigate forward request with CDP."""
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 try:
 # Get navigation history
 history = await cdp_session.cdp_client.send.Page.getNavigationHistory(session_id=cdp_session.session_id)
 current_index = history['currentIndex']
 entries = history['entries']

 # Check if we can go forward
 if current_index >= len(entries) - 1:
 self.logger.warning('Cannot go forward - no next entry in history')
 return

 # Navigate to the next entry
 next_entry_id = entries[current_index + 1]['id']
 await cdp_session.cdp_client.send.Page.navigateToHistoryEntry(
 params={'entryId': next_entry_id}, session_id=cdp_session.session_id
)

 # Wait for navigation
 await asyncio.sleep(0.5)
 # Navigation is handled by BrowserSession via events

 self.logger.info(f'Navigated forward to {entries[current_index + 1]["url"]}')
 except Exception as e:
 raise

 async def on_RefreshEvent(self, event: RefreshEvent) -> None:
 """Handle target refresh request with CDP."""
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 try:
 # Reload the target
 await cdp_session.cdp_client.send.Page.reload(session_id=cdp_session.session_id)

 # Wait for reload
 await asyncio.sleep(1.0)

 # Note: We don't clear cached state here - let the next state fetch rebuild as needed

 # Navigation is handled by BrowserSession via events

 self.logger.info('Target refreshed')
 except Exception as e:
 raise

 @observe_debug(ignore_input=True, ignore_output=True, name='wait_event_handler')
 async def on_WaitEvent(self, event: WaitEvent) -> None:
 """Handle wait request."""
 try:
 # Cap wait time at maximum
 actual_seconds = min(max(event.seconds, 0), event.max_seconds)
 if actual_seconds != event.seconds:
 self.logger.info(f'Waiting for {actual_seconds} seconds (capped from {event.seconds}s)')
 else:
 self.logger.info(f'Waiting for {actual_seconds} seconds')

 await asyncio.sleep(actual_seconds)
 except Exception as e:
 raise

```

```

 async def _dispatch_key_event(self, cdp_session, event_type: str, key: str, modifiers: int = 0) -> None:
 """Helper to dispatch a keyboard event with proper key codes."""
 code, vk_code = get_key_info(key)
 params: DispatchKeyEventParameters = {
 'type': event_type,
 'key': key,
 'code': code,
 }
 if modifiers:
 params['modifiers'] = modifiers
 if vk_code is not None:
 params['windowsVirtualKeyCode'] = vk_code
 await cdp_session.cdp_client.send.Input.dispatchKeyEvent(params=params, session_id=cdp_session.session_id)

 async def on_SendKeysEvent(self, event: SendKeysEvent) -> None:
 """Handle send keys request with CDP."""
 cdp_session = await self.browser_session.get_or_create_cdp_session(focus=True)
 try:
 # Normalize key names from common aliases
 key_aliases = {
 'ctrl': 'Control',
 'control': 'Control',
 'alt': 'Alt',
 'option': 'Alt',
 'meta': 'Meta',
 'cmd': 'Meta',
 'command': 'Meta',
 'shift': 'Shift',
 'enter': 'Enter',
 'return': 'Enter',
 'tab': 'Tab',
 'delete': 'Delete',
 'backspace': 'Backspace',
 'escape': 'Escape',
 'esc': 'Escape',
 'space': ' ',
 'up': 'ArrowUp',
 'down': 'ArrowDown',
 'left': 'ArrowLeft',
 'right': 'ArrowRight',
 'pageup': 'PageUp',
 'pagedown': 'PageDown',
 'home': 'Home',
 'end': 'End',
 }

 # Parse and normalize the key string
 keys = event.keys
 if '+' in keys:
 # Handle key combinations like "ctrl+a"
 parts = keys.split('+')
 normalized_parts = []
 for part in parts:
 part_lower = part.strip().lower()
 normalized = key_aliases.get(part_lower, part)
 normalized_parts.append(normalized)
 normalized_keys = '+'.join(normalized_parts)
 else:
 # Single key
 keys_lower = keys.strip().lower()
 normalized_keys = key_aliases.get(keys_lower, keys)

 # Handle key combinations like "Control+A"
 if '+' in normalized_keys:
 parts = normalized_keys.split('+')
 modifiers = parts[:-1]
 main_key = parts[-1]

 # Calculate modifier bitmask
 modifier_value = 0
 modifier_map = {'Alt': 1, 'Control': 2, 'Meta': 4, 'Shift': 8}
 for mod in modifiers:

```



```

#####modifier_value |= modifier_map.get(mod, 0)

Press modifier keys
#####for mod in modifiers:
#####await self._dispatch_key_event(cdp_session, 'keyDown', mod)

Press main key with modifiers bitmask
#####await self._dispatch_key_event(cdp_session, 'keyDown', main_key, modifier_value)

#####await self._dispatch_key_event(cdp_session, 'keyUp', main_key, modifier_value)

Release modifier keys
#####for mod in reversed(modifiers):
#####await self._dispatch_key_event(cdp_session, 'keyUp', mod)
#####else:
Check if this is a text string or special key
#####special_keys = {
#####'Enter',
#####'Tab',
#####'Delete',
#####'Backspace',
#####'Escape',
#####'ArrowUp',
#####'ArrowDown',
#####'ArrowLeft',
#####'ArrowRight',
#####'PageUp',
#####'PageDown',
#####'Home',
#####'End',
#####'Control',
#####'Alt',
#####'Meta',
#####'Shift',
#####'F1',
#####'F2',
#####'F3',
#####'F4',
#####'F5',
#####'F6',
#####'F7',
#####'F8',
#####'F9',
#####'F10',
#####'F11',
#####'F12',
#####}

If it's a special key, use original logic
#####if normalized_keys in special_keys:
#####await self._dispatch_key_event(cdp_session, 'keyDown', normalized_keys)
For Enter key, also dispatch a char event to trigger keypress listeners
#####if normalized_keys == 'Enter':
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'char',
#####'text': '\r',
#####'key': 'Enter',
#####},
#####session_id=cdp_session.session_id,
#####)
#####await self._dispatch_key_event(cdp_session, 'keyUp', normalized_keys)
#####else:
It's text (single character or string) - send each character as text input
This is crucial for text to appear in focused input fields
#####for char in normalized_keys:
Special-case newline characters to dispatch as Enter
#####if char in ('\n', '\r'):
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'rawKeyDown',
#####'windowsVirtualKeyCode': 13,

```

```

#####'unmodifiedText': '\r',
#####'text': '\r',
#####},
#####session_id=cdp_session.session_id,
#####)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'char',
#####'windowsVirtualKeyCode': 13,
#####'unmodifiedText': '\r',
#####'text': '\r',
#####},
#####session_id=cdp_session.session_id,
#####)
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyUp',
#####'windowsVirtualKeyCode': 13,
#####'unmodifiedText': '\r',
#####'text': '\r',
#####},
#####session_id=cdp_session.session_id,
#####)
#####continue

Get proper modifiers and key info for the character
#####modifiers, vk_code, base_key = self._get_char_modifiers_and_vk(char)
#####key_code = self._get_key_code_for_char(base_key)

Send keyDown
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyDown',
#####'key': base_key,
#####'code': key_code,
#####'modifiers': modifiers,
#####'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)

Send char event with text - this is what makes text appear in input fields
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'char',
#####'text': char,
#####'key': char,
#####},
#####session_id=cdp_session.session_id,
#####)

Send keyUp
#####await cdp_session.cdp_client.send.Input.dispatchKeyEvent(
#####params={
#####'type': 'keyUp',
#####'key': base_key,
#####'code': key_code,
#####'modifiers': modifiers,
#####'windowsVirtualKeyCode': vk_code,
#####},
#####session_id=cdp_session.session_id,
#####)

Small delay between characters (10ms)
#####await asyncio.sleep(0.010)

#####self.logger.info(f'##### Sent keys: {event.keys}')

Note: We don't clear cached state on Enter; multi_act will detect DOM changes
and rebuild explicitly. We still wait briefly for potential navigation.
#####if 'enter' in event.keys.lower() or 'return' in event.keys.lower():
#####await asyncio.sleep(0.1)

```

```

 except Exception as e:
 raise

 async def on_UploadFileEvent(self, event: UploadFileEvent) -> None:
 """Handle file upload request with CDP."""
 try:
 # Use the provided node
 element_node = event.node
 index_for_logging = element_node.backend_node_id or 'unknown'

 # Check if it's a file input
 if not self.browser_session.is_file_input(element_node):
 msg = f'Upload failed - element {index_for_logging} is not a file input.'
 raise BrowserError(message=msg, long_term_memory=msg)

 # Get CDP client and session
 cdp_client = self.browser_session.cdp_client
 session_id = await self._get_session_id_for_element(element_node)

 # Validate file before upload
 if os.path.exists(event.file_path):
 file_size = os.path.getsize(event.file_path)
 if file_size == 0:
 msg = f'Upload failed - file {event.file_path} is empty (0 bytes).'
 raise BrowserError(message=msg, long_term_memory=msg)
 self.logger.debug(f'File {event.file_path} validated ({file_size} bytes)')

 # Set file(s) to upload
 backend_node_id = element_node.backend_node_id
 await cdp_client.send.DOM.setFileInputFiles(
 params={
 'files': [event.file_path],
 'backendNodeId': backend_node_id,
 },
 session_id=session_id,
)

 self.logger.info(f'Uploaded file {event.file_path} to element {index_for_logging}')
 except Exception as e:
 raise

 async def on_ScrollToTextEvent(self, event: ScrollToTextEvent) -> None:
 """Handle scroll to text request with CDP. Raises exception if text not found."""

 # TODO: handle looking for text inside cross-origin iframes as well

 # Get focused CDP session using public API (validates and waits for recovery if needed)
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 cdp_client = cdp_session.cdp_client
 session_id = cdp_session.session_id

 # Enable DOM
 await cdp_client.send.DOM.enable(session_id=session_id)

 # Get document
 doc = await cdp_client.send.DOM.getDocument(params={'depth': -1}, session_id=session_id)
 root_node_id = doc['root']['nodeId']

 # Search for text using XPath
 search_queries = [
 f'//*[contains(text(), "{event.text}")]',
 f'//*[contains(., "{event.text}")]',
 f'//*[@*[contains(., "{event.text}")]]',
]

 found = False
 for query in search_queries:
 try:
 # Perform search
 search_result = await cdp_client.send.DOM.performSearch(params={'query': query}, session_id=session_id)
 search_id = search_result['searchId']
 result_count = search_result['resultCount']

```

```

#####if result_count > 0:
Get the first match
node_ids = await cdp_client.send.DOM.getSearchResults(
params={'searchId': search_id, 'fromIndex': 0, 'toIndex': 1},
session_id=session_id,
)

#####if node_ids['nodeIds']:
node_id = node_ids['nodeIds'][0]

Scroll the element into view
await cdp_client.send.DOM.scrollToViewIfNeeded(params={'nodeId': node_id}, session_id=session_id)

found = True
self.logger.debug(f'■ Scrolled to text: "{event.text}")')
break

Clean up search
await cdp_client.send.DOM.discardSearchResults(params={'searchId': search_id}, session_id=session_id)
except Exception as e:
self.logger.debug(f'Search query failed: {query}, error: {e}')
continue

#####if not found:
Fallback: Try JavaScript search
js_result = await cdp_client.send.Runtime.evaluate(
params={
'expression': f'''
(() => {{
const walker = document.createTreeWalker(
document.body,
NodeFilter.SHOW_TEXT,
null,
false
});
let node;
while (node = walker.nextNode()) {{
if (node.textContent.includes("{event.text}") {{
node.parentElement.scrollToView({{behavior: 'smooth', block: 'center'}});
return true;
}}
}}
return false;
}})()
'''
},
session_id=session_id,
)

#####if js_result.get('result', {}).get('value'):
self.logger.debug(f'■ Scrolled to text: "{event.text}" (via JS)')
return None
#####else:
self.logger.warning(f'■ Text not found: "{event.text}")')
raise BrowserError(f'Text not found: "{event.text}"', details={'text': event.text})

If we got here and found is True, return None (success)
if found:
return None
else:
raise BrowserError(f'Text not found: "{event.text}"', details={'text': event.text})

#####async def on_GetDropdownOptionsEvent(self, event: GetDropdownOptionsEvent) -> dict[str, str]:
"""Handle get dropdown options request with CDP."""
try:
Use the provided node
element_node = event.node
index_for_logging = element_node.backend_node_id or 'unknown'

Get CDP session for this node
cdp_session = await self.browser_session.cdp_client_for_node(element_node)
Convert node to object ID for CDP operations

```

```

try:
 object_result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': element_node.backend_node_id}, session_id=cdp_session.session_id
)
 remote_object = object_result.get('object', {})
 object_id = remote_object.get('objectId')
 if not object_id:
 raise ValueError('Could not get object ID from resolved node')
 except Exception as e:
 raise ValueError(f'Failed to resolve node to object: {e}') from e

 # Check if this is an ARIA combobox that needs expansion
 # ARIA comboboxes have options in a separate element referenced by aria-controls
 check_combobox_script = """
function() {
 const element = this;
 const role = element.getAttribute('role');
 const ariaControls = element.getAttribute('aria-controls');
 const ariaExpanded = element.getAttribute('aria-expanded');

 if (role === 'combobox' && ariaControls) {
 return {
 isCombobox: true,
 ariaControls: ariaControls,
 isExpanded: ariaExpanded === 'true',
 tagName: element.tagName.toLowerCase()
 };
 }
 return { isCombobox: false };
}
"""

 combobox_check = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': check_combobox_script,
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)
 combobox_info = combobox_check.get('result', {}).get('value', {})

 # If it's an ARIA combobox with aria-controls, handle it specially
 if combobox_info.get('isCombobox'):
 return await self._handle_aria_combobox_options(cdp_session, object_id, combobox_info, index_for_lo

 # Use JavaScript to extract dropdown options (existing logic for non-combobox elements)
 options_script = """
function() {
 const startElement = this;

 // Function to check if an element is a dropdown and extract options
 function checkDropdownElement(element) {
 // Check if it's a native select element
 if (element.tagName.toLowerCase() === 'select') {
 return {
 type: 'select',
 options: Array.from(element.options).map((opt, idx) => ({
 text: opt.text.trim(),
 value: opt.value,
 index: idx,
 selected: opt.selected
 })),
 id: element.id || '',
 name: element.name || '',
 source: 'target'
 };
 }
 }

 // Check if it's an ARIA dropdown/menu (not combobox - handled separately)
 const role = element.getAttribute('role');
 if (role === 'menu' || role === 'listbox') {

```

```

 // Find all menu items/options
 const menuItems = element.querySelectorAll('[role="menuitem"], [role="option"]');
 const options = [];

 menuItems.forEach((item, idx) => {
 const text = item.textContent ? item.textContent.trim() : '';
 if (text) {
 options.push({
 text: text,
 value: item.getAttribute('data-value') || text,
 index: idx,
 selected: item.getAttribute('aria-selected') === 'true' || item.classList.contains('selected')
 });
 }
 });

 return {
 type: 'aria',
 options: options,
 id: element.id || '',
 name: element.getAttribute('aria-label') || '',
 source: 'target'
 };

 // Check if it's a Semantic UI dropdown or similar
 if (element.classList.contains('dropdown') || element.classList.contains('ui')) {
 const menuItems = element.querySelectorAll('.item, .option, [data-value]');
 const options = [];

 menuItems.forEach((item, idx) => {
 const text = item.textContent ? item.textContent.trim() : '';
 if (text) {
 options.push({
 text: text,
 value: item.getAttribute('data-value') || text,
 index: idx,
 selected: item.classList.contains('selected') || item.classList.contains('active')
 });
 }
 });

 if (options.length > 0) {
 return {
 type: 'custom',
 options: options,
 id: element.id || '',
 name: element.getAttribute('aria-label') || '',
 source: 'target'
 };
 }
 }

 return null;
 }

 // Function to recursively search children up to specified depth
 function searchChildrenForDropdowns(element, maxDepth, currentDepth = 0) {
 if (currentDepth >= maxDepth) return null;

 // Check all direct children
 for (let child of element.children) {
 // Check if this child is a dropdown
 const result = checkDropdownElement(child);
 if (result) {
 result.source = `child-depth-${currentDepth + 1}`;
 return result;
 }
 }

 // Recursively check this child's children
 const childResult = searchChildrenForDropdowns(child, maxDepth, currentDepth + 1);
 if (childResult) {

```

```

 return childResult;
 }

 return null;
}

// First check the target element itself
let dropdownResult = checkDropdownElement(startElement);
if (dropdownResult) {
 return dropdownResult;
}

// If target element is not a dropdown, search children up to depth 4
dropdownResult = searchChildrenForDropdowns(startElement, 4);
if (dropdownResult) {
 return dropdownResult;
}

return {
 error: `Element and its children (depth 4) are not recognizable dropdown types (tag: ${startElement.tagName})`
};
}

result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 {
 params: {
 'functionDeclaration': options_script,
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id: cdp_session.session_id,
 }
)

dropdown_data = result.get('result', {}).get('value', {})

if dropdown_data.get('error'):
 raise BrowserError(message=dropdown_data['error'], long_term_memory=dropdown_data['error'])

if not dropdown_data.get('options'):
 msg = f'No options found in dropdown at index {index_for_logging}'
 return {
 'error': msg,
 'short_term_memory': msg,
 'long_term_memory': msg,
 'backend_node_id': str(index_for_logging),
 }

Format options for display
formatted_options = []
for opt in dropdown_data['options']:
 # Use JSON encoding to ensure exact string matching
 encoded_text = json.dumps(opt['text'])
 status = ' (selected)' if opt.get('selected') else ''
 formatted_options.append(f'{opt["index"]}: text={encoded_text}, value={json.dumps(opt["value"])}{status}')

dropdown_type = dropdown_data.get('type', 'select')
element_info = f'Index: {index_for_logging}, Type: {dropdown_type}, ID: {dropdown_data.get("id", "unknown")}'
source_info = dropdown_data.get('source', 'unknown')

if source_info == 'target':
 msg = f'Found {dropdown_type} dropdown ({element_info}):' + '\n' + '\n'.join(formatted_options)
else:
 msg = f'Found {dropdown_type} dropdown in {source_info} ({element_info}):' + '\n' + '\n'.join(formatted_options)
 msg += (
 f'\n\nUse the exact text or value string (without quotes) in select_dropdown(index={index_for_logging}, value={value})'
)

if source_info == 'target':
 self.logger.info(f'Found {len(dropdown_data["options"])} dropdown options for index {index_for_logging}')
else:
 self.logger.info(

```

```

#####f' Found {len(dropdown_data["options"])} dropdown options for index {index_for_logging} in {source}
#####)

Create structured memory for the response
####short_term_memory = msg
####long_term_memory = f'Got dropdown options for index {index_for_logging}'

Return the dropdown data as a dict with structured memory
####return {
#####'type': dropdown_type,
#####'options': json.dumps(dropdown_data['options']), # Convert list to JSON string for dict[str, str]
#####'element_info': element_info,
#####'source': source_info,
#####'formatted_options': '\n'.join(formatted_options),
#####'message': msg,
#####'short_term_memory': short_term_memory,
#####'long_term_memory': long_term_memory,
#####'backend_node_id': str(index_for_logging),
#####}

####except BrowserError:
Re-raise BrowserError as-is to preserve structured memory
####raise
####except TimeoutError:
####msg = f'Failed to get dropdown options for index {index_for_logging} due to timeout.'
####self.logger.error(msg)
####raise BrowserError(message=msg, long_term_memory=msg)
####except Exception as e:
####msg = 'Failed to get dropdown options'
####error_msg = f'{msg}: {str(e)}'
####self.logger.error(error_msg)
####raise BrowserError(
####message=error_msg, long_term_memory=f'Failed to get dropdown options for index {index_for_logging}.
####)

####async def _handle_aria_combobox_options(
####self,
####cdp_session,
####object_id: str,
####combobox_info: dict,
####index_for_logging: int | str,
####) -> dict[str, str]:
####"""Handle ARIA combobox elements with options in a separate listbox element.

####ARIA comboboxes (role="combobox") have options in a separate element referenced
####by aria-controls. Options may only be rendered when the combobox is expanded.

####This method:
####1. Expands the combobox if collapsed (by clicking/focusing it)
####2. Waits for options to render
####3. Finds options in the aria-controls referenced element
####4. Collapses the combobox after extracting options
####"""
####aria_controls_id = combobox_info.get('ariaControls')
####was_expanded = combobox_info.get('isExpanded', False)

If combobox is collapsed, expand it first to trigger option rendering
####if not was_expanded:
Use more robust expansion: dispatch proper DOM events that trigger event listeners
####expand_script = """
####function() {
#####const element = this;
#####
#####// Dispatch focus event properly
#####const focusEvent = new FocusEvent('focus', { bubbles: true, cancelable: true });
#####element.dispatchEvent(focusEvent);
#####
#####// Also call native focus
#####element.focus();
#####
#####// Dispatch focusin event (bubbles, unlike focus)
#####const focusInEvent = new FocusEvent('focusin', { bubbles: true, cancelable: true });

```



```

element.dispatchEvent(focusInEvent);
// For some comboboxes, a click is needed
const clickEvent = new MouseEvent('click', {
 bubbles: true,
 cancelable: true,
 view: window
});
element.dispatchEvent(clickEvent);
// Some comboboxes respond to mousedown
const mousedownEvent = new MouseEvent('mousedown', {
 bubbles: true,
 cancelable: true,
 view: window
});
element.dispatchEvent(mousedownEvent);
return {
 success: true,
 ariaExpanded: element.getAttribute('aria-expanded')
};
}
"""
await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': expand_script,
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)
await asyncio.sleep(0.5)

Now extract options from the aria-controls referenced element
extract_options_script = """
function(ariaControlsId) {
 const combobox = this;

 // Find the listbox element referenced by aria-controls
 const listbox = document.getElementById(ariaControlsId);

 if (!listbox) {
 return {
 error: `Could not find listbox element with id "${ariaControlsId}" referenced by aria-controls`,
 ariaControlsId: ariaControlsId
 };
 }

 // Find all option elements in the listbox
 const optionElements = listbox.querySelectorAll('[role="option"]');
 const options = [];

 optionElements.forEach((item, idx) => {
 const text = item.textContent ? item.textContent.trim() : '';
 if (text) {
 options.push({
 text: text,
 value: item.getAttribute('data-value') || item.getAttribute('value') || text,
 index: idx,
 selected: item.getAttribute('aria-selected') === 'true' || item.classList.contains('selected')
 });
 }
 });

 // If no options with role="option", try other common patterns
 if (options.length === 0) {
 // Try li elements inside
 const liElements = listbox.querySelectorAll('li');
 liElements.forEach((item, idx) => {
 const text = item.textContent ? item.textContent.trim() : '';
 if (text) {

```

```

#####options.push({
#####text: text,
#####value: item.getAttribute('data-value') || item.getAttribute('value') || text,
#####index: idx,
#####selected: item.getAttribute('aria-selected') === 'true' || item.classList.contains('selected')
#####});
#####}
#####});
#####}
#####}
#####return {
#####type: 'aria-combobox',
#####options: options,
#####id: combobox.id || '',
#####name: combobox.getAttribute('aria-label') || combobox.getAttribute('name') || '',
#####listboxId: ariaControlsId,
#####source: 'aria-controls'
#####};
#####}
#####}
#####"""

#####result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': extract_options_script,
#####'objectId': object_id,
#####'arguments': [{ 'value': aria_controls_id }],
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)

#####dropdown_data = result.get('result', {}).get('value', {})

Collapse the combobox if we expanded it (blur to close)
#####if not was_expanded:
#####collapse_script = """
#####function() {
#####this.blur();
#####// Also dispatch escape key to close dropdowns
#####const escEvent = new KeyboardEvent('keydown', { key: 'Escape', bubbles: true });
#####this.dispatchEvent(escEvent);
#####return true;
#####}
#####}
#####"""
#####await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': collapse_script,
#####'objectId': object_id,
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)

Handle errors
#####if dropdown_data.get('error'):
#####raise BrowserError(message=dropdown_data['error'], long_term_memory=dropdown_data['error'])

#####if not dropdown_data.get('options'):
#####msg = f'No options found in ARIA combobox at index {index_for_logging} (listbox: {aria_controls_id})'
#####return {
#####'error': msg,
#####'short_term_memory': msg,
#####'long_term_memory': msg,
#####'backend_node_id': str(index_for_logging),
#####}

Format options for display
#####formatted_options = []
#####for opt in dropdown_data['options']:
#####encoded_text = json.dumps(opt['text'])
#####status = ' (selected)' if opt.get('selected') else ''
#####formatted_options.append(f'{opt["index"]}: text={encoded_text}, value={json.dumps(opt["value"])}{sta

```

```

 dropdown_type = dropdown_data.get('type', 'aria-combobox')
 element_info = f'Index: {index_for_logging}, Type: {dropdown_type}, ID: {dropdown_data.get("id", "none")}'
 source_info = f'aria-controls → {aria_controls_id}'

 msg = f'Found {dropdown_type} dropdown ({element_info}): \n' + '\n'.join(formatted_options)
 msg += f'\n\nUse the exact text or value string (without quotes) in select_dropdown(index={index_for_logging})'

 self.logger.info(f'Found {len(dropdown_data["options"])} options in ARIA combobox at index {index_for_logging}')

 return {
 'type': dropdown_type,
 'options': json.dumps(dropdown_data['options']),
 'element_info': element_info,
 'source': source_info,
 'formatted_options': '\n'.join(formatted_options),
 'message': msg,
 'short_term_memory': msg,
 'long_term_memory': f'Got dropdown options for ARIA combobox at index {index_for_logging}',
 'backend_node_id': str(index_for_logging),
 }

 async def on_SelectDropdownOptionEvent(self, event: SelectDropdownOptionEvent) -> dict[str, str]:
 """Handle select dropdown option request with CDP."""
 try:
 # Use the provided node
 element_node = event.node
 index_for_logging = element_node.backend_node_id or 'unknown'
 target_text = event.text

 # Get CDP session for this node
 cdp_session = await self.browser_session.cdp_client_for_node(element_node)

 # Convert node to object ID for CDP operations
 try:
 object_result = await cdp_session.cdp_client.send.DOM.resolveNode(
 params={'backendNodeId': element_node.backend_node_id}, session_id=cdp_session.session_id
)
 remote_object = object_result.get('object', {})
 object_id = remote_object.get('objectId')
 if not object_id:
 raise ValueError('Could not get object ID from resolved node')
 except Exception as e:
 raise ValueError(f'Failed to resolve node to object: {e}') from e

 try:
 # Use JavaScript to select the option
 selection_script = """
 function(targetText) {
 const startElement = this;

 // Function to attempt selection on a dropdown element
 function attemptSelection(element) {
 // Handle native select elements
 if (element.tagName.toLowerCase() === 'select') {
 const options = Array.from(element.options);
 const targetTextLower = targetText.toLowerCase();

 for (const option of options) {
 const optionTextLower = option.text.trim().toLowerCase();
 const optionValueLower = option.value.toLowerCase();

 // Match against both text and value (case-insensitive)
 if (optionTextLower === targetTextLower || optionValueLower === targetTextLower) {
 const expectedValue = option.value;

 // Focus the element FIRST (important for Svelte/Vue/React and other reactive frameworks)
 // This simulates the user focusing on the dropdown before changing it
 element.focus();

 // Then set the value using multiple methods for maximum compatibility
 element.value = expectedValue;
 option.selected = true;
 }
 }
 }
 }
 """

```

```

element.selectedIndex = option.index;

// Trigger all necessary events for reactive frameworks
// 1. input event - critical for Vue's v-model and Svelte's bind:value
const inputEvent = new Event('input', { bubbles: true, cancelable: true });
element.dispatchEvent(inputEvent);

// 2. change event - traditional form validation and framework reactivity
const changeEvent = new Event('change', { bubbles: true, cancelable: true });
element.dispatchEvent(changeEvent);

// 3. blur event - completes the interaction, triggers validation
element.blur();

// Verification: Check if the selection actually stuck (avoid intercepting and resetting the value)
if (element.value !== expectedValue) {
 // Selection was reverted - need to try clicking instead
 return {
 success: false,
 error: `Selection was set but reverted by page framework. The dropdown may require clicking again`,
 selectionReverted: true,
 targetOption: {
 text: option.text.trim(),
 value: expectedValue,
 index: option.index
 },
 availableOptions: Array.from(element.options).map(opt => ({
 text: opt.text.trim(),
 value: opt.value
 })))
 };
}

return {
 success: true,
 message: `Selected option: ${option.text.trim()} (value: ${option.value})`,
 value: option.value
};
}

// Return available options as separate field
const availableOptions = options.map(opt => ({
 text: opt.text.trim(),
 value: opt.value
})));

return {
 success: false,
 error: `Option with text or value '${targetText}' not found in select element`,
 availableOptions: availableOptions
};
}

// Handle ARIA dropdowns/menus
const role = element.getAttribute('role');
if (role === 'menu' || role === 'listbox' || role === 'combobox') {
 const menuItems = element.querySelectorAll('[role="menuitem"], [role="option"]');
 const targetTextLower = targetText.toLowerCase();

 for (const item of menuItems) {
 if (item.textContent) {
 const itemTextLower = item.textContent.trim().toLowerCase();
 const itemValueLower = (item.getAttribute('data-value') || '').toLowerCase();

 // Match against both text and data-value (case-insensitive)
 if (itemTextLower === targetTextLower || itemValueLower === targetTextLower) {
 // Clear previous selections
 menuItems.forEach(mi => {
 mi.setAttribute('aria-selected', 'false');
 mi.classList.remove('selected');
 });
 }
 }
 }
}

```

```

#####// Select this item
#####item.setAttribute('aria-selected', 'true');
#####item.classList.add('selected');

#####// Trigger click and change events
#####item.click();
#####const clickEvent = new MouseEvent('click', { view: window, bubbles: true, cancelable: true });
#####item.dispatchEvent(clickEvent);

#####return {
#####success: true,
#####message: `Selected ARIA menu item: ${item.textContent.trim()}`
#####};
#####}
#####}
#####}

#####// Return available options as separate field
#####const availableOptions = Array.from(menuItems).map(item => ({
#####text: item.textContent ? item.textContent.trim() : '',
#####value: item.getAttribute('data-value') || ''
#####})).filter(opt => opt.text || opt.value);

#####return {
#####success: false,
#####error: `Menu item with text or value '${targetText}' not found`,
#####availableOptions: availableOptions
#####};
#####}

#####// Handle Semantic UI or custom dropdowns
#####if (element.classList.contains('dropdown') || element.classList.contains('ui')) {
#####const menuItems = element.querySelectorAll('.item, .option, [data-value]');
#####const targetTextLower = targetText.toLowerCase();

#####for (const item of menuItems) {
#####if (item.textContent) {
#####const itemTextLower = item.textContent.trim().toLowerCase();
#####const itemValueLower = (item.getAttribute('data-value') || '').toLowerCase();

#####// Match against both text and data-value (case-insensitive)
#####if (itemTextLower === targetTextLower || itemValueLower === targetTextLower) {
#####// Clear previous selections
#####menuItems.forEach(mi => {
#####mi.classList.remove('selected', 'active');
#####});

#####// Select this item
#####item.classList.add('selected', 'active');

#####// Update dropdown text if there's a text element
#####const textElement = element.querySelector('.text');
#####if (textElement) {
#####textElement.textContent = item.textContent.trim();
#####}

#####// Trigger click and change events
#####item.click();
#####const clickEvent = new MouseEvent('click', { view: window, bubbles: true, cancelable: true });
#####item.dispatchEvent(clickEvent);

#####// Also dispatch on the main dropdown element
#####const dropdownChangeEvent = new Event('change', { bubbles: true });
#####element.dispatchEvent(dropdownChangeEvent);

#####return {
#####success: true,
#####message: `Selected custom dropdown item: ${item.textContent.trim()}`
#####};
#####}
#####}
#####}

```

```

#####// Return available options as separate field
#####const availableOptions = Array.from(menuItems).map(item => ({
#####text: item.textContent ? item.textContent.trim() : '',
#####value: item.getAttribute('data-value') || ''
#####})).filter(opt => opt.text || opt.value);

#####return {
#####success: false,
#####error: `Custom dropdown item with text or value '${targetText}' not found`,
#####availableOptions: availableOptions
#####};
#####}

#####return null; // Not a dropdown element
#####}

#####// Function to recursively search children for dropdowns
#####function searchChildrenForSelection(element, maxDepth, currentDepth = 0) {
#####if (currentDepth >= maxDepth) return null;

#####// Check all direct children
#####for (let child of element.children) {
#####// Try selection on this child
#####const result = attemptSelection(child);
#####if (result && result.success) {
#####return result;
#####}

#####// Recursively check this child's children
#####const childResult = searchChildrenForSelection(child, maxDepth, currentDepth + 1);
#####if (childResult && childResult.success) {
#####return childResult;
#####}
#####}

#####return null;
#####}

#####// First try the target element itself
#####let selectionResult = attemptSelection(startElement);
#####if (selectionResult) {
#####// If attemptSelection returned a result (success or failure), use it
#####// Don't search children if we found a dropdown element but selection failed
#####return selectionResult;
#####}

#####// Only search children if target element is not a dropdown element
#####selectionResult = searchChildrenForSelection(startElement, 4);
#####if (selectionResult && selectionResult.success) {
#####return selectionResult;
#####}

#####return {
#####success: false,
#####error: `Element and its children (depth 4) do not contain a dropdown with option '${targetText}'
#####};
#####}
#####""

#####result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': selection_script,
#####'arguments': [{ 'value': target_text }],
#####'objectId': object_id,
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)

#####selection_result = result.get('result', {}).get('value', {})

If selection failed and all options are empty, the dropdown may be lazily populated.

```

```

Focus the element (triggers lazy loaders) and retry once after a wait.
#####if not selection_result.get('success'):
#####available_options = selection_result.get('availableOptions', [])
#####all_empty = available_options and all(
#####(not opt.get('text', '').strip() and not opt.get('value', '').strip())
#####if isinstance(opt, dict)
#####else not str(opt).strip()
#####for opt in available_options
#####)
#####if all_empty:
#####self.logger.info(
#####'All dropdown options are empty – options may be lazily loaded. Focusing element and retrying')
#####)

Use element.focus() only – no synthetic mouse events that leak isTrusted=false
#####try:
#####await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': 'function() { this.focus(); }',
#####'objectId': object_id,
#####},
#####session_id=cdp_session.session_id,
#####)
#####except Exception:
#####pass # non-fatal, best-effort

#####await asyncio.sleep(1.0)

#####retry_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
#####params={
#####'functionDeclaration': selection_script,
#####'arguments': [{ 'value': target_text }],
#####'objectId': object_id,
#####'returnByValue': True,
#####},
#####session_id=cdp_session.session_id,
#####)
#####selection_result = retry_result.get('result', {}).get('value', {})

Check if selection was reverted by framework - try clicking as fallback
#####if selection_result.get('selectionReverted'):
#####self.logger.info('Selection was reverted by page framework, trying click fallback...')
#####target_option = selection_result.get('targetOption', {})
#####option_index = target_option.get('index', 0)

Try clicking on the option element directly
#####click_fallback_script = """
#####function(optionIndex) {
#####const select = this;
#####if (select.tagName.toLowerCase() !== 'select') return { success: false, error: 'Not a select element' };
#####const option = select.options[optionIndex];
#####if (!option) return { success: false, error: 'Option not found at index ' + optionIndex };

#####// Method 1: Try using the native selectedIndex setter with a small delay
#####const originalValue = select.value;

#####// Simulate opening the dropdown (some frameworks need this)
#####select.focus();
#####const mouseDown = new MouseEvent('mousedown', { bubbles: true, cancelable: true, view: window });
#####select.dispatchEvent(mouseDown);

#####// Set using selectedIndex (more reliable for some frameworks)
#####select.selectedIndex = optionIndex;

#####// Click the option
#####option.selected = true;
#####const optionClick = new MouseEvent('click', { bubbles: true, cancelable: true, view: window });
#####option.dispatchEvent(optionClick);

#####// Close dropdown
#####const mouseUp = new MouseEvent('mouseup', { bubbles: true, cancelable: true, view: window });

```

```

 select.dispatchEvent(mouseUp);

 // Fire change event
 const changeEvent = new Event('change', { bubbles: true, cancelable: true });
 select.dispatchEvent(changeEvent);

 // Blur to finalize
 select.blur();

 // Verify
 if (select.value === option.value || select.selectedIndex === optionIndex) {
 return {
 success: true,
 message: 'Selected via click fallback: ' + option.text.trim(),
 value: option.value
 };
 }

 return {
 success: false,
 error: 'Click fallback also failed - framework may block all programmatic selection',
 finalValue: select.value,
 expectedValue: option.value
 };
 }
}

fallback_result = await cdp_session.cdp_client.send.Runtime.callFunctionOn(
 params={
 'functionDeclaration': click_fallback_script,
 'arguments': [{ 'value': option_index }],
 'objectId': object_id,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)

fallback_data = fallback_result.get('result', {}).get('value', {})
if fallback_data.get('success'):
 msg = fallback_data.get('message', f'Selected option via click: {target_text}')
 self.logger.info(f'█ {msg}')
 return {
 'success': 'true',
 'message': msg,
 'value': fallback_data.get('value', target_text),
 'backend_node_id': str(index_for_logging),
 }
else:
 self.logger.warning(f'█ Click fallback also failed: {fallback_data.get("error", "unknown")}')
 # Continue to error handling below

if selection_result.get('success'):
 msg = selection_result.get('message', f'Selected option: {target_text}')
 self.logger.debug(f'{msg}')

Return the result as a dict
return {
 'success': 'true',
 'message': msg,
 'value': selection_result.get('value', target_text),
 'backend_node_id': str(index_for_logging),
}
else:
 error_msg = selection_result.get('error', f'Failed to select option: {target_text}')
 available_options = selection_result.get('availableOptions', [])
 self.logger.error(f'█ {error_msg}')
 self.logger.debug(f'Available options from JavaScript: {available_options}')

If we have available options, return structured error data
if available_options:
 # Format options for short_term_memory (simple bulleted list)
 short_term_options = []

```



```

#####for opt in available_options:
#####if isinstance(opt, dict):
#####text = opt.get('text', '').strip()
#####value = opt.get('value', '').strip()
#####if text:
#####short_term_options.append(f'- {text}')
#####elif value:
#####short_term_options.append(f'- {value}')
#####elif isinstance(opt, str):
#####short_term_options.append(f'- {opt}')

#####if short_term_options:
#####short_term_memory = 'Available dropdown options are:\n' + '\n'.join(short_term_options)
#####long_term_memory = (
#####f"Couldn't select the dropdown option as '{target_text}' is not one of the available options."
#####)

Return error result with structured memory instead of raising exception
#####return {
#####'success': 'false',
#####'error': error_msg,
#####'short_term_memory': short_term_memory,
#####'long_term_memory': long_term_memory,
#####'backend_node_id': str(index_for_logging),
#####}

Fallback to regular error result if no available options
#####return {
#####'success': 'false',
#####'error': error_msg,
#####'backend_node_id': str(index_for_logging),
#####}

#####except Exception as e:
#####error_msg = f'Failed to select dropdown option: {str(e)}'
#####self.logger.error(error_msg)
#####raise ValueError(error_msg) from e

#####except Exception as e:
#####error_msg = f'Failed to select dropdown option "{target_text}" for element {index_for_logging}: {str(e)}'
#####self.logger.error(error_msg)
#####raise ValueError(error_msg) from e

```

## **`.system\browser\watchdogs\dom_watchdog.py`**

```

"""DOM watchdog for browser DOM tree management using CDP."""

import asyncio
import time
from typing import TYPE_CHECKING

from system.browser.events import (
 BrowserErrorEvent,
 BrowserStateRequestEvent,
 ScreenshotEvent,
 TabCreatedEvent,
)
from system.browser.watchdog_base import BaseWatchdog
from system.dom.service import DomService
from system.dom.views import (
 EnhancedDOMTreeNode,
 SerializedDOMState,
)
from system.observability import observe_debug
from system.utils import create_task_with_error_handling, time_execution_async

if TYPE_CHECKING:
 from system.browser.views import BrowserStateSummary, NetworkRequest, PageInfo, PaginationButton

```

```

class DOMWatchdog(BaseWatchdog):
 """Handles DOM tree building, serialization, and element access via CDP.

 This watchdog acts as a bridge between the event-driven browser session
 and the DomService implementation, maintaining cached state and providing
 helper methods for other watchdogs.
 """

 LISTENS_TO = [TabCreatedEvent, BrowserStateRequestEvent]
 EMITS = [BrowserErrorEvent]

 # Public properties for other watchdogs
 selector_map: dict[int, EnhancedDOMTreeNode] | None = None
 current_dom_state: SerializedDOMState | None = None
 enhanced_dom_tree: EnhancedDOMTreeNode | None = None

 # Internal DOM service
 _dom_service: DomService | None = None

 # Network tracking - maps request_id to (url, start_time, method, resource_type)
 _pending_requests: dict[str, tuple[str, float, str, str | None]] = {}

 @async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 """# self.logger.debug('Setting up init scripts in browser')
 """return None

 def _get_recent_events_str(self, limit: int = 10) -> str | None:
 """Get the most recent events from the event bus as JSON.

 Args:
 limit: Maximum number of recent events to include

 Returns:
 JSON string of recent events or None if not available
 """
 import json

 try:
 """# Get all events from history, sorted by creation time (most recent first)
 """all_events = sorted(
 """self.browser_session.event_bus.event_history.values(), key=lambda e: e.event_created_at.timestamp()
 """
)

 """# Take the most recent events and create JSON-serializable data
 """recent_events_data = []
 """for event in all_events[:limit]:
 """event_data = {
 """ 'event_type': event.event_type,
 """ 'timestamp': event.event_created_at.isoformat(),
 """}
 """# Add specific fields for certain event types
 """if hasattr(event, 'url'):
 """event_data['url'] = getattr(event, 'url')
 """if hasattr(event, 'error_message'):
 """event_data['error_message'] = getattr(event, 'error_message')
 """if hasattr(event, 'target_id'):
 """event_data['target_id'] = getattr(event, 'target_id')
 """recent_events_data.append(event_data)

 """return json.dumps(recent_events_data) # Return empty array if no events
 """except Exception as e:
 """self.logger.debug(f'Failed to get recent events: {e}')

 """return json.dumps([]) # Return empty JSON array on error

 """async def _get_pending_network_requests(self) -> list['NetworkRequest']:
 """Get list of currently pending network requests.

 Uses document.readyState and performance API to detect pending requests.
 Filters out ads, tracking, and other noise.

 Returns:

```

```

■■■■List of NetworkRequest objects representing currently loading resources
■■■■""
■■from system.browser.views import NetworkRequest

■■try:
■■■# get_or_create_cdp_session() now handles focus validation automatically
■■■cdp_session = await self.browser_session.get_or_create_cdp_session(focus=True)

■■■# Use performance API to get pending requests
■■■js_code = """
(function() {
■■const now = performance.now();
■■const resources = performance.getEntriesByType('resource');
■■const pending = [];

■■// Check document readyState
■■const docLoading = document.readyState !== 'complete';

■■// Common ad/tracking domains and patterns to filter out
■■const adDomains = [
■■■// Standard ad/tracking networks
■■■'doubleclick.net', 'googlesyndication.com', 'googletagmanager.com',
■■■'facebook.net', 'analytics', 'ads', 'tracking', 'pixel',
■■■'hotjar.com', 'clarity.ms', 'mixpanel.com', 'segment.com',
■■■// Analytics platforms
■■■'demdex.net', 'omtrdc.net', 'adobedtm.com', 'ensighten.com',
■■■'newrelic.com', 'nr-data.net', 'google-analytics.com',
■■■// Social media trackers
■■■'connect.facebook.net', 'platform.twitter.com', 'platform.linkedin.com',
■■■// CDN/image hosts (usually not critical for functionality)
■■■'.cloudfront.net/image/', '.akamaized.net/image/',
■■■// Common tracking paths
■■■'/tracker/', '/collector/', '/beacon/', '/telemetry/', '/log/',
■■■'/events/', '/eventBatch', '/track.', '/metrics/'
■■];

■■// Get resources that are still loading (responseEnd is 0)
■■let totalResourcesChecked = 0;
■■let filteredByResponseEnd = 0;
■■const allDomains = new Set();

■■for (const entry of resources) {
■■■totalResourcesChecked++;

■■■// Track all domains from recent resources (for logging)
■■■try {
■■■■const hostname = new URL(entry.name).hostname;
■■■■if (hostname) allDomains.add(hostname);
■■■} catch (e) {}

■■■if (entry.responseEnd === 0) {
■■■■filteredByResponseEnd++;
■■■■const url = entry.name;

■■■■// Filter out ads and tracking
■■■■const isAd = adDomains.some(domain => url.includes(domain));
■■■■if (isAd) continue;

■■■■// Filter out data: URLs and very long URLs (often inline resources)
■■■■if (url.startsWith('data:') || url.length > 500) continue;

■■■■const loadingDuration = now - entry.startTime;

■■■■// Skip requests that have been loading for >10 seconds (likely stuck/polling)
■■■■if (loadingDuration > 10000) continue;

■■■■const resourceType = entry.initiatorType || 'unknown';

■■■■// Filter out non-critical resources (images, fonts, icons) if loading >3 seconds
■■■■const nonCriticalTypes = ['img', 'image', 'icon', 'font'];
■■■■if (nonCriticalTypes.includes(resourceType) && loadingDuration > 3000) continue;
■■■■// Filter out image URLs even if type is unknown

```

```

#####const isImageUrl = /\.(jpg|jpeg|png|gif|webp|svg|ico)(\?|$)/i.test(url);
#####if (isImageUrl && loadingDuration > 3000) continue;

#####pending.push({
#####url: url,
#####method: 'GET',
#####loading_duration_ms: Math.round(loadingDuration),
#####resource_type: resourceType
#####});
#####}
#####}

#####return {
#####pending_requests: pending,
#####document_loading: docLoading,
#####document_ready_state: document.readyState,
#####debug: {
#####total_resources: totalResourcesChecked,
#####with_response_end_zero: filteredByResponseEnd,
#####after_all_filters: pending.length,
#####all_domains: Array.from(allDomains)
#####}
#####};
#####})();
#####"""

#####result = await cdp_session.cdp_client.send.Runtime.evaluate(
#####params={'expression': js_code, 'returnByValue': True}, session_id=cdp_session.session_id
#####)

#####if result.get('result', {}).get('type') == 'object':
#####data = result['result'].get('value', {})
#####pending = data.get('pending_requests', [])
#####doc_state = data.get('document_ready_state', 'unknown')
#####doc_loading = data.get('document_loading', False)
#####debug_info = data.get('debug', {})

Get all domains that had recent activity (from JS)
#####all_domains = debug_info.get('all_domains', [])
#####all_domains_str = ', '.join(sorted(all_domains)[:5]) if all_domains else 'none'
#####if len(all_domains) > 5:
#####all_domains_str += f' +{len(all_domains) - 5} more'

Debug logging
#####self.logger.debug(
#####f'■ Network check: document.readyState={doc_state}, loading={doc_loading}, '
#####f'total_resources={debug_info.get("total_resources", 0)}, '
#####f'responseEnd=0: {debug_info.get("with_response_end_zero", 0)}, '
#####f'after_filters={len(pending)}, domains=[{all_domains_str}]'
#####)

Convert to NetworkRequest objects
#####network_requests = []
#####for req in pending[:20]: # Limit to 20 to avoid overwhelming the context
#####network_requests.append(
#####NetworkRequest(
#####url=req['url'],
#####method=req.get('method', 'GET'),
#####loading_duration_ms=req.get('loading_duration_ms', 0.0),
#####resource_type=req.get('resource_type'),
#####)
#####)

#####return network_requests

#####except Exception as e:
#####self.logger.debug(f'Failed to get pending network requests: {e}')

#####return []

#####@observe_debug(ignore_input=True, ignore_output=True, name='browser_state_request_event')
#####async def on_BrowserStateRequestEvent(self, event: BrowserStateRequestEvent) -> 'BrowserStateSummary':

```



```

#####try:
#####page_info = await self._get_page_info()
#####except Exception as e:
#####self.logger.debug(f'Failed to get page info from CDP for empty page: {e}, using fallback')
Use default viewport dimensions
#####viewport = self.browser_session.browser_profile.viewport or {'width': 1280, 'height': 720}
#####page_info = PageInfo(
#####viewport_width=viewport['width'],
#####viewport_height=viewport['height'],
#####page_width=viewport['width'],
#####page_height=viewport['height'],
#####scroll_x=0,
#####scroll_y=0,
#####pixels_above=0,
#####pixels_below=0,
#####pixels_left=0,
#####pixels_right=0,
#####)

#####return BrowserStateSummary(
#####dom_state=content,
#####url=page_url,
#####title='Empty Tab',
#####tabs=tabs_info,
#####screenshot=screenshot_b64,
#####page_info=page_info,
#####pixels_above=0,
#####pixels_below=0,
#####browser_errors=[],
#####is_pdf_viewer=False,
#####recent_events=self._get_recent_events_str() if event.include_recent_events else None,
#####pending_network_requests=[], # Empty page has no pending requests
#####pagination_buttons=[], # Empty page has no pagination
#####closed_popup_messages=self.browser_session._closed_popup_messages.copy(),
#####)

Execute DOM building and screenshot capture in parallel
#####dom_task = None
#####screenshot_task = None

Start DOM building task if requested
#####if event.include_dom:
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: ■ Starting DOM tree build task...')

#####previous_state = (
#####self.browser_session._cached_browser_state_summary.dom_state
#####if self.browser_session._cached_browser_state_summary
#####else None
#####)

#####dom_task = create_task_with_error_handling(
#####self._build_dom_tree_without_highlights(previous_state),
#####name='build_dom_tree',
#####logger_instance=self.logger,
#####suppress_exceptions=True,
#####)

Start clean screenshot task if requested (without JS highlights)
#####if event.include_screenshot:
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: ■ Starting clean screenshot task...')
#####screenshot_task = create_task_with_error_handling(
#####self._capture_clean_screenshot(),
#####name='capture_screenshot',
#####logger_instance=self.logger,
#####suppress_exceptions=True,
#####)

Wait for both tasks to complete
#####content = None
#####screenshot_b64 = None

#####if dom_task:

```

```

#####try:
#####content = await dom_task
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: ■ DOM tree build completed')
#####except Exception as e:
#####self.logger.warning(f'■ DOMWatchdog.on_BrowserStateRequestEvent: DOM build failed: {e}, using min')
#####content = SerializedDOMState(_root=None, selector_map={})
#####else:
#####content = SerializedDOMState(_root=None, selector_map={})

####if screenshot_task:
#####try:
#####screenshot_b64 = await screenshot_task
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: ■ Clean screenshot captured')
#####except Exception as e:
#####self.logger.warning(f'■ DOMWatchdog.on_BrowserStateRequestEvent: Clean screenshot failed: {e}')
#####screenshot_b64 = None

Add browser-side highlights for user visibility
####if content and content.selector_map and self.browser_session.browser_profile.dom_highlight_elements:
#####try:
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: ■ Adding browser-side highlights...')
#####await self.browser_session.add_highlights(content.selector_map)
#####self.logger.debug(
#####f'■ DOMWatchdog.on_BrowserStateRequestEvent: ■ Added browser highlights for {len(content.selector_map)} elements')
#####except Exception as e:
#####self.logger.warning(f'■ DOMWatchdog.on_BrowserStateRequestEvent: Browser highlighting failed: {e}')

Ensure we have valid content
####if not content:
#####content = SerializedDOMState(_root=None, selector_map={})

Tabs info already fetched at the beginning

Get target title safely
####try:
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: Getting page title...')
#####title = await asyncio.wait_for(self.browser_session.get_current_page_title(), timeout=1.0)
#####self.logger.debug(f'■ DOMWatchdog.on_BrowserStateRequestEvent: Got title: {title}')
#####except Exception as e:
#####self.logger.debug(f'■ DOMWatchdog.on_BrowserStateRequestEvent: Failed to get title: {e}')
#####title = 'Page'

Get comprehensive page info from CDP with timeout
####try:
#####self.logger.debug('■ DOMWatchdog.on_BrowserStateRequestEvent: Getting page info from CDP...')
#####page_info = await asyncio.wait_for(self._get_page_info(), timeout=1.0)
#####self.logger.debug(f'■ DOMWatchdog.on_BrowserStateRequestEvent: Got page info from CDP: {page_info}')
#####except Exception as e:
#####self.logger.debug(
#####f'■ DOMWatchdog.on_BrowserStateRequestEvent: Failed to get page info from CDP: {e}, using fallback')
Fallback to default viewport dimensions
#####viewport = self.browser_session.browser_profile.viewport or {'width': 1280, 'height': 720}
#####page_info = PageInfo(
#####viewport_width=viewport['width'],
#####viewport_height=viewport['height'],
#####page_width=viewport['width'],
#####page_height=viewport['height'],
#####scroll_x=0,
#####scroll_y=0,
#####pixels_above=0,
#####pixels_below=0,
#####pixels_left=0,
#####pixels_right=0,
#####)

Check for PDF viewer
####is_pdf_viewer = page_url.endswith('.pdf') or '/pdf/' in page_url

Detect pagination buttons from the DOM
####pagination_buttons_data = []

```

```

 if content and content.selector_map:
 pagination_buttons_data = self._detect_pagination_buttons(content.selector_map)

 # Build and cache the browser state summary
 if screenshot_b64:
 self.logger.debug(
 f'DOMWatchdog.on_BrowserStateRequestEvent: ■ Creating BrowserStateSummary with screenshot, length {len(screenshot_b64)}')
 else:
 self.logger.debug(
 f'DOMWatchdog.on_BrowserStateRequestEvent: ■ Creating BrowserStateSummary WITHOUT screenshot')

 browser_state = BrowserStateSummary(
 dom_state=content,
 url=page_url,
 title=title,
 tabs=tabs_info,
 screenshot=screenshot_b64,
 page_info=page_info,
 pixels_above=0,
 pixels_below=0,
 browser_errors=[],
 is_pdf_viewer=is_pdf_viewer,
 recent_events=self._get_recent_events_str() if event.include_recent_events else None,
 pending_network_requests=pending_requests,
 pagination_buttons=pagination_buttons_data,
 closed_popup_messages=self.browser_session._closed_popup_messages.copy(),
)

 # Cache the state
 self.browser_session._cached_browser_state_summary = browser_state

 # Cache viewport size for coordinate conversion (if llm_screenshot_size is enabled)
 if page_info:
 self.browser_session._original_viewport_size = (page_info.viewport_width, page_info.viewport_height)

 self.logger.debug(f'DOMWatchdog.on_BrowserStateRequestEvent: ■ COMPLETED - Returning browser state')
 return browser_state

except Exception as e:
 self.logger.error(f'Failed to get browser state: {e}')

Return minimal recovery state
return BrowserStateSummary(
 dom_state=SerializedDOMState(_root=None, selector_map={}),
 url=page_url if 'page_url' in locals() else '',
 title='Error',
 tabs=[],
 screenshot=None,
 page_info=PageInfo(
 viewport_width=1280,
 viewport_height=720,
 page_width=1280,
 page_height=720,
 scroll_x=0,
 scroll_y=0,
 pixels_above=0,
 pixels_below=0,
 pixels_left=0,
 pixels_right=0,
),
 pixels_above=0,
 pixels_below=0,
 browser_errors=[str(e)],
 is_pdf_viewer=False,
 recent_events=None,
 pending_network_requests=[], # Error state has no pending requests
 pagination_buttons=[], # Error state has no pagination
 closed_popup_messages=self.browser_session._closed_popup_messages.copy()
 if hasattr(self, 'browser_session') and self.browser_session is not None
 else [],
)

```



```

)

 @time_execution_async('build_dom_tree_without_highlights')
 @observe_debug(ignore_input=True, ignore_output=True, name='build_dom_tree_without_highlights')
 async def _build_dom_tree_without_highlights(self, previous_state: SerializedDOMState | None = None) -> SerializedDOMState:
 """Build DOM tree without injecting JavaScript highlights (for parallel execution)."""
 try:
 self.logger.debug('■ DOMWatchdog._build_dom_tree_without_highlights: STARTING DOM tree build')

 # Create or reuse DOM service
 if self._dom_service is None:
 self._dom_service = DomService(
 browser_session=self.browser_session,
 logger=self.logger,
 cross_origin_iframes=self.browser_session.browser_profile.cross_origin_iframes,
 paint_order_filtering=self.browser_session.browser_profile.paint_order_filtering,
 max_iframes=self.browser_session.browser_profile.max_iframes,
 max_iframe_depth=self.browser_session.browser_profile.max_iframe_depth,
)

 # Get serialized DOM tree using the service
 self.logger.debug('■ DOMWatchdog._build_dom_tree_without_highlights: Calling DomService.get_serialized_dom_tree')
 start = time.time()
 self.current_dom_state, self.enhanced_dom_tree, timing_info = await self._dom_service.get_serialized_dom_tree(
 previous_cached_state=previous_state,
)
 end = time.time()
 total_time_ms = (end - start) * 1000
 self.logger.debug(
 '■ DOMWatchdog._build_dom_tree_without_highlights: ■ DomService.get_serialized_dom_tree completed'
)

 # Build hierarchical timing breakdown as single multi-line string
 timing_lines = [f'■ Total DOM tree time: {total_time_ms:.2f}ms', '■ Timing breakdown:']

 # get_all_trees breakdown
 get_all_trees_ms = timing_info.get('get_all_trees_total_ms', 0)
 if get_all_trees_ms > 0:
 timing_lines.append(f'■ get_all_trees: {get_all_trees_ms:.2f}ms')
 iframe_scroll_ms = timing_info.get('iframe_scroll_detection_ms', 0)
 cdp_parallel_ms = timing_info.get('cdp_parallel_calls_ms', 0)
 snapshot_proc_ms = timing_info.get('snapshot_processing_ms', 0)
 if iframe_scroll_ms > 0.01:
 timing_lines.append(f'■ iframe_scroll_detection: {iframe_scroll_ms:.2f}ms')
 if cdp_parallel_ms > 0.01:
 timing_lines.append(f'■ cdp_parallel_calls: {cdp_parallel_ms:.2f}ms')
 if snapshot_proc_ms > 0.01:
 timing_lines.append(f'■ snapshot_processing: {snapshot_proc_ms:.2f}ms')

 # build_ax_lookup
 build_ax_ms = timing_info.get('build_ax_lookup_ms', 0)
 if build_ax_ms > 0.01:
 timing_lines.append(f'■ build_ax_lookup: {build_ax_ms:.2f}ms')

 # build_snapshot_lookup
 build_snapshot_ms = timing_info.get('build_snapshot_lookup_ms', 0)
 if build_snapshot_ms > 0.01:
 timing_lines.append(f'■ build_snapshot_lookup: {build_snapshot_ms:.2f}ms')

 # construct_enhanced_tree
 construct_tree_ms = timing_info.get('construct_enhanced_tree_ms', 0)
 if construct_tree_ms > 0.01:
 timing_lines.append(f'■ construct_enhanced_tree: {construct_tree_ms:.2f}ms')

 # serialize_accessible_elements breakdown
 serialize_total_ms = timing_info.get('serialize_accessible_elements_total_ms', 0)
 if serialize_total_ms > 0.01:
 timing_lines.append(f'■ serialize_accessible_elements: {serialize_total_ms:.2f}ms')
 create_simp_ms = timing_info.get('create_simplified_tree_ms', 0)
 paint_order_ms = timing_info.get('calculate_paint_order_ms', 0)
 optimize_ms = timing_info.get('optimize_tree_ms', 0)
 bbox_ms = timing_info.get('bbox_filtering_ms', 0)

```

```

#####assign_idx_ms = timing_info.get('assign_interactive_indices_ms', 0)
#####clickable_ms = timing_info.get('clickable_detection_time_ms', 0)

#####if create_simp_ms > 0.01:
timing_lines.append(f' ■ ■■ create_simplified_tree: {create_simp_ms:.2f}ms')
if clickable_ms > 0.01:
timing_lines.append(f' ■ ■ ■■ clickable_detection: {clickable_ms:.2f}ms')
if paint_order_ms > 0.01:
timing_lines.append(f' ■ ■■ calculate_paint_order: {paint_order_ms:.2f}ms')
if optimize_ms > 0.01:
timing_lines.append(f' ■ ■■ optimize_tree: {optimize_ms:.2f}ms')
if bbox_ms > 0.01:
timing_lines.append(f' ■ ■■ bbox_filtering: {bbox_ms:.2f}ms')
if assign_idx_ms > 0.01:
timing_lines.append(f' ■ ■■ assign_interactive_indices: {assign_idx_ms:.2f}ms')

Overheads
#####get_dom_overhead_ms = timing_info.get('get_dom_tree_overhead_ms', 0)
#####serialize_overhead_ms = timing_info.get('serialization_overhead_ms', 0)
#####get_serialized_overhead_ms = timing_info.get('get_serialized_dom_tree_overhead_ms', 0)

#####if get_dom_overhead_ms > 0.1:
timing_lines.append(f' ■■ get_dom_tree_overhead: {get_dom_overhead_ms:.2f}ms')
if serialize_overhead_ms > 0.1:
timing_lines.append(f' ■■ serialization_overhead: {serialize_overhead_ms:.2f}ms')
if get_serialized_overhead_ms > 0.1:
timing_lines.append(f' ■■ get_serialized_dom_tree_overhead: {get_serialized_overhead_ms:.2f}ms')

Calculate total tracked time for validation
#####main_operations_ms = (
get_all_trees_ms
+ build_ax_ms
+ build_snapshot_ms
+ construct_tree_ms
+ serialize_total_ms
+ get_dom_overhead_ms
+ serialize_overhead_ms
+ get_serialized_overhead_ms
#####)
#####untracked_time_ms = total_time_ms - main_operations_ms

#####if untracked_time_ms > 1.0: # Only log if significant
timing_lines.append(f' ■■ untracked_time: {untracked_time_ms:.2f}ms')

Single log call with all timing info
#####self.logger.debug('\n'.join(timing_lines))

Update selector map for other watchdogs
#####self.logger.debug('■ DOMWatchdog._build_dom_tree_without_highlights: Updating selector maps...')
#####self.selector_map = self.current_dom_state.selector_map
Update BrowserSession's cached selector map
#####if self.browser_session:
self.browser_session.update_cached_selector_map(self.selector_map)
self.logger.debug(
f'■ DOMWatchdog._build_dom_tree_without_highlights: ■ Selector maps updated, {len(self.selector_ma
)

Skip JavaScript highlighting injection - Python highlighting will be applied later
#####self.logger.debug('■ DOMWatchdog._build_dom_tree_without_highlights: ■ COMPLETED DOM tree build (no
#####return self.current_dom_state

#####except Exception as e:
self.logger.error(f'Failed to build DOM tree without highlights: {e}')
self.event_bus.dispatch(
BrowserErrorEvent(
error_type='DOMBuildFailed',
message=str(e),
)
)
raise

#####@time_execution_async('capture_clean_screenshot')

```

```

@observe_debug(ignore_input=True, ignore_output=True, name='capture_clean_screenshot')
async def _capture_clean_screenshot(self) -> str:
 """Capture a clean screenshot without JavaScript highlights."""
 try:
 self.logger.debug('■ DOMWatchdog._capture_clean_screenshot: Capturing clean screenshot...')

 await self.browser_session.get_or_create_cdp_session(target_id=self.browser_session.agent_focus_target_id)

 # Check if handler is registered
 handlers = self.event_bus.handlers.get('ScreenshotEvent', [])
 handler_names = [getattr(h, '__name__', str(h)) for h in handlers]
 self.logger.debug(f'■ ScreenshotEvent handlers registered: {len(handlers)} - {handler_names}')

 screenshot_event = self.event_bus.dispatch(ScreenshotEvent(full_page=False))
 self.logger.debug('■ Dispatched ScreenshotEvent, waiting for event to complete...')

 # Wait for the event itself to complete (this waits for all handlers)
 await screenshot_event

 # Get the single handler result
 screenshot_b64 = await screenshot_event.event_result(raise_if_any=True, raise_if_none=True)
 if screenshot_b64 is None:
 raise RuntimeError('Screenshot handler returned None')
 self.logger.debug('■ DOMWatchdog._capture_clean_screenshot: ■ Clean screenshot captured successfully')
 return str(screenshot_b64)

 except TimeoutError:
 self.logger.warning('■ Clean screenshot timed out after 6 seconds - no handler registered or slow page')
 raise
 except Exception as e:
 self.logger.warning(f'■ Clean screenshot failed: {type(e).__name__}: {e}')
 raise

def _detect_pagination_buttons(self, selector_map: dict[int, EnhancedDOMTreeNode]) -> list[PaginationButton]:
 """Detect pagination buttons from the DOM selector map.

 Args:
 selector_map: Dictionary mapping element indices to DOM tree nodes

 Returns:
 List of PaginationButton instances found in the DOM
 """
 from system.browser.views import PaginationButton

 pagination_buttons_data = []
 try:
 self.logger.debug('■ DOMWatchdog._detect_pagination_buttons: Detecting pagination buttons...')
 pagination_buttons_raw = DomService.detect_pagination_buttons(selector_map)
 # Convert to PaginationButton instances
 pagination_buttons_data = [
 PaginationButton(
 button_type=btn['button_type'], # type: ignore
 backend_node_id=btn['backend_node_id'], # type: ignore
 text=btn['text'], # type: ignore
 selector=btn['selector'], # type: ignore
 is_disabled=btn['is_disabled'], # type: ignore
)
 for btn in pagination_buttons_raw
]
 if pagination_buttons_data:
 self.logger.debug(
 f'■ DOMWatchdog._detect_pagination_buttons: Found {len(pagination_buttons_data)} pagination buttons'
)
 except Exception as e:
 self.logger.warning(f'■ DOMWatchdog._detect_pagination_buttons: Pagination detection failed: {e}')

 return pagination_buttons_data

async def _get_page_info(self) -> 'PageInfo':
 """Get comprehensive page information using a single CDP call.

 TODO: should we make this an event as well?

```

```

 Returns:
 PageInfo with all viewport, page dimensions, and scroll information
 """

 from system.browser.views import PageInfo

 # get_or_create_cdp_session() handles focus validation automatically
 cdp_session = await self.browser_session.get_or_create_cdp_session(
 target_id=self.browser_session.agent_focus_target_id, focus=True
)

 # Get layout metrics which includes all the information we need
 metrics = await asyncio.wait_for(
 cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=cdp_session.session_id), timeout=10.0
)

 # Extract different viewport types
 layout_viewport = metrics.get('layoutViewport', {})
 visual_viewport = metrics.get('visualViewport', {})
 css_visual_viewport = metrics.get('cssVisualViewport', {})
 css_layout_viewport = metrics.get('cssLayoutViewport', {})
 content_size = metrics.get('contentSize', {})

 # Calculate device pixel ratio to convert between device pixels and CSS pixels
 # This matches the approach in dom/service.py _get_viewport_ratio method
 css_width = css_visual_viewport.get('clientWidth', css_layout_viewport.get('clientWidth', 1280.0))
 device_width = visual_viewport.get('clientWidth', css_width)
 device_pixel_ratio = device_width / css_width if css_width > 0 else 1.0

 # For viewport dimensions, use CSS pixels (what JavaScript sees)
 # Prioritize CSS layout viewport, then fall back to layout viewport
 viewport_width = int(css_layout_viewport.get('clientWidth') or layout_viewport.get('clientWidth', 1280))
 viewport_height = int(css_layout_viewport.get('clientHeight') or layout_viewport.get('clientHeight', 1280))

 # For total page dimensions, content size is typically in device pixels, so convert to CSS pixels
 # by dividing by device pixel ratio
 raw_page_width = content_size.get('width', viewport_width * device_pixel_ratio)
 raw_page_height = content_size.get('height', viewport_height * device_pixel_ratio)
 page_width = int(raw_page_width / device_pixel_ratio)
 page_height = int(raw_page_height / device_pixel_ratio)

 # For scroll position, use CSS visual viewport if available, otherwise CSS layout viewport
 # These should already be in CSS pixels
 scroll_x = int(css_visual_viewport.get('pageX') or css_layout_viewport.get('pageX', 0))
 scroll_y = int(css_visual_viewport.get('pageY') or css_layout_viewport.get('pageY', 0))

 # Calculate scroll information - pixels that are above/below/left/right of current viewport
 pixels_above = scroll_y
 pixels_below = max(0, page_height - viewport_height - scroll_y)
 pixels_left = scroll_x
 pixels_right = max(0, page_width - viewport_width - scroll_x)

 page_info = PageInfo(
 viewport_width=viewport_width,
 viewport_height=viewport_height,
 page_width=page_width,
 page_height=page_height,
 scroll_x=scroll_x,
 scroll_y=scroll_y,
 pixels_above=pixels_above,
 pixels_below=pixels_below,
 pixels_left=pixels_left,
 pixels_right=pixels_right,
)

 return page_info

 # ===== Public Helper Methods =====

 async def get_element_by_index(self, index: int) -> EnhancedDOMTreeNode | None:
 """Get DOM element by index from cached selector map.
 Builds DOM if not cached.

```

```

 Returns:
 EnhancedDOMTreeNode or None if index not found
 """
 if not self.selector_map:
 # Build DOM if not cached
 await self._build_dom_tree_without_highlights()

 return self.selector_map.get(index) if self.selector_map else None

 def clear_cache(self) -> None:
 """Clear cached DOM state to force rebuild on next access."""
 self.selector_map = None
 self.current_dom_state = None
 self.enhanced_dom_tree = None
 # Keep the DOM service instance to reuse its CDP client connection

 def is_file_input(self, element: EnhancedDOMTreeNode) -> bool:
 """Check if element is a file input."""
 return element.node_name.upper() == 'INPUT' and element.attributes.get('type', '').lower() == 'file'

 @staticmethod
 def is_element_visible_according_to_all_parents(node: EnhancedDOMTreeNode, html_frames: list[EnhancedDOMTreeNode]) -> bool:
 """Check if the element is visible according to all its parent HTML frames.

 Delegates to the DomService static method.
 """
 return DomService.is_element_visible_according_to_all_parents(node, html_frames)

 async def __aexit__(self, exc_type, exc_value, traceback):
 """Clean up DOM service on exit."""
 if self._dom_service:
 await self._dom_service.__aexit__(exc_type, exc_value, traceback)
 self._dom_service = None

 def __del__(self):
 """Clean up DOM service on deletion."""
 super().__del__()
 # DOM service will clean up its own CDP client
 self._dom_service = None

```

## ***.\system\browser\watchdogs\downloads\_watchdog.py***

```

"""Downloads watchdog for monitoring and handling file downloads."""

import asyncio
import json
import os
import tempfile
from pathlib import Path
from typing import TYPE_CHECKING, Any, ClassVar
from urllib.parse import urlparse

import anyio
from bubus import BaseEvent
from cdp_use.cdp.browser import DownloadProgressEvent as CDPDownloadProgressEvent
from cdp_use.cdp.browser import DownloadWillBeginEvent
from cdp_use.cdp.network import ResponseReceivedEvent
from cdp_use.cdp.target import SessionID, TargetID
from pydantic import PrivateAttr

from system.browser.events import (
 BrowserLaunchEvent,
 BrowserStateRequestEvent,
 BrowserStoppedEvent,
 DownloadProgressEvent,
 DownloadStartedEvent,
 FileDownloadedEvent,
 NavigationCompleteEvent,

```

```

 ■TabClosedEvent,
 ■TabCreatedEvent,
)
 from system.browser.watchdog_base import BaseWatchdog
 from system.utils import create_task_with_error_handling

 if TYPE_CHECKING:
 ■pass

class DownloadsWatchdog(BaseWatchdog):
 ■"""Monitors downloads and handles file download events."""

 ■# Events this watchdog listens to (for documentation)
 ■LISTENS_TO: ClassVar[list[type[BaseEvent[Any]]]] = [
 ■BrowserLaunchEvent,
 ■BrowserStateRequestEvent,
 ■BrowserStoppedEvent,
 ■TabCreatedEvent,
 ■TabClosedEvent,
 ■NavigationCompleteEvent,
 ■]

 ■# Events this watchdog emits
 ■EMITS: ClassVar[list[type[BaseEvent[Any]]]] = [
 ■DownloadProgressEvent,
 ■DownloadStartedEvent,
 ■FileDownloadedEvent,
 ■]

 ■# Private state
 ■_sessions_with_listeners: set[str] = PrivateAttr(default_factory=set) # Track sessions that already have
 ■_active_downloads: dict[str, Any] = PrivateAttr(default_factory=dict)
 ■_pdf_viewer_cache: dict[str, bool] = PrivateAttr(default_factory=dict) # Cache PDF viewer status by tab
 ■_download_cdp_session_setup: bool = PrivateAttr(default=False) # Track if CDP session is set up
 ■_download_cdp_session: Any = PrivateAttr(default=None) # Store CDP session reference
 ■_cdp_event_tasks: set[asyncio.Task] = PrivateAttr(default_factory=set) # Track CDP event handler tasks
 ■_cdp_downloads_info: dict[str, dict[str, Any]] = PrivateAttr(default_factory=dict) # Map guid -> info
 ■_session_pdf_urls: dict[str, str] = PrivateAttr(default_factory=dict) # URL -> path for PDFs downloaded
 ■_initial_downloads_snapshot: set[str] = PrivateAttr(default_factory=set) # Files present when watchdog
 ■_network_monitored_targets: set[str] = PrivateAttr(default_factory=set) # Track targets with network monitored
 ■_detected_downloads: set[str] = PrivateAttr(default_factory=set) # Track detected download URLs to avoid
 ■_network_callback_registered: bool = PrivateAttr(default=False) # Track if global network callback is

 ■# Direct callback support for download waiting (bypasses event bus for synchronization)
 ■_download_start_callbacks: list[Any] = PrivateAttr(default_factory=list) # Callbacks for download start
 ■_download_progress_callbacks: list[Any] = PrivateAttr(default_factory=list) # Callbacks for download progress
 ■_download_complete_callbacks: list[Any] = PrivateAttr(default_factory=list) # Callbacks for download complete

 ■def register_download_callbacks(
 ■self,
 ■on_start: Any | None = None,
 ■on_progress: Any | None = None,
 ■on_complete: Any | None = None,
 ■) -> None:
 ■"""Register direct callbacks for download events

 ■# Callbacks called sync from CDP event handlers, so click
 ■handlers receive download notification without waiting for event bus to process
 ■"""
 ■self.logger.debug(
 ■■■f'[DownloadsWatchdog] Registering callbacks: start={on_start is not None}, progress={on_progress is not None}'
)
 ■if on_start:
 ■self._download_start_callbacks.append(on_start)
 ■self.logger.debug(
 ■■■f'[DownloadsWatchdog] Registered start callback, now have {len(self._download_start_callbacks)} start callbacks'
)
 ■if on_progress:
 ■self._download_progress_callbacks.append(on_progress)
 ■if on_complete:
 ■self._download_complete_callbacks.append(on_complete)

```

```

def unregister_download_callbacks(
 self,
 on_start: Any | None = None,
 on_progress: Any | None = None,
 on_complete: Any | None = None,
) -> None:
 """Unregister previously registered download callbacks."""
 if on_start and on_start in self._download_start_callbacks:
 self._download_start_callbacks.remove(on_start)
 if on_progress and on_progress in self._download_progress_callbacks:
 self._download_progress_callbacks.remove(on_progress)
 if on_complete and on_complete in self._download_complete_callbacks:
 self._download_complete_callbacks.remove(on_complete)

async def on_BrowserLaunchEvent(self, event: BrowserLaunchEvent) -> None:
 self.logger.debug(f'[Downloadswatchdog] Received BrowserLaunchEvent, EventBus ID: {id(self.event_bus)}')
 # Ensure downloads directory exists
 downloads_path = self.browser_session.browser_profile.downloads_path
 if downloads_path:
 expanded_path = Path(downloads_path).expanduser().resolve()
 expanded_path.mkdir(parents=True, exist_ok=True)
 self.logger.debug(f'[Downloadswatchdog] Ensured downloads directory exists: {expanded_path}')

 # Capture initial files to detect new downloads reliably
 if expanded_path.exists():
 for f in expanded_path.iterdir():
 if f.is_file() and not f.name.startswith('.'):
 self._initial_downloads_snapshot.add(f.name)
 self.logger.debug(
 f'[Downloadswatchdog] Captured initial downloads: {len(self._initial_downloads_snapshot)} files'
)

async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 """Monitor new tabs for downloads."""
 # logger.info(f'[Downloadswatchdog] TabCreatedEvent received for tab {event.target_id[-4:]}: {event.url}')

 # Assert downloads path is configured (should always be set by BrowserProfile default)
 assert self.browser_session.browser_profile.downloads_path is not None, 'Downloads path must be configured'

 if event.target_id:
 # logger.info(f'[Downloadswatchdog] Found target for tab {event.target_id}, calling attach_to_target')
 await self.attach_to_target(event.target_id)
 else:
 self.logger.warning(f'[Downloadswatchdog] No target found for tab {event.target_id}')

async def on_TabClosedEvent(self, event: TabClosedEvent) -> None:
 """Stop monitoring closed tabs."""
 pass # No cleanup needed, browser context handles target lifecycle

async def on_BrowserStateRequestEvent(self, event: BrowserStateRequestEvent) -> None:
 """Handle browser state request events."""
 # Use public API - automatically validates and waits for recovery if needed
 self.logger.debug(f'[Downloadswatchdog] on_BrowserStateRequestEvent started, event_id={event.event_id}')
 try:
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 except ValueError:
 self.logger.warning(f'[Downloadswatchdog] No valid focus, skipping BrowserStateRequestEvent {event.event_id}')
 return # No valid focus, skip

 self.logger.debug(
 f'[Downloadswatchdog] About to call get_current_page_url(), target_id={cdp_session.target_id[-4:]} if'
)
 url = await self.browser_session.get_current_page_url()
 self.logger.debug(f'[Downloadswatchdog] Got URL: {url[:80]} if url else "None"')

 if not url:
 self.logger.warning(f'[Downloadswatchdog] No URL found for BrowserStateRequestEvent {event.event_id}')
 return

 target_id = cdp_session.target_id
 self.logger.debug(f'[Downloadswatchdog] About to dispatch NavigationCompleteEvent for target {target_id}')
 self.event_bus.dispatch(

```

```

■■■■NavigationCompleteEvent(
■■■■event_type='NavigationCompleteEvent',
■■■■url=url,
■■■■target_id=target_id,
■■■■event_parent_id=event.event_id,
■■■■)
■■■)
■■■self.logger.debug('[Downloadswatchdog] Successfully completed BrowserStateRequestEvent')

■■■async def on_BrowserStoppedEvent(self, event: BrowserStoppedEvent) -> None:
■■■ """Clean up when browser stops."""
■■■ # Cancel all CDP event handler tasks
■■■ for task in list(self._cdp_event_tasks):
■■■ if not task.done():
■■■ task.cancel()
■■■ # Wait for all tasks to complete cancellation
■■■ if self._cdp_event_tasks:
■■■ await asyncio.gather(*self._cdp_event_tasks, return_exceptions=True)
■■■ self._cdp_event_tasks.clear()

■■■ # Clean up CDP session
■■■ # CDP sessions are now cached and managed by BrowserSession
■■■ self._download_cdp_session = None
■■■ self._download_cdp_session_setup = False

■■■ # Clear other state
■■■ self._sessions_with_listeners.clear()
■■■ self._active_downloads.clear()
■■■ self._pdf_viewer_cache.clear()
■■■ self._session_pdf_urls.clear()
■■■ self._network_monitored_targets.clear()
■■■ self._detected_downloads.clear()
■■■ self._initial_downloads_snapshot.clear()
■■■ self._network_callback_registered = False

■■■ async def on_NavigationCompleteEvent(self, event: NavigationCompleteEvent) -> None:
■■■ """Check for PDFs after navigation completes."""
■■■ self.logger.debug(f'[Downloadswatchdog] NavigationCompleteEvent received for {event.url}, tab #{event.tab_id}')

■■■ # Clear PDF cache for the navigated URL since content may have changed
■■■ if event.url in self._pdf_viewer_cache:
■■■ del self._pdf_viewer_cache[event.url]

■■■ # Check if auto-download is enabled
■■■ auto_download_enabled = self._is_auto_download_enabled()
■■■ if not auto_download_enabled:
■■■ return

■■■ # Note: Using network-based PDF detection that doesn't require JavaScript

■■■ target_id = event.target_id
■■■ self.logger.debug(f'[Downloadswatchdog] Got target_id={target_id} for tab #{event.target_id[-4:]}')

■■■ is_pdf = await self.check_for_pdf_viewer(target_id)

■■■ if is_pdf:
■■■ self.logger.debug(f'[Downloadswatchdog] ■ PDF detected at {event.url}, triggering auto-download...')
■■■ download_path = await self.trigger_pdf_download(target_id)
■■■ if not download_path:
■■■ self.logger.warning(f'[Downloadswatchdog] ■■ PDF download failed for {event.url}')

■■■ def _is_auto_download_enabled(self) -> bool:
■■■ """Check if auto-download PDFs is enabled in browser profile."""
■■■ return self.browser_session.browser_profile.auto_download_pdfs

■■■ async def attach_to_target(self, target_id: TargetID) -> None:
■■■ """Set up download monitoring for a specific target."""

■■■ # Define CDP event handlers outside of try to avoid indentation/scope issues
■■■ def download_will_begin_handler(event: DownloadWillBeginEvent, session_id: SessionID | None) -> None:
■■■ self.logger.debug(f'[Downloadswatchdog] Download will begin: {event}')
■■■ # Cache info for later completion event handling (esp. remote browsers)

```



```

 guid = event.get('guid', '')
 url = event.get('url', '')
 suggested_filename = event.get('suggestedFilename', 'download')
 try:
 assert suggested_filename, 'CDP DownloadWillBegin missing suggestedFilename'
 self._cdp_downloads_info[guid] = {
 'url': url,
 'suggested_filename': suggested_filename,
 'handled': False,
 }
 except (AssertionError, KeyError):
 pass

 # Call direct callbacks first (for click handlers waiting for downloads)
 download_info = {
 'guid': guid,
 'url': url,
 'suggested_filename': suggested_filename,
 'auto_download': False,
 }
 self.logger.debug(f'[Downloadswatchdog] Calling {len(self._download_start_callbacks)} start callbacks')
 for callback in self._download_start_callbacks:
 try:
 self.logger.debug(f'[Downloadswatchdog] Calling start callback: {callback}')
 callback(download_info)
 except Exception as e:
 self.logger.debug(f'[Downloadswatchdog] Error in download start callback: {e}')

 # Emit DownloadStartedEvent so other components can react
 self.event_bus.dispatch(
 DownloadStartedEvent(
 guid=guid,
 url=url,
 suggested_filename=suggested_filename,
 auto_download=False, # CDP-triggered downloads are user-initiated
)
)

 # Create and track the task
 task = create_task_with_error_handling(
 self._handle_cdp_download(event, target_id, session_id),
 name='handle_cdp_download',
 logger_instance=self.logger,
 suppress_exceptions=True,
)
 self._cdp_event_tasks.add(task)
 # Remove from set when done
 task.add_done_callback(lambda t: self._cdp_event_tasks.discard(t))

 def download_progress_handler(event: CDPDownloadProgressEvent, session_id: SessionID | None) -> None:
 guid = event.get('guid', '')
 state = event.get('state', '')
 received_bytes = int(event.get('receivedBytes', 0))
 total_bytes = int(event.get('totalBytes', 0))

 # Call direct callbacks first (for click handlers tracking progress)
 progress_info = {
 'guid': guid,
 'received_bytes': received_bytes,
 'total_bytes': total_bytes,
 'state': state,
 }
 for callback in self._download_progress_callbacks:
 try:
 callback(progress_info)
 except Exception as e:
 self.logger.debug(f'[Downloadswatchdog] Error in download progress callback: {e}')

 # Emit progress event for all states so listeners can track progress
 from system.browser.events import DownloadProgressEvent as DownloadProgressEventInternal

 self.event_bus.dispatch(

```

```

#####DownloadProgressEventInternal(
#####guid=guid,
#####received_bytes=received_bytes,
#####total_bytes=total_bytes,
#####state=state,
#####)
#####)

Check if download is complete
####if state == 'completed':
#####file_path = event.get('filePath')
#####if self.browser_session.is_local:
#####if file_path:
#####self.logger.debug(f'[Downloadswatchdog] Download completed: {file_path}')
Track the download
#####self._track_download(file_path, guid=guid)
Mark as handled to prevent fallback duplicate dispatch
#####try:
#####if guid in self._cdp_downloads_info:
#####self._cdp_downloads_info[guid]['handled'] = True
#####except (KeyError, AttributeError):
#####pass
#####else:
No filePath provided - detect by comparing with initial snapshot
#####self.logger.debug(f'[Downloadswatchdog] No filePath in progress event; detecting via filesystem')
#####downloads_path = self.browser_session.browser_profile.downloads_path
#####if downloads_path:
#####downloads_dir = Path(downloads_path).expanduser().resolve()
#####if downloads_dir.exists():
#####for f in downloads_dir.iterdir():
#####if (
#####f.is_file()
#####and not f.name.startswith('.')
#####and f.name not in self._initial_downloads_snapshot
#####):
Check file has content before processing
#####if f.stat().st_size > 4:
Found a new file! Add to snapshot immediately to prevent duplicate detection
#####self._initial_downloads_snapshot.add(f.name)
#####self.logger.debug(f'[Downloadswatchdog] Detected new download: {f.name}')
#####self._track_download(str(f))
Mark as handled
#####try:
#####if guid in self._cdp_downloads_info:
#####self._cdp_downloads_info[guid]['handled'] = True
#####except (KeyError, AttributeError):
#####pass
#####break
#####else:
Remote browser: do not touch local filesystem. Fallback to downloadPath+suggestedFilename
#####info = self._cdp_downloads_info.get(guid, {})
#####try:
#####suggested_filename = info.get('suggested_filename') or (Path(file_path).name if file_path else '')
#####downloads_path = str(self.browser_session.browser_profile.downloads_path or '')
#####effective_path = file_path or str(Path(downloads_path) / suggested_filename)
#####file_name = Path(effective_path).name
#####file_ext = Path(file_name).suffix.lower().lstrip('.')
#####self.event_bus.dispatch(
#####FileDownloadedEvent(
#####guid=guid,
#####url=info.get('url', ''),
#####path=str(effective_path),
#####file_name=file_name,
#####file_size=0,
#####file_type=file_ext if file_ext else None,
#####)
#####)
#####self.logger.debug(f'[Downloadswatchdog] ■ (remote) Download completed: {effective_path}')
#####finally:
#####if guid in self._cdp_downloads_info:
#####del self._cdp_downloads_info[guid]
#####try:

```

```

#####downloads_path_raw = self.browser_session.browser_profile.downloads_path
#####if not downloads_path_raw:
logger.info(f'[Downloadswatchdog] No downloads path configured, skipping target: {target_id}')
return # No downloads path configured

Check if we already have a download listener on this session
to prevent duplicate listeners from being added
Note: Since download listeners are set up once per browser session, not per target,
we just track if we've set up the browser-level listener
#####if self._download_cdp_session_setup:
self.logger.debug(f'[Downloadswatchdog] Download listener already set up for browser session')
return

logger.debug(f'[Downloadswatchdog] Setting up CDP download listener for target: {target_id}')

Use CDP session for download events but store reference in watchdog
#####if not self._download_cdp_session_setup:
Set up CDP session for downloads (only once per browser session)
cdp_client = self.browser_session.cdp_client

Set download behavior to allow downloads and enable events
#####downloads_path = self.browser_session.browser_profile.downloads_path
#####if not downloads_path:
self.logger.warning(f'[Downloadswatchdog] No downloads path configured, skipping CDP download setup')
return
Ensure path is properly expanded (~ -> absolute path)
#####expanded_downloads_path = Path(downloads_path).expanduser().resolve()
#####await cdp_client.send(Browser.setDownloadBehavior(
params={
'behavior': 'allow',
'downloadPath': str(expanded_downloads_path), # Use expanded absolute path
'eventsEnabled': True,
})
#####)

Register the handlers with CDP
#####cdp_client.register.Browser.downloadWillBegin(download_will_begin_handler) # type: ignore[arg-type]
#####cdp_client.register.Browser.downloadProgress(download_progress_handler) # type: ignore[arg-type]

#####self._download_cdp_session_setup = True
#####self.logger.debug(f'[Downloadswatchdog] Set up CDP download listeners')

No need to track individual targets since download listener is browser-level
logger.debug(f'[Downloadswatchdog] Successfully set up CDP download listener for target: {target_id}')

#####except Exception as e:
self.logger.warning(f'[Downloadswatchdog] Failed to set up CDP download listener for target {target_id}')

Set up network monitoring for this target (catches ALL download variants)
#####await self._setup_network_monitoring(target_id)

#####async def _setup_network_monitoring(self, target_id: TargetID) -> None:
"""Set up network monitoring to detect PDFs and downloads from ALL sources.

This catches:
- Direct PDF navigation
- PDFs in iframes
- PDFs with embed/object tags
- JavaScript-triggered downloads
- Any Content-Disposition: attachment headers
"""
Skip if already monitoring this target
if target_id in self._network_monitored_targets:
self.logger.debug(f'[Downloadswatchdog] Network monitoring already enabled for target {target_id[-4:]}')
return

Check if auto-download is enabled
if not self._is_auto_download_enabled():
self.logger.debug(f'[Downloadswatchdog] Auto-download disabled, skipping network monitoring')
return

try:

```

```

■■■■cdp_client = self.browser_session.cdp_client

■■■■# Register the global callback once
■■■■if not self._network_callback_registered:

■■■■def on_response_received(event: ResponseReceivedEvent, session_id: str | None) -> None:
■■■■ """Handle Network.responseReceived event to detect downloadable content.

■■■■ This callback is registered globally and uses session_id to determine the correct target.
■■■■ """
■■■■ try:
■■■■ # Check if session_manager exists (may be None during browser shutdown)
■■■■ if not self.browser_session.session_manager:
■■■■ self.logger.warning('[DownloadsWatchdog] Session manager not found, skipping network monitoring')
■■■■ return

■■■■ # Look up target_id from session_id
■■■■ event_target_id = self.browser_session.session_manager.get_target_id_from_session_id(session_id)
■■■■ if not event_target_id:
■■■■ # Session not in pool - might be a stale session or not yet tracked
■■■■ return

■■■■ # Only process events for targets we're monitoring
■■■■ if event_target_id not in self._network_monitored_targets:
■■■■ return

■■■■ response = event.get('response', {})
■■■■ url = response.get('url', '')
■■■■ content_type = response.get('mimeType', '').lower()
■■■■ headers = {
■■■■ k.lower(): v for k, v in response.get('headers', {}).items()
■■■■ } # Normalize for case-insensitive lookup
■■■■ request_type = event.get('type', '')

■■■■ # Skip non-HTTP URLs (data:, about:, chrome-extension:, etc.)
■■■■ if not url.startswith('http'):
■■■■ return

■■■■ # Skip fetch/XHR - real browsers don't download PDFs from programmatic requests
■■■■ if request_type in ('Fetch', 'XHR'):
■■■■ return

■■■■ # Check if it's a PDF
■■■■ is_pdf = 'application/pdf' in content_type

■■■■ # Check if it's marked as download via Content-Disposition header
■■■■ content_disposition = str(headers.get('content-disposition', '')).lower()
■■■■ is_download_attachment = 'attachment' in content_disposition

■■■■ # Filter out image/video/audio files even if marked as attachment
■■■■ # These are likely resources, not intentional downloads
■■■■ unwanted_content_types = [
■■■■ 'image/',
■■■■ 'video/',
■■■■ 'audio/',
■■■■ 'text/css',
■■■■ 'text/javascript',
■■■■ 'application/javascript',
■■■■ 'application/x-javascript',
■■■■ 'text/html',
■■■■ 'application/json',
■■■■ 'font/',
■■■■ 'application/font',
■■■■ 'application/x-font',
■■■■]
■■■■ is_unwanted_type = any(content_type.startswith(prefix) for prefix in unwanted_content_types)
■■■■ if is_unwanted_type:
■■■■ return

■■■■ # Check URL extension to filter out obvious images/resources
■■■■ url_lower = url.lower().split('?')[0] # Remove query params
■■■■ unwanted_extensions = [

```

```

#####'.jpg',
#####'.jpeg',
#####'.png',
#####'.gif',
#####'.webp',
#####'.svg',
#####'.ico',
#####'.css',
#####'.js',
#####'.woff',
#####'.woff2',
#####'.ttf',
#####'.eot',
#####'.mp4',
#####'.webm',
#####'.mp3',
#####'.wav',
#####'.ogg',
#####]
#####if any(url_lower.endswith(ext) for ext in unwanted_extensions):
#####return

Only process if it's a PDF or download
#####if not (is_pdf or is_download_attachment):
#####return

If already downloaded this URL and file still exists, do nothing
#####existing_path = self._session_pdf_urls.get(url)
#####if existing_path:
#####if os.path.exists(existing_path):
#####return
Stale cache entry, allow re-download
#####del self._session_pdf_urls[url]

Check if we've already processed this URL in this session
#####if url in self._detected_downloads:
#####self.logger.debug(f'[Downloadswatchdog] Already detected download: {url[:80]}...')
#####return

Mark as detected to avoid duplicates
#####self._detected_downloads.add(url)

Extract filename from Content-Disposition if available
#####suggested_filename = None
#####if 'filename=' in content_disposition:
Parse filename from Content-Disposition header
#####import re

#####filename_match = re.search(r'filename[^\n]*=((["']).*?\2|[\^;\n]*)', content_disposition)
#####if filename_match:
#####suggested_filename = filename_match.group(1).strip('\\"')

#####self.logger.info(f'[Downloadswatchdog] ■ Detected downloadable content via network: {url[:80]}...')
#####self.logger.debug(
#####f'[Downloadswatchdog] Content-Type: {content_type}, Is PDF: {is_pdf}, Is Attachment: {is_down
#####)

Trigger download asynchronously in background (don't block event handler)
#####async def download_in_background():
Don't permanently block re-processing this URL if download fails
#####try:
#####download_path = await self.download_file_from_url(
#####url=url,
#####target_id=event_target_id, # Use target_id from session_id lookup
#####content_type=content_type,
#####suggested_filename=suggested_filename,
#####)

#####if download_path:
#####self.logger.info(f'[Downloadswatchdog] ■ Successfully downloaded: {download_path}')
#####else:
#####self.logger.warning(f'[Downloadswatchdog] ■ Failed to download: {url[:80]}...')

```

```

except Exception as e:
 self.logger.error(f'[Downloadswatchdog] Error downloading in background: {type(e).__name__}: {e}')
 finally:
 # Allow future detections of the same URL
 self._detected_downloads.discard(url)

 # Create background task
 task = create_task_with_error_handling(
 download_in_background(),
 name='download_in_background',
 logger_instance=self.logger,
 suppress_exceptions=True,
)
 self._cdp_event_tasks.add(task)
 task.add_done_callback(lambda t: self._cdp_event_tasks.discard(t))

except Exception as e:
 self.logger.error(f'[Downloadswatchdog] Error in network response handler: {type(e).__name__}: {e}')

Register the callback globally (once)
cdp_client.register.Network.responseReceived(on_response_received)
self._network_callback_registered = True
self.logger.debug(f'[Downloadswatchdog] ■ Registered global network response callback')

Get or create CDP session for this target
cdp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)

Enable Network domain to monitor HTTP responses (per-target/per-session)
await cdp_client.send.Network.enable(session_id=cdp_session.session_id)
self.logger.debug(f'[Downloadswatchdog] Enabled Network domain for target {target_id[-4:]}')

Mark this target as monitored
self._network_monitored_targets.add(target_id)
self.logger.debug(f'[Downloadswatchdog] ■ Network monitoring enabled for target {target_id[-4:]}')

except Exception as e:
 self.logger.warning(f'[Downloadswatchdog] Failed to set up network monitoring for target {target_id[-4:]}')

async def download_file_from_url(
 self, url: str, target_id: TargetID, content_type: str | None = None, suggested_filename: str | None = None
) -> str | None:
 """Generic method to download any file from a URL.

 Args:
 url: The URL to download
 target_id: The target ID for CDP session
 content_type: Optional content type (e.g., 'application/pdf')
 suggested_filename: Optional filename from Content-Disposition header

 Returns:
 Path to downloaded file, or None if download failed
 """
 if not self.browser_session.browser_profile.downloads_path:
 self.logger.warning(f'[Downloadswatchdog] No downloads path configured')
 return None

 # Check if already downloaded in this session
 if url in self._session_pdf_urls:
 existing_path = self._session_pdf_urls[url]
 if os.path.exists(existing_path):
 self.logger.debug(f'[Downloadswatchdog] File already downloaded in session: {existing_path}')
 return existing_path

 # Stale cache entry: the file was removed/cleaned up after we cached it.
 self.logger.debug(f'[Downloadswatchdog] Cached download path no longer exists, re-downloading: {existing_path}')
 del self._session_pdf_urls[url]

 try:
 # Get or create CDP session for this target
 temp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)

 # Determine filename

```

```

#####if suggested_filename:
#####filename = suggested_filename
#####else:
Extract from URL
#####filename = os.path.basename(url.split('?')[0]) # Remove query params
#####if not filename or '.' not in filename:
Fallback: use content type to determine extension
#####if content_type and 'pdf' in content_type:
#####filename = 'document.pdf'
#####else:
#####filename = 'download'

Ensure downloads directory exists
#####downloads_dir = str(self.browser_session.browser_profile.downloads_path)
#####os.makedirs(downloads_dir, exist_ok=True)

Generate unique filename if file exists
#####final_filename = filename
#####existing_files = os.listdir(downloads_dir)
#####if filename in existing_files:
#####base, ext = os.path.splitext(filename)
#####counter = 1
#####while f'{base} ({counter}){ext}' in existing_files:
#####counter += 1
#####final_filename = f'{base} ({counter}){ext}'
#####self.logger.debug(f'[Downloadswatchdog] File exists, using: {final_filename}')

#####self.logger.debug(f'[Downloadswatchdog] Downloading from: {url[:100]}...')

Download using JavaScript fetch to leverage browser cache
#####escaped_url = json.dumps(url)

#####result = await asyncio.wait_for(
#####temp_session.cdp_client.send(Runtime.evaluate(
#####params={
#####'expression': f"""
#####(async () => {{
#####try {{
#####const response = await fetch({escaped_url}, {{
#####cache: 'force-cache'
#####}});
#####if (!response.ok) {{
#####throw new Error(`HTTP error! status: ${response.status}`);
#####}}
#####const blob = await response.blob();
#####const arrayBuffer = await blob.arrayBuffer();
#####const uint8Array = new Uint8Array(arrayBuffer);

#####return {{
#####data: Array.from(uint8Array),
#####responseSize: uint8Array.length
#####}};
#####}} catch (error) {{
#####throw new Error(`Fetch failed: ${error.message}`);
#####}}
#####}})()
#####""",
#####'awaitPromise': True,
#####'returnByValue': True,
#####},
#####session_id=temp_session.session_id,
#####),
#####timeout=15.0, # 15 second timeout
#####)

#####download_result = result.get('result', {}).get('value', {})

#####if download_result and download_result.get('data') and len(download_result['data']) > 0:
#####download_path = os.path.join(downloads_dir, final_filename)

Save the file asynchronously
#####async with await anyio.open_file(download_path, 'wb') as f:

```

```

#####await f.write(bytes(download_result['data']))

Verify file was written successfully
#####if os.path.exists(download_path):
actual_size = os.path.getsize(download_path)
self.logger.debug(f'[Downloadswatchdog] File written: {download_path} ({actual_size} bytes)')

Determine file type
#####file_ext = Path(final_filename).suffix.lower().lstrip('.')
#####mime_type = content_type or f'application/{file_ext}'

Store URL->path mapping for this session
#####self._session_pdf_urls[url] = download_path

Emit file downloaded event
#####self.logger.debug(f'[Downloadswatchdog] Dispatching FileDownloadedEvent for {final_filename}')
#####self.event_bus.dispatch(
FileDownloadedEvent(
url=url,
path=download_path,
file_name=final_filename,
file_size=actual_size,
file_type=file_ext if file_ext else None,
mime_type=mime_type,
auto_download=True,
)
#####)

#####return download_path
#####else:
self.logger.error(f'[Downloadswatchdog] Failed to write file: {download_path}')
return None
#####else:
self.logger.warning(f'[Downloadswatchdog] No data received when downloading from {url}')
return None

#####except TimeoutError:
self.logger.warning(f'[Downloadswatchdog] Download timed out: {url[:80]}...')
return None
#####except Exception as e:
self.logger.warning(f'[Downloadswatchdog] Download failed: {type(e).__name__}: {e}')
return None

#####def _track_download(self, file_path: str, guid: str | None = None) -> None:
"""Track a completed download and dispatch the appropriate event.

Args:
file_path: The path to the downloaded file
guid: Optional CDP download GUID for correlation with DownloadStartedEvent
"""
try:
Get file info
path = Path(file_path)
if path.exists():
file_size = path.stat().st_size
self.logger.debug(f'[Downloadswatchdog] Tracked download: {path.name} ({file_size} bytes)')

Get file extension for file_type
file_ext = path.suffix.lower().lstrip('.')

Call direct callbacks first (for click handlers waiting for downloads)
complete_info = {
'guid': guid,
'url': str(path),
'path': str(path),
'file_name': path.name,
'file_size': file_size,
'file_type': file_ext if file_ext else None,
'auto_download': False,
}
for callback in self._download_complete_callbacks:
try:

```



```

#####callback(complete_info)
#####except Exception as e:
#####self.logger.debug(f'[Downloadswatchdog] Error in download complete callback: {e}')

Dispatch download event
#####from system.browser.events import FileDownloadedEvent

#####self.event_bus.dispatch(
#####FileDownloadedEvent(
#####guid=guid,
#####url=str(path), # Use the file path as URL for local files
#####path=str(path),
#####file_name=path.name,
#####file_size=file_size,
#####)
#####)
#####else:
#####self.logger.warning(f'[Downloadswatchdog] Downloaded file not found: {file_path}')
#####except Exception as e:
#####self.logger.error(f'[Downloadswatchdog] Error tracking download: {e}')

#####async def _handle_cdp_download(
#####self, event: DownloadWillBeginEvent, target_id: TargetID, session_id: SessionID | None
#####) -> None:
#####"""Handle a CDP Page.downloadWillBegin event."""
#####downloads_dir = (
#####Path(
#####self.browser_session.browser_profile.downloads_path
#####or f'{tempfile.gettempdir()}/browser_use_downloads.{str(self.browser_session.id)[-4:]}'
#####)
#####).expanduser()
#####.resolve()
#####) # Ensure path is properly expanded

Initialize variables that may be used outside try blocks
#####unique_filename = None
#####file_size = 0
#####expected_path = None
#####download_result = None
#####download_url = event.get('url', '')
#####suggested_filename = event.get('suggestedFilename', 'download')
#####guid = event.get('guid', '')

#####try:
#####self.logger.debug(f'[Downloadswatchdog] ■■ File download starting: {suggested_filename} from {download_url}')
#####self.logger.debug(f'[Downloadswatchdog] Full CDP event: {event}')

Since Browser.setDownloadBehavior is already configured, the browser will download the file
We just need to wait for it to appear in the downloads directory
#####expected_path = downloads_dir / suggested_filename

For remote browsers, don't poll local filesystem; downloadProgress handler will emit the event
#####if not self.browser_session.is_local:
#####return
#####except Exception as e:
#####self.logger.error(f'[Downloadswatchdog] ■ Error handling CDP download: {type(e).__name__} {e}')

If we reach here, the fetch method failed, so wait for native download
Poll the downloads directory for new files
#####self.logger.debug(f'[Downloadswatchdog] Checking if browser auto-download saved the file for us: {suggested_filename}')

Poll for new files
#####max_wait = 20 # seconds
#####start_time = asyncio.get_event_loop().time()

#####while asyncio.get_event_loop().time() - start_time < max_wait: # noqa: ASYNC110
#####await asyncio.sleep(5.0) # Check every 5 seconds

#####if Path(downloads_dir).exists():
#####for file_path in Path(downloads_dir).iterdir():
Skip hidden files and files that were already there
if (

```

```

#####file_path.is_file()
#####and not file_path.name.startswith('.')
#####and file_path.name not in self._initial_downloads_snapshot
#####):
Add to snapshot immediately to prevent duplicate detection
#####self._initial_downloads_snapshot.add(file_path.name)
Check if file has content (> 4 bytes)
#####try:
#####file_size = file_path.stat().st_size
#####if file_size > 4:
Found a new download!
#####self.logger.debug(
#####f'[Downloadswatchdog] ■ Found downloaded file: {file_path} ({file_size} bytes)'
#####)

Determine file type from extension
#####file_ext = file_path.suffix.lower().lstrip('.')
#####file_type = file_ext if file_ext else None

Dispatch download event
Skip if already handled by progress/JS fetch
#####info = self._cdp_downloads_info.get(guid, {})
#####if info.get('handled'):
#####return
#####self.event_bus.dispatch(
#####FileDownloadedEvent(
#####guid=guid,
#####url=download_url,
#####path=str(file_path),
#####file_name=file_path.name,
#####file_size=file_size,
#####file_type=file_type,
#####)
#####)
Mark as handled after dispatch
#####try:
#####if guid in self._cdp_downloads_info:
#####self._cdp_downloads_info[guid]['handled'] = True
#####except (KeyError, AttributeError):
#####pass
#####return
#####except Exception as e:
#####self.logger.debug(f'[Downloadswatchdog] Error checking file {file_path}: {e}')

#####self.logger.warning(f'[Downloadswatchdog] Download did not complete within {max_wait} seconds')

#####async def _handle_download(self, download: Any) -> None:
#####"""Handle a download event."""
#####download_id = f'{id(download)}'
#####self._active_downloads[download_id] = download
#####self.logger.debug(f'[Downloadswatchdog] ■■ Handling download: {download.suggested_filename} from {download.url}')

Debug: Check if download is already being handled elsewhere
#####failure = (
#####await download.failure()
#####) # TODO: it always fails for some reason, figure out why connect_over_cdp makes accept_downloads no
#####self.logger.warning(f'[Downloadswatchdog] ■ Download state - canceled: {failure}, url: {download.url}')
logger.info(f'[Downloadswatchdog] Active downloads count: {len(self._active_downloads)}')

#####try:
#####current_step = 'getting_download_info'
Get download info immediately
#####url = download.url
#####suggested_filename = download.suggested_filename

#####current_step = 'determining_download_directory'
Determine download directory from browser profile
#####downloads_dir = self.browser_session.browser_profile.downloads_path
#####if not downloads_dir:
#####downloads_dir = str(Path.home() / 'Downloads')
#####else:
#####downloads_dir = str(downloads_dir) # Ensure it's a string

```

```

Check if Playwright already auto-downloaded the file (due to CDP setup)
#####original_path = Path(downloads_dir) / suggested_filename
#####if original_path.exists() and original_path.stat().st_size > 0:
#####self.logger.debug(
#####f'[DownloadsWatchdog] File already downloaded by Playwright: {original_path} ({original_path.stat()
#####)

Use the existing file instead of creating a duplicate
#####download_path = original_path
#####file_size = original_path.stat().st_size
#####unique_filename = suggested_filename
#####else:
#####current_step = 'generating_unique_filename'
Ensure unique filename
#####unique_filename = await self._get_unique_filename(downloads_dir, suggested_filename)
#####download_path = Path(downloads_dir) / unique_filename

#####self.logger.debug(f'[DownloadsWatchdog] Download started: {unique_filename} from {url[:100]}...')

#####current_step = 'calling_save_as'
Save the download using Playwright's save_as method
#####self.logger.debug(f'[DownloadsWatchdog] Saving download to: {download_path}')
#####self.logger.debug(f'[DownloadsWatchdog] Download path exists: {download_path.parent.exists()}')
#####self.logger.debug(f'[DownloadsWatchdog] Download path writable: {os.access(download_path.parent, os

#####try:
#####self.logger.debug(f'[DownloadsWatchdog] About to call download.save_as()...')
#####await download.save_as(str(download_path))
#####self.logger.debug(f'[DownloadsWatchdog] Successfully saved download to: {download_path}')
#####current_step = 'save_as_completed'
#####except Exception as save_error:
#####self.logger.error(f'[DownloadsWatchdog] save_as() failed with error: {save_error}')
#####raise save_error

Get file info
#####file_size = download_path.stat().st_size if download_path.exists() else 0

Determine file type from extension
#####file_ext = download_path.suffix.lower().lstrip('.')
#####file_type = file_ext if file_ext else None

Try to get MIME type from response headers if available
#####mime_type = None
Note: Playwright doesn't expose response headers directly from Download object

Check if this was a PDF auto-download
#####auto_download = False
#####if file_type == 'pdf':
#####auto_download = self._is_auto_download_enabled()

Emit download event
#####self.event_bus.dispatch(
#####FileDownloadedEvent(
#####url=url,
#####path=str(download_path),
#####file_name=suggested_filename,
#####file_size=file_size,
#####file_type=file_type,
#####mime_type=mime_type,
#####from_cache=False,
#####auto_download=auto_download,
#####)
#####)

#####self.logger.debug(
#####f'[DownloadsWatchdog] ■ Download completed: {suggested_filename} ({file_size} bytes) saved to {down
#####)

File is now tracked on filesystem, no need to track in memory

#####except Exception as e:
#####self.logger.error(

```

```

 """f'[DownloadsWatchdog] Error handling download at step "{locals().get("current_step", "unknown")}',
 """
 self.logger.error(
 """f'[DownloadsWatchdog] Download state - URL: {download.url}, filename: {download.suggested_filename}
 """
)
 finally:
 # Clean up tracking
 if download_id in self._active_downloads:
 del self._active_downloads[download_id]

 async def check_for_pdf_viewer(self, target_id: TargetID) -> bool:
 """Check if the current target is a PDF using network-based detection.

 This method avoids JavaScript execution that can crash WebSocket connections.
 Returns True if a PDF is detected and should be downloaded.
 """
 self.logger.debug(f'[DownloadsWatchdog] Checking if target {target_id} is PDF viewer...')

 # Use safe API - focus=False to avoid changing focus during PDF check
 try:
 session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)
 except ValueError as e:
 self.logger.warning(f'[DownloadsWatchdog] No session found for {target_id}: {e}')
 return False

 # Get URL from target
 target = self.browser_session.session_manager.get_target(target_id)
 if not target:
 self.logger.warning(f'[DownloadsWatchdog] No target found for {target_id}')
 return False
 page_url = target.url

 # Check cache first
 if page_url in self._pdf_viewer_cache:
 cached_result = self._pdf_viewer_cache[page_url]
 self.logger.debug(f'[DownloadsWatchdog] Using cached PDF check result for {page_url}: {cached_result}')
 return cached_result

 try:
 # Method 1: Check URL patterns (fastest, most reliable)
 url_is_pdf = self._check_url_for_pdf(page_url)
 if url_is_pdf:
 self.logger.debug(f'[DownloadsWatchdog] PDF detected via URL pattern: {page_url}')
 self._pdf_viewer_cache[page_url] = True
 return True
 chrome_pdf_viewer = self._is_chrome_pdf_viewer_url(page_url)
 if chrome_pdf_viewer:
 self.logger.debug(f'[DownloadsWatchdog] Chrome PDF viewer detected: {page_url}')
 self._pdf_viewer_cache[page_url] = True
 return True

 # Not a PDF
 self._pdf_viewer_cache[page_url] = False
 return False

 except Exception as e:
 self.logger.warning(f'[DownloadsWatchdog] Error checking for PDF viewer: {e}')
 self._pdf_viewer_cache[page_url] = False
 return False

 def _check_url_for_pdf(self, url: str) -> bool:
 """Check if URL indicates a PDF file."""
 if not url:
 return False

 url_lower = url.lower()

 # Direct PDF file extensions
 if url_lower.endswith('.pdf'):
 return True

 # PDF in path

```

```

 if '.pdf' in url_lower:
 return True

 # PDF MIME type in URL parameters
 if any(
 param in url_lower
 for param in [
 'content-type=application/pdf',
 'content-type=application%2fpdf',
 'mimetype=application/pdf',
 'type=application/pdf',
]
):
 return True

 return False

def _is_chrome_pdf_viewer_url(self, url: str) -> bool:
 """Check if this is Chrome's internal PDF viewer URL."""
 if not url:
 return False

 url_lower = url.lower()

 # Chrome PDF viewer uses chrome-extension:// URLs
 if 'chrome-extension://' in url_lower and 'pdf' in url_lower:
 return True

 # Chrome PDF viewer internal URLs
 if url_lower.startswith('chrome://') and 'pdf' in url_lower:
 return True

 return False

async def _check_network_headers_for_pdf(self, target_id: TargetID) -> bool:
 """Infer PDF via navigation history/URL; headers are not available post-navigation in this context."""
 try:
 import asyncio

 # Get CDP session
 temp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)

 # Get navigation history to find the main resource
 history = await asyncio.wait_for(
 temp_session.cdp_client.send.Page.getNavigationHistory(session_id=temp_session.session_id), timeout=
)

 current_entry = history.get('entries', [])
 if current_entry:
 current_index = history.get('currentIndex', 0)
 if 0 <= current_index < len(current_entry):
 current_url = current_entry[current_index].get('url', '')

 # Check if the URL itself suggests PDF
 if self._check_url_for_pdf(current_url):
 return True

 # Note: CDP doesn't easily expose response headers for completed navigations
 # For more complex cases, we'd need to set up Network.responseReceived listeners
 # before navigation, but that's overkill for most PDF detection cases

 return False

 except Exception as e:
 self.logger.debug(f'[DownloadSwatdog] Network headers check failed (non-critical): {e}')
 return False

 async def trigger_pdf_download(self, target_id: TargetID) -> str | None:
 """Trigger download of a PDF from Chrome's PDF viewer.

 Returns the download path if successful, None otherwise.
 """

```

```

self.logger.debug(f'[Downloadswatchdog] trigger_pdf_download called for target_id={target_id}')

if not self.browser_session.browser_profile.downloads_path:
 self.logger.warning(f'[Downloadswatchdog] ■ No downloads path configured, cannot save PDF download')
 return None

downloads_path = self.browser_session.browser_profile.downloads_path
self.logger.debug(f'[Downloadswatchdog] Downloads path: {downloads_path}')

try:
 # Create a temporary CDP session for this target without switching focus
 import asyncio

 self.logger.debug(f'[Downloadswatchdog] Creating CDP session for PDF download from target {target_id}')
 temp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)

 # Try to get the PDF URL with timeout
 result = await asyncio.wait_for(
 temp_session.cdp_client.send(Runtime.evaluate(
 params={
 'expression': """
 (() => {
 // For Chrome's PDF viewer, the actual URL is in window.location.href
 // The embed element's src is often "about:blank"
 const embedElement = document.querySelector('embed[type="application/x-google-chrome-pdf"]') ||
 document.querySelector('embed[type="application/pdf"]');
 if (embedElement) {
 // Chrome PDF viewer detected - use the page URL
 return { url: window.location.href };
 }
 // Fallback to window.location.href anyway
 return { url: window.location.href };
 })()
 """,
 'returnByValue': True,
 },
 session_id=temp_session.session_id,
)),
 timeout=5.0, # 5 second timeout to prevent hanging
)
 pdf_info = result.get('result', {}).get('value', {})

 pdf_url = pdf_info.get('url', '')
 if not pdf_url:
 self.logger.warning(f'[Downloadswatchdog] ■ Could not determine PDF URL for download {pdf_info}')
 return None

 # Generate filename from URL
 pdf_filename = os.path.basename(pdf_url.split('?')[0]) # Remove query params
 if not pdf_filename or not pdf_filename.endswith('.pdf'):
 parsed = urlparse(pdf_url)
 pdf_filename = os.path.basename(parsed.path) or 'document.pdf'
 if not pdf_filename.endswith('.pdf'):
 pdf_filename += '.pdf'

 self.logger.debug(f'[Downloadswatchdog] Generated filename: {pdf_filename}')

 # Check if already downloaded in this session
 self.logger.debug(f'[Downloadswatchdog] PDF_URL: {pdf_url}, session_pdf_urls: {self._session_pdf_urls}')
 if pdf_url in self._session_pdf_urls:
 existing_path = self._session_pdf_urls[pdf_url]
 self.logger.debug(f'[Downloadswatchdog] PDF already downloaded in session: {existing_path}')
 return existing_path

 # Generate unique filename if file exists from previous run
 downloads_dir = str(self.browser_session.browser_profile.downloads_path)
 os.makedirs(downloads_dir, exist_ok=True)
 final_filename = pdf_filename
 existing_files = os.listdir(downloads_dir)
 if pdf_filename in existing_files:
 # Generate unique name with (1), (2), etc.
 base, ext = os.path.splitext(pdf_filename)

```

```

#####counter = 1
#####while f'{base} ({counter}){ext}' in existing_files:
#####counter += 1
#####final_filename = f'{base} ({counter}){ext}'
#####self.logger.debug(f'[Downloadswatchdog] File exists, using: {final_filename}')

#####self.logger.debug(f'[Downloadswatchdog] Starting PDF download from: {pdf_url[:100]}...')

Download using JavaScript fetch to leverage browser cache
#####try:
Properly escape the URL to prevent JavaScript injection
#####escaped_pdf_url = json.dumps(pdf_url)

#####result = await asyncio.wait_for(
#####temp_session.cdp_client.send.Runtime.evaluate(
#####params={
#####'expression': f"""
#####(async () => {{
#####try {{
#####// Use fetch with cache: 'force-cache' to prioritize cached version
#####const response = await fetch({escaped_pdf_url}, {{
#####cache: 'force-cache'
#####}});
#####if (!response.ok) {{
#####throw new Error(`HTTP error! status: ${response.status}`);
#####}}
#####const blob = await response.blob();
#####const arrayBuffer = await blob.arrayBuffer();
#####const uint8Array = new Uint8Array(arrayBuffer);
#####
#####// Check if served from cache
#####const fromCache = response.headers.has('age') ||
#####!response.headers.has('date');
#####
#####return {{
#####data: Array.from(uint8Array),
#####fromCache: fromCache,
#####responseSize: uint8Array.length,
#####transferSize: response.headers.get('content-length') || 'unknown'
#####}};
#####}} catch (error) {{
#####throw new Error(`Fetch failed: ${error.message}`);
#####}}
#####}})()
#####"""
#####'awaitPromise': True,
#####'returnByValue': True,
#####},
#####session_id=temp_session.session_id,
#####),
#####timeout=10.0, # 10 second timeout for download operation
#####)
#####download_result = result.get('result', {}).get('value', {})

#####if download_result and download_result.get('data') and len(download_result['data']) > 0:
Ensure downloads directory exists
#####downloads_dir = str(self.browser_session.browser_profile.downloads_path)
#####os.makedirs(downloads_dir, exist_ok=True)
#####download_path = os.path.join(downloads_dir, final_filename)

Save the PDF asynchronously
#####async with await anyio.open_file(download_path, 'wb') as f:
#####await f.write(bytes(download_result['data']))

Verify file was written successfully
#####if os.path.exists(download_path):
#####actual_size = os.path.getsize(download_path)
#####self.logger.debug(
#####f'[Downloadswatchdog] PDF file written successfully: {download_path} ({actual_size} bytes)'
#####)
#####else:
#####self.logger.error(f'[Downloadswatchdog] ■ Failed to write PDF file to: {download_path}')

```

```

 #####return None

 ###### Log cache information
 #####cache_status = 'from cache' if download_result.get('fromCache') else 'from network'
 #####response_size = download_result.get('responseSize', 0)
 #####self.logger.debug(
 #####f'[Downloadswatchdog] ■ Auto-downloaded PDF ({cache_status}, {response_size:,} bytes): {download
 #####)

 ###### Store URL->path mapping for this session
 #####self._session_pdf_urls[pdf_url] = download_path

 ###### Emit file downloaded event
 #####self.logger.debug(f'[Downloadswatchdog] Dispatching FileDownloadedEvent for {final_filename}')
 #####self.event_bus.dispatch(
 #####FileDownloadedEvent(
 #####url=pdf_url,
 #####path=download_path,
 #####file_name=final_filename,
 #####file_size=response_size,
 #####file_type='pdf',
 #####mime_type='application/pdf',
 #####from_cache=download_result.get('fromCache', False),
 #####auto_download=True,
 #####)
 #####)

 ###### No need to detach - session is cached
 #####return download_path
 #####else:
 #####self.logger.warning(f'[Downloadswatchdog] No data received when downloading PDF from {pdf_url}')
 #####return None

 #####except Exception as e:
 #####self.logger.warning(f'[Downloadswatchdog] Failed to auto-download PDF from {pdf_url}: {type(e).__na
 #####return None

 #####except TimeoutError:
 #####self.logger.debug('[Downloadswatchdog] PDF download operation timed out')
 #####return None
 #####except Exception as e:
 #####self.logger.error(f'[Downloadswatchdog] Error in PDF download: {type(e).__name__}: {e}')
 #####return None

 #####@staticmethod
 #####def _get_unique_filename(directory: str, filename: str) -> str:
 ##### """Generate a unique filename for downloads by appending (1), (2), etc., if a file already exists."""
 ##### base, ext = os.path.splitext(filename)
 ##### counter = 1
 ##### new_filename = filename
 ##### while os.path.exists(os.path.join(directory, new_filename)):
 ##### new_filename = f'{base} ({counter}){ext}'
 ##### counter += 1
 ##### return new_filename

Fix Pydantic circular dependency - this will be called from session.py after BrowserSession is defined

```

## ***.\system\browser\watchdogs\har\_recording\_watchdog.py***

```

"""HAR Recording Watchdog for Browser-Use sessions.

Captures HTTPS network activity via CDP Network domain and writes a HAR 1.2
file on browser shutdown. Respects `record_har_content` (omit/embed/attach)
and `record_har_mode` (full/minimal).
"""

from __future__ import annotations

```



```

import base64
import hashlib
import json
from dataclasses import dataclass, field
from importlib import metadata as importlib_metadata
from pathlib import Path
from typing import ClassVar

from bubus import BaseEvent
from cdp_use.cdp.network.events import (
 ■DataReceivedEvent,
 ■LoadingFailedEvent,
 ■LoadingFinishedEvent,
 ■RequestWillBeSentEvent,
 ■ResponseReceivedEvent,
)
from cdp_use.cdp.page.events import FrameNavigatedEvent, LifecycleEventEvent

from system.browser.events import BrowserConnectedEvent, BrowserStopEvent
from system.browser.watchdog_base import BaseWatchdog

@dataclass
class _HarContent:
 ■mime_type: str | None = None
 ■text_b64: str | None = None # for embed
 ■file_rel: str | None = None # for attach
 ■size: int | None = None

@dataclass
class _HarEntryBuilder:
 ■request_id: str = ''
 ■frame_id: str | None = None
 ■document_url: str | None = None
 ■url: str | None = None
 ■method: str | None = None
 ■request_headers: dict = field(default_factory=dict)
 ■request_body: bytes | None = None
 ■post_data: str | None = None # CDP postData field
 ■status: int | None = None
 ■status_text: str | None = None
 ■response_headers: dict = field(default_factory=dict)
 ■mime_type: str | None = None
 ■encoded_data: bytearray = field(default_factory=bytearray)
 ■failed: bool = False
 ■# timing info (CDP timestamps are monotonic seconds); wallTime is epoch seconds
 ■ts_request: float | None = None
 ■wall_time_request: float | None = None
 ■ts_response: float | None = None
 ■ts_finished: float | None = None
 ■encoded_data_length: int | None = None
 ■response_body: bytes | None = None
 ■content_length: int | None = None # From Content-Length header
 ■protocol: str | None = None
 ■server_ip_address: str | None = None
 ■server_port: int | None = None
 ■security_details: dict | None = None
 ■transfer_size: int | None = None

def _is_https(url: str | None) -> bool:
 ■return bool(url and url.lower().startswith('https://'))

def _origin(url: str) -> str:
 ■# Very small origin extractor, assumes https URLs
 ■# https://host[:port]/...
 ■if not url:
 ■ ■return ''
 ■try:
 ■ ■without_scheme = url.split('://', 1)[1]

```

```

 ■■host_port = without_scheme.split('/', 1)[0]
 ■■return f'https://{host_port}'
 ■■except Exception:
 ■■return ''

def _mime_to_extension(mime_type: str | None) -> str:
 ■■■"Map MIME type to file extension, matching Playwright's behavior."
 ■■if not mime_type:
 ■■return 'bin'

 ■■mime_lower = mime_type.lower().split(';')[0].strip()

 ■■# Common MIME type to extension mapping
 ■■mime_map = {
 ■■'text/html': 'html',
 ■■'text/css': 'css',
 ■■'text/javascript': 'js',
 ■■'application/javascript': 'js',
 ■■'application/x-javascript': 'js',
 ■■'application/json': 'json',
 ■■'application/xml': 'xml',
 ■■'text/xml': 'xml',
 ■■'text/plain': 'txt',
 ■■'image/png': 'png',
 ■■'image/jpeg': 'jpg',
 ■■'image/jpg': 'jpg',
 ■■'image/gif': 'gif',
 ■■'image/webp': 'webp',
 ■■'image/svg+xml': 'svg',
 ■■'image/x-icon': 'ico',
 ■■'font/woff': 'woff',
 ■■'font/woff2': 'woff2',
 ■■'application/font-woff': 'woff',
 ■■'application/font-woff2': 'woff2',
 ■■'application/x-font-woff': 'woff',
 ■■'application/x-font-woff2': 'woff2',
 ■■'font/ttf': 'ttf',
 ■■'application/x-font-ttf': 'ttf',
 ■■'font/otf': 'otf',
 ■■'application/x-font-opentype': 'otf',
 ■■'application/pdf': 'pdf',
 ■■'application/zip': 'zip',
 ■■'application/x-zip-compressed': 'zip',
 ■■'video/mp4': 'mp4',
 ■■'video/webm': 'webm',
 ■■'audio/mpeg': 'mp3',
 ■■'audio/mp3': 'mp3',
 ■■'audio/wav': 'wav',
 ■■'audio/ogg': 'ogg',
 ■■}

 ■■return mime_map.get(mime_lower, 'bin')

def _generate_har_filename(content: bytes, mime_type: str | None) -> str:
 ■■■"Generate a hash-based filename for HAR attach mode, matching Playwright's format."
 ■■content_hash = hashlib.sha1(content).hexdigest()
 ■■extension = _mime_to_extension(mime_type)
 ■■return f'{content_hash}.{extension}'

class HarRecordingWatchdog(BaseWatchdog):
 ■■■"Collects HTTPS requests/responses and writes a HAR 1.2 file on stop."

 ■■LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [BrowserConnectedEvent, BrowserStopEvent]
 ■■EMITS: ClassVar[list[type[BaseEvent]]] = []

 ■■def __init__(self, *args, **kwargs) -> None:
 ■■super().__init__(*args, **kwargs)
 ■■self._enabled: bool = False
 ■■self._entries: dict[str, _HarEntryBuilder] = {}

```

```

 self._top_level_pages: dict[
 str, dict
] = {} # frameId -> {url, title, startedDateTime, monotonic_start, onContentLoaded, onLoad}

 async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 profile = self.browser_session.browser_profile
 if not profile.record_har_path:
 return

 ## Normalize config
 self._content_mode = (profile.record_har_content or 'embed').lower()
 self._mode = (profile.record_har_mode or 'full').lower()
 self._har_path = Path(str(profile.record_har_path)).expanduser().resolve()
 self._har_dir = self._har_path.parent
 self._har_dir.mkdir(parents=True, exist_ok=True)

 try:
 ## Enable Network and Page domains for events
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 await cdp_session.cdp_client.send.Network.enable(session_id=cdp_session.session_id)
 await cdp_session.cdp_client.send.Page.enable(session_id=cdp_session.session_id)

 ## Query browser version for HAR log.browser
 try:
 version_info = await self.browser_session.cdp_client.send.Browser.getVersion()
 self._browser_name = version_info.get('product') or 'Chromium'
 self._browser_version = version_info.get('jsVersion') or ''
 except Exception:
 self._browser_name = 'Chromium'
 self._browser_version = ''

 cdp = self.browser_session.cdp_client.register
 cdp.Network.requestWillBeSent(self._on_request_will_be_sent)
 cdp.Network.responseReceived(self._on_response_received)
 cdp.Network.dataReceived(self._on_data_received)
 cdp.Network.loadingFinished(self._on_loading_finished)
 cdp.Network.loadingFailed(self._on_loading_failed)
 cdp.Page.lifecycleEvent(self._on_lifecycle_event)
 cdp.Page.frameNavigated(self._on_frame_navigated)

 self._enabled = True
 self.logger.info(f'Starting HAR recording to {self._har_path}')
 except Exception as e:
 self.logger.warning(f'Failed to enable HAR recording: {e}')
 self._enabled = False

 async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 if not self._enabled:
 return
 try:
 await self._write_har()
 self.logger.info(f'HAR file saved: {self._har_path}')
 except Exception as e:
 self.logger.warning(f'Failed to write HAR: {e}')

 ## ===== CDP Event Handlers (sync) =====
 def _on_request_will_be_sent(self, params: RequestWillBeSentEvent, session_id: str | None) -> None:
 try:
 req = params.get('request', {}) if hasattr(params, 'get') else getattr(params, 'request', {})
 url = req.get('url') if isinstance(req, dict) else getattr(req, 'url', None)
 if not _is_https(url):
 return # HTTPS-only requirement (only HTTPS requests are recorded for now)

 request_id = params.get('requestId') if hasattr(params, 'get') else getattr(params, 'requestId', None)
 if not request_id:
 return

 entry = self._entries.setdefault(request_id, _HarEntryBuilder(request_id=request_id))
 entry.url = url
 entry.method = req.get('method') if isinstance(req, dict) else getattr(req, 'method', None)
 entry.post_data = req.get('postData') if isinstance(req, dict) else getattr(req, 'postData', None)
 ## Convert headers to plain dict, handling various formats

```

```

#####headers_raw = req.get('headers') if isinstance(req, dict) else getattr(req, 'headers', None)
#####if headers_raw is None:
entry.request_headers = {}
#####elif isinstance(headers_raw, dict):
entry.request_headers = {k.lower(): str(v) for k, v in headers_raw.items()}
#####elif isinstance(headers_raw, list):
entry.request_headers = {
h.get('name', '').lower(): str(h.get('value') or '') for h in headers_raw if isinstance(h, dict)
}
#####else:
Handle Headers type or other formats - convert to dict
try:
headers_dict = dict(headers_raw) if hasattr(headers_raw, '__iter__') else {}
entry.request_headers = {k.lower(): str(v) for k, v in headers_dict.items()}
except Exception:
entry.request_headers = {}

#####entry.frame_id = params.get('frameId') if hasattr(params, 'get') else getattr(params, 'frameId', None)
#####entry.document_url = (
params.get('documentURL')
if hasattr(params, 'get')
else getattr(params, 'documentURL', None) or entry.document_url
#####)

Timing anchors
#####entry.ts_request = params.get('timestamp') if hasattr(params, 'get') else getattr(params, 'timestamp', None)
#####entry.wall_time_request = params.get('wallTime') if hasattr(params, 'get') else getattr(params, 'wallTime', None)

Track top-level navigations for page context
#####req_type = params.get('type') if hasattr(params, 'get') else getattr(params, 'type', None)
#####is_same_doc = (
params.get('isSameDocument', False) if hasattr(params, 'get') else getattr(params, 'isSameDocument', None)
#####)
#####if req_type == 'Document' and not is_same_doc:
best-effort: consider as navigation
if entry.frame_id and url:
if entry.frame_id not in self._top_level_pages:
self._top_level_pages[entry.frame_id] = {
'url': str(url),
'title': str(url), # Default to URL, will be updated from DOM
'startedDateTime': entry.wall_time_request,
'monotonic_start': entry.ts_request, # Track monotonic start time for timing calculations
'onContentLoaded': -1,
'onLoad': -1,
}
else:
Update startedDateTime and monotonic_start if this is earlier
page_info = self._top_level_pages[entry.frame_id]
if entry.wall_time_request and (
page_info['startedDateTime'] is None or entry.wall_time_request < page_info['startedDateTime']
):
page_info['startedDateTime'] = entry.wall_time_request
page_info['monotonic_start'] = entry.ts_request
except Exception as e:
self.logger.debug(f'requestWillBeSent handling error: {e}')

#####def _on_response_received(self, params: ResponseReceivedEvent, session_id: str | None) -> None:
try:
request_id = params.get('requestId') if hasattr(params, 'get') else getattr(params, 'requestId', None)
if not request_id or request_id not in self._entries:
return
response = params.get('response', {}) if hasattr(params, 'get') else getattr(params, 'response', {})
entry = self._entries[request_id]
entry.status = response.get('status') if isinstance(response, dict) else getattr(response, 'status', None)
entry.status_text = (
response.get('statusText') if isinstance(response, dict) else getattr(response, 'statusText', None)
)

Extract Content-Length for compression calculation (before converting headers)
#####headers_raw = response.get('headers') if isinstance(response, dict) else getattr(response, 'headers', None)
#####if headers_raw:
if isinstance(headers_raw, dict):

```

```

#####cl_str = headers_raw.get('content-length') or headers_raw.get('Content-Length')
#####elif isinstance(headers_raw, list):
#####cl_header = next(
#####(h for h in headers_raw if isinstance(h, dict) and h.get('name', '').lower() == 'content-length')
#####)
#####cl_str = cl_header.get('value') if cl_header else None
#####else:
#####cl_str = None
#####if cl_str:
#####try:
#####entry.content_length = int(cl_str)
#####except Exception:
#####pass

Convert headers to plain dict, handling various formats
#####if headers_raw is None:
#####entry.response_headers = {}
#####elif isinstance(headers_raw, dict):
#####entry.response_headers = {k.lower(): str(v) for k, v in headers_raw.items()}
#####elif isinstance(headers_raw, list):
#####entry.response_headers = {
#####h.get('name', '').lower(): str(h.get('value') or '') for h in headers_raw if isinstance(h, dict)
#####}
#####else:
Handle Headers type or other formats - convert to dict
#####try:
#####headers_dict = dict(headers_raw) if hasattr(headers_raw, '__iter__') else {}
#####entry.response_headers = {k.lower(): str(v) for k, v in headers_dict.items()}
#####except Exception:
#####entry.response_headers = {}

#####entry.mime_type = response.get('mimeType') if isinstance(response, dict) else getattr(response, 'mimeType', None)
#####entry.ts_response = params.get('timestamp') if hasattr(params, 'get') else getattr(params, 'timestamp', None)

#####protocol_raw = response.get('protocol') if isinstance(response, dict) else getattr(response, 'protocol', None)
#####if protocol_raw:
#####protocol_lower = str(protocol_raw).lower()
#####if protocol_lower == 'h2' or protocol_lower.startswith('http/2'):
#####entry.protocol = 'HTTP/2.0'
#####elif protocol_lower.startswith('http/1.1'):
#####entry.protocol = 'HTTP/1.1'
#####elif protocol_lower.startswith('http/1.0'):
#####entry.protocol = 'HTTP/1.0'
#####else:
#####entry.protocol = str(protocol_raw).upper()

#####entry.server_ip_address = (
#####response.get('remoteIPAddress') if isinstance(response, dict) else getattr(response, 'remoteIPAddress', None)
#####)
#####server_port_raw = response.get('remotePort') if isinstance(response, dict) else getattr(response, 'remotePort', None)
#####if server_port_raw is not None:
#####try:
#####entry.server_port = int(server_port_raw)
#####except (ValueError, TypeError):
#####pass

Extract security details (TLS info)
#####security_details_raw = (
#####response.get('securityDetails') if isinstance(response, dict) else getattr(response, 'securityDetails', None)
#####)
#####if security_details_raw:
#####try:
#####entry.security_details = dict(security_details_raw)
#####except Exception:
#####pass
#####except Exception as e:
#####self.logger.debug(f'responseReceived handling error: {e}')

#####def _on_data_received(self, params: DataReceivedEvent, session_id: str | None) -> None:
#####try:
#####request_id = params.get('requestId') if hasattr(params, 'get') else getattr(params, 'requestId', None)
#####if not request_id or request_id not in self._entries:

```

```

 return
 data = params.get('data') if hasattr(params, 'get') else getattr(params, 'data', None)
 if isinstance(data, str):
 try:
 self._entries[request_id].encoded_data.extend(data.encode('latin1'))
 except Exception:
 pass
 except Exception as e:
 self.logger.debug(f'dataReceived handling error: {e}')

 def _on_loading_finished(self, params: LoadingFinishedEvent, session_id: str | None) -> None:
 try:
 request_id = params.get('requestId') if hasattr(params, 'get') else getattr(params, 'requestId', None)
 if not request_id or request_id not in self._entries:
 return
 entry = self._entries[request_id]
 entry.ts_finished = params.get('timestamp')
 # Fetch response body via CDP as dataReceived may be incomplete
 import asyncio as _asyncio

 async def _fetch_body(self_ref, req_id, sess_id):
 try:
 resp = await self_ref.browser_session.cdp_client.send.Network.getResponseBody(
 params={'requestId': req_id}, session_id=sess_id
)
 data = resp.get('body', b'')
 if resp.get('base64Encoded'):
 import base64 as _b64

 data = _b64.b64decode(data)
 else:
 # Ensure data is bytes even if CDP returns a string
 if isinstance(data, str):
 data = data.encode('utf-8', errors='replace')
 # Ensure we always have bytes
 if not isinstance(data, bytes):
 data = bytes(data) if data else b''
 entry.response_body = data
 except Exception:
 pass

 # Always schedule the response body fetch task
 _asyncio.create_task(_fetch_body(self, request_id, session_id))

 encoded_length = (
 params.get('encodedDataLength') if hasattr(params, 'get') else getattr(params, 'encodedDataLength', None)
)
 if encoded_length is not None:
 try:
 entry.encoded_data_length = int(encoded_length)
 entry.transfer_size = entry.encoded_data_length
 except Exception:
 entry.encoded_data_length = None
 except Exception as e:
 self.logger.debug(f'loadingFinished handling error: {e}')

 def _on_loading_failed(self, params: LoadingFailedEvent, session_id: str | None) -> None:
 try:
 request_id = params.get('requestId') if hasattr(params, 'get') else getattr(params, 'requestId', None)
 if request_id and request_id in self._entries:
 self._entries[request_id].failed = True
 except Exception as e:
 self.logger.debug(f'loadingFailed handling error: {e}')

 # ===== HAR Writing =====
 def _on_lifecycle_event(self, params: LifecycleEventEvent, session_id: str | None) -> None:
 """Handle Page.lifecycleEvent for tracking page load timings."""
 try:
 frame_id = params.get('frameId') if hasattr(params, 'get') else getattr(params, 'frameId', None)
 name = params.get('name') if hasattr(params, 'get') else getattr(params, 'name', None)
 timestamp = params.get('timestamp') if hasattr(params, 'get') else getattr(params, 'timestamp', None)
 if not frame_id or not name or frame_id not in self._top_level_pages:

```

```

 return

 page_info = self._top_level_pages[frame_id]
 # Use monotonic_start instead of startedDateTime (wall-clock) for timing calculations
 monotonic_start = page_info.get('monotonic_start')

 if name == 'DOMContentLoaded' and monotonic_start is not None:
 # Calculate milliseconds since page start using monotonic timestamps
 try:
 elapsed_ms = int(round((timestamp - monotonic_start) * 1000))
 page_info['onContentLoaded'] = max(0, elapsed_ms)
 except Exception:
 pass
 elif name == 'load' and monotonic_start is not None:
 try:
 elapsed_ms = int(round((timestamp - monotonic_start) * 1000))
 page_info['onLoad'] = max(0, elapsed_ms)
 except Exception:
 pass
 except Exception as e:
 self.logger.debug(f'lifecycleEvent handling error: {e}')

 def _on_frame_navigated(self, params: FrameNavigatedEvent, session_id: str | None) -> None:
 """Handle Page.frameNavigated to update page title from DOM."""
 try:
 frame = params.get('frame') if hasattr(params, 'get') else getattr(params, 'frame', None)
 if not frame:
 return

 frame_id = frame.get('id') if isinstance(frame, dict) else getattr(frame, 'id', None)
 title = (
 frame.get('name') or frame.get('url')
 if isinstance(frame, dict)
 else getattr(frame, 'name', None) or getattr(frame, 'url', None)
)

 if frame_id and frame_id in self._top_level_pages:
 # Try to get actual page title via Runtime.evaluate if possible
 # For now, use frame name or URL as fallback
 if title:
 self._top_level_pages[frame_id]['title'] = str(title)
 except Exception as e:
 self.logger.debug(f'frameNavigated handling error: {e}')

 # ===== HAR Writing =====
 async def _write_har(self) -> None:
 # Filter by mode and HTTPS already respected at collection time
 entries = [e for e in self._entries.values() if self._include_entry(e)]

 har_entries = []
 sidecar_dir: Path | None = None
 if self._content_mode == 'attach':
 sidecar_dir = self._har_dir / f'{self._har_path.stem}_har_parts'
 sidecar_dir.mkdir(parents=True, exist_ok=True)

 for e in entries:
 content_obj: dict = {'mimeType': e.mime_type or ''}

 # Get body data, preferring response_body over encoded_data
 if e.response_body is not None:
 body_data = e.response_body
 else:
 body_data = e.encoded_data

 # Defensive conversion: ensure body_data is always bytes
 if isinstance(body_data, str):
 body_bytes = body_data.encode('utf-8', errors='replace')
 elif isinstance(body_data, bytearray):
 body_bytes = bytes(body_data)
 elif isinstance(body_data, bytes):
 body_bytes = body_data
 else:

```

```

Fallback: try to convert to bytes
try:
body_bytes = bytes(body_data) if body_data else b''
except (TypeError, ValueError):
body_bytes = b''

content_size = len(body_bytes)

Calculate compression (bytes saved by compression)
compression = 0
if e.content_length is not None and e.encoded_data_length is not None:
compression = max(0, e.content_length - e.encoded_data_length)

if self._content_mode == 'embed' and content_size > 0:
Prefer plain text; fallback to base64 only if decoding fails
try:
text_decoded = body_bytes.decode('utf-8')
content_obj['text'] = text_decoded
content_obj['size'] = content_size
content_obj['compression'] = compression
except UnicodeDecodeError:
content_obj['text'] = base64.b64encode(body_bytes).decode('ascii')
content_obj['encoding'] = 'base64'
content_obj['size'] = content_size
content_obj['compression'] = compression
elif self._content_mode == 'attach' and content_size > 0 and sidecar_dir is not None:
filename = _generate_har_filename(body_bytes, e.mime_type)
(sidecar_dir / filename).write_bytes(body_bytes)
content_obj['_file'] = filename
content_obj['size'] = content_size
content_obj['compression'] = compression
else:
omit or empty
content_obj['size'] = content_size
if content_size > 0:
content_obj['compression'] = compression

started_date_time, total_time_ms, timings = self._compute_timings(e)
req_headers_list = [{'name': k, 'value': str(v)} for k, v in (e.request_headers or {}).items()]
resp_headers_list = [{'name': k, 'value': str(v)} for k, v in (e.response_headers or {}).items()]
request_headers_size = self._calc_headers_size(e.method or 'GET', e.url or '', req_headers_list)
response_headers_size = self._calc_headers_size(None, None, resp_headers_list)
request_body_size = self._calc_request_body_size(e)
request_post_data = None
if e.post_data and self._content_mode != 'omit':
if self._content_mode == 'embed':
request_post_data = {'mimeType': e.request_headers.get('content-type', ''), 'text': e.post_data}
elif self._content_mode == 'attach' and sidecar_dir is not None:
post_data_bytes = e.post_data.encode('utf-8')
req_mime_type = e.request_headers.get('content-type', 'text/plain')
req_filename = _generate_har_filename(post_data_bytes, req_mime_type)
(sidecar_dir / req_filename).write_bytes(post_data_bytes)
request_post_data = {
'mimeType': req_mime_type,
'_file': req_filename,
}

http_version = e.protocol if e.protocol else 'HTTP/1.1'

response_body_size = e.transfer_size
if response_body_size is None:
response_body_size = e.encoded_data_length
if response_body_size is None:
response_body_size = content_size if content_size > 0 else -1

entry_dict = {
'startedDateTime': started_date_time,
'time': total_time_ms,
'request': {
'method': e.method or 'GET',
'url': e.url or '',
'httpVersion': http_version,

```



```

#####'headers': req_headers_list,
#####'queryString': [],
#####'cookies': [],
#####'headersSize': request_headers_size,
#####'bodySize': request_body_size,
#####'postData': request_post_data,
#####},
#####'response': {
#####'status': e.status or 0,
#####'statusText': e.status_text or '',
#####'httpVersion': http_version,
#####'headers': resp_headers_list,
#####'cookies': [],
#####'content': content_obj,
#####'redirectURL': '',
#####'headersSize': response_headers_size,
#####'bodySize': response_body_size,
#####},
#####'cache': {},
#####'timings': timings,
#####'pageref': self._page_ref_for_entry(e),
#####}

Add security/TLS details if available
#####if e.server_ip_address:
#####entry_dict['serverIPAddress'] = e.server_ip_address
#####if e.server_port is not None:
#####entry_dict['_serverPort'] = e.server_port
#####if e.security_details:
Filter to match Playwright's minimal security details set
#####security_filtered = {}
#####if 'protocol' in e.security_details:
#####security_filtered['protocol'] = e.security_details['protocol']
#####if 'subjectName' in e.security_details:
#####security_filtered['subjectName'] = e.security_details['subjectName']
#####if 'issuer' in e.security_details:
#####security_filtered['issuer'] = e.security_details['issuer']
#####if 'validFrom' in e.security_details:
#####security_filtered['validFrom'] = e.security_details['validFrom']
#####if 'validTo' in e.security_details:
#####security_filtered['validTo'] = e.security_details['validTo']
#####if security_filtered:
#####entry_dict['_securityDetails'] = security_filtered
#####if e.transfer_size is not None:
#####entry_dict['response']['_transferSize'] = e.transfer_size

#####har_entries.append(entry_dict)

Try to include our library version in creator
#####try:
#####bu_version = importlib_metadata.version('browser-use')
#####except Exception:
Fallback when running from source without installed package metadata
#####bu_version = 'dev'

#####har_obj = {
#####'log': {
#####'version': '1.2',
#####'creator': {'name': 'browser-use', 'version': bu_version},
#####'browser': {'name': self._browser_name, 'version': self._browser_version},
#####'pages': [
#####{
#####'id': f'page@{pid}', # Use Playwright format: "page@{frame_id}"
#####'title': page_info.get('title', page_info.get('url', '')),
#####'startedDateTime': self._format_page_started_datetime(page_info.get('startedDateTime')),
#####'pageTimings': (
#####(lambda _ocl, _ol: ({k: v for k, v in ((('onContentLoaded', _ocl), ('onLoad', _ol)) if v is not None
#####(page_info.get('onContentLoaded') if page_info.get('onContentLoaded', -1) >= 0 else None),
#####(page_info.get('onLoad') if page_info.get('onLoad', -1) >= 0 else None),
#####)),
#####),
#####}
#####]
#####}

```

```

 for pid, page_info in self._top_level_pages.items():
],
 'entries': har_entries,
 }
 }

 tmp_path = self._har_path.with_suffix(self._har_path.suffix + '.tmp')
 # Write as bytes explicitly to avoid any text/binary mode confusion in different environments
 tmp_path.write_bytes(json.dumps(har_obj, indent=2).encode('utf-8'))
 tmp_path.replace(self._har_path)

 def _format_page_started_datetime(self, timestamp: float | None) -> str:
 """Format page startedDate from timestamp."""
 if timestamp is None:
 return ''
 try:
 from datetime import datetime, timezone

 return datetime.fromtimestamp(timestamp, tz=timezone.utc).isoformat().replace('+00:00', 'Z')
 except Exception:
 return ''

 def _page_ref_for_entry(self, e: _HarEntryBuilder) -> str | None:
 # Use Playwright format: "page@{frame_id}" if frame_id is known
 if e.frame_id and e.frame_id in self._top_level_pages:
 return f'page@{e.frame_id}'
 return None

 def _include_entry(self, e: _HarEntryBuilder) -> bool:
 if not _is_https(e.url):
 return False
 # Filter out favicon requests (matching Playwright behavior)
 if e.url and '/favicon.ico' in e.url.lower():
 return False
 if getattr(self, '_mode', 'full') == 'full':
 return True
 # minimal: include main document and same-origin subresources
 if e.frame_id and e.frame_id in self._top_level_pages:
 page_info = self._top_level_pages[e.frame_id]
 page_url = page_info.get('url') if isinstance(page_info, dict) else page_info
 return _origin(e.url or '') == _origin(page_url or '')
 return False

 # ===== Helpers =====
 def _compute_timings(self, e: _HarEntryBuilder) -> tuple[str, int, dict]:
 # startedDate from wall_time_request in ISO8601 Z
 started = ''
 try:
 if e.wall_time_request is not None:
 from datetime import datetime, timezone

 started = datetime.fromtimestamp(e.wall_time_request, tz=timezone.utc).isoformat().replace('+00:00', 'Z')
 except Exception:
 started = ''

 # Calculate timings - CDP doesn't always provide DNS/connect/SSL breakdown
 # Default to 0 for unavailable timings, calculate what we can from timestamps
 dns_ms = 0
 connect_ms = 0
 ssl_ms = 0
 send_ms = 0
 wait_ms = 0
 receive_ms = 0

 if e.ts_request is not None and e.ts_response is not None:
 wait_ms = max(0, int(round((e.ts_response - e.ts_request) * 1000)))

 if e.ts_response is not None and e.ts_finished is not None:
 receive_ms = max(0, int(round((e.ts_finished - e.ts_response) * 1000)))

 # Note: DNS, connect, and SSL timings would require additional CDP events or ResourceTiming API
 # For now, we structure the timings dict to match Playwright format

```

```

 ## but leave DNS/connect/SSL as 0 since CDP doesn't provide this breakdown directly

 total = dns_ms + connect_ms + ssl_ms + send_ms + wait_ms + receive_ms
 return (
 started,
 total,
 {
 'dns': dns_ms,
 'connect': connect_ms,
 'ssl': ssl_ms,
 'send': send_ms,
 'wait': wait_ms,
 'receive': receive_ms,
 },
)

 def _calc_headers_size(self, method: str | None, url: str | None, headers_list: list[dict]) -> int:
 try:
 # Approximate per RFC: sum of header lines + CRLF; include request/status line only for request
 size = 0
 if method and url:
 # Use HTTP/1.1 request line approximation
 size += len(f'{method} {url} HTTP/1.1\r\n'.encode('latin1'))
 for h in headers_list:
 size += len(f'{h.get("name", "")}: {h.get("value", "")}\r\n'.encode('latin1'))
 size += len(b'\r\n')
 return size
 except Exception:
 return -1

 def _calc_request_body_size(self, e: _HarEntryBuilder) -> int:
 ## Try Content-Length header first; else post_data; else request_body; else 0 for GET/HEAD, -1 if unknown
 try:
 cl = None
 if e.request_headers:
 cl = e.request_headers.get('content-length') or e.request_headers.get('Content-Length')
 if cl is not None:
 return int(cl)
 if e.post_data:
 return len(e.post_data.encode('utf-8'))
 if e.request_body is not None:
 return len(e.request_body)
 # GET/HEAD requests typically have no body
 if e.method and e.method.upper() in ('GET', 'HEAD'):
 return 0
 except Exception:
 pass
 return -1

```

## ***.\system\browser\watchdogs\local\_browser\_watchdog.py***

```

"""Local browser watchdog for managing browser subprocess lifecycle."""

import asyncio
import os
import shutil
import tempfile
from pathlib import Path
from typing import TYPE_CHECKING, Any, ClassVar

import psutil
from bubus import BaseEvent
from pydantic import PrivateAttr

from system.browser.events import (
 BrowserKillEvent,
 BrowserLaunchEvent,
 BrowserLaunchResult,

```

```

 BrowserStopEvent,
)
from system.browser.watchdog_base import BaseWatchdog
from system.observability import observe_debug

if TYPE_CHECKING:
 pass

class LocalBrowserWatchdog(BaseWatchdog):
 """Manages local browser subprocess lifecycle."""

 # Events this watchdog listens to
 LISTENS_TO: ClassVar[list[type[BaseEvent[Any]]]] = [
 BrowserLaunchEvent,
 BrowserKillEvent,
 BrowserStopEvent,
]

 # Events this watchdog emits
 EMITS: ClassVar[list[type[BaseEvent[Any]]]] = []

 # Private state for subprocess management
 _subprocess: psutil.Process | None = PrivateAttr(default=None)
 _owns_browser_resources: bool = PrivateAttr(default=True)
 _temp_dirs_to_cleanup: list[Path] = PrivateAttr(default_factory=list)
 _original_user_data_dir: str | None = PrivateAttr(default=None)

 @observe_debug(ignore_input=True, ignore_output=True, name='browser_launch_event')
 async def on_BrowserLaunchEvent(self, event: BrowserLaunchEvent) -> BrowserLaunchResult:
 """Launch a local browser process."""

 try:
 self.logger.debug('[LocalBrowserWatchdog] Received BrowserLaunchEvent, launching local browser...')

 # self.logger.debug('[LocalBrowserWatchdog] Calling _launch_browser...')
 process, cdp_url = await self._launch_browser()
 self._subprocess = process
 # self.logger.debug(f'[LocalBrowserWatchdog] _launch_browser returned: process={process}, cdp_url={cdp_url}')

 return BrowserLaunchResult(cdp_url=cdp_url)
 except Exception as e:
 self.logger.error(f'[LocalBrowserWatchdog] Exception in on_BrowserLaunchEvent: {e}', exc_info=True)
 raise

 async def on_BrowserKillEvent(self, event: BrowserKillEvent) -> None:
 """Kill the local browser subprocess."""
 self.logger.debug('[LocalBrowserWatchdog] Killing local browser process')

 if self._subprocess:
 await self._cleanup_process(self._subprocess)
 self._subprocess = None

 # Clean up temp directories if any were created
 for temp_dir in self._temp_dirs_to_cleanup:
 self._cleanup_temp_dir(temp_dir)
 self._temp_dirs_to_cleanup.clear()

 # Restore original user_data_dir if it was modified
 if self._original_user_data_dir is not None:
 self.browser_session.browser_profile.user_data_dir = self._original_user_data_dir
 self._original_user_data_dir = None

 self.logger.debug('[LocalBrowserWatchdog] Browser cleanup completed')

 async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 """Listen for BrowserStopEvent and dispatch BrowserKillEvent without awaiting it."""
 if self.browser_session.is_local and self._subprocess:
 self.logger.debug('[LocalBrowserWatchdog] BrowserStopEvent received, dispatching BrowserKillEvent')
 # Dispatch BrowserKillEvent without awaiting so it gets processed after all BrowserStopEvent handlers
 self.event_bus.dispatch(BrowserKillEvent())
 @observe_debug(ignore_input=True, ignore_output=True, name='launch_browser_process')

```

```

 async def _launch_browser(self, max_retries: int = 3) -> tuple[psutil.Process, str]:
 """Launch browser process and return (process, cdp_url).

 Handles launch errors by falling back to temporary directories if needed.

 Returns:
 Tuple of (psutil.Process, cdp_url)
 """
 # Keep track of original user_data_dir to restore if needed
 profile = self.browser_session.browser_profile
 self._original_user_data_dir = str(profile.user_data_dir) if profile.user_data_dir else None
 self._temp_dirs_to_cleanup = []

 for attempt in range(max_retries):
 try:
 # Get launch args from profile
 launch_args = profile.get_args()

 # Add debugging port
 debug_port = self._find_free_port()
 launch_args.extend(
 [
 f'--remote-debugging-port={debug_port}',
]
)
 assert '--user-data-dir' in str(launch_args), (
 'User data dir must be set somewhere in launch args to a non-default path, otherwise Chrome will not work'
)

 # Get browser executable
 # Priority: custom executable > fallback paths > playwright subprocess
 if profile.executable_path:
 browser_path = profile.executable_path
 self.logger.debug(f'[LocalBrowserWatchdog] ■ Using custom local browser executable_path= {browser_path}')
 else:
 # self.logger.debug('[LocalBrowserWatchdog] ■ Looking for local browser binary path...')
 # Try fallback paths first (system browsers preferred)
 browser_path = self._find_installed_browser_path()
 if not browser_path:
 self.logger.error(
 '[LocalBrowserWatchdog] ■ No local browser binary found, installing browser using playwright'
)
 browser_path = await self._install_browser_with_playwright()

 self.logger.debug(f'[LocalBrowserWatchdog] ■ Found local browser installed at executable_path= {browser_path}')
 if not browser_path:
 raise RuntimeError('No local Chrome/Chromium install found, and failed to install with playwright')

 # Launch browser subprocess directly
 self.logger.debug(f'[LocalBrowserWatchdog] ■ Launching browser subprocess with {len(launch_args)} args')
 self.logger.debug(
 f'[LocalBrowserWatchdog] ■ user_data_dir={profile.user_data_dir}, profile_directory={profile.profile_directory}'
)
 subprocess = await asyncio.create_subprocess_exec(
 browser_path,
 *launch_args,
 stdout=asyncio.subprocess.PIPE,
 stderr=asyncio.subprocess.PIPE,
)
 self.logger.debug(
 f'[LocalBrowserWatchdog] ■ Browser running with browser_pid= {subprocess.pid} ■ listening on CDP'
)

 # Convert to psutil.Process
 process = psutil.Process(subprocess.pid)

 # Wait for CDP to be ready and get the URL
 cdp_url = await self._wait_for_cdp_url(debug_port)

 # Success! Clean up only the temp dirs we created but didn't use
 currently_used_dir = str(profile.user_data_dir)
 unused_temp_dirs = [tmp_dir for tmp_dir in self._temp_dirs_to_cleanup if str(tmp_dir) != currently_used_dir]

```

```

 for tmp_dir in unused_temp_dirs:
 try:
 shutil.rmtree(tmp_dir, ignore_errors=True)
 except Exception:
 pass

 # Keep only the in-use directory for cleanup during browser kill
 if currently_used_dir and 'browseruse-tmp-' in currently_used_dir:
 self._temp_dirs_to_cleanup = [Path(currently_used_dir)]
 else:
 self._temp_dirs_to_cleanup = []

 return process, cdp_url

except Exception as e:
 error_str = str(e).lower()

 # Check if this is a user_data_dir related error
 if any(err in error_str for err in ['singletonlock', 'user data directory', 'cannot create', 'already exists']):
 self.logger.warning(f'Browser launch failed (attempt {attempt + 1}/{max_retries}): {e}')

 if attempt < max_retries - 1:
 # Create a temporary directory for next attempt
 tmp_dir = Path(tempfile.mkdtemp(prefix='browseruse-tmp-'))
 self._temp_dirs_to_cleanup.append(tmp_dir)

 # Update profile to use temp directory
 profile.user_data_dir = str(tmp_dir)
 self.logger.debug(f'Retrying with temporary user_data_dir: {tmp_dir}')

 # Small delay before retry
 await asyncio.sleep(0.5)
 continue

 # Not a recoverable error or last attempt failed
 # Restore original user_data_dir before raising
 if self._original_user_data_dir is not None:
 profile.user_data_dir = self._original_user_data_dir

 # Clean up any temp dirs we created
 for tmp_dir in self._temp_dirs_to_cleanup:
 try:
 shutil.rmtree(tmp_dir, ignore_errors=True)
 except Exception:
 pass

 raise

Should not reach here, but just in case
if self._original_user_data_dir is not None:
 profile.user_data_dir = self._original_user_data_dir
raise RuntimeError(f'Failed to launch browser after {max_retries} attempts')

@staticmethod
def _find_installed_browser_path() -> str | None:
 """Try to find browser executable from common fallback locations.

 Prioritizes:
 1. System Chrome Stable
 1. Playwright chromium
 2. Other system native browsers (Chromium -> Chrome Canary/Dev -> Brave)
 3. Playwright headless-shell fallback

 Returns:
 Path to browser executable or None if not found
 """
 import glob
 import platform
 from pathlib import Path

 system = platform.system()
 patterns = []

```

```

■■■# Get playwright browsers path from environment variable if set
■■■playwright_path = os.environ.get('PLAYWRIGHT_BROWSERS_PATH')

■■■if system == 'Darwin': # macOS
■■■if not playwright_path:
■■■playwright_path = '~/Library/Caches/ms-playwright'
■■■patterns = [
■■■f'/Applications/Google Chrome.app/Contents/MacOS/Google Chrome',
■■■f'{playwright_path}/chromium-*/chrome-mac/Chromium.app/Contents/MacOS/Chromium',
■■■f'/Applications/Chromium.app/Contents/MacOS/Chromium',
■■■f'/Applications/Google Chrome Canary.app/Contents/MacOS/Google Chrome Canary',
■■■f'/Applications/Brave Browser.app/Contents/MacOS/Brave Browser',
■■■f'{playwright_path}/chromium_headless_shell-*/chrome-mac/Chromium.app/Contents/MacOS/Chromium',
■■■]
■■■elif system == 'Linux':
■■■if not playwright_path:
■■■playwright_path = '~/.cache/ms-playwright'
■■■patterns = [
■■■'/usr/bin/google-chrome-stable',
■■■'/usr/bin/google-chrome',
■■■'/usr/local/bin/google-chrome',
■■■f'{playwright_path}/chromium-*/chrome-linux*/chrome',
■■■'/usr/bin/chromium',
■■■'/usr/bin/chromium-browser',
■■■'/usr/local/bin/chromium',
■■■'/snap/bin/chromium',
■■■'/usr/bin/google-chrome-beta',
■■■'/usr/bin/google-chrome-dev',
■■■'/usr/bin/brave-browser',
■■■f'{playwright_path}/chromium_headless_shell-*/chrome-linux*/chrome',
■■■]
■■■elif system == 'Windows':
■■■if not playwright_path:
■■■playwright_path = r'%LOCALAPPDATA%\ms-playwright'
■■■patterns = [
■■■r'C:\Program Files\Google\Chrome\Application\chrome.exe',
■■■r'C:\Program Files (x86)\Google\Chrome\Application\chrome.exe',
■■■r'%LOCALAPPDATA%\Google\Chrome\Application\chrome.exe',
■■■r'%PROGRAMFILES%\Google\Chrome\Application\chrome.exe',
■■■r'%PROGRAMFILES(X86)%\Google\Chrome\Application\chrome.exe',
■■■f'{playwright_path}\\chromium-*\\chrome-win\\chrome.exe',
■■■r'C:\Program Files\Chromium\Application\chrome.exe',
■■■r'C:\Program Files (x86)\Chromium\Application\chrome.exe',
■■■r'%LOCALAPPDATA%\Chromium\Application\chrome.exe',
■■■r'C:\Program Files\BraveSoftware\Brave-Browser\Application\brave.exe',
■■■r'C:\Program Files (x86)\BraveSoftware\Brave-Browser\Application\brave.exe',
■■■r'C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe',
■■■r'C:\Program Files\Microsoft\Edge\Application\msedge.exe',
■■■r'%LOCALAPPDATA%\Microsoft\Edge\Application\msedge.exe',
■■■f'{playwright_path}\\chromium_headless_shell-*\\chrome-win\\chrome.exe',
■■■]

■■■for pattern in patterns:
■■■# Expand user home directory
■■■expanded_pattern = Path(pattern).expanduser()

■■■# Handle Windows environment variables
■■■if system == 'Windows':
■■■pattern_str = str(expanded_pattern)
■■■for env_var in ['%LOCALAPPDATA%', '%PROGRAMFILES%', '%PROGRAMFILES(X86)%']:
■■■if env_var in pattern_str:
■■■env_key = env_var.strip('%').replace('(x86)', '(x86)')
■■■env_value = os.environ.get(env_key, '')
■■■if env_value:
■■■pattern_str = pattern_str.replace(env_var, env_value)
■■■expanded_pattern = Path(pattern_str)

■■■# Convert to string for glob
■■■pattern_str = str(expanded_pattern)

■■■# Check if pattern contains wildcards
■■■if '*' in pattern_str:

```

```

Use glob to expand the pattern
matches = glob.glob(pattern_str)
if matches:
Sort matches and take the last one (alphanumerically highest version)
matches.sort()
browser_path = matches[-1]
if Path(browser_path).exists() and Path(browser_path).is_file():
return browser_path
else:
Direct path check
if expanded_pattern.exists() and expanded_pattern.is_file():
return str(expanded_pattern)

return None

async def _install_browser_with_playwright(self) -> str:
"""Get browser executable path from playwright in a subprocess to avoid thread issues."""
import platform

Build command - only use --with-deps on Linux (it fails on Windows/macOS)
cmd = ['uvx', 'playwright', 'install', 'chrome']
if platform.system() == 'Linux':
cmd.append('--with-deps')

Run in subprocess with timeout
process = await asyncio.create_subprocess_exec(
*cmd,
stdout=asyncio.subprocess.PIPE,
stderr=asyncio.subprocess.PIPE,
)

try:
stdout, stderr = await asyncio.wait_for(process.communicate(), timeout=60.0)
self.logger.debug(f'[LocalBrowserWatchdog] ■ Playwright install output: {stdout}')
browser_path = self._find_installed_browser_path()
if browser_path:
return browser_path
self.logger.error(f'[LocalBrowserWatchdog] ■ Playwright local browser installation error: \n{stdout}')
raise RuntimeError('No local browser path found after: uvx playwright install chrome')
except TimeoutError:
Kill the subprocess if it times out
process.kill()
await process.wait()
raise RuntimeError('Timeout getting browser path from playwright')
except Exception as e:
Make sure subprocess is terminated
if process.returncode is None:
process.kill()
await process.wait()
raise RuntimeError(f'Error getting browser path: {e}')

@staticmethod
def _find_free_port() -> int:
"""Find a free port for the debugging interface."""
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
s.bind(('127.0.0.1', 0))
s.listen(1)
port = s.getsockname()[1]
return port

@staticmethod
async def _wait_for_cdp_url(port: int, timeout: float = 30) -> str:
"""Wait for the browser to start and return the CDP URL."""
import aiohttp

start_time = asyncio.get_event_loop().time()

while asyncio.get_event_loop().time() - start_time < timeout:
try:
async with aiohttp.ClientSession() as session:

```



```

#####async with session.get(f'http://127.0.0.1:{port}/json/version') as resp:
#####if resp.status == 200:
Chrome is ready
return f'http://127.0.0.1:{port}/'
#####else:
Chrome is starting up and returning 502/500 errors
await asyncio.sleep(0.1)
#####except Exception:
Connection error - Chrome might not be ready yet
await asyncio.sleep(0.1)

#####raise TimeoutError(f'Browser did not start within {timeout} seconds')

#####@staticmethod
#####async def _cleanup_process(process: psutil.Process) -> None:
"""Clean up browser process.

Args:
process: psutil.Process to terminate
"""
if not process:
return

try:
Try graceful shutdown first
process.terminate()

Use async wait instead of blocking wait
for _ in range(50): # Wait up to 5 seconds (50 * 0.1)
if not process.is_running():
return
await asyncio.sleep(0.1)

If still running after 5 seconds, force kill
if process.is_running():
process.kill()
Give it a moment to die
await asyncio.sleep(0.1)

except psutil.NoSuchProcess:
Process already gone
pass
except Exception:
Ignore any other errors during cleanup
pass

#####def _cleanup_temp_dir(self, temp_dir: Path | str) -> None:
"""Clean up temporary directory.

Args:
temp_dir: Path to temporary directory to remove
"""
if not temp_dir:
return

try:
temp_path = Path(temp_dir)
Only remove if it's actually a temp directory we created
if 'browseruse-tmp-' in str(temp_path):
shutil.rmtree(temp_path, ignore_errors=True)
except Exception as e:
self.logger.debug(f'Failed to cleanup temp dir {temp_dir}: {e}')

#####@property
#####def browser_pid(self) -> int | None:
"""Get the browser process ID."""
if self._subprocess:
return self._subprocess.pid
return None

#####@staticmethod
#####async def get_browser_pid_via_cdp(browser) -> int | None:

```

```

 """Get the browser process ID via CDP SystemInfo.getProcessInfo.

 Args:
 browser: Playwright Browser instance

 Returns:
 Process ID or None if failed
 """
 try:
 cdp_session = await browser.new_browser_cdp_session()
 result = await cdp_session.send('SystemInfo.getProcessInfo')
 process_info = result.get('processInfo', {})
 pid = process_info.get('id')
 await cdp_session.detach()
 return pid
 except Exception:
 # If we can't get PID via CDP, it's not critical
 return None

```

## ***.\system\browser\watchdogs\permissions\_watchdog.py***

```

"""Permissions watchdog for granting browser permissions on connection."""

from typing import TYPE_CHECKING, ClassVar

from bubus import BaseEvent

from system.browser.events import BrowserConnectedEvent
from system.browser.watchdog_base import BaseWatchdog

if TYPE_CHECKING:
 pass

class PermissionsWatchdog(BaseWatchdog):
 """Grants browser permissions when browser connects."""

 # Event contracts
 LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 BrowserConnectedEvent,
]
 EMITS: ClassVar[list[type[BaseEvent]]] = []

 async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 """Grant permissions when browser connects."""
 permissions = self.browser_session.browser_profile.permissions

 if not permissions:
 self.logger.debug('No permissions to grant')
 return

 self.logger.debug(f'Granting browser permissions: {permissions}')

 try:
 # Grant permissions using CDP Browser.grantPermissions
 # origin=None means grant to all origins
 # Browser domain commands don't use session_id
 await self.browser_session.cdp_client.send(Browser.grantPermissions(
 params={'permissions': permissions} # type: ignore
))
 self.logger.debug(f'Successfully granted permissions: {permissions}')
 except Exception as e:
 self.logger.error(f'Failed to grant permissions: {str(e)}')
 # Don't raise - permissions are not critical to browser operation

```

## **`.\system\browser\watchdogs\popups_watchdog.py`**

```
"""Watchdog for handling JavaScript dialogs (alert, confirm, prompt) automatically."""

import asyncio
from typing import ClassVar

from bubus import BaseEvent
from pydantic import PrivateAttr

from system.browser.events import TabCreatedEvent
from system.browser.watchdog_base import BaseWatchdog

class PopupsWatchdog(BaseWatchdog):
 """Handles JavaScript dialogs (alert, confirm, prompt) by automatically accepting them immediately."""

 # Events this watchdog listens to and emits
 LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [TabCreatedEvent]
 EMITS: ClassVar[list[type[BaseEvent]]] = []

 # Track which targets have dialog handlers registered
 _dialog_listeners_registered: set[str] = PrivateAttr(default_factory=set)

 def __init__(self, **kwargs):
 super().__init__(**kwargs)
 self.logger.debug(f"PopupsWatchdog initialized with browser_session={self.browser_session}, ID={id(self)}")

 async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 """Set up JavaScript dialog handling when a new tab is created."""
 target_id = event.target_id
 self.logger.debug(f"PopupsWatchdog received TabCreatedEvent for target {target_id}")

 # Skip if we've already registered for this target
 if target_id in self._dialog_listeners_registered:
 self.logger.debug(f"Already registered dialog handlers for target {target_id}")
 return

 self.logger.debug(f"Starting dialog handler setup for target {target_id}")
 try:
 # Get all CDP sessions for this target and any child frames
 cdp_session = await self.browser_session.get_or_create_cdp_session(
 target_id, focus=False
) # don't auto-focus new tabs! sometimes we need to open tabs in background

 # CRITICAL: Enable Page domain to receive dialog events
 try:
 await cdp_session.cdp_client.send.Page.enable(session_id=cdp_session.session_id)
 self.logger.debug(f"Enabled Page domain for session {cdp_session.session_id[-8:]}")
 except Exception as e:
 self.logger.debug(f"Failed to enable Page domain: {e}")

 # Also register for the root CDP client to catch dialogs from any frame
 if self.browser_session._cdp_client_root:
 self.logger.debug("Also registering handler on root CDP client")
 try:
 # Enable Page domain on root client too
 await self.browser_session._cdp_client_root.send.Page.enable()
 self.logger.debug("Enabled Page domain on root CDP client")
 except Exception as e:
 self.logger.debug(f"Failed to enable Page domain on root: {e}")

 # Set up async handler for JavaScript dialogs - accept immediately without event dispatch
 async def handle_dialog(event_data, session_id: str | None = None):
 """Handle JavaScript dialog events - accept immediately."""
 try:
 dialog_type = event_data.get('type', 'alert')
 message = event_data.get('message', '')

 # Store the popup message in browser session for inclusion in browser state
```

```

#####if message:
#####formatted_message = f'[{dialog_type}] {message}'
#####self.browser_session._closed_popup_messages.append(formatted_message)
#####self.logger.debug(f'■ Stored popup message: {formatted_message[:100]}')

Choose action based on dialog type:
- alert: accept=true (click OK to dismiss)
- confirm: accept=true (click OK to proceed - safer for automation)
- prompt: accept=false (click Cancel since we can't provide input)
- beforeunload: accept=true (allow navigation)
#####should_accept = dialog_type in ('alert', 'confirm', 'beforeunload')

#####action_str = 'accepting (OK)' if should_accept else 'dismissing (Cancel)'
#####self.logger.info(f"■ JavaScript {dialog_type} dialog: '{message[:100]}' - {action_str}...")

#####dismissed = False

Approach 1: Use the session that detected the dialog (most reliable)
#####if self.browser_session._cdp_client_root and session_id:
#####try:
#####self.logger.debug(f'■ Approach 1: Using detecting session {session_id[-8:]}')
#####await asyncio.wait_for(
#####self.browser_session._cdp_client_root.send.Page.handleJavaScriptDialog(
#####params={'accept': should_accept},
#####session_id=session_id,
#####),
#####timeout=0.5,
#####)
#####dismissed = True
#####self.logger.info('■ Dialog handled successfully via detecting session')
#####except (TimeoutError, Exception) as e:
#####self.logger.debug(f'Approach 1 failed: {type(e).__name__}')

Approach 2: Try with current agent focus session
#####if not dismissed and self.browser_session._cdp_client_root and self.browser_session.agent_focus_target_id:
#####try:
Use public API with focus=False to avoid changing focus during popup dismissal
#####cdp_session = await self.browser_session.get_or_create_cdp_session(
#####self.browser_session.agent_focus_target_id, focus=False
#####)
#####self.logger.debug(f'■ Approach 2: Using agent focus session {cdp_session.session_id[-8:]}')
#####await asyncio.wait_for(
#####self.browser_session._cdp_client_root.send.Page.handleJavaScriptDialog(
#####params={'accept': should_accept},
#####session_id=cdp_session.session_id,
#####),
#####timeout=0.5,
#####)
#####dismissed = True
#####self.logger.info('■ Dialog handled successfully via agent focus session')
#####except (TimeoutError, Exception) as e:
#####self.logger.debug(f'Approach 2 failed: {type(e).__name__}')

#####except Exception as e:
#####self.logger.error(f'■ Critical error in dialog handler: {type(e).__name__}: {e}')

Register handler on the specific session
#####cdp_session.cdp_client.register.Page.javascriptDialogOpening(handle_dialog) # type: ignore[arg-type]
#####self.logger.debug(
#####f'Successfully registered Page.javascriptDialogOpening handler for session {cdp_session.session_id}'
#####)

Also register on root CDP client to catch dialogs from any frame
#####if hasattr(self.browser_session._cdp_client_root, 'register'):
#####try:
#####self.browser_session._cdp_client_root.register.Page.javascriptDialogOpening(handle_dialog) # type: ignore[arg-type]
#####self.logger.debug('Successfully registered dialog handler on root CDP client for all frames')
#####except Exception as root_error:
#####self.logger.warning(f'Failed to register on root CDP client: {root_error}')

Mark this target as having dialog handling set up
#####self._dialog_listeners_registered.add(target_id)

```

```

■■■self.logger.debug(f'Set up JavaScript dialog handling for tab {target_id}')

■■except Exception as e:
■■■self.logger.warning(f'Failed to set up popup handling for tab {target_id}: {e}')

```

## **`.system\browser\watchdogs\recording_watchdog.py`**

```

"""Recording Watchdog for Browser Use Sessions."""

import asyncio
from pathlib import Path
from typing import Any, ClassVar

from bubus import BaseEvent
from cdp_use.cdp.page.events import ScreenshotFrameEvent
from pydantic import PrivateAttr
from uuid_extensions import uuid7str

from system.browser.events import AgentFocusChangedEvent, BrowserConnectedEvent, BrowserStopEvent
from system.browser.profile import ViewportSize
from system.browser.video_recorder import VideoRecorderService
from system.browser.watchdog_base import BaseWatchdog
from system.utils import create_task_with_error_handling

class RecordingWatchdog(BaseWatchdog):
 """
 ■■■Manages video recording of a browser session using CDP screencasting.
 """

 ■■■LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [BrowserConnectedEvent, BrowserStopEvent, AgentFocusChangedEvent]
 ■■■EMITS: ClassVar[list[type[BaseEvent]]] = []

 ■■■_recorder: VideoRecorderService | None = PrivateAttr(default=None)
 ■■■_current_session_id: str | None = PrivateAttr(default=None)
 ■■■_screenshot_params: dict[str, Any] | None = PrivateAttr(default=None)

 ■■■async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 """
 ■■■Starts video recording if it is configured in the browser profile.
 """
 ■■■profile = self.browser_session.browser_profile
 ■■■if not profile.record_video_dir:
 ■■■return

 ■■■# Dynamically determine video size
 ■■■size = profile.record_video_size
 ■■■if not size:
 ■■■self.logger.debug('record_video_size not specified, detecting viewport size...')
 ■■■size = await self._get_current_viewport_size()

 ■■■if not size:
 ■■■self.logger.warning('Cannot start video recording: viewport size could not be determined.')
 ■■■return

 ■■■video_format = getattr(profile, 'record_video_format', 'mp4').strip('.')
 ■■■output_path = Path(profile.record_video_dir) / f'{uuid7str()}.{video_format}'

 ■■■self.logger.debug(f'Initializing video recorder for format: {video_format}')
 ■■■self._recorder = VideoRecorderService(output_path=output_path, size=size, framerate=profile.record_video_framerate)
 ■■■self._recorder.start()

 ■■■if not self._recorder._is_active:
 ■■■self._recorder = None
 ■■■return

 ■■■self.browser_session.cdp_client.register.Page.screenshotFrame(self.on_screenshotFrame)
 ■■■self._screenshot_params = {

```

```

 'format': 'png',
 'quality': 90,
 'maxWidth': size['width'],
 'maxHeight': size['height'],
 'everyNthFrame': 1,
 }

 await self._start_screencast()

 async def on_AgentFocusChangedEvent(self, event: AgentFocusChangedEvent) -> None:
 """
 Switches video recording to the new tab.
 """
 if self._recorder:
 self.logger.debug(f'Agent focus changed to {event.target_id}, switching screencast...')
 await self._start_screencast()

 async def _start_screencast(self) -> None:
 """Starts screencast on the currently focused tab."""
 if not self._recorder or not self._screencast_params:
 return

 try:
 # Get the current session (for the focused target)
 cdp_session = await self.browser_session.get_or_create_cdp_session()

 # If we are already recording this session, do nothing
 if self._current_session_id == cdp_session.session_id:
 return

 # Stop recording on the previous session
 if self._current_session_id:
 try:
 # Use the root client to stop screencast on the specific session
 await self.browser_session.cdp_client.send.Page.stopScreencast(session_id=self._current_session_id)
 except Exception as e:
 # It's possible the session is already closed
 self.logger.debug(f'Failed to stop screencast on old session {self._current_session_id}: {e}')

 self._current_session_id = cdp_session.session_id

 # Start recording on the new session
 await cdp_session.cdp_client.send.Page.startScreencast(
 params=self._screencast_params, # type: ignore
 session_id=cdp_session.session_id,
)
 self.logger.info(f'Started/Switched video recording to target {cdp_session.target_id}')
 except Exception as e:
 self.logger.error(f'Failed to switch screencast via CDP: {e}')
 # If we fail to start on the new tab, we reset current session id
 self._current_session_id = None

 async def _get_current_viewport_size(self) -> ViewportSize | None:
 """Gets the current viewport size directly from the browser via CDP."""
 try:
 cdp_session = await self.browser_session.get_or_create_cdp_session()
 metrics = await cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=cdp_session.session_id)

 # Use cssVisualViewport for the most accurate representation of the visible area
 viewport = metrics.get('cssVisualViewport', {})
 width = viewport.get('clientWidth')
 height = viewport.get('clientHeight')

 if width and height:
 self.logger.debug(f'Detected viewport size: {width}x{height}')
 return ViewportSize(width=int(width), height=int(height))
 except Exception as e:
 self.logger.warning(f'Failed to get viewport size from browser: {e}')

 return None

 def on_screencastFrame(self, event: ScreencastFrameEvent, session_id: str | None) -> None:

```

```

"""
Synchronous handler for incoming screencast frames.
"""
Only process frames from the current session we intend to record
This handles race conditions where old session might still send frames before stop completes
if self._current_session_id and session_id != self._current_session_id:
 return

if not self._recorder:
 return
self._recorder.add_frame(event['data'])
create_task_with_error_handling(
 self._ack_screencast_frame(event, session_id),
 name='ack_screencast_frame',
 logger_instance=self.logger,
 suppress_exceptions=True,
)

async def _ack_screencast_frame(self, event: ScreencastFrameEvent, session_id: str | None) -> None:
 """
 Asynchronously acknowledges a screencast frame.
 """
 try:
 await self.browser_session.cdp_client.send.Page.screencastFrameAck(
 params={'sessionId': event['sessionId']}, session_id=session_id
)
 except Exception as e:
 self.logger.debug(f'Failed to acknowledge screencast frame: {e}')

async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 """
 Stops the video recording and finalizes the video file.
 """
 if self._recorder:
 recorder = self._recorder
 self._recorder = None
 self._current_session_id = None
 self._screencast_params = None

 self.logger.debug('Stopping video recording and saving file...')
 loop = asyncio.get_event_loop()
 await loop.run_in_executor(None, recorder.stop_and_save)

```

## **`.system\browser\watchdogs\screenshot_watchdog.py`**

```

"""Screenshot watchdog for handling screenshot requests using CDP."""

from typing import TYPE_CHECKING, Any, ClassVar

from bubus import BaseEvent
from cdp_use.cdp.page import CaptureScreenshotParameters

from system.browser.events import ScreenshotEvent
from system.browser.views import BrowserError
from system.browser.watchdog_base import BaseWatchdog
from system.observability import observe_debug

if TYPE_CHECKING:
 pass

class ScreenshotWatchdog(BaseWatchdog):
 """Handles screenshot requests using CDP."""

 # Events this watchdog listens to
 LISTENS_TO: ClassVar[list[type[BaseEvent[Any]]]] = [ScreenshotEvent]

 # Events this watchdog emits

```

```

■EMITS: ClassVar[list[type[BaseEvent[Any]]]] = []

■@observe_debug(ignore_input=True, ignore_output=True, name='screenshot_event_handler')
■async def on_ScreenshotEvent(self, event: ScreenshotEvent) -> str:
■ """Handle screenshot request using CDP.

■ Args:
■ event: ScreenshotEvent with optional full_page and clip parameters

■ Returns:
■ Dict with 'screenshot' key containing base64-encoded screenshot or None
■ """
■ self.logger.debug('[ScreenshotWatchdog] Handler START - on_ScreenshotEvent called')
■ try:
■ # Validate focused target is a top-level page (not iframe/worker)
■ # CDP Page.captureScreenshot only works on page/tab targets
■ focused_target = self.browser_session.get_focused_target()

■ if focused_target and focused_target.target_type in ('page', 'tab'):
■ target_id = focused_target.target_id
■ else:
■ # Focused target is iframe/worker/missing - fall back to any page target
■ target_type_str = focused_target.target_type if focused_target else 'None'
■ self.logger.warning(f'[ScreenshotWatchdog] Focused target is {target_type_str}, falling back to page targets')
■ page_targets = self.browser_session.get_page_targets()
■ if not page_targets:
■ raise BrowserError('[ScreenshotWatchdog] No page targets available for screenshot')
■ target_id = page_targets[-1].target_id

■ cdp_session = await self.browser_session.get_or_create_cdp_session(target_id, focus=True)

■ # Prepare screenshot parameters
■ params = CaptureScreenshotParameters(format='png', captureBeyondViewport=False)

■ # Take screenshot using CDP
■ self.logger.debug(f'[ScreenshotWatchdog] Taking screenshot with params: {params}')
■ result = await cdp_session.cdp_client.send.Page.captureScreenshot(params=params, session_id=cdp_session.session_id)

■ # Return base64-encoded screenshot data
■ if result and 'data' in result:
■ self.logger.debug('[ScreenshotWatchdog] Screenshot captured successfully')
■ return result['data']

■ raise BrowserError('[ScreenshotWatchdog] Screenshot result missing data')
■ except Exception as e:
■ self.logger.error(f'[ScreenshotWatchdog] Screenshot failed: {e}')
■ raise
■ finally:
■ # Try to remove highlights even on failure
■ try:
■ await self.browser_session.remove_highlights()
■ except Exception:
■ pass

```

## ***.\system\browser\watchdogs\security\_watchdog.py***

```

"""Security watchdog for enforcing URL access policies."""

from typing import TYPE_CHECKING, ClassVar

from bubus import BaseEvent

from system.browser.events import (
 ■BrowserErrorEvent,
 ■NavigateToUrlEvent,
 ■NavigationCompleteEvent,
 ■TabCreatedEvent,
)

```



```

from system.browser.watchdog_base import BaseWatchdog

if TYPE_CHECKING:
 pass

Track if we've shown the glob warning
_GLOB_WARNING_SHOWN = False

class SecurityWatchdog(BaseWatchdog):
 """Monitors and enforces security policies for URL access."""

 ## Event contracts
 LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 NavigateToUrlEvent,
 NavigationCompleteEvent,
 TabCreatedEvent,
]
 EMITS: ClassVar[list[type[BaseEvent]]] = [
 BrowserErrorEvent,
]

 async def on_NavigateToUrlEvent(self, event: NavigateToUrlEvent) -> None:
 """Check if navigation URL is allowed before navigation starts."""
 ## Security check BEFORE navigation
 if not self._is_url_allowed(event.url):
 self.logger.warning(f"Blocking navigation to disallowed URL: {event.url}")
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='NavigationBlocked',
 message=f'Navigation blocked to disallowed URL: {event.url}',
 details={'url': event.url, 'reason': 'not_in_allowed_domains'},
)
)
 ## Stop event propagation by raising exception
 raise ValueError(f'Navigation to {event.url} blocked by security policy')

 async def on_NavigationCompleteEvent(self, event: NavigationCompleteEvent) -> None:
 """Check if navigated URL is allowed (catches redirects to blocked domains)."""
 ## Check if the navigated URL is allowed (in case of redirects)
 if not self._is_url_allowed(event.url):
 self.logger.warning(f"Navigation to non-allowed URL detected: {event.url}")

 ## Dispatch browser error
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='NavigationBlocked',
 message=f'Navigation blocked to non-allowed URL: {event.url} - redirecting to about:blank',
 details={'url': event.url, 'target_id': event.target_id},
)
)
 ## Navigate to about:blank to keep session alive
 ## Agent will see the error and can continue with other tasks
 try:
 session = await self.browser_session.get_or_create_cdp_session(target_id=event.target_id)
 await session.cdp_client.send.Page.navigate(params={'url': 'about:blank'}, session_id=session.session_id)
 self.logger.info(f"Navigated to about:blank after blocked URL: {event.url}")
 except Exception as e:
 pass
 self.logger.error(f"Failed to navigate to about:blank: {type(e).__name__} {e}")

 async def on_TabCreatedEvent(self, event: TabCreatedEvent) -> None:
 """Check if new tab URL is allowed."""
 if not self._is_url_allowed(event.url):
 self.logger.warning(f"New tab created with disallowed URL: {event.url}")

 ## Dispatch error and try to close the tab
 self.event_bus.dispatch(
 BrowserErrorEvent(
 error_type='TabCreationBlocked',
 message=f'Tab created with non-allowed URL: {event.url}',
 details={'url': event.url, 'target_id': event.target_id},
)
)

```

```

)
)

Try to close the offending tab
try:
 await self.browser_session._cdp_close_page(event.target_id)
 self.logger.info(f'Closed new tab with non-allowed URL: {event.url}')
except Exception as e:
 self.logger.error(f'Failed to close new tab with non-allowed URL: {type(e).__name__} {e}')

def _is_root_domain(self, domain: str) -> bool:
 """Check if a domain is a root domain (no subdomain present).

 Simple heuristic: only add www for domains with exactly 1 dot (domain.tld).
 For complex cases like country TLDs or subdomains, users should configure explicitly.

 Args:
 domain: The domain to check

 Returns:
 True if it's a simple root domain, False otherwise
 """
 # Skip if it contains wildcards or protocol
 if '*' in domain or '://' in domain:
 return False

 return domain.count('.') == 1

def _log_glob_warning(self) -> None:
 """Log a warning about glob patterns in allowed_domains."""
 global _GLOB_WARNING_SHOWN
 if not _GLOB_WARNING_SHOWN:
 _GLOB_WARNING_SHOWN = True
 self.logger.warning(
 'Using glob patterns in allowed_domains. '
 'Note: Patterns like "*.example.com" will match both subdomains AND the main domain.'
)

def _get_domain_variants(self, host: str) -> tuple[str, str]:
 """Get both variants of a domain (with and without www prefix).

 Args:
 host: The hostname to process

 Returns:
 Tuple of (original_host, variant_host)
 - If host starts with www., variant is without www.
 - Otherwise, variant is with www. prefix
 """
 if host.startswith('www.'):
 return (host, host[4:]) # ('www.example.com', 'example.com')
 else:
 return (host, f'www.{host}') # ('example.com', 'www.example.com')

def _is_ip_address(self, host: str) -> bool:
 """Check if a hostname is an IP address (IPv4 or IPv6).

 Args:
 host: The hostname to check

 Returns:
 True if the host is an IP address, False otherwise
 """
 import ipaddress

 try:
 # Try to parse as IP address (handles both IPv4 and IPv6)
 ipaddress.ip_address(host)
 return True
 except ValueError:
 return False
 except Exception:

```

```

 return False

def _is_url_allowed(self, url: str) -> bool:
 """Check if a URL is allowed based on the allowed_domains configuration.

 Args:
 url: The URL to check

 Returns:
 True if the URL is allowed, False otherwise
 """

 ## Always allow internal browser targets (before any other checks)
 if url in ['about:blank', 'chrome://new-tab-page/', 'chrome://new-tab-page', 'chrome://newtab/']:
 return True

 ## Parse the URL to extract components
 from urllib.parse import urlparse

 try:
 parsed = urlparse(url)
 except Exception:
 ## Invalid URL
 return False

 ## Allow data: and blob: URLs (they don't have hostnames)
 if parsed.scheme in ['data', 'blob']:
 return True

 ## Get the actual host (domain)
 host = parsed.hostname
 if not host:
 return False

 ## Check if IP addresses should be blocked (before domain checks)
 if self.browser_session.browser_profile.block_ip_addresses:
 if self._is_ip_address(host):
 return False

 ## If no allowed_domains specified, allow all URLs
 if (
 not self.browser_session.browser_profile.allowed_domains
 and not self.browser_session.browser_profile.prohibited_domains
):
 return True

 ## Check allowed domains (fast path for sets, slow path for lists with patterns)
 if self.browser_session.browser_profile.allowed_domains:
 allowed_domains = self.browser_session.browser_profile.allowed_domains

 if isinstance(allowed_domains, set):
 ## Fast path: O(1) exact hostname match - check both www and non-www variants
 host_variant, host_alt = self._get_domain_variants(host)
 return host_variant in allowed_domains or host_alt in allowed_domains
 else:
 ## Slow path: O(n) pattern matching for lists
 for pattern in allowed_domains:
 if self._is_url_match(url, host, parsed.scheme, pattern):
 return True
 return False

 ## Check prohibited domains (fast path for sets, slow path for lists with patterns)
 if self.browser_session.browser_profile.prohibited_domains:
 prohibited_domains = self.browser_session.browser_profile.prohibited_domains

 if isinstance(prohibited_domains, set):
 ## Fast path: O(1) exact hostname match - check both www and non-www variants
 host_variant, host_alt = self._get_domain_variants(host)
 return host_variant not in prohibited_domains and host_alt not in prohibited_domains
 else:
 ## Slow path: O(n) pattern matching for lists
 for pattern in prohibited_domains:

```

```

 #####if self._is_url_match(url, host, parsed.scheme, pattern):
 #####return False
 #####return True

 #####return True

 def _is_url_match(self, url: str, host: str, scheme: str, pattern: str) -> bool:
 #####"""Check if a URL matches a pattern."""

 ###### Full URL for matching (scheme + host)
 #####full_url_pattern = f'{scheme}://{host}'

 ###### Handle glob patterns
 #####if '*' in pattern:
 #####self._log_glob_warning()
 #####import fnmatch

 ###### Check if pattern matches the host
 #####if pattern.startswith('*'):
 ##### ###### Pattern like *.example.com should match subdomains and main domain
 ##### domain_part = pattern[2:] # Remove *.
 ##### if host == domain_part or host.endswith('.' + domain_part):
 ##### ###### Only match http/https URLs for domain-only patterns
 ##### if scheme in ['http', 'https']:
 ##### #####return True
 ##### elif pattern.endswith('/*'):
 ##### ###### Pattern like brave://* or http*://example.com/*
 ##### if fnmatch.fnmatch(url, pattern):
 ##### #####return True
 ##### else:
 ##### ###### Use fnmatch for other glob patterns
 ##### if fnmatch.fnmatch(
 ##### full_url_pattern if '://' in pattern else host,
 ##### pattern,
 #####):
 ##### #####return True
 ##### else:
 ##### ###### Exact match
 ##### if '://' in pattern:
 ##### ###### Full URL pattern
 ##### if url.startswith(pattern):
 ##### #####return True
 ##### else:
 ##### ###### Domain-only pattern (case-insensitive comparison)
 ##### if host.lower() == pattern.lower():
 ##### #####return True
 ##### ###### If pattern is a root domain, also check www subdomain
 ##### if self._is_root_domain(pattern) and host.lower() == f'www.{pattern.lower()}':
 ##### #####return True

 #####return False

```

## **.lsystem\browser\watchdogs\storage\_state\_watchdog.py**

```

 """Storage state watchdog for managing browser cookies and storage persistence."""

 import asyncio
 import json
 import os
 from pathlib import Path
 from typing import Any, ClassVar

 from bubus import BaseEvent
 from cdp_use.cdp.network import Cookie
 from pydantic import Field, PrivateAttr

 from system.browser.events import (
 #####BrowserConnectedEvent,

```

```

 ■BrowserStopEvent,
 ■LoadStorageStateEvent,
 ■SaveStorageStateEvent,
 ■StorageStateLoadedEvent,
 ■StorageStateSavedEvent,
)
from system.browser.watchdog_base import BaseWatchdog
from system.utils import create_task_with_error_handling

class StorageStateWatchdog(BaseWatchdog):
 ■"""Monitors and persists browser storage state including cookies and localStorage."""

 ■# Event contracts
 ■LISTENS_TO: ClassVar[list[type[BaseEvent]]] = [
 ■■BrowserConnectedEvent,
 ■■BrowserStopEvent,
 ■■SaveStorageStateEvent,
 ■■LoadStorageStateEvent,
 ■]
 ■EMITS: ClassVar[list[type[BaseEvent]]] = [
 ■■StorageStateSavedEvent,
 ■■StorageStateLoadedEvent,
 ■]

 ■# Configuration
 ■auto_save_interval: float = Field(default=30.0) # Auto-save every 30 seconds
 ■save_on_change: bool = Field(default=True) # Save immediately when cookies change

 ■# Private state
 ■_monitoring_task: asyncio.Task | None = PrivateAttr(default=None)
 ■_last_cookie_state: list[dict] = PrivateAttr(default_factory=list)
 ■_save_lock: asyncio.Lock = PrivateAttr(default_factory=asyncio.Lock)

 ■async def on_BrowserConnectedEvent(self, event: BrowserConnectedEvent) -> None:
 ■■"""Start monitoring when browser starts."""
 ■■self.logger.debug('[StorageStateWatchdog] ■ Initializing auth/cookies sync <-> with storage_state.json')

 ■■# Start monitoring
 ■■await self._start_monitoring()

 ■■# Automatically load storage state after browser start
 ■■await self.event_bus.dispatch(LoadStorageStateEvent())

 ■async def on_BrowserStopEvent(self, event: BrowserStopEvent) -> None:
 ■■"""Stop monitoring when browser stops."""
 ■■self.logger.debug('[StorageStateWatchdog] Stopping storage_state monitoring')
 ■■await self._stop_monitoring()

 ■async def on_SaveStorageStateEvent(self, event: SaveStorageStateEvent) -> None:
 ■■"""Handle storage state save request."""
 ■■# Use provided path or fall back to profile default
 ■■path = event.path
 ■■if path is None:
 ■■# Use profile default path if available
 ■■if self.browser_session.browser_profile.storage_state:
 ■■path = str(self.browser_session.browser_profile.storage_state)
 ■■else:
 ■■path = None # Skip saving if no path available
 ■■await self._save_storage_state(path)

 ■async def on_LoadStorageStateEvent(self, event: LoadStorageStateEvent) -> None:
 ■■"""Handle storage state load request."""
 ■■# Use provided path or fall back to profile default
 ■■path = event.path
 ■■if path is None:
 ■■# Use profile default path if available
 ■■if self.browser_session.browser_profile.storage_state:
 ■■path = str(self.browser_session.browser_profile.storage_state)
 ■■else:
 ■■path = None # Skip loading if no path available
 ■■await self._load_storage_state(path)

```

```

 async def _start_monitoring(self) -> None:
 """Start the monitoring task."""
 if self._monitoring_task and not self._monitoring_task.done():
 return

 assert self.browser_session.cdp_client is not None

 self._monitoring_task = create_task_with_error_handling(
 self._monitor_storage_changes(), name='monitor_storage_changes', logger_instance=self.logger, suppress=True
)
 # self.logger'[StorageStateWatchdog] Started storage monitoring task')

 async def _stop_monitoring(self) -> None:
 """Stop the monitoring task."""
 if self._monitoring_task and not self._monitoring_task.done():
 self._monitoring_task.cancel()
 try:
 await self._monitoring_task
 except asyncio.CancelledError:
 pass
 # self.logger.debug('[StorageStateWatchdog] Stopped storage monitoring task')

 async def _check_for_cookie_changes_cdp(self, event: dict) -> None:
 """Check if a CDP network event indicates cookie changes.

 This would be called by Network.responseReceivedExtraInfo events
 if we set up CDP event listeners.
 """
 try:
 # Check for Set-Cookie headers in the response
 headers = event.get('headers', {})
 if 'set-cookie' in headers or 'Set-Cookie' in headers:
 self.logger.debug('[StorageStateWatchdog] Cookie change detected via CDP')

 # If save on change is enabled, trigger save immediately
 if self.save_on_change:
 await self._save_storage_state()
 except Exception as e:
 self.logger.warning(f'[StorageStateWatchdog] Error checking for cookie changes: {e}')

 async def _monitor_storage_changes(self) -> None:
 """Periodically check for storage changes and auto-save."""
 while True:
 try:
 await asyncio.sleep(self.auto_save_interval)

 # Check if cookies have changed
 if await self._have_cookies_changed():
 self.logger.debug('[StorageStateWatchdog] Detected changes to sync with storage_state.json')
 await self._save_storage_state()

 except asyncio.CancelledError:
 break
 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Error in monitoring loop: {e}')

 async def _have_cookies_changed(self) -> bool:
 """Check if cookies have changed since last save."""
 if not self.browser_session.cdp_client:
 return False

 try:
 # Get current cookies using CDP
 current_cookies = await self.browser_session._cdp_get_cookies()

 # Convert to comparable format, using .get() for optional fields
 current_cookie_set = {
 (c.get('name', ''), c.get('domain', ''), c.get('path', '')): c.get('value', '') for c in current_cookies
 }

 last_cookie_set = {
 (c.get('name', ''), c.get('domain', ''), c.get('path', '')): c.get('value', '') for c in self._last_cookies
 }

 return current_cookie_set != last_cookie_set
 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Error getting cookies: {e}')
 return False

```

```

 }

 return current_cookie_set != last_cookie_set
except Exception as e:
 self.logger.debug(f'[StorageStateWatchdog] Error comparing cookies: {e}')
 return False

 async def _save_storage_state(self, path: str | None = None) -> None:
 """Save browser storage state to file."""
 async with self._save_lock:
 # Check if CDP client is available
 assert await self.browser_session.get_or_create_cdp_session(target_id=None)

 save_path = path or self.browser_session.browser_profile.storage_state
 if not save_path:
 return

 # Skip saving if the storage state is already a dict (indicates it was loaded from memory)
 # We only save to file if it started as a file path
 if isinstance(save_path, dict):
 self.logger.debug('[StorageStateWatchdog] Storage state is already a dict, skipping file save')
 return

 try:
 # Get current storage state using CDP
 storage_state = await self.browser_session._cdp_get_storage_state()

 # Update our last known state
 self._last_cookie_state = storage_state.get('cookies', []).copy()

 # Convert path to Path object
 json_path = Path(save_path).expanduser().resolve()
 json_path.parent.mkdir(parents=True, exist_ok=True)

 # Merge with existing state if file exists
 merged_state = storage_state
 if json_path.exists():
 try:
 existing_state = json.loads(json_path.read_text())
 merged_state = self._merge_storage_states(existing_state, dict(storage_state))
 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Failed to merge with existing state: {e}')

 # Write atomically
 temp_path = json_path.with_suffix('.json.tmp')
 temp_path.write_text(json.dumps(merged_state, indent=4))

 # Backup existing file
 if json_path.exists():
 backup_path = json_path.with_suffix('.json.bak')
 json_path.replace(backup_path)

 # Move temp to final
 temp_path.replace(json_path)

 # Emit success event
 self.event_bus.dispatch(
 StorageStateSavedEvent(
 path=str(json_path),
 cookies_count=len(merged_state.get('cookies', [])),
 origins_count=len(merged_state.get('origins', [])),
)
)

 self.logger.debug(
 f'[StorageStateWatchdog] Saved storage state to {json_path} '
 f'({len(merged_state.get("cookies", []))} cookies, '
 f'{len(merged_state.get("origins", []))} origins)'
)

 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Failed to save storage state: {e}')

```

```

 async def _load_storage_state(self, path: str | None = None) -> None:
 """Load browser storage state from file."""
 if not self.browser_session.cdp_client:
 self.logger.warning('[StorageStateWatchdog] No CDP client available for loading')
 return

 load_path = path or self.browser_session.browser_profile.storage_state
 if not load_path or not os.path.exists(str(load_path)):
 return

 try:
 # Read the storage state file asynchronously
 import anyio

 content = await anyio.Path(str(load_path)).read_text()
 storage = json.loads(content)

 # Apply cookies if present
 if 'cookies' in storage and storage['cookies']:
 # Playwright exports session cookies with expires=0/-1. CDP treats expires=0 as expired.
 # Normalize session cookies by omitting expires
 normalized_cookies: list[Cookie] = []
 for cookie in storage['cookies']:
 if not isinstance(cookie, dict):
 normalized_cookies.append(cookie) # type: ignore[arg-type]
 continue
 c = dict(cookie)
 expires = c.get('expires')
 if expires in (0, 0.0, -1, -1.0):
 c.pop('expires', None)
 normalized_cookies.append(Cookie(**c))

 await self.browser_session._cdp_set_cookies(normalized_cookies)
 self._last_cookie_state = storage['cookies'].copy()
 self.logger.debug(f'[StorageStateWatchdog] Added {len(storage["cookies"])} cookies from storage state')

 # Apply origins (localStorage/sessionStorage) if present
 if 'origins' in storage and storage['origins']:
 for origin in storage['origins']:
 origin_value = origin.get('origin')
 if not origin_value:
 continue

 # Scope storage restoration to its origin to avoid cross-site pollution.
 if origin.get('localStorage'):
 lines = []
 for item in origin['localStorage']:
 lines.append(f>window.localStorage.setItem({json.dumps(item["name"])}, {json.dumps(item["value"])});')
 script = (
 '(function(){\n'
 f' if (window.location && window.location.origin !== {json.dumps(origin_value)}) return;\n'
 ' try {\n'
 f' {" ".join(lines)}\n'
 ' } catch (e) {{\n'
 ' }}();\n'
 ')')
 await self.browser_session._cdp_add_init_script(script)

 if origin.get('sessionStorage'):
 lines = []
 for item in origin['sessionStorage']:
 lines.append(
 f>window.sessionStorage.setItem({json.dumps(item["name"])}, {json.dumps(item["value"])});')
 script = (
 '(function(){\n'
 f' if (window.location && window.location.origin !== {json.dumps(origin_value)}) return;\n'
 ' try {\n'
 f' {" ".join(lines)}\n'
 ' } catch (e) {{\n'
 ' }}();\n'
 ')')

```



```

#####await self.browser_session._cdp_add_init_script(script)
#####self.logger.debug(
#####f'[StorageStateWatchdog] Applied localStorage/sessionStorage from {len(storage["origins"])} origins'
#####)

###self.event_bus.dispatch(
#####StorageStateLoadedEvent(
#####path=str(load_path),
#####cookies_count=len(storage.get('cookies', [])),
#####origins_count=len(storage.get('origins', [])),
#####)
###)

###self.logger.debug(f'[StorageStateWatchdog] Loaded storage state from: {load_path}')

###except Exception as e:
###self.logger.error(f'[StorageStateWatchdog] Failed to load storage state: {e}')

@staticmethod
def _merge_storage_states(existing: dict[str, Any], new: dict[str, Any]) -> dict[str, Any]:
 """Merge two storage states, with new values taking precedence."""
 merged = existing.copy()

 # Merge cookies
 existing_cookies = {(c['name'], c['domain'], c['path']): c for c in existing.get('cookies', [])}

 for cookie in new.get('cookies', []):
 key = (cookie['name'], cookie['domain'], cookie['path'])
 existing_cookies[key] = cookie

 merged['cookies'] = list(existing_cookies.values())

 # Merge origins
 existing_origins = {origin['origin']: origin for origin in existing.get('origins', [])}

 for origin in new.get('origins', []):
 existing_origins[origin['origin']] = origin

 merged['origins'] = list(existing_origins.values())

 return merged

async def get_current_cookies(self) -> list[dict[str, Any]]:
 """Get current cookies using CDP."""
 if not self.browser_session.cdp_client:
 return []

 try:
 cookies = await self.browser_session._cdp_get_cookies()
 # Cookie is a TypedDict, cast to dict for compatibility
 return [dict(cookie) for cookie in cookies]
 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Failed to get cookies: {e}')
 return []

async def add_cookies(self, cookies: list[dict[str, Any]]) -> None:
 """Add cookies using CDP."""
 if not self.browser_session.cdp_client:
 self.logger.warning('[StorageStateWatchdog] No CDP client available for adding cookies')
 return

 try:
 # Convert dicts to Cookie objects
 cookie_objects = [Cookie(**cookie_dict) if isinstance(cookie_dict, dict) else cookie_dict for cookie_dict in cookies]
 # Set cookies using CDP
 await self.browser_session._cdp_set_cookies(cookie_objects)
 self.logger.debug(f'[StorageStateWatchdog] Added {len(cookie_objects)} cookies')
 except Exception as e:
 self.logger.error(f'[StorageStateWatchdog] Failed to add cookies: {e}')

```

**`.system\browser\watchdogs\__init__.py`**

**`.system\dom\enhanced_snapshot.py`**

```
"""
Enhanced snapshot processing for browser-use DOM tree extraction.

This module provides stateless functions for parsing Chrome DevTools Protocol (CDP) DOMSnapshot data
to extract visibility, clickability, cursor styles, and other layout information.
"""

from cdp_use.cdp.domsnapshot.commands import CaptureSnapshotReturns
from cdp_use.cdp.domsnapshot.types import (
 LayoutTreeSnapshot,
 NodeTreeSnapshot,
 RareBooleanData,
)

from system.dom.views import DOMRect, EnhancedSnapshotNode

Only the ESSENTIAL computed styles for interactivity and visibility detection
REQUIRED_COMPUTED_STYLES = [
 # Only styles actually accessed in the codebase (prevents Chrome crashes on heavy sites)
 'display', # Used in service.py visibility detection
 'visibility', # Used in service.py visibility detection
 'opacity', # Used in service.py visibility detection
 'overflow', # Used in views.py scrollability detection
 'overflow-x', # Used in views.py scrollability detection
 'overflow-y', # Used in views.py scrollability detection
 'cursor', # Used in enhanced_snapshot.py cursor extraction
 'pointer-events', # Used for clickability logic
 'position', # Used for visibility logic
 'background-color', # Used for visibility logic
]

def _parse_rare_boolean_data(rare_data: RareBooleanData, index: int) -> bool | None:
 """Parse rare boolean data from snapshot - returns True if index is in the rare data."""
 return index in rare_data['index']

def _parse_computed_styles(strings: list[str], style_indices: list[int]) -> dict[str, str]:
 """Parse computed styles from layout tree using string indices."""
 styles = {}
 for i, style_index in enumerate(style_indices):
 if i < len(REQUIRED_COMPUTED_STYLES) and 0 <= style_index < len(strings):
 styles[REQUIRED_COMPUTED_STYLES[i]] = strings[style_index]
 return styles

def build_snapshot_lookup(
 snapshot: CaptureSnapshotReturns,
 device_pixel_ratio: float = 1.0,
) -> dict[int, EnhancedSnapshotNode]:
 """Build a lookup table of backend node ID to enhanced snapshot data with everything calculated upfront"""
 import logging

 logger = logging.getLogger('system.dom.enhanced_snapshot')
 snapshot_lookup: dict[int, EnhancedSnapshotNode] = {}

 if not snapshot['documents']:
 return snapshot_lookup

 strings = snapshot['strings']
 logger.debug(f"SNAPSHOT: Processing {len(snapshot['documents'])} documents with {len(strings)} strings")
```

```

for doc_idx, document in enumerate(snapshot['documents']):
 nodes: NodeTreeSnapshot = document['nodes']
 layout: LayoutTreeSnapshot = document['layout']

 ## Build backend node id to snapshot index lookup
 backend_node_to_snapshot_index = {}
 if 'backendNodeId' in nodes:
 for i, backend_node_id in enumerate(nodes['backendNodeId']):
 backend_node_to_snapshot_index[backend_node_id] = i

 ## Log document info
 doc_url = strings[document.get('documentURL', 0)] if document.get('documentURL', 0) < len(strings) else None
 logger.debug(
 f'■ SNAPSHOT doc[{doc_idx}]: url={doc_url[:80]}... has {len(backend_node_to_snapshot_index)} nodes, '
 f'layout has {len(layout.get("nodeIndex", []))} entries'
)

 ## PERFORMANCE: Pre-build layout index map to eliminate O(n²) double lookups
 ## Preserve original behavior: use FIRST occurrence for duplicates
 layout_index_map = {}
 if layout and 'nodeIndex' in layout:
 for layout_idx, node_index in enumerate(layout['nodeIndex']):
 if node_index not in layout_index_map: # Only store first occurrence
 layout_index_map[node_index] = layout_idx

 ## Build snapshot lookup for each backend node id
 for backend_node_id, snapshot_index in backend_node_to_snapshot_index.items():
 is_clickable = None
 if 'isClickable' in nodes:
 is_clickable = _parse_rare_boolean_data(nodes['isClickable'], snapshot_index)

 ## Find corresponding layout node
 cursor_style = None
 is_visible = None
 bounding_box = None
 computed_styles = {}

 ## Look for layout tree node that corresponds to this snapshot node
 paint_order = None
 client_rects = None
 scroll_rects = None
 stacking_contexts = None
 if snapshot_index in layout_index_map:
 layout_idx = layout_index_map[snapshot_index]
 if layout_idx < len(layout.get('bounds', [])):
 ## Parse bounding box
 bounds = layout['bounds'][layout_idx]
 if len(bounds) >= 4:
 ## IMPORTANT: CDP coordinates are in device pixels, convert to CSS pixels
 ## by dividing by the device pixel ratio
 raw_x, raw_y, raw_width, raw_height = bounds[0], bounds[1], bounds[2], bounds[3]

 ## Apply device pixel ratio scaling to convert device pixels to CSS pixels
 bounding_box = DOMRect(
 x=raw_x / device_pixel_ratio,
 y=raw_y / device_pixel_ratio,
 width=raw_width / device_pixel_ratio,
 height=raw_height / device_pixel_ratio,
)

 ## Parse computed styles for this layout node
 if layout_idx < len(layout.get('styles', [])):
 style_indices = layout['styles'][layout_idx]
 computed_styles = _parse_computed_styles(strings, style_indices)
 cursor_style = computed_styles.get('cursor')

 ## Extract paint order if available
 if layout_idx < len(layout.get('paintOrders', [])):
 paint_order = layout.get('paintOrders', [])[layout_idx]

 ## Extract client rects if available
 client_rects_data = layout.get('clientRects', [])

```

```

 if layout_idx < len(client_rects_data):
 client_rect_data = client_rects_data[layout_idx]
 if client_rect_data and len(client_rect_data) >= 4:
 client_rects = DOMRect(
 x=client_rect_data[0],
 y=client_rect_data[1],
 width=client_rect_data[2],
 height=client_rect_data[3],
)

 # Extract scroll rects if available
 scroll_rects_data = layout.get('scrollRects', [])
 if layout_idx < len(scroll_rects_data):
 scroll_rect_data = scroll_rects_data[layout_idx]
 if scroll_rect_data and len(scroll_rect_data) >= 4:
 scroll_rects = DOMRect(
 x=scroll_rect_data[0],
 y=scroll_rect_data[1],
 width=scroll_rect_data[2],
 height=scroll_rect_data[3],
)

 # Extract stacking contexts if available
 if layout_idx < len(layout.get('stackingContexts', [])):
 stacking_contexts = layout.get('stackingContexts', {}).get('index', [])[layout_idx]

 snapshot_lookup[backend_node_id] = EnhancedSnapshotNode(
 is_clickable=is_clickable,
 cursor_style=cursor_style,
 bounds=bounding_box,
 clientRects=client_rects,
 scrollRects=scroll_rects,
 computed_styles=computed_styles if computed_styles else None,
 paint_order=paint_order,
 stacking_contexts=stacking_contexts,
)

 # Count how many have bounds (are actually visible/laid out)
 with_bounds = sum(1 for n in snapshot_lookup.values() if n.bounds)
 logger.debug(f'■ SNAPSHOT: Built lookup with {len(snapshot_lookup)} total entries, {with_bounds} have bounds')
 return snapshot_lookup

```

## **`.\system\dom\markdown_extractor.py`**

```

"""
Shared markdown extraction utilities for browser content processing.

This module provides a unified interface for extracting clean markdown from browser content,
used by both the tools service and page actor.
"""

import re
from dataclasses import dataclass
from enum import Enum, auto
from typing import TYPE_CHECKING, Any

from system.dom.serializer.html_serializer import HTMLSerializer
from system.dom.service import DomService
from system.dom.views import MarkdownChunk

if TYPE_CHECKING:
 from system.browser.session import BrowserSession
 from system.browser.watchdogs.dom_watchdog import DOMWatchdog

async def extract_clean_markdown(
 browser_session: 'BrowserSession | None' = None,
 dom_service: DomService | None = None,

```

```

■target_id: str | None = None,
■extract_links: bool = False,
) -> tuple[str, dict[str, Any]]:
■"""Extract clean markdown from browser content using enhanced DOM tree.

■This unified function can extract markdown using either a browser session (for tools service)
■or a DOM service with target ID (for page actor).

■Args:
■ browser_session: Browser session to extract content from (tools service path)
■ dom_service: DOM service instance (page actor path)
■ target_id: Target ID for the page (required when using dom_service)
■ extract_links: Whether to preserve links in markdown

■Returns:
■ tuple: (clean_markdown_content, content_statistics)

■Raises:
■ ValueError: If neither browser_session nor (dom_service + target_id) are provided
■"""
■# Validate input parameters
■if browser_session is not None:
■ if dom_service is not None or target_id is not None:
■ raise ValueError('Cannot specify both browser_session and dom_service/target_id')
■ # Browser session path (tools service)
■ enhanced_dom_tree = await _get_enhanced_dom_tree_from_browser_session(browser_session)
■ current_url = await browser_session.get_current_page_url()
■ method = 'enhanced_dom_tree'
■elif dom_service is not None and target_id is not None:
■ # DOM service path (page actor)
■ # Lazy fetch all_frames inside get_dom_tree if needed (for cross-origin iframes)
■ enhanced_dom_tree, _ = await dom_service.get_dom_tree(target_id=target_id, all_frames=None)
■ current_url = None # Not available via DOM service
■ method = 'dom_service'
■else:
■ raise ValueError('Must provide either browser_session or both dom_service and target_id')

■# Use the HTML serializer with the enhanced DOM tree
■html_serializer = HTMLSerializer(extract_links=extract_links)
■page_html = html_serializer.serialize(enhanced_dom_tree)

■original_html_length = len(page_html)

■# Use markdownify for clean markdown conversion
■from markdownify import markdownify as md

■content = md(
■ page_html,
■ heading_style='ATX', # Use # style headings
■ strip=['script', 'style'], # Remove these tags
■ bullets='-', # Use - for unordered lists
■ code_language='', # Don't add language to code blocks
■ escape_asterisks=False, # Don't escape asterisks (cleaner output)
■ escape_underscores=False, # Don't escape underscores (cleaner output)
■ escape_misc=False, # Don't escape other characters (cleaner output)
■ autolinks=False, # Don't convert URLs to <> format
■ default_title=False, # Don't add default title attributes
■ keep_inline_images_in=[], # Don't keep inline images in any tags (we already filter base64 in HTML)
■)

■initial_markdown_length = len(content)

■# Minimal cleanup - markdownify already does most of the work
■content = re.sub(r'%[0-9A-Fa-f]{2}', '', content) # Remove any remaining URL encoding

■# Apply light preprocessing to clean up excessive whitespace
■content, chars_filtered = _preprocess_markdown_content(content)

■final_filtered_length = len(content)

■# Content statistics
■stats = {

```

```

 ■■'method': method,
 ■■'original_html_chars': original_html_length,
 ■■'initial_markdown_chars': initial_markdown_length,
 ■■'filtered_chars_removed': chars_filtered,
 ■■'final_filtered_chars': final_filtered_length,
 ■}

 ■# Add URL to stats if available
 ■if current_url:
 ■ stats['url'] = current_url

 ■return content, stats

async def _get_enhanced_dom_tree_from_browser_session(browser_session: 'BrowserSession'):
 ■"""Get enhanced DOM tree from browser session via DOMWatchdog."""
 ■# Get the enhanced DOM tree from DOMWatchdog
 ■# This captures the current state of the page including dynamic content, shadow roots, etc.
 ■dom_watchdog: DOMWatchdog | None = browser_session._dom_watchdog
 ■assert dom_watchdog is not None, 'DOMWatchdog not available'

 ■# Use cached enhanced DOM tree if available, otherwise build it
 ■if dom_watchdog.enhanced_dom_tree is not None:
 ■ return dom_watchdog.enhanced_dom_tree

 ■# Build the enhanced DOM tree if not cached
 ■await dom_watchdog._build_dom_tree_without_highlights()
 ■enhanced_dom_tree = dom_watchdog.enhanced_dom_tree
 ■assert enhanced_dom_tree is not None, 'Enhanced DOM tree not available'

 ■return enhanced_dom_tree

Legacy aliases removed - all code now uses the unified extract_clean_markdown function

def _preprocess_markdown_content(content: str, max_newlines: int = 3) -> tuple[str, int]:
 ■"""
 ■Light preprocessing of markdown output - minimal cleanup with JSON blob removal.

 ■Args:
 ■ content: Markdown content to lightly filter
 ■ max_newlines: Maximum consecutive newlines to allow

 ■Returns:
 ■ tuple: (filtered_content, chars_filtered)
 ■"""
 ■original_length = len(content)

 ■# Remove JSON blobs (common in SPAs like LinkedIn, Facebook, etc.)
 ■# These are often embedded as `{"key":"value",...}` and can be massive
 ■# Match JSON objects/arrays that are at least 100 chars long
 ■# This catches SPA state/config data without removing small inline JSON
 ■content = re.sub(r'`(?:\{(?:[^\{\\}"]|\\.)*\}|(?:\[(?:[^\[\]\\]|\\.)*\])`', '', content, flags=re.DOTALL) # Remove JSON in code blocks
 ■content = re.sub(r'`(?:\{"$type":(?:[^\{\\}"]|\\.)*\}|(?:\[(?:[^\[\]\\]|\\.)*\])`', '', content) # Remove JSON with $type fields (common pattern)
 ■content = re.sub(r'`(?:\{"(?:[^\{\\}"]|\\.)*\}":(?:\{(?:[^\{\\}"]|\\.)*\}|(?:\[(?:[^\[\]\\]|\\.)*\])`', '', content) # Remove nested JSON objects

 ■# Compress consecutive newlines (4+ newlines become max_newlines)
 ■content = re.sub(r'\n{4,}', '\n' * max_newlines, content)

 ■# Remove lines that are only whitespace
 ■lines = content.split('\n')
 ■filtered_lines = []
 ■for line in lines:
 ■ stripped = line.strip()
 ■ ■# Keep all non-empty lines
 ■ if stripped:
 ■ ■# Skip lines that look like JSON (start with { or [and are very long)
 ■ if (stripped.startswith('{') or stripped.startswith '[') and len(stripped) > 100:
 ■ continue
 ■ filtered_lines.append(line)
 ■content = '\n'.join(filtered_lines)

```

```

 content = content.strip()

 chars_filtered = original_length - len(content)
 return content, chars_filtered

Structure-aware markdown chunking

class _BlockType(Enum):
 HEADER = auto()
 CODE_FENCE = auto()
 TABLE = auto()
 LIST_ITEM = auto()
 PARAGRAPH = auto()
 BLANK = auto()

@dataclass(slots=True)
class _AtomicBlock:
 block_type: _BlockType
 lines: list[str]
 char_start: int # offset in original content
 char_end: int # offset in original content (exclusive)

_TABLE_ROW_RE = re.compile(r'^\s*\|.*\|s*$')
_LIST_ITEM_RE = re.compile(r'^(\s*)([-*+]|d+[.]) ')
_LIST_CONTINUATION_RE = re.compile(r'^(\s{2,}|\t)')

def _parse_atomic_blocks(content: str) -> list[_AtomicBlock]:
 """Phase 1: Walk lines, group into unsplittable blocks."""
 lines = content.split('\n')
 blocks: list[_AtomicBlock] = []
 i = 0
 offset = 0 # char offset tracking

 while i < len(lines):
 line = lines[i]
 line_len = len(line) + 1 # +1 for the newline we split on

 ## BLANK
 if not line.strip():
 blocks.append(
 _AtomicBlock(
 block_type=_BlockType.BLANK,
 lines=[line],
 char_start=offset,
 char_end=offset + line_len,
)
)
 offset += line_len
 i += 1
 continue

 ## CODE FENCE
 if line.strip().startswith('```'):
 fence_lines = [line]
 fence_end = offset + line_len
 i += 1
 ## Consume until closing fence or EOF
 while i < len(lines):
 fence_line = lines[i]
 fence_line_len = len(fence_line) + 1
 fence_lines.append(fence_line)
 fence_end += fence_line_len
 i += 1
 if fence_line.strip().startswith('```') and len(fence_lines) > 1:
 break

```

```

 blocks.append(
 _AtomicBlock(
 block_type=_BlockType.CODE_FENCE,
 lines=fence_lines,
 char_start=offset,
 char_end=fence_end,
)
)
 offset = fence_end
 continue

 ## HEADER
 if line.lstrip().startswith('#'):
 blocks.append(
 _AtomicBlock(
 block_type=_BlockType.HEADER,
 lines=[line],
 char_start=offset,
 char_end=offset + line_len,
)
)
 offset += line_len
 i += 1
 continue

 ## TABLE (consecutive [...] lines)
 ## Header + separator row stay together; each data row is its own block
 if _TABLE_ROW_RE.match(line):
 ## Collect header line
 header_lines = [line]
 header_end = offset + line_len
 i += 1
 ## Check if next line is separator (contains ---)
 if i < len(lines) and _TABLE_ROW_RE.match(lines[i]) and '---' in lines[i]:
 sep = lines[i]
 sep_len = len(sep) + 1
 header_lines.append(sep)
 header_end += sep_len
 i += 1
 ## Emit header+separator as one atomic block
 blocks.append(
 _AtomicBlock(
 block_type=_BlockType.TABLE,
 lines=header_lines,
 char_start=offset,
 char_end=header_end,
)
)
 offset = header_end
 ## Each subsequent table row is its own TABLE block (splittable between rows)
 while i < len(lines) and _TABLE_ROW_RE.match(lines[i]):
 row = lines[i]
 row_len = len(row) + 1
 blocks.append(
 _AtomicBlock(
 block_type=_BlockType.TABLE,
 lines=[row],
 char_start=offset,
 char_end=offset + row_len,
)
)
 offset += row_len
 i += 1
 continue

 ## LIST ITEM (with indented continuations)
 if _LIST_ITEM_RE.match(line):
 list_lines = [line]
 list_end = offset + line_len
 i += 1
 ## Consume continuation lines (indented or blank between items)
 while i < len(lines):

```



```

#####next_line = lines[i]
#####next_len = len(next_line) + 1
Another list item at same or deeper indent → still part of this block
#####if _LIST_ITEM_RE.match(next_line):
#####list_lines.append(next_line)
#####list_end += next_len
#####i += 1
#####continue
Indented continuation
#####if next_line.strip() and _LIST_CONTINUATION_RE.match(next_line):
#####list_lines.append(next_line)
#####list_end += next_len
#####i += 1
#####continue
#####break
#####blocks.append(
#####_AtomicBlock(
#####block_type=_BlockType.LIST_ITEM,
#####lines=list_lines,
#####char_start=offset,
#####char_end=list_end,
#####)
#####)
#####offset = list_end
#####continue

PARAGRAPH (everything else, up to next blank line)
#####para_lines = [line]
#####para_end = offset + line_len
#####i += 1
#####while i < len(lines) and lines[i].strip():
Stop if next line starts a different block type
#####nl = lines[i]
#####if nl.lstrip().startswith('#') or nl.strip().startswith('```') or _TABLE_ROW_RE.match(nl) or _LIST_ITEM_RE.match(nl):
#####break
#####nl_len = len(nl) + 1
#####para_lines.append(nl)
#####para_end += nl_len
#####i += 1
#####blocks.append(
#####_AtomicBlock(
#####block_type=_BlockType.PARAGRAPH,
#####lines=para_lines,
#####char_start=offset,
#####char_end=para_end,
#####)
#####)
#####offset = para_end

Fix last block char_end: content may not end with \n
#####if blocks and content and not content.endswith('\n'):
#####blocks[-1] = _AtomicBlock(
#####block_type=blocks[-1].block_type,
#####lines=blocks[-1].lines,
#####char_start=blocks[-1].char_start,
#####char_end=len(content),
#####)

#####return blocks

def _block_text(block: _AtomicBlock) -> str:
#####return '\n'.join(block.lines)

def _get_table_header(block: _AtomicBlock) -> str | None:
#####"""Extract table header + separator rows from a TABLE block."""
#####assert block.block_type == _BlockType.TABLE
#####if len(block.lines) < 2:
#####return None
Header is first line, separator is second line (must contain ---)
#####sep_line = block.lines[1]

```

```

■if '---' in sep_line or '- -' in sep_line:
■return block.lines[0] + '\n' + block.lines[1]
■return None

def chunk_markdown_by_structure(
■content: str,
■max_chunk_chars: int = 100_000,
■overlap_lines: int = 5,
■start_from_char: int = 0,
) -> list[MarkdownChunk]:
■"""Split markdown into structure-aware chunks.

■Algorithm:
■ Phase 1 – Parse atomic blocks (headers, code fences, tables, list items, paragraphs).
■ Phase 2 – Greedy chunk assembly: accumulate blocks until exceeding max_chunk_chars.
■ A single block exceeding the limit is allowed (soft limit).
■ Phase 3 – Build overlap prefixes for context carry between chunks.

■Args:
■ content: Full markdown string.
■ max_chunk_chars: Target maximum chars per chunk (soft limit for single blocks).
■ overlap_lines: Number of trailing lines from previous chunk to prepend.
■ start_from_char: Return chunks starting from the chunk that contains this offset.

■Returns:
■ List of MarkdownChunk. Empty if start_from_char is past end of content.
■"""
■if not content:
■return [
■ MarkdownChunk(
■ content='',
■ chunk_index=0,
■ total_chunks=1,
■ char_offset_start=0,
■ char_offset_end=0,
■ overlap_prefix='',
■ has_more=False,
■)
■]

■if start_from_char >= len(content):
■return []

■# Phase 1: parse atomic blocks
■blocks = _parse_atomic_blocks(content)
■if not blocks:
■return []

■# Phase 2: greedy chunk assembly with header-preferred splitting
■raw_chunks: list[list[_AtomicBlock]] = []
■current_chunk: list[_AtomicBlock] = []
■current_size = 0

■for block in blocks:
■ block_size = block.char_end - block.char_start
■ # If adding this block would exceed limit AND we already have content, emit chunk
■ if current_size + block_size > max_chunk_chars and current_chunk:
■ # Prefer splitting at a header boundary within the current chunk.
■ # Scan backwards for the last HEADER block; if found and it wouldn't
■ # create a tiny chunk (< 50% of limit), split right before it so the
■ # header starts the next chunk for better semantic coherence.
■ best_split = len(current_chunk)
■ for j in range(len(current_chunk) - 1, 0, -1):
■ if current_chunk[j].block_type == _BlockType.HEADER:
■ prefix_size = sum(b.char_end - b.char_start for b in current_chunk[:j])
■ if prefix_size >= max_chunk_chars * 0.5:
■ best_split = j
■ break
■ raw_chunks.append(current_chunk[:best_split])
■ # Carry remaining blocks (from the header onward) into the next chunk
■ current_chunk = current_chunk[best_split:]

```

```

 current_size = sum(b.char_end - b.char_start for b in current_chunk)
 current_chunk.append(block)
 current_size += block_size

 if current_chunk:
 raw_chunks.append(current_chunk)

 total_chunks = len(raw_chunks)

 ## Phase 3: build MarkdownChunk objects with overlap prefixes
 chunks: list[MarkdownChunk] = []
 ## Track table header from previous chunk for table continuations
 prev_chunk_last_table_header: str | None = None

 for idx, chunk_blocks in enumerate(raw_chunks):
 chunk_text = '\n'.join(_block_text(b) for b in chunk_blocks)
 char_start = chunk_blocks[0].char_start
 char_end = chunk_blocks[-1].char_end

 ## Build overlap prefix
 overlap = ''
 if idx > 0:
 prev_blocks = raw_chunks[idx - 1]
 prev_text = '\n'.join(_block_text(b) for b in prev_blocks)
 prev_lines = prev_text.split('\n')

 ## Check if current chunk starts with a table continuation
 first_block = chunk_blocks[0]
 if first_block.block_type == _BlockType.TABLE and prev_chunk_last_table_header:
 ## Always prepend table header for continuation
 trailing = prev_lines[-(overlap_lines):] if overlap_lines > 0 else []
 header_lines = prev_chunk_last_table_header.split('\n')
 ## Deduplicate: don't repeat header lines if they're already in trailing
 combined = list(header_lines)
 for tl in trailing:
 if tl not in combined:
 combined.append(tl)
 overlap = '\n'.join(combined)
 elif overlap_lines > 0:
 overlap = '\n'.join(prev_lines[-(overlap_lines):])

 ## Track table header from this chunk for next iteration.
 ## Only overwrite if this chunk contains a new header+separator block;
 ## otherwise preserve the previous header so tables spanning 3+ chunks
 ## still get the header carried forward.
 for b in chunk_blocks:
 if b.block_type == _BlockType.TABLE:
 hdr = _get_table_header(b)
 if hdr is not None:
 prev_chunk_last_table_header = hdr

 has_more = idx < total_chunks - 1
 chunks.append(
 MarkdownChunk(
 content=chunk_text,
 chunk_index=idx,
 total_chunks=total_chunks,
 char_offset_start=char_start,
 char_offset_end=char_end,
 overlap_prefix=overlap,
 has_more=has_more,
)
)

 ## Apply start_from_char filter: return chunks from the one containing that offset
 if start_from_char > 0:
 for i, chunk in enumerate(chunks):
 if chunk.char_offset_end > start_from_char:
 return chunks[i:]
 return [] # offset past all chunks

return chunks

```

## ***.\system\dom\service.py***

```
import asyncio
import logging
import time
from typing import TYPE_CHECKING, Any

from cdp_use.cdp.accessibility.commands import GetFullAXTreeReturns
from cdp_use.cdp.accessibility.types import AXNode
from cdp_use.cdp.dom.types import Node
from cdp_use.cdp.target import TargetID

from system.dom.enhanced_snapshot import (
 ■REQUIRED_COMPUTED_STYLES,
 ■build_snapshot_lookup,
)
from system.dom.serializer.clickable_elements import ClickableElementDetector
from system.dom.serializer.serializer import DOMTreeSerializer
from system.dom.views import (
 ■DOMRect,
 ■EnhancedAXNode,
 ■EnhancedAXProperty,
 ■EnhancedDOMTreeNode,
 ■NodeType,
 ■SerializedDOMState,
 ■TargetAllTrees,
)
from system.observability import observe_debug
from system.utils import create_task_with_error_handling

if TYPE_CHECKING:
 ■from system.browser.session import BrowserSession

Note: iframe limits are now configurable via BrowserProfile.max_iframes and BrowserProfile.max_iframe_

class DomService:
 ■"""
 ■Service for getting the DOM tree and other DOM-related information.

 ■Either browser or page must be provided.

 ■TODO: currently we start a new websocket connection PER STEP, we should definitely keep this persistent
 ■"""

 ■logger: logging.Logger

 ■def __init__(
 ■ ■self,
 ■ ■browser_session: 'BrowserSession',
 ■ ■logger: logging.Logger | None = None,
 ■ ■cross_origin_iframes: bool = False,
 ■ ■paint_order_filtering: bool = True,
 ■ ■max_iframes: int = 100,
 ■ ■max_iframe_depth: int = 5,
 ■ ■viewport_threshold: int | None = 1000,
 ■):
 ■ ■self.browser_session = browser_session
 ■ ■self.logger = logger or browser_session.logger
 ■ ■self.cross_origin_iframes = cross_origin_iframes
 ■ ■self.paint_order_filtering = paint_order_filtering
 ■ ■self.max_iframes = max_iframes
 ■ ■self.max_iframe_depth = max_iframe_depth
 ■ ■self.viewport_threshold = viewport_threshold

 ■async def __aenter__(self):
 ■ ■return self
```

```

 async def __aexit__(self, exc_type, exc_value, traceback):
 pass # no need to cleanup anything, browser_session auto handles cleaning up session cache

 def _count_hidden_elements_in_iframes(self, node: EnhancedDOMTreeNode) -> None:
 """Collect hidden interactive elements in iframes for LLM hints.

 For each iframe, collects details of hidden interactive elements including
 tag, text/name, and scroll distance in pages so the agent knows how far to scroll.
 """

 def is_hidden_by_threshold(element: EnhancedDOMTreeNode) -> bool:
 """Check if element is hidden by viewport threshold (not CSS)."""
 if element.is_visible or not element.snapshot_node or not element.snapshot_node.bounds:
 return False

 computed_styles = element.snapshot_node.computed_styles or {}
 display = computed_styles.get('display', '').lower()
 visibility = computed_styles.get('visibility', '').lower()
 opacity = computed_styles.get('opacity', '1')

 css_hidden = display == 'none' or visibility == 'hidden'
 try:
 css_hidden = css_hidden or float(opacity) <= 0
 except (ValueError, TypeError):
 pass

 return not css_hidden

 def collect_hidden_elements(subtree_root: EnhancedDOMTreeNode, viewport_height: float) -> list[dict[str, Any]]:
 """Collect hidden interactive elements from subtree."""
 hidden: list[dict[str, Any]] = []

 if subtree_root.node_type == NodeType.ELEMENT_NODE:
 is_interactive = ClickableElementDetector.is_interactive(subtree_root)

 if is_interactive and is_hidden_by_threshold(subtree_root):
 # Get element text/name
 text = ''
 if subtree_root.ax_node and subtree_root.ax_node.name:
 text = subtree_root.ax_node.name[:40]
 elif subtree_root.attributes:
 text = (
 subtree_root.attributes.get('placeholder', '')
 or subtree_root.attributes.get('title', '')
 or subtree_root.attributes.get('aria-label', '')
)[:40]

 # Get y position and convert to pages
 y_pos = 0.0
 if subtree_root.snapshot_node and subtree_root.snapshot_node.bounds:
 y_pos = subtree_root.snapshot_node.bounds.y
 pages_down = round(y_pos / viewport_height, 1) if viewport_height > 0 else 0

 hidden.append(
 {
 'tag': subtree_root.tag_name or '?',
 'text': text or '(no label)',
 'pages': pages_down,
 }
)

 for child in subtree_root.children_nodes or []:
 hidden.extend(collect_hidden_elements(child, viewport_height))

 for shadow_root in subtree_root.shadow_roots or []:
 hidden.extend(collect_hidden_elements(shadow_root, viewport_height))

 return hidden

 def has_any_hidden_content(subtree_root: EnhancedDOMTreeNode) -> bool:
 """Check if there's any hidden content (interactive or not) in subtree."""
 if is_hidden_by_threshold(subtree_root):

```

```

#####return True

#####for child in subtree_root.children_nodes or []:
#####if has_any_hidden_content(child):
#####return True

#####for shadow_root in subtree_root.shadow_roots or []:
#####if has_any_hidden_content(shadow_root):
#####return True

#####return False

#####def process_node(current_node: EnhancedDOMTreeNode) -> None:
#####"""Process node and descendants, collecting hidden elements for iframes."""
#####if (
#####current_node.node_type == NodeType.ELEMENT_NODE
#####and current_node.tag_name
#####and current_node.tag_name.upper() in ('IFRAME', 'FRAME')
#####and current_node.content_document
#####):
Get viewport height from iframe's client rect
#####viewport_height = 0.0
#####if current_node.snapshot_node and current_node.snapshot_node.clientRects:
#####viewport_height = current_node.snapshot_node.clientRects.height

#####hidden = collect_hidden_elements(current_node.content_document, viewport_height)
Sort by pages and limit to avoid bloating context
#####hidden.sort(key=lambda x: x['pages'])
#####current_node.hidden_elements_info = hidden[:10] # Limit to 10

Check for hidden non-interactive content when no interactive elements found
#####if not hidden and has_any_hidden_content(current_node.content_document):
#####current_node.has_hidden_content = True

#####for child in current_node.children_nodes or []:
#####process_node(child)

#####if current_node.content_document:
#####process_node(current_node.content_document)

#####for shadow_root in current_node.shadow_roots or []:
#####process_node(shadow_root)

#####process_node(node)

#####def _build_enhanced_ax_node(self, ax_node: AXNode) -> EnhancedAXNode:
#####properties: list[EnhancedAXProperty] | None = None
#####if 'properties' in ax_node and ax_node['properties']:
#####properties = []
#####for property in ax_node['properties']:
#####try:
test whether property name can go into the enum (sometimes Chrome returns some random properties)
#####properties.append(
#####EnhancedAXProperty(
#####name=property['name'],
#####value=property.get('value', {}).get('value', None),
related_nodes=[], # TODO: add related nodes
#####)
#####)
#####except ValueError:
#####pass

#####enhanced_ax_node = EnhancedAXNode(
#####ax_node_id=ax_node['nodeId'],
#####ignored=ax_node['ignored'],
#####role=ax_node.get('role', {}).get('value', None),
#####name=ax_node.get('name', {}).get('value', None),
#####description=ax_node.get('description', {}).get('value', None),
#####properties=properties,
#####child_ids=ax_node.get('childIds', []) if ax_node.get('childIds') else None,
#####)
#####return enhanced_ax_node

```

```

 async def _get_viewport_ratio(self, target_id: TargetID) -> float:
 """Get viewport dimensions, device pixel ratio, and scroll position using CDP."""
 cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=target_id, focus=False)

 try:
 # Get the layout metrics which includes the visual viewport
 metrics = await cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=cdp_session.session_id)

 visual_viewport = metrics.get('visualViewport', {})

 # IMPORTANT: Use CSS viewport instead of device pixel viewport
 # This fixes the coordinate mismatch on high-DPI displays
 css_visual_viewport = metrics.get('cssVisualViewport', {})
 css_layout_viewport = metrics.get('cssLayoutViewport', {})

 # Use CSS pixels (what JavaScript sees) instead of device pixels
 width = css_visual_viewport.get('clientWidth', css_layout_viewport.get('clientWidth', 1920.0))

 # Calculate device pixel ratio
 device_width = visual_viewport.get('clientWidth', width)
 css_width = css_visual_viewport.get('clientWidth', width)
 device_pixel_ratio = device_width / css_width if css_width > 0 else 1.0

 return float(device_pixel_ratio)
 except Exception as e:
 self.logger.debug(f'Viewport size detection failed: {e}')
 # Fallback to default viewport size
 return 1.0

 @classmethod
 def is_element_visible_according_to_all_parents(
 cls, node: EnhancedDOMTreeNode, html_frames: list[EnhancedDOMTreeNode], viewport_threshold: int | None
) -> bool:
 """Check if the element is visible according to all its parent HTML frames.

 Args:
 node: The DOM node to check visibility for
 html_frames: List of parent HTML frame nodes
 viewport_threshold: Pixel threshold beyond viewport to consider visible.
 Default 1000px. Set to None to disable threshold checking entirely.
 """

 if not node.snapshot_node:
 return False

 computed_styles = node.snapshot_node.computed_styles or {}

 display = computed_styles.get('display', '').lower()
 visibility = computed_styles.get('visibility', '').lower()
 opacity = computed_styles.get('opacity', '1')

 if display == 'none' or visibility == 'hidden':
 return False

 try:
 if float(opacity) <= 0:
 return False
 except (ValueError, TypeError):
 pass

 # Start with the element's local bounds (in its own frame's coordinate system)
 current_bounds = node.snapshot_node.bounds

 if not current_bounds:
 return False # If there are no bounds, the element is not visible

 # If threshold is None, skip all viewport-based filtering (only check CSS visibility)
 if viewport_threshold is None:
 return True

 """
 Reverse iterate through the html frames (that can be either iframe or document -> if it's a document

```

```

"""
for frame in reversed(html_frames):
 if (
 frame.node_type == NodeType.ELEMENT_NODE
 and (frame.node_name.upper() == 'IFRAME' or frame.node_name.upper() == 'FRAME')
 and frame.snapshot_node
 and frame.snapshot_node.bounds
):
 iframe_bounds = frame.snapshot_node.bounds

 # negate the values added in `_construct_enhanced_node`
 current_bounds.x += iframe_bounds.x
 current_bounds.y += iframe_bounds.y

 if (
 frame.node_type == NodeType.ELEMENT_NODE
 and frame.node_name == 'HTML'
 and frame.snapshot_node
 and frame.snapshot_node.scrollRects
 and frame.snapshot_node.clientRects
):
 # For iframe content, we need to check visibility within the iframe's viewport
 # The scrollRects represent the current scroll position
 # The clientRects represent the viewport size
 # Elements are visible if they fall within the viewport after accounting for scroll

 # The viewport of the frame (what's actually visible)
 viewport_left = 0 # Viewport always starts at 0 in frame coordinates
 viewport_top = 0
 viewport_right = frame.snapshot_node.clientRects.width
 viewport_bottom = frame.snapshot_node.clientRects.height

 # Adjust element bounds by the scroll offset to get position relative to viewport
 # When scrolled down, scrollRects.y is positive, so we subtract it from element's y
 adjusted_x = current_bounds.x - frame.snapshot_node.scrollRects.x
 adjusted_y = current_bounds.y - frame.snapshot_node.scrollRects.y

 frame_intersects = (
 adjusted_x < viewport_right
 and adjusted_x + current_bounds.width > viewport_left
 and adjusted_y < viewport_bottom + viewport_threshold
 and adjusted_y + current_bounds.height > viewport_top - viewport_threshold
)

 if not frame_intersects:
 return False

 # Keep the original coordinate adjustment to maintain consistency
 # This adjustment is needed for proper coordinate transformation
 current_bounds.x -= frame.snapshot_node.scrollRects.x
 current_bounds.y -= frame.snapshot_node.scrollRects.y

 # If we reach here, element is visible in main viewport and all containing iframes
 return True

async def _get_ax_tree_for_all_frames(self, target_id: TargetID) -> GetFullAXTreeReturns:
 """Recursively collect all frames and merge their accessibility trees into a single array."""

 cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=target_id, focus=False)
 frame_tree = await cdp_session.cdp_client.send.Page.getFrameTree(session_id=cdp_session.session_id)

 def collect_all_frame_ids(frame_tree_node) -> list[str]:
 """Recursively collect all frame IDs from the frame tree."""
 frame_ids = [frame_tree_node['frame']['id']]

 if 'childFrames' in frame_tree_node and frame_tree_node['childFrames']:
 for child_frame in frame_tree_node['childFrames']:
 frame_ids.extend(collect_all_frame_ids(child_frame))

 return frame_ids

Collect all frame IDs recursively

```



```

 all_frame_ids = collect_all_frame_ids(frame_tree['frameTree'])

 ## Get accessibility tree for each frame
 ax_tree_requests = []
 for frame_id in all_frame_ids:
 ax_tree_request = cdp_session.cdp_client.send.Accessibility.getFullAXTree(
 params={'frameId': frame_id}, session_id=cdp_session.session_id
)
 ax_tree_requests.append(ax_tree_request)

 ## Wait for all requests to complete
 ax_trees = await asyncio.gather(*ax_tree_requests)

 ## Merge all AX nodes into a single array
 merged_nodes: list[AXNode] = []
 for ax_tree in ax_trees:
 merged_nodes.extend(ax_tree['nodes'])

 return {'nodes': merged_nodes}

 async def _get_all_trees(self, target_id: TargetID) -> TargetAllTrees:
 cdp_session = await self.browser_session.get_or_create_cdp_session(target_id=target_id, focus=False)

 ## Wait for the page to be ready first
 try:
 ready_state = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': 'document.readyState'}, session_id=cdp_session.session_id
)
 except Exception as e:
 pass # Page might not be ready yet
 ## DEBUG: Log before capturing snapshot
 self.logger.debug(f"DEBUG: Capturing DOM snapshot for target {target_id}")

 ## Get actual scroll positions for all iframes before capturing snapshot
 start_iframe_scroll = time.time()
 iframe_scroll_positions = {}
 try:
 scroll_result = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={
 'expression': """
 (() => {
 const scrollData = {};
 const iframes = document.querySelectorAll('iframe');
 iframes.forEach((iframe, index) => {
 try {
 const doc = iframe.contentDocument || iframe.contentWindow.document;
 if (doc) {
 scrollData[index] = {
 scrollTop: doc.documentElement.scrollTop || doc.body.scrollTop || 0,
 scrollLeft: doc.documentElement.scrollLeft || doc.body.scrollLeft || 0
 };
 }
 } catch (e) {
 // Cross-origin iframe, can't access
 }
 });
 return scrollData;
 })()
 """,
 'returnByValue': True,
 },
 session_id=cdp_session.session_id,
)
 if scroll_result and 'result' in scroll_result and 'value' in scroll_result['result']:
 iframe_scroll_positions = scroll_result['result']['value']
 for idx, scroll_data in iframe_scroll_positions.items():
 self.logger.debug(
 f"DEBUG: Iframe {idx} actual scroll position - scrollTop={scroll_data.get(\"scrollTop\", 0)}, scrollLeft={scroll_data.get(\"scrollLeft\", 0)}"
)
 except Exception as e:
 self.logger.debug(f'Failed to get iframe scroll positions: {e}')
 iframe_scroll_ms = (time.time() - start_iframe_scroll) * 1000

```

```

Detect elements with JavaScript click event listeners (without mutating DOM)
#####start_js_listener_detection = time.time()
#####js_click_listener_backend_ids: set[int] = set()
#####try:
Step 1: Run JS to find elements with click listeners and return them by reference
#####js_listener_result = await cdp_session.cdp_client.send.Runtime.evaluate(
#####params={
#####'expression': """
#####(() => {
#####// getEventListeners is only available in DevTools context via includeCommandLineAPI
#####if (typeof getEventListeners !== 'function') {
#####return null;
#####}

#####const elementsWithListeners = [];
#####const allElements = document.querySelectorAll('*');

#####for (const el of allElements) {
#####try {
#####const listeners = getEventListeners(el);
#####// Check for click-related event listeners
#####if (listeners.click || listeners.mousedown || listeners.mouseup || listeners.pointerdown || li
#####elementsWithListeners.push(el);
#####} catch (e) {
#####// Ignore errors for individual elements (e.g., cross-origin)
#####}
#####}

#####return elementsWithListeners;
#####})()
#####"""
#####'includeCommandLineAPI': True, # enables getEventListeners()
#####'returnByValue': False, # Return object references, not values
#####},
#####session_id=cdp_session.session_id,
#####)

#####result_object_id = js_listener_result.get('result', {}).get('objectId')
#####if result_object_id:
Step 2: Get array properties to access each element
#####array_props = await cdp_session.cdp_client.send.Runtime.getProperties(
#####params={
#####'objectId': result_object_id,
#####'ownProperties': True,
#####},
#####session_id=cdp_session.session_id,
#####)

Step 3: For each element, get its backend node ID via DOM.describeNode
#####element_object_ids: list[str] = []
#####for prop in array_props.get('result', []):
Array indices are numeric property names
#####prop_name = prop.get('name', '') if isinstance(prop, dict) else ''
#####if isinstance(prop_name, str) and prop_name.isdigit():
#####prop_value = prop.get('value', {}) if isinstance(prop, dict) else {}
#####if isinstance(prop_value, dict):
#####object_id = prop_value.get('objectId')
#####if object_id and isinstance(object_id, str):
#####element_object_ids.append(object_id)

Batch resolve backend node IDs (run in parallel)
#####async def get_backend_node_id(object_id: str) -> int | None:
#####try:
#####node_info = await cdp_session.cdp_client.send.DOM.describeNode(
#####params={'objectId': object_id},
#####session_id=cdp_session.session_id,
#####)
#####return node_info.get('node', {}).get('backendNodeId')
#####except Exception:
#####return None
Resolve all element object IDs to backend node IDs in parallel

```

```

#####backend_ids = await asyncio.gather(*[get_backend_node_id(oid) for oid in element_object_ids])
#####js_click_listener_backend_ids = {bid for bid in backend_ids if bid is not None}

Release the array object to avoid memory leaks
#####try:
await cdp_session.cdp_client.send.Runtime.releaseObject(
params={'objectId': result_object_id},
session_id=cdp_session.session_id,
)
#####except Exception:
pass # Best effort cleanup

#####self.logger.debug(f'Detected {len(js_click_listener_backend_ids)} elements with JS click listeners')
#####except Exception as e:
self.logger.debug(f'Failed to detect JS event listeners: {e}')
#####js_listener_detection_ms = (time.time() - start_js_listener_detection) * 1000

Define CDP request factories to avoid duplication
#####def create_snapshot_request():
return cdp_session.cdp_client.send.DOMSnapshot.captureSnapshot(
params={
'computedStyles': REQUIRED_COMPUTED_STYLES,
'includePaintOrder': True,
'includeDOMRects': True,
'includeBlendedBackgroundColors': False,
'includeTextColorOpacities': False,
},
session_id=cdp_session.session_id,
)

#####def create_dom_tree_request():
return cdp_session.cdp_client.send.DOM.getDocument(
params={'depth': -1, 'pierce': True}, session_id=cdp_session.session_id
)

#####start_cdp_calls = time.time()

Create initial tasks
#####tasks = {
'snapshot': create_task_with_error_handling(create_snapshot_request(), name='get_snapshot'),
'dom_tree': create_task_with_error_handling(create_dom_tree_request(), name='get_dom_tree'),
'ax_tree': create_task_with_error_handling(self._get_ax_tree_for_all_frames(target_id), name='get_ax_tree'),
'device_pixel_ratio': create_task_with_error_handling(self._get_viewport_ratio(target_id), name='get_device_pixel_ratio')
#####}

Wait for all tasks with timeout
#####done, pending = await asyncio.wait(tasks.values(), timeout=10.0)

Retry any failed or timed out tasks
#####if pending:
for task in pending:
task.cancel()

Retry mapping for pending tasks
#####retry_map = {
tasks['snapshot']: lambda: create_task_with_error_handling(create_snapshot_request(), name='get_snapshot'),
tasks['dom_tree']: lambda: create_task_with_error_handling(create_dom_tree_request(), name='get_dom_tree'),
tasks['ax_tree']: lambda: create_task_with_error_handling(self._get_ax_tree_for_all_frames(target_id), name='get_ax_tree'),
self._get_ax_tree_for_all_frames(target_id), name='get_ax_tree_retry'
#####),
tasks['device_pixel_ratio']: lambda: create_task_with_error_handling(
self._get_viewport_ratio(target_id), name='get_viewport_ratio_retry'
),
#####}

Create new tasks only for the ones that didn't complete
#####for key, task in tasks.items():
if task in pending and task in retry_map:
tasks[key] = retry_map[task]()

Wait again with shorter timeout
#####done2, pending2 = await asyncio.wait([t for t in tasks.values() if not t.done()], timeout=2.0)

```

```

 if pending2:
 for task in pending2:
 task.cancel()

 ## Extract results, tracking which ones failed
 results = {}
 failed = []
 for key, task in tasks.items():
 if task.done() and not task.cancelled():
 try:
 results[key] = task.result()
 except Exception as e:
 self.logger.warning(f'CDP request {key} failed with exception: {e}')
 failed.append(key)
 else:
 self.logger.warning(f'CDP request {key} timed out')
 failed.append(key)

 ## If any required tasks failed, raise an exception
 if failed:
 raise TimeoutError(f'CDP requests failed or timed out: {" ".join(failed)}')

 snapshot = results['snapshot']
 dom_tree = results['dom_tree']
 ax_tree = results['ax_tree']
 device_pixel_ratio = results['device_pixel_ratio']
 end_cdp_calls = time.time()
 cdp_calls_ms = (end_cdp_calls - start_cdp_calls) * 1000

 ## Calculate total time for _get_all_trees and overhead
 start_snapshot_processing = time.time()

 ## DEBUG: Log snapshot info and limit documents to prevent explosion
 if snapshot and 'documents' in snapshot:
 original_doc_count = len(snapshot['documents'])
 ## Limit to max_iframes documents to prevent iframe explosion
 if original_doc_count > self.max_iframes:
 self.logger.warning(
 f'Limiting processing of {original_doc_count} iframes on page to only first {self.max_iframes}'
)
 snapshot['documents'] = snapshot['documents'][: self.max_iframes]

 total_nodes = sum(len(doc.get('nodes', [])) for doc in snapshot['documents'])
 self.logger.debug(f'DEBUG: Snapshot contains {len(snapshot["documents"])} frames with {total_nodes} nodes')
 ## Log iframe-specific info
 for doc_idx, doc in enumerate(snapshot['documents']):
 if doc_idx > 0: # Not the main document
 self.logger.debug(
 f'DEBUG: Iframe #{doc_idx} {doc.get("frameId", "no-frame-id")} {doc.get("url", "no-url")} has'
)

 snapshot_processing_ms = (time.time() - start_snapshot_processing) * 1000

 ## Return with detailed timing breakdown
 return TargetAllTrees(
 snapshot=snapshot,
 dom_tree=dom_tree,
 ax_tree=ax_tree,
 device_pixel_ratio=device_pixel_ratio,
 cdp_timing={
 'iframe_scroll_detection_ms': iframe_scroll_ms,
 'js_listener_detection_ms': js_listener_detection_ms,
 'cdp_parallel_calls_ms': cdp_calls_ms,
 'snapshot_processing_ms': snapshot_processing_ms,
 },
 js_click_listener_backend_ids=js_click_listener_backend_ids if js_click_listener_backend_ids else None
)

 @observe_debug(ignore_input=True, ignore_output=True, name='get_dom_tree')
 async def get_dom_tree(
 self,
 target_id: TargetID,

```

```

 all_frames: dict | None = None,
 initial_html_frames: list[EnhancedDOMTreeNode] | None = None,
 initial_total_frame_offset: DOMRect | None = None,
 iframe_depth: int = 0,
) -> tuple[EnhancedDOMTreeNode, dict[str, float]]:
 """Get the DOM tree for a specific target.

 Args:
 target_id: Target ID of the page to get the DOM tree for.
 all_frames: Pre-fetched frame hierarchy to avoid redundant CDP calls (optional, lazy fetch if None)
 initial_html_frames: List of HTML frame nodes encountered so far
 initial_total_frame_offset: Accumulated coordinate offset
 iframe_depth: Current depth of iframe nesting to prevent infinite recursion

 Returns:
 Tuple of (enhanced_dom_tree_node, timing_info)
 """
 timing_info: dict[str, float] = {}
 timing_start_total = time.time()

 ## Get all trees from CDP (snapshot, DOM, AX, viewport ratio)
 start_get_trees = time.time()
 trees = await self._get_all_trees(target_id)
 get_trees_ms = (time.time() - start_get_trees) * 1000
 timing_info.update(trees.cdp_timing)
 timing_info['get_all_trees_total_ms'] = get_trees_ms

 dom_tree = trees.dom_tree
 ax_tree = trees.ax_tree
 snapshot = trees.snapshot
 device_pixel_ratio = trees.device_pixel_ratio
 js_click_listener_backend_ids = trees.js_click_listener_backend_ids or set()

 ## Build AX tree lookup
 start_ax = time.time()
 ax_tree_lookup: dict[int, AXNode] = {
 ax_node['backendDOMNodeId']: ax_node for ax_node in ax_tree['nodes'] if 'backendDOMNodeId' in ax_node
 }
 timing_info['build_ax_lookup_ms'] = (time.time() - start_ax) * 1000

 enhanced_dom_tree_node_lookup: dict[int, EnhancedDOMTreeNode] = {}
 """ NodeId (NOT backend node id) -> enhanced dom tree node""" # way to get the parent/content node

 ## Parse snapshot data with everything calculated upfront
 start_snapshot = time.time()
 snapshot_lookup = build_snapshot_lookup(snapshot, device_pixel_ratio)
 timing_info['build_snapshot_lookup_ms'] = (time.time() - start_snapshot) * 1000

 async def _construct_enhanced_node(
 node: Node,
 html_frames: list[EnhancedDOMTreeNode] | None,
 total_frame_offset: DOMRect | None,
 all_frames: dict | None,
) -> EnhancedDOMTreeNode:
 """
 Recursively construct enhanced DOM tree nodes.

 Args:
 node: The DOM node to construct
 html_frames: List of HTML frame nodes encountered so far
 total_frame_offset: Accumulated coordinate translation from parent iframes (includes scroll correct)
 all_frames: Pre-fetched frame hierarchy to avoid redundant CDP calls
 """

 ## Initialize lists if not provided
 if html_frames is None:
 html_frames = []

 ## to get rid of the pointer references
 if total_frame_offset is None:
 total_frame_offset = DOMRect(x=0.0, y=0.0, width=0.0, height=0.0)
 else:

```

```

#####total_frame_offset = DOMRect(
#####total_frame_offset.x, total_frame_offset.y, total_frame_offset.width, total_frame_offset.height
#####)

memoize the mf (I don't know if some nodes are duplicated)
#####if node['nodeId'] in enhanced_dom_tree_node_lookup:
#####return enhanced_dom_tree_node_lookup[node['nodeId']]

#####ax_node = ax_tree_lookup.get(node['backendNodeId'])
#####if ax_node:
#####enhanced_ax_node = self._build_enhanced_ax_node(ax_node)
#####else:
#####enhanced_ax_node = None

To make attributes more readable
#####attributes: dict[str, str] | None = None
#####if 'attributes' in node and node['attributes']:
#####attributes = {}
#####for i in range(0, len(node['attributes']), 2):
#####attributes[node['attributes'][i]] = node['attributes'][i + 1]

#####shadow_root_type = None
#####if 'shadowRootType' in node and node['shadowRootType']:
#####try:
#####shadow_root_type = node['shadowRootType']
#####except ValueError:
#####pass

Get snapshot data and calculate absolute position
#####snapshot_data = snapshot_lookup.get(node['backendNodeId'], None)

DIAGNOSTIC: Log when interactive elements don't have snapshot data
#####if not snapshot_data and node['nodeName'].upper() in ['INPUT', 'BUTTON', 'SELECT', 'TEXTAREA', 'A']:
#####parent_has_shadow = False
#####parent_info = ''
#####if 'parentId' in node and node['parentId'] in enhanced_dom_tree_node_lookup:
#####parent = enhanced_dom_tree_node_lookup[node['parentId']]
#####if parent.shadow_root_type:
#####parent_has_shadow = True
#####parent_info = f'parent={parent.tag_name}(shadow={parent.shadow_root_type})'
#####attr_str = ''
#####if 'attributes' in node and node['attributes']:
#####attrs_dict = {node['attributes'][i]: node['attributes'][i + 1] for i in range(0, len(node['attributes']), 2)}
#####attr_str = f'name={attrs_dict.get("name", "N/A")} id={attrs_dict.get("id", "N/A")}'
#####self.logger.debug(
#####f' NO SNAPSHOT DATA for <{node["nodeName"]} backendNodeId={node["backendNodeId"]} '
#####f'{attr_str} {parent_info} (snapshot_lookup has {len(snapshot_lookup)} entries)'
#####)

#####absolute_position = None
#####if snapshot_data and snapshot_data.bounds:
#####absolute_position = DOMRect(
#####x=snapshot_data.bounds.x + total_frame_offset.x,
#####y=snapshot_data.bounds.y + total_frame_offset.y,
#####width=snapshot_data.bounds.width,
#####height=snapshot_data.bounds.height,
#####)

#####try:
#####session = await self.browser_session.get_or_create_cdp_session(target_id, focus=False)
#####session_id = session.session_id
#####except ValueError:
Target may have detached during DOM construction
#####session_id = None

#####dom_tree_node = EnhancedDOMTreeNode(
#####node_id=node['nodeId'],
#####backend_node_id=node['backendNodeId'],
#####node_type=NodeType(node['nodeType']),
#####node_name=node['nodeName'],
#####node_value=node['nodeValue'],
#####attributes=attributes or {},

```

```

#####is_scrollable=node.get('isScrollable', None),
#####frame_id=node.get('frameId', None),
#####session_id=session_id,
#####target_id=target_id,
#####content_document=None,
#####shadow_root_type=shadow_root_type,
#####shadow_roots=None,
#####parent_node=None,
#####children_nodes=None,
#####ax_node=enhanced_ax_node,
#####snapshot_node=snapshot_data,
#####is_visible=None,
#####has_js_click_listener=node['backendNodeId'] in js_click_listener_backend_ids,
#####absolute_position=absolute_position,
#####)

#####enhanced_dom_tree_node_lookup[node['nodeId']] = dom_tree_node

#####if 'parentId' in node and node['parentId']:
#####dom_tree_node.parent_node = enhanced_dom_tree_node_lookup[
#####node['parentId']
#####] # parents should always be in the lookup

Check if this is an HTML frame node and add it to the list
#####updated_html_frames = html_frames.copy()
#####if node['nodeType'] == NodeType.ELEMENT_NODE.value and node['nodeName'] == 'HTML' and node.get('frame
#####updated_html_frames.append(dom_tree_node)

and adjust the total frame offset by scroll
#####if snapshot_data and snapshot_data.scrollRects:
#####total_frame_offset.x -= snapshot_data.scrollRects.x
#####total_frame_offset.y -= snapshot_data.scrollRects.y
DEBUG: Log iframe scroll information
#####self.logger.debug(
#####f'■ DEBUG: HTML frame scroll - scrollY={snapshot_data.scrollRects.y}, scrollX={snapshot_data.scr
#####)

Calculate new iframe offset for content documents, accounting for iframe scroll
#####if (
#####(node['nodeName'].upper() == 'IFRAME' or node['nodeName'].upper() == 'FRAME')
#####and snapshot_data
#####and snapshot_data.bounds
#####):
#####if snapshot_data.bounds:
#####updated_html_frames.append(dom_tree_node)

#####total_frame_offset.x += snapshot_data.bounds.x
#####total_frame_offset.y += snapshot_data.bounds.y

#####if 'contentDocument' in node and node['contentDocument']:
#####dom_tree_node.content_document = await _construct_enhanced_node(
#####node['contentDocument'], updated_html_frames, total_frame_offset, all_frames
#####)
#####dom_tree_node.content_document.parent_node = dom_tree_node
forcefully set the parent node to the content document node (helps traverse the tree)

#####if 'shadowRoots' in node and node['shadowRoots']:
#####dom_tree_node.shadow_roots = []
#####for shadow_root in node['shadowRoots']:
#####shadow_root_node = await _construct_enhanced_node(
#####shadow_root, updated_html_frames, total_frame_offset, all_frames
#####)
forcefully set the parent node to the shadow root node (helps traverse the tree)
#####shadow_root_node.parent_node = dom_tree_node
#####dom_tree_node.shadow_roots.append(shadow_root_node)

#####if 'children' in node and node['children']:
#####dom_tree_node.children_nodes = []
Build set of shadow root node IDs to filter them out from children
#####shadow_root_node_ids = set()
#####if 'shadowRoots' in node and node['shadowRoots']:
#####for shadow_root in node['shadowRoots']:

```

```

#####shadow_root_node_ids.add(shadow_root['nodeId'])

#####for child in node['children']:
Skip shadow roots - they should only be in shadow_roots list
#####if child['nodeId'] in shadow_root_node_ids:
#####continue
#####dom_tree_node.children_nodes.append(
#####await _construct_enhanced_node(child, updated_html_frames, total_frame_offset, all_frames)
#####)

Set visibility using the collected HTML frames and viewport threshold
#####dom_tree_node.is_visible = self.is_element_visible_according_to_all_parents(
#####dom_tree_node, updated_html_frames, self.viewport_threshold
#####)

DEBUG: Log visibility info for form elements in iframes
#####if dom_tree_node.tag_name and dom_tree_node.tag_name.upper() in ['INPUT', 'SELECT', 'TEXTAREA', 'LAB
#####attrs = dom_tree_node.attributes or {}
#####elem_id = attrs.get('id', '')
#####elem_name = attrs.get('name', '')
#####if (
#####'city' in elem_id.lower()
#####or 'city' in elem_name.lower()
#####or 'state' in elem_id.lower()
#####or 'state' in elem_name.lower()
#####or 'zip' in elem_id.lower()
#####or 'zip' in elem_name.lower()
#####):
#####self.logger.debug(
#####if"##### DEBUG: Form element {dom_tree_node.tag_name} id='{elem_id}' name='{elem_name}' - visible={do
#####)

handle cross origin iframe (just recursively call the main function with the proper target if it e
only do this if the iframe is visible (otherwise it's not worth it)

#####if (
TODO: hacky way to disable cross origin iframes for now
#####self.cross_origin_iframes and node['nodeName'].upper() == 'IFRAME' and node.get('contentDocument',
#####): # None meaning there is no content
Check iframe depth to prevent infinite recursion
#####if iframe_depth >= self.max_iframe_depth:
#####self.logger.debug(
#####if'Skipping iframe at depth {iframe_depth} to prevent infinite recursion (max depth: {self.max_ifr
#####)
#####else:
Check if iframe is visible and large enough (>= 50px in both dimensions)
#####should_process_iframe = False

First check if the iframe element itself is visible
#####if dom_tree_node.is_visible:
Check iframe dimensions
#####if dom_tree_node.snapshot_node and dom_tree_node.snapshot_node.bounds:
#####bounds = dom_tree_node.snapshot_node.bounds
#####width = bounds.width
#####height = bounds.height

Only process if iframe is at least 50px in both dimensions
#####if width >= 50 and height >= 50:
#####should_process_iframe = True
#####self.logger.debug(f'Processing cross-origin iframe: visible=True, width={width}, height={height}
#####else:
#####self.logger.debug(
#####if'Skipping small cross-origin iframe: width={width}, height={height} (needs >= 50px)'
#####)
#####else:
#####self.logger.debug('Skipping cross-origin iframe: no bounds available')
#####else:
#####self.logger.debug('Skipping invisible cross-origin iframe')

#####if should_process_iframe:
Lazy fetch all_frames only when actually needed (for cross-origin iframes)
#####if all_frames is None:

```



```

#####all_frames, _ = await self.browser_session.get_all_frames()

Use pre-fetched all_frames to find the iframe's target (no redundant CDP call)
#####frame_id = node.get('frameId', None)
#####if frame_id:
frame_info = all_frames.get(frame_id)
iframe_document_target = None
if frame_info and frame_info.get('frameTargetId'):
iframe_target_id = frame_info['frameTargetId']
iframe_target = self.browser_session.session_manager.get_target(iframe_target_id)
if iframe_target:
iframe_document_target = {
'targetId': iframe_target.target_id,
'url': iframe_target.url,
'title': iframe_target.title,
'type': iframe_target.target_type,
}
else:
iframe_document_target = None
if target actually exists in one of the frames, just recursively build the dom tree for it
if iframe_document_target:
self.logger.debug(
f'Getting content document for iframe {node.get("frameId", None)} at depth {iframe_depth + 1}'
)
content_document, _ = await self.get_dom_tree(
target_id=iframe_document_target['targetId'],
all_frames=all_frames,
TODO: experiment with this values -> not sure whether the whole cross origin iframe should b
Current config: if the cross origin iframe is AT ALL visible, then just include everything i
initial_html_frames=updated_html_frames,
initial_total_frame_offset=total_frame_offset,
iframe_depth=iframe_depth + 1,
)

#####dom_tree_node.content_document = content_document
#####dom_tree_node.content_document.parent_node = dom_tree_node

###return dom_tree_node

Build enhanced DOM tree recursively
Note: all_frames stays None and will be lazily fetched inside _construct_enhanced_node
only if/when a cross-origin iframe is encountered
###start_construct = time.time()
###enhanced_dom_tree_node = await _construct_enhanced_node(
dom_tree['root'], initial_html_frames, initial_total_frame_offset, all_frames
###)
###timing_info['construct_enhanced_tree_ms'] = (time.time() - start_construct) * 1000

Count hidden elements per iframe for LLM hints
###self._count_hidden_elements_in_iframes(enhanced_dom_tree_node)

Calculate total time for get_dom_tree
###total_get_dom_tree_ms = (time.time() - timing_start_total) * 1000
###timing_info['get_dom_tree_total_ms'] = total_get_dom_tree_ms

Calculate overhead in get_dom_tree (time not accounted for by sub-operations)
###tracked_sub_operations_ms = (
timing_info.get('get_all_trees_total_ms', 0)
+ timing_info.get('build_ax_lookup_ms', 0)
+ timing_info.get('build_snapshot_lookup_ms', 0)
+ timing_info.get('construct_enhanced_tree_ms', 0)
###)
###get_dom_tree_overhead_ms = total_get_dom_tree_ms - tracked_sub_operations_ms
###if get_dom_tree_overhead_ms > 0.1:
timing_info['get_dom_tree_overhead_ms'] = get_dom_tree_overhead_ms

###return enhanced_dom_tree_node, timing_info

###@observe_debug(ignore_input=True, ignore_output=True, name='get_serialized_dom_tree')
###async def get_serialized_dom_tree(
self, previous_cached_state: SerializedDOMState | None = None
###) -> tuple[SerializedDOMState, EnhancedDOMTreeNode, dict[str, float]]:

```

```

 """Get the serialized DOM tree representation for LLM consumption.

 Returns:
 Tuple of (serialized_dom_state, enhanced_dom_tree_root, timing_info)
 """
 timing_info: dict[str, float] = {}
 start_total = time.time()

 # Use current target (None means use current)
 assert self.browser_session.agent_focus_target_id is not None

 session_id = self.browser_session.id

 # Build DOM tree (includes CDP calls for snapshot, DOM, AX tree)
 # Note: all_frames is fetched lazily inside get_dom_tree only if cross-origin iframes need it
 enhanced_dom_tree, dom_tree_timing = await self.get_dom_tree(
 target_id=self.browser_session.agent_focus_target_id,
 all_frames=None, # Lazy - will fetch if needed
)

 # Add sub-timings from DOM tree construction
 timing_info.update(dom_tree_timing)

 # Serialize DOM tree for LLM
 start_serialize = time.time()

 serialized_dom_state, serializer_timing = DOMTreeSerializer(
 enhanced_dom_tree, previous_cached_state, paint_order_filtering=self.paint_order_filtering, session_id=session_id
).serialize_accessible_elements()
 total_serialization_ms = (time.time() - start_serialize) * 1000

 # Add serializer sub-timings (convert to ms)
 for key, value in serializer_timing.items():
 timing_info[f'{key}_ms'] = value * 1000

 # Calculate untracked time in serialization
 tracked_serialization_ms = sum(value * 1000 for value in serializer_timing.values())
 serialization_overhead_ms = total_serialization_ms - tracked_serialization_ms
 if serialization_overhead_ms > 0.1: # Only log if significant
 timing_info['serialization_overhead_ms'] = serialization_overhead_ms

 # Calculate total time for get_serialized_dom_tree
 total_get_serialized_dom_tree_ms = (time.time() - start_total) * 1000
 timing_info['get_serialized_dom_tree_total_ms'] = total_get_serialized_dom_tree_ms

 # Calculate overhead in get_serialized_dom_tree (time not accounted for)
 tracked_major_operations_ms = timing_info.get('get_dom_tree_total_ms', 0) + total_serialization_ms
 get_serialized_overhead_ms = total_get_serialized_dom_tree_ms - tracked_major_operations_ms
 if get_serialized_overhead_ms > 0.1:
 timing_info['get_serialized_dom_tree_overhead_ms'] = get_serialized_overhead_ms

 return serialized_dom_state, enhanced_dom_tree, timing_info

 @staticmethod
 def detect_pagination_buttons(selector_map: dict[int, EnhancedDOMTreeNode]) -> list[dict[str, str | int]]:
 """Detect pagination buttons from the selector map.

 Args:
 selector_map: Map of element indices to EnhancedDOMTreeNode

 Returns:
 List of pagination button information dicts with:
 - button_type: 'next', 'prev', 'first', 'last', 'page_number'
 - backend_node_id: Backend node ID for clicking
 - text: Button text/label
 - selector: XPath selector
 - is_disabled: Whether the button appears disabled
 """
 pagination_buttons: list[dict[str, str | int | bool]] = []

 # Common pagination patterns to look for
 next_patterns = ['next', '>', '»', '→', 'siguiente', 'suivant', 'weiter', 'volgende']

```

```

prev_patterns = ['prev', 'previous', '<', '<<', '←', 'anterior', 'précédent', 'zurück', 'vorige']
first_patterns = ['first', '■', '«', 'primera', 'première', 'erste', 'eerste']
last_patterns = ['last', '■', '»', 'última', 'dernier', 'letzte', 'laatste']

for index, node in selector_map.items():
 # Skip non-clickable elements
 if not node.snapshot_node or not node.snapshot_node.is_clickable:
 continue

 # Get element text and attributes
 text = node.get_all_children_text().lower().strip()
 aria_label = node.attributes.get('aria-label', '').lower()
 title = node.attributes.get('title', '').lower()
 class_name = node.attributes.get('class', '').lower()
 role = node.attributes.get('role', '').lower()

 # Combine all text sources for pattern matching
 all_text = f'{text} {aria_label} {title} {class_name}'.strip()

 # Check if it's disabled
 is_disabled = (
 node.attributes.get('disabled') == 'true'
 or node.attributes.get('aria-disabled') == 'true'
 or 'disabled' in class_name
)

 button_type: str | None = None

 # Check for next button
 if any(pattern in all_text for pattern in next_patterns):
 button_type = 'next'
 # Check for previous button
 elif any(pattern in all_text for pattern in prev_patterns):
 button_type = 'prev'
 # Check for first button
 elif any(pattern in all_text for pattern in first_patterns):
 button_type = 'first'
 # Check for last button
 elif any(pattern in all_text for pattern in last_patterns):
 button_type = 'last'
 # Check for numeric page buttons (single or double digit)
 elif text.isdigit() and len(text) <= 2 and role in ['button', 'link', '']:
 button_type = 'page_number'

 if button_type:
 pagination_buttons.append(
 {
 'button_type': button_type,
 'backend_node_id': index,
 'text': node.get_all_children_text().strip() or aria_label or title,
 'selector': node.xpath,
 'is_disabled': is_disabled,
 }
)

return pagination_buttons

```

## ***.lsystem\dom\utils.py***

```

def cap_text_length(text: str, max_length: int) -> str:
 """Cap text length for display."""
 if len(text) <= max_length:
 return text
 return text[:max_length] + '...'

def generate_css_selector_for_element(enhanced_node) -> str | None:
 """Generate a CSS selector using node properties from version 0.5.0 approach."""

```

```

import re

if not enhanced_node or not hasattr(enhanced_node, 'tag_name') or not enhanced_node.tag_name:
 return None

Get base selector from tag name (simplified since we don't have xpath in EnhancedDOMTreeNode)
tag_name = enhanced_node.tag_name.lower().strip()
if not tag_name or not re.match(r'^[a-zA-Z][a-zA-Z0-9-]*$', tag_name):
 return None

css_selector = tag_name

Add ID if available (most specific)
if enhanced_node.attributes and 'id' in enhanced_node.attributes:
 element_id = enhanced_node.attributes['id']
 if element_id and element_id.strip():
 element_id = element_id.strip()
 # Validate ID contains only valid characters for # selector
 if re.match(r'^[a-zA-Z][a-zA-Z0-9-]*$', element_id):
 return f'#{element_id}'
 else:
 # For IDs with special characters ($, ., :, etc.), use attribute selector
 # Escape quotes in the ID value
 escaped_id = element_id.replace('"', '\\"')
 return f'{tag_name}[id="{escaped_id}"]'

Handle class attributes (from version 0.5.0 approach)
if enhanced_node.attributes and 'class' in enhanced_node.attributes and enhanced_node.attributes['class']:
 # Define a regex pattern for valid class names in CSS
 valid_class_name_pattern = re.compile(r'^[a-zA-Z_][a-zA-Z0-9-]*$')

 # Iterate through the class attribute values
 classes = enhanced_node.attributes['class'].split()
 for class_name in classes:
 # Skip empty class names
 if not class_name.strip():
 continue

 # Check if the class name is valid
 if valid_class_name_pattern.match(class_name):
 # Append the valid class name to the CSS selector
 css_selector += f'.{class_name}'

Expanded set of safe attributes that are stable and useful for selection (from v0.5.0)
SAFE_ATTRIBUTES = {
 # Data attributes (if they're stable in your application)
 'id',
 # Standard HTML attributes
 'name',
 'type',
 'placeholder',
 # Accessibility attributes
 'aria-label',
 'aria-labelledby',
 'aria-describedby',
 'role',
 # Common form attributes
 'for',
 'autocomplete',
 'required',
 'readonly',
 # Media attributes
 'alt',
 'title',
 'src',
 # Custom stable attributes (add any application-specific ones)
 'href',
 'target',
}

Always include dynamic attributes (include_dynamic_attributes=True equivalent)
include_dynamic_attributes = True

```

```

 if include_dynamic_attributes:
 dynamic_attributes = {
 'data-id',
 'data-qa',
 'data-cy',
 'data-testid',
 }
 SAFE_ATTRIBUTES.update(dynamic_attributes)

 # Handle other attributes (from version 0.5.0 approach)
 if enhanced_node.attributes:
 for attribute, value in enhanced_node.attributes.items():
 if attribute == 'class':
 continue

 # Skip invalid attribute names
 if not attribute.strip():
 continue

 if attribute not in SAFE_ATTRIBUTES:
 continue

 # Escape special characters in attribute names
 safe_attribute = attribute.replace(':', r'\:')

 # Handle different value cases
 if value == '':
 css_selector += f'[{safe_attribute}]'
 elif any(char in value for char in '"\'<>\n\r\t'):
 # Use contains for values with special characters
 # For newline-containing text, only use the part before the newline
 if '\n' in value:
 value = value.split('\n')[0]
 # Regex-substitute *any* whitespace with a single space, then strip.
 collapsed_value = re.sub(r'\s+', ' ', value).strip()
 # Escape embedded double-quotes.
 safe_value = collapsed_value.replace('"', '\\"')
 css_selector += f'[{safe_attribute}*="{safe_value}"]'
 else:
 css_selector += f'[{safe_attribute}="{value}"]'

 # Final validation: ensure the selector is safe and doesn't contain problematic characters
 # Note: quotes are allowed in attribute selectors like [name="value"]
 if css_selector and not any(char in css_selector for char in ['\n', '\r', '\t']):
 return css_selector

 # If we get here, the selector was problematic, return just the tag name as fallback
 return tag_name

```

## ***.lsystem\dom\views.py***

```

import hashlib
from dataclasses import asdict, dataclass, field
from enum import Enum
from typing import Any

from cdp_use.cdp.accessibility.commands import GetFullAXTreeReturns
from cdp_use.cdp.accessibility.types import AXPropertyName
from cdp_use.cdp.dom.commands import GetDocumentReturns
from cdp_use.cdp.dom.types import ShadowRootType
from cdp_use.cdp.domsnapshot.commands import CaptureSnapshotReturns
from cdp_use.cdp.target.types import SessionID, TargetID, TargetInfo
from uuid_extensions import uuid7str

from system.dom.utils import cap_text_length
from system.observability import observe_debug

Serializer types

```

```

DEFAULT_INCLUDE_ATTRIBUTES = [
 ■'title',
 ■'type',
 ■'checked',
 ■# 'class',
 ■'id',
 ■'name',
 ■'role',
 ■'value',
 ■'placeholder',
 ■'data-date-format',
 ■'alt',
 ■'aria-label',
 ■'aria-expanded',
 ■'data-state',
 ■'aria-checked',
 ■# ARIA value attributes for datetime/range inputs
 ■'aria-valuemin',
 ■'aria-valuemax',
 ■'aria-valuenow',
 ■'aria-placeholder',
 ■# Validation attributes - help agents avoid brute force attempts
 ■'pattern',
 ■'min',
 ■'max',
 ■'minlength',
 ■'maxlength',
 ■'step',
 ■'accept', # File input types (e.g., accept="image/*" or accept=".pdf")
 ■'multiple', # Whether multiple files/selections are allowed
 ■'inputmode', # Virtual keyboard hint (numeric, tel, email, url, etc.)
 ■'autocomplete', # Autocomplete behavior hint
 ■'aria-autocomplete', # ARIA autocomplete type (list, inline, both)
 ■'list', # Associated datalist element ID
 ■'data-mask', # Input mask format (e.g., phone numbers, credit cards)
 ■'data-inputmask', # Alternative input mask attribute
 ■'data-datepicker', # jQuery datepicker indicator
 ■'format', # Synthetic attribute for date/time input format (e.g., MM/dd/yyyy)
 ■'expected_format', # Synthetic attribute for explicit expected format (e.g., AngularJS datepickers)
 ■'contenteditable', # Rich text editor detection
 ■# Webkit shadow DOM identifiers
 ■'pseudo',
 ■# Accessibility properties from ax_node (ordered by importance for automation)
 ■'checked',
 ■'selected',
 ■'expanded',
 ■'pressed',
 ■'disabled',
 ■'invalid', # Current validation state from AX node
 ■'valuemin', # Min value from AX node (for datetime/range)
 ■'valuemax', # Max value from AX node (for datetime/range)
 ■'valuenow',
 ■'keyshortcuts',
 ■'haspopup',
 ■'multiselectable',
 ■# Less commonly needed (uncomment if required):
 ■# 'readonly',
 ■'required',
 ■'valuetext',
 ■'level',
 ■'busy',
 ■'live',
 ■# Accessibility name (contains text content for StaticText elements)
 ■'ax_name',
]

STATIC_ATTRIBUTES = {
 ■'class',
 ■'id',
 ■'name',
 ■'type',
 ■'placeholder',

```

```

■ 'aria-label',
■ 'title',
■# 'aria-expanded',
■ 'role',
■ 'data-testid',
■ 'data-test',
■ 'data-cy',
■ 'data-selenium',
■ 'for',
■ 'required',
■ 'disabled',
■ 'readonly',
■ 'checked',
■ 'selected',
■ 'multiple',
■ 'accept',
■ 'href',
■ 'target',
■ 'rel',
■ 'aria-describedby',
■ 'aria-labelledby',
■ 'aria-controls',
■ 'aria-owns',
■ 'aria-live',
■ 'aria-atomic',
■ 'aria-busy',
■ 'aria-disabled',
■ 'aria-hidden',
■ 'aria-pressed',
■ 'aria-autocomplete',
■ 'aria-checked',
■ 'aria-selected',
■ 'list',
■ 'tabindex',
■ 'alt',
■ 'src',
■ 'lang',
■ 'itemscope',
■ 'itemtype',
■ 'itemprop',
■# Webkit shadow DOM attributes
■ 'pseudo',
■ 'aria-valuemin',
■ 'aria-valuemax',
■ 'aria-valuenow',
■ 'aria-placeholder',
}

```

```

Class patterns that indicate dynamic/transient UI state - excluded from stable hash
DYNAMIC_CLASS_PATTERNS = frozenset(

```

```

■{
■ ■ 'focus',
■ ■ 'hover',
■ ■ 'active',
■ ■ 'selected',
■ ■ 'disabled',
■ ■ 'animation',
■ ■ 'transition',
■ ■ 'loading',
■ ■ 'open',
■ ■ 'closed',
■ ■ 'expanded',
■ ■ 'collapsed',
■ ■ 'visible',
■ ■ 'hidden',
■ ■ 'pressed',
■ ■ 'checked',
■ ■ 'highlighted',
■ ■ 'current',
■ ■ 'entering',
■ ■ 'leaving',
■}

```

)

```
class MatchLevel(Enum):
 """Element matching strictness levels for history replay."""

 EXACT = 1 # Full hash with all attributes (current behavior)
 STABLE = 2 # Hash with dynamic classes filtered out
 XPATH = 3 # XPath string comparison
 AX_NAME = 4 # Accessible name (ax_name) from accessibility tree
 ATTRIBUTE = 5 # Unique attribute match (name, id, aria-label)

def filter_dynamic_classes(class_str: str | None) -> str:
 """
 Remove dynamic state classes, keep semantic/identifying ones.
 Returns sorted classes for deterministic hashing.
 """
 if not class_str:
 return ''
 classes = class_str.split()
 stable = [c for c in classes if not any(pattern in c.lower() for pattern in DYNAMIC_CLASS_PATTERNS)]
 return ' '.join(sorted(stable))

@dataclass
class CurrentPageTargets:
 page_session: TargetInfo
 iframe_sessions: list[TargetInfo]
 """
 Iframe sessions are ALL the iframes sessions of all the pages (not just the current page)
 """

@dataclass
class TargetAllTrees:
 snapshot: CaptureSnapshotReturns
 dom_tree: GetDocumentReturns
 ax_tree: GetFullAXTreeReturns
 device_pixel_ratio: float
 cdp_timing: dict[str, float]
 js_click_listener_backend_ids: set[int] | None = None
 """Backend node IDs of elements with JS click/mouse event listeners (detected via CDP getEventListeners)"""

@dataclass(slots=True)
class PropagatingBounds:
 """Track bounds that propagate from parent elements to filter children."""

 tag: str # The tag that started propagation ('a' or 'button')
 bounds: 'DOMRect' # The bounding box
 node_id: int # Node ID for debugging
 depth: int # How deep in tree this started (for debugging)

@dataclass(slots=True)
class SimplifiedNode:
 """Simplified tree node for optimization."""

 original_node: 'EnhancedDOMTreeNode'
 children: list['SimplifiedNode']
 should_display: bool = True
 is_interactive: bool = False # True if element is in selector_map

 is_new: bool = False

 ignored_by_paint_order: bool = False # More info in dom/serializer/paint_order.py
 excluded_by_parent: bool = False # New field for bbox filtering
 is_shadow_host: bool = False # New field for shadow DOM hosts
 is_compound_component: bool = False # True for virtual components of compound controls

 def _clean_original_node_json(self, node_json: dict) -> dict:
```



```

 """Recursively remove children_nodes and shadow_roots from original_node JSON."""
 # Remove the fields we don't want in SimplifiedNode serialization
 if 'children_nodes' in node_json:
 del node_json['children_nodes']
 if 'shadow_roots' in node_json:
 del node_json['shadow_roots']

 # Clean nested content_document if it exists
 if node_json.get('content_document'):
 node_json['content_document'] = self._clean_original_node_json(node_json['content_document'])

 return node_json

 def __json__(self) -> dict:
 original_node_json = self.original_node.__json__()
 # Remove children_nodes and shadow_roots to avoid duplication with SimplifiedNode.children
 cleaned_original_node_json = self._clean_original_node_json(original_node_json)
 return {
 'should_display': self.should_display,
 'is_interactive': self.is_interactive,
 'ignored_by_paint_order': self.ignored_by_paint_order,
 'excluded_by_parent': self.excluded_by_parent,
 'original_node': cleaned_original_node_json,
 'children': [c.__json__() for c in self.children],
 }

class NodeType(int, Enum):
 """DOM node types based on the DOM specification."""

 ELEMENT_NODE = 1
 ATTRIBUTE_NODE = 2
 TEXT_NODE = 3
 CDATA_SECTION_NODE = 4
 ENTITY_REFERENCE_NODE = 5
 ENTITY_NODE = 6
 PROCESSING_INSTRUCTION_NODE = 7
 COMMENT_NODE = 8
 DOCUMENT_NODE = 9
 DOCUMENT_TYPE_NODE = 10
 DOCUMENT_FRAGMENT_NODE = 11
 NOTATION_NODE = 12

 @dataclass(slots=True)
 class DOMRect:
 x: float
 y: float
 width: float
 height: float

 def to_dict(self) -> dict[str, Any]:
 return {
 'x': self.x,
 'y': self.y,
 'width': self.width,
 'height': self.height,
 }

 def __json__(self) -> dict:
 return self.to_dict()

 @dataclass(slots=True)
 class EnhancedAXProperty:
 """we don't need `sources` and `related_nodes` for now (not sure how to use them)

 TODO: there is probably some way to determine whether it has a value or related nodes or not, but for now
 """
 name: AXPropertyName
 value: str | bool | None

```

```
■# related_nodes: list[EnhancedAXRelatedNode] | None
```

```
@dataclass(slots=True)
class EnhancedAXNode:
 ■ax_node_id: str
 ■"""Not to be confused the DOM node_id. Only useful for AX node tree"""
 ■ignored: bool
 ■# we don't need ignored_reasons as we anyway ignore the node otherwise
 ■role: str | None
 ■name: str | None
 ■description: str | None

 ■properties: list[EnhancedAXProperty] | None
 ■child_ids: list[str] | None
```

```
@dataclass(slots=True)
class EnhancedSnapshotNode:
 ■"""Snapshot data extracted from DOMSnapshot for enhanced functionality."""

 ■is_clickable: bool | None
 ■cursor_style: str | None
 ■bounds: DOMRect | None
 ■"""
 ■Document coordinates (origin = top-left of the page, ignores current scroll).
 ■Equivalent JS API: layoutNode.boundingBox in the older API.
 ■Typical use: Quick hit-test that doesn't care about scroll position.
 ■"""

 ■client_rects: DOMRect | None
 ■"""
 ■Viewport coordinates (origin = top-left of the visible scrollport).
 ■Equivalent JS API: element.getClientRects() / getBoundingClientRect().
 ■Typical use: Pixel-perfect hit-testing on screen, taking current scroll into account.
 ■"""

 ■scroll_rects: DOMRect | None
 ■"""
 ■Scrollable area of the element.
 ■"""

 ■computed_styles: dict[str, str] | None
 ■"""Computed styles from the layout tree"""
 ■paint_order: int | None
 ■"""Paint order from the layout tree"""
 ■stacking_contexts: int | None
 ■"""Stacking contexts from the layout tree"""
```

```
@dataclass(slots=True)
class SuperSelector:
■node_id: int
■backend_node_id: int
■frame_id: str | None
■target_id: TargetID

■node_type: NodeType
■node_name: str

■# is_visible: bool | None
■# is_scrollable: bool | None

■element_index: int | None
```

```
@dataclass(slots=True)
class EnhancedDOMTreeNode:
 ■"""
 ■Enhanced DOM tree node that contains information from AX, DOM, and Snapshot trees. It's mostly based on
 ■@dev when serializing check if the value is a valid value first!
```

```

■ Learn more about the fields:
■ - (DOM node) https://chromedevtools.github.io/devtools-protocol/tot/DOM/#type-BackendNode
■ - (AX node) https://chromedevtools.github.io/devtools-protocol/tot/Accessibility/#type-AXNode
■ - (Snapshot node) https://chromedevtools.github.io/devtools-protocol/tot/DOMSnapshot/#type-DOMNode
■ ""

■ # region - DOM Node data

■ node_id: int
■ backend_node_id: int

■ node_type: NodeType
■ ""Node types, defined in `NodeType` enum.""
■ node_name: str
■ ""Only applicable for `NodeType.ELEMENT_NODE`""
■ node_value: str
■ ""this is where the value from `NodeType.TEXT_NODE` is stored usually""
■ attributes: dict[str, str]
■ ""slightly changed from the original attributes to be more readable""
■ is_scrollable: bool | None
■ ""
■ Whether the node is scrollable.
■ ""
■ is_visible: bool | None
■ ""
■ Whether the node is visible according to the upper most frame node.
■ ""

■ absolute_position: DOMRect | None
■ ""
■ Absolute position of the node in the document according to the top-left of the page.
■ ""

■ # frames
■ target_id: TargetID
■ frame_id: str | None
■ session_id: SessionID | None
■ content_document: 'EnhancedDOMTreeNode | None'
■ ""
■ Content document is the document inside a new iframe.
■ ""
■ # Shadow DOM
■ shadow_root_type: ShadowRootType | None
■ shadow_roots: list['EnhancedDOMTreeNode'] | None
■ ""
■ Shadow roots are the shadow DOMs of the element.
■ ""

■ # Navigation
■ parent_node: 'EnhancedDOMTreeNode | None'
■ children_nodes: list['EnhancedDOMTreeNode'] | None

■ # endregion - DOM Node data

■ # region - AX Node data
■ ax_node: EnhancedAXNode | None

■ # endregion - AX Node data

■ # region - Snapshot Node data
■ snapshot_node: EnhancedSnapshotNode | None

■ # endregion - Snapshot Node data

■ # Compound control child components information
■ _compound_children: list[dict[str, Any]] = field(default_factory=list)

■ has_js_click_listener: bool = False
■ ""
■ Whether this element has JS click/mouse event listeners attached (detected via CDP getEventListeners)
■ Used to identify clicks that don't use native interactive HTML tags
■ ""

```

```

■hidden_elements_info: list[dict[str, Any]] = field(default_factory=list)
■"""
■Details of interactive elements hidden due to viewport threshold (for iframes).
■Each dict contains: tag, text, pages (scroll distance in viewport pages).
■Used to show specific element info in the LLM representation.
■"""

■has_hidden_content: bool = False
■"""
■Whether this iframe has hidden non-interactive content below the viewport threshold.
■"""

■uuid: str = field(default_factory=uuid7str)

■@property
■def parent(self) -> 'EnhancedDOMTreeNode | None':
■ return self.parent_node

■@property
■def children(self) -> list['EnhancedDOMTreeNode']:
■ return self.children_nodes or []

■@property
■def children_and_shadow_roots(self) -> list['EnhancedDOMTreeNode']:
■ """
■ Returns all children nodes, including shadow roots
■ """
■ # IMPORTANT: Make a copy to avoid mutating the original children_nodes list!
■ children = list(self.children_nodes) if self.children_nodes else []
■ if self.shadow_roots:
■ children.extend(self.shadow_roots)
■ return children

■@property
■def tag_name(self) -> str:
■ return self.node_name.lower()

■@property
■def xpath(self) -> str:
■ """Generate XPath for this DOM node, stopping at shadow boundaries or iframes."""
■ segments = []
■ current_element = self

■ while current_element and (
■ current_element.node_type == NodeType.ELEMENT_NODE or current_element.node_type == NodeType.DOCUMENT
■):
■ # just pass through shadow roots
■ if current_element.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
■ current_element = current_element.parent_node
■ continue

■ # stop ONLY if we hit iframe
■ if current_element.parent_node and current_element.parent_node.node_name.lower() == 'iframe':
■ break

■ position = self._get_element_position(current_element)
■ tag_name = current_element.node_name.lower()
■ xpath_index = f'[{position}]' if position > 0 else ''
■ segments.insert(0, f'{tag_name}{xpath_index}')

■ current_element = current_element.parent_node

■ return '/'.join(segments)

■def _get_element_position(self, element: 'EnhancedDOMTreeNode') -> int:
■ """Get the position of an element among its siblings with the same tag name.
■ Returns 0 if it's the only element of its type, otherwise returns 1-based index."""
■ if not element.parent_node or not element.parent_node.children_nodes:
■ return 0

■ same_tag_siblings = [
■ child

```

```

 for child in element.parent_node.children_nodes
 if child.node_type == NodeType.ELEMENT_NODE and child.node_name.lower() == element.node_name.lower()
]

 if len(same_tag_siblings) <= 1:
 return 0 # No index needed if it's the only one

 try:
 # XPath is 1-indexed
 position = same_tag_siblings.index(element) + 1
 return position
 except ValueError:
 return 0

 def __json__(self) -> dict:
 """Serializes the node and its descendants to a dictionary, omitting parent references."""
 return {
 'node_id': self.node_id,
 'backend_node_id': self.backend_node_id,
 'node_type': self.node_type.name,
 'node_name': self.node_name,
 'node_value': self.node_value,
 'is_visible': self.is_visible,
 'attributes': self.attributes,
 'is_scrollable': self.is_scrollable,
 'session_id': self.session_id,
 'target_id': self.target_id,
 'frame_id': self.frame_id,
 'content_document': self.content_document.__json__() if self.content_document else None,
 'shadow_root_type': self.shadow_root_type,
 'ax_node': asdict(self.ax_node) if self.ax_node else None,
 'snapshot_node': asdict(self.snapshot_node) if self.snapshot_node else None,
 # these two in the end, so it's easier to read json
 'shadow_roots': [r.__json__() for r in self.shadow_roots] if self.shadow_roots else [],
 'children_nodes': [c.__json__() for c in self.children_nodes] if self.children_nodes else [],
 }

 def get_all_children_text(self, max_depth: int = -1) -> str:
 text_parts = []

 def collect_text(node: EnhancedDOMTreeNode, current_depth: int) -> None:
 if max_depth != -1 and current_depth > max_depth:
 return

 # Skip this branch if we hit a highlighted element (except for the current node)
 # TODO: think whether it makes sense to add text until the next clickable element or everything from
 # if node.node_type == NodeType.ELEMENT_NODE
 # if isinstance(node, DOMElementNode) and node != self and node.highlight_index is not None:
 # return

 if node.node_type == NodeType.TEXT_NODE:
 text_parts.append(node.node_value)
 elif node.node_type == NodeType.ELEMENT_NODE:
 for child in node.children:
 collect_text(child, current_depth + 1)

 collect_text(self, 0)
 return '\n'.join(text_parts).strip()

 def __repr__(self) -> str:
 """
 @DEV ! don't display this to the LLM, it's SUPER long
 """
 attributes = ', '.join([f'{k}={v}' for k, v in self.attributes.items()])
 is_scrollable = getattr(self, 'is_scrollable', False)
 num_children = len(self.children_nodes or [])
 return (
 f'<{self.tag_name} {attributes} is_scrollable={is_scrollable} '
 f'num_children={num_children} >{self.node_value}</{self.tag_name}>'
)

 def llm_representation(self, max_text_length: int = 100) -> str:

```

```

"""
Token friendly representation of the node, used in the LLM
"""

return f'<{self.tag_name}>{cap_text_length(self.get_all_children_text(), max_text_length) or ""}'

def get_meaningful_text_for_llm(self) -> str:
 """
 Get the meaningful text content that the LLM actually sees for this element.
 This matches exactly what goes into the DOMTreeSerializer output.
 """
 meaningful_text = ''
 if hasattr(self, 'attributes') and self.attributes:
 # Priority order: value, aria-label, title, placeholder, alt, text content
 for attr in ['value', 'aria-label', 'title', 'placeholder', 'alt']:
 if attr in self.attributes and self.attributes[attr]:
 meaningful_text = self.attributes[attr]
 break

 # Fallback to text content if no meaningful attributes
 if not meaningful_text:
 meaningful_text = self.get_all_children_text()

 return meaningful_text.strip()

@property
def is_actually_scrollable(self) -> bool:
 """
 Enhanced scroll detection that combines CDP detection with CSS analysis.

 This detects scrollable elements that Chrome's CDP might miss, which is common
 in iframes and dynamically sized containers.
 """
 # First check if CDP already detected it as scrollable
 if self.is_scrollable:
 return True

 # Enhanced detection for elements CDP missed
 if not self.snapshot_node:
 return False

 # Check scroll vs client rects - this is the most reliable indicator
 scroll_rects = self.snapshot_node.scrollRects
 client_rects = self.snapshot_node.clientRects

 if scroll_rects and client_rects:
 # Content is larger than visible area = scrollable
 has_vertical_scroll = scroll_rects.height > client_rects.height + 1 # +1 for rounding
 has_horizontal_scroll = scroll_rects.width > client_rects.width + 1

 if has_vertical_scroll or has_horizontal_scroll:
 # Also check CSS to make sure scrolling is allowed
 if self.snapshot_node.computed_styles:
 styles = self.snapshot_node.computed_styles

 overflow = styles.get('overflow', 'visible').lower()
 overflow_x = styles.get('overflow-x', overflow).lower()
 overflow_y = styles.get('overflow-y', overflow).lower()

 # Only allow scrolling if overflow is explicitly set to auto, scroll, or overlay
 # Do NOT consider 'visible' overflow as scrollable - this was causing the issue
 allows_scroll = (
 overflow in ['auto', 'scroll', 'overlay']
 or overflow_x in ['auto', 'scroll', 'overlay']
 or overflow_y in ['auto', 'scroll', 'overlay']
)

 return allows_scroll
 else:
 # No CSS info, but content overflows - be more conservative
 # Only consider it scrollable if it's a common scrollable container element
 scrollable_tags = {'div', 'main', 'section', 'article', 'aside', 'body', 'html'}

```

```

 return self.tag_name.lower() in scrollable_tags

 return False

 @property
 def should_show_scroll_info(self) -> bool:
 """
 Simple check: show scroll info only if this element is scrollable
 and doesn't have a scrollable parent (to avoid nested scroll spam).

 Special case for iframes: Always show scroll info since Chrome might not
 always detect iframe scrollability correctly (scrollHeight: 0 issue).
 """
 # Special case: Always show scroll info for iframe elements
 # Even if not detected as scrollable, they might have scrollable content
 if self.tag_name.lower() == 'iframe':
 return True

 # Must be scrollable first for non-iframe elements
 if not (self.is_scrollable or self.is_actually_scrollable):
 return False

 # Always show for iframe content documents (body/html)
 if self.tag_name.lower() in {'body', 'html'}:
 return True

 # Don't show if parent is already scrollable (avoid nested spam)
 if self.parent_node and (self.parent_node.is_scrollable or self.parent_node.is_actually_scrollable):
 return False

 return True

 def _find_html_in_content_document(self) -> 'EnhancedDOMTreeNode | None':
 """Find HTML element in iframe content document."""
 if not self.content_document:
 return None

 # Check if content document itself is HTML
 if self.content_document.tag_name.lower() == 'html':
 return self.content_document

 # Look through children for HTML element
 if self.content_document.children_nodes:
 for child in self.content_document.children_nodes:
 if child.tag_name.lower() == 'html':
 return child

 return None

 @property
 def scroll_info(self) -> dict[str, Any] | None:
 """Calculate scroll information for this element if it's scrollable."""
 if not self.is_actually_scrollable or not self.snapshot_node:
 return None

 # Get scroll and client rects from snapshot data
 scroll_rects = self.snapshot_node.scrollRects
 client_rects = self.snapshot_node.clientRects
 bounds = self.snapshot_node.bounds

 if not scroll_rects or not client_rects:
 return None

 # Calculate scroll position and percentages
 scroll_top = scroll_rects.y
 scroll_left = scroll_rects.x

 # Total scrollable height and width
 scrollable_height = scroll_rects.height
 scrollable_width = scroll_rects.width

 # Visible (client) dimensions

```

```

■■visible_height = client_rects.height
■■visible_width = client_rects.width

■■# Calculate how much content is above/below/left/right of current view
■■content_above = max(0, scroll_top)
■■content_below = max(0, scrollable_height - visible_height - scroll_top)
■■content_left = max(0, scroll_left)
■■content_right = max(0, scrollable_width - visible_width - scroll_left)

■■# Calculate scroll percentages
■■vertical_scroll_percentage = 0
■■horizontal_scroll_percentage = 0

■■if scrollable_height > visible_height:
■■■max_scroll_top = scrollable_height - visible_height
■■■vertical_scroll_percentage = (scroll_top / max_scroll_top) * 100 if max_scroll_top > 0 else 0

■■if scrollable_width > visible_width:
■■■max_scroll_left = scrollable_width - visible_width
■■■horizontal_scroll_percentage = (scroll_left / max_scroll_left) * 100 if max_scroll_left > 0 else 0

■■# Calculate pages equivalent (using visible height as page unit)
■■pages_above = content_above / visible_height if visible_height > 0 else 0
■■pages_below = content_below / visible_height if visible_height > 0 else 0
■■total_pages = scrollable_height / visible_height if visible_height > 0 else 1

■■return {
■■■'scroll_top': scroll_top,
■■■'scroll_left': scroll_left,
■■■'scrollable_height': scrollable_height,
■■■'scrollable_width': scrollable_width,
■■■'visible_height': visible_height,
■■■'visible_width': visible_width,
■■■'content_above': content_above,
■■■'content_below': content_below,
■■■'content_left': content_left,
■■■'content_right': content_right,
■■■'vertical_scroll_percentage': round(vertical_scroll_percentage, 1),
■■■'horizontal_scroll_percentage': round(horizontal_scroll_percentage, 1),
■■■'pages_above': round(pages_above, 1),
■■■'pages_below': round(pages_below, 1),
■■■'total_pages': round(total_pages, 1),
■■■'can_scroll_up': content_above > 0,
■■■'can_scroll_down': content_below > 0,
■■■'can_scroll_left': content_left > 0,
■■■'can_scroll_right': content_right > 0,
■■}

■■def get_scroll_info_text(self) -> str:
■■■"""Get human-readable scroll information text for this element."""
■■■# Special case for iframes: check content document for scroll info
■■■if self.tag_name.lower() == 'iframe':
■■■# Try to get scroll info from the HTML document inside the iframe
■■■if self.content_document:
■■■# Look for HTML element in content document
■■■html_element = self._find_html_in_content_document()
■■■if html_element and html_element.scroll_info:
■■■info = html_element.scroll_info
■■■# Provide minimal but useful scroll info
■■■pages_below = info.get('pages_below', 0)
■■■pages_above = info.get('pages_above', 0)
■■■v_pct = int(info.get('vertical_scroll_percentage', 0))

■■■if pages_below > 0 or pages_above > 0:
■■■return f'scroll: {pages_above:.1f}↑ {pages_below:.1f}↓ {v_pct}%'

■■return 'scroll'

■■scroll_info = self.scroll_info
■■if not scroll_info:
■■return ''
■■parts = []

```



```

 ## Vertical scroll info (concise format)
 if scroll_info['scrollable_height'] > scroll_info['visible_height']:
 parts.append(f'{scroll_info["pages_above"]:.1f} pages above, {scroll_info["pages_below"]:.1f} pages

 ## Horizontal scroll info (concise format)
 if scroll_info['scrollable_width'] > scroll_info['visible_width']:
 parts.append(f'horizontal {scroll_info["horizontal_scroll_percentage"]:.0f}%')

 return ' '.join(parts)

@property
def element_hash(self) -> int:
 return hash(self)

def compute_stable_hash(self) -> int:
 """
 Compute hash with dynamic classes filtered out.
 More stable across sessions than element_hash since it excludes
 transient CSS state classes like focus, hover, animation, etc.
 """
 parent_branch_path = self._get_parent_branch_path()
 parent_branch_path_string = '/'.join(parent_branch_path)

 ## Filter dynamic classes before building attributes string
 filtered_attrs: dict[str, str] = {}
 for k, v in self.attributes.items():
 if k not in STATIC_ATTRIBUTES:
 continue
 if k == 'class':
 v = filter_dynamic_classes(v)
 if not v: # Skip empty class after filtering
 continue
 filtered_attrs[k] = v

 attributes_string = ''.join(f'{k}={v}' for k, v in sorted(filtered_attrs.items()))

 ax_name = ''
 if self.ax_node and self.ax_node.name:
 ax_name = f'|ax_name={self.ax_node.name}'

 combined_string = f'{parent_branch_path_string}|{attributes_string}{ax_name}'
 hash_hex = hashlib.sha256(combined_string.encode()).hexdigest()
 return int(hash_hex[:16], 16)

def __str__(self) -> str:
 return f'<{self.tag_name}>#{self.frame_id[-4:] if self.frame_id else "?"}:{self.backend_node_id}'

def __hash__(self) -> int:
 """
 Hash the element based on its parent branch path, attributes, and accessibility name.

 TODO: migrate this to use only backendNodeId + current SessionId
 """

 ## Get parent branch path
 parent_branch_path = self._get_parent_branch_path()
 parent_branch_path_string = '/'.join(parent_branch_path)

 attributes_string = ''.join(
 f'{k}={v}' for k, v in sorted((k, v) for k, v in self.attributes.items() if k in STATIC_ATTRIBUTES)
)

 ## Include accessibility name (ax_name) if available - this helps distinguish
 ## elements that have identical structure and attributes but different visible text
 ax_name = ''
 if self.ax_node and self.ax_node.name:
 ax_name = f'|ax_name={self.ax_node.name}'

 ## Combine all for final hash
 combined_string = f'{parent_branch_path_string}|{attributes_string}{ax_name}'
 element_hash = hashlib.sha256(combined_string.encode()).hexdigest()
 ## Convert to int for __hash__ return type - use first 16 chars and convert from hex to int

```

```

 return int(element_hash[:16], 16)

 def parent_branch_hash(self) -> int:
 """
 Hash the element based on its parent branch path and attributes.
 """
 parent_branch_path = self._get_parent_branch_path()
 parent_branch_path_string = '/'.join(parent_branch_path)
 element_hash = hashlib.sha256(parent_branch_path_string.encode()).hexdigest()

 return int(element_hash[:16], 16)

 def _get_parent_branch_path(self) -> list[str]:
 """Get the parent branch path as a list of tag names from root to current element."""
 parents: list['EnhancedDOMTreeNode'] = []
 current_element: 'EnhancedDOMTreeNode | None' = self

 while current_element is not None:
 if current_element.node_type == NodeType.ELEMENT_NODE:
 parents.append(current_element)
 current_element = current_element.parent_node

 parents.reverse()
 return [parent.tag_name for parent in parents]

DOMSelectorMap = dict[int, EnhancedDOMTreeNode]

@dataclass(slots=True)
class MarkdownChunk:
 """A structure-aware chunk of markdown content."""

 content: str
 chunk_index: int
 total_chunks: int
 char_offset_start: int # in original content
 char_offset_end: int # in original content
 overlap_prefix: str # context from prev chunk (e.g. table headers)
 has_more: bool

@dataclass
class SerializedDOMState:
 _root: SimplifiedNode | None
 """Not meant to be used directly, use `llm_representation` instead"""

 selector_map: DOMSelectorMap

 @observe_debug(ignore_input=True, ignore_output=True, name='llm_representation')
 def llm_representation(
 self,
 include_attributes: list[str] | None = None,
) -> str:
 """Kinda ugly, but leaving this as an internal method because include_attributes are a parameter on the
 from system.dom.serializer.serializer import DOMTreeSerializer

 if not self._root:
 return 'Empty DOM tree (you might have to wait for the page to load)'

 include_attributes = include_attributes or DEFAULT_INCLUDE_ATTRIBUTES

 return DOMTreeSerializer.serialize_tree(self._root, include_attributes)

 @observe_debug(ignore_input=True, ignore_output=True, name='eval_representation')
 def eval_representation(
 self,
 include_attributes: list[str] | None = None,
) -> str:
 """
 Evaluation-focused DOM representation without interactive indexes.
 This serializer is designed for evaluation/judge contexts where:

```

```

 ■- No interactive indexes are needed (we're not clicking)
 ■- Full HTML structure should be preserved for context
 ■- More attribute information is helpful
 ■- Text content is important for understanding page structure
 """
 from system.dom.serializer.eval_serializer import DOMEvalSerializer

 if not self._root:
 return 'Empty DOM tree (you might have to wait for the page to load)'

 include_attributes = include_attributes or DEFAULT_INCLUDE_ATTRIBUTES

 return DOMEvalSerializer.serialize_tree(self._root, include_attributes)

@dataclass
class DOMInteractedElement:
 """
 DOMInteractedElement is a class that represents a DOM element that has been interacted with.
 It is used to store the DOM element that has been interacted with and to store the DOM element that has
 """
 TODO: this is a bit of a hack, we should probably have a better way to do this
 """

 node_id: int
 backend_node_id: int
 frame_id: str | None

 node_type: NodeType
 node_value: str
 node_name: str
 attributes: dict[str, str] | None

 bounds: DOMRect | None

 x_path: str

 element_hash: int

 # Stable hash with dynamic classes filtered - computed at save time for consistent matching
 stable_hash: int | None = None

 # Accessibility name (visible text) - used for fallback matching when hash/xpath fail
 ax_name: str | None = None

 def to_dict(self) -> dict[str, Any]:
 return {
 'node_id': self.node_id,
 'backend_node_id': self.backend_node_id,
 'frame_id': self.frame_id,
 'node_type': self.node_type.value,
 'node_value': self.node_value,
 'node_name': self.node_name,
 'attributes': self.attributes,
 'x_path': self.x_path,
 'element_hash': self.element_hash,
 'stable_hash': self.stable_hash,
 'bounds': self.bounds.to_dict() if self.bounds else None,
 'ax_name': self.ax_name,
 }

 @classmethod
 def load_from_enhanced_dom_tree(cls, enhanced_dom_tree: EnhancedDOMTreeNode) -> 'DOMInteractedElement':
 # Extract accessibility name if available
 ax_name = None
 if enhanced_dom_tree.ax_node and enhanced_dom_tree.ax_node.name:
 ax_name = enhanced_dom_tree.ax_node.name

 return cls(
 node_id=enhanced_dom_tree.node_id,
 backend_node_id=enhanced_dom_tree.backend_node_id,
 frame_id=enhanced_dom_tree.frame_id,

```

```

■■■■node_type=enhanced_dom_tree.node_type,
■■■■node_value=enhanced_dom_tree.node_value,
■■■■node_name=enhanced_dom_tree.node_name,
■■■■attributes=enhanced_dom_tree.attributes,
■■■■bounds=enhanced_dom_tree.snapshot_node.bounds if enhanced_dom_tree.snapshot_node else None,
■■■■x_path=enhanced_dom_tree.xpath,
■■■■element_hash=hash(enhanced_dom_tree),
■■■■stable_hash=enhanced_dom_tree.compute_stable_hash(), # Compute from source for single source of truth
■■■■ax_name=ax_name,
■■■)

```

## ***.\system\dom\playground\extraction.py***

```

import asyncio
import json
import os
import time

import anyio
import pyperclip
import tiktoken

from system.agent.prompts import AgentMessagePrompt
from system.browser import BrowserProfile, BrowserSession
from system.browser.events import ClickElementEvent, TypeTextEvent
from system.browser.profile import ViewportSize
from system.dom.service import DomService
from system.dom.views import DEFAULT_INCLUDE_ATTRIBUTES
from system.filesystem.file_system import FileSystem

TIMEOUT = 60

async def test_focus_vs_all_elements():
 browser_session = BrowserSession(
 browser_profile=BrowserProfile(
 # executable_path='/Applications/Google Chrome.app/Contents/MacOS/Google Chrome',
 window_size=ViewportSize(width=1100, height=1000),
 disable_security=False,
 wait_for_network_idle_page_load_time=1,
 headless=False,
 args=['--incognito'],
 paint_order_filtering=True,
),
)

 ## 10 Sample websites with various interactive elements
 sample_websites = [
 'https://browser-use.github.io/stress-tests/challenges/iframe-inception-level2.html',
 'https://www.google.com/travel/flights',
 'https://v0-simple-ui-test-site.vercel.app',
 'https://browser-use.github.io/stress-tests/challenges/iframe-inception-level1.html',
 'https://browser-use.github.io/stress-tests/challenges/angular-form.html',
 'https://www.google.com/travel/flights',
 'https://www.amazon.com/s?k=laptop',
 'https://github.com/trending',
 'https://www.reddit.com',
 'https://www.ycombinator.com/companies',
 'https://www.kayak.com/flights',
 'https://www.booking.com',
 'https://www.airbnb.com',
 'https://www.linkedin.com/jobs',
 'https://stackoverflow.com/questions',
]

 ## 5 Difficult websites with complex elements (iframes, canvas, dropdowns, etc.)
 difficult_websites = [
 'https://www.w3schools.com/html/tryit.asp?filename=tryhtml_iframe', # Nested iframes
]

```

```

 'https://semantic-ui.com/modules/dropdown.html', # Complex dropdowns
 'https://www.dezlearn.com/nested-iframes-example/', # Cross-origin nested iframes
 'https://codepen.io/towc/pen/mJzOWJ', # Canvas elements with interactions
 'https://jqueryui.com/accordion/', # Complex accordion/dropdown widgets
 'https://v0-simple-landing-page-seven-xi.vercel.app/', # Simple landing page with iframe
 'https://www.unesco.org/en',
]

Descriptions for difficult websites
difficult_descriptions = {
 'https://www.w3schools.com/html/tryit.asp?filename=tryhtml_iframe': '■ NESTED IFRAMES: Multiple iframes',
 'https://semantic-ui.com/modules/dropdown.html': '■ COMPLEX DROPDOWNS: Custom dropdown components',
 'https://www.dezlearn.com/nested-iframes-example/': '■ CROSS-ORIGIN IFRAMES: Different domain iframes',
 'https://codepen.io/towc/pen/mJzOWJ': '■ CANVAS ELEMENTS: Interactive canvas graphics',
 'https://jqueryui.com/accordion/': '■ ACCORDION WIDGETS: Collapsible content sections',
}

websites = sample_websites + difficult_websites
current_website_index = 0

def get_website_list_for_prompt() -> str:
 """Get a compact website list for the input prompt."""
 lines = []
 lines.append('■ Websites:')

 ## Sample websites (1-10)
 for i, site in enumerate(sample_websites, 1):
 current_marker = ' ←' if (i - 1) == current_website_index else ''
 domain = site.replace('https://', '').split('/')[0]
 lines.append(f' {i:2d}. {domain[:15]:<15}{current_marker}')

 ## Difficult websites (11-15)
 for i, site in enumerate(difficult_websites, len(sample_websites) + 1):
 current_marker = ' ←' if (i - 1) == current_website_index else ''
 domain = site.replace('https://', '').split('/')[0]
 desc = difficult_descriptions.get(site, '')
 challenge = desc.split(': ')[1][:15] if ':' in desc else ''
 lines.append(f' {i:2d}. {domain[:15]:<15} ({challenge}){current_marker}')

 return '\n'.join(lines)

await browser_session.start()

Show startup info
print('\n■ BROWSER-USE DOM EXTRACTION TESTER')
print(f'■ {len(websites)} websites total: {len(sample_websites)} standard + {len(difficult_websites)} difficult')
print('■ Controls: Type 1-15 to jump | Enter to re-run | "n" next | "q" quit')
print('■ Outputs: tmp/user_message.txt & tmp/element_tree.json\n')

dom_service = DomService(browser_session)

while True:
 ## Cycle through websites
 if current_website_index >= len(websites):
 current_website_index = 0
 print('Cycled back to first website!')

 website = websites[current_website_index]
 ## sleep 2
 await browser_session._cdp_navigate(website)
 await asyncio.sleep(1)

 last_clicked_index = None # Track the index for text input
 while True:
 try:
 ## all_elements_state = await dom_service.get_serialized_dom_tree()

 website_type = 'DIFFICULT' if website in difficult_websites else 'SAMPLE'
 print(f'\n{" " * 60}')
 print(f'[{current_website_index + 1}/{len(websites)}] [{website_type}] Testing: {website}')
 if website in difficult_descriptions:
 print(f'{difficult_descriptions[website]}')

```

```

#####print(f'{"=" * 60}')

Get/refresh the state (includes removing old highlights)
#####print('\nGetting page state...')

#####start_time = time.time()
#####all_elements_state = await browser_session.get_browser_state_summary(True)
#####end_time = time.time()
#####get_state_time = end_time - start_time
#####print(f'get_state_summary took {get_state_time:.2f} seconds')

Get detailed timing info from DOM service
#####print('\nGetting detailed DOM timing...')
#####serialized_state, _, timing_info = await dom_service.get_serialized_dom_tree()

Combine all timing info
#####all_timing = {'get_state_summary_total': get_state_time, **timing_info}

#####selector_map = all_elements_state.dom_state.selector_map
#####total_elements = len(selector_map.keys())
#####print(f'Total number of elements: {total_elements}')

print(all_elements_state.element_tree.clickable_elements_to_string())
#####prompt = AgentMessagePrompt(
browser_state_summary=all_elements_state,
file_system=FileSystem(base_dir='./tmp'),
include_attributes=DEFAULT_INCLUDE_ATTRIBUTES,
step_info=None,
#####)
Write the user message to a file for analysis
#####user_message = prompt.get_user_message(use_vision=False).text

clickable_elements_str = all_elements_state.element_tree.clickable_elements_to_string()

#####text_to_save = user_message

#####os.makedirs('./tmp', exist_ok=True)
#####async with await anyio.open_file('./tmp/user_message.txt', 'w', encoding='utf-8') as f:
await f.write(text_to_save)

save pure clickable elements to a file
#####if all_elements_state.dom_state._root:
async with await anyio.open_file('./tmp/simplified_element_tree.json', 'w', encoding='utf-8') as f:
await f.write(json.dumps(all_elements_state.dom_state._root.__json__(), indent=2))

async with await anyio.open_file('./tmp/original_element_tree.json', 'w', encoding='utf-8') as f:
await f.write(json.dumps(all_elements_state.dom_state._root.original_node.__json__(), indent=2))

copy the user message to the clipboard
pyperclip.copy(text_to_save)

#####encoding = tiktoken.encoding_for_model('gpt-4.1-mini')
#####token_count = len(encoding.encode(text_to_save))
#####print(f'Token count: {token_count}')

#####print('User message written to ./tmp/user_message.txt')
#####print('Element tree written to ./tmp/simplified_element_tree.json')
#####print('Original element tree written to ./tmp/original_element_tree.json')

Save timing information
#####timing_text = '■ DOM EXTRACTION PERFORMANCE ANALYSIS\n'
#####timing_text += f'{"=" * 50}\n\n'
#####timing_text += f'■ Website: {website}\n'
#####timing_text += f'■ Total Elements: {total_elements}\n'
#####timing_text += f'■ Token Count: {token_count}\n\n'

#####timing_text += '■ TIMING BREAKDOWN:\n'
#####timing_text += f'{"■" * 30}\n'
#####for key, value in all_timing.items():
timing_text += f'{key:<35}: {value * 1000:>8.2f} ms\n'

Calculate percentages

```

```

#####total_time = all_timing.get('get_state_summary_total', 0)
#####if total_time > 0 and total_elements > 0:
timing_text += '\n■ PERCENTAGE BREAKDOWN:\n'
timing_text += f'{"■" * 30}\n'
for key, value in all_timing.items():
if key != 'get_state_summary_total':
percentage = (value / total_time) * 100
timing_text += f'{key:<35}: {percentage:>7.1f}%\n'

#####timing_text += '\n■ CLICKABLE DETECTION ANALYSIS:\n'
#####timing_text += f'{"■" * 35}\n'
#####clickable_time = all_timing.get('clickable_detection_time', 0)
#####if clickable_time > 0 and total_elements > 0:
avg_per_element = (clickable_time / total_elements) * 1000000 # microseconds
timing_text += f'Total clickable detection time: {clickable_time * 1000:.2f} ms\n'
timing_text += f'Average per element: {avg_per_element:.2f} μs\n'
timing_text += f'Clickable detection calls: ~{total_elements} (approx)\n'

#####async with await anyio.open_file('./tmp/timing_analysis.txt', 'w', encoding='utf-8') as f:
await f.write(timing_text)

#####print('Timing analysis written to ./tmp/timing_analysis.txt')

also save all_elements_state.element_tree.clickable_elements_to_string() to a file
with open('./tmp/clickable_elements.json', 'w', encoding='utf-8') as f:
f.write(json.dumps(all_elements_state.element_tree.__json__(), indent=2))
print('Clickable elements written to ./tmp/clickable_elements.json')

#####website_list = get_website_list_for_prompt()
#####answer = input(
"■ Enter: element index | 'index' click (clickable) | 'index,text' input | 'c,index' copy | Enter\n#####)

#####if answer.lower() == 'q':
return # Exit completely
#####elif answer.lower() == 'n':
print('Moving to next website...')
current_website_index += 1
break # Break inner loop to go to next website
#####elif answer.strip() == '':
print('Re-running extraction on current page state...')
continue # Continue inner loop to re-extract DOM without reloading page
#####elif answer.strip().isdigit():
Click element format: index
try:
clicked_index = int(answer)
if clicked_index in selector_map:
element_node = selector_map[clicked_index]
print(f'Clicking element {clicked_index}: {element_node.tag_name}')
event = browser_session.event_bus.dispatch(ClickElementEvent(node=element_node))
await event
print('Click successful.')
except ValueError:
print(f"Invalid input: '{answer}'. Enter an index, 'index,text', 'c,index', or 'q'.")
continue

#####try:
if answer.lower().startswith('c,'):
Copy element JSON format: c,index
parts = answer.split(',', 1)
if len(parts) == 2:
try:
target_index = int(parts[1].strip())
if target_index in selector_map:
element_node = selector_map[target_index]
element_json = json.dumps(element_node.__json__(), indent=2, default=str)
pyperclip.copy(element_json)
print(f'Copied element {target_index} JSON to clipboard: {element_node.tag_name}')
else:
print(f'Invalid index: {target_index}')
except ValueError:
print(f'Invalid index format: {parts[1]}')

```

```

#####else:
#####print("Invalid input format. Use 'c,index'.")
#####elif ',' in answer:
Input text format: index,text
parts = answer.split(',', 1)
if len(parts) == 2:
try:
target_index = int(parts[0].strip())
text_to_input = parts[1]
if target_index in selector_map:
element_node = selector_map[target_index]
print(
f"Inputting text '{text_to_input}' into element {target_index}: {element_node.tag_name}"
)

event = await browser_session.event_bus.dispatch(
TypeTextEvent(node=element_node, text=text_to_input)
)

print('Input successful.')
else:
print(f'Invalid index: {target_index}')
except ValueError:
print(f'Invalid index format: {parts[0]}')
else:
print("Invalid input format. Use 'index,text'.")

#####except Exception as action_e:
print(f'Action failed: {action_e}')

No explicit highlight removal here, get_state handles it at the start of the loop

#####except Exception as e:
print(f'Error in loop: {e}')
Optionally add a small delay before retrying
await asyncio.sleep(1)

if __name__ == '__main__':
 asyncio.run(test_focus_vs_all_elements())
 # asyncio.run(test_process_html_file()) # Commented out the other test

```

## ***.\system\dom\playground\multi\_act.py***

```

from system import Agent
from system.browser import BrowserProfile, BrowserSession
from system.browser.profile import ViewportSize
from system.llm import ChatAzureOpenAI

Initialize the Azure OpenAI client
llm = ChatAzureOpenAI(
 model='gpt-4.1-mini',
)

TASK = """
Go to https://browser-use.github.io/stress-tests/challenges/react-native-web-form.html and complete the
"""

async def main():
 browser = BrowserSession(
 browser_profile=BrowserProfile(
 window_size=ViewportSize(width=1100, height=1000),
)
)

 agent = Agent(task=TASK, llm=llm)

```



```

■await agent.run()

if __name__ == '__main__':
■import asyncio

■asyncio.run(main())

```

## ***.\system\dom\serializer\clickable\_elements.py***

```

from system.dom.views import EnhancedDOMTreeNode, NodeType

class ClickableElementDetector:
■@staticmethod
■def is_interactive(node: EnhancedDOMTreeNode) -> bool:
■ """Check if this node is clickable/interactive using enhanced scoring."""

■ def has_form_control_descendant(element: EnhancedDOMTreeNode, max_depth: int = 2) -> bool:
■ """Detect nested form controls within limited depth (handles label/span wrappers)."""
■ if max_depth <= 0:
■ return False

■ for child in element.children_and_shadow_roots:
■ if child.node_type != NodeType.ELEMENT_NODE:
■ continue

■ tag_name = child.tag_name
■ if tag_name in {'input', 'select', 'textarea'}:
■ return True

■ if has_form_control_descendant(child, max_depth=max_depth - 1):
■ return True

■ return False

■ # Skip non-element nodes
■ if node.node_type != NodeType.ELEMENT_NODE:
■ return False

■ # if ax ignored skip
■ if node.ax_node and node.ax_node.ignored:
■ return False

■ # remove html and body nodes
■ if node.tag_name in {'html', 'body'}:
■ return False

■ # Check for JavaScript click event listeners detected via CDP (without DOM mutation)
■ # this handles vue.js @click, react onClick, angular (click), etc.
■ if node.has_js_click_listener:
■ return True

■ # IFRAME elements should be interactive if they're large enough to potentially need scrolling
■ # Small iframes (< 100px width or height) are unlikely to have scrollable content
■ if node.tag_name and node.tag_name.upper() == 'IFRAME' or node.tag_name.upper() == 'FRAME':
■ if node.snapshot_node and node.snapshot_node.bounds:
■ width = node.snapshot_node.bounds.width
■ height = node.snapshot_node.bounds.height
■ # Only include iframes larger than 100x100px
■ if width > 100 and height > 100:
■ return True

■ # RELAXED SIZE CHECK: Allow all elements including size 0 (they might be interactive overlays, etc.)
■ # Note: Size 0 elements can still be interactive (e.g., invisible clickable overlays)
■ # Visibility is determined separately by CSS styles, not just bounding box size

■ # Specialized handling for labels used as component wrappers (e.g., Ant Design radio/checkbox)

```

```

#####if node.tag_name == 'label':
Skip labels that proxy via "for" to avoid double-activating external inputs
if node.attributes and node.attributes.get('for'):
return False

Detect labels that wrap form controls up to two levels deep (label > span > input)
if has_form_control_descendant(node, max_depth=2):
return True
Fall through to pointer/role/attribute heuristics for other label cases

Span wrappers for UI components (detect clear interactive signals only)
if node.tag_name == 'span':
if has_form_control_descendant(node, max_depth=2):
return True
Allow other heuristics (aria roles, event handlers, pointer) to decide

SEARCH ELEMENT DETECTION: Check for search-related classes and attributes
if node.attributes:
search_indicators = {
'search',
'magnify',
'glass',
'lookup',
'find',
'query',
'search-icon',
'search-btn',
'search-button',
'searchbox',
}

Check class names for search indicators
class_list = node.attributes.get('class', '').lower().split()
if any(indicator in ' '.join(class_list) for indicator in search_indicators):
return True

Check id for search indicators
element_id = node.attributes.get('id', '').lower()
if any(indicator in element_id for indicator in search_indicators):
return True

Check data attributes for search functionality
for attr_name, attr_value in node.attributes.items():
if attr_name.startswith('data-') and any(indicator in attr_value.lower() for indicator in search_in
return True

Enhanced accessibility property checks - direct clear indicators only
if node.ax_node and node.ax_node.properties:
for prop in node.ax_node.properties:
try:
aria disabled
if prop.name == 'disabled' and prop.value:
return False

aria hidden
if prop.name == 'hidden' and prop.value:
return False

Direct interactiveness indicators
if prop.name in ['focusable', 'editable', 'settable'] and prop.value:
return True

Interactive state properties (presence indicates interactive widget)
if prop.name in ['checked', 'expanded', 'pressed', 'selected']:
These properties only exist on interactive elements
return True

Form-related interactiveness
if prop.name in ['required', 'autocomplete'] and prop.value:
return True

Elements with keyboard shortcuts are interactive

```

```

#####if prop.name == 'keyshortcuts' and prop.value:
#####return True
#####except (AttributeError, ValueError):
Skip properties we can't process
#####continue

ENHANCED TAG CHECK: Include truly interactive elements
Note: 'label' removed - labels are handled by other attribute checks below - other wise labels with
#####interactive_tags = {
#####'button',
#####'input',
#####'select',
#####'textarea',
#####'a',
#####'details',
#####'summary',
#####'option',
#####'optgroup',
#####}
Check with case-insensitive comparison
#####if node.tag_name and node.tag_name.lower() in interactive_tags:
#####return True

SVG elements need special handling - only interactive if they have explicit handlers
svg_tags = {'svg', 'path', 'circle', 'rect', 'polygon', 'ellipse', 'line', 'polyline', 'g'}
if node.tag_name in svg_tags:
Only consider SVG elements interactive if they have:
1. Explicit event handlers
2. Interactive role attributes
3. Cursor pointer style
if node.attributes:
Check for event handlers
if any(attr.startswith('on') for attr in node.attributes):
return True
Check for interactive roles
if node.attributes.get('role') in {'button', 'link', 'menuitem'}:
return True
Check for cursor pointer (indicating clickability)
if node.attributes.get('style') and 'cursor: pointer' in node.attributes.get('style', ''):
return True
Otherwise, SVG elements are decorative
return False

Tertiary check: elements with interactive attributes
#####if node.attributes:
Check for event handlers or interactive attributes
#####interactive_attributes = {'onclick', 'onmousedown', 'onmouseup', 'onkeydown', 'onkeyup', 'tabindex'}
#####if any(attr in node.attributes for attr in interactive_attributes):
#####return True

Check for interactive ARIA roles
#####if 'role' in node.attributes:
#####interactive_roles = {
#####'button',
#####'link',
#####'menuitem',
#####'option',
#####'radio',
#####'checkbox',
#####'tab',
#####'textbox',
#####'combobox',
#####'slider',
#####'spinbutton',
#####'search',
#####'searchbox',
#####'row',
#####'cell',
#####'gridcell',
#####}
#####if node.attributes['role'] in interactive_roles:
#####return True

```

```

■■■# Quaternary check: accessibility tree roles
■■■if node.ax_node and node.ax_node.role:
■■■ interactive_ax_roles = {
■■■ 'button',
■■■ 'link',
■■■ 'menuitem',
■■■ 'option',
■■■ 'radio',
■■■ 'checkbox',
■■■ 'tab',
■■■ 'textbox',
■■■ 'combobox',
■■■ 'slider',
■■■ 'spinbutton',
■■■ 'listbox',
■■■ 'search',
■■■ 'searchbox',
■■■ 'row',
■■■ 'cell',
■■■ 'gridcell',
■■■ }
■■■ if node.ax_node.role in interactive_ax_roles:
■■■ return True

■■■# ICON AND SMALL ELEMENT CHECK: Elements that might be icons
■■■if (
■■■ node.snapshot_node
■■■ and node.snapshot_node.bounds
■■■ and 10 <= node.snapshot_node.bounds.width <= 50 # Icon-sized elements
■■■ and 10 <= node.snapshot_node.bounds.height <= 50
■■■):
■■■ # Check if this small element has interactive properties
■■■ if node.attributes:
■■■ # Small elements with these attributes are likely interactive icons
■■■ icon_attributes = {'class', 'role', 'onclick', 'data-action', 'aria-label'}
■■■ if any(attr in node.attributes for attr in icon_attributes):
■■■ return True

■■■# Final fallback: cursor style indicates interactivity (for cases Chrome missed)
■■■if node.snapshot_node and node.snapshot_node.cursor_style and node.snapshot_node.cursor_style == 'pointer':
■■■ return True

■■■return False

```

## ***.\system\dom\serializer\code\_use\_serializer.py***

```

@file purpose: Ultra-compact serializer optimized for code-use agents
Focuses on minimal token usage while preserving essential interactive context

from system.dom.utils import cap_text_length
from system.dom.views import (
 EnhancedDOMTreeNode,
 NodeType,
 SimplifiedNode,
)

Minimal but sufficient attribute list for code agents
CODE_USE_KEY_ATTRIBUTES = [
 'id', # Essential for element selection
 'name', # For form inputs
 'type', # For input types
 'placeholder', # For empty inputs
 'aria-label', # For buttons without text
 'value', # Current values
 'alt', # For images
 'class', # Keep top 2 classes for common selectors
]
Interactive elements agent can use

```

```

INTERACTIVE_ELEMENTS = {
 ■'a',
 ■'button',
 ■'input',
 ■'textarea',
 ■'select',
 ■'form',
}

Semantic structure elements - expanded to include more content containers
SEMANTIC_STRUCTURE = {
 ■'h1',
 ■'h2',
 ■'h3',
 ■'h4',
 ■'h5',
 ■'h6',
 ■'nav',
 ■'main',
 ■'header',
 ■'footer',
 ■'article',
 ■'section',
 ■'p', # Paragraphs often contain prices and product info
 ■'span', # Spans often contain prices and labels
 ■'div', # Divs with useful attributes (id/class) should be shown
 ■'ul',
 ■'ol',
 ■'li',
 ■'label',
 ■'img',
}

class DOMCodeAgentSerializer:
 ■"""Optimized DOM serializer for code-use agents - balances token efficiency with context."""

 ■@staticmethod
 ■def serialize_tree(node: SimplifiedNode | None, include_attributes: list[str], depth: int = 0) -> str:
 ■ ■"""
 ■ ■Serialize DOM tree with smart token optimization.

 ■ ■Strategy:
 ■ ■- Keep top 2 CSS classes for querySelector compatibility
 ■ ■- Show div/span/p elements with useful attributes or text
 ■ ■- Show all interactive + semantic elements
 ■ ■- Inline text up to 80 chars for better context
 ■ ■"""
 ■ ■if not node:
 ■ ■return ''

 ■ ■# Skip excluded/hidden nodes
 ■ ■if hasattr(node, 'excluded_by_parent') and node.excluded_by_parent:
 ■ ■return DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth)

 ■ ■if not node.should_display:
 ■ ■return DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth)

 ■ ■formatted_text = []
 ■ ■depth_str = ' ' * depth # Use 2 spaces instead of tabs for compactness

 ■ ■if node.original_node.node_type == NodeType.ELEMENT_NODE:
 ■ ■tag = node.original_node.tag_name.lower()
 ■ ■is_visible = node.original_node.snapshot_node and node.original_node.is_visible

 ■ ■# Skip invisible (except iframes)
 ■ ■if not is_visible and tag not in ['iframe', 'frame']:
 ■ ■return DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth)

 ■ ■# Special handling for iframes
 ■ ■if tag in ['iframe', 'frame']:
 ■ ■return DOMCodeAgentSerializer._serialize_iframe(node, include_attributes, depth)

```

```

Build minimal attributes
#####attributes_str = DOMCodeAgentSerializer._build_minimal_attributes(node.original_node)

Decide if element should be shown
#####is_interactive = tag in INTERACTIVE_ELEMENTS
#####is_semantic = tag in SEMANTIC_STRUCTURE
#####has_useful_attrs = bool(attributes_str)
#####has_text = DOMCodeAgentSerializer._has_direct_text(node)

Skip non-semantic, non-interactive containers without attributes
#####if not is_interactive and not is_semantic and not has_useful_attrs and not has_text:
#####return DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth)

Collapse pointless wrappers
#####if tag in {'div', 'span'} and not has_useful_attrs and not has_text and len(node.children) == 1:
#####return DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth)

Build element
#####line = f'{depth_str}<{tag}'

#####if attributes_str:
#####line += f' {attributes_str}'

Inline text
#####inline_text = DOMCodeAgentSerializer._get_inline_text(node)
#####if inline_text:
#####line += f'>{inline_text}'
#####else:
#####line += '>'

#####formatted_text.append(line)

Children (only if no inline text)
#####if node.children and not inline_text:
#####children_text = DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth + 1)
#####if children_text:
#####formatted_text.append(children_text)

#####elif node.original_node.node_type == NodeType.TEXT_NODE:
Handled inline with parent
#####pass

#####elif node.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
Shadow DOM - minimal marker
#####if node.children:
#####formatted_text.append(f'{depth_str}#shadow')
#####children_text = DOMCodeAgentSerializer._serialize_children(node, include_attributes, depth + 1)
#####if children_text:
#####formatted_text.append(children_text)

#####return '\n'.join(formatted_text)

#####@staticmethod
#####def _serialize_children(node: SimplifiedNode, include_attributes: list[str], depth: int) -> str:
"""Serialize children."""
children_output = []
for child in node.children:
child_text = DOMCodeAgentSerializer.serialize_tree(child, include_attributes, depth)
if child_text:
children_output.append(child_text)
return '\n'.join(children_output)

#####@staticmethod
#####def _build_minimal_attributes(node: EnhancedDOMTreeNode) -> str:
"""Build minimal but useful attributes - keep top 2 classes for selectors."""
attrs = []

if node.attributes:
for attr in CODE_USE_KEY_ATTRIBUTES:
if attr in node.attributes:
value = str(node.attributes[attr]).strip()
if value:

```

```

Special handling for class - keep only first 2 classes
if attr == 'class':
classes = value.split()[:2]
value = ' '.join(classes)
Cap at 25 chars
value = cap_text_length(value, 25)
attrs.append(f'{attr}="{value}"')

return ' '.join(attrs)

@staticmethod
def _has_direct_text(node: SimplifiedNode) -> bool:
"""Check if node has direct text children."""
for child in node.children:
if child.original_node.node_type == NodeType.TEXT_NODE:
text = child.original_node.node_value.strip() if child.original_node.node_value else ''
if len(text) > 1:
return True
return False

@staticmethod
def _get_inline_text(node: SimplifiedNode) -> str:
"""Get inline text (max 80 chars for better context)."""
text_parts = []
for child in node.children:
if child.original_node.node_type == NodeType.TEXT_NODE:
text = child.original_node.node_value.strip() if child.original_node.node_value else ''
if text and len(text) > 1:
text_parts.append(text)

if not text_parts:
return ''

combined = ' '.join(text_parts)
return cap_text_length(combined, 40)

@staticmethod
def _serialize_iframe(node: SimplifiedNode, include_attributes: list[str], depth: int) -> str:
"""Handle iframe minimally."""
formatted_text = []
depth_str = ' ' * depth
tag = node.original_node.tag_name.lower()

Minimal iframe marker
attributes_str = DOMCodeAgentSerializer._build_minimal_attributes(node.original_node)
line = f'{depth_str}<{tag}'
if attributes_str:
line += f' {attributes_str}'
line += '>'
formatted_text.append(line)

Iframe content
if node.original_node.content_document:
formatted_text.append(f'{depth_str} #iframe-content')

Find and serialize body content only
for child_node in node.original_node.content_document.children_nodes or []:
if child_node.tag_name.lower() == 'html':
for html_child in child_node.children:
if html_child.tag_name.lower() == 'body':
for body_child in html_child.children:
DOMCodeAgentSerializer._serialize_document_node(
body_child, formatted_text, include_attributes, depth + 2
)
break

return '\n'.join(formatted_text)

@staticmethod
def _serialize_document_node(
dom_node: EnhancedDOMTreeNode, output: list[str], include_attributes: list[str], depth: int
) -> None:

```

```

"""Serialize document node without SimplifiedNode wrapper."""
depth_str = ' ' * depth

if dom_node.node_type == NodeType.ELEMENT_NODE:
 tag = dom_node.tag_name.lower()

 # Skip invisible
 is_visible = dom_node.snapshot_node and dom_node.is_visible
 if not is_visible:
 return

 # Check if worth showing
 is_interactive = tag in INTERACTIVE_ELEMENTS
 is_semantic = tag in SEMANTIC_STRUCTURE
 attributes_str = DOMCodeAgentSerializer._build_minimal_attributes(dom_node)

 if not is_interactive and not is_semantic and not attributes_str:
 # Skip but process children
 for child in dom_node.children:
 DOMCodeAgentSerializer._serialize_document_node(child, output, include_attributes, depth)
 return

 # Build element
 line = f'{depth_str}<{tag}'
 if attributes_str:
 line += f' {attributes_str}'

 # Get text
 text_parts = []
 for child in dom_node.children:
 if child.node_type == NodeType.TEXT_NODE and child.node_value:
 text = child.node_value.strip()
 if text and len(text) > 1:
 text_parts.append(text)

 if text_parts:
 combined = ' '.join(text_parts)
 line += f'>{cap_text_length(combined, 25)}'
 else:
 line += '>'

 output.append(line)

 # Process non-text children
 for child in dom_node.children:
 if child.node_type != NodeType.TEXT_NODE:
 DOMCodeAgentSerializer._serialize_document_node(child, output, include_attributes, depth + 1)

```

## ***.\system\dom\serializer\eval\_serializer.py***

# @file purpose: Concise evaluation serializer for DOM trees - optimized for LLM query writing

```

from system.dom.utils import cap_text_length
from system.dom.views import (
 EnhancedDOMTreeNode,
 NodeType,
 SimplifiedNode,
)

Critical attributes for query writing and form interaction
NOTE: Removed 'id' and 'class' to force more robust structural selectors
EVAL_KEY_ATTRIBUTES = [
 'id', # Removed - can have special chars, forces structural selectors
 'class', # Removed - can have special chars like +, forces structural selectors
 'name',
 'type',
 'placeholder',

```



```

■'aria-label',
■'role',
■'value',
■# 'href',
■'data-testid',
■'alt', # for images
■'title', # useful for tooltips/link context
■# State attributes (critical for form interaction)
■'checked',
■'selected',
■'disabled',
■'required',
■'readonly',
■# ARIA states
■'aria-expanded',
■'aria-pressed',
■'aria-checked',
■'aria-selected',
■'aria-invalid',
■# Validation attributes (help agents avoid brute force)
■'pattern',
■'min',
■'max',
■'minlength',
■'maxlength',
■'step',
■'aria-valuemin',
■'aria-valuemax',
■'aria-valuenow',
]

Semantic elements that should always be shown
SEMANTIC_ELEMENTS = {
■'html', # Always show document root
■'body', # Always show body
■'h1',
■'h2',
■'h3',
■'h4',
■'h5',
■'h6',
■'a',
■'button',
■'input',
■'textarea',
■'select',
■'form',
■'label',
■'nav',
■'header',
■'footer',
■'main',
■'article',
■'section',
■'table',
■'thead',
■'tbody',
■'tr',
■'th',
■'td',
■'ul',
■'ol',
■'li',
■'img',
■'iframe',
■'video',
■'audio',
}

Container elements that can be collapsed if they only wrap one child
COLLAPSIBLE_CONTAINERS = {'div', 'span', 'section', 'article'}
SVG child elements to skip (decorative only, no interaction value)

```

```

SVG_ELEMENTS = {
 ■'path',
 ■'rect',
 ■'g',
 ■'circle',
 ■'ellipse',
 ■'line',
 ■'polyline',
 ■'polygon',
 ■'use',
 ■'defs',
 ■'clipPath',
 ■'mask',
 ■'pattern',
 ■'image',
 ■'text',
 ■'tspan',
}

```

```

class DOMEvalSerializer:

```

```

 ■"""Ultra-concise DOM serializer for quick LLM query writing."""

```

```

 ■@staticmethod

```

```

 ■def serialize_tree(node: SimplifiedNode | None, include_attributes: list[str], depth: int = 0) -> str:
 ■ """

```

```

 ■ Serialize complete DOM tree structure for LLM understanding.

```

```

 ■ Strategy:

```

```

 ■ ■- Show ALL elements to preserve DOM structure

```

```

 ■ ■- Non-interactive elements show just tag name

```

```

 ■ ■- Interactive elements show full attributes + [index]

```

```

 ■ ■- Self-closing tags only (no closing tags)

```

```

 ■ """

```

```

 ■ ■if not node:

```

```

 ■ ■ return ''

```

```

 ■ ■# Skip excluded nodes but process children

```

```

 ■ ■if hasattr(node, 'excluded_by_parent') and node.excluded_by_parent:

```

```

 ■ ■ return DOMEvalSerializer._serialize_children(node, include_attributes, depth)

```

```

 ■ ■# Skip nodes marked as should_display=False

```

```

 ■ ■if not node.should_display:

```

```

 ■ ■ return DOMEvalSerializer._serialize_children(node, include_attributes, depth)

```

```

 ■ ■formatted_text = []

```

```

 ■ ■depth_str = depth * '\t'

```

```

 ■ ■if node.original_node.node_type == NodeType.ELEMENT_NODE:

```

```

 ■ ■ tag = node.original_node.tag_name.lower()

```

```

 ■ ■ is_visible = node.original_node.snapshot_node and node.original_node.is_visible

```

```

 ■ ■# Container elements that should be shown even if invisible (might have visible children)

```

```

 ■ ■container_tags = {'html', 'body', 'div', 'main', 'section', 'article', 'aside', 'header', 'footer',

```

```

 ■ ■# Skip invisible elements UNLESS they're containers or iframes (which might have visible children)

```

```

 ■ ■if not is_visible and tag not in container_tags and tag not in ['iframe', 'frame']:

```

```

 ■ ■ return DOMEvalSerializer._serialize_children(node, include_attributes, depth)

```

```

 ■ ■# Special handling for iframes - show them with their content

```

```

 ■ ■if tag in ['iframe', 'frame']:

```

```

 ■ ■ return DOMEvalSerializer._serialize_iframe(node, include_attributes, depth)

```

```

 ■ ■# Skip SVG elements entirely - they're just decorative graphics with no interaction value

```

```

 ■ ■# Show the <svg> tag itself to indicate graphics, but don't recurse into children

```

```

 ■ ■if tag == 'svg':

```

```

 ■ ■ line = f'{depth_str}'

```

```

 ■ ■# Add [i_X] for interactive SVG elements only

```

```

 ■ ■if node.is_interactive:

```

```

 ■ ■ line += f'[i_{node.original_node.backend_node_id}] '

```

```

 ■ ■ line += '<svg'

```

```

 ■ ■ attributes_str = DOMEvalSerializer._build_compact_attributes(node.original_node)

```

```

#####if attributes_str:
#####line += f' {attributes_str}'
#####line += ' /> <!-- SVG content collapsed -->'
#####return line

Skip SVG child elements entirely (path, rect, g, circle, etc.)
####if tag in SVG_ELEMENTS:
#####return ''

Build compact attributes string
####attributes_str = DOMEvalSerializer._build_compact_attributes(node.original_node)

Decide if this element should be shown
####is_semantic = tag in SEMANTIC_ELEMENTS
####has_useful_attrs = bool(attributes_str)
####has_text_content = DOMEvalSerializer._has_direct_text(node)
####has_children = len(node.children) > 0

Build compact element representation
####line = f'{depth_str}'
Add backend node ID notation - [i_X] for interactive elements only
####if node.is_interactive:
#####line += f'[i_{node.original_node.backend_node_id}] '
Non-interactive elements don't get an index notation
####line += f'<{tag}'

####if attributes_str:
#####line += f' {attributes_str}'

Add scroll info if element is scrollable
####if node.original_node.should_show_scroll_info:
#####scroll_text = node.original_node.get_scroll_info_text()
#####if scroll_text:
#####line += f' scroll="{scroll_text}"'

Add inline text if present (keep it on same line for compactness)
####inline_text = DOMEvalSerializer._get_inline_text(node)

For containers (html, body, div, etc.), always show children even if there's inline text
For other elements, inline text replaces children (more compact)
####is_container = tag in container_tags

####if inline_text and not is_container:
#####line += f'>{inline_text}'
####else:
#####line += ' />'

####formatted_text.append(line)

Process children (always for containers, only if no inline_text for others)
####if has_children and (is_container or not inline_text):
#####children_text = DOMEvalSerializer._serialize_children(node, include_attributes, depth + 1)
#####if children_text:
#####formatted_text.append(children_text)

####elif node.original_node.node_type == NodeType.TEXT_NODE:
Text nodes are handled inline with their parent
####pass

####elif node.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
Shadow DOM - just show children directly with minimal marker
####if node.children:
#####formatted_text.append(f'{depth_str}#shadow')
#####children_text = DOMEvalSerializer._serialize_children(node, include_attributes, depth + 1)
#####if children_text:
#####formatted_text.append(children_text)

####return '\n'.join(formatted_text)

#####@staticmethod
#####def _serialize_children(node: SimplifiedNode, include_attributes: list[str], depth: int) -> str:
"""Helper to serialize all children of a node."""

```

```

 children_output = []

 ## Check if parent is a list container (ul, ol)
 is_list_container = node.original_node.node_type == NodeType.ELEMENT_NODE and node.original_node.tag_name in ['ul', 'ol']

 ## Track list items and consecutive links
 li_count = 0
 max_list_items = 50
 consecutive_link_count = 0
 max_consecutive_links = 50
 total_links_skipped = 0

 for child in node.children:
 ## Get tag name for this child
 current_tag = None
 if child.original_node.node_type == NodeType.ELEMENT_NODE:
 current_tag = child.original_node.tag_name.lower()

 ## If we're in a list container and this child is an li element
 if is_list_container and current_tag == 'li':
 li_count += 1
 ## Skip li elements after the 5th one
 if li_count > max_list_items:
 continue

 ## Track consecutive anchor tags (links)
 if current_tag == 'a':
 consecutive_link_count += 1
 ## Skip links after the 5th consecutive one
 if consecutive_link_count > max_consecutive_links:
 total_links_skipped += 1
 continue
 else:
 ## Reset counter when we hit a non-link element
 ## But first add truncation message if we skipped links
 if total_links_skipped > 0:
 depth_str = depth * '\t'
 children_output.append(f'{depth_str}... ({total_links_skipped} more links in this list)')
 total_links_skipped = 0
 consecutive_link_count = 0

 child_text = DOMEvalSerializer.serialize_tree(child, include_attributes, depth)
 if child_text:
 children_output.append(child_text)

 ## Add truncation message if we skipped items at the end
 if is_list_container and li_count > max_list_items:
 depth_str = depth * '\t'
 children_output.append(f'{depth_str}... ({li_count - max_list_items} more items in this list (truncated) use evaluate to get more)')

 ## Add truncation message for links if we skipped any at the end
 if total_links_skipped > 0:
 depth_str = depth * '\t'
 children_output.append(f'{depth_str}... ({total_links_skipped} more links in this list) (truncated) use evaluate to get more')

 return '\n'.join(children_output)

 @staticmethod
 def _build_compact_attributes(node: EnhancedDOMTreeNode) -> str:
 """Build ultra-compact attributes string with only key attributes."""
 attrs = []

 ## Prioritize attributes that help with query writing
 if node.attributes:
 for attr in EVAL_KEY_ATTRIBUTES:

```

```

#####if attr in node.attributes:
#####value = str(node.attributes[attr]).strip()
#####if not value:
#####continue

Special handling for different attributes
#####if attr == 'class':
For class, limit to first 2 classes to save space
#####classes = value.split()[:3]
#####value = ' '.join(classes)
#####elif attr == 'href':
For href, cap at 20 chars to save space
#####value = cap_text_length(value, 80)
#####else:
Cap at 25 chars for other attributes
#####value = cap_text_length(value, 80)

#####attrs.append(f'{attr}="{value}"')

Note: We intentionally don't add role from ax_node here because:
1. If role is explicitly set in HTML, it's already captured above via EVAL_KEY_ATTRIBUTES
2. Inferred roles from AX tree (like link, listitem, LineBreak) are redundant with the tag name
3. This reduces noise - is redundant, we already know <a> is a link

#####return ' '.join(attrs)

#####@staticmethod
#####def _has_direct_text(node: SimplifiedNode) -> bool:
"""Check if node has direct text children (not nested in other elements)."""
for child in node.children:
if child.original_node.node_type == NodeType.TEXT_NODE:
text = child.original_node.node_value.strip() if child.original_node.node_value else ''
if len(text) > 1:
return True
return False

#####@staticmethod
#####def _get_inline_text(node: SimplifiedNode) -> str:
"""Get text content to display inline (max 40 chars)."""
text_parts = []
for child in node.children:
if child.original_node.node_type == NodeType.TEXT_NODE:
text = child.original_node.node_value.strip() if child.original_node.node_value else ''
if text and len(text) > 1:
text_parts.append(text)

if not text_parts:
return ''

combined = ' '.join(text_parts)
return cap_text_length(combined, 80)

#####@staticmethod
#####def _serialize_iframe(node: SimplifiedNode, include_attributes: list[str], depth: int) -> str:
"""Handle iframe serialization with content document."""
formatted_text = []
depth_str = depth * '\t'
tag = node.original_node.tag_name.lower()

Build minimal iframe marker with key attributes
attributes_str = DOMEvalSerializer._build_compact_attributes(node.original_node)
line = f'{depth_str}<{tag}'
if attributes_str:
line += f' {attributes_str}'

Add scroll info for iframe content
if node.original_node.should_show_scroll_info:
scroll_text = node.original_node.get_scroll_info_text()
if scroll_text:
line += f' scroll="{scroll_text}"'

line += ' />'

```

```

 formatted_text.append(line)

 ## If iframe has content document, serialize its content
 if node.original_node.content_document:
 ## Add marker for iframe content
 formatted_text.append(f'{depth_str}\t#iframe-content')

 ## Process content document children
 for child_node in node.original_node.content_document.children_nodes or []:
 ## Process html documents
 if child_node.tag_name.lower() == 'html':
 ## Find and serialize body content only (skip head)
 for html_child in child_node.children:
 if html_child.tag_name.lower() == 'body':
 for body_child in html_child.children:
 ## Recursively process body children (iframe content)
 DOMEvalSerializer._serialize_document_node(
 body_child, formatted_text, include_attributes, depth + 2, is_iframe_content=True
)
 break # Stop after processing body
 else:
 ## Not an html element - serialize directly
 DOMEvalSerializer._serialize_document_node(
 child_node, formatted_text, include_attributes, depth + 1, is_iframe_content=True
)

 return '\n'.join(formatted_text)

 @staticmethod
 def _serialize_document_node(
 dom_node: EnhancedDOMTreeNode,
 output: list[str],
 include_attributes: list[str],
 depth: int,
 is_iframe_content: bool = True,
) -> None:
 """Helper to serialize a document node without SimplifiedNode wrapper.

 Args:
 is_iframe_content: If True, be more permissive with visibility checks since
 iframe content might not have snapshot data from parent page.
 """
 depth_str = depth * '\t'

 if dom_node.node_type == NodeType.ELEMENT_NODE:
 tag = dom_node.tag_name.lower()

 ## For iframe content, be permissive - show all semantic elements even without snapshot data
 ## For regular content, skip invisible elements
 if is_iframe_content:
 ## Only skip if we have snapshot data AND it's explicitly invisible
 ## If no snapshot data, assume visible (cross-origin iframe content)
 is_visible = (not dom_node.snapshot_node) or dom_node.is_visible
 else:
 ## Regular strict visibility check
 is_visible = dom_node.snapshot_node and dom_node.is_visible

 if not is_visible:
 return

 ## Check if semantic or has useful attributes
 is_semantic = tag in SEMANTIC_ELEMENTS
 attributes_str = DOMEvalSerializer._build_compact_attributes(dom_node)

 if not is_semantic and not attributes_str:
 ## Skip but process children
 for child in dom_node.children:
 DOMEvalSerializer._serialize_document_node(
 child, output, include_attributes, depth, is_iframe_content=is_iframe_content
)
 return
 ## Build element line

```

```

 line = f'{depth_str}<{tag}'
 if attributes_str:
 line += f' {attributes_str}'

 # Get direct text content
 text_parts = []
 for child in dom_node.children:
 if child.node_type == NodeType.TEXT_NODE and child.node_value:
 text = child.node_value.strip()
 if text and len(text) > 1:
 text_parts.append(text)

 if text_parts:
 combined = ' '.join(text_parts)
 line += f'>{cap_text_length(combined, 100)}'
 else:
 line += ' />'

 output.append(line)

 # Process non-text children
 for child in dom_node.children:
 if child.node_type != NodeType.TEXT_NODE:
 DOMEvalSerializer._serialize_document_node(
 child, output, include_attributes, depth + 1, is_iframe_content=is_iframe_content
)

```

## ***.system\dom\serializer\html\_serializer.py***

```

@file purpose: Serializes enhanced DOM trees to HTML format including shadow roots

```

```

from system.dom.views import EnhancedDOMTreeNode, NodeType

```

```

class HTMLSerializer:
 """Serializes enhanced DOM trees back to HTML format.

 This serializer reconstructs HTML from the enhanced DOM tree, including:
 - Shadow DOM content (both open and closed)
 - Iframe content documents
 - All attributes and text nodes
 - Proper HTML structure

 Unlike getOuterHTML which only captures light DOM, this captures the full
 enhanced tree including shadow roots that are crucial for modern SPAs.
 """

 def __init__(self, extract_links: bool = False):
 """Initialize the HTML serializer.

 Args:
 extract_links: If True, preserves all links. If False, removes href attributes.
 """
 self.extract_links = extract_links

 def serialize(self, node: EnhancedDOMTreeNode, depth: int = 0) -> str:
 """Serialize an enhanced DOM tree node to HTML.

 Args:
 node: The enhanced DOM tree node to serialize
 depth: Current depth for indentation (internal use)

 Returns:
 HTML string representation of the node and its descendants
 """
 if node.node_type == NodeType.DOCUMENT_NODE:
 # Process document root - serialize all children
 parts = []

```

```

#####for child in node.children_and_shadow_roots:
#####child_html = self.serialize(child, depth)
#####if child_html:
#####parts.append(child_html)
#####return ''.join(parts)

#####elif node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
Shadow DOM root - wrap in template with shadowrootmode attribute
#####parts = []

Add shadow root opening
#####shadow_type = node.shadow_root_type or 'open'
#####parts.append(f'<template shadowroot="{shadow_type.lower()}">')

Serialize shadow children
#####for child in node.children:
#####child_html = self.serialize(child, depth + 1)
#####if child_html:
#####parts.append(child_html)

Close shadow root
#####parts.append('</template>')

#####return ''.join(parts)

#####elif node.node_type == NodeType.ELEMENT_NODE:
#####parts = []
#####tag_name = node.tag_name.lower()

Skip non-content elements
#####if tag_name in {'style', 'script', 'head', 'meta', 'link', 'title'}:
#####return ''

Skip code tags with display:none - these often contain JSON state for SPAs
#####if tag_name == 'code' and node.attributes:
#####style = node.attributes.get('style', '')
Check if element is hidden (display:none) - likely JSON data
#####if 'display:none' in style.replace(' ', '') or 'display: none' in style:
#####return ''
Also check for bpr-guid IDs (LinkedIn's JSON data pattern)
#####element_id = node.attributes.get('id', '')
#####if 'bpr-guid' in element_id or 'data' in element_id or 'state' in element_id:
#####return ''

Skip base64 inline images - these are usually placeholders or tracking pixels
#####if tag_name == 'img' and node.attributes:
#####src = node.attributes.get('src', '')
#####if src.startswith('data:image/'):
#####return ''

Opening tag
#####parts.append(f'<{tag_name}>')

Add attributes
#####if node.attributes:
#####attrs = self._serialize_attributes(node.attributes)
#####if attrs:
#####parts.append(' ' + attrs)

Handle void elements (self-closing)
#####void_elements = {
#####'area',
#####'base',
#####'br',
#####'col',
#####'embed',
#####'hr',
#####'img',
#####'input',
#####'link',
#####'meta',
#####'param',

```



```

 'source',
 'track',
 'wbr',
]
 if tag_name in void_elements:
 parts.append(' />')
 return ''.join(parts)

 parts.append('>')

 # Handle table normalization (ensure thead/tbody for markdownify)
 if tag_name == 'table':
 # Serialize shadow roots first (same as the general path)
 if node.shadow_roots:
 for shadow_root in node.shadow_roots:
 child_html = self.serialize(shadow_root, depth + 1)
 if child_html:
 parts.append(child_html)
 table_html = self._serialize_table_children(node, depth)
 parts.append(table_html)
 # Handle iframe content document
 elif tag_name in {'iframe', 'frame'} and node.content_document:
 # Serialize iframe content
 for child in node.content_document.children_nodes or []:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)
 else:
 # Serialize shadow roots FIRST (for declarative shadow DOM)
 if node.shadow_roots:
 for shadow_root in node.shadow_roots:
 child_html = self.serialize(shadow_root, depth + 1)
 if child_html:
 parts.append(child_html)

 # Then serialize light DOM children (for slot projection)
 for child in node.children:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)

 # Closing tag
 parts.append(f'</{tag_name}>')

 return ''.join(parts)

 elif node.node_type == NodeType.TEXT_NODE:
 # Return text content with basic HTML escaping
 if node.node_value:
 return self._escape_html(node.node_value)
 return ''

 elif node.node_type == NodeType.COMMENT_NODE:
 # Skip comments to reduce noise
 return ''

 else:
 # Unknown node type - skip
 return ''

 def _serialize_table_children(self, table_node: EnhancedDOMTreeNode, depth: int) -> str:
 """Normalize table structure to ensure thead/tbody for markdownify.

 When a <table> has no <thead> but the first <tr> contains <th> cells,
 wrap that row in <thead> and remaining rows in <tbody>.
 """
 children = table_node.children
 if not children:
 return ''

 # Check if table already has thead
 child_tags = [c.tag_name for c in children if c.node_type == NodeType.ELEMENT_NODE]

```

```

 has_thead = 'thead' in child_tags
 has_tbody = 'tbody' in child_tags

 if has_thead or not child_tags:
 # Already normalized or empty – serialize normally
 parts = []
 for child in children:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)
 return ''.join(parts)

 # Find the first <tr> with <th> cells
 first_tr = None
 first_tr_idx = -1
 for i, child in enumerate(children):
 if child.nodeType == NodeType.ELEMENT_NODE and child.tag_name == 'tr':
 # Check if this row contains <th> cells
 has_th = any(c.nodeType == NodeType.ELEMENT_NODE and c.tag_name == 'th' for c in child.children)
 if has_th:
 first_tr = child
 first_tr_idx = i
 break # Only check the first <tr>

 if first_tr is None:
 # No header row detected – serialize normally
 parts = []
 for child in children:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)
 return ''.join(parts)

 # Wrap first_tr in <thead>, remaining <tr> in <tbody>
 parts = []

 # Emit any children before the header row (e.g. colgroup, caption)
 for child in children[:first_tr_idx]:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)

 # Emit <thead>
 parts.append('<thead>')
 parts.append(self.serialize(first_tr, depth + 2))
 parts.append('</thead>')

 # Collect remaining rows
 remaining = children[first_tr_idx + 1 :]
 if remaining and not has_tbody:
 parts.append('<tbody>')
 for child in remaining:
 child_html = self.serialize(child, depth + 2)
 if child_html:
 parts.append(child_html)
 parts.append('</tbody>')
 else:
 for child in remaining:
 child_html = self.serialize(child, depth + 1)
 if child_html:
 parts.append(child_html)

 return ''.join(parts)

def _serialize_attributes(self, attributes: dict[str, str]) -> str:
 """Serialize element attributes to HTML attribute string.

 Args:
 attributes: Dictionary of attribute names to values

 Returns:
 HTML attribute string (e.g., 'class="foo" id="bar"')

```

```

 """
 parts = []
 for key, value in attributes.items():
 # Skip href if not extracting links
 if not self.extract_links and key == 'href':
 continue

 # Skip data-* attributes as they often contain JSON payloads
 # These are used by modern SPAs (React, Vue, Angular) for state management
 if key.startswith('data-'):
 continue

 # Handle boolean attributes
 if value == '' or value is None:
 parts.append(key)
 else:
 # Escape attribute value
 escaped_value = self._escape_attribute(value)
 parts.append(f'{key}="{escaped_value}"')

 return ' '.join(parts)

def _escape_html(self, text: str) -> str:
 """Escape HTML special characters in text content.

 Args:
 text: Raw text content

 Returns:
 HTML-escaped text
 """
 return text.replace('&', '&').replace('<', '<').replace('>', '>')

def _escape_attribute(self, value: str) -> str:
 """Escape HTML special characters in attribute values.

 Args:
 value: Raw attribute value

 Returns:
 HTML-escaped attribute value
 """
 return value.replace('&', '&').replace('<', '<').replace('>', '>').replace('"', '"').r

```

## ***.\system\dom\serializer\paint\_order.py***

```

from collections import defaultdict
from dataclasses import dataclass

from system.dom.views import SimplifiedNode

"""
Helper class for maintaining a union of rectangles (used for order of elements calculation)
"""

@dataclass(frozen=True, slots=True)
class Rect:
 """Closed axis-aligned rectangle with (x1,y1) bottom-left, (x2,y2) top-right."""

 x1: float
 y1: float
 x2: float
 y2: float

 def __post_init__(self):
 if not (self.x1 <= self.x2 and self.y1 <= self.y2):
 return False

```

```

■# --- fast relations -----
■def area(self) -> float:
■ return (self.x2 - self.x1) * (self.y2 - self.y1)

■def intersects(self, other: 'Rect') -> bool:
■ return not (self.x2 <= other.x1 or other.x2 <= self.x1 or self.y2 <= other.y1 or other.y2 <= self.y1)

■def contains(self, other: 'Rect') -> bool:
■ return self.x1 <= other.x1 and self.y1 <= other.y1 and self.x2 >= other.x2 and self.y2 >= other.y2

class RectUnionPure:
 """
 ■Maintains a *disjoint* set of rectangles.
 ■No external dependencies - fine for a few thousand rectangles.
 """

 ■__slots__ = ('_rects',)

 ■def __init__(self):
 ■ self._rects: list[Rect] = []

 ■# -----
 ■def _split_diff(self, a: Rect, b: Rect) -> list[Rect]:
 ■ r"""
 ■ Return list of up to 4 rectangles = a \ b.
 ■ Assumes a intersects b.
 ■ """
 ■ parts = []

 ■ ■# Bottom slice
 ■ ■if a.y1 < b.y1:
 ■ ■parts.append(Rect(a.x1, a.y1, a.x2, b.y1))
 ■ ■# Top slice
 ■ ■if b.y2 < a.y2:
 ■ ■parts.append(Rect(a.x1, b.y2, a.x2, a.y2))

 ■ ■# Middle (vertical) strip: y overlap is [max(a.y1,b.y1), min(a.y2,b.y2)]
 ■ ■y_lo = max(a.y1, b.y1)
 ■ ■y_hi = min(a.y2, b.y2)

 ■ ■# Left slice
 ■ ■if a.x1 < b.x1:
 ■ ■parts.append(Rect(a.x1, y_lo, b.x1, y_hi))
 ■ ■# Right slice
 ■ ■if b.x2 < a.x2:
 ■ ■parts.append(Rect(b.x2, y_lo, a.x2, y_hi))

 ■ return parts

 ■# -----
 ■def contains(self, r: Rect) -> bool:
 ■ r"""
 ■ True iff r is fully covered by the current union.
 ■ """
 ■ if not self._rects:
 ■ return False

 ■ stack = [r]
 ■ for s in self._rects:
 ■ new_stack = []
 ■ for piece in stack:
 ■ if s.contains(piece):
 ■ ■# piece completely gone
 ■ continue
 ■ if piece.intersects(s):
 ■ new_stack.extend(self._split_diff(piece, s))
 ■ else:
 ■ new_stack.append(piece)
 ■ if not new_stack: # everything eaten - covered
 ■ return True
 ■ stack = new_stack

```

```

 return False # something survived

 # -----
 def add(self, r: Rect) -> bool:
 """
 Insert r unless it is already covered.
 Returns True if the union grew.
 """
 if self.contains(r):
 return False

 pending = [r]
 i = 0
 while i < len(self._rects):
 s = self._rects[i]
 new_pending = []
 changed = False
 for piece in pending:
 if piece.intersects(s):
 new_pending.extend(self._split_diff(piece, s))
 changed = True
 else:
 new_pending.append(piece)
 pending = new_pending
 if changed:
 # s unchanged; proceed with next existing rectangle
 i += 1
 else:
 i += 1

 # Any left over pieces are new, non overlapping areas
 self._rects.extend(pending)
 return True

class PaintOrderRemover:
 """
 Calculates which elements should be removed based on the paint order parameter.
 """

 def __init__(self, root: SimplifiedNode):
 self.root = root

 def calculate_paint_order(self) -> None:
 all_simplified_nodes_with_paint_order: list[SimplifiedNode] = []

 def collect_paint_order(node: SimplifiedNode) -> None:
 if (
 node.original_node.snapshot_node
 and node.original_node.snapshot_node.paint_order is not None
 and node.original_node.snapshot_node.bounds is not None
):
 all_simplified_nodes_with_paint_order.append(node)

 for child in node.children:
 collect_paint_order(child)

 collect_paint_order(self.root)

 grouped_by_paint_order: defaultdict[int, list[SimplifiedNode]] = defaultdict(list)

 for node in all_simplified_nodes_with_paint_order:
 if node.original_node.snapshot_node and node.original_node.snapshot_node.paint_order is not None:
 grouped_by_paint_order[node.original_node.snapshot_node.paint_order].append(node)

 rect_union = RectUnionPure()

 for paint_order, nodes in sorted(grouped_by_paint_order.items(), key=lambda x: -x[0]):
 rects_to_add = []

 for node in nodes:
 if not node.original_node.snapshot_node or not node.original_node.snapshot_node.bounds:

```

```

#####continue # shouldn't happen by how we filter them out in the first place

#####rect = Rect(
#####x1=node.original_node.snapshot_node.bounds.x,
#####y1=node.original_node.snapshot_node.bounds.y,
#####x2=node.original_node.snapshot_node.bounds.x + node.original_node.snapshot_node.bounds.width,
#####y2=node.original_node.snapshot_node.bounds.y + node.original_node.snapshot_node.bounds.height,
#####)

#####if rect_union.contains(rect):
#####node.ignored_by_paint_order = True

don't add to the nodes if opacity is less then 0.95 or background-color is transparent
#####if (
#####node.original_node.snapshot_node.computed_styles
#####and node.original_node.snapshot_node.computed_styles.get('background-color', 'rgba(0, 0, 0, 0)')
#####== 'rgba(0, 0, 0, 0)'
#####) or (
#####node.original_node.snapshot_node.computed_styles
#####and float(node.original_node.snapshot_node.computed_styles.get('opacity', '1'))
#####< 0.8 # this is highly vibes based number
#####):
#####continue

#####rects_to_add.append(rect)

#####for rect in rects_to_add:
#####rect_union.add(rect)

#####return None

```

## ***.\system\dom\serializer\serializer.py***

```

@file purpose: Serializes enhanced DOM trees to string format for LLM consumption

from typing import Any

from system.dom.serializer.clickable_elements import ClickableElementDetector
from system.dom.serializer.paint_order import PaintOrderRemover
from system.dom.utils import cap_text_length
from system.dom.views import (
 DOMRect,
 DOMSelectorMap,
 EnhancedDOMTreeNode,
 NodeType,
 PropagatingBounds,
 SerializedDOMState,
 SimplifiedNode,
)

DISABLED_ELEMENTS = {'style', 'script', 'head', 'meta', 'link', 'title'}

SVG child elements to skip (decorative only, no interaction value)
SVG_ELEMENTS = {
 'path',
 'rect',
 'g',
 'circle',
 'ellipse',
 'line',
 'polyline',
 'polygon',
 'use',
 'defs',
 'clipPath',
 'mask',
 'pattern',
 'image',
}

```

```

 'text',
 'tspan',
}

```

```

class DOMTreeSerializer:

```

```

 """Serializes enhanced DOM trees to string format."""

```

```

 ## Configuration - elements that propagate bounds to their children

```

```

 PROPAGATING_ELEMENTS = [

```

```

 {'tag': 'a', 'role': None}, # Any <a> tag

```

```

 {'tag': 'button', 'role': None}, # Any <button> tag

```

```

 {'tag': 'div', 'role': 'button'}, # <div role="button">

```

```

 {'tag': 'div', 'role': 'combobox'}, # <div role="combobox"> - dropdowns/selects

```

```

 {'tag': 'span', 'role': 'button'}, #

```

```

 {'tag': 'span', 'role': 'combobox'}, #

```

```

 {'tag': 'input', 'role': 'combobox'}, # <input role="combobox"> - autocomplete inputs

```

```

 {'tag': 'input', 'role': 'combobox'}, # <input type="text"> - text inputs with suggestions

```

```

 {'tag': 'div', 'role': 'link'}, # <div role="link">

```

```

 {'tag': 'span', 'role': 'link'}, #

```

```

]

```

```

 DEFAULT_CONTAINMENT_THRESHOLD = 0.99 # 99% containment by default

```

```

 def __init__(

```

```

 self,

```

```

 root_node: EnhancedDOMTreeNode,

```

```

 previous_cached_state: SerializedDOMState | None = None,

```

```

 enable_bbox_filtering: bool = True,

```

```

 containment_threshold: float | None = None,

```

```

 paint_order_filtering: bool = True,

```

```

 session_id: str | None = None,

```

```

):

```

```

 self.root_node = root_node

```

```

 self._interactive_counter = 1

```

```

 self._selector_map: DOMSelectorMap = {}

```

```

 self._previous_cached_selector_map = previous_cached_state.selector_map if previous_cached_state else

```

```

 ## Add timing tracking

```

```

 self.timing_info: dict[str, float] = {}

```

```

 ## Cache for clickable element detection to avoid redundant calls

```

```

 self._clickable_cache: dict[int, bool] = {}

```

```

 ## Bounding box filtering configuration

```

```

 self.enable_bbox_filtering = enable_bbox_filtering

```

```

 self.containment_threshold = containment_threshold or self.DEFAULT_CONTAINMENT_THRESHOLD

```

```

 ## Paint order filtering configuration

```

```

 self.paint_order_filtering = paint_order_filtering

```

```

 ## Session ID for session-specific exclude attribute

```

```

 self.session_id = session_id

```

```

 def _safe_parse_number(self, value_str: str, default: float) -> float:

```

```

 """Parse string to float, handling negatives and decimals."""

```

```

 try:

```

```

 return float(value_str)

```

```

 except (ValueError, TypeError):

```

```

 return default

```

```

 def _safe_parse_optional_number(self, value_str: str | None) -> float | None:

```

```

 """Parse string to float, returning None for invalid values."""

```

```

 if not value_str:

```

```

 return None

```

```

 try:

```

```

 return float(value_str)

```

```

 except (ValueError, TypeError):

```

```

 return None

```

```

 def serialize_accessible_elements(self) -> tuple[SerializedDOMState, dict[str, float]]:

```

```

 import time

```

```

 start_total = time.time()

```

```

 ## Reset state

```

```

 self._interactive_counter = 1

```

```

 self._selector_map = {}

```

```

self._semantic_groups = []
self._clickable_cache = {} # Clear cache for new serialization

Step 1: Create simplified tree (includes clickable element detection)
start_step1 = time.time()
simplified_tree = self._create_simplified_tree(self.root_node)
end_step1 = time.time()
self.timing_info['create_simplified_tree'] = end_step1 - start_step1

Step 2: Remove elements based on paint order
start_step3 = time.time()
if self.paint_order_filtering and simplified_tree:
 PaintOrderRemover(simplified_tree).calculate_paint_order()
end_step3 = time.time()
self.timing_info['calculate_paint_order'] = end_step3 - start_step3

Step 3: Optimize tree (remove unnecessary parents)
start_step2 = time.time()
optimized_tree = self._optimize_tree(simplified_tree)
end_step2 = time.time()
self.timing_info['optimize_tree'] = end_step2 - start_step2

Step 3: Apply bounding box filtering (NEW)
if self.enable_bbox_filtering and optimized_tree:
 start_step3 = time.time()
 filtered_tree = self._apply_bounding_box_filtering(optimized_tree)
 end_step3 = time.time()
 self.timing_info['bbox_filtering'] = end_step3 - start_step3
else:
 filtered_tree = optimized_tree

Step 4: Assign interactive indices to clickable elements
start_step4 = time.time()
self._assign_interactive_indices_and_mark_new_nodes(filtered_tree)
end_step4 = time.time()
self.timing_info['assign_interactive_indices'] = end_step4 - start_step4

end_total = time.time()
self.timing_info['serialize_accessible_elements_total'] = end_total - start_total

return SerializedDOMState(_root=filtered_tree, selector_map=self._selector_map), self.timing_info

def _add_compound_components(self, simplified: SimplifiedNode, node: EnhancedDOMTreeNode) -> None:
 """Enhance compound controls with information from their child components."""
 ## Only process elements that might have compound components
 if node.tag_name not in ['input', 'select', 'details', 'audio', 'video']:
 return

 ## For input elements, check for compound input types
 if node.tag_name == 'input':
 if not node.attributes or node.attributes.get('type') not in [
 'date',
 'time',
 'datetime-local',
 'month',
 'week',
 'range',
 'number',
 'color',
 'file',
]:
 return
 ## For other elements, check if they have AX child indicators
 elif not node.ax_node or not node.ax_node.child_ids:
 return

 ## Add compound component information based on element type
 element_type = node.tag_name
 input_type = node.attributes.get('type', '') if node.attributes else ''

 if element_type == 'input':
 ## NOTE: For date/time inputs, we DON'T add compound components because:

```



```

1. They confuse the model (seeing "Day, Month, Year" suggests DD.MM.YYYY format)
2. HTML5 date/time inputs ALWAYS require ISO format (YYYY-MM-DD, HH:MM, etc.)
3. The placeholder attribute clearly shows the required format
4. These inputs use direct value assignment, not sequential typing
if input_type in ['date', 'time', 'datetime-local', 'month', 'week']:
Skip compound components for date/time inputs - format is shown in placeholder
pass
elif input_type == 'range':
Range slider with value indicator
min_val = node.attributes.get('min', '0') if node.attributes else '0'
max_val = node.attributes.get('max', '100') if node.attributes else '100'

node._compound_children.append(
{
'role': 'slider',
'name': 'Value',
'valuemin': self._safe_parse_number(min_val, 0.0),
'valuemax': self._safe_parse_number(max_val, 100.0),
'valuenow': None,
}
)
simplified.is_compound_component = True
elif input_type == 'number':
Number input with increment/decrement buttons
min_val = node.attributes.get('min') if node.attributes else None
max_val = node.attributes.get('max') if node.attributes else None

node._compound_children.extend(
[
{'role': 'button', 'name': 'Increment', 'valuemin': None, 'valuemax': None, 'valuenow': None},
{'role': 'button', 'name': 'Decrement', 'valuemin': None, 'valuemax': None, 'valuenow': None},
{
'role': 'textbox',
'name': 'Value',
'valuemin': self._safe_parse_optional_number(min_val),
'valuemax': self._safe_parse_optional_number(max_val),
'valuenow': None,
},
]
)
simplified.is_compound_component = True
elif input_type == 'color':
Color picker with components
node._compound_children.extend(
[
{'role': 'textbox', 'name': 'Hex Value', 'valuemin': None, 'valuemax': None, 'valuenow': None},
{'role': 'button', 'name': 'Color Picker', 'valuemin': None, 'valuemax': None, 'valuenow': None},
]
)
simplified.is_compound_component = True
elif input_type == 'file':
File input with browse button
multiple = 'multiple' in node.attributes if node.attributes else False

Extract current file selection state from AX tree
current_value = 'None' # Default to explicit "None" string for clarity
if node.ax_node and node.ax_node.properties:
for prop in node.ax_node.properties:
Try valuetext first (human-readable display like "file.pdf")
if prop.name == 'valuetext' and prop.value:
value_str = str(prop.value).strip()
if value_str and value_str.lower() not in ['', 'no file chosen', 'no file selected']:
current_value = value_str
break
Also try 'value' property (may include full path)
elif prop.name == 'value' and prop.value:
value_str = str(prop.value).strip()
if value_str:
For file inputs, value might be a full path - extract just filename
if '\\' in value_str:
current_value = value_str.split('\\')[-1]
elif '/' in value_str:

```

```

#####current_value = value_str.split('/')[-1]
#####else:
#####current_value = value_str
#####break

#####node._compound_children.extend(
#####[
#####{'role': 'button', 'name': 'Browse Files', 'valuemin': None, 'valuemax': None, 'valuenow': None},
#####{
#####'role': 'textbox',
#####'name': f'{"Files" if multiple else "File"} Selected',
#####'valuemin': None,
#####'valuemax': None,
#####'valuenow': current_value, # Always shows state: filename or "None"
#####},
#####]
#####)
#####simplified.is_compound_component = True

#####elif element_type == 'select':
Select dropdown with option list and detailed option information
#####base_components = [
#####{'role': 'button', 'name': 'Dropdown Toggle', 'valuemin': None, 'valuemax': None, 'valuenow': None}
#####]

Extract option information from child nodes
#####options_info = self._extract_select_options(node)
#####if options_info:
#####options_component = {
#####'role': 'listbox',
#####'name': 'Options',
#####'valuemin': None,
#####'valuemax': None,
#####'valuenow': None,
#####'options_count': options_info['count'],
#####'first_options': options_info['first_options'],
#####}
#####if options_info['format_hint']:
#####options_component['format_hint'] = options_info['format_hint']
#####base_components.append(options_component)
#####else:
#####base_components.append(
#####{'role': 'listbox', 'name': 'Options', 'valuemin': None, 'valuemax': None, 'valuenow': None}
#####)

#####node._compound_children.extend(base_components)
#####simplified.is_compound_component = True

#####elif element_type == 'details':
Details/summary disclosure widget
#####node._compound_children.extend(
#####[
#####{'role': 'button', 'name': 'Toggle Disclosure', 'valuemin': None, 'valuemax': None, 'valuenow': None},
#####{'role': 'region', 'name': 'Content Area', 'valuemin': None, 'valuemax': None, 'valuenow': None},
#####]
#####)
#####simplified.is_compound_component = True

#####elif element_type == 'audio':
Audio player controls
#####node._compound_children.extend(
#####[
#####{'role': 'button', 'name': 'Play/Pause', 'valuemin': None, 'valuemax': None, 'valuenow': None},
#####{'role': 'slider', 'name': 'Progress', 'valuemin': 0, 'valuemax': 100, 'valuenow': None},
#####{'role': 'button', 'name': 'Mute', 'valuemin': None, 'valuemax': None, 'valuenow': None},
#####{'role': 'slider', 'name': 'Volume', 'valuemin': 0, 'valuemax': 100, 'valuenow': None},
#####]
#####)
#####simplified.is_compound_component = True

#####elif element_type == 'video':
Video player controls

```

```

node._compound_children.extend(
 [
 {'role': 'button', 'name': 'Play/Pause', 'valuemin': None, 'valuemax': None, 'valuenow': None},
 {'role': 'slider', 'name': 'Progress', 'valuemin': 0, 'valuemax': 100, 'valuenow': None},
 {'role': 'button', 'name': 'Mute', 'valuemin': None, 'valuemax': None, 'valuenow': None},
 {'role': 'slider', 'name': 'Volume', 'valuemin': 0, 'valuemax': 100, 'valuenow': None},
 {'role': 'button', 'name': 'Fullscreen', 'valuemin': None, 'valuemax': None, 'valuenow': None},
]
)
simplified.is_compound_component = True

def _extract_select_options(self, select_node: EnhancedDOMTreeNode) -> dict[str, Any] | None:
 """Extract option information from a select element."""
 if not select_node.children:
 return None

 options = []
 option_values = []

 def extract_options_recursive(node: EnhancedDOMTreeNode) -> None:
 """Recursively extract option elements, including from optgroups."""
 if node.tag_name.lower() == 'option':
 # Extract option text and value
 option_text = ''
 option_value = ''

 # Get value attribute if present
 if node.attributes and 'value' in node.attributes:
 option_value = str(node.attributes['value']).strip()

 # Get text content from direct child text nodes only to avoid duplication
 def get_direct_text_content(n: EnhancedDOMTreeNode) -> str:
 text = ''
 for child in n.children:
 if child.node_type == NodeType.TEXT_NODE and child.node_value:
 text += child.node_value.strip() + ' '
 return text.strip()

 option_text = get_direct_text_content(node)

 # Use text as value if no explicit value
 if not option_value and option_text:
 option_value = option_text

 if option_text or option_value:
 options.append({'text': option_text, 'value': option_value})
 option_values.append(option_value)

 elif node.tag_name.lower() == 'optgroup':
 # Process optgroup children
 for child in node.children:
 extract_options_recursive(child)
 # Process other children that might contain options
 for child in node.children:
 extract_options_recursive(child)

 # Extract all options from select children
 for child in select_node.children:
 extract_options_recursive(child)

 if not options:
 return None

 # Prepare first 4 options for display
 first_options = []
 for option in options[:4]:
 # Always use text if available, otherwise use value
 display_text = option['text'] if option['text'] else option['value']
 if display_text:
 # Limit individual option text to avoid overly long attributes
 text = display_text[:30] + ('...' if len(display_text) > 30 else '')

```

```

#####first_options.append(text)

Add ellipsis indicator if there are more options than shown
#####if len(options) > 4:
#####first_options.append(f'... {len(options) - 4} more options...')

Try to infer format hint from option values
#####format_hint = None
#####if len(option_values) >= 2:
Check for common patterns
#####if all(val.isdigit() for val in option_values[:5] if val):
#####format_hint = 'numeric'
#####elif all(len(val) == 2 and val.isupper() for val in option_values[:5] if val):
#####format_hint = 'country/state codes'
#####elif all('/') in val or '-' in val for val in option_values[:5] if val):
#####format_hint = 'date/path format'
#####elif any('@' in val for val in option_values[:5] if val):
#####format_hint = 'email addresses'

#####return {'count': len(options), 'first_options': first_options, 'format_hint': format_hint}

#####def _is_interactive_cached(self, node: EnhancedDOMTreeNode) -> bool:
#####"""Cached version of clickable element detection to avoid redundant calls."""

#####if node.node_id not in self._clickable_cache:
#####import time

#####start_time = time.time()
#####result = ClickableElementDetector.is_interactive(node)
#####end_time = time.time()

#####if 'clickable_detection_time' not in self.timing_info:
#####self.timing_info['clickable_detection_time'] = 0
#####self.timing_info['clickable_detection_time'] += end_time - start_time

#####self._clickable_cache[node.node_id] = result

#####return self._clickable_cache[node.node_id]

#####def _create_simplified_tree(self, node: EnhancedDOMTreeNode, depth: int = 0) -> SimplifiedNode | None:
#####"""Step 1: Create a simplified tree with enhanced element detection."""

#####if node.node_type == NodeType.DOCUMENT_NODE:
for all cldren including shadow roots
#####for child in node.children_and_shadow_roots:
#####simplified_child = self._create_simplified_tree(child, depth + 1)
#####if simplified_child:
#####return simplified_child

#####return None

#####if node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
ENHANCED shadow DOM processing - always include shadow content
#####simplified = SimplifiedNode(original_node=node, children=[])
#####for child in node.children_and_shadow_roots:
#####simplified_child = self._create_simplified_tree(child, depth + 1)
#####if simplified_child:
#####simplified.children.append(simplified_child)

Always return shadow DOM fragments, even if children seem empty
Shadow DOM often contains the actual interactive content in SPAs
#####return simplified if simplified.children else SimplifiedNode(original_node=node, children=[])

#####elif node.node_type == NodeType.ELEMENT_NODE:
Skip non-content elements
#####if node.node_name.lower() in DISABLED_ELEMENTS:
#####return None

Skip SVG child elements entirely (path, rect, g, circle, etc.)
#####if node.node_name.lower() in SVG_ELEMENTS:
#####return None
#####attributes = node.attributes or {}

```

```

Check for session-specific exclude attribute first, then fall back to legacy attribute
#####exclude_attr = None
#####attr_type = None
#####if self.session_id:
session_specific_attr = f'data-browser-use-exclude-{self.session_id}'
exclude_attr = attributes.get(session_specific_attr)
if exclude_attr:
attr_type = 'session-specific'
Fall back to legacy attribute if session-specific not found
if not exclude_attr:
exclude_attr = attributes.get('data-browser-use-exclude')
if isinstance(exclude_attr, str) and exclude_attr.lower() == 'true':
return None

#####if node.node_name == 'IFRAME' or node.node_name == 'FRAME':
if node.content_document:
simplified = SimplifiedNode(original_node=node, children=[])
for child in node.content_document.children_nodes or []:
simplified_child = self._create_simplified_tree(child, depth + 1)
if simplified_child is not None:
simplified.children.append(simplified_child)
return simplified

#####is_visible = node.is_visible
#####is_scrollable = node.is_actually_scrollable
#####has_shadow_content = bool(node.children_and_shadow_roots)

ENHANCED SHADOW DOM DETECTION: Include shadow hosts even if not visible
#####is_shadow_host = any(child.node_type == NodeType.DOCUMENT_FRAGMENT_NODE for child in node.children_and_shadow_roots)

Override visibility for elements with validation attributes
#####if not is_visible and node.attributes:
has_validation_attrs = any(attr.startswith(('aria-', 'pseudo')) for attr in node.attributes.keys())
if has_validation_attrs:
is_visible = True # Force visibility for validation elements

EXCEPTION: File inputs are often hidden with opacity:0 but are still functional
Bootstrap and other frameworks use this pattern with custom-styled file pickers
#####is_file_input = (
node.tag_name and node.tag_name.lower() == 'input' and node.attributes and node.attributes.get('type') == 'file'
#####)
#####if not is_visible and is_file_input:
is_visible = True # Force visibility for file inputs

Include if visible, scrollable, has children, or is shadow host
#####if is_visible or is_scrollable or has_shadow_content or is_shadow_host:
simplified = SimplifiedNode(original_node=node, children=[], is_shadow_host=is_shadow_host)

Process ALL children including shadow roots with enhanced logging
#####for child in node.children_and_shadow_roots:
simplified_child = self._create_simplified_tree(child, depth + 1)
if simplified_child:
simplified.children.append(simplified_child)

COMPOUND CONTROL PROCESSING: Add virtual components for compound controls
#####self._add_compound_components(simplified, node)

SHADOW DOM SPECIAL CASE: Always include shadow hosts even if not visible
Many SPA frameworks (React, Vue) render content in shadow DOM
#####if is_shadow_host and simplified.children:
return simplified

Return if meaningful or has meaningful children
#####if is_visible or is_scrollable or simplified.children:
return simplified
#####elif node.node_type == NodeType.TEXT_NODE:
Include meaningful text nodes
is_visible = node.snapshot_node and node.is_visible
if is_visible and node.node_value and node.node_value.strip() and len(node.node_value.strip()) > 1:
return SimplifiedNode(original_node=node, children=[])

#####return None

```

```

def _optimize_tree(self, node: SimplifiedNode | None) -> SimplifiedNode | None:
 """Step 2: Optimize tree structure."""
 if not node:
 return None

 # Process children
 optimized_children = []
 for child in node.children:
 optimized_child = self._optimize_tree(child)
 if optimized_child:
 optimized_children.append(optimized_child)

 node.children = optimized_children

 # Keep meaningful nodes
 is_visible = node.original_node.snapshot_node and node.original_node.is_visible

 # EXCEPTION: File inputs are often hidden with opacity:0 but are still functional
 is_file_input = (
 node.original_node.tag_name
 and node.original_node.tag_name.lower() == 'input'
 and node.original_node.attributes
 and node.original_node.attributes.get('type') == 'file'
)

 if (
 is_visible # Keep all visible nodes
 or node.original_node.is_actually_scrollable
 or node.original_node.node_type == NodeType.TEXT_NODE
 or node.children
 or is_file_input # Keep file inputs even if not visible
):
 return node

 return None

def _collect_interactive_elements(self, node: SimplifiedNode, elements: list[SimplifiedNode]) -> None:
 """Recursively collect interactive elements that are also visible."""
 is_interactive = self._is_interactive_cached(node.original_node)
 is_visible = node.original_node.snapshot_node and node.original_node.is_visible

 # Only collect elements that are both interactive AND visible
 if is_interactive and is_visible:
 elements.append(node)

 for child in node.children:
 self._collect_interactive_elements(child, elements)

def _has_interactive_descendants(self, node: SimplifiedNode) -> bool:
 """Check if a node has any interactive descendants (not including the node itself)."""
 # Check children for interactivity
 for child in node.children:
 # Check if child itself is interactive
 if self._is_interactive_cached(child.original_node):
 return True
 # Recursively check child's descendants
 if self._has_interactive_descendants(child):
 return True

 return False

def _is_inside_shadow_dom(self, node: SimplifiedNode) -> bool:
 """Check if a node is inside a shadow DOM by walking up the parent chain.

 Shadow DOM elements are descendants of a #document-fragment node (shadow root).
 The shadow root node has node_type == DOCUMENT_FRAGMENT_NODE and shadow_root_type set.
 """
 current = node.original_node.parent_node
 while current is not None:
 # Shadow roots are DOCUMENT_FRAGMENT nodes with shadow_root_type
 if current.node_type == NodeType.DOCUMENT_FRAGMENT_NODE and current.shadow_root_type is not None:
 return True
 current = current.parent_node
 return False

```

```

 current = current.parent_node
 return False

def _assign_interactive_indices_and_mark_new_nodes(self, node: SimplifiedNode | None) -> None:
 """Assign interactive indices to clickable elements that are also visible."""
 if not node:
 return

 ## Skip assigning index to excluded nodes, or ignored by paint order
 if not node.excluded_by_parent and not node.ignored_by_paint_order:
 ## Regular interactive element assignment (including enhanced compound controls)
 is_interactive_assign = self._is_interactive_cached(node.original_node)
 is_visible = node.original_node.snapshot_node and node.original_node.is_visible
 is_scrollable = node.original_node.is_actually_scrollable

 ## DIAGNOSTIC: Log when interactive elements don't have snapshot_node
 if is_interactive_assign and not node.original_node.snapshot_node:
 import logging

 logger = logging.getLogger('system.dom.serializer')
 attrs = node.original_node.attributes or {}
 attr_str = f'name={attrs.get("name", "")} id={attrs.get("id", "")} type={attrs.get("type", "")}'
 in_shadow = self._is_inside_shadow_dom(node)
 if (
 in_shadow
 and node.original_node.tag_name
 and node.original_node.tag_name.lower() in ['input', 'button', 'select', 'textarea', 'a']
):
 logger.debug(
 f'INCLUDING shadow DOM <{node.original_node.tag_name}> (no snapshot_node but in shadow DOM): '
 f'backendNodeId={node.original_node.backend_node_id} {attr_str}'
)
 else:
 logger.debug(
 f'SKIPPING interactive <{node.original_node.tag_name}> (no snapshot_node, not in shadow DOM): '
 f'backendNodeId={node.original_node.backend_node_id} {attr_str}'
)

 ## EXCEPTION: File inputs are often hidden with opacity:0 but are still functional
 ## Bootstrap and other frameworks use this pattern with custom-styled file pickers
 is_file_input = (
 node.original_node.tag_name
 and node.original_node.tag_name.lower() == 'input'
 and node.original_node.attributes
 and node.original_node.attributes.get('type') == 'file'
)

 ## EXCEPTION: Shadow DOM form elements may not have snapshot layout data from CDP's
 ## DOMSnapshot.captureSnapshot, but they're still functional/interactive.
 ## This handles login forms, custom web components, etc. inside shadow DOM.
 is_shadow_dom_element = (
 is_interactive_assign
 and not node.original_node.snapshot_node
 and node.original_node.tag_name
 and node.original_node.tag_name.lower() in ['input', 'button', 'select', 'textarea', 'a']
 and self._is_inside_shadow_dom(node)
)

 ## Check if scrollable container should be made interactive
 ## For scrollable elements, ONLY make them interactive if they have no interactive descendants
 should_make_interactive = False
 if is_scrollable:
 ## Check if this is a dropdown container that needs to be indexed regardless of descendants
 attrs = node.original_node.attributes or {}
 role = attrs.get('role', '').lower()
 tag_name = (node.original_node.tag_name or '').lower()
 class_attr = attrs.get('class', '').lower()
 class_list = class_attr.split() if class_attr else []

 ## Detect dropdown containers by role, tag, or class
 is_dropdown_by_role = role in ('listbox', 'menu', 'combobox', 'menubar', 'tree', 'grid')
 is_dropdown_by_tag = tag_name == 'select'

```

```

Match common dropdown class patterns
is_dropdown_by_class = (
'dropdown' in class_list
or 'dropdown-menu' in class_list
or 'select-menu' in class_list
or ('ui' in class_list and 'dropdown' in class_attr) # Semantic UI
#####)
is_dropdown_container = is_dropdown_by_role or is_dropdown_by_tag or is_dropdown_by_class

if is_dropdown_container:
Always index dropdown containers - need to be targetable for select_dropdown
should_make_interactive = True
else:
For other scrollable elements, check if they have interactive children
has_interactive_desc = self._has_interactive_descendants(node)
Only make scrollable container interactive if it has no interactive descendants
if not has_interactive_desc:
should_make_interactive = True
elif is_interactive_assign and (is_visible or is_file_input or is_shadow_dom_element):
Non-scrollable interactive elements: make interactive if visible (or file input or shadow DOM for
should_make_interactive = True

Add to selector map if element should be interactive
if should_make_interactive:
Mark node as interactive
node.is_interactive = True
Store backend_node_id in selector map (model outputs backend_node_id)
self._selector_map[node.original_node.backend_node_id] = node.original_node
self._interactive_counter += 1

Mark compound components as new for visibility
if node.is_compound_component:
node.is_new = True
elif self._previous_cached_selector_map:
Check if node is new for regular elements
previous_backend_node_ids = {node.backend_node_id for node in self._previous_cached_selector_map}
if node.original_node.backend_node_id not in previous_backend_node_ids:
node.is_new = True

Process children
for child in node.children:
self._assign_interactive_indices_and_mark_new_nodes(child)

def _apply_bounding_box_filtering(self, node: SimplifiedNode | None) -> SimplifiedNode | None:
"""Filter children contained within propagating parent bounds."""
if not node:
return None

Start with no active bounds
self._filter_tree_recursive(node, active_bounds=None, depth=0)

Log statistics
excluded_count = self._count_excluded_nodes(node)
if excluded_count > 0:
import logging

logging.debug(f'BBox filtering excluded {excluded_count} nodes')

return node

def _filter_tree_recursive(self, node: SimplifiedNode, active_bounds: PropagatingBounds | None = None,
"""
Recursively filter tree with bounding box propagation.
Bounds propagate to ALL descendants until overridden.
"""

Check if this node should be excluded by active bounds
if active_bounds and self._should_exclude_child(node, active_bounds):
node.excluded_by_parent = True
Important: Still check if this node starts NEW propagation

Check if this node starts new propagation (even if excluded!)

```



```

new_bounds = None
tag = node.original_node.tag_name.lower()
role = node.original_node.attributes.get('role') if node.original_node.attributes else None
attributes = {
 'tag': tag,
 'role': role,
}
Check if this element matches any propagating element pattern
if self._is_propagating_element(attributes):
 ## This node propagates bounds to ALL its descendants
 if node.original_node.snapshot_node and node.original_node.snapshot_node.bounds:
 new_bounds = PropagatingBounds(
 tag=tag,
 bounds=node.original_node.snapshot_node.bounds,
 node_id=node.original_node.node_id,
 depth=depth,
)

Propagate to ALL children
Use new_bounds if this node starts propagation, otherwise continue with active_bounds
propagate_bounds = new_bounds if new_bounds else active_bounds

for child in node.children:
 self._filter_tree_recursive(child, propagate_bounds, depth + 1)

def _should_exclude_child(self, node: SimplifiedNode, active_bounds: PropagatingBounds) -> bool:
 """
 Determine if child should be excluded based on propagating bounds.
 """

 ## Never exclude text nodes - we always want to preserve text content
 if node.original_node.node_type == NodeType.TEXT_NODE:
 return False

 ## Get child bounds
 if not node.original_node.snapshot_node or not node.original_node.snapshot_node.bounds:
 return False # No bounds = can't determine containment

 child_bounds = node.original_node.snapshot_node.bounds

 ## Check containment with configured threshold
 if not self._is_contained(child_bounds, active_bounds.bounds, self.containment_threshold):
 return False # Not sufficiently contained

 ## EXCEPTION RULES - Keep these even if contained:

 child_tag = node.original_node.tag_name.lower()
 child_role = node.original_node.attributes.get('role') if node.original_node.attributes else None
 child_attributes = {
 'tag': child_tag,
 'role': child_role,
 }

 ## 1. Never exclude form elements (they need individual interaction)
 if child_tag in ['input', 'select', 'textarea', 'label']:
 return False

 ## 2. Keep if child is also a propagating element
 ## (might have stopPropagation, e.g., button in button)
 if self._is_propagating_element(child_attributes):
 return False

 ## 3. Keep if has explicit onclick handler
 if node.original_node.attributes and 'onclick' in node.original_node.attributes:
 return False

 ## 4. Keep if has aria-label suggesting it's independently interactive
 if node.original_node.attributes:
 aria_label = node.original_node.attributes.get('aria-label')
 if aria_label and aria_label.strip():
 ## Has meaningful aria-label, likely interactive
 return False

```

```

 ## 5. Keep if has role suggesting interactivity
 if node.original_node.attributes:
 role = node.original_node.attributes.get('role')
 if role in ['button', 'link', 'checkbox', 'radio', 'tab', 'menuitem', 'option']:
 return False

 ## Default: exclude this child
 return True

def _is_contained(self, child: DOMRect, parent: DOMRect, threshold: float) -> bool:
 """
 Check if child is contained within parent bounds.

 Args:
 threshold: Percentage (0.0-1.0) of child that must be within parent
 """
 ## Calculate intersection
 x_overlap = max(0, min(child.x + child.width, parent.x + parent.width) - max(child.x, parent.x))
 y_overlap = max(0, min(child.y + child.height, parent.y + parent.height) - max(child.y, parent.y))

 intersection_area = x_overlap * y_overlap
 child_area = child.width * child.height

 if child_area == 0:
 return False # Zero-area element

 containment_ratio = intersection_area / child_area
 return containment_ratio >= threshold

def _count_excluded_nodes(self, node: SimplifiedNode, count: int = 0) -> int:
 """Count how many nodes were excluded (for debugging)."""
 if hasattr(node, 'excluded_by_parent') and node.excluded_by_parent:
 count += 1
 for child in node.children:
 count = self._count_excluded_nodes(child, count)
 return count

def _is_propagating_element(self, attributes: dict[str, str | None]) -> bool:
 """
 Check if an element should propagate bounds based on attributes.
 If the element satisfies one of the patterns, it propagates bounds to all its children.
 """
 keys_to_check = ['tag', 'role']
 for pattern in self.PROPAGATING_ELEMENTS:
 ## Check if the element satisfies the pattern
 check = [pattern.get(key) is None or pattern.get(key) == attributes.get(key) for key in keys_to_check]
 if all(check):
 return True

 return False

 @staticmethod
 def serialize_tree(node: SimplifiedNode | None, include_attributes: list[str], depth: int = 0) -> str:
 """Serialize the optimized tree to string format."""
 if not node:
 return ''

 ## Skip rendering excluded nodes, but process their children
 if hasattr(node, 'excluded_by_parent') and node.excluded_by_parent:
 formatted_text = []
 for child in node.children:
 child_text = DOMTreeSerializer.serialize_tree(child, include_attributes, depth)
 if child_text:
 formatted_text.append(child_text)
 return '\n'.join(formatted_text)

 formatted_text = []
 depth_str = depth * '\t'
 next_depth = depth

 if node.original_node.node_type == NodeType.ELEMENT_NODE:
 ## Skip displaying nodes marked as should_display=False

```

```

#####if not node.should_display:
#####for child in node.children:
#####child_text = DOMTreeSerializer.serialize_tree(child, include_attributes, depth)
#####if child_text:
#####formatted_text.append(child_text)
#####return '\n'.join(formatted_text)

Special handling for SVG elements - show the tag but collapse children
####if node.original_node.tag_name.lower() == 'svg':
####shadow_prefix = ''
####if node.is_shadow_host:
#####has_closed_shadow = any(
#####child.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE
#####and child.original_node.shadow_root_type
#####and child.original_node.shadow_root_type.lower() == 'closed'
#####)for child in node.children
#####)
#####shadow_prefix = '|SHADOW(closed)|' if has_closed_shadow else '|SHADOW(open)|'

#####line = f'{depth_str}{shadow_prefix}'
Add interactive marker if clickable
#####if node.is_interactive:
#####new_prefix = '*' if node.is_new else ''
#####line += f'{new_prefix}[{node.original_node.backend_node_id}]'
#####line += '<svg'
#####attributes_html_str = DOMTreeSerializer._build_attributes_string(node.original_node, include_attri
#####if attributes_html_str:
#####line += f' {attributes_html_str}'
#####line += ' /> <!-- SVG content collapsed -->'
#####formatted_text.append(line)
Don't process children for SVG
#####return '\n'.join(formatted_text)

Add element if clickable, scrollable, or iframe
####is_any_scrollable = node.original_node.is_actually_scrollable or node.original_node.is_scrollable
####should_show_scroll = node.original_node.should_show_scroll_info
####if (
####node.is_interactive
####or is_any_scrollable
####or node.original_node.tag_name.upper() == 'IFRAME'
####or node.original_node.tag_name.upper() == 'FRAME'
####):
####next_depth += 1

Build attributes string with compound component info
####text_content = ''
####attributes_html_str = DOMTreeSerializer._build_attributes_string(
####node.original_node, include_attributes, text_content
####)

Add compound component information to attributes if present
####if node.original_node._compound_children:
####compound_info = []
####for child_info in node.original_node._compound_children:
#####parts = []
#####if child_info['name']:
#####parts.append(f'name={child_info["name"]}')
#####if child_info['role']:
#####parts.append(f'role={child_info["role"]}')
#####if child_info['valuemin'] is not None:
#####parts.append(f'min={child_info["valuemin"]}')
#####if child_info['valuemax'] is not None:
#####parts.append(f'max={child_info["valuemax"]}')
#####if child_info['valuenow'] is not None:
#####parts.append(f'current={child_info["valuenow"]}')

Add select-specific information
#####if 'options_count' in child_info and child_info['options_count'] is not None:
#####parts.append(f'count={child_info["options_count"]}')
#####if 'first_options' in child_info and child_info['first_options']:
#####options_str = '|'.join(child_info['first_options'][:4]) # Limit to 4 options
#####parts.append(f'options={options_str}')

```

```

#####if 'format_hint' in child_info and child_info['format_hint']:
#####parts.append(f'format={child_info["format_hint"]}')

#####if parts:
#####compound_info.append(f'({",".join(parts)})')

#####if compound_info:
#####compound_attr = f'compound_components={",".join(compound_info)}'
#####if attributes_html_str:
#####attributes_html_str += f' {compound_attr}'
#####else:
#####attributes_html_str = compound_attr

Build the line with shadow host indicator
#####shadow_prefix = ''
#####if node.is_shadow_host:
Check if any shadow children are closed
#####has_closed_shadow = any(
#####child.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE
#####and child.original_node.shadow_root_type
#####and child.original_node.shadow_root_type.lower() == 'closed'
#####for child in node.children
#####)
#####shadow_prefix = '|SHADOW(closed)|' if has_closed_shadow else '|SHADOW(open)|'

#####if should_show_scroll and not node.is_interactive:
Scrollable container but not clickable
#####line = f'{depth_str}{shadow_prefix}|scroll element|<{node.original_node.tag_name}'
#####elif node.is_interactive:
Clickable (and possibly scrollable) - show backend_node_id
#####new_prefix = '*' if node.is_new else ''
#####scroll_prefix = '|scroll element[' if should_show_scroll else '['
#####line = f'{depth_str}{shadow_prefix}{new_prefix}{scroll_prefix}{node.original_node.backend_node_id}'
#####elif node.original_node.tag_name.upper() == 'IFRAME':
Iframe element (not interactive)
#####line = f'{depth_str}{shadow_prefix}|IFRAME|<{node.original_node.tag_name}'
#####elif node.original_node.tag_name.upper() == 'FRAME':
Frame element (not interactive)
#####line = f'{depth_str}{shadow_prefix}|FRAME|<{node.original_node.tag_name}'
#####else:
#####line = f'{depth_str}{shadow_prefix}<{node.original_node.tag_name}'

#####if attributes_html_str:
#####line += f' {attributes_html_str}'

#####line += ' />'

Add scroll information only when we should show it
#####if should_show_scroll:
#####scroll_info_text = node.original_node.get_scroll_info_text()
#####if scroll_info_text:
#####line += f' ({scroll_info_text})'

#####formatted_text.append(line)

#####elif node.original_node.node_type == NodeType.DOCUMENT_FRAGMENT_NODE:
Shadow DOM representation - show clearly to LLM
#####if node.original_node.shadow_root_type and node.original_node.shadow_root_type.lower() == 'closed':
#####formatted_text.append(f'{depth_str}Closed Shadow')
#####else:
#####formatted_text.append(f'{depth_str}Open Shadow')

#####next_depth += 1

Process shadow DOM children
#####for child in node.children:
#####child_text = DOMTreeSerializer.serialize_tree(child, include_attributes, next_depth)
#####if child_text:
#####formatted_text.append(child_text)

Close shadow DOM indicator
#####if node.children: # Only show close if we had content

```

```

#####formatted_text.append(f'{depth_str}Shadow End')

#####elif node.original_node.node_type == NodeType.TEXT_NODE:
Include visible text
#####is_visible = node.original_node.snapshot_node and node.original_node.is_visible
#####if (
#####is_visible
#####and node.original_node.node_value
#####and node.original_node.node_value.strip()
#####and len(node.original_node.node_value.strip()) > 1
#####):
#####clean_text = node.original_node.node_value.strip()
#####formatted_text.append(f'{depth_str}{clean_text}')

Process children (for non-shadow elements)
#####if node.original_node.node_type != NodeType.DOCUMENT_FRAGMENT_NODE:
#####for child in node.children:
#####child_text = DOMTreeSerializer.serialize_tree(child, include_attributes, next_depth)
#####if child_text:
#####formatted_text.append(child_text)

Add hidden content hint for iframes
#####if (
#####node.original_node.node_type == NodeType.ELEMENT_NODE
#####and node.original_node.tag_name
#####and node.original_node.tag_name.upper() in ('IFRAME', 'FRAME')
#####):
#####if node.original_node.hidden_elements_info:
Show specific interactive elements with scroll distances
#####hidden = node.original_node.hidden_elements_info
#####hint_lines = [f'{depth_str}... ({len(hidden)} more elements below - scroll to reveal):']
#####for elem in hidden:
#####hint_lines.append(f'{depth_str} <{elem["tag"]} > "{elem["text"]}" ~{elem["pages"]} pages down')
#####formatted_text.extend(hint_lines)
#####elif node.original_node.has_hidden_content:
Generic hint for non-interactive hidden content
#####formatted_text.append(f'{depth_str}... (more content below viewport - scroll to reveal)')

#####return '\n'.join(formatted_text)

#####@staticmethod
#####def _build_attributes_string(node: EnhancedDOMTreeNode, include_attributes: list[str], text: str) -> str:
#####"""Build the attributes string for an element."""
#####attributes_to_include = {}

Include HTML attributes
#####if node.attributes:
#####attributes_to_include.update(
#####{
#####key: str(value).strip()
#####for key, value in node.attributes.items()
#####if key in include_attributes and str(value).strip() != ''
#####}
#####)

Add format hints for date/time inputs to help LLMs use the correct format
NOTE: These formats are standardized by HTML5 specification (ISO 8601), NOT locale-dependent
The browser may DISPLAY dates in locale format (MM/DD/YYYY in US, DD/MM/YYYY in EU),
but the .value attribute and programmatic setting ALWAYS uses these ISO formats:
- date: YYYY-MM-DD (e.g., "2024-03-15")
- time: HH:MM or HH:MM:SS (24-hour, e.g., "14:30")
- datetime-local: YYYY-MM-DDTHH:MM (e.g., "2024-03-15T14:30")
Reference: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input/date
#####if node.tag_name and node.tag_name.lower() == 'input' and node.attributes:
#####input_type = node.attributes.get('type', '').lower()

For HTML5 date/time inputs, add a highly visible "format" attribute
This makes it IMPOSSIBLE for the model to miss the required format
#####if input_type in ['date', 'time', 'datetime-local', 'month', 'week']:
#####format_map = {
#####'date': 'YYYY-MM-DD',
#####'time': 'HH:MM',

```

```

'datetime-local': 'YYYY-MM-DDTHH:MM',
'month': 'YYYY-MM',
'week': 'YYYY-W##',
#####}
Add format as a special attribute that appears prominently
This appears BEFORE placeholder in the serialized output
attributes_to_include['format'] = format_map[input_type]

Only add placeholder if it doesn't already exist
if 'placeholder' in include_attributes and 'placeholder' not in attributes_to_include:
Native HTML5 date/time inputs - ISO format required
if input_type == 'date':
attributes_to_include['placeholder'] = 'YYYY-MM-DD'
elif input_type == 'time':
attributes_to_include['placeholder'] = 'HH:MM'
elif input_type == 'datetime-local':
attributes_to_include['placeholder'] = 'YYYY-MM-DDTHH:MM'
elif input_type == 'month':
attributes_to_include['placeholder'] = 'YYYY-MM'
elif input_type == 'week':
attributes_to_include['placeholder'] = 'YYYY-W##'
Tel - suggest format if no pattern attribute
elif input_type == 'tel' and 'pattern' not in attributes_to_include:
attributes_to_include['placeholder'] = '123-456-7890'
jQuery/Bootstrap/AngularJS datepickers (text inputs with datepicker classes/attributes)
elif input_type in {'text', ''}:
class_attr = node.attributes.get('class', '').lower()

Check for AngularJS UI Bootstrap datepicker (uib-datepicker-popup attribute)
This takes precedence as it's the most specific indicator
if 'uib-datepicker-popup' in node.attributes:
Extract format from uib-datepicker-popup="MM/dd/yyyy"
date_format = node.attributes.get('uib-datepicker-popup', '')
if date_format:
Use 'expected_format' for clarity - this is the required input format
attributes_to_include['expected_format'] = date_format
Also keep format for consistency with HTML5 date inputs
attributes_to_include['format'] = date_format
Detect jQuery/Bootstrap datepickers by class names
elif any(indicator in class_attr for indicator in ['datepicker', 'datetimepicker', 'daterangepicker'])
Try to get format from data-date-format attribute
date_format = node.attributes.get('data-date-format', '')
if date_format:
attributes_to_include['placeholder'] = date_format
attributes_to_include['format'] = date_format # Also add format for jQuery datepickers
else:
Default to common US format for jQuery datepickers
attributes_to_include['placeholder'] = 'mm/dd/yyyy'
attributes_to_include['format'] = 'mm/dd/yyyy'
Also detect by data-* attributes
elif any(attr in node.attributes for attr in ['data-datepicker']):
date_format = node.attributes.get('data-date-format', '')
if date_format:
attributes_to_include['placeholder'] = date_format
attributes_to_include['format'] = date_format
else:
attributes_to_include['placeholder'] = 'mm/dd/yyyy'
attributes_to_include['format'] = 'mm/dd/yyyy'

Include accessibility properties
if node.ax_node and node.ax_node.properties:
for prop in node.ax_node.properties:
try:
if prop.name in include_attributes and prop.value is not None:
Convert boolean to lowercase string, keep others as-is
if isinstance(prop.value, bool):
attributes_to_include[prop.name] = str(prop.value).lower()
else:
prop_value_str = str(prop.value).strip()
if prop_value_str:
attributes_to_include[prop.name] = prop_value_str
except (AttributeError, ValueError):

```

```

#####continue

Special handling for form elements - ensure current value is shown
For text inputs, textareas, and selects, prioritize showing the current value from AX tree
if node.tag_name and node.tag_name.lower() in ['input', 'textarea', 'select']:
ALWAYS check AX tree - it reflects actual typed value, DOM attribute may not update
if node.ax_node and node.ax_node.properties:
#####for prop in node.ax_node.properties:
Try valuetext first (human-readable display value)
#####if prop.name == 'valuetext' and prop.value:
#####value_str = str(prop.value).strip()
#####if value_str:
#####attributes_to_include['value'] = value_str
#####break
Also try 'value' property directly
#####elif prop.name == 'value' and prop.value:
#####value_str = str(prop.value).strip()
#####if value_str:
#####attributes_to_include['value'] = value_str
#####break

if not attributes_to_include:
return ''

Remove duplicate values
ordered_keys = [key for key in include_attributes if key in attributes_to_include]

if len(ordered_keys) > 1:
keys_to_remove = set()
seen_values = {}

Attributes that should never be removed as duplicates (they serve distinct purposes)
protected_attrs = {'format', 'expected_format', 'placeholder', 'value', 'aria-label', 'title'}

for key in ordered_keys:
value = attributes_to_include[key]
if len(value) > 5:
#####if value in seen_values and key not in protected_attrs:
#####keys_to_remove.add(key)
#####else:
#####seen_values[value] = key

for key in keys_to_remove:
del attributes_to_include[key]

Remove attributes that duplicate accessibility data
role = node.ax_node.role if node.ax_node else None
if role and node.node_name == role:
attributes_to_include.pop('role', None)

Remove type attribute if it matches the tag name (e.g. <button type="button">)
if 'type' in attributes_to_include and attributes_to_include['type'].lower() == node.node_name.lower():
del attributes_to_include['type']

Remove invalid attribute if it's false (only show when true)
if 'invalid' in attributes_to_include and attributes_to_include['invalid'].lower() == 'false':
del attributes_to_include['invalid']

boolean_attrs = {'required'}
for attr in boolean_attrs:
if attr in attributes_to_include and attributes_to_include[attr].lower() in {'false', '0', 'no'}:
del attributes_to_include[attr]

Remove aria-expanded if we have expanded (prefer AX tree over HTML attribute)
if 'expanded' in attributes_to_include and 'aria-expanded' in attributes_to_include:
del attributes_to_include['aria-expanded']

attrs_to_remove_if_text_matches = ['aria-label', 'placeholder', 'title']
for attr in attrs_to_remove_if_text_matches:
if attributes_to_include.get(attr) and attributes_to_include.get(attr, '').strip().lower() == text.s
del attributes_to_include[attr]
if attributes_to_include:

```

```

Format attributes, wrapping empty values in quotes for clarity
#####formatted_attrs = []
#####for key, value in attributes_to_include.items():
#####capped_value = cap_text_length(value, 100)
Show empty values as key='' instead of key=
#####if not capped_value:
#####formatted_attrs.append(f'{key}=''')
#####else:
#####formatted_attrs.append(f'{key}={capped_value}')
#####return ' '.join(formatted_attrs)

#####return ''

```

## ***.\system\filesystem\file\_system.py***

```

import asyncio
import base64
import csv
import io
import os
import re
import shutil
from abc import ABC, abstractmethod
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path
from typing import Any

from pydantic import BaseModel, Field

UNSUPPORTED_BINARY_EXTENSIONS = {
 'png',
 'jpg',
 'jpeg',
 'gif',
 'bmp',
 'svg',
 'webp',
 'ico',
 'mp3',
 'mp4',
 'wav',
 'avi',
 'mov',
 'zip',
 'tar',
 'gz',
 'rar',
 'exe',
 'bin',
 'dll',
 'so',
}

def _build_filename_error_message(file_name: str, supported_extensions: list[str]) -> str:
 """Build a specific error message explaining why the filename was rejected and how to fix it."""
 base = os.path.basename(file_name)

 # Check for binary/image extension
 if '.' in base:
 _, ext = base.rsplit('.', 1)
 ext_lower = ext.lower()
 if ext_lower in UNSUPPORTED_BINARY_EXTENSIONS:
 return (
 f"Error: Cannot write binary/image file '{base}'."
 f"The write_file tool only supports text-based files. "
 f"Supported extensions: {', '.join('.' + e for e in supported_extensions)}. "
 f"For screenshots, the browser automatically captures them - do not try to save screenshots as file

```



```

)
 if ext_lower not in supported_extensions:
 return (
 f"Error: Unsupported file extension '{ext_lower}' in '{base}'. "
 f"Supported extensions: {", ".join("." + e for e in supported_extensions)}."
 f"Please rename the file to use a supported extension."
)

 ## No extension or no dot
 if '.' not in base:
 return (
 f"Error: Filename '{base}' has no extension. "
 f"Please add a supported extension: {", ".join"." + e for e in supported_extensions)}."
)

 return (
 f"Error: Invalid filename '{base}'. "
 f"Filenames must contain only letters, numbers, underscores, hyphens, dots, parentheses, and spaces. "
 f"Supported extensions: {", ".join"." + e for e in supported_extensions)}."
)

DEFAULT_FILE_SYSTEM_PATH = 'browseruse_agent_data'

class FileSystemError(Exception):
 """Custom exception for file system operations that should be shown to LLM"""

 pass

class BaseFile(BaseModel, ABC):
 """Base class for all file types"""

 name: str
 content: str = ''

 ## --- Subclass must define this ---
 @property
 @abstractmethod
 def extension(self) -> str:
 """File extension (e.g. 'txt', 'md')"""
 pass

 def write_file_content(self, content: str) -> None:
 """Update internal content (formatted)"""
 self.update_content(content)

 def append_file_content(self, content: str) -> None:
 """Append content to internal content"""
 self.update_content(self.content + content)

 ## --- These are shared and implemented here ---

 def update_content(self, content: str) -> None:
 self.content = content

 def sync_to_disk_sync(self, path: Path) -> None:
 file_path = path / self.full_name
 file_path.write_text(self.content)

 async def sync_to_disk(self, path: Path) -> None:
 file_path = path / self.full_name
 with ThreadPoolExecutor() as executor:
 await asyncio.get_event_loop().run_in_executor(executor, lambda: file_path.write_text(self.content))

 async def write(self, content: str, path: Path) -> None:
 self.write_file_content(content)
 await self.sync_to_disk(path)

 async def append(self, content: str, path: Path) -> None:
 self.append_file_content(content)

```

```

 await self.sync_to_disk(path)

 def read(self) -> str:
 return self.content

 @property
 def full_name(self) -> str:
 return f'{self.name}.{self.extension}'

 @property
 def get_size(self) -> int:
 return len(self.content)

 @property
 def get_line_count(self) -> int:
 return len(self.content.splitlines())

class MarkdownFile(BaseFile):
 """Markdown file implementation"""

 @property
 def extension(self) -> str:
 return 'md'

class TxtFile(BaseFile):
 """Plain text file implementation"""

 @property
 def extension(self) -> str:
 return 'txt'

class JsonFile(BaseFile):
 """JSON file implementation"""

 @property
 def extension(self) -> str:
 return 'json'

class CsvFile(BaseFile):
 """CSV file implementation with automatic RFC 4180 normalization.

 LLMs frequently produce malformed CSV (missing quotes around fields with commas,
 inconsistent empty fields, unescaped internal quotes). This class parses the raw
 content through Python's csv module on every write to guarantee well-formed output.
 """

 @property
 def extension(self) -> str:
 return 'csv'

 @staticmethod
 def _normalize_csv(raw: str) -> str:
 """Parse and re-serialize CSV content to fix quoting, empty fields, and escaping.

 Handles common LLM mistakes: unquoted fields containing commas,
 unescaped quotes inside fields, inconsistent empty fields,
 trailing/leading blank lines, and double-escaped JSON output
 (literal backslash-n and backslash-quote instead of real newlines/quotes).
 """
 stripped = raw.strip('\n\r')
 if not stripped:
 return raw

 ## Detect double-escaped LLM tool call output: if the content has no real
 ## newlines but contains literal \n sequences, the entire string is likely
 ## double-escaped JSON. Unescape \" -> " first, then \n -> newline.
 if '\n' not in stripped and '\\n' in stripped:
 stripped = stripped.replace('\\\"', '"')

```

```

■■■stripped = stripped.replace('\n', '\n')

■■reader = csv.reader(io.StringIO(stripped))
■■rows: list[list[str]] = []
■■for row in reader:
■■■# Skip completely empty rows (artifacts of blank lines)
■■■if row:
■■■■rows.append(row)

■■if not rows:
■■■return raw

■■out = io.StringIO()
■■writer = csv.writer(out, lineterminator='\n')
■■writer.writerows(rows)
■■# Strip trailing newline so callers (write_file action) control line endings
■■return out.getvalue().rstrip('\n')

■■def write_file_content(self, content: str) -> None:
■■■"""Normalize CSV content before storing."""
■■■self.update_content(self._normalize_csv(content))

■■def append_file_content(self, content: str) -> None:
■■■"""Normalize the appended CSV rows and merge with existing content."""
■■■normalized_new = self._normalize_csv(content)
■■■if not normalized_new.strip('\n\r'):
■■■return
■■■existing = self.content
■■■if existing and not existing.endswith('\n'):
■■■existing += '\n'
■■■combined = existing + normalized_new
■■■self.update_content(self._normalize_csv(combined))

class JsonlFile(BaseFile):
■■■"""JSONL (JSON Lines) file implementation"""

■■@property
■■def extension(self) -> str:
■■return 'jsonl'

class PdfFile(BaseFile):
■■■"""PDF file implementation"""

■■@property
■■def extension(self) -> str:
■■return 'pdf'

■■def sync_to_disk_sync(self, path: Path) -> None:
■■■# Lazy import reportlab
■■■from reportlab.lib.pagesizes import letter
■■■from reportlab.lib.styles import getSampleStyleSheet
■■■from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer

■■file_path = path / self.full_name
■■try:
■■■# Create PDF document
■■■doc = SimpleDocTemplate(str(file_path), pagesize=letter)
■■■styles = getSampleStyleSheet()
■■■story = []

■■■# Convert markdown content to simple text and add to PDF
■■■# For basic implementation, we'll treat content as plain text
■■■# This avoids the AGPL license issue while maintaining functionality
■■■content_lines = self.content.split('\n')

■■■for line in content_lines:
■■■■if line.strip():
■■■■■# Handle basic markdown headers
■■■■■if line.startswith('# '):
■■■■■para = Paragraph(line[2:], styles['Title'])

```

```

 elif line.startswith('## '):
 para = Paragraph(line[3:], styles['Heading1'])
 elif line.startswith('### '):
 para = Paragraph(line[4:], styles['Heading2'])
 else:
 para = Paragraph(line, styles['Normal'])
 story.append(para)
 else:
 story.append(Spacer(1, 6))

 doc.build(story)
except Exception as e:
 raise FilesystemError(f"Error: Could not write to file '{self.full_name}'. {str(e)}")

async def sync_to_disk(self, path: Path) -> None:
 with ThreadPoolExecutor() as executor:
 await asyncio.get_event_loop().run_in_executor(executor, lambda: self.sync_to_disk_sync(path))

class DocxFile(BaseFile):
 """DOCX file implementation"""

 @property
 def extension(self) -> str:
 return 'docx'

 def sync_to_disk_sync(self, path: Path) -> None:
 file_path = path / self.full_name
 try:
 from docx import Document

 doc = Document()

 # Convert content to DOCX paragraphs
 content_lines = self.content.split('\n')

 for line in content_lines:
 if line.strip():
 # Handle basic markdown headers
 if line.startswith('# '):
 doc.add_heading(line[2:], level=1)
 elif line.startswith('## '):
 doc.add_heading(line[3:], level=2)
 elif line.startswith('### '):
 doc.add_heading(line[4:], level=3)
 else:
 doc.add_paragraph(line)
 else:
 doc.add_paragraph() # Empty paragraph for spacing

 doc.save(str(file_path))
 except Exception as e:
 raise FilesystemError(f"Error: Could not write to file '{self.full_name}'. {str(e)}")

 async def sync_to_disk(self, path: Path) -> None:
 with ThreadPoolExecutor() as executor:
 await asyncio.get_event_loop().run_in_executor(executor, lambda: self.sync_to_disk_sync(path))

class HtmlFile(BaseFile):
 """HTML file implementation"""

 @property
 def extension(self) -> str:
 return 'html'

class XmlFile(BaseFile):
 """XML file implementation"""

 @property
 def extension(self) -> str:

```

```

 return 'xml'

class FileSystemState(BaseModel):
 """Serializable state of the file system"""

 files: dict[str, dict[str, Any]] = Field(default_factory=dict) # full filename -> file data
 base_dir: str
 extracted_content_count: int = 0

class FileSystem:
 """Enhanced file system with in-memory storage and multiple file type support"""

 def __init__(self, base_dir: str | Path, create_default_files: bool = True):
 """# Handle the Path conversion before calling super().__init__"""
 self.base_dir = Path(base_dir) if isinstance(base_dir, str) else base_dir
 self.base_dir.mkdir(parents=True, exist_ok=True)

 """# Create and use a dedicated subfolder for all operations"""
 self.data_dir = self.base_dir / DEFAULT_FILE_SYSTEM_PATH
 if self.data_dir.exists():
 """# clean the data directory"""
 shutil.rmtree(self.data_dir)
 self.data_dir.mkdir(exist_ok=True)

 self._file_types: dict[str, type[BaseFile]] = {
 'md': MarkdownFile,
 'txt': TxtFile,
 'json': JsonFile,
 'jsonl': JsonlFile,
 'csv': CsvFile,
 'pdf': PdfFile,
 'docx': DocxFile,
 'html': HtmlFile,
 'xml': XmlFile,
 }

 self.files = {}
 if create_default_files:
 self.default_files = ['todo.md']
 self._create_default_files()

 self.extracted_content_count = 0

 def get_allowed_extensions(self) -> list[str]:
 """Get allowed extensions"""
 return list(self._file_types.keys())

 def _get_file_type_class(self, extension: str) -> type[BaseFile] | None:
 """Get the appropriate file class for an extension."""
 return self._file_types.get(extension.lower(), None)

 def _create_default_files(self) -> None:
 """Create default results and todo files"""
 for full_filename in self.default_files:
 name_without_ext, extension = self._parse_filename(full_filename)
 file_class = self._get_file_type_class(extension)
 if not file_class:
 raise ValueError(f"Error: Invalid file extension '{extension}' for file '{full_filename}'.")

 file_obj = file_class(name=name_without_ext)
 self.files[full_filename] = file_obj # Use full filename as key
 file_obj.sync_to_disk_sync(self.data_dir)

 def _is_valid_filename(self, file_name: str) -> bool:
 """Check if filename matches the required pattern: name.extension

 Allows letters, numbers, underscores, hyphens, dots, parentheses, spaces, and Chinese characters
 in the name part, followed by a dot and a supported extension.
 """
 extensions = '|'.join(self._file_types.keys())

```

```

 ■■# Allow dots, spaces, parens in the name part - match everything up to the last dot
 ■■pattern = rf'^[a-zA-Z0-9_\\.\(\) \u4e00-\u9fff]+\.{extensions}$'
 ■■file_name_base = os.path.basename(file_name)
 ■■if not re.match(pattern, file_name_base):
 ■■■return False
 ■■# Ensure the name part (before last dot) is non-empty
 ■■name_part = file_name_base.rsplit('.', 1)[0]
 ■■return len(name_part.strip()) > 0

 ■■@staticmethod
 ■■def sanitize_filename(file_name: str) -> str:
 ■■ """Sanitize a filename by replacing/removing invalid characters.

 ■■- Replaces spaces with hyphens
 ■■- Removes characters that are not alphanumeric, underscore, hyphen, dot, parentheses, or Chinese
 ■■- Preserves the extension
 ■■- Collapses multiple consecutive hyphens
 ■■ """
 ■■ base = os.path.basename(file_name)
 ■■ if '.' not in base:
 ■■■return base

 ■■ name_part, ext = base.rsplit('.', 1)
 ■■ # Replace spaces with hyphens
 ■■ name_part = name_part.replace(' ', '-')
 ■■ # Remove invalid characters (keep alphanumeric, underscore, hyphen, dot, parens, Chinese)
 ■■ name_part = re.sub(r'^[a-zA-Z0-9_\\.\(\) \u4e00-\u9fff]', '', name_part)
 ■■ # Collapse multiple hyphens
 ■■ name_part = re.sub(r'-{2,}', '-', name_part)
 ■■ # Strip leading/trailing hyphens and dots
 ■■ name_part = name_part.strip('-.')

 ■■ if not name_part:
 ■■■name_part = 'file'

 ■■ return f'{name_part}.{ext.lower()}'

 ■■def _resolve_filename(self, file_name: str) -> tuple[str, bool]:
 ■■ """Resolve a filename, attempting sanitization if the original is invalid.

 ■■Normalizes to basename first to prevent directory traversal (e.g. ../secret.md).

 ■■Returns:
 ■■■(resolved_name, was_changed): The resolved filename and whether it differs from the input.
 ■■■If resolution fails, returns (basename, was_changed).
 ■■ """
 ■■ base_name = os.path.basename(file_name)
 ■■ was_changed = base_name != file_name

 ■■ if self._is_valid_filename(base_name):
 ■■■return base_name, was_changed

 ■■ sanitized = self.sanitize_filename(base_name)
 ■■ if sanitized != base_name and self._is_valid_filename(sanitized):
 ■■■return sanitized, True

 ■■ return base_name, was_changed

 ■■def _parse_filename(self, filename: str) -> tuple[str, str]:
 ■■ """Parse filename into name and extension. Always check _is_valid_filename first."""
 ■■ name, extension = filename.rsplit('.', 1)
 ■■ return name, extension.lower()

 ■■def get_dir(self) -> Path:
 ■■ """Get the file system directory"""
 ■■ return self.data_dir

 ■■def get_file(self, full_filename: str) -> BaseFile | None:
 ■■ """Get a file object by full filename, trying sanitization if the name is invalid."""
 ■■ resolved, _ = self._resolve_filename(full_filename)
 ■■ if not self._is_valid_filename(resolved):
 ■■■return None

```

```

 ■■# Use resolved filename as key
 ■■return self.files.get(resolved)

 ■■def list_files(self) -> list[str]:
 ■■"""List all files in the system"""
 ■■return [file_obj.full_name for file_obj in self.files.values()]

 ■■def display_file(self, full_filename: str) -> str | None:
 ■■"""Display file content using file-specific display method"""
 ■■resolved, _ = self._resolve_filename(full_filename)
 ■■if not self._is_valid_filename(resolved):
 ■■ return None

 ■■file_obj = self.files.get(resolved)
 ■■if not file_obj:
 ■■ return None

 ■■return file_obj.read()

 ■■async def read_file_structured(self, full_filename: str, external_file: bool = False) -> dict[str, Any]:
 ■■"""Read file and return structured data including images if applicable.

 ■■Returns:
 ■■■dict with keys:
 ■■■- 'message': str - The message to display
 ■■■- 'images': list[dict] | None - Image data if file is an image: [{"name": str, "data": base64_str}]
 ■■"""
 ■■result: dict[str, Any] = {'message': '', 'images': None}

 ■■if external_file:
 ■■ try:
 ■■ try:
 ■■ _, extension = self._parse_filename(full_filename)
 ■■ except Exception:
 ■■ result['message'] = (
 ■■ f'Error: Invalid filename format {full_filename}. Must be alphanumeric with a supported extension'
 ■■)
 ■■ return result

 ■■ # Text-based extensions: derive from _file_types, excluding those with special readers
 ■■ _special_extensions = {'docx', 'pdf', 'jpg', 'jpeg', 'png'}
 ■■ text_extensions = [ext for ext in self._file_types if ext not in _special_extensions]

 ■■ if extension in text_extensions:
 ■■ import anyio

 ■■ async with await anyio.open_file(full_filename, 'r') as f:
 ■■ content = await f.read()
 ■■ result['message'] = f'Read from file {full_filename}.\n<content>\n{content}\n</content>'
 ■■ return result

 ■■ elif extension == 'docx':
 ■■ from docx import Document

 ■■ doc = Document(full_filename)
 ■■ content = '\n'.join([para.text for para in doc.paragraphs])
 ■■ result['message'] = f'Read from file {full_filename}.\n<content>\n{content}\n</content>'
 ■■ return result

 ■■ elif extension == 'pdf':
 ■■ import pypdf

 ■■ reader = pypdf.PdfReader(full_filename)
 ■■ num_pages = len(reader.pages)
 ■■ MAX_CHARS = 60000 # character-based limit

 ■■ # Extract text from all pages with page markers
 ■■ page_texts: list[tuple[int, str]] = []
 ■■ total_chars = 0
 ■■ for i, page in enumerate(reader.pages, 1):
 ■■ text = page.extract_text() or ''
 ■■ page_texts.append((i, text))

```

```

#####total_chars += len(text)

If small enough, return everything
#####if total_chars <= MAX_CHARS:
content_parts = []
for page_num, text in page_texts:
if text.strip():
content_parts.append(f'--- Page {page_num} ---\n{text}')
extracted_text = '\n\n'.join(content_parts)
result['message'] = (
f'Read from file {full_filename} ({num_pages} pages, {total_chars:,} chars).\n'
f'<content>\n{extracted_text}\n</content>'
)
return result

Large PDF - use search to prioritize pages with distinctive content
#####import math
#####import re

Extract words from each page and count which pages they appear on
#####word_to_pages: dict[str, set[int]] = {}
#####page_words: dict[int, set[str]] = {}

#####for page_num, text in page_texts:
Extract words (lowercase, 4+ chars to filter noise)
words = set(re.findall(r'\b[a-zA-Z]{4,}\b', text.lower()))
page_words[page_num] = words
for word in words:
if word not in word_to_pages:
word_to_pages[word] = set()
word_to_pages[word].add(page_num)

Score pages using inverse document frequency (IDF)
words appearing on fewer pages get higher weight
#####page_scores: dict[int, float] = {}
#####for page_num, words in page_words.items():
score = 0.0
for word in words:
pages_with_word = len(word_to_pages[word])
IDF: log(total_pages / pages_with_word) - higher for rarer words
score += math.log(num_pages / pages_with_word)
page_scores[page_num] = score

Sort pages by score (highest first), always include page 1
#####sorted_pages = sorted(page_scores.items(), key=lambda x: -x[1])
#####priority_pages = [1]
#####for page_num, _ in sorted_pages:
if page_num not in priority_pages:
priority_pages.append(page_num)

Add remaining pages in order (for pages with no distinctive content)
#####for page_num, _ in page_texts:
if page_num not in priority_pages:
priority_pages.append(page_num)

Build content from prioritized pages, respecting char limit
#####content_parts = []
#####chars_used = 0
#####pages_included = []

First pass: add pages in priority order
#####for page_num in priority_pages:
text = page_texts[page_num - 1][1]
if not text.strip():
continue
page_header = f'--- Page {page_num} ---\n'
truncation_suffix = '\n[...truncated]'
remaining = MAX_CHARS - chars_used
Need room for header + suffix + at least some content
min_useful = len(page_header) + len(truncation_suffix) + 50
if remaining < min_useful:
break # no room left for meaningful content

```



```

#####page_content = page_header + text
#####if len(page_content) > remaining:
Truncate page to fit remaining budget exactly
page_content = page_content[: remaining - len(truncation_suffix)] + truncation_suffix
content_parts.append((page_num, page_content))
chars_used += len(page_content)
pages_included.append(page_num)
if chars_used >= MAX_CHARS:
break

Sort included pages by page number for readability
content_parts.sort(key=lambda x: x[0])
extracted_text = '\n\n'.join(part for _, part in content_parts)

pages_not_shown = num_pages - len(pages_included)
if pages_not_shown > 0:
skipped = [p for p in range(1, num_pages + 1) if p not in pages_included]
truncation_note = (
f'\n\n[Showing {len(pages_included)} of {num_pages} pages. '
f'Skipped pages: {skipped[:10]}{"..." if len(skipped) > 10 else ""}. '
f'Use read_long_content with a specific goal to find relevant sections.]'
)
else:
truncation_note = ''

result['message'] = (
f'Read from file {full_filename} ({num_pages} pages, {total_chars:,} chars total).\n'
f'<content>\n{extracted_text}{truncation_note}\n</content>'
)
return result

elif extension in ['.jpg', '.jpeg', '.png']:
import anyio

Read image file and convert to base64
async with await anyio.open_file(full_filename, 'rb') as f:
img_data = await f.read()

base64_str = base64.b64encode(img_data).decode('utf-8')

result['message'] = f'Read image file {full_filename}.'
result['images'] = [{'name': os.path.basename(full_filename), 'data': base64_str}]
return result

else:
result['message'] = f'Error: Cannot read file {full_filename} as {extension} extension is not supported'
return result

except FileNotFoundError:
result['message'] = f'Error: File '{full_filename}' not found.'
return result
except PermissionError:
result['message'] = f'Error: Permission denied to read file '{full_filename}'.'
return result
except Exception as e:
result['message'] = f'Error: Could not read file '{full_filename}'. {str(e)}'
return result

For internal files, only non-image types are supported
resolved, was_sanitized = self._resolve_filename(full_filename)
if not self._is_valid_filename(resolved):
result['message'] = _build_filename_error_message(full_filename, self.get_allowed_extensions())
return result

file_obj = self.files.get(resolved)
if not file_obj:
if was_sanitized:
result['message'] = f'File '{resolved}' not found. (Filename was auto-corrected from '{full_filename}')
else:
result['message'] = f'File '{full_filename}' not found.'
return result
try:

```

```

 """content = file_obj.read()
 """sanitize_note = f"Note: filename was auto-corrected from '{full_filename}' to '{resolved}'. " if was
 """result['message'] = f'{sanitize_note}Read from file {resolved}.\n<content>\n{content}\n</content>'
 """return result
 """except FileSystemError as e:
 """result['message'] = str(e)
 """return result
 """except Exception as e:
 """result['message'] = f"Error: Could not read file '{full_filename}'. {str(e)}"
 """return result

 async def read_file(self, full_filename: str, external_file: bool = False) -> str:
 """Read file content using file-specific read method and return appropriate message to LLM.

 Note: For image files, use read_file_structured() to get image data.
 """
 """result = await self.read_file_structured(full_filename, external_file)
 """return result['message']

 async def write_file(self, full_filename: str, content: str) -> str:
 """Write content to file using file-specific write method"""
 """original_filename = full_filename
 """resolved, was_sanitized = self._resolve_filename(full_filename)
 """if not self._is_valid_filename(resolved):
 """return _build_filename_error_message(full_filename, self.get_allowed_extensions())
 """full_filename = resolved

 """try:
 """name_without_ext, extension = self._parse_filename(full_filename)
 """file_class = self._get_file_type_class(extension)
 """if not file_class:
 """raise ValueError(f"Error: Invalid file extension '{extension}' for file '{full_filename}'.")

 """# Create or get existing file using full filename as key
 """if full_filename in self.files:
 """file_obj = self.files[full_filename]
 """else:
 """file_obj = file_class(name=name_without_ext)
 """self.files[full_filename] = file_obj # Use full filename as key

 """# Use file-specific write method
 """await file_obj.write(content, self.data_dir)
 """sanitize_note = f" (auto-corrected from '{original_filename}')" if was_sanitized else ''
 """return f'Data written to file {full_filename} successfully.{sanitize_note}'
 """except FileSystemError as e:
 """return str(e)
 """except Exception as e:
 """return f"Error: Could not write to file '{full_filename}'. {str(e)}"

 async def append_file(self, full_filename: str, content: str) -> str:
 """Append content to file using file-specific append method"""
 """original_filename = full_filename
 """resolved, was_sanitized = self._resolve_filename(full_filename)
 """if not self._is_valid_filename(resolved):
 """return _build_filename_error_message(full_filename, self.get_allowed_extensions())
 """full_filename = resolved

 """file_obj = self.files.get(full_filename)
 """if not file_obj:
 """if was_sanitized:
 """return f"File '{full_filename}' not found. (Filename was auto-corrected from '{original_filename}')"
 """return f"File '{full_filename}' not found."

 """try:
 """await file_obj.append(content, self.data_dir)
 """sanitize_note = f" (auto-corrected from '{original_filename}')" if was_sanitized else ''
 """return f'Data appended to file {full_filename} successfully.{sanitize_note}'
 """except FileSystemError as e:
 """return str(e)
 """except Exception as e:
 """return f"Error: Could not append to file '{full_filename}'. {str(e)}"
 """async def replace_file_str(self, full_filename: str, old_str: str, new_str: str) -> str:

```

```

 """Replace old_str with new_str in file_name"""
 original_filename = full_filename
 resolved, was_sanitized = self._resolve_filename(full_filename)
 if not self._is_valid_filename(resolved):
 return _build_filename_error_message(full_filename, self.get_allowed_extensions())
 full_filename = resolved

 if not old_str:
 return 'Error: Cannot replace empty string. Please provide a non-empty string to replace.'

 file_obj = self.files.get(full_filename)
 if not file_obj:
 if was_sanitized:
 return f"File '{full_filename}' not found. (Filename was auto-corrected from '{original_filename}')"
 return f"File '{full_filename}' not found."

 try:
 content = file_obj.read()
 content = content.replace(old_str, new_str)
 await file_obj.write(content, self.data_dir)
 sanitize_note = f" (auto-corrected from '{original_filename}')" if was_sanitized else ''
 return f'Successfully replaced all occurrences of "{old_str}" with "{new_str}" in file {full_filename}'
 except FileNotFoundError as e:
 return str(e)
 except Exception as e:
 return f"Error: Could not replace string in file '{full_filename}'. {str(e)}"

 async def save_extracted_content(self, content: str) -> str:
 """Save extracted content to a numbered file"""
 initial_filename = f'extracted_content_{self.extracted_content_count}'
 extracted_filename = f'{initial_filename}.md'
 file_obj = MarkdownFile(name=initial_filename)
 await file_obj.write(content, self.data_dir)
 self.files[extracted_filename] = file_obj
 self.extracted_content_count += 1
 return extracted_filename

 def describe(self) -> str:
 """List all files with their content information using file-specific display methods"""
 DISPLAY_CHARS = 400
 description = ''

 for file_obj in self.files.values():
 # Skip todo.md from description
 if file_obj.full_name == 'todo.md':
 continue

 content = file_obj.read()

 # Handle empty files
 if not content:
 description += f'<file>\n{file_obj.full_name} - [empty file]\n</file>\n'
 continue

 lines = content.splitlines()
 line_count = len(lines)

 # For small files, display the entire content
 whole_file_description = (
 f'<file>\n{file_obj.full_name} - {line_count} lines\n<content>\n{content}\n</content>\n</file>\n'
)
 if len(content) < int(1.5 * DISPLAY_CHARS):
 description += whole_file_description
 continue

 # For larger files, display start and end previews
 half_display_chars = DISPLAY_CHARS // 2

 # Get start preview
 start_preview = ''
 start_line_count = 0
 chars_count = 0

```

```

 for line in lines:
 if chars_count + len(line) + 1 > half_display_chars:
 break
 start_preview += line + '\n'
 chars_count += len(line) + 1
 start_line_count += 1

 # Get end preview
 end_preview = ''
 end_line_count = 0
 chars_count = 0
 for line in reversed(lines):
 if chars_count + len(line) + 1 > half_display_chars:
 break
 end_preview = line + '\n' + end_preview
 chars_count += len(line) + 1
 end_line_count += 1

 # Calculate lines in between
 middle_line_count = line_count - start_line_count - end_line_count
 if middle_line_count <= 0:
 description += whole_file_description
 continue

 start_preview = start_preview.strip('\n').rstrip()
 end_preview = end_preview.strip('\n').rstrip()

 # Format output
 if not (start_preview or end_preview):
 description += f'<file>\n{file_obj.full_name} - {line_count} lines\n<content>\n{middle_line_count}'
 else:
 description += f'<file>\n{file_obj.full_name} - {line_count} lines\n<content>\n{start_preview}\n'
 description += f'... {middle_line_count} more lines ...\n'
 description += f'{end_preview}\n'
 description += '</content>\n</file>\n'

 return description.strip('\n')

def get_todo_contents(self) -> str:
 """Get todo file contents"""
 todo_file = self.get_file('todo.md')
 return todo_file.read() if todo_file else ''

def get_state(self) -> FileSystemState:
 """Get serializable state of the file system"""
 files_data = {}
 for full_filename, file_obj in self.files.items():
 files_data[full_filename] = {'type': file_obj.__class__.__name__, 'data': file_obj.model_dump()}

 return FileSystemState(
 files=files_data, base_dir=str(self.base_dir), extracted_content_count=self.extracted_content_count
)

def nuke(self) -> None:
 """Delete the file system directory"""
 shutil.rmtree(self.data_dir)

@classmethod
def from_state(cls, state: FileSystemState) -> 'FileSystem':
 """Restore file system from serializable state at the exact same location"""
 # Create file system without default files
 fs = cls(base_dir=Path(state.base_dir), create_default_files=False)
 fs.extracted_content_count = state.extracted_content_count

 # Restore all files
 for full_filename, file_data in state.files.items():
 file_type = file_data['type']
 file_info = file_data['data']

 # Create the appropriate file object based on type
 file_type_map: dict[str, type[BaseFile]] = {
 'MarkdownFile': MarkdownFile,

```

```

 'TxtFile': TxtFile,
 'JsonFile': JsonFile,
 'JsonlFile': JsonlFile,
 'CsvFile': CsvFile,
 'PdfFile': PdfFile,
 'DocxFile': DocxFile,
 'HtmlFile': HtmlFile,
 'XmlFile': XmlFile,
 }

 file_class = file_type_map.get(file_type)
 if not file_class:
 # Skip unknown file types
 continue
 file_obj = file_class(**file_info)

 # Add to files dict and sync to disk
 fs.files[full_filename] = file_obj
 file_obj.sync_to_disk_sync(fs.data_dir)

 return fs

```

## ***.\system\filesystem\\_\_init\_\_.py***

## ***.\system\llm\base.py***

```

from typing import Any, Protocol, TypeVar, overload, runtime_checkable

from pydantic import BaseModel

from system.llm.messages import BaseMessage
from system.llm.views import ChatInvokeCompletion

T = TypeVar('T', bound=BaseModel)

@runtime_checkable
class BaseChatModel(Protocol):
 _verified_api_keys: bool = False

 model: str

 @property
 def provider(self) -> str: ...

 @property
 def name(self) -> str: ...

 @property
 def model_name(self) -> str:
 # for legacy support
 return self.model

 @overload
 async def ainvoke(
 self, messages: list[BaseMessage], output_format: None = None, **kwargs: Any
) -> ChatInvokeCompletion[str]: ...

 @overload
 async def ainvoke(self, messages: list[BaseMessage], output_format: type[T], **kwargs: Any) -> ChatInvokeCompletion[T]: ...

 async def ainvoke(
 self, messages: list[BaseMessage], output_format: type[T] | None = None, **kwargs: Any
) -> ChatInvokeCompletion[T]: ...

```

```

■) -> ChatInvokeCompletion[T] | ChatInvokeCompletion[str]: ...

■@classmethod
■def __get_pydantic_core_schema__(
■ cls,
■ source_type: type,
■ handler: Any,
■) -> Any:
■ """
■ Allow this Protocol to be used in Pydantic models -> very useful to typesafe the agent settings for e
■ Returns a schema that allows any object (since this is a Protocol).
■ """
■ from pydantic_core import core_schema

■ # Return a schema that accepts any object for Protocol types
■ return core_schema.any_schema()

```

## ***.\system\llm\exceptions.py***

```

class ModelError(Exception):
 pass

class ModelProviderError(ModelError):
 """Exception raised when a model provider returns an error."""

 def __init__(
 self,
 message: str,
 status_code: int = 502,
 model: str | None = None,
):
 super().__init__(message)
 self.message = message
 self.status_code = status_code
 self.model = model

class ModelRateLimitError(ModelProviderError):
 """Exception raised when a model provider returns a rate limit error."""

 def __init__(
 self,
 message: str,
 status_code: int = 429,
 model: str | None = None,
):
 super().__init__(message, status_code, model)

```

## ***.\system\llm\messages.py***

```

"""
This implementation is based on the OpenAI types, while removing all the parts that are not needed for B
"""

region - Content parts
from typing import Literal, Union

from pydantic import BaseModel

def _truncate(text: str, max_length: int = 50) -> str:
 """Truncate text to max_length characters, adding ellipsis if truncated."""
 if len(text) <= max_length:

```

```

 return text
 return text[: max_length - 3] + '...'

```

```

def _format_image_url(url: str, max_length: int = 50) -> str:
 """Format image URL for display, truncating if necessary."""
 if url.startswith('data:'):
 # Base64 image
 media_type = url.split(';')[0].split(':')[1] if ';' in url else 'image'
 return f'<base64 {media_type}>'
 else:
 # Regular URL
 return _truncate(url, max_length)

```

```

class ContentPartTextParam(BaseModel):
 text: str
 type: Literal['text'] = 'text'

 def __str__(self) -> str:
 return f'Text: {_truncate(self.text)}'

 def __repr__(self) -> str:
 return f'ContentPartTextParam(text={_truncate(self.text)})'

```

```

class ContentPartRefusalParam(BaseModel):
 refusal: str
 type: Literal['refusal'] = 'refusal'

 def __str__(self) -> str:
 return f'Refusal: {_truncate(self.refusal)}'

 def __repr__(self) -> str:
 return f'ContentPartRefusalParam(refusal={_truncate(repr(self.refusal), 50)})'

```

```

SupportedImageMediaType = Literal['image/jpeg', 'image/png', 'image/gif', 'image/webp']

```

```

class ImageURL(BaseModel):
 url: str
 """Either a URL of the image or the base64 encoded image data."""
 detail: Literal['auto', 'low', 'high'] = 'auto'
 """Specifies the detail level of the image.

```

Learn more in the  
[Vision guide](<https://platform.openai.com/docs/guides/vision#low-or-high-fidelity-image-understanding>)

```

 """
 # needed for Anthropic
 media_type: SupportedImageMediaType = 'image/png'

 def __str__(self) -> str:
 url_display = _format_image_url(self.url)
 return f'Image[{self.media_type}, detail={self.detail}]: {url_display}'

 def __repr__(self) -> str:
 url_repr = _format_image_url(self.url, 30)
 return f'ImageURL(url={repr(url_repr)}, detail={repr(self.detail)}, media_type={repr(self.media_type)})'

```

```

class ContentPartImageParam(BaseModel):
 image_url: ImageURL
 type: Literal['image_url'] = 'image_url'

 def __str__(self) -> str:
 return str(self.image_url)

 def __repr__(self) -> str:
 return f'ContentPartImageParam(image_url={repr(self.image_url)})'
class Function(BaseModel):
 arguments: str

```

```

 """
 The arguments to call the function with, as generated by the model in JSON
 format. Note that the model does not always generate valid JSON, and may
 hallucinate parameters not defined by your function schema. Validate the
 arguments in your code before calling your function.
 """
 name: str
 """The name of the function to call."""

 def __str__(self) -> str:
 args_preview = _truncate(self.arguments, 80)
 return f'{self.name}({args_preview})'

 def __repr__(self) -> str:
 args_repr = _truncate(repr(self.arguments), 50)
 return f'Function(name={repr(self.name)}, arguments={args_repr})'

class ToolCall(BaseModel):
 id: str
 """The ID of the tool call."""
 function: Function
 """The function that the model called."""
 type: Literal['function'] = 'function'
 """The type of the tool. Currently, only `function` is supported."""

 def __str__(self) -> str:
 return f'ToolCall[{self.id}]: {self.function}'

 def __repr__(self) -> str:
 return f'ToolCall(id={repr(self.id)}, function={repr(self.function)})'

endregion

region - Message types
class _MessageBase(BaseModel):
 """Base class for all message types"""

 role: Literal['user', 'system', 'assistant']

 cache: bool = False
 """Whether to cache this message. This is only applicable when using Anthropic models.
 """

class UserMessage(_MessageBase):
 role: Literal['user'] = 'user'
 """The role of the messages author, in this case `user`."""

 content: str | list[ContentPartTextParam | ContentPartImageParam]
 """The contents of the user message."""

 name: str | None = None
 """An optional name for the participant.

 Provides the model information to differentiate between participants of the same
 role.
 """

 @property
 def text(self) -> str:
 """
 Automatically parse the text inside content, whether it's a string or a list of content parts.
 """
 if isinstance(self.content, str):
 return self.content
 elif isinstance(self.content, list):
 return '\n'.join([part.text for part in self.content if part.type == 'text'])
 else:
 return ''

```



```

 def __str__(self) -> str:
 return f'UserMessage(content={self.text})'

 def __repr__(self) -> str:
 return f'UserMessage(content={repr(self.text)})'

class SystemMessage(_MessageBase):
 role: Literal['system'] = 'system'
 """The role of the messages author, in this case `system`."""

 content: str | list[ContentPartTextParam]
 """The contents of the system message."""

 name: str | None = None

 @property
 def text(self) -> str:
 """
 Automatically parse the text inside content, whether it's a string or a list of content parts.
 """
 if isinstance(self.content, str):
 return self.content
 elif isinstance(self.content, list):
 return '\n'.join([part.text for part in self.content if part.type == 'text'])
 else:
 return ''

 def __str__(self) -> str:
 return f'SystemMessage(content={self.text})'

 def __repr__(self) -> str:
 return f'SystemMessage(content={repr(self.text)})'

class AssistantMessage(_MessageBase):
 role: Literal['assistant'] = 'assistant'
 """The role of the messages author, in this case `assistant`."""

 content: str | list[ContentPartTextParam | ContentPartRefusalParam] | None
 """The contents of the assistant message."""

 name: str | None = None

 refusal: str | None = None
 """The refusal message by the assistant."""

 tool_calls: list[ToolCall] = []
 """The tool calls generated by the model, such as function calls."""

 @property
 def text(self) -> str:
 """
 Automatically parse the text inside content, whether it's a string or a list of content parts.
 """
 if isinstance(self.content, str):
 return self.content
 elif isinstance(self.content, list):
 text = ''
 for part in self.content:
 if part.type == 'text':
 text += part.text
 elif part.type == 'refusal':
 text += f'[Refusal] {part.refusal}'
 return text
 else:
 return ''

 def __str__(self) -> str:
 return f'AssistantMessage(content={self.text})'

 def __repr__(self) -> str:

```

```
 return f'AssistantMessage(content={repr(self.text)})'
```

```
BaseMessage = Union[UserMessage, SystemMessage, AssistantMessage]
```

```
endregion
```

## ***.lsystem\llm\models.py***

```
"""
Convenient access to LLM models.

Usage:
 from system import llm

 # Simple model access
 model = llm.azure_gpt_4_1_mini
 model = llm.openai_gpt_4o
 model = llm.google_gemini_2_5_pro
 model = llm.bu_latest # or bu_1_0, bu_2_0
"""

import os
from typing import TYPE_CHECKING

from system.llm.azure.chat import ChatAzureOpenAI
from system.llm.system.chat import ChatBrowserUse
from system.llm.cerebras.chat import ChatCerebras
from system.llm.google.chat import ChatGoogle
from system.llm.mistral.chat import ChatMistral
from system.llm.openai.chat import ChatOpenAI

Optional OCI import
try:
 from system.llm.oci_raw.chat import ChatOCIRaw

 OCI_AVAILABLE = True
except ImportError:
 ChatOCIRaw = None
 OCI_AVAILABLE = False

if TYPE_CHECKING:
 from system.llm.base import BaseChatModel

Type stubs for IDE autocomplete
openai_gpt_4o: 'BaseChatModel'
openai_gpt_4o_mini: 'BaseChatModel'
openai_gpt_4_1_mini: 'BaseChatModel'
openai_o1: 'BaseChatModel'
openai_o1_mini: 'BaseChatModel'
openai_o1_pro: 'BaseChatModel'
openai_o3: 'BaseChatModel'
openai_o3_mini: 'BaseChatModel'
openai_o3_pro: 'BaseChatModel'
openai_o4_mini: 'BaseChatModel'
openai_gpt_5: 'BaseChatModel'
openai_gpt_5_mini: 'BaseChatModel'
openai_gpt_5_nano: 'BaseChatModel'

azure_gpt_4o: 'BaseChatModel'
azure_gpt_4o_mini: 'BaseChatModel'
azure_gpt_4_1_mini: 'BaseChatModel'
azure_o1: 'BaseChatModel'
azure_o1_mini: 'BaseChatModel'
azure_o1_pro: 'BaseChatModel'
azure_o3: 'BaseChatModel'
azure_o3_mini: 'BaseChatModel'
azure_o3_pro: 'BaseChatModel'
```

```
azure_gpt_5: 'BaseChatModel'
azure_gpt_5_mini: 'BaseChatModel'
```

```
google_gemini_2_0_flash: 'BaseChatModel'
google_gemini_2_0_pro: 'BaseChatModel'
google_gemini_2_5_pro: 'BaseChatModel'
google_gemini_2_5_flash: 'BaseChatModel'
google_gemini_2_5_flash_lite: 'BaseChatModel'
mistral_large: 'BaseChatModel'
mistral_medium: 'BaseChatModel'
mistral_small: 'BaseChatModel'
codestral: 'BaseChatModel'
pixtral_large: 'BaseChatModel'
```

```
cerebras_llama3_1_8b: 'BaseChatModel'
cerebras_llama3_3_70b: 'BaseChatModel'
cerebras_gpt_oss_120b: 'BaseChatModel'
cerebras_llama_4_scout_17b_16e_instruct: 'BaseChatModel'
cerebras_llama_4_maverick_17b_128e_instruct: 'BaseChatModel'
cerebras_qwen_3_32b: 'BaseChatModel'
cerebras_qwen_3_235b_a22b_instruct_2507: 'BaseChatModel'
cerebras_qwen_3_235b_a22b_thinking_2507: 'BaseChatModel'
cerebras_qwen_3_coder_480b: 'BaseChatModel'
```

```
bu_latest: 'BaseChatModel'
bu_1_0: 'BaseChatModel'
bu_2_0: 'BaseChatModel'
```

```
def get_llm_by_name(model_name: str):
```

```
 """
```

```
 ■ Factory function to create LLM instances from string names with API keys from environment.
```

```
 ■ Args:
```

```
 ■ model_name: String name like 'azure_gpt_4_1_mini', 'openai_gpt_4o', etc.
```

```
 ■ Returns:
```

```
 ■ LLM instance with API keys from environment variables
```

```
 ■ Raises:
```

```
 ■ ValueError: If model_name is not recognized
```

```
 """
```

```
 ■ if not model_name:
```

```
 ■ raise ValueError('Model name cannot be empty')
```

```
 ■ # Handle top-level Mistral aliases without provider prefix
```

```
 ■ mistral_aliases = {
```

```
 ■ 'mistral_large': 'mistral-large-latest',
```

```
 ■ 'mistral_medium': 'mistral-medium-latest',
```

```
 ■ 'mistral_small': 'mistral-small-latest',
```

```
 ■ 'codestral': 'codestral-latest',
```

```
 ■ 'pixtral_large': 'pixtral-large-latest',
```

```
 ■ }
```

```
 ■ if model_name in mistral_aliases:
```

```
 ■ api_key = os.getenv('MISTRAL_API_KEY')
```

```
 ■ base_url = os.getenv('MISTRAL_BASE_URL', 'https://api.mistral.ai/v1')
```

```
 ■ return ChatMistral(model=mistral_aliases[model_name], api_key=api_key, base_url=base_url)
```

```
 ■ # Parse model name
```

```
 ■ parts = model_name.split('_', 1)
```

```
 ■ if len(parts) < 2:
```

```
 ■ raise ValueError(f"Invalid model name format: '{model_name}'. Expected format: 'provider_model_name'")
```

```
 ■ provider = parts[0]
```

```
 ■ model_part = parts[1]
```

```
 ■ # Convert underscores back to dots/dashes for actual model names
```

```
 ■ if 'gpt_4_1_mini' in model_part:
```

```
 ■ model = model_part.replace('gpt_4_1_mini', 'gpt-4.1-mini')
```

```
 ■ elif 'gpt_4o_mini' in model_part:
```

```
 ■ model = model_part.replace('gpt_4o_mini', 'gpt-4o-mini')
```

```
 ■ elif 'gpt_4o' in model_part:
```

```

 model = model_part.replace('gpt_4o', 'gpt-4o')
 elif 'gemini_2_0' in model_part:
 model = model_part.replace('gemini_2_0', 'gemini-2.0').replace('_', '-')
 elif 'gemini_2_5' in model_part:
 model = model_part.replace('gemini_2_5', 'gemini-2.5').replace('_', '-')
 elif 'llama3_1' in model_part:
 model = model_part.replace('llama3_1', 'llama3.1').replace('_', '-')
 elif 'llama3_3' in model_part:
 model = model_part.replace('llama3_3', 'llama-3.3').replace('_', '-')
 elif 'llama_4_scout' in model_part:
 model = model_part.replace('llama_4_scout', 'llama-4-scout').replace('_', '-')
 elif 'llama_4_maverick' in model_part:
 model = model_part.replace('llama_4_maverick', 'llama-4-maverick').replace('_', '-')
 elif 'gpt_oss_120b' in model_part:
 model = model_part.replace('gpt_oss_120b', 'gpt-oss-120b')
 elif 'qwen_3_32b' in model_part:
 model = model_part.replace('qwen_3_32b', 'qwen-3-32b')
 elif 'qwen_3_235b_a22b_instruct' in model_part:
 if model_part.endswith('_2507'):
 model = model_part.replace('qwen_3_235b_a22b_instruct_2507', 'qwen-3-235b-a22b-instruct-2507')
 else:
 model = model_part.replace('qwen_3_235b_a22b_instruct', 'qwen-3-235b-a22b-instruct-2507')
 elif 'qwen_3_235b_a22b_thinking' in model_part:
 if model_part.endswith('_2507'):
 model = model_part.replace('qwen_3_235b_a22b_thinking_2507', 'qwen-3-235b-a22b-thinking-2507')
 else:
 model = model_part.replace('qwen_3_235b_a22b_thinking', 'qwen-3-235b-a22b-thinking-2507')
 elif 'qwen_3_coder_480b' in model_part:
 model = model_part.replace('qwen_3_coder_480b', 'qwen-3-coder-480b')
 else:
 model = model_part.replace('_', '-')

 ## OpenAI Models
 if provider == 'openai':
 api_key = os.getenv('OPENAI_API_KEY')
 return ChatOpenAI(model=model, api_key=api_key)

 ## Azure OpenAI Models
 elif provider == 'azure':
 api_key = os.getenv('AZURE_OPENAI_KEY') or os.getenv('AZURE_OPENAI_API_KEY')
 azure_endpoint = os.getenv('AZURE_OPENAI_ENDPOINT')
 return ChatAzureOpenAI(model=model, api_key=api_key, azure_endpoint=azure_endpoint)

 ## Google Models
 elif provider == 'google':
 api_key = os.getenv('GOOGLE_API_KEY')
 return ChatGoogle(model=model, api_key=api_key)

 ## Mistral Models
 elif provider == 'mistral':
 api_key = os.getenv('MISTRAL_API_KEY')
 base_url = os.getenv('MISTRAL_BASE_URL', 'https://api.mistral.ai/v1')
 mistral_map = {
 'large': 'mistral-large-latest',
 'medium': 'mistral-medium-latest',
 'small': 'mistral-small-latest',
 'codestral': 'codestral-latest',
 'pixtral-large': 'pixtral-large-latest',
 }
 normalized_model_part = model_part.replace('_', '-')
 resolved_model = mistral_map.get(normalized_model_part, model_part.replace('_', '-'))
 return ChatMistral(model=resolved_model, api_key=api_key, base_url=base_url)

 ## OCI Models
 elif provider == 'oci':
 ## OCI requires more complex configuration that can't be easily inferred from env vars
 ## Users should use ChatOCIRaw directly with proper configuration
 raise ValueError('OCI models require manual configuration. Use ChatOCIRaw directly with your OCI credentials')

 ## Cerebras Models
 elif provider == 'cerebras':
 api_key = os.getenv('CEREBRAS_API_KEY')

```

```

 return ChatCerebras(model=model, api_key=api_key)

 # Browser Use Models
 elif provider == 'bu':
 # Handle bu_latest -> bu-latest conversion (need to prepend 'bu-' back)
 model = f'bu-{{model_part.replace("_", "-")}}'
 api_key = os.getenv('BROWSER_USE_API_KEY')
 return ChatBrowserUse(model=model, api_key=api_key)

 else:
 available_providers = ['openai', 'azure', 'google', 'oci', 'cerebras', 'bu']

 raise ValueError(f"Unknown provider: '{provider}'. Available providers: {' '.join(available_providers)}")

Pre-configured model instances (lazy loaded via __getattr__)
def __getattr__(name: str) -> 'BaseChatModel':
 """Create model instances on demand with API keys from environment."""
 # Handle chat classes first
 if name == 'ChatOpenAI':
 return ChatOpenAI # type: ignore
 elif name == 'ChatAzureOpenAI':
 return ChatAzureOpenAI # type: ignore
 elif name == 'ChatGoogle':
 return ChatGoogle # type: ignore

 elif name == 'ChatMistral':
 return ChatMistral # type: ignore

 elif name == 'ChatOCIRaw':
 if not OCI_AVAILABLE:
 raise ImportError('OCI integration not available. Install with: pip install "browser-use[oci]"')
 return ChatOCIRaw # type: ignore
 elif name == 'ChatCerebras':
 return ChatCerebras # type: ignore
 elif name == 'ChatBrowserUse':
 return ChatBrowserUse # type: ignore

 # Handle model instances - these are the main use case
 try:
 return get_llm_by_name(name)
 except ValueError:
 raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

Export all classes and preconfigured instances, conditionally including ChatOCIRaw
__all__ = [
 'ChatOpenAI',
 'ChatAzureOpenAI',
 'ChatGoogle',
 'ChatMistral',
 'ChatCerebras',
 'ChatBrowserUse',
]

if OCI_AVAILABLE:
 __all__.append('ChatOCIRaw')

__all__ += [
 'get_llm_by_name',
 # OpenAI instances - created on demand
 'openai_gpt_4o',
 'openai_gpt_4o_mini',
 'openai_gpt_4_1_mini',
 'openai_o1',
 'openai_o1_mini',
 'openai_o1_pro',
 'openai_o3',
 'openai_o3_mini',
 'openai_o3_pro',
 'openai_o4_mini',
 'openai_gpt_5',

```

```

■'openai_gpt_5_mini',
■'openai_gpt_5_nano',
■# Azure instances - created on demand
■'azure_gpt_4o',
■'azure_gpt_4o_mini',
■'azure_gpt_4_1_mini',
■'azure_o1',
■'azure_o1_mini',
■'azure_o1_pro',
■'azure_o3',
■'azure_o3_mini',
■'azure_o3_pro',
■'azure_gpt_5',
■'azure_gpt_5_mini',
■# Google instances - created on demand
■'google_gemini_2_0_flash',
■'google_gemini_2_0_pro',
■'google_gemini_2_5_pro',
■'google_gemini_2_5_flash',
■'google_gemini_2_5_flash_lite',
■# Mistral instances - created on demand
■'mistral_large',
■'mistral_medium',
■'mistral_small',
■'codestral',
■'pixtral_large',
■# Cerebras instances - created on demand
■'cerebras_llama3_1_8b',
■'cerebras_llama3_3_70b',
■'cerebras_gpt_oss_120b',
■'cerebras_llama_4_scout_17b_16e_instruct',
■'cerebras_llama_4_maverick_17b_128e_instruct',
■'cerebras_qwen_3_32b',
■'cerebras_qwen_3_235b_a22b_instruct_2507',
■'cerebras_qwen_3_235b_a22b_thinking_2507',
■'cerebras_qwen_3_coder_480b',
■# Browser Use instances - created on demand
■'bu_latest',
■'bu_1_0',
■'bu_2_0',
]

```

```

NOTE: OCI backend is optional. The try/except ImportError and conditional __all__ are required
so this module can be imported without browser-use[oci] installed.

```

## ***.\system\llm\schema.py***

```

"""
Utilities for creating optimized Pydantic schemas for LLM usage.
"""

from typing import Any

from pydantic import BaseModel

class SchemaOptimizer:
 ■@staticmethod
 ■def create_optimized_json_schema(
 ■ model: type[BaseModel],
 ■ *,
 ■ remove_min_items: bool = False,
 ■ remove_defaults: bool = False,
 ■) -> dict[str, Any]:
 ■ """
 ■ Create the most optimized schema by flattening all $ref/$defs while preserving
 ■ FULL descriptions and ALL action definitions. Also ensures OpenAI strict mode compatibility.
 ■ Args:
 """

```

```

#####model: The Pydantic model to optimize
#####remove_min_items: If True, remove minItems from the schema
#####remove_defaults: If True, remove default values from the schema

#####Returns:
#####Optimized schema with all $refs resolved and strict mode compatibility
#####"""
Generate original schema
#####original_schema = model.model_json_schema()

Extract $defs for reference resolution, then flatten everything
#####defs_lookup = original_schema.get('$defs', {})

Create optimized schema with flattening
Pass flags to optimize_schema via closure
#####def optimize_schema(obj: Any, defs_lookup: dict[str, Any] | None = None, *, in_properties: bool = False):
"""Apply all optimization techniques including flattening all $ref/$defs"""
if isinstance(obj, dict):
optimized: dict[str, Any] = {}
flattened_ref: dict[str, Any] | None = None

Skip unnecessary fields AND $defs (we'll inline everything)
skip_fields = ['additionalProperties', '$defs']

for key, value in obj.items():
if key in skip_fields:
continue

Skip metadata "title" unless we're iterating inside an actual `properties` map
if key == 'title' and not in_properties:
continue

Preserve FULL descriptions without truncation, skip empty ones
elif key == 'description':
if value: # Only include non-empty descriptions
optimized[key] = value

Handle type field - must recursively process in case value contains $ref
elif key == 'type':
optimized[key] = value if not isinstance(value, (dict, list)) else optimize_schema(value, defs_lookup)

FLATTEN: Resolve $ref by inlining the actual definition
elif key == '$ref' and defs_lookup:
ref_path = value.split('/')[1] # Get the definition name from "#/$defs/SomeName"
if ref_path in defs_lookup:
Get the referenced definition and flatten it
referenced_def = defs_lookup[ref_path]
flattened_ref = optimize_schema(referenced_def, defs_lookup)

Skip minItems/min_items and default if requested (check BEFORE processing)
elif key in ('minItems', 'min_items') and remove_min_items:
continue # Skip minItems/min_items
elif key == 'default' and remove_defaults:
continue # Skip default values

Keep all anyOf structures (action unions) and resolve any $refs within
elif key == 'anyOf' and isinstance(value, list):
optimized[key] = [optimize_schema(item, defs_lookup) for item in value]

Recursively optimize nested structures
elif key in ['properties', 'items']:
optimized[key] = optimize_schema(
value,
defs_lookup,
in_properties=(key == 'properties'),
)

Keep essential validation fields
elif key in [
'type',
'required',
'minimum',

```

```

 'maximum',
 'minItems',
 'min_items',
 'maxItems',
 'pattern',
 'default',
]:
 optimized[key] = value if not isinstance(value, (dict, list)) else optimize_schema(value, defs_lookup)

 # Recursively process all other fields
 else:
 optimized[key] = optimize_schema(value, defs_lookup) if isinstance(value, (dict, list)) else value

 # If we have a flattened reference, merge it with the optimized properties
 if flattened_ref is not None and isinstance(flattened_ref, dict):
 # Start with the flattened reference as the base
 result = flattened_ref.copy()

 # Merge in any sibling properties that were processed
 for key, value in optimized.items():
 # Preserve descriptions from the original object if they exist
 if key == 'description' and 'description' not in result:
 result[key] = value
 elif key != 'description': # Don't overwrite description from flattened ref
 result[key] = value

 return result
 else:
 # No $ref, just return the optimized object
 # CRITICAL: Add additionalProperties: false to ALL objects for OpenAI strict mode
 if optimized.get('type') == 'object':
 optimized['additionalProperties'] = False

 return optimized

elif isinstance(obj, list):
 return [optimize_schema(item, defs_lookup, in_properties=in_properties) for item in obj]
return obj

optimized_result = optimize_schema(original_schema, defs_lookup)

Ensure we have a dictionary (should always be the case for schema root)
if not isinstance(optimized_result, dict):
 raise ValueError('Optimized schema result is not a dictionary')

optimized_schema: dict[str, Any] = optimized_result

Additional pass to ensure ALL objects have additionalProperties: false
def ensure_additional_properties_false(obj: Any) -> None:
 """Ensure all objects have additionalProperties: false"""
 if isinstance(obj, dict):
 # If it's an object type, ensure additionalProperties is false
 if obj.get('type') == 'object':
 obj['additionalProperties'] = False

 # Recursively apply to all values
 for value in obj.values():
 if isinstance(value, (dict, list)):
 ensure_additional_properties_false(value)
 elif isinstance(obj, list):
 for item in obj:
 if isinstance(item, (dict, list)):
 ensure_additional_properties_false(item)

ensure_additional_properties_false(optimized_schema)
SchemaOptimizer._make_strict_compatible(optimized_schema)

Final pass to remove minItems/min_items and default values if requested
if remove_min_items or remove_defaults:

 def remove_forbidden_fields(obj: Any) -> None:
 """Recursively remove minItems/min_items and default values"""

```



```

 if isinstance(obj, dict):
 # Remove forbidden keys
 if remove_min_items:
 obj.pop('minItems', None)
 obj.pop('min_items', None)
 if remove_defaults:
 obj.pop('default', None)
 # Recursively process all values
 for value in obj.values():
 if isinstance(value, (dict, list)):
 remove_forbidden_fields(value)
 elif isinstance(obj, list):
 for item in obj:
 if isinstance(item, (dict, list)):
 remove_forbidden_fields(item)

 remove_forbidden_fields(optimized_schema)

 return optimized_schema

 @staticmethod
 def _make_strict_compatible(schema: dict[str, Any] | list[Any]) -> None:
 """Ensure all properties are required for OpenAI strict mode"""
 if isinstance(schema, dict):
 # First recursively apply to nested objects
 for key, value in schema.items():
 if isinstance(value, (dict, list)) and key != 'required':
 SchemaOptimizer._make_strict_compatible(value)

 # Then update required for this level
 if 'properties' in schema and 'type' in schema and schema['type'] == 'object':
 # Add all properties to required array
 all_props = list(schema['properties'].keys())
 schema['required'] = all_props # Set all properties as required

 elif isinstance(schema, list):
 for item in schema:
 SchemaOptimizer._make_strict_compatible(item)

 @staticmethod
 def create_gemini_optimized_schema(model: type[BaseModel]) -> dict[str, Any]:
 """
 Create Gemini-optimized schema, preserving explicit `required` arrays so Gemini
 respects mandatory fields defined by the caller.

 Args:
 model: The Pydantic model to optimize

 Returns:
 Optimized schema suitable for Gemini structured output
 """
 return SchemaOptimizer.create_optimized_json_schema(model)

```

## **`.\system\llm\views.py`**

```

from typing import Generic, TypeVar, Union

from pydantic import BaseModel

T = TypeVar('T', bound=Union[BaseModel, str])

class ChatInvokeUsage(BaseModel):
 """
 Usage information for a chat model invocation.
 """

 prompt_tokens: int

```

```

■"""The number of tokens in the prompt (this includes the cached tokens as well. When calculating the co

■prompt_cached_tokens: int | None
■"""The number of cached tokens."""

■prompt_cache_creation_tokens: int | None
■"""Anthropic only: The number of tokens used to create the cache."""

■prompt_image_tokens: int | None
■"""Google only: The number of tokens in the image (prompt tokens is the text tokens + image tokens in t

■completion_tokens: int
■"""The number of tokens in the completion."""

■total_tokens: int
■"""The total number of tokens in the response."""

class ChatInvokeCompletion(BaseModel, Generic[T]):
 """
 ■Response from a chat model invocation.
 """

 ■completion: T
 ■"""The completion of the response."""

 ■# Thinking stuff
 ■thinking: str | None = None
 ■redacted_thinking: str | None = None

 ■usage: ChatInvokeUsage | None
 ■"""The usage of the response."""

 ■stop_reason: str | None = None
 ■"""The reason the model stopped generating. Common values: 'end_turn', 'max_tokens', 'stop_sequence'."""

```

## **`.\system\llm\__init__.py`**

```

"""
LLM Abstraction for MIRA
Based on browser-use's openai.types.chat parameter layer.
"""

from system.llm.base import BaseChatModel
from system.llm.messages import (
 BaseMessage,
 UserMessage,
 SystemMessage,
 AssistantMessage,
 ContentPartTextParam as ContentText,
 ContentPartImageParam as ContentImage,
)

Lazy importing
_LAZY_IMPORTS = {
 'ChatOpenAI': ('system.llm.openai.chat', 'ChatOpenAI'),
 'ChatGoogle': ('system.llm.google.chat', 'ChatGoogle'),
}

Cache for model instances - only created when accessed
_model_cache: dict[str, 'BaseChatModel'] = {}

def __getattr__(name: str):
 """Lazy import mechanism for heavy chat model imports and model instances."""
 if name in _LAZY_IMPORTS:
 module_path, attr_name = _LAZY_IMPORTS[name]
 try:
 from importlib import import_module

```

```

 module = import_module(module_path)
 attr = getattr(module, attr_name)
 return attr
 except ImportError as e:
 raise ImportError(f'Failed to import {name} from {module_path}: {e}') from e

Check cache first for model instances
if name in _model_cache:
 return _model_cache[name]

Try to get model instances from models module on-demand
try:
 from system.llm.models import __getattr__ as models_getattr

 attr = models_getattr(name)
 # Cache in our clean cache dict
 _model_cache[name] = attr
 return attr
except (AttributeError, ImportError):
 pass

raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

__all__ = [
 'BaseMessage',
 'UserMessage',
 'SystemMessage',
 'AssistantMessage',
 'ContentText',
 'ContentImage',
 'BaseChatModel',
 'ChatOpenAI',
 'ChatGoogle',
]

```

## ***.\system\llm\google\chat.py***

```

import asyncio
import json
import logging
import random
import time
from dataclasses import dataclass, field
from typing import Any, Literal, TypeVar, overload

from google import genai
from google.auth.credentials import Credentials
from google.genai import types
from google.genai.types import MediaModality
from pydantic import BaseModel

from system.llm.base import BaseChatModel
from system.llm.exceptions import ModelProviderError
from system.llm.google.serializer import GoogleMessageSerializer
from system.llm.messages import BaseMessage
from system.llm.schema import SchemaOptimizer
from system.llm.views import ChatInvokeCompletion, ChatInvokeUsage

T = TypeVar('T', bound=BaseModel)

VerifiedGeminiModels = Literal[
 ■'gemini-2.0-flash',
 ■'gemini-2.0-flash-exp',
 ■'gemini-2.0-flash-lite-preview-02-05',
 ■'Gemini-2.0-exp',
 ■'gemini-2.5-flash',

```

```

 'gemini-2.5-flash-lite',
 'gemini-flash-latest',
 'gemini-flash-lite-latest',
 'gemini-2.5-pro',
 'gemini-3-pro-preview',
 'gemini-3-flash-preview',
 'gemma-3-27b-it',
 'gemma-3-4b',
 'gemma-3-12b',
 'gemma-3n-e2b',
 'gemma-3n-e4b',
]

```

```
@dataclass
```

```
class ChatGoogle(BaseChatModel):
```

```
 """
```

```
 A wrapper around Google's Gemini chat model using the genai client.
```

```

 This class accepts all genai.Client parameters while adding model,
 temperature, and config parameters for the LLM interface.

```

```
 Args:
```

```
 model: The Gemini model to use
```

```
 temperature: Temperature for response generation
```

```
 config: Additional configuration parameters to pass to generate_content
```

```
 (e.g., tools, safety_settings, etc.).
```

```
 api_key: Google API key
```

```
 vertexai: Whether to use Vertex AI
```

```
 credentials: Google credentials object
```

```
 project: Google Cloud project ID
```

```
 location: Google Cloud location
```

```
 http_options: HTTP options for the client
```

```
 include_system_in_user: If True, system messages are included in the first user message
```

```
 supports_structured_output: If True, uses native JSON mode; if False, uses prompt-based fallback
```

```
 max_retries: Number of retries for retryable errors (default: 5)
```

```
 retryable_status_codes: List of HTTP status codes to retry on (default: [429, 500, 502, 503, 504])
```

```
 retry_base_delay: Base delay in seconds for exponential backoff (default: 1.0)
```

```
 retry_max_delay: Maximum delay in seconds between retries (default: 60.0)
```

```
 Example:
```

```
 from google.genai import types
```

```
 llm = ChatGoogle(
```

```
 model='gemini-2.0-flash-exp',
```

```
 config={
```

```
 'tools': [types.Tool(code_execution=types.ToolCodeExecution())]
```

```
 },
```

```
 max_retries=5,
```

```
 retryable_status_codes=[429, 500, 502, 503, 504],
```

```
 retry_base_delay=1.0,
```

```
 retry_max_delay=60.0,
```

```
)
```

```
 """
```

```
 # Model configuration
```

```
 model: VerifiedGeminiModels | str
```

```
 temperature: float | None = 0.5
```

```
 top_p: float | None = None
```

```
 seed: int | None = None
```

```
 thinking_budget: int | None = None # for Gemini 2.5: -1 for dynamic (default), 0 disables, or token count
```

```
 thinking_level: Literal['minimal', 'low', 'medium', 'high'] | None = (
```

```
 None # for Gemini 3: Pro supports low/high, Flash supports all levels
```

```
)
```

```
 max_output_tokens: int | None = 8096
```

```
 config: types.GenerateContentConfigDict | None = None
```

```
 include_system_in_user: bool = False
```

```
 supports_structured_output: bool = True # New flag
```

```
 max_retries: int = 5 # Number of retries for retryable errors
```

```
 retryable_status_codes: list[int] = field(default_factory=lambda: [429, 500, 502, 503, 504]) # Status
```

```
 retry_base_delay: float = 1.0 # Base delay in seconds for exponential backoff
```

```
 retry_max_delay: float = 60.0 # Maximum delay in seconds between retries
```

```

■# Client initialization parameters
■api_key: str | None = None
■vertexai: bool | None = None
■credentials: Credentials | None = None
■project: str | None = None
■location: str | None = None
■http_options: types.HttpOptions | types.HttpOptionsDict | None = None

■# Internal client cache to prevent connection issues
■_client: genai.Client | None = None

■# Static
■@property
■def provider(self) -> str:
■ return 'google'

■@property
■def logger(self) -> logging.Logger:
■ """Get logger for this chat instance"""
■ return logging.getLogger(f'system.llm.google.{self.model}')

■def _get_client_params(self) -> dict[str, Any]:
■ """Prepare client parameters dictionary."""
■ # Define base client params
■ base_params = {
■ 'api_key': self.api_key,
■ 'vertexai': self.vertexai,
■ 'credentials': self.credentials,
■ 'project': self.project,
■ 'location': self.location,
■ 'http_options': self.http_options,
■ }

■ # Create client_params dict with non-None values
■ client_params = {k: v for k, v in base_params.items() if v is not None}

■ return client_params

■def get_client(self) -> genai.Client:
■ """
■ Returns a genai.Client instance.

■ Returns:
■ genai.Client: An instance of the Google genai client.
■ """
■ if self._client is not None:
■ return self._client

■ client_params = self._get_client_params()
■ self._client = genai.Client(**client_params)
■ return self._client

■@property
■def name(self) -> str:
■ return str(self.model)

■def _get_stop_reason(self, response: types.GenerateContentResponse) -> str | None:
■ """Extract stop_reason from Google response."""
■ if hasattr(response, 'candidates') and response.candidates:
■ return str(response.candidates[0].finish_reason) if hasattr(response.candidates[0], 'finish_reason')
■ return None

■def _get_usage(self, response: types.GenerateContentResponse) -> ChatInvokeUsage | None:
■ usage: ChatInvokeUsage | None = None

■ if response.usage_metadata is not None:
■ image_tokens = 0
■ if response.usage_metadata.prompt_tokens_details is not None:
■ image_tokens = sum(
■ detail.token_count or 0
■ for detail in response.usage_metadata.prompt_tokens_details
■ if detail.modality == MediaModality.IMAGE

```

```

)

 usage = ChatInvokeUsage(
 prompt_tokens=response.usage_metadata.prompt_token_count or 0,
 completion_tokens=(response.usage_metadata.candidates_token_count or 0)
 + (response.usage_metadata.thoughts_token_count or 0),
 total_tokens=response.usage_metadata.total_token_count or 0,
 prompt_cached_tokens=response.usage_metadata.cached_content_token_count,
 prompt_cache_creation_tokens=None,
 prompt_image_tokens=image_tokens,
)

 return usage

 @overload
 async def ainvoke(
 self, messages: list[BaseMessage], output_format: None = None, **kwargs: Any
) -> ChatInvokeCompletion[str]: ...

 @overload
 async def ainvoke(self, messages: list[BaseMessage], output_format: type[T], **kwargs: Any) -> ChatInvokeCompletion[T]: ...

 async def ainvoke(
 self, messages: list[BaseMessage], output_format: type[T] | None = None, **kwargs: Any
) -> ChatInvokeCompletion[T] | ChatInvokeCompletion[str]:
 """
 Invoke the model with the given messages.

 Args:
 messages: List of chat messages
 output_format: Optional Pydantic model class for structured output

 Returns:
 Either a string response or an instance of output_format
 """

 # Serialize messages to Google format with the include_system_in_user flag
 contents, system_instruction = GoogleMessageSerializer.serialize_messages(
 messages, include_system_in_user=self.include_system_in_user
)

 # Build config dictionary starting with user-provided config
 config: types.GenerateContentConfigDict = {}
 if self.config:
 config = self.config.copy()

 # Apply model-specific configuration (these can override config)
 if self.temperature is not None:
 config['temperature'] = self.temperature

 # Add system instruction if present
 if system_instruction:
 config['system_instruction'] = system_instruction

 if self.top_p is not None:
 config['top_p'] = self.top_p

 if self.seed is not None:
 config['seed'] = self.seed

 # Configure thinking based on model version
 # Gemini 3 Pro: uses thinking_level only
 # Gemini 3 Flash: supports both, defaults to thinking_budget=-1
 # Gemini 2.5: uses thinking_budget only
 is_gemini_3_pro = 'gemini-3-pro' in self.model
 is_gemini_3_flash = 'gemini-3-flash' in self.model

 if is_gemini_3_pro:
 # Validate: thinking_budget should not be set for Gemini 3 Pro
 if self.thinking_budget is not None:
 self.logger.warning(
 f'thinking_budget={self.thinking_budget} is deprecated for Gemini 3 Pro and may cause '

```

```

#####f'suboptimal performance. Use thinking_level instead.'
#####)

Validate: minimal/medium only supported on Flash, not Pro
####if self.thinking_level in ('minimal', 'medium'):
####self.logger.warning(
#####f'thinking_level="{self.thinking_level}" is not supported for Gemini 3 Pro. '
#####f'Only "low" and "high" are valid. Falling back to "low".'
#####)
####self.thinking_level = 'low'

Default to 'low' for Gemini 3 Pro
####if self.thinking_level is None:
####self.thinking_level = 'low'

Map to ThinkingLevel enum (SDK accepts string values)
####level = types.ThinkingLevel(self.thinking_level.upper())
####config['thinking_config'] = types.ThinkingConfigDict(thinking_level=level)
####elif is_gemini_3_flash:
Gemini 3 Flash supports both thinking_level and thinking_budget
If user set thinking_level, use that; otherwise default to thinking_budget=-1
####if self.thinking_level is not None:
####level = types.ThinkingLevel(self.thinking_level.upper())
####config['thinking_config'] = types.ThinkingConfigDict(thinking_level=level)
####else:
####if self.thinking_budget is None:
####self.thinking_budget = -1
####config['thinking_config'] = types.ThinkingConfigDict(thinking_budget=self.thinking_budget)
####else:
Gemini 2.5 and earlier: use thinking_budget only
####if self.thinking_level is not None:
####self.logger.warning(
#####f'thinking_level="{self.thinking_level}" is not supported for this model. '
#####f'Use thinking_budget instead (0 to disable, -1 for dynamic, or token count).'
#####)
Default to -1 for dynamic/auto on 2.5 models
####if self.thinking_budget is None and ('gemini-2.5' in self.model or 'gemini-flash' in self.model):
####self.thinking_budget = -1
####if self.thinking_budget is not None:
####config['thinking_config'] = types.ThinkingConfigDict(thinking_budget=self.thinking_budget)

####if self.max_output_tokens is not None:
####config['max_output_tokens'] = self.max_output_tokens

####async def _make_api_call():
####start_time = time.time()
####self.logger.debug(f'■ Starting API call to {self.model}')

####try:
####if output_format is None:
Return string response
#####self.logger.debug('■ Requesting text response')

#####response = await self.get_client().ai.models.generate_content(
#####model=self.model,
#####contents=contents, # type: ignore
#####config=config,
#####)

#####elapsed = time.time() - start_time
#####self.logger.debug(f'■ Got text response in {elapsed:.2f}s')

Handle case where response.text might be None
#####text = response.text or ''
#####if not text:
#####self.logger.warning('■ Empty text response received')

#####usage = self._get_usage(response)

#####return ChatInvokeCompletion(
#####completion=text,
#####usage=usage,

```

```

 stop_reason=self._get_stop_reason(response),
)

 else:
 # Handle structured output
 if self.supports_structured_output:
 # Use native JSON mode
 self.logger.debug(f'■ Requesting structured output for {output_format.__name__}')
 config['response_mime_type'] = 'application/json'
 # Convert Pydantic model to Gemini-compatible schema
 optimized_schema = SchemaOptimizer.create_gemini_optimized_schema(output_format)

 gemini_schema = self._fix_gemini_schema(optimized_schema)
 config['response_schema'] = gemini_schema

 response = await self.get_client().aio.models.generate_content(
 model=self.model,
 contents=contents,
 config=config,
)

 elapsed = time.time() - start_time
 self.logger.debug(f'■ Got structured response in {elapsed:.2f}s')

 usage = self._get_usage(response)

 # Handle case where response.parsed might be None
 if response.parsed is None:
 self.logger.debug('■ Parsing JSON from text response')
 # When using response_schema, Gemini returns JSON as text
 if response.text:
 try:
 # Handle JSON wrapped in markdown code blocks (common Gemini behavior)
 text = response.text.strip()
 if text.startswith('```json') and text.endswith('```'):
 text = text[7:-3].strip()
 self.logger.debug('■ Stripped ```json``` wrapper from response')
 elif text.startswith('```') and text.endswith('```'):
 text = text[3:-3].strip()
 self.logger.debug('■ Stripped ``` wrapper from response')

 # Parse the JSON text and validate with the Pydantic model
 parsed_data = json.loads(text)
 return ChatInvokeCompletion(
 completion=output_format.model_validate(parsed_data),
 usage=usage,
 stop_reason=self._get_stop_reason(response),
)
 except (json.JSONDecodeError, ValueError) as e:
 self.logger.error(f'■ Failed to parse JSON response: {str(e)}')
 self.logger.debug(f'Raw response text: {response.text[:200]}...')
 raise ModelProviderError(
 message=f'Failed to parse or validate response {response}: {str(e)}',
 status_code=500,
 model=self.model,
) from e
 else:
 self.logger.error('■ No response text received')
 raise ModelProviderError(
 message=f'No response from model {response}',
 status_code=500,
 model=self.model,
)

 # Ensure we return the correct type
 if isinstance(response.parsed, output_format):
 return ChatInvokeCompletion(
 completion=response.parsed,
 usage=usage,
 stop_reason=self._get_stop_reason(response),
)
 else:

```



```

If it's not the expected type, try to validate it
return ChatInvokeCompletion(
completion=output_format.model_validate(response.parsed),
usage=usage,
stop_reason=self._get_stop_reason(response),
)
else:
Fallback: Request JSON in the prompt for models without native JSON mode
self.logger.debug(f'■ Using fallback JSON mode for {output_format.__name__}')
Create a copy of messages to modify
modified_messages = [m.model_copy(deep=True) for m in messages]

Add JSON instruction to the last message
if modified_messages and isinstance(modified_messages[-1].content, str):
json_instruction = f'\n\nPlease respond with a valid JSON object that matches this schema: {Sch
modified_messages[-1].content += json_instruction

Re-serialize with modified messages
fallback_contents, fallback_system = GoogleMessageSerializer.serialize_messages(
modified_messages, include_system_in_user=self.include_system_in_user
)

Update config with fallback system instruction if present
fallback_config = config.copy()
if fallback_system:
fallback_config['system_instruction'] = fallback_system

response = await self.get_client().aio.models.generate_content(
model=self.model,
contents=fallback_contents, # type: ignore
config=fallback_config,
)

elapsed = time.time() - start_time
self.logger.debug(f'■ Got fallback response in {elapsed:.2f}s')

usage = self._get_usage(response)

Try to extract JSON from the text response
if response.text:
try:
Try to find JSON in the response
text = response.text.strip()

Common patterns: JSON wrapped in markdown code blocks
if text.startswith('```json') and text.endswith('```'):
text = text[7:-3].strip()
elif text.startswith('```') and text.endswith('```'):
text = text[3:-3].strip()

Parse and validate
parsed_data = json.loads(text)
return ChatInvokeCompletion(
completion=output_format.model_validate(parsed_data),
usage=usage,
stop_reason=self._get_stop_reason(response),
)
except (json.JSONDecodeError, ValueError) as e:
self.logger.error(f'■ Failed to parse fallback JSON: {str(e)}')
self.logger.debug(f'Raw response text: {response.text[:200]}...')
raise ModelProviderError(
message=f'Model does not support JSON mode and failed to parse JSON from text response: {str
status_code=500,
model=self.model,
) from e
else:
self.logger.error('■ No response text in fallback mode')
raise ModelProviderError(
message='No response from model',
status_code=500,
model=self.model,
)

```

```

 except Exception as e:
 elapsed = time.time() - start_time
 self.logger.error(f'API call failed after {elapsed:.2f}s: {type(e).__name__}: {e}')
 # Re-raise the exception
 raise

 # Retry logic for certain errors with exponential backoff
 assert self.max_retries >= 1, 'max_retries must be at least 1'

 for attempt in range(self.max_retries):
 try:
 return await _make_api_call()
 except ModelProviderError as e:
 # Retry if status code is in retryable list and we have attempts left
 if e.status_code in self.retryable_status_codes and attempt < self.max_retries - 1:
 # Exponential backoff with jitter: base_delay * 2^attempt + random jitter
 delay = min(self.retry_base_delay * (2**attempt), self.retry_max_delay)
 jitter = random.uniform(0, delay * 0.1) # 10% jitter
 total_delay = delay + jitter
 self.logger.warning(
 f'Got {e.status_code} error, retrying in {total_delay:.1f}s... (attempt {attempt + 1}/{self.max_retries})')
 await asyncio.sleep(total_delay)
 continue
 # Otherwise raise
 raise
 except Exception as e:
 # For non-ModelProviderError, wrap and raise
 error_message = str(e)
 status_code: int | None = None

 # Try to extract status code if available
 if hasattr(e, 'response'):
 response_obj = getattr(e, 'response', None)
 if response_obj and hasattr(response_obj, 'status_code'):
 status_code = getattr(response_obj, 'status_code', None)

 # Enhanced timeout error handling
 if 'timeout' in error_message.lower() or 'cancelled' in error_message.lower():
 if isinstance(e, asyncio.CancelledError) or 'CancelledError' in str(type(e)):
 error_message = 'Gemini API request was cancelled (likely timeout). Consider: 1) Reducing input'
 status_code = 504
 else:
 status_code = 408
 elif any(indicator in error_message.lower() for indicator in ['forbidden', '403']):
 status_code = 403
 elif any(
 indicator in error_message.lower()
 for indicator in ['rate limit', 'resource exhausted', 'quota exceeded', 'too many requests', '429']
):
 status_code = 429
 elif any(
 indicator in error_message.lower()
 for indicator in ['service unavailable', 'internal server error', 'bad gateway', '503', '502', '500']
):
 status_code = 503

 raise ModelProviderError(
 message=error_message,
 status_code=status_code or 502,
 model=self.name,
) from e

 raise RuntimeError('Retry loop completed without return or exception')

def _fix_gemini_schema(self, schema: dict[str, Any]) -> dict[str, Any]:
 """
 Convert a Pydantic model to a Gemini-compatible schema.

 This function removes unsupported properties like 'additionalProperties' and resolves
 $ref references that Gemini doesn't support.
 """

```

```

 ## Handle $defs and $ref resolution
 if '$defs' in schema:
 defs = schema.pop('$defs')

 def resolve_refs(obj: Any) -> Any:
 if isinstance(obj, dict):
 if '$ref' in obj:
 ref = obj.pop('$ref')
 ref_name = ref.split('/')[1]
 if ref_name in defs:
 # Replace the reference with the actual definition
 resolved = defs[ref_name].copy()
 # Merge any additional properties from the reference
 for key, value in obj.items():
 if key != '$ref':
 resolved[key] = value
 return resolve_refs(resolved)
 return obj
 else:
 # Recursively process all dictionary values
 return {k: resolve_refs(v) for k, v in obj.items()}
 elif isinstance(obj, list):
 return [resolve_refs(item) for item in obj]
 return obj

 schema = resolve_refs(schema)

 ## Remove unsupported properties
 def clean_schema(obj: Any, parent_key: str | None = None) -> Any:
 if isinstance(obj, dict):
 # Remove unsupported properties
 cleaned = {}
 for key, value in obj.items():
 # Only strip 'title' when it's a JSON Schema metadata field (not inside 'properties')
 # 'title' as a metadata field appears at schema level, not as a property name
 is_metadata_title = key == 'title' and parent_key != 'properties'
 if key not in ['additionalProperties', 'default'] and not is_metadata_title:
 cleaned_value = clean_schema(value, parent_key=key)
 # Handle empty object properties - Gemini doesn't allow empty OBJECT types
 if (
 key == 'properties'
 and isinstance(cleaned_value, dict)
 and len(cleaned_value) == 0
 and isinstance(obj.get('type', ''), str)
 and obj.get('type', '').upper() == 'OBJECT'
):
 # Convert empty object to have at least one property
 cleaned['properties'] = {'_placeholder': {'type': 'string'}}
 else:
 cleaned[key] = cleaned_value

 # If this is an object type with empty properties, add a placeholder
 if (
 isinstance(cleaned.get('type', ''), str)
 and cleaned.get('type', '').upper() == 'OBJECT'
 and 'properties' in cleaned
 and isinstance(cleaned['properties'], dict)
 and len(cleaned['properties']) == 0
):
 cleaned['properties'] = {'_placeholder': {'type': 'string'}}

 return cleaned
 elif isinstance(obj, list):
 return [clean_schema(item, parent_key=parent_key) for item in obj]
 return obj

 return clean_schema(schema)

```

**.\system\llm\google\serializer.py**

```

import base64

from google.genai.types import Content, ContentListUnion, Part

from system.llm.messages import (
 AssistantMessage,
 BaseMessage,
 SystemMessage,
 UserMessage,
)

class GoogleMessageSerializer:
 """Serializer for converting messages to Google Gemini format."""

 @staticmethod
 def serialize_messages(
 messages: list[BaseMessage], include_system_in_user: bool = False
) -> tuple[ContentListUnion, str | None]:
 """
 Convert a list of BaseMessages to Google format, extracting system message.

 Google handles system instructions separately from the conversation, so we need to:
 1. Extract any system messages and return them separately as a string (or include in first user message)
 2. Convert the remaining messages to Content objects

 Args:
 messages: List of messages to convert
 include_system_in_user: If True, system/developer messages are prepended to the first user message

 Returns:
 A tuple of (formatted_messages, system_message) where:
 - formatted_messages: List of Content objects for the conversation
 - system_message: System instruction string or None
 """

 messages = [m.model_copy(deep=True) for m in messages]

 formatted_messages: ContentListUnion = []
 system_message: str | None = None
 system_parts: list[str] = []

 for i, message in enumerate(messages):
 role = message.role if hasattr(message, 'role') else None

 # Handle system/developer messages
 if isinstance(message, SystemMessage) or role in ['system', 'developer']:
 # Extract system message content as string
 if isinstance(message.content, str):
 if include_system_in_user:
 system_parts.append(message.content)
 else:
 system_message = message.content
 elif message.content is not None:
 # Handle Iterable of content parts
 parts = []
 for part in message.content:
 if part.type == 'text':
 parts.append(part.text)
 combined_text = '\n'.join(parts)
 if include_system_in_user:
 system_parts.append(combined_text)
 else:
 system_message = combined_text
 continue

 # Determine the role for non-system messages
 if isinstance(message, UserMessage):
 role = 'user'
 elif isinstance(message, AssistantMessage):
 role = 'model'

```

```

 else:
 # Default to user for any unknown message types
 role = 'user'

 # Initialize message parts
 message_parts: list[Part] = []

 # If this is the first user message and we have system parts, prepend them
 if include_system_in_user and system_parts and role == 'user' and not formatted_messages:
 system_text = '\n\n'.join(system_parts)
 if isinstance(message.content, str):
 message_parts.append(Part.from_text(text=f'{system_text}\n\n{message.content}'))
 else:
 # Add system text as the first part
 message_parts.append(Part.from_text(text=system_text))
 system_parts = [] # Clear after using
 else:
 # Extract content and create parts normally
 if isinstance(message.content, str):
 # Regular text content
 message_parts = [Part.from_text(text=message.content)]
 elif message.content is not None:
 # Handle Iterable of content parts
 for part in message.content:
 if part.type == 'text':
 message_parts.append(Part.from_text(text=part.text))
 elif part.type == 'refusal':
 message_parts.append(Part.from_text(text=f'[Refusal] {part.refusal}'))
 elif part.type == 'image_url':
 # Handle images
 url = part.image_url.url

 # Format: data:image/jpeg;base64,<data>
 header, data = url.split(',', 1)
 # Decode base64 to bytes
 image_bytes = base64.b64decode(data)

 # Use the media_type from ImageURL, which correctly identifies the image format
 mime_type = part.image_url.media_type

 # Add image part
 image_part = Part.from_bytes(data=image_bytes, mime_type=mime_type)

 message_parts.append(image_part)

 # Create the Content object
 if message_parts:
 final_message = Content(role=role, parts=message_parts)
 # for some reason, the type checker is not able to infer the type of formatted_messages
 formatted_messages.append(final_message) # type: ignore

 return formatted_messages, system_message

```

## ***.\system\lm\google\\_\_init\_\_.py***

```

from system.llm.google.chat import ChatGoogle

__all__ = ['ChatGoogle']

```

## ***.\system\lm\openai\chat.py***

```

from collections.abc import Iterable, Mapping
from dataclasses import dataclass, field
from typing import Any, Literal, TypeVar, overload

```

```

import httpx
from openai import APIConnectionError, APIStatusError, AsyncOpenAI, RateLimitError
from openai.types.chat import ChatCompletionContentPartTextParam
from openai.types.chat.chat_completion import ChatCompletion
from openai.types.shared_params.chat_model import ChatModel
from openai.types.shared_params.reasoning_effort import ReasoningEffort
from openai.types.shared_params.response_format_json_schema import JSONSchema, ResponseFormatJSONSchema
from pydantic import BaseModel

from system.llm.base import BaseChatModel
from system.llm.exceptions import ModelProviderError, ModelRateLimitError
from system.llm.messages import BaseMessage
from system.llm.openai.serializer import OpenAIMessageSerializer
from system.llm.schema import SchemaOptimizer
from system.llm.views import ChatInvokeCompletion, ChatInvokeUsage

T = TypeVar('T', bound=BaseModel)

@dataclass
class ChatOpenAI(BaseChatModel):
 """
 A wrapper around AsyncOpenAI that implements the BaseLLM protocol.

 This class accepts all AsyncOpenAI parameters while adding model
 and temperature parameters for the LLM interface (if temperature is not `None`).
 """

 ## Model configuration
 model: ChatModel | str

 ## Model params
 temperature: float | None = 0.2
 frequency_penalty: float | None = 0.3 # this avoids infinite generation of \t for models like 4.1-mini
 reasoning_effort: ReasoningEffort = 'low'
 seed: int | None = None
 service_tier: Literal['auto', 'default', 'flex', 'priority', 'scale'] | None = None
 top_p: float | None = None
 add_schema_to_system_prompt: bool = False # Add JSON schema to system prompt instead of using response
 dont_force_structured_output: bool = False # If True, the model will not be forced to output a structured
 remove_min_items_from_schema: bool = (
 False # If True, remove minItems from JSON schema (for compatibility with some providers)
)
 remove_defaults_from_schema: bool = (
 False # If True, remove default values from JSON schema (for compatibility with some providers)
)

 ## Client initialization parameters
 api_key: str | None = None
 organization: str | None = None
 project: str | None = None
 base_url: str | httpx.URL | None = None
 websocket_base_url: str | httpx.URL | None = None
 timeout: float | httpx.Timeout | None = None
 max_retries: int = 5 # Increase default retries for automation reliability
 default_headers: Mapping[str, str] | None = None
 default_query: Mapping[str, object] | None = None
 http_client: httpx.AsyncClient | None = None
 _strict_response_validation: bool = False
 max_completion_tokens: int | None = 4096
 reasoning_models: list[ChatModel | str] | None = field(
 default_factory=lambda: [
 'o4-mini',
 'o3',
 'o3-mini',
 'o1',
 'o1-pro',
 'o3-pro',
 'gpt-5',
 'gpt-5-mini',
 'gpt-5-nano',
]
)

```

```

)

 # Static
 @property
 def provider(self) -> str:
 return 'openai'

 def _get_client_params(self) -> dict[str, Any]:
 """Prepare client parameters dictionary."""
 # Define base client params
 base_params = {
 'api_key': self.api_key,
 'organization': self.organization,
 'project': self.project,
 'base_url': self.base_url,
 'websocket_base_url': self.websocket_base_url,
 'timeout': self.timeout,
 'max_retries': self.max_retries,
 'default_headers': self.default_headers,
 'default_query': self.default_query,
 '_strict_response_validation': self._strict_response_validation,
 }

 # Create client_params dict with non-None values
 client_params = {k: v for k, v in base_params.items() if v is not None}

 # Add http_client if provided
 if self.http_client is not None:
 client_params['http_client'] = self.http_client

 return client_params

 def get_client(self) -> AsyncOpenAI:
 """
 Returns an AsyncOpenAI client.

 Returns:
 AsyncOpenAI: An instance of the AsyncOpenAI client.
 """
 client_params = self._get_client_params()
 return AsyncOpenAI(**client_params)

 @property
 def name(self) -> str:
 return str(self.model)

 def _get_usage(self, response: ChatCompletion) -> ChatInvokeUsage | None:
 if response.usage is not None:
 # Note: completion_tokens already includes reasoning_tokens per OpenAI API docs.
 # Unlike Google Gemini where thinking_tokens are reported separately,
 # OpenAI's reasoning_tokens are a subset of completion_tokens.
 usage = ChatInvokeUsage(
 prompt_tokens=response.usage.prompt_tokens,
 prompt_cached_tokens=response.usage.prompt_tokens_details.cached_tokens
)
 if response.usage.prompt_tokens_details is not None:
 else None,
 prompt_cache_creation_tokens=None,
 prompt_image_tokens=None,
)
 # Completion
 completion_tokens=response.usage.completion_tokens,
 total_tokens=response.usage.total_tokens,
)
 else:
 usage = None

 return usage

 @overload
 async def ainvoke(
 self, messages: list[BaseMessage], output_format: None = None, **kwargs: Any
) -> ChatInvokeCompletion[str]: ...
 @overload

```

```

 async def ainvoke(self, messages: list[BaseMessage], output_format: type[T], **kwargs: Any) -> ChatInvokeCompletion[T] | ChatInvokeCompletion[str]:
 """
 Invoke the model with the given messages.

 Args:
 messages: List of chat messages
 output_format: Optional Pydantic model class for structured output

 Returns:
 Either a string response or an instance of output_format
 """
 openai_messages = OpenAIMessageSerializer.serialize_messages(messages)

 try:
 model_params: dict[str, Any] = {}

 if self.temperature is not None:
 model_params['temperature'] = self.temperature

 if self.frequency_penalty is not None:
 model_params['frequency_penalty'] = self.frequency_penalty

 if self.max_completion_tokens is not None:
 model_params['max_completion_tokens'] = self.max_completion_tokens

 if self.top_p is not None:
 model_params['top_p'] = self.top_p

 if self.seed is not None:
 model_params['seed'] = self.seed

 if self.service_tier is not None:
 model_params['service_tier'] = self.service_tier

 if self.reasoning_models and any(str(m).lower() in str(self.model).lower() for m in self.reasoning_models):
 model_params['reasoning_effort'] = self.reasoning_effort
 model_params.pop('temperature', None)
 model_params.pop('frequency_penalty', None)

 if output_format is None:
 # Return string response
 response = await self.get_client().chat.completions.create(
 model=self.model,
 messages=openai_messages,
 **model_params,
)

 choice = response.choices[0] if response.choices else None
 if choice is None:
 base_url = str(self.base_url) if self.base_url is not None else None
 hint = f' (base_url={base_url})' if base_url is not None else ''
 raise ModelProviderError(
 message=(
 'Invalid OpenAI chat completion response: missing or empty `choices`.'
 ' If you are using a proxy via `base_url`, ensure it implements the OpenAI'
 ' `/v1/chat/completions` schema and returns `choices` as a non-empty list.'
) + f'{hint}'
),
 status_code=502,
 model=self.name,
)

 usage = self._get_usage(response)
 return ChatInvokeCompletion(
 completion=choice.message.content or '',
 usage=usage,
 stop_reason=choice.finish_reason,
)

```



```

)

 else:
 response_format: JSONSchema = {
 'name': 'agent_output',
 'strict': True,
 'schema': SchemaOptimizer.create_optimized_json_schema(
 output_format,
 remove_min_items=self.remove_min_items_from_schema,
 remove_defaults=self.remove_defaults_from_schema,
),
 }

 # Add JSON schema to system prompt if requested
 if self.add_schema_to_system_prompt and openai_messages and openai_messages[0]['role'] == 'system':
 schema_text = f'\n<json_schema>\n{response_format}\n</json_schema>'
 if isinstance(openai_messages[0]['content'], str):
 openai_messages[0]['content'] += schema_text
 elif isinstance(openai_messages[0]['content'], Iterable):
 openai_messages[0]['content'] = list(openai_messages[0]['content']) + [
 ChatCompletionContentPartTextParam(text=schema_text, type='text')
]

 if self.dont_force_structured_output:
 response = await self.get_client().chat.completions.create(
 model=self.model,
 messages=openai_messages,
 **model_params,
)
 else:
 # Return structured response
 response = await self.get_client().chat.completions.create(
 model=self.model,
 messages=openai_messages,
 response_format=ResponseFormatJSONSchema(json_schema=response_format, type='json_schema'),
 **model_params,
)

 choice = response.choices[0] if response.choices else None
 if choice is None:
 base_url = str(self.base_url) if self.base_url is not None else None
 hint = f' (base_url={base_url})' if base_url is not None else ''
 raise ModelProviderError(
 message=(
 'Invalid OpenAI chat completion response: missing or empty `choices`.'
 ' If you are using a proxy via `base_url`, ensure it implements the OpenAI'
 ' `/v1/chat/completions` schema and returns `choices` as a non-empty list.'
) + hint,
 status_code=502,
 model=self.name,
)

 if choice.message.content is None:
 raise ModelProviderError(
 message='Failed to parse structured output from model response',
 status_code=500,
 model=self.name,
)

 usage = self._get_usage(response)

 parsed = output_format.model_validate_json(choice.message.content)

 return ChatInvokeCompletion(
 completion=parsed,
 usage=usage,
 stop_reason=choice.finish_reason,
)

 except ModelProviderError:
 # Preserve status_code and message from validation errors

```

```

 raise

 except RateLimitError as e:
 raise ModelRateLimitError(message=e.message, model=self.name) from e

 except APIConnectionError as e:
 raise ModelProviderError(message=str(e), model=self.name) from e

 except APIStatusError as e:
 raise ModelProviderError(message=e.message, status_code=e.status_code, model=self.name) from e

 except Exception as e:
 raise ModelProviderError(message=str(e), model=self.name) from e

```

### ***.\system\llm\openai\like.py***

```

from dataclasses import dataclass

from system.llm.openai.chat import ChatOpenAI

@dataclass
class ChatOpenAILike(ChatOpenAI):
 """
 A class for to interact with any provider using the OpenAI API schema.

 Args:
 model (str): The name of the OpenAI model to use.

 model: str

```

### ***.\system\llm\openai\responses\_serializer.py***

```

"""Serializer for converting messages to OpenAI Responses API input format."""

from typing import overload

from openai.types.responses.easy_input_message_param import EasyInputMessageParam
from openai.types.responses.response_input_image_param import ResponseInputImageParam
from openai.types.responses.response_input_message_content_list_param import (
 ResponseInputMessageContentListParam,
)
from openai.types.responses.response_input_text_param import ResponseInputTextParam

from system.llm.messages import (
 AssistantMessage,
 BaseMessage,
 ContentPartImageParam,
 ContentPartRefusalParam,
 ContentPartTextParam,
 SystemMessage,
 UserMessage,
)

class ResponsesAPIMessageSerializer:
 """Serializer for converting between custom message types and OpenAI Responses API input format."""

 @staticmethod
 def _serialize_content_part_text(part: ContentPartTextParam) -> ResponseInputTextParam:
 return ResponseInputTextParam(text=part.text, type='input_text')

 @staticmethod

```

```

def _serialize_content_part_image(part: ContentPartImageParam) -> ResponseInputImageParam:
 return ResponseInputImageParam(
 image_url=part.image_url.url,
 detail=part.image_url.detail,
 type='input_image',
)

 @staticmethod
 def _serialize_user_content(
 content: str | list[ContentPartTextParam | ContentPartImageParam],
) -> str | ResponseInputMessageContentListParam:
 """Serialize content for user messages (text and images allowed)."""
 if isinstance(content, str):
 return content

 serialized_parts: ResponseInputMessageContentListParam = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(ResponseAPIMessageSerializer._serialize_content_part_text(part))
 elif part.type == 'image_url':
 serialized_parts.append(ResponseAPIMessageSerializer._serialize_content_part_image(part))
 return serialized_parts

 @staticmethod
 def _serialize_system_content(
 content: str | list[ContentPartTextParam],
) -> str | ResponseInputMessageContentListParam:
 """Serialize content for system messages (text only)."""
 if isinstance(content, str):
 return content

 serialized_parts: ResponseInputMessageContentListParam = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(ResponseAPIMessageSerializer._serialize_content_part_text(part))
 return serialized_parts

 @staticmethod
 def _serialize_assistant_content(
 content: str | list[ContentPartTextParam | ContentPartRefusalParam] | None,
) -> str | ResponseInputMessageContentListParam | None:
 """Serialize content for assistant messages (text only for Responses API)."""
 if content is None:
 return None
 if isinstance(content, str):
 return content

 serialized_parts: ResponseInputMessageContentListParam = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(ResponseAPIMessageSerializer._serialize_content_part_text(part))
 # Refusals are converted to text for the Responses API
 elif part.type == 'refusal':
 serialized_parts.append(ResponseInputTextParam(text=f'[Refusal: {part.refusal}]', type='input_text'))
 return serialized_parts

 @overload
 @staticmethod
 def serialize(message: UserMessage) -> EasyInputMessageParam: ...

 @overload
 @staticmethod
 def serialize(message: SystemMessage) -> EasyInputMessageParam: ...

 @overload
 @staticmethod
 def serialize(message: AssistantMessage) -> EasyInputMessageParam: ...

 @staticmethod
 def serialize(message: BaseMessage) -> EasyInputMessageParam:
 """Serialize a custom message to an OpenAI Responses API input message param."""
 if isinstance(message, UserMessage):

```

```

 return EasyInputMessageParam(
 role='user',
 content=ResponsesAPIMessageSerializer._serialize_user_content(message.content),
)

 elif isinstance(message, SystemMessage):
 # Note: Responses API uses 'developer' role for system messages in some contexts,
 # but 'system' is also supported via EasyInputMessageParam
 return EasyInputMessageParam(
 role='system',
 content=ResponsesAPIMessageSerializer._serialize_system_content(message.content),
)

 elif isinstance(message, AssistantMessage):
 content = ResponsesAPIMessageSerializer._serialize_assistant_content(message.content)
 # For assistant messages, we need to provide content
 # If content is None but there are tool calls, we represent them as text
 if content is None:
 if message.tool_calls:
 # Convert tool calls to a text representation for context
 tool_call_text = '\n'.join(
 f'[Tool call: {tc.function.name}({tc.function.arguments})]' for tc in message.tool_calls
)
 content = tool_call_text
 else:
 content = ''

 return EasyInputMessageParam(
 role='assistant',
 content=content,
)

 else:
 raise ValueError(f'Unknown message type: {type(message)}')

 @staticmethod
 def serialize_messages(messages: list[BaseMessage]) -> list[EasyInputMessageParam]:
 """Serialize a list of messages to Responses API input format."""
 return [ResponsesAPIMessageSerializer.serialize(m) for m in messages]

```

## ***.\system\llm\openai\serializer.py***

```

from typing import overload

from openai.types.chat import (
 ChatCompletionAssistantMessageParam,
 ChatCompletionContentPartImageParam,
 ChatCompletionContentPartRefusalParam,
 ChatCompletionContentPartTextParam,
 ChatCompletionMessageFunctionToolCallParam,
 ChatCompletionMessageParam,
 ChatCompletionSystemMessageParam,
 ChatCompletionUserMessageParam,
)
from openai.types.chat.chat_completion_content_part_image_param import ImageURL
from openai.types.chat.chat_completion_message_function_tool_call_param import Function

from system.llm.messages import (
 AssistantMessage,
 BaseMessage,
 ContentPartImageParam,
 ContentPartRefusalParam,
 ContentPartTextParam,
 SystemMessage,
 ToolCall,
 UserMessage,
)
class OpenAIMessageSerializer:

```

```

 """Serializer for converting between custom message types and OpenAI message param types."""

 @staticmethod
 def _serialize_content_part_text(part: ContentPartTextParam) -> ChatCompletionContentPartTextParam:
 return ChatCompletionContentPartTextParam(text=part.text, type='text')

 @staticmethod
 def _serialize_content_part_image(part: ContentPartImageParam) -> ChatCompletionContentPartImageParam:
 return ChatCompletionContentPartImageParam(
 image_url=ImageURL(url=part.image_url.url, detail=part.image_url.detail),
 type='image_url',
)

 @staticmethod
 def _serialize_content_part_refusal(part: ContentPartRefusalParam) -> ChatCompletionContentPartRefusalParam:
 return ChatCompletionContentPartRefusalParam(refusal=part.refusal, type='refusal')

 @staticmethod
 def _serialize_user_content(
 content: str | list[ContentPartTextParam | ContentPartImageParam],
) -> str | list[ChatCompletionContentPartTextParam | ChatCompletionContentPartImageParam]:
 """Serialize content for user messages (text and images allowed)."""
 if isinstance(content, str):
 return content

 serialized_parts: list[ChatCompletionContentPartTextParam | ChatCompletionContentPartImageParam] = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(OpenAIMessageSerializer._serialize_content_part_text(part))
 elif part.type == 'image_url':
 serialized_parts.append(OpenAIMessageSerializer._serialize_content_part_image(part))
 return serialized_parts

 @staticmethod
 def _serialize_system_content(
 content: str | list[ContentPartTextParam],
) -> str | list[ChatCompletionContentPartTextParam]:
 """Serialize content for system messages (text only)."""
 if isinstance(content, str):
 return content

 serialized_parts: list[ChatCompletionContentPartTextParam] = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(OpenAIMessageSerializer._serialize_content_part_text(part))
 return serialized_parts

 @staticmethod
 def _serialize_assistant_content(
 content: str | list[ContentPartTextParam | ContentPartRefusalParam] | None,
) -> str | list[ChatCompletionContentPartTextParam | ChatCompletionContentPartRefusalParam] | None:
 """Serialize content for assistant messages (text and refusal allowed)."""
 if content is None:
 return None
 if isinstance(content, str):
 return content

 serialized_parts: list[ChatCompletionContentPartTextParam | ChatCompletionContentPartRefusalParam] = []
 for part in content:
 if part.type == 'text':
 serialized_parts.append(OpenAIMessageSerializer._serialize_content_part_text(part))
 elif part.type == 'refusal':
 serialized_parts.append(OpenAIMessageSerializer._serialize_content_part_refusal(part))
 return serialized_parts

 @staticmethod
 def _serialize_tool_call(tool_call: ToolCall) -> ChatCompletionMessageFunctionToolCallParam:
 return ChatCompletionMessageFunctionToolCallParam(
 id=tool_call.id,
 function=Function(name=tool_call.function.name, arguments=tool_call.function.arguments),
 type='function',
)

```

```

■# endregion

■# region - Serialize overloads
■@overload
■@staticmethod
■def serialize(message: UserMessage) -> ChatCompletionUserMessageParam: ...

■@overload
■@staticmethod
■def serialize(message: SystemMessage) -> ChatCompletionSystemMessageParam: ...

■@overload
■@staticmethod
■def serialize(message: AssistantMessage) -> ChatCompletionAssistantMessageParam: ...

■@staticmethod
■def serialize(message: BaseMessage) -> ChatCompletionMessageParam:
■ """Serialize a custom message to an OpenAI message param."""

 ■if isinstance(message, UserMessage):
 ■ user_result: ChatCompletionUserMessageParam = {
 ■ 'role': 'user',
 ■ 'content': OpenAIMessageSerializer._serialize_user_content(message.content),
 ■ }
 ■ if message.name is not None:
 ■ user_result['name'] = message.name
 ■ return user_result

 ■elif isinstance(message, SystemMessage):
 ■ system_result: ChatCompletionSystemMessageParam = {
 ■ 'role': 'system',
 ■ 'content': OpenAIMessageSerializer._serialize_system_content(message.content),
 ■ }
 ■ if message.name is not None:
 ■ system_result['name'] = message.name
 ■ return system_result

 ■elif isinstance(message, AssistantMessage):
 ■ # Handle content serialization
 ■ content = None
 ■ if message.content is not None:
 ■ content = OpenAIMessageSerializer._serialize_assistant_content(message.content)

 ■ assistant_result: ChatCompletionAssistantMessageParam = {'role': 'assistant'}

 ■ # Only add content if it's not None
 ■ if content is not None:
 ■ assistant_result['content'] = content

 ■ if message.name is not None:
 ■ assistant_result['name'] = message.name
 ■ if message.refusal is not None:
 ■ assistant_result['refusal'] = message.refusal
 ■ if message.tool_calls:
 ■ assistant_result['tool_calls'] = [OpenAIMessageSerializer._serialize_tool_call(tc) for tc in message.tool_calls]

 ■ return assistant_result

 ■else:
 ■ raise ValueError(f'Unknown message type: {type(message)}')

■@staticmethod
■def serialize_messages(messages: list[BaseMessage]) -> list[ChatCompletionMessageParam]:
■ return [OpenAIMessageSerializer.serialize(m) for m in messages]

```

***.\system\screenshots\service.py***

"""

```
Screenshot storage service for browser-use agents.
"""
```

```
import base64
from pathlib import Path
```

```
import anyio
```

```
from system.observability import observe_debug
```

```
class ScreenshotService:
```

```
 """Simple screenshot storage service that saves screenshots to disk"""
```

```
 def __init__(self, agent_directory: str | Path):
```

```
 """Initialize with agent directory path"""
```

```
 self.agent_directory = Path(agent_directory) if isinstance(agent_directory, str) else agent_directory
```

```
 ## Create screenshots subdirectory
```

```
 self.screenshots_dir = self.agent_directory / 'screenshots'
```

```
 self.screenshots_dir.mkdir(parents=True, exist_ok=True)
```

```
 @observe_debug(ignore_input=True, ignore_output=True, name='store_screenshot')
```

```
 async def store_screenshot(self, screenshot_b64: str, step_number: int) -> str:
```

```
 """Store screenshot to disk and return the full path as string"""
```

```
 screenshot_filename = f'step_{step_number}.png'
```

```
 screenshot_path = self.screenshots_dir / screenshot_filename
```

```
 ## Decode base64 and save to disk
```

```
 screenshot_data = base64.b64decode(screenshot_b64)
```

```
 ## async with await anyio.open_file(screenshot_path, 'wb') as f:
```

```
 await f.write(screenshot_data)
```

```
 ## return str(screenshot_path)
```

```
 @observe_debug(ignore_input=True, ignore_output=True, name='get_screenshot_from_disk')
```

```
 async def get_screenshot(self, screenshot_path: str) -> str | None:
```

```
 """Load screenshot from disk path and return as base64"""
```

```
 if not screenshot_path:
```

```
 return None
```

```
 path = Path(screenshot_path)
```

```
 if not path.exists():
```

```
 return None
```

```
 ## Load from disk and encode to base64
```

```
 async with await anyio.open_file(path, 'rb') as f:
```

```
 screenshot_data = await f.read()
```

```
 ## return base64.b64encode(screenshot_data).decode('utf-8')
```

## **`.lsystem\screenshots\__init__.py`**

```
Screenshots package for browser-use
```

## **`.lsystem\sync\auth.py`**

```
"""
```

```
OAuth2 Device Authorization Grant flow client for browser-use.
```

```
"""
```

```
import asyncio
```

```
import json
```

```

import os
import shutil
import time
from datetime import datetime

import httpx
from pydantic import BaseModel
from uuid_extensions import uuid7str

from system.config import CONFIG

Temporary user ID for pre-auth events (matches cloud backend)
TEMP_USER_ID = '99999999-9999-9999-9999-999999999999'

def get_or_create_device_id() -> str:
 """Get or create a persistent device ID for this installation."""
 device_id_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'device_id'

 ## Try to read existing device ID
 if device_id_path.exists():
 try:
 device_id = device_id_path.read_text().strip()
 if device_id: # Make sure it's not empty
 return device_id
 except Exception:
 ## If we can't read it, we'll create a new one
 pass

 ## Create new device ID
 device_id = uuid7str()

 ## Ensure config directory exists
 CONFIG.BROWSER_USE_CONFIG_DIR.mkdir(parents=True, exist_ok=True)

 ## Write device ID to file
 device_id_path.write_text(device_id)

 return device_id

class CloudAuthConfig(BaseModel):
 """Configuration for cloud authentication"""

 api_token: str | None = None
 user_id: str | None = None
 authorized_at: datetime | None = None

 @classmethod
 def load_from_file(cls) -> 'CloudAuthConfig':
 """Load auth config from local file"""

 config_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'cloud_auth.json'
 if config_path.exists():
 try:
 with open(config_path) as f:
 data = json.load(f)
 return cls.model_validate(data)
 except Exception:
 ## Return empty config if file is corrupted
 pass
 return cls()

 def save_to_file(self) -> None:
 """Save auth config to local file"""

 CONFIG.BROWSER_USE_CONFIG_DIR.mkdir(parents=True, exist_ok=True)

 config_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'cloud_auth.json'
 with open(config_path, 'w') as f:
 json.dump(self.model_dump(mode='json'), f, indent=2, default=str)
 ## Set restrictive permissions (owner read/write only) for security

```



```

 try:
 os.chmod(config_path, 0o600)
 except Exception:
 # Some systems may not support chmod, continue anyway
 pass

class DeviceAuthClient:
 """Client for OAuth2 device authorization flow"""

 def __init__(self, base_url: str | None = None, http_client: httpx.AsyncClient | None = None):
 """Backend API URL for OAuth requests - can be passed directly or defaults to env var"""
 self.base_url = base_url or CONFIG.BROWSER_USE_CLOUD_API_URL
 self.client_id = 'library'
 self.scope = 'read write'

 """If no client provided, we'll create one per request"""
 self.http_client = http_client

 """Temporary user ID for pre-auth events"""
 self.temp_user_id = TEMP_USER_ID

 """Get or create persistent device ID"""
 self.device_id = get_or_create_device_id()

 """Load existing auth if available"""
 self.auth_config = CloudAuthConfig.load_from_file()

 @property
 def is_authenticated(self) -> bool:
 """Check if we have valid authentication"""
 return bool(self.auth_config.api_token and self.auth_config.user_id)

 @property
 def api_token(self) -> str | None:
 """Get the current API token"""
 return self.auth_config.api_token

 @property
 def user_id(self) -> str:
 """Get the current user ID (temporary or real)"""
 return self.auth_config.user_id or self.temp_user_id

 async def start_device_authorization(
 self,
 agent_session_id: str | None = None,
) -> dict:
 """
 Start the device authorization flow.
 Returns device authorization details including user code and verification URL.
 """
 if self.http_client:
 response = await self.http_client.post(
 f'{self.base_url.rstrip("/")}/api/v1/oauth/device/authorize',
 data={
 'client_id': self.client_id,
 'scope': self.scope,
 'agent_session_id': agent_session_id or '',
 'device_id': self.device_id,
 },
)
 response.raise_for_status()
 return response.json()
 else:
 async with httpx.AsyncClient() as client:
 response = await client.post(
 f'{self.base_url.rstrip("/")}/api/v1/oauth/device/authorize',
 data={
 'client_id': self.client_id,
 'scope': self.scope,
 'agent_session_id': agent_session_id or '',
 'device_id': self.device_id,
 },
)

```

```

 },
)
 response.raise_for_status()
 return response.json()

 async def poll_for_token(
 self,
 device_code: str,
 interval: float = 3.0,
 timeout: float = 1800.0,
) -> dict | None:
 """
 Poll for the access token.
 Returns token info when authorized, None if timeout.
 """
 start_time = time.time()

 if self.http_client:
 # Use injected client for all requests
 while time.time() - start_time < timeout:
 try:
 response = await self.http_client.post(
 f'{self.base_url.rstrip("/")}/api/v1/oauth/device/token',
 data={
 'grant_type': 'urn:ietf:params:oauth:grant-type:device_code',
 'device_code': device_code,
 'client_id': self.client_id,
 },
)

 if response.status_code == 200:
 data = response.json()

 # Check for pending authorization
 if data.get('error') == 'authorization_pending':
 await asyncio.sleep(interval)
 continue

 # Check for slow down
 if data.get('error') == 'slow_down':
 interval = data.get('interval', interval * 2)
 await asyncio.sleep(interval)
 continue

 # Check for other errors
 if 'error' in data:
 print(f'Error: {data.get("error_description", data["error"])}')
 return None

 # Success! We have a token
 if 'access_token' in data:
 return data

 elif response.status_code == 400:
 # Error response
 data = response.json()
 if data.get('error') not in ['authorization_pending', 'slow_down']:
 print(f'Error: {data.get("error_description", "Unknown error")}')
 return None

 except:
 else:
 print(f'Unexpected status code: {response.status_code}')
 return None

 except Exception as e:
 print(f'Error polling for token: {e}')

 await asyncio.sleep(interval)
 else:
 # Create a new client for polling
 async with httpx.AsyncClient() as client:
 while time.time() - start_time < timeout:

```

```

 try:
 response = await client.post(
 f'{self.base_url.rstrip("/")}/api/v1/oauth/device/token',
 data={
 'grant_type': 'urn:ietf:params:oauth:grant-type:device_code',
 'device_code': device_code,
 'client_id': self.client_id,
 },
)

 if response.status_code == 200:
 data = response.json()

 # Check for pending authorization
 if data.get('error') == 'authorization_pending':
 await asyncio.sleep(interval)
 continue

 # Check for slow down
 if data.get('error') == 'slow_down':
 interval = data.get('interval', interval * 2)
 await asyncio.sleep(interval)
 continue

 # Check for other errors
 if 'error' in data:
 print(f'Error: {data.get("error_description", data["error"])}')
 return None

 # Success! We have a token
 if 'access_token' in data:
 return data

 elif response.status_code == 400:
 # Error response
 data = response.json()
 if data.get('error') not in ['authorization_pending', 'slow_down']:
 print(f'Error: {data.get("error_description", "Unknown error")}')
 return None

 else:
 print(f'Unexpected status code: {response.status_code}')
 return None

 except Exception as e:
 print(f'Error polling for token: {e}')

 await asyncio.sleep(interval)

 return None

 async def authenticate(
 self,
 agent_session_id: str | None = None,
 show_instructions: bool = True,
) -> bool:
 """
 Run the full authentication flow.
 Returns True if authentication successful.
 """
 import logging

 logger = logging.getLogger(__name__)

 try:
 # Start device authorization
 device_auth = await self.start_device_authorization(agent_session_id)

 # Use frontend URL for user-facing links
 frontend_url = CONFIG.BROWSER_USE_CLOUD_UI_URL or self.base_url.replace('//api.', '//cloud.')

 # Replace backend URL with frontend URL in verification URIs

```

```

#####verification_uri = device_auth['verification_uri'].replace(self.base_url, frontend_url)
#####verification_uri_complete = device_auth['verification_uri_complete'].replace(self.base_url, frontend

#####terminal_width, _terminal_height = shutil.get_terminal_size((80, 20))
#####if show_instructions and CONFIG.BROWSER_USE_CLOUD_SYNC:
#####logger.info('■' * max(terminal_width - 40, 20))
#####logger.info('■ View the details of this run in Browser Use Cloud:')
#####logger.info(f'■ {verification_uri_complete}')
#####logger.info('■' * max(terminal_width - 40, 20) + '\n')

Poll for token
#####token_data = await self.poll_for_token(
#####device_code=device_auth['device_code'],
#####interval=device_auth.get('interval', 5),
#####)

#####if token_data and token_data.get('access_token'):
Save authentication
#####self.auth_config.api_token = token_data['access_token']
#####self.auth_config.user_id = token_data.get('user_id', self.temp_user_id)
#####self.auth_config.authorized_at = datetime.now()
#####self.auth_config.save_to_file()

#####if show_instructions:
#####logger.debug('■ Authentication successful! Cloud sync is now enabled with your browser-use account')

#####return True

#####except httpx.HTTPStatusError as e:
HTTP error with response
#####if e.response.status_code == 404:
#####logger.warning(
#####'Cloud sync authentication endpoint not found (404). Check your BROWSER_USE_CLOUD_API_URL setting')
#####)
#####else:
#####logger.warning(f'Failed to authenticate with cloud service: HTTP {e.response.status_code} - {e.response.text}')
#####except httpx.RequestError as e:
Connection/network errors
logger.warning(f'Failed to connect to cloud service: {type(e).__name__}: {e}')
#####pass
#####except Exception as e:
Other unexpected errors
#####logger.warning(f'■ Unexpected error during cloud sync authentication: {type(e).__name__}: {e}')

#####if show_instructions:
#####logger.debug(f'■ Sync authentication failed or timed out with {CONFIG.BROWSER_USE_CLOUD_API_URL}')

#####return False

#####def get_headers(self) -> dict:
#####"""Get headers for API requests"""
#####if self.api_token:
#####return {'Authorization': f'Bearer {self.api_token}'}
#####return {}

#####def clear_auth(self) -> None:
#####"""Clear stored authentication"""
#####self.auth_config = CloudAuthConfig()

Remove the config file entirely instead of saving empty values
#####config_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'cloud_auth.json'
#####config_path.unlink(missing_ok=True)

```

## ***.lsystems\sync\service.py***

```

"""
Cloud sync service for sending events to the Browser Use cloud.
"""

```

```

import logging

import httpx
from bubus import BaseEvent

from system.config import CONFIG
from system.sync.auth import TEMP_USER_ID, DeviceAuthClient

logger = logging.getLogger(__name__)

class CloudSync:
 """Service for syncing events to the Browser Use cloud"""

 def __init__(self, base_url: str | None = None, allow_session_events_for_auth: bool = False):
 """# Backend API URL for all API requests - can be passed directly or defaults to env var
 self.base_url = base_url or CONFIG.BROWSER_USE_CLOUD_API_URL
 self.auth_client = DeviceAuthClient(base_url=self.base_url)
 self.session_id: str | None = None
 self.allow_session_events_for_auth = allow_session_events_for_auth
 self.auth_flow_active = False # Flag to indicate auth flow is running
 """# Check if cloud sync is actually enabled - if not, we should remain silent
 self.enabled = CONFIG.BROWSER_USE_CLOUD_SYNC

 async def handle_event(self, event: BaseEvent) -> None:
 """Handle an event by sending it to the cloud"""
 try:
 """# If cloud sync is disabled, don't handle any events
 if not self.enabled:
 return

 """# Extract session ID from CreateAgentSessionEvent
 if event.event_type == 'CreateAgentSessionEvent' and hasattr(event, 'id'):
 self.session_id = str(event.id) # type: ignore

 """# Send events based on authentication status and context
 if self.auth_client.is_authenticated:
 """# User is authenticated - send all events
 await self._send_event(event)
 elif self.allow_session_events_for_auth:
 """# Special case: allow ALL events during auth flow
 await self._send_event(event)
 """# Mark auth flow as active when we see a session event
 if event.event_type == 'CreateAgentSessionEvent':
 self.auth_flow_active = True
 else:
 """# User is not authenticated and no auth in progress - don't send anything
 logger.debug(f'Skipping event {event.event_type} - user not authenticated')

 except Exception as e:
 logger.error(f'Failed to handle {event.event_type} event: {type(e).__name__}: {e}', exc_info=True)

 async def _send_event(self, event: BaseEvent) -> None:
 """Send event to cloud API"""
 try:
 headers = {}

 """# Override user_id only if it's not already set to a specific value
 """# This allows CLI and other code to explicitly set temp user_id when needed
 if self.auth_client and self.auth_client.is_authenticated:
 """# Only override if we're fully authenticated and event doesn't have temp user_id
 current_user_id = getattr(event, 'user_id', None)
 if current_user_id != TEMP_USER_ID:
 setattr(event, 'user_id', str(self.auth_client.user_id))
 else:
 """# Set temp user_id if not already set
 if not hasattr(event, 'user_id') or not getattr(event, 'user_id', None):
 setattr(event, 'user_id', TEMP_USER_ID)

 """# Add auth headers if available
 if self.auth_client:
 headers.update(self.auth_client.get_headers())

```

```

Send event (batch format with direct BaseEvent serialization)
async with httpx.AsyncClient() as client:
Serialize event and add device_id to all events
event_data = event.model_dump(mode='json')
if self.auth_client and self.auth_client.device_id:
event_data['device_id'] = self.auth_client.device_id

response = await client.post(
f'{self.base_url.rstrip("/")}/api/v1/events',
json={'events': [event_data]},
headers=headers,
timeout=10.0,
)

if response.status_code >= 400:
Log error but don't raise - we want to fail silently
logger.debug(
f'Failed to send sync event: POST {response.request.url} {response.status_code} - {response.text}'
)
except httpx.TimeoutException:
logger.debug(f'Event send timed out after 10 seconds: {event}')
except httpx.ConnectError as e:
logger.warning(f'Failed to connect to cloud service at {self.base_url}: {e}')
pass
except httpx.HTTPError as e:
logger.debug(f'HTTP error sending event {event}: {type(e).__name__}: {e}')
except Exception as e:
logger.debug(f'Unexpected error sending event {event}: {type(e).__name__}: {e}')

async def _update_wal_user_ids(self, session_id: str) -> None:
"""Update user IDs in WAL file after authentication"""
try:
assert self.auth_client, 'Cloud sync must be authenticated to update WAL user ID'

wal_path = CONFIG.BROWSER_USE_CONFIG_DIR / 'events' / f'{session_id}.jsonl'
if not await anyio.Path(wal_path).exists():
raise FileNotFoundError(
f'CloudSync failed to update saved event user_ids after auth: Agent EventBus WAL file not found'
)

Read all events
events = []
content = await anyio.Path(wal_path).read_text()
for line in content.splitlines():
if line.strip():
events.append(json.loads(line))

Update user_id and device_id
user_id = self.auth_client.user_id
device_id = self.auth_client.device_id
for event in events:
if 'user_id' in event:
event['user_id'] = user_id
Add device_id to all events
event['device_id'] = device_id

Write back
updated_content = '\n'.join(json.dumps(event) for event in events) + '\n'
await anyio.Path(wal_path).write_text(updated_content)

except Exception as e:
logger.warning(f'Failed to update WAL user IDs: {e}')

def set_auth_flow_active(self) -> None:
"""Mark auth flow as active to allow all events"""
self.auth_flow_active = True

async def authenticate(self, show_instructions: bool = True) -> bool:
"""Authenticate with the cloud service"""
If cloud sync is disabled, don't authenticate
if not self.enabled:
return False

```

```

 ■■# Check if already authenticated first
 ■■if self.auth_client.is_authenticated:
 ■■■import logging

 ■■■logger = logging.getLogger(__name__)
 ■■■if show_instructions:
 ■■■■logger.info('■ Already authenticated! Skipping OAuth flow.')
 ■■■return True

 ■■# Not authenticated - run OAuth flow
 ■■return await self.auth_client.authenticate(agent_session_id=self.session_id, show_instructions=show_i

```

## ***.\system\sync\\_\_init\_\_.py***

```

"""Cloud sync module for Browser Use."""

from system.sync.auth import CloudAuthConfig, DeviceAuthClient
from system.sync.service import CloudSync

__all__ = ['CloudAuthConfig', 'DeviceAuthClient', 'CloudSync']

```

## ***.\system\telemetry\service.py***

```

import logging
import os

from dotenv import load_dotenv
from posthog import Posthog
from uuid_extensions import uuid7str

from system.telemetry.views import BaseTelemetryEvent
from system.utils import singleton

load_dotenv()

from system.config import CONFIG

logger = logging.getLogger(__name__)

POSTHOG_EVENT_SETTINGS = {
 ■'process_person_profile': True,
}

@singleton
class ProductTelemetry:
 ■"""
 ■Service for capturing anonymized telemetry data.

 ■If the environment variable `ANONYMIZED_TELEMETRY=False`, anonymized telemetry will be disabled.
 ■"""

 ■USER_ID_PATH = str(CONFIG.BROWSER_USE_CONFIG_DIR / 'device_id')
 ■PROJECT_API_KEY = 'phc_F8JMNjW1i2KbGUTaw1unnDdLSPCoyc52SGRU0JecaUh'
 ■HOST = 'https://eu.i.posthog.com'
 ■UNKNOWN_USER_ID = 'UNKNOWN'

 ■_curr_user_id = None

 ■def __init__(self) -> None:
 ■■telemetry_disabled = not CONFIG.ANONYMIZED_TELEMETRY
 ■■self.debug_logging = CONFIG.BROWSER_USE_LOGGING_LEVEL == 'debug'
 ■■if telemetry_disabled:

```

```

 self._posthog_client = None
 else:
 logger.info('Using anonymized telemetry, see https://docs.browser-use.com/development/monitoring/telemetry')
 self._posthog_client = Posthog(
 project_api_key=self.PROJECT_API_KEY,
 host=self.HOST,
 disable_geoip=False,
 enable_exception_autocapture=True,
)

 # Silence posthog's logging
 if not self.debug_logging:
 posthog_logger = logging.getLogger('posthog')
 posthog_logger.disabled = True

 if self._posthog_client is None:
 logger.debug('Telemetry disabled')

 def capture(self, event: BaseTelemetryEvent) -> None:
 if self._posthog_client is None:
 return

 self._direct_capture(event)

 def _direct_capture(self, event: BaseTelemetryEvent) -> None:
 """
 Should not be thread blocking because posthog magically handles it
 """
 if self._posthog_client is None:
 return

 try:
 self._posthog_client.capture(
 distinct_id=self.user_id,
 event=event.name,
 properties={**event.properties, **POSTHOG_EVENT_SETTINGS},
)
 except Exception as e:
 logger.error(f'Failed to send telemetry event {event.name}: {e}')

 def flush(self) -> None:
 if self._posthog_client:
 try:
 self._posthog_client.flush()
 logger.debug('PostHog client telemetry queue flushed.')
 except Exception as e:
 logger.error(f'Failed to flush PostHog client: {e}')
 else:
 logger.debug('PostHog client not available, skipping flush.')

 @property
 def user_id(self) -> str:
 if self._curr_user_id:
 return self._curr_user_id

 # File access may fail due to permissions or other reasons. We don't want to
 # crash so we catch all exceptions.
 try:
 if not os.path.exists(self.USER_ID_PATH):
 os.makedirs(os.path.dirname(self.USER_ID_PATH), exist_ok=True)
 with open(self.USER_ID_PATH, 'w') as f:
 new_user_id = uuid7str()
 f.write(new_user_id)
 self._curr_user_id = new_user_id
 else:
 with open(self.USER_ID_PATH) as f:
 self._curr_user_id = f.read()
 except Exception:
 self._curr_user_id = 'UNKNOWN_USER_ID'
 return self._curr_user_id

```



## ***.\system\telemetry\views.py***

```
from abc import ABC, abstractmethod
from collections.abc import Sequence
from dataclasses import asdict, dataclass
from typing import Any, Literal

from system.config import CONFIG

@dataclass
class BaseTelemetryEvent(ABC):
 ■@property
 ■@abstractmethod
 ■def name(self) -> str:
 ■ pass

 ■@property
 ■def properties(self) -> dict[str, Any]:
 ■ props = {k: v for k, v in asdict(self).items() if k != 'name'}
 ■ ■# Add Docker context if running in Docker
 ■ props['is_docker'] = CONFIG.IN_DOCKER
 ■ return props

@dataclass
class AgentTelemetryEvent(BaseTelemetryEvent):
 ■# start details
 ■task: str
 ■model: str
 ■model_provider: str
 ■max_steps: int
 ■max_actions_per_step: int
 ■use_vision: bool | Literal['auto']
 ■version: str
 ■source: str
 ■cdp_url: str | None
 ■agent_type: str | None # 'code' for CodeAgent, None for regular Agent
 ■# step details
 ■action_errors: Sequence[str | None]
 ■action_history: Sequence[list[dict] | None]
 ■urls_visited: Sequence[str | None]
 ■# end details
 ■steps: int
 ■total_input_tokens: int
 ■total_output_tokens: int
 ■prompt_cached_tokens: int
 ■total_tokens: int
 ■total_duration_seconds: float
 ■success: bool | None
 ■final_result_response: str | None
 ■error_message: str | None
 ■# judge details
 ■judge_verdict: bool | None = None
 ■judge_reasoning: str | None = None
 ■judge_failure_reason: str | None = None
 ■judge_reached_captcha: bool | None = None
 ■judge_impossible_task: bool | None = None

 ■name: str = 'agent_event'

@dataclass
class MCPClientTelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for MCP client usage"""

 ■server_name: str
 ■command: str
 ■tools_discovered: int
 ■version: str
```

```

 ■action: str # 'connect', 'disconnect', 'tool_call'
 ■tool_name: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None

 ■name: str = 'mcp_client_event'

@dataclass
class MCPServerTelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for MCP server usage"""

 ■version: str
 ■action: str # 'start', 'stop', 'tool_call'
 ■tool_name: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None
 ■parent_process_cmdline: str | None = None

 ■name: str = 'mcp_server_event'

@dataclass
class CLITelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for CLI usage"""

 ■version: str
 ■action: str # 'start', 'message_sent', 'task_completed', 'error'
 ■mode: str # 'interactive', 'oneshot', 'mcp_server'
 ■model: str | None = None
 ■model_provider: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None

 ■name: str = 'cli_event'

```

## ***.\system\telemetry\\_\_init\_\_.py***

```

"""
Telemetry for Browser Use.
"""

from typing import TYPE_CHECKING

Type stubs for lazy imports
if TYPE_CHECKING:
 ■from system.telemetry.service import ProductTelemetry
 ■from system.telemetry.views import (
 ■■BaseTelemetryEvent,
 ■■CLITelemetryEvent,
 ■■MCPClientTelemetryEvent,
 ■■MCPServerTelemetryEvent,
 ■)

Lazy imports mapping
_LAZY_IMPORTS = {
 ■'ProductTelemetry': ('system.telemetry.service', 'ProductTelemetry'),
 ■'BaseTelemetryEvent': ('system.telemetry.views', 'BaseTelemetryEvent'),
 ■'CLITelemetryEvent': ('system.telemetry.views', 'CLITelemetryEvent'),
 ■'MCPClientTelemetryEvent': ('system.telemetry.views', 'MCPClientTelemetryEvent'),
 ■'MCPServerTelemetryEvent': ('system.telemetry.views', 'MCPServerTelemetryEvent'),
}

def __getattr__(name: str):
 ■"""Lazy import mechanism for telemetry components."""
 ■if name in _LAZY_IMPORTS:
 ■■module_path, attr_name = _LAZY_IMPORTS[name]

```

```

 try:
 from importlib import import_module

 module = import_module(module_path)
 attr = getattr(module, attr_name)
 # Cache the imported attribute in the module's globals
 globals()[name] = attr
 return attr
 except ImportError as e:
 raise ImportError(f'Failed to import {name} from {module_path}: {e}') from e

 raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

__all__ = [
 'BaseTelemetryEvent',
 'ProductTelemetry',
 'CLITelemetryEvent',
 'MCPClientTelemetryEvent',
 'MCPServerTelemetryEvent',
]

```

## ***.system\telemetry\telemetry\service.py***

```

import logging
import os

from dotenv import load_dotenv
from posthog import Posthog
from uuid_extensions import uuid7str

from system.telemetry.views import BaseTelemetryEvent
from system.utils import singleton

load_dotenv()

from system.config import CONFIG

logger = logging.getLogger(__name__)

POSTHOG_EVENT_SETTINGS = {
 'process_person_profile': True,
}

@singleton
class ProductTelemetry:
 """
 Service for capturing anonymized telemetry data.

 If the environment variable `ANONYMIZED_TELEMETRY=False`, anonymized telemetry will be disabled.
 """

 USER_ID_PATH = str(CONFIG.BROWSER_USE_CONFIG_DIR / 'device_id')
 PROJECT_API_KEY = 'phc_F8JMNjW1i2KbGUTaW1unnDdLSPCoYc52SGRU0JecaUh'
 HOST = 'https://eu.i.posthog.com'
 UNKNOWN_USER_ID = 'UNKNOWN'

 _curr_user_id = None

 def __init__(self) -> None:
 self.telemetry_disabled = not CONFIG.ANONYMIZED_TELEMETRY
 self.debug_logging = CONFIG.BROWSER_USE_LOGGING_LEVEL == 'debug'

 if telemetry_disabled:
 self._posthog_client = None
 else:

```

```

 logger.info('Using anonymized telemetry, see https://docs.browser-use.com/development/monitoring/telemetry')
 self._posthog_client = Posthog(
 project_api_key=self.PROJECT_API_KEY,
 host=self.HOST,
 disable_geoip=False,
 enable_exception_autocapture=True,
)

 # Silence posthog's logging
 if not self.debug_logging:
 posthog_logger = logging.getLogger('posthog')
 posthog_logger.disabled = True

 if self._posthog_client is None:
 logger.debug('Telemetry disabled')

 def capture(self, event: BaseTelemetryEvent) -> None:
 if self._posthog_client is None:
 return

 self._direct_capture(event)

 def _direct_capture(self, event: BaseTelemetryEvent) -> None:
 """
 Should not be thread blocking because posthog magically handles it
 """
 if self._posthog_client is None:
 return

 try:
 self._posthog_client.capture(
 distinct_id=self.user_id,
 event=event.name,
 properties={**event.properties, **POSTHOG_EVENT_SETTINGS},
)
 except Exception as e:
 logger.error(f'Failed to send telemetry event {event.name}: {e}')

 def flush(self) -> None:
 if self._posthog_client:
 try:
 self._posthog_client.flush()
 logger.debug('PostHog client telemetry queue flushed.')
 except Exception as e:
 logger.error(f'Failed to flush PostHog client: {e}')
 else:
 logger.debug('PostHog client not available, skipping flush.')

 @property
 def user_id(self) -> str:
 if self._curr_user_id:
 return self._curr_user_id

 # File access may fail due to permissions or other reasons. We don't want to
 # crash so we catch all exceptions.
 try:
 if not os.path.exists(self.USER_ID_PATH):
 os.makedirs(os.path.dirname(self.USER_ID_PATH), exist_ok=True)
 with open(self.USER_ID_PATH, 'w') as f:
 new_user_id = uuid7str()
 f.write(new_user_id)
 self._curr_user_id = new_user_id
 except Exception:
 self._curr_user_id = 'UNKNOWN_USER_ID'
 return self._curr_user_id

```

## ***.\system\telemetry\telemetry\views.py***

```
from abc import ABC, abstractmethod
from collections.abc import Sequence
from dataclasses import asdict, dataclass
from typing import Any, Literal

from system.config import is_running_in_docker

@dataclass
class BaseTelemetryEvent(ABC):
 ■@property
 ■@abstractmethod
 ■def name(self) -> str:
 ■ pass

 ■@property
 ■def properties(self) -> dict[str, Any]:
 ■ props = {k: v for k, v in asdict(self).items() if k != 'name'}
 ■ ■# Add Docker context if running in Docker
 ■ props['is_docker'] = is_running_in_docker()
 ■ return props

@dataclass
class AgentTelemetryEvent(BaseTelemetryEvent):
 ■# start details
 ■task: str
 ■model: str
 ■model_provider: str
 ■max_steps: int
 ■max_actions_per_step: int
 ■use_vision: bool | Literal['auto']
 ■version: str
 ■source: str
 ■cdp_url: str | None
 ■agent_type: str | None # 'code' for CodeAgent, None for regular Agent
 ■# step details
 ■action_errors: Sequence[str | None]
 ■action_history: Sequence[list[dict] | None]
 ■urls_visited: Sequence[str | None]
 ■# end details
 ■steps: int
 ■total_input_tokens: int
 ■total_output_tokens: int
 ■prompt_cached_tokens: int
 ■total_tokens: int
 ■total_duration_seconds: float
 ■success: bool | None
 ■final_result_response: str | None
 ■error_message: str | None
 ■# judge details
 ■judge_verdict: bool | None = None
 ■judge_reasoning: str | None = None
 ■judge_failure_reason: str | None = None
 ■judge_reached_captcha: bool | None = None
 ■judge_impossible_task: bool | None = None

 ■name: str = 'agent_event'

@dataclass
class MCPClientTelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for MCP client usage"""

 ■server_name: str
 ■command: str
 ■tools_discovered: int
 ■version: str
```

```

 ■action: str # 'connect', 'disconnect', 'tool_call'
 ■tool_name: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None

 ■name: str = 'mcp_client_event'

@dataclass
class MCPServerTelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for MCP server usage"""

 ■version: str
 ■action: str # 'start', 'stop', 'tool_call'
 ■tool_name: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None
 ■parent_process_cmdline: str | None = None

 ■name: str = 'mcp_server_event'

@dataclass
class CLITelemetryEvent(BaseTelemetryEvent):
 ■"""Telemetry event for CLI usage"""

 ■version: str
 ■action: str # 'start', 'message_sent', 'task_completed', 'error'
 ■mode: str # 'interactive', 'oneshot', 'mcp_server'
 ■model: str | None = None
 ■model_provider: str | None = None
 ■duration_seconds: float | None = None
 ■error_message: str | None = None

 ■name: str = 'cli_event'

```

## **`.system\telemetry\telemetry\__init__.py`**

```

"""
Telemetry for Browser Use.
"""

from typing import TYPE_CHECKING

Type stubs for lazy imports
if TYPE_CHECKING:
 ■from system.telemetry.service import ProductTelemetry
 ■from system.telemetry.views import (
 ■■BaseTelemetryEvent,
 ■■CLITelemetryEvent,
 ■■MCPClientTelemetryEvent,
 ■■MCPServerTelemetryEvent,
 ■)

Lazy imports mapping
_LAZY_IMPORTS = {
 ■'ProductTelemetry': ('system.telemetry.service', 'ProductTelemetry'),
 ■'BaseTelemetryEvent': ('system.telemetry.views', 'BaseTelemetryEvent'),
 ■'CLITelemetryEvent': ('system.telemetry.views', 'CLITelemetryEvent'),
 ■'MCPClientTelemetryEvent': ('system.telemetry.views', 'MCPClientTelemetryEvent'),
 ■'MCPServerTelemetryEvent': ('system.telemetry.views', 'MCPServerTelemetryEvent'),
}

def __getattr__(name: str):
 ■"""Lazy import mechanism for telemetry components."""
 ■if name in _LAZY_IMPORTS:
 ■■module_path, attr_name = _LAZY_IMPORTS[name]

```

```

try:
 from importlib import import_module

 module = import_module(module_path)
 attr = getattr(module, attr_name)
 # Cache the imported attribute in the module's globals
 globals()[name] = attr
 return attr
except ImportError as e:
 raise ImportError(f'Failed to import {name} from {module_path}: {e}') from e

raise AttributeError(f"module '{__name__}' has no attribute '{name}'")

__all__ = [
 'BaseTelemetryEvent',
 'ProductTelemetry',
 'CLITelemetryEvent',
 'MCPClientTelemetryEvent',
 'MCPServerTelemetryEvent',
]

```

## ***.system\tokens\custom\_pricing.py***

```

"""
Custom model pricing for models not available in LiteLLM's pricing data.

Prices are per token (not per 1M tokens).
"""

from typing import Any

Custom model pricing data
Format matches LiteLLM's model_prices_and_context_window.json structure
CUSTOM_MODEL_PRICING: dict[str, dict[str, Any]] = {
 'bu-1-0': {
 'input_cost_per_token': 0.2 / 1_000_000, # $0.20 per 1M tokens
 'output_cost_per_token': 2.00 / 1_000_000, # $2.00 per 1M tokens
 'cache_read_input_token_cost': 0.02 / 1_000_000, # $0.02 per 1M tokens
 'cache_creation_input_token_cost': None, # Not specified
 'max_tokens': None, # Not specified
 'max_input_tokens': None, # Not specified
 'max_output_tokens': None, # Not specified
 },
 'bu-2-0': {
 'input_cost_per_token': 0.60 / 1_000_000, # $0.60 per 1M tokens
 'output_cost_per_token': 3.50 / 1_000_000, # $3.50 per 1M tokens
 'cache_read_input_token_cost': 0.06 / 1_000_000, # $0.06 per 1M tokens
 'cache_creation_input_token_cost': None, # Not specified
 'max_tokens': None, # Not specified
 'max_input_tokens': None, # Not specified
 'max_output_tokens': None, # Not specified
 },
}
CUSTOM_MODEL_PRICING['bu-latest'] = CUSTOM_MODEL_PRICING['bu-1-0']

CUSTOM_MODEL_PRICING['smart'] = CUSTOM_MODEL_PRICING['bu-1-0']

```

## ***.system\tokens\mappings.py***

```

Mapping from model_name to LiteLLM model name
MODEL_TO_LITELLM: dict[str, str] = {
 'gemini-flash-latest': 'gemini/gemini-flash-latest',
}

```

## **`.\system\tokens\service.py`**

```
"""
Token cost service that tracks LLM token usage and costs.

Fetches pricing data from LiteLLM repository and caches it for 1 day.
Automatically tracks token usage when LLMs are registered and invoked.
"""

import logging
import os
from datetime import datetime, timedelta
from pathlib import Path
from typing import Any

import anyio
import httpx
from dotenv import load_dotenv

from system.llm.base import BaseChatModel
from system.llm.views import ChatInvokeUsage
from system.tokens.custom_pricing import CUSTOM_MODEL_PRICING
from system.tokens.mappings import MODEL_TO_LITELLM
from system.tokens.views import (
 ■CachedPricingData,
 ■ModelPricing,
 ■ModelUsageStats,
 ■ModelUsageTokens,
 ■TokenCostCalculated,
 ■TokenUsageEntry,
 ■UsageSummary,
)
from system.utils import create_task_with_error_handling

load_dotenv()

from system.config import CONFIG

logger = logging.getLogger(__name__)
cost_logger = logging.getLogger('cost')

def xdg_cache_home() -> Path:
 ■default = Path.home() / '.cache'
 ■if CONFIG.XDG_CACHE_HOME and (path := Path(CONFIG.XDG_CACHE_HOME)).is_absolute():
 ■ return path
 ■return default

class TokenCost:
 ■"""Service for tracking token usage and calculating costs"""

 ■CACHE_DIR_NAME = 'system/token_cost'
 ■CACHE_DURATION = timedelta(days=1)
 ■PRICING_URL = 'https://raw.githubusercontent.com/BerriAI/litellm/main/model_prices_and_context_window.json'

 ■def __init__(self, include_cost: bool = False):
 ■ self.include_cost = include_cost or os.getenv('BROWSER_USE_CALCULATE_COST', 'false').lower() == 'true'

 ■ self.usage_history: list[TokenUsageEntry] = []
 ■ self.registered_llms: dict[str, BaseChatModel] = {}
 ■ self._pricing_data: dict[str, Any] | None = None
 ■ self._initialized = False
 ■ self._cache_dir = xdg_cache_home() / self.CACHE_DIR_NAME

 ■ async def initialize(self) -> None:
 ■ ■"""Initialize the service by loading pricing data"""
```



```

 if not self._initialized:
 if self.include_cost:
 await self._load_pricing_data()
 self._initialized = True

 async def _load_pricing_data(self) -> None:
 """Load pricing data from cache or fetch from GitHub"""
 # Try to find a valid cache file
 cache_file = await self._find_valid_cache()

 if cache_file:
 await self._load_from_cache(cache_file)
 else:
 await self._fetch_and_cache_pricing_data()

 async def _find_valid_cache(self) -> Path | None:
 """Find the most recent valid cache file"""
 try:
 # Ensure cache directory exists
 self._cache_dir.mkdir(parents=True, exist_ok=True)

 # List all JSON files in the cache directory
 cache_files = list(self._cache_dir.glob('*.json'))

 if not cache_files:
 return None

 # Sort by modification time (most recent first)
 cache_files.sort(key=lambda f: f.stat().st_mtime, reverse=True)

 # Check each file until we find a valid one
 for cache_file in cache_files:
 if await self._is_cache_valid(cache_file):
 return cache_file
 else:
 # Clean up old cache files
 try:
 os.remove(cache_file)
 except Exception:
 pass

 return None
 except Exception:
 return None

 async def _is_cache_valid(self, cache_file: Path) -> bool:
 """Check if a specific cache file is valid and not expired"""
 try:
 if not cache_file.exists():
 return False

 # Read the cached data
 cached = CachedPricingData.model_validate_json(await anyio.Path(cache_file).read_text())

 # Check if cache is still valid
 return datetime.now() - cached.timestamp < self.CACHE_DURATION
 except Exception:
 return False

 async def _load_from_cache(self, cache_file: Path) -> None:
 """Load pricing data from a specific cache file"""
 try:
 content = await anyio.Path(cache_file).read_text()
 cached = CachedPricingData.model_validate_json(content)
 self._pricing_data = cached.data
 except Exception as e:
 logger.debug(f'Error loading cached pricing data from {cache_file}: {e}')
 # Fall back to fetching
 await self._fetch_and_cache_pricing_data()

 async def _fetch_and_cache_pricing_data(self) -> None:
 """Fetch pricing data from LiteLLM GitHub and cache it with timestamp"""

```

```

 try:
 async with httpx.AsyncClient() as client:
 response = await client.get(self.PRICING_URL, timeout=30)
 response.raise_for_status()

 self._pricing_data = response.json()

 # Create cache object with timestamp
 cached = CachedPricingData(timestamp=datetime.now(), data=self._pricing_data or {})

 # Ensure cache directory exists
 self._cache_dir.mkdir(parents=True, exist_ok=True)

 # Create cache file with timestamp in filename
 timestamp_str = datetime.now().strftime('%Y%m%d_%H%M%S')
 cache_file = self._cache_dir / f'pricing_{timestamp_str}.json'

 await anyio.Path(cache_file).write_text(cached.model_dump_json(indent=2))
 except Exception as e:
 logger.debug(f'Error fetching pricing data: {e}')
 # Fall back to empty pricing data
 self._pricing_data = {}

 async def get_model_pricing(self, model_name: str) -> ModelPricing | None:
 """Get pricing information for a specific model"""
 # Ensure we're initialized
 if not self._initialized:
 await self.initialize()

 # Check custom pricing first
 if model_name in CUSTOM_MODEL_PRICING:
 data = CUSTOM_MODEL_PRICING[model_name]
 return ModelPricing(
 model=model_name,
 input_cost_per_token=data.get('input_cost_per_token'),
 output_cost_per_token=data.get('output_cost_per_token'),
 max_tokens=data.get('max_tokens'),
 max_input_tokens=data.get('max_input_tokens'),
 max_output_tokens=data.get('max_output_tokens'),
 cache_read_input_token_cost=data.get('cache_read_input_token_cost'),
 cache_creation_input_token_cost=data.get('cache_creation_input_token_cost'),
)

 # Map model name to LiteLLM model name if needed
 litellm_model_name = MODEL_TO_LITELLM.get(model_name, model_name)

 if not self._pricing_data or litellm_model_name not in self._pricing_data:
 return None

 data = self._pricing_data[litellm_model_name]
 return ModelPricing(
 model=model_name,
 input_cost_per_token=data.get('input_cost_per_token'),
 output_cost_per_token=data.get('output_cost_per_token'),
 max_tokens=data.get('max_tokens'),
 max_input_tokens=data.get('max_input_tokens'),
 max_output_tokens=data.get('max_output_tokens'),
 cache_read_input_token_cost=data.get('cache_read_input_token_cost'),
 cache_creation_input_token_cost=data.get('cache_creation_input_token_cost'),
)

 async def calculate_cost(self, model: str, usage: ChatInvokeUsage) -> TokenCostCalculated | None:
 if not self.include_cost:
 return None

 data = await self.get_model_pricing(model)
 if data is None:
 return None

 uncached_prompt_tokens = usage.prompt_tokens - (usage.prompt_cached_tokens or 0)

 return TokenCostCalculated(

```

```

new_prompt_tokens=usage.prompt_tokens,
new_prompt_cost=uncached_prompt_tokens * (data.input_cost_per_token or 0),
Cached tokens
prompt_read_cached_tokens=usage.prompt_cached_tokens,
prompt_read_cached_cost=usage.prompt_cached_tokens * data.cache_read_input_token_cost
if usage.prompt_cached_tokens and data.cache_read_input_token_cost
else None,
Cache creation tokens
prompt_cached_creation_tokens=usage.prompt_cache_creation_tokens,
prompt_cache_creation_cost=usage.prompt_cache_creation_tokens * data.cache_creation_input_token_cost
if data.cache_creation_input_token_cost and usage.prompt_cache_creation_tokens
else None,
Completion tokens
completion_tokens=usage.completion_tokens,
completion_cost=usage.completion_tokens * float(data.output_cost_per_token or 0),
)

def add_usage(self, model: str, usage: ChatInvokeUsage) -> TokenUsageEntry:
 """Add token usage entry to history (without calculating cost)"""
 entry = TokenUsageEntry(
 model=model,
 timestamp=datetime.now(),
 usage=usage,
)

 self.usage_history.append(entry)

 return entry

async def _log_non_usage_llm(self, llm: BaseChatModel) -> None:
"""Log non-usage to the logger"""
C_CYAN = '\033[96m'
C_RESET = '\033[0m'

cost_logger.debug(f'{C_CYAN}{llm.model}{C_RESET} (no usage found)')

 async def _log_usage(self, model: str, usage: TokenUsageEntry) -> None:
 """Log usage to the logger"""
 if not self._initialized:
 await self.initialize()

 # ANSI color codes
 C_CYAN = '\033[96m'
 C_YELLOW = '\033[93m'
 C_GREEN = '\033[92m'
 C_BLUE = '\033[94m'
 C_RESET = '\033[0m'

 # Always get cost breakdown for token details (even if not showing costs)
 cost = await self.calculate_cost(model, usage.usage)

 # Build input tokens breakdown
 input_part = self._build_input_tokens_display(usage.usage, cost)

 # Build output tokens display
 completion_tokens_fmt = self._format_tokens(usage.usage.completion_tokens)
 if self.include_cost and cost and cost.completion_cost > 0:
 output_part = f'{C_GREEN}{completion_tokens_fmt} (${cost.completion_cost:.4f}){C_RESET}'
 else:
 output_part = f'{C_GREEN}{completion_tokens_fmt}{C_RESET}'

 cost_logger.debug(f'{C_CYAN}{model}{C_RESET} | {input_part} | {output_part}')

 def _build_input_tokens_display(self, usage: ChatInvokeUsage, cost: TokenCostCalculated | None) -> str:
 """Build a clear display of input tokens breakdown with emojis and optional costs"""
 C_YELLOW = '\033[93m'
 C_BLUE = '\033[94m'
 C_RESET = '\033[0m'

 parts = []

 # Always show token breakdown if we have cache information, regardless of cost tracking

```

```

 if usage.prompt_cached_tokens or usage.prompt_cache_creation_tokens:
 # Calculate actual new tokens (non-cached)
 new_tokens = usage.prompt_tokens - (usage.prompt_cached_tokens or 0)

 if new_tokens > 0:
 new_tokens_fmt = self._format_tokens(new_tokens)
 if self.include_cost and cost and cost.new_prompt_cost > 0:
 parts.append(f'■ {C_YELLOW}{new_tokens_fmt} (${cost.new_prompt_cost:.4f}){C_RESET}')
 else:
 parts.append(f'■ {C_YELLOW}{new_tokens_fmt}{C_RESET}')

 if usage.prompt_cached_tokens:
 cached_tokens_fmt = self._format_tokens(usage.prompt_cached_tokens)
 if self.include_cost and cost and cost.prompt_read_cached_cost:
 parts.append(f'■ {C_BLUE}{cached_tokens_fmt} (${cost.prompt_read_cached_cost:.4f}){C_RESET}')
 else:
 parts.append(f'■ {C_BLUE}{cached_tokens_fmt}{C_RESET}')

 if usage.prompt_cache_creation_tokens:
 creation_tokens_fmt = self._format_tokens(usage.prompt_cache_creation_tokens)
 if self.include_cost and cost and cost.prompt_cache_creation_cost:
 parts.append(f'■ {C_BLUE}{creation_tokens_fmt} (${cost.prompt_cache_creation_cost:.4f}){C_RESET}')
 else:
 parts.append(f'■ {C_BLUE}{creation_tokens_fmt}{C_RESET}')

 if not parts:
 # Fallback to simple display when no cache information available
 total_tokens_fmt = self._format_tokens(usage.prompt_tokens)
 if self.include_cost and cost and cost.new_prompt_cost > 0:
 parts.append(f'■ {C_YELLOW}{total_tokens_fmt} (${cost.new_prompt_cost:.4f}){C_RESET}')
 else:
 parts.append(f'■ {C_YELLOW}{total_tokens_fmt}{C_RESET}')

 return ' ' + '.join(parts)

def register_llm(self, llm: BaseChatModel) -> BaseChatModel:
 """
 Register an LLM to automatically track its token usage

 @dev Guarantees that the same instance is not registered multiple times
 """
 # Use instance ID as key to avoid collisions between multiple instances
 instance_id = str(id(llm))

 # Check if this exact instance is already registered
 if instance_id in self.registered_llms:
 logger.debug(f'LLM instance {instance_id} ({llm.provider}_{llm.model}) is already registered')
 return llm

 self.registered_llms[instance_id] = llm

 # Store the original method
 original_ainvoke = llm.ainvoke
 # Store reference to self for use in the closure
 token_cost_service = self

 # Create a wrapped version that tracks usage
 async def tracked_ainvoke(messages, output_format=None, **kwargs):
 # Call the original method, passing through any additional kwargs
 result = await original_ainvoke(messages, output_format, **kwargs)

 # Track usage if available (no await needed since add_usage is now sync)
 # Use llm.model instead of llm.name for consistency with get_usage_tokens_for_model()
 if result.usage:
 usage = token_cost_service.add_usage(llm.model, result.usage)

 logger.debug(f'Token cost service: {usage}')

 create_task_with_error_handling(
 token_cost_service._log_usage(llm.model, usage), name='log_token_usage', suppress_exceptions=True
)
 # else:

```

```

 await token_cost_service._log_non_usage_llm(llm)

 return result

 ## Replace the method with our tracked version
 ## Using setattr to avoid type checking issues with overloaded methods
 setattr(llm, 'ainvoke', tracked_ainvoke)

 return llm

def get_usage_tokens_for_model(self, model: str) -> ModelUsageTokens:
 """Get usage tokens for a specific model"""
 filtered_usage = [u for u in self.usage_history if u.model == model]

 return ModelUsageTokens(
 model=model,
 prompt_tokens=sum(u.usage.prompt_tokens for u in filtered_usage),
 prompt_cached_tokens=sum(u.usage.prompt_cached_tokens or 0 for u in filtered_usage),
 completion_tokens=sum(u.usage.completion_tokens for u in filtered_usage),
 total_tokens=sum(u.usage.prompt_tokens + u.usage.completion_tokens for u in filtered_usage),
)

 async def get_usage_summary(self, model: str | None = None, since: datetime | None = None) -> UsageSummary:
 """Get summary of token usage and costs (costs calculated on-the-fly)"""
 filtered_usage = self.usage_history

 if model:
 filtered_usage = [u for u in filtered_usage if u.model == model]

 if since:
 filtered_usage = [u for u in filtered_usage if u.timestamp >= since]

 if not filtered_usage:
 return UsageSummary(
 total_prompt_tokens=0,
 total_prompt_cost=0.0,
 total_prompt_cached_tokens=0,
 total_prompt_cached_cost=0.0,
 total_completion_tokens=0,
 total_completion_cost=0.0,
 total_tokens=0,
 total_cost=0.0,
 entry_count=0,
)

 ## Calculate totals
 total_prompt = sum(u.usage.prompt_tokens for u in filtered_usage)
 total_completion = sum(u.usage.completion_tokens for u in filtered_usage)
 total_tokens = total_prompt + total_completion
 total_prompt_cached = sum(u.usage.prompt_cached_tokens or 0 for u in filtered_usage)
 models = list({u.model for u in filtered_usage})

 ## Calculate per-model stats with record-by-record cost calculation
 model_stats: dict[str, ModelUsageStats] = {}
 total_prompt_cost = 0.0
 total_completion_cost = 0.0
 total_prompt_cached_cost = 0.0

 for entry in filtered_usage:
 if entry.model not in model_stats:
 model_stats[entry.model] = ModelUsageStats(model=entry.model)

 stats = model_stats[entry.model]
 stats.prompt_tokens += entry.usage.prompt_tokens
 stats.completion_tokens += entry.usage.completion_tokens
 stats.total_tokens += entry.usage.prompt_tokens + entry.usage.completion_tokens
 stats.invocations += 1

 if self.include_cost:
 ## Calculate cost record by record using the updated calculate_cost function
 cost = await self.calculate_cost(entry.model, entry.usage)
 if cost:

```

```

 stats.cost += cost.total_cost
 total_prompt_cost += cost.prompt_cost
 total_completion_cost += cost.completion_cost
 total_prompt_cached_cost += cost.prompt_read_cached_cost or 0

 ## Calculate averages
 for stats in model_stats.values():
 if stats.invocations > 0:
 stats.average_tokens_per_invocation = stats.total_tokens / stats.invocations

 return UsageSummary(
 total_prompt_tokens=total_prompt,
 total_prompt_cost=total_prompt_cost,
 total_prompt_cached_tokens=total_prompt_cached,
 total_prompt_cached_cost=total_prompt_cached_cost,
 total_completion_tokens=total_completion,
 total_completion_cost=total_completion_cost,
 total_tokens=total_tokens,
 total_cost=total_prompt_cost + total_completion_cost + total_prompt_cached_cost,
 entry_count=len(filtered_usage),
 by_model=model_stats,
)

 def _format_tokens(self, tokens: int) -> str:
 """Format token count with k suffix for thousands"""
 if tokens >= 1000000000:
 return f'{tokens / 1000000000:.1f}B'
 if tokens >= 1000000:
 return f'{tokens / 1000000:.1f}M'
 if tokens >= 1000:
 return f'{tokens / 1000:.1f}k'
 return str(tokens)

 async def log_usage_summary(self) -> None:
 """Log a comprehensive usage summary per model with colors and nice formatting"""
 if not self.usage_history:
 return

 summary = await self.get_usage_summary()

 if summary.entry_count == 0:
 return

 ## ANSI color codes
 C_CYAN = '\033[96m'
 C_YELLOW = '\033[93m'
 C_GREEN = '\033[92m'
 C_BLUE = '\033[94m'
 C_MAGENTA = '\033[95m'
 C_RESET = '\033[0m'
 C_BOLD = '\033[1m'

 ## Log overall summary
 total_tokens_fmt = self._format_tokens(summary.total_tokens)
 prompt_tokens_fmt = self._format_tokens(summary.total_prompt_tokens)
 completion_tokens_fmt = self._format_tokens(summary.total_completion_tokens)

 ## Format cost breakdowns for input and output (only if cost tracking is enabled)
 if self.include_cost and summary.total_cost > 0:
 total_cost_part = f'({C_MAGENTA}{summary.total_cost:.4f}{C_RESET})'
 prompt_cost_part = f'({C_MAGENTA}{summary.total_prompt_cost:.4f})'
 completion_cost_part = f'({C_MAGENTA}{summary.total_completion_cost:.4f})'
 else:
 total_cost_part = ''
 prompt_cost_part = ''
 completion_cost_part = ''

 if len(summary.by_model) > 1:
 cost_logger.debug(
 f'{C_BOLD}Total Usage Summary{C_RESET}: {C_BLUE}{total_tokens_fmt} tokens{C_RESET}{total_cost_part}'
 f'{C_YELLOW}{prompt_tokens_fmt}{prompt_cost_part}{C_RESET} | ➡ {C_GREEN}{completion_tokens_fmt}'
)

```

```

 for model, stats in summary.by_model.items():
 # Format tokens
 model_total_fmt = self._format_tokens(stats.total_tokens)
 model_prompt_fmt = self._format_tokens(stats.prompt_tokens)
 model_completion_fmt = self._format_tokens(stats.completion_tokens)
 avg_tokens_fmt = self._format_tokens(int(stats.average_tokens_per_invocation))

 # Format cost display (only if cost tracking is enabled)
 if self.include_cost:
 # Calculate per-model costs on-the-fly
 total_model_cost = 0.0
 model_prompt_cost = 0.0
 model_completion_cost = 0.0

 # Calculate costs for this model
 for entry in self.usage_history:
 if entry.model == model:
 cost = await self.calculate_cost(entry.model, entry.usage)
 if cost:
 model_prompt_cost += cost.prompt_cost
 model_completion_cost += cost.completion_cost

 total_model_cost = model_prompt_cost + model_completion_cost

 if total_model_cost > 0:
 cost_part = f'({C_MAGENTA}{total_model_cost:.4f}{C_RESET})'
 prompt_part = f'{C_YELLOW}{model_prompt_fmt}({model_prompt_cost:.4f}){C_RESET}'
 completion_part = f'{C_GREEN}{model_completion_fmt}({model_completion_cost:.4f}){C_RESET}'
 else:
 cost_part = ''
 prompt_part = f'{C_YELLOW}{model_prompt_fmt}{C_RESET}'
 completion_part = f'{C_GREEN}{model_completion_fmt}{C_RESET}'
 else:
 cost_part = ''
 prompt_part = f'{C_YELLOW}{model_prompt_fmt}{C_RESET}'
 completion_part = f'{C_GREEN}{model_completion_fmt}{C_RESET}'

 cost_logger.debug(
 f' {C_CYAN}{model}{C_RESET}: {C_BLUE}{model_total_fmt} tokens{C_RESET}{cost_part} | '
 f' {C_YELLOW}{model_prompt_fmt}{C_RESET} | {C_GREEN}{model_completion_fmt}{C_RESET} | '
 f' {stats.invocations} calls | {avg_tokens_fmt}/call'
)

 async def get_cost_by_model(self) -> dict[str, ModelUsageStats]:
 """Get cost breakdown by model"""
 summary = await self.get_usage_summary()
 return summary.by_model

 def clear_history(self) -> None:
 """Clear usage history"""
 self.usage_history = []

 async def refresh_pricing_data(self) -> None:
 """Force refresh of pricing data from GitHub"""
 if self.include_cost:
 await self._fetch_and_cache_pricing_data()

 async def clean_old_caches(self, keep_count: int = 3) -> None:
 """Clean up old cache files, keeping only the most recent ones"""
 try:
 # List all JSON files in the cache directory
 cache_files = list(self._cache_dir.glob('*.json'))

 if len(cache_files) <= keep_count:
 return

 # Sort by modification time (oldest first)
 cache_files.sort(key=lambda f: f.stat().st_mtime)

 # Remove all but the most recent files
 for cache_file in cache_files[:-keep_count]:
 try:

```

```

 os.remove(cache_file)
 except Exception:
 pass
 except Exception as e:
 logger.debug(f'Error cleaning old cache files: {e}')

 async def ensure_pricing_loaded(self) -> None:
 """Ensure pricing data is loaded in the background. Call this after creating the service."""
 if not self._initialized and self.include_cost:
 # This will run in the background and won't block
 await self.initialize()

```

## ***.\system\tokens\views.py***

```

from datetime import datetime
from typing import Any, TypeVar

from pydantic import BaseModel, Field

from system.llm.views import ChatInvokeUsage

T = TypeVar('T', bound=BaseModel)

class TokenUsageEntry(BaseModel):
 """Single token usage entry"""

 model: str
 timestamp: datetime
 usage: ChatInvokeUsage

class TokenCostCalculated(BaseModel):
 """Token cost"""

 new_prompt_tokens: int
 new_prompt_cost: float

 prompt_read_cached_tokens: int | None
 prompt_read_cached_cost: float | None

 prompt_cached_creation_tokens: int | None
 prompt_cache_creation_cost: float | None
 """Anthropic only: The cost of creating the cache."""

 completion_tokens: int
 completion_cost: float

 @property
 def prompt_cost(self) -> float:
 return self.new_prompt_cost + (self.prompt_read_cached_cost or 0) + (self.prompt_cache_creation_cost or 0)

 @property
 def total_cost(self) -> float:
 return (
 self.new_prompt_cost
 + (self.prompt_read_cached_cost or 0)
 + (self.prompt_cache_creation_cost or 0)
 + self.completion_cost
)

class ModelPricing(BaseModel):
 """Pricing information for a model"""

 model: str
 input_cost_per_token: float | None
 output_cost_per_token: float | None

```



```

■cache_read_input_token_cost: float | None
■cache_creation_input_token_cost: float | None

■max_tokens: int | None
■max_input_tokens: int | None
■max_output_tokens: int | None

class CachedPricingData(BaseModel):
 """Cached pricing data with timestamp"""

 ■timestamp: datetime
 ■data: dict[str, Any]

class ModelUsageStats(BaseModel):
 """Usage statistics for a single model"""

 ■model: str
 ■prompt_tokens: int = 0
 ■completion_tokens: int = 0
 ■total_tokens: int = 0
 ■cost: float = 0.0
 ■invocations: int = 0
 ■average_tokens_per_invocation: float = 0.0

class ModelUsageTokens(BaseModel):
 """Usage tokens for a single model"""

 ■model: str
 ■prompt_tokens: int
 ■prompt_cached_tokens: int
 ■completion_tokens: int
 ■total_tokens: int

class UsageSummary(BaseModel):
 """Summary of token usage and costs"""

 ■total_prompt_tokens: int
 ■total_prompt_cost: float

 ■total_prompt_cached_tokens: int
 ■total_prompt_cached_cost: float

 ■total_completion_tokens: int
 ■total_completion_cost: float
 ■total_tokens: int
 ■total_cost: float
 ■entry_count: int

 ■by_model: dict[str, ModelUsageStats] = Field(default_factory=dict)

```

***.\system\tokens\\_\_init\_\_.py***

***.\system\tokens\tests\test\_cost.py***

```

"""
Simple test for token cost tracking with real LLM calls.

Tests ChatOpenAI and ChatGoogle by iteratively generating countries.
"""

```

```

import asyncio
import logging

from system.llm import ChatGoogle, ChatOpenAI
from system.llm.messages import AssistantMessage, SystemMessage, UserMessage
from system.tokens.service import TokenCost

Optional OCI import
try:
 from examples.models.oci_models import meta_llm

 OCI_MODELS_AVAILABLE = True
except ImportError:
 meta_llm = None
 OCI_MODELS_AVAILABLE = False

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

def get_oci_model_if_available():
 """Create OCI model for testing if credentials are available."""
 if not OCI_MODELS_AVAILABLE:
 return None

 # Try to create OCI model with mock/test configuration
 # These values should be replaced with real ones if testing with actual OCI
 try:
 # get any of the llm xai_llm or cohere_llm
 return meta_llm

 except Exception as e:
 logger.info(f'OCI model not available for testing: {e}')
 return None

async def test_iterative_country_generation():
 """Test token cost tracking with iterative country generation"""

 # Initialize token cost service
 tc = TokenCost(include_cost=True)

 # System prompt that explains the iterative task
 system_prompt = """You are a country name generator. When asked, you will provide exactly ONE country name. Each time you're asked to continue, provide the next country name that hasn't been mentioned yet. Keep track of which countries you've already said and don't repeat them. Only output the country name, no numbers, no punctuation, just the name."""

 # Test with different models
 models = []
 models.append(ChatOpenAI(model='gpt-4.1')) # Commented out - requires OPENAI_API_KEY
 models.append(ChatGoogle(model='gemini-2.0-flash-exp'))

 # Add OCI model if available
 oci_model = get_oci_model_if_available()
 if oci_model:
 models.append(oci_model)
 print(f'OCI model added to test: {oci_model.name}')
 else:
 print('OCI model not available (install with pip install browser-use[oci] and configure credentials)')

 print('\nIterative Country Generation Test')
 print('=' * 80)

 for llm in models:
 print(f'\nTesting {llm.model}')
 print('-' * 60)

 # Register the LLM for automatic tracking
 tc.register_llm(llm)

 # Initialize conversation

```

```

■ messages = [SystemMessage(content=system_prompt), UserMessage(content='Give me a country name')]
■ countries = []

■ # Generate 10 countries iteratively
■ for i in range(10):
■ # Call the LLM
■ result = await llm.ainvoke(messages)
■ country = result.completion.strip()
■ countries.append(country)

■ # Add the response to messages
■ messages.append(AssistantMessage(content=country))

■ # Add the next request (except for the last iteration)
■ if i < 9:
■ messages.append(UserMessage(content='Next country please'))

■ print(f' Country {i + 1}: {country}')

■ print(f'\n Generated countries: {"", ".join(countries)}')

■ # Display cost summary
■ print('\n■ Cost Summary')
■ print('=' * 80)

■ summary = await tc.get_usage_summary()
■ print(f'Total calls: {summary.entry_count}')
■ print(f'Total tokens: {summary.total_tokens:,}')
■ print(f'Total cost: ${summary.total_cost:.6f}')

■ expected_cost = 0
■ expected_invocations = 0

■ print('\n■ Cost breakdown by model:')
■ for model, stats in summary.by_model.items():
■ expected_cost += stats.cost
■ expected_invocations += stats.invocations

■ print(f'\n{model}:')
■ print(f' Calls: {stats.invocations}')
■ print(f' Prompt tokens: {stats.prompt_tokens:,}')
■ print(f' Completion tokens: {stats.completion_tokens:,}')
■ print(f' Total tokens: {stats.total_tokens:,}')
■ print(f' Cost: ${stats.cost:.6f}')
■ print(f' Average tokens per call: {stats.average_tokens_per_invocation:.1f}')

■ assert summary.entry_count == expected_invocations, f'Expected {expected_invocations} invocations, got {summary.entry_count}'
■ assert abs(summary.total_cost - expected_cost) < 1e-6, (
■ f'Expected total cost ${expected_cost:.6f}, got ${summary.total_cost:.6f}'
■)

if __name__ == '__main__':
■ # Run the test
■ asyncio.run(test_iterative_country_generation())

```

## ***.\systemtools\service.py***

```

import asyncio
import json
import logging
import os
from typing import Generic, TypeVar

import anyio

try:

```

```

from lmnr import Laminar # type: ignore
except ImportError:
 Laminar = None # type: ignore
from pydantic import BaseModel

from system.agent.views import ActionModel, ActionResult
from system.browser import BrowserSession
from system.browser.events import (
 ClickCoordinateEvent,
 ClickElementEvent,
 CloseTabEvent,
 GetDropdownOptionsEvent,
 GoBackEvent,
 NavigateToUrlEvent,
 ScrollEvent,
 ScrollToTextEvent,
 SendKeysEvent,
 SwitchTabEvent,
 TypeTextEvent,
 UploadFileEvent,
)
from system.browser.views import BrowserError
from system.dom.service import EnhancedDOMTreeNode
from system.filesystem.file_system import FileSystem
from system.llm.base import BaseChatModel
from system.llm.messages import SystemMessage, UserMessage
from system.observability import observe_debug
from system.tools.registry.service import Registry
from system.tools.utils import get_click_description
from system.tools.views import (
 ClickElementAction,
 ClickElementActionIndexOnly,
 CloseTabAction,
 DoneAction,
 ExtractAction,
 FindElementsAction,
 GetDropdownOptionsAction,
 InputTextAction,
 NavigateAction,
 NoParamsAction,
 ReadContentAction,
 SaveAsPdfAction,
 ScreenshotAction,
 ScrollAction,
 SearchAction,
 SearchPageAction,
 SelectDropdownOptionAction,
 SendKeysAction,
 StructuredOutputAction,
 SwitchTabAction,
 UploadFileAction,
)
from system.utils import create_task_with_error_handling, sanitize_surrogates, time_execution_sync

logger = logging.getLogger(__name__)

Import EnhancedDOMTreeNode and rebuild event models that have forward references to it
This must be done after all imports are complete
ClickElementEvent.model_rebuild()
TypeTextEvent.model_rebuild()
ScrollEvent.model_rebuild()
UploadFileEvent.model_rebuild()

Context = TypeVar('Context')

T = TypeVar('T', bound=BaseModel)

def _detect_sensitive_key_name(text: str, sensitive_data: dict[str, str | dict[str, str]] | None) -> str:
 """Detect which sensitive key name corresponds to the given text value."""
 if not sensitive_data or not text:
 return None

```

```

 ## Collect all sensitive values and their keys
 for domain_or_key, content in sensitive_data.items():
 if isinstance(content, dict):
 ## New format: {domain: {key: value}}
 for key, value in content.items():
 if value and value == text:
 return key
 elif content: # Old format: {key: value}
 if content == text:
 return domain_or_key

 return None

def handle_browser_error(e: BrowserError) -> ActionResult:
 if e.long_term_memory is not None:
 if e.short_term_memory is not None:
 return ActionResult(
 extracted_content=e.short_term_memory, error=e.long_term_memory, include_extracted_content_only_once=True
)
 else:
 return ActionResult(error=e.long_term_memory)
 # Fallback to original error handling if long_term_memory is None
 logger.warning(
 'A BrowserError was raised without long_term_memory - always set long_term_memory when raising BrowserError'
)
 raise e

--- JS templates for search_page and find_elements ---

_SEARCH_PAGE_JS_BODY = """\
try {
 var scope = CSS_SCOPE ? document.querySelector(CSS_SCOPE) : document.body;
 if (!scope) {
 return {error: 'CSS scope selector not found: ' + CSS_SCOPE, matches: [], total: 0};
 }
 var walker = document.createTreeWalker(scope, NodeFilter.SHOW_TEXT);
 var fullText = '';
 var nodeOffsets = [];
 while (walker.nextNode()) {
 var node = walker.currentNode;
 var text = node.textContent;
 if (text && text.trim()) {
 nodeOffsets.push({offset: fullText.length, length: text.length, node: node});
 fullText += text;
 }
 }
 var re;
 try {
 var flags = CASE_SENSITIVE ? 'g' : 'gi';
 if (IS_REGEX) {
 re = new RegExp(PATTERN, flags);
 } else {
 re = new RegExp(PATTERN.replace(/[*+?^$(){}|\[\]\\\\]/g, '\\\\$&'), flags);
 }
 } catch (e) {
 return {error: 'Invalid regex pattern: ' + e.message, matches: [], total: 0};
 }
 var matches = [];
 var match;
 var totalFound = 0;
 while ((match = re.exec(fullText)) !== null) {
 totalFound++;
 if (matches.length < MAX_RESULTS) {
 var start = Math.max(0, match.index - CONTEXT_CHARS);
 var end = Math.min(fullText.length, match.index + match[0].length + CONTEXT_CHARS);
 var context = fullText.slice(start, end);
 var elementPath = '';
 for (var i = 0; i < nodeOffsets.length; i++) {
 var no = nodeOffsets[i];
 if (no.offset <= match.index && no.offset + no.length > match.index) {

```

```

#####elementPath = _getPath(no.node.parentElement);
#####break;
#####}
#####}
#####matches.push({
#####match_text: match[0],
#####context: (start > 0 ? '...' : '') + context + (end < fullText.length ? '...' : ''),
#####element_path: elementPath,
#####char_position: match.index
#####});
#####}
#####if (match[0].length === 0) re.lastIndex++;
#####}
#####return {matches: matches, total: totalFound, has_more: totalFound > MAX_RESULTS};
#####} catch (e) {
#####return {error: 'search_page error: ' + e.message, matches: [], total: 0};
#####}
#####function _getPath(el) {
#####var parts = [];
#####var current = el;
#####while (current && current !== document.body && current !== document) {
#####var desc = current.tagName ? current.tagName.toLowerCase() : '';
#####if (!desc) break;
#####if (current.id) desc += '#' + current.id;
#####else if (current.className && typeof current.className === 'string') {
#####var classes = current.className.trim().split(/\s+/).slice(0, 2).join('.');
#####if (classes) desc += '.' + classes;
#####}
#####parts.unshift(desc);
#####current = current.parentElement;
#####}
#####return parts.join(' > ');
#####}
#####}

_FIND_ELEMENTS_JS_BODY = """"\
try {
#####var elements;
#####try {
#####elements = document.querySelectorAll(SELECTOR);
#####} catch (e) {
#####return {error: 'Invalid CSS selector: ' + e.message, elements: [], total: 0};
#####}
#####var total = elements.length;
#####var limit = Math.min(total, MAX_RESULTS);
#####var results = [];
#####for (var i = 0; i < limit; i++) {
#####var el = elements[i];
#####var item = {index: i, tag: el.tagName.toLowerCase()};
#####if (INCLUDE_TEXT) {
#####var text = (el.textContent || '').trim();
#####item.text = text.length > 300 ? text.slice(0, 300) + '...' : text;
#####}
#####if (ATTRIBUTES && ATTRIBUTES.length > 0) {
#####item.attrs = {};
#####for (var j = 0; j < ATTRIBUTES.length; j++) {
#####var val = el.getAttribute(ATTRIBUTES[j]);
#####if (val !== null) {
#####item.attrs[ATTRIBUTES[j]] = val.length > 500 ? val.slice(0, 500) + '...' : val;
#####}
#####}
#####}
#####item.children_count = el.children.length;
#####results.push(item);
#####}
#####return {elements: results, total: total, showing: limit};
#####} catch (e) {
#####return {error: 'find_elements error: ' + e.message, elements: [], total: 0};
#####}
#####}

def _build_search_page_js(
#####pattern: str,

```

```

■regex: bool,
■case_sensitive: bool,
■context_chars: int,
■css_scope: str | None,
■max_results: int,
) -> str:
■"""Build JS IIFE for search_page with safe parameter injection."""
■params_js = (
■■f'var PATTERN = {json.dumps(pattern)};\n'
■■f'var IS_REGEX = {json.dumps(regex)};\n'
■■f'var CASE_SENSITIVE = {json.dumps(case_sensitive)};\n'
■■f'var CONTEXT_CHARS = {json.dumps(context_chars)};\n'
■■f'var CSS_SCOPE = {json.dumps(css_scope)};\n'
■■f'var MAX_RESULTS = {json.dumps(max_results)};\n'
■■)
■return '(function() {\n' + params_js + _SEARCH_PAGE_JS_BODY + '\n})();)'

```

```

def _build_find_elements_js(
■selector: str,
■attributes: list[str] | None,
■max_results: int,
■include_text: bool,
) -> str:
■"""Build JS IIFE for find_elements with safe parameter injection."""
■params_js = (
■■f'var SELECTOR = {json.dumps(selector)};\n'
■■f'var ATTRIBUTES = {json.dumps(attributes)};\n'
■■f'var MAX_RESULTS = {json.dumps(max_results)};\n'
■■f'var INCLUDE_TEXT = {json.dumps(include_text)};\n'
■■)
■return '(function() {\n' + params_js + _FIND_ELEMENTS_JS_BODY + '\n})();)'

```

```

def _format_search_results(data: dict, pattern: str) -> str:
■"""Format search_page CDP result into human-readable text for the agent."""
■if not isinstance(data, dict):
■■return f'search_page returned unexpected result: {data}'

```

```

■matches = data.get('matches', [])
■total = data.get('total', 0)
■has_more = data.get('has_more', False)

```

```

■if total == 0:
■■return f'No matches found for "{pattern}" on page.'

```

```

■lines = [f'Found {total} match{"es" if total != 1 else ""} for "{pattern}" on page:']
■lines.append('')
■for i, m in enumerate(matches):
■■context = m.get('context', '')
■■path = m.get('element_path', '')
■■loc = f' (in {path})' if path else ''
■■lines.append(f'[{i + 1}] {context}{loc}')

```

```

■if has_more:
■■lines.append(f'\n... showing {len(matches)} of {total} total matches. Increase max_results to see more')

```

```

■return '\n'.join(lines)

```

```

def _format_find_results(data: dict, selector: str) -> str:
■"""Format find_elements CDP result into human-readable text for the agent."""
■if not isinstance(data, dict):
■■return f'find_elements returned unexpected result: {data}'

```

```

■elements = data.get('elements', [])
■total = data.get('total', 0)
■showing = data.get('showing', 0)

```

```

■if total == 0:
■■return f'No elements found matching "{selector}".'
■lines = [f'Found {total} element{"s" if total != 1 else ""} matching "{selector}":']

```

```

 lines.append('')
 for el in elements:
 idx = el.get('index', 0)
 tag = el.get('tag', '?')
 text = el.get('text', '')
 attrs = el.get('attrs', {})
 children = el.get('children_count', 0)

 ## Build element description
 parts = [f'[{idx}] <{tag}>']
 if text:
 ## Collapse whitespace for readability
 display_text = ' '.join(text.split())
 if len(display_text) > 120:
 display_text = display_text[:120] + '...'
 parts.append(f'"{display_text}"')
 if attrs:
 attr_strs = [f'{k}="{v}"' for k, v in attrs.items()]
 parts.append('{' + ', '.join(attr_strs) + '}')
 parts.append(f'({children} children)')
 lines.append(' '.join(parts))

 if showing < total:
 lines.append(f'\nShowing {showing} of {total} total elements. Increase max_results to see more.')

 return '\n'.join(lines)

def _is_autocomplete_field(node: EnhancedDOMTreeNode) -> bool:
 """Detect if a node is an autocomplete/combobox field from its attributes."""
 attrs = node.attributes or {}
 if attrs.get('role') == 'combobox':
 return True
 aria_ac = attrs.get('aria-autocomplete', '')
 if aria_ac and aria_ac != 'none':
 return True
 if attrs.get('list'):
 return True
 haspopup = attrs.get('aria-haspopup', '')
 if haspopup and haspopup != 'false' and (attrs.get('aria-controls') or attrs.get('aria-owns')):
 return True
 return False

class Tools(Generic[Context]):
 def __init__(
 self,
 exclude_actions: list[str] | None = None,
 output_model: type[T] | None = None,
 display_files_in_done_text: bool = True,
):
 self.registry = Registry[Context](exclude_actions if exclude_actions is not None else [])
 self.display_files_in_done_text = display_files_in_done_text
 self._output_model: type[BaseModel] | None = output_model
 self._coordinate_clicking_enabled: bool = False

 """Register all default browser actions"""

 self._register_done_action(output_model)

 ## Basic Navigation Actions
 @self.registry.action(
 '',
 param_model=SearchAction,
 terminates_sequence=True,
)
 async def search(params: SearchAction, browser_session: BrowserSession):
 import urllib.parse

 ## Encode query for URL safety
 encoded_query = urllib.parse.quote_plus(params.query)
 ## Build search URL based on search engine

```



```

search_engines = {
 'duckduckgo': f'https://duckduckgo.com/?q={encoded_query}',
 'google': f'https://www.google.com/search?q={encoded_query}&udm=14',
 'bing': f'https://www.bing.com/search?q={encoded_query}',
}

if params.engine.lower() not in search_engines:
 return ActionResult(error=f'Unsupported search engine: {params.engine}. Options: duckduckgo, google')

search_url = search_engines[params.engine.lower()]

Simple tab logic: use current tab by default
use_new_tab = False

Dispatch navigation event
try:
 event = browser_session.event_bus.dispatch(
 NavigateToUrlEvent(
 url=search_url,
 new_tab=use_new_tab,
)
)
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)
 memory = f"Searched {params.engine.title()} for '{params.query}'"
 msg = f'■ {memory}'
 logger.info(msg)
 return ActionResult(extracted_content=memory, long_term_memory=memory)
except Exception as e:
 logger.error(f'Failed to search {params.engine}: {e}')
 return ActionResult(error=f'Failed to search {params.engine} for "{params.query}": {str(e)}')

@self.registry.action(
 '',
 param_model=NavigateAction,
 terminates_sequence=True,
)
async def navigate(params: NavigateAction, browser_session: BrowserSession):
 try:
 # Dispatch navigation event
 event = browser_session.event_bus.dispatch(NavigateToUrlEvent(url=params.url, new_tab=params.new_tab))
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)

 if params.new_tab:
 memory = f'Opened new tab with URL {params.url}'
 msg = f'■ Opened new tab with url {params.url}'
 else:
 memory = f'Navigated to {params.url}'
 msg = f'■ {memory}'

 logger.info(msg)
 return ActionResult(extracted_content=msg, long_term_memory=memory)
 except Exception as e:
 error_msg = str(e)
 # Always log the actual error first for debugging
 browser_session.logger.error(f'■ Navigation failed: {error_msg}')

 # Check if it's specifically a RuntimeError about CDP client
 if isinstance(e, RuntimeError) and 'CDP client not initialized' in error_msg:
 browser_session.logger.error('■ Browser connection failed - CDP client not properly initialized')
 return ActionResult(error=f'Browser connection error: {error_msg}')
 # Check for network-related errors
 elif any(
 err in error_msg
 for err in [
 'ERR_NAME_NOT_RESOLVED',
 'ERR_INTERNET_DISCONNECTED',
 'ERR_CONNECTION_REFUSED',
 'ERR_TIMED_OUT',
 'net::',
]

```

```

):
 site_unavailable_msg = f'Navigation failed - site unavailable: {params.url}'
 browser_session.logger.warning(f'■ {site_unavailable_msg} - {error_msg}')
 return ActionResult(error=site_unavailable_msg)
 else:
 # Return error in ActionResult instead of re-raising
 return ActionResult(error=f'Navigation failed: {str(e)}')

 @self.registry.action('Go back', param_model=NoParamsAction, terminates_sequence=True)
 async def go_back(_: NoParamsAction, browser_session: BrowserSession):
 try:
 event = browser_session.event_bus.dispatch(GoBackEvent())
 await event
 memory = 'Navigated back'
 msg = f'■ {memory}'
 logger.info(msg)
 return ActionResult(extracted_content=memory)
 except Exception as e:
 logger.error(f'Failed to dispatch GoBackEvent: {type(e).__name__}: {e}')
 error_msg = f'Failed to go back: {str(e)}'
 return ActionResult(error=error_msg)

 @self.registry.action('Wait for x seconds.')
 async def wait(seconds: int = 3):
 # Cap wait time at maximum 30 seconds
 # Reduce the wait time by 3 seconds to account for the llm call which takes at least 3 seconds
 # So if the model decides to wait for 5 seconds, the llm call took at least 3 seconds, so we only need 2 more
 # Note by Mert: the above doesn't make sense because we do the LLM call right after this or this could be before
 # so I revert this.
 actual_seconds = min(max(seconds - 1, 0), 30)
 memory = f'Waited for {seconds} seconds'
 logger.info(f'■ waited for {seconds} second{" " if seconds == 1 else "s"}')
 await asyncio.sleep(actual_seconds)
 return ActionResult(extracted_content=memory, long_term_memory=memory)

 ## Helper function for coordinate conversion
 def _convert_llm_coordinates_to_viewport(llm_x: int, llm_y: int, browser_session: BrowserSession) -> tuple[int, int]:
 """Convert coordinates from LLM screenshot size to original viewport size."""
 if browser_session.llm_screenshot_size and browser_session._original_viewport_size:
 original_width, original_height = browser_session._original_viewport_size
 llm_width, llm_height = browser_session.llm_screenshot_size

 ## Convert coordinates using fractions
 actual_x = int((llm_x / llm_width) * original_width)
 actual_y = int((llm_y / llm_height) * original_height)

 logger.info(
 f'■ Converting coordinates: LLM ({llm_x}, {llm_y}) @ {llm_width}x{llm_height} '
 f'→ Viewport ({actual_x}, {actual_y}) @ {original_width}x{original_height}'
)
 return actual_x, actual_y
 return llm_x, llm_y

 ## Element Interaction Actions
 async def _detect_new_tab_opened(
 browser_session: BrowserSession,
 tabs_before: set[str],
) -> str:
 """Detect if a click opened a new tab, and return a note for the agent.
 Waits briefly for CDP events to propagate, then checks if any new tabs appeared.
 """
 try:
 # Brief delay to allow CDP Target.attachedToTarget events to propagate
 # and be processed by SessionManager._handle_target_attached
 await asyncio.sleep(0.05)

 tabs_after = await browser_session.get_tabs()
 new_tabs = [t for t in tabs_after if t.target_id not in tabs_before]
 if new_tabs:
 new_tab_id = new_tabs[0].target_id[-4:]
 return f'. Note: This opened a new tab (tab_id: {new_tab_id}) - switch to it if you need to interact with it.'
 except Exception:

```

```

#####pass
#####return ''

#####async def _click_by_coordinate(params: ClickElementAction, browser_session: BrowserSession) -> ActionResult:
Ensure coordinates are provided (type safety)
if params.coordinate_x is None or params.coordinate_y is None:
return ActionResult(error='Both coordinate_x and coordinate_y must be provided')

try:
Convert coordinates from LLM size to original viewport size if resizing was used
actual_x, actual_y = _convert_llm_coordinates_to_viewport(
params.coordinate_x, params.coordinate_y, browser_session
)

Capture tab IDs before click to detect new tabs
tabs_before = {t.target_id for t in await browser_session.get_tabs()}

Highlight the coordinate being clicked (truly non-blocking)
asyncio.create_task(browser_session.highlight_coordinate_click(actual_x, actual_y))

Dispatch ClickCoordinateEvent - handler will check for safety and click
event = browser_session.event_bus.dispatch(
ClickCoordinateEvent(coordinate_x=actual_x, coordinate_y=actual_y, force=True)
)
await event
Wait for handler to complete and get any exception or metadata
click_metadata = await event.event_result(raise_if_any=True, raise_if_none=False)

Check for validation errors (only happens when force=False)
if isinstance(click_metadata, dict) and 'validation_error' in click_metadata:
error_msg = click_metadata['validation_error']
return ActionResult(error=error_msg)

memory = f'Clicked on coordinate {params.coordinate_x}, {params.coordinate_y}'
memory += await _detect_new_tab_opened(browser_session, tabs_before)
logger.info(f'### {memory}')

return ActionResult(
extracted_content=memory,
metadata={'click_x': actual_x, 'click_y': actual_y},
)
except BrowserError as e:
return handle_browser_error(e)
except Exception as e:
error_msg = f'Failed to click at coordinates ({params.coordinate_x}, {params.coordinate_y}).'
return ActionResult(error=error_msg)

async def _click_by_index(
params: ClickElementAction | ClickElementActionIndexOnly, browser_session: BrowserSession
) -> ActionResult:
assert params.index is not None
try:
assert params.index != 0, (
'Cannot click on element with index 0. If there are no interactive elements use wait(), refresh(),
)

Look up the node from the selector map
node = await browser_session.get_element_by_index(params.index)
if node is None:
msg = f'Element index {params.index} not available - page may have changed. Try refreshing browser'
logger.warning(f'### {msg}')
return ActionResult(extracted_content=msg)

Get description of clicked element
element_desc = get_click_description(node)

Capture tab IDs before click to detect new tabs
tabs_before = {t.target_id for t in await browser_session.get_tabs()}

Highlight the element being clicked (truly non-blocking)
create_task_with_error_handling(
browser_session.highlight_interaction_element(node), name='highlight_click_element', suppress_exc

```

```

)

 event = browser_session.event_bus.dispatch(ClickElementEvent(node=node))
 await event
 # Wait for handler to complete and get any exception or metadata
 click_metadata = await event.event_result(raise_if_any=True, raise_if_none=False)

 # Check if result contains validation error (e.g., trying to click <select> or file input)
 if isinstance(click_metadata, dict) and 'validation_error' in click_metadata:
 error_msg = click_metadata['validation_error']
 # If it's a select element, try to get dropdown options as a helpful shortcut
 if 'Cannot click on <select> elements.' in error_msg:
 try:
 return await dropdown_options(
 params=GetDropdownOptionsAction(index=params.index), browser_session=browser_session
)
 except Exception as dropdown_error:
 logger.debug(
 f'Failed to get dropdown options as shortcut during click on dropdown: {type(dropdown_error)}.'
)
 return ActionResult(error=error_msg)

 # Build memory with element info
 memory = f'Clicked {element_desc}'
 memory += await _detect_new_tab_opened(browser_session, tabs_before)
 logger.info(f'█ {memory}')

 # Include click coordinates in metadata if available
 return ActionResult(
 extracted_content=memory,
 metadata=click_metadata if isinstance(click_metadata, dict) else None,
)
 except BrowserError as e:
 return handle_browser_error(e)
 except Exception as e:
 error_msg = f'Failed to click element {params.index}: {str(e)}'
 return ActionResult(error=error_msg)

 ## Store click handlers for re-registration
 self._click_by_index = _click_by_index
 self._click_by_coordinate = _click_by_coordinate

 ## Register click action (index-only by default)
 self._register_click_action()

 @self.registry.action(
 'Input text into element by index.',
 param_model=InputTextAction,
)
 async def input(
 params: InputTextAction,
 browser_session: BrowserSession,
 has_sensitive_data: bool = False,
 sensitive_data: dict[str, str] | dict[str, str] | None = None,
):
 # Look up the node from the selector map
 node = await browser_session.get_element_by_index(params.index)
 if node is None:
 msg = f'Element index {params.index} not available - page may have changed. Try refreshing browser'
 logger.warning(f'█ {msg}')
 return ActionResult(extracted_content=msg)

 # Highlight the element being typed into (truly non-blocking)
 create_task_with_error_handling(
 browser_session.highlight_interaction_element(node), name='highlight_type_element', suppress_exceptions=True
)

 # Dispatch type text event with node
 try:
 # Detect which sensitive key is being used
 sensitive_key_name = None
 if has_sensitive_data and sensitive_data:

```

```

#####sensitive_key_name = _detect_sensitive_key_name(params.text, sensitive_data)

#####event = browser_session.event_bus.dispatch(
#####TypeTextEvent(
#####node=node,
#####text=params.text,
#####clear=params.clear,
#####is_sensitive=has_sensitive_data,
#####sensitive_key_name=sensitive_key_name,
#####)
#####)
#####await event
#####input_metadata = await event.event_result(raise_if_any=True, raise_if_none=False)

Create message with sensitive data handling
#####if has_sensitive_data:
#####if sensitive_key_name:
#####msg = f'Typed {sensitive_key_name}'
#####log_msg = f'Typed <{sensitive_key_name}>'
#####else:
#####msg = 'Typed sensitive data'
#####log_msg = 'Typed <sensitive>'
#####else:
#####msg = f'Typed '{params.text}''
#####log_msg = f'Typed '{params.text}''

#####logger.debug(log_msg)

Check for value mismatch (non-sensitive only)
#####actual_value = None
#####if isinstance(input_metadata, dict):
#####actual_value = input_metadata.pop('actual_value', None)

#####if not has_sensitive_data and actual_value is not None and actual_value != params.text:
#####msg += f"\n■ Note: the field's actual value '{actual_value}' differs from typed text '{params.te

Check for autocomplete/combobox field – add mechanical delay for dropdown
#####if _is_autocomplete_field(node):
#####msg += '\n■ This is an autocomplete field. Wait for suggestions to appear, then click the correct
Only delay for true JS-driven autocomplete (combobox / aria-autocomplete),
not native <datalist> or loose aria-haspopup which the browser handles instantly
#####attrs = node.attributes or {}
#####if attrs.get('role') == 'combobox' or (attrs.get('aria-autocomplete', '') not in ('', 'none')):
#####await asyncio.sleep(0.4) # let JS dropdown populate before next action

Include input coordinates in metadata if available
#####return ActionResult(
#####extracted_content=msg,
#####long_term_memory=msg,
#####metadata=input_metadata if isinstance(input_metadata, dict) else None,
#####)
#####except BrowserError as e:
#####return handle_browser_error(e)
#####except Exception as e:
Log the full error for debugging
#####logger.error(f'Failed to dispatch TypeTextEvent: {type(e).__name__}: {e}')
#####error_msg = f'Failed to type text into element {params.index}: {e}'
#####return ActionResult(error=error_msg)

#####@self.registry.action(
#####'',
#####param_model=UploadFileAction,
#####)
#####async def upload_file(
#####params: UploadFileAction, browser_session: BrowserSession, available_file_paths: list[str], file_sys
#####):
Check if file is in available_file_paths (user-provided or downloaded files)
For remote browsers (is_local=False), we allow absolute remote paths even if not tracked locally
#####if params.path not in available_file_paths:
Also check if it's a recently downloaded file that might not be in available_file_paths yet
#####downloaded_files = browser_session.downloaded_files
#####if params.path not in downloaded_files:

```

```

Finally, check if it's a file in the FileSystem service
if file_system and file_system.get_dir():
Check if the file is actually managed by the FileSystem service
The path should be just the filename for FileSystem files
file_obj = file_system.get_file(params.path)
if file_obj:
File is managed by FileSystem, construct the full path
file_system_path = str(file_system.get_dir() / params.path)
params = UploadFileAction(index=params.index, path=file_system_path)
else:
If browser is remote, allow passing a remote-accessible absolute path
if not browser_session.is_local:
pass
else:
msg = f'File path {params.path} is not available. To fix: The user must add this file path to t
logger.error(f'█ {msg}')
return ActionResult(error=msg)
else:
If browser is remote, allow passing a remote-accessible absolute path
if not browser_session.is_local:
pass
else:
msg = f'File path {params.path} is not available. To fix: The user must add this file path to t
raise BrowserError(message=msg, long_term_memory=msg)

For local browsers, ensure the file exists and has content
if browser_session.is_local:
if not os.path.exists(params.path):
msg = f'File {params.path} does not exist'
return ActionResult(error=msg)
file_size = os.path.getsize(params.path)
if file_size == 0:
msg = f'File {params.path} is empty (0 bytes). The file may not have been saved correctly.'
return ActionResult(error=msg)

Get the selector map to find the node
selector_map = await browser_session.get_selector_map()
if params.index not in selector_map:
msg = f'Element with index {params.index} does not exist.'
return ActionResult(error=msg)

node = selector_map[params.index]

Helper function to find file input near the selected element
def find_file_input_near_element(
node: EnhancedDOMTreeNode, max_height: int = 3, max_descendant_depth: int = 3
) -> EnhancedDOMTreeNode | None:
"""Find the closest file input to the selected element."""

def find_file_input_in_descendants(n: EnhancedDOMTreeNode, depth: int) -> EnhancedDOMTreeNode | None:
if depth < 0:
return None
if browser_session.is_file_input(n):
return n
for child in n.children_nodes or []:
result = find_file_input_in_descendants(child, depth - 1)
if result:
return result
return None

current = node
for _ in range(max_height + 1):
Check the current node itself
if browser_session.is_file_input(current):
return current
Check all descendants of the current node
result = find_file_input_in_descendants(current, max_descendant_depth)
if result:
return result
Check all siblings and their descendants
if current.parent_node:
for sibling in current.parent_node.children_nodes or []:

```

```

#####if sibling is current:
#####continue
#####if browser_session.is_file_input(sibling):
#####return sibling
#####result = find_file_input_in_descendants(sibling, max_descendant_depth)
#####if result:
#####return result
#####current = current.parent_node
#####if not current:
#####break
#####return None

Try to find a file input element near the selected element
###file_input_node = find_file_input_near_element(node)

Highlight the file input element if found (truly non-blocking)
###if file_input_node:
###create_task_with_error_handling(
#####browser_session.highlight_interaction_element(file_input_node),
#####name='highlight_file_input',
#####suppress_exceptions=True,
#####)

If not found near the selected element, fallback to finding the closest file input to current scroll position
###if file_input_node is None:
###logger.info(
#####f'No file upload element found near index {params.index}, searching for closest file input to scroll position')

Get current scroll position
#####cdp_session = await browser_session.get_or_create_cdp_session()
#####try:
#####scroll_info = await cdp_session.cdp_client.send.Runtime.evaluate(
#####params={'expression': 'window.scrollY || window.pageYOffset || 0'}, session_id=cdp_session.session_id)
#####current_scroll_y = scroll_info.get('result', {}).get('value', 0)
#####except Exception:
#####current_scroll_y = 0

Find all file inputs in the selector map and pick the closest one to scroll position
#####closest_file_input = None
#####min_distance = float('inf')

#####for idx, element in selector_map.items():
#####if browser_session.is_file_input(element):
Get element's Y position
#####if element.absolute_position:
#####element_y = element.absolute_position.y
#####distance = abs(element_y - current_scroll_y)
#####if distance < min_distance:
#####min_distance = distance
#####closest_file_input = element

#####if closest_file_input:
#####file_input_node = closest_file_input
#####logger.info(f'Found file input closest to scroll position (distance: {min_distance}px)')

Highlight the fallback file input element (truly non-blocking)
#####create_task_with_error_handling(
#####browser_session.highlight_interaction_element(file_input_node),
#####name='highlight_file_input_fallback',
#####suppress_exceptions=True,
#####)
#####else:
#####msg = 'No file upload element found on the page'
#####logger.error(msg)
#####raise BrowserError(msg)
TODO: figure out why this fails sometimes + add fallback hail mary, just look for any file input element

Dispatch upload file event with the file input node
#####try:
#####event = browser_session.event_bus.dispatch(UploadFileEvent(node=file_input_node, file_path=params.path))

```

```

#####await event
#####await event.event_result(raise_if_any=True, raise_if_none=False)
#####msg = f'Successfully uploaded file to index {params.index}'
#####logger.info(f'■ {msg}')
#####return ActionResult(
extracted_content=msg,
long_term_memory=f'Uploaded file {params.path} to element {params.index}',
#####)
#####except Exception as e:
logger.error(f'Failed to upload file: {e}')
raise BrowserError(f'Failed to upload file: {e}')

Tab Management Actions

#####@self.registry.action(
'Switch to another open tab by tab_id. Tab IDs are shown in browser state tabs list (last 4 chars of
param_model=SwitchTabAction,
terminates_sequence=True,
#####)
#####async def switch(params: SwitchTabAction, browser_session: BrowserSession):
Simple switch tab logic
try:
target_id = await browser_session.get_target_id_from_tab_id(params.tab_id)

event = browser_session.event_bus.dispatch(SwitchTabEvent(target_id=target_id))
await event
new_target_id = await event.event_result(raise_if_any=False, raise_if_none=False) # Don't raise on
#####
if new_target_id:
memory = f'Switched to tab #{new_target_id[-4:]}'
else:
memory = f'Switched to tab #{params.tab_id}'

logger.info(f'■ {memory}')
return ActionResult(extracted_content=memory, long_term_memory=memory)
except Exception as e:
logger.warning(f'Tab switch may have failed: {e}')
memory = f'Attempted to switch to tab #{params.tab_id}'
return ActionResult(extracted_content=memory, long_term_memory=memory)

#####@self.registry.action(
'Close a tab by tab_id. Tab IDs are shown in browser state tabs list (last 4 chars of target_id). US
param_model=CloseTabAction,
#####)
#####async def close(params: CloseTabAction, browser_session: BrowserSession):
Simple close tab logic
try:
target_id = await browser_session.get_target_id_from_tab_id(params.tab_id)

Dispatch close tab event - handle stale target IDs gracefully
event = browser_session.event_bus.dispatch(CloseTabEvent(target_id=target_id))
await event
await event.event_result(raise_if_any=False, raise_if_none=False) # Don't raise on errors

memory = f'Closed tab #{params.tab_id}'
logger.info(f'■ {memory}')
return ActionResult(
extracted_content=memory,
long_term_memory=memory,
)
except Exception as e:
Handle stale target IDs gracefully
logger.warning(f'Tab {params.tab_id} may already be closed: {e}')
memory = f'Tab #{params.tab_id} closed (was already closed or invalid)'
return ActionResult(
extracted_content=memory,
long_term_memory=memory,
)

#####@self.registry.action(
"""LLM extracts structured data from page markdown. Use when: on right page, know what to extract, h
param_model=ExtractAction,

```



```

)
 async def extract(
 params: ExtractAction,
 browser_session: BrowserSession,
 page_extraction_llm: BaseChatModel,
 file_system: FileSystem,
 extraction_schema: dict | None = None,
):
 # Constants
 MAX_CHAR_LIMIT = 100000
 query = params['query'] if isinstance(params, dict) else params.query
 extract_links = params['extract_links'] if isinstance(params, dict) else params.extract_links
 start_from_char = params['start_from_char'] if isinstance(params, dict) else params.start_from_char
 output_schema: dict | None = params.get('output_schema') if isinstance(params, dict) else params.output_schema

 # If the LLM didn't provide an output_schema, use the agent-injected extraction_schema
 if output_schema is None and extraction_schema is not None:
 output_schema = extraction_schema

 # Attempt to convert output_schema to a pydantic model upfront; fall back to free-text on failure
 structured_model: type[BaseModel] | None = None
 if output_schema is not None:
 try:
 from system.tools.extraction.schema_utils import schema_dict_to_pydantic_model

 structured_model = schema_dict_to_pydantic_model(output_schema)
 except (ValueError, TypeError) as exc:
 logger.warning(f'Invalid output_schema, falling back to free-text extraction: {exc}')
 output_schema = None

 # Extract clean markdown using the unified method
 try:
 from system.dom.markdown_extractor import extract_clean_markdown

 content, content_stats = await extract_clean_markdown(
 browser_session=browser_session, extract_links=extract_links
)
 except Exception as e:
 raise RuntimeError(f'Could not extract clean markdown: {type(e).__name__}')

 # Original content length for processing
 final_filtered_length = content_stats['final_filtered_chars']

 # Structure-aware chunking replaces naive char-based truncation
 from system.dom.markdown_extractor import chunk_markdown_by_structure

 chunks = chunk_markdown_by_structure(content, max_chunk_chars=MAX_CHAR_LIMIT, start_from_char=start_from_char)
 if not chunks:
 return ActionResult(
 error=f'start_from_char ({start_from_char}) exceeds content length {final_filtered_length} characters'
)
 chunk = chunks[0]
 content = chunk.content
 truncated = chunk.has_more

 # Prepend overlap context for continuation chunks (e.g. table headers)
 if chunk.overlap_prefix:
 content = chunk.overlap_prefix + '\n' + content

 if start_from_char > 0:
 content_stats['started_from_char'] = start_from_char
 if truncated:
 content_stats['truncated_at_char'] = chunk.char_offset_end
 content_stats['next_start_char'] = chunk.char_offset_end
 content_stats['chunk_index'] = chunk.chunk_index
 content_stats['total_chunks'] = chunk.total_chunks

 # Add content statistics to the result
 original_html_length = content_stats['original_html_chars']
 initial_markdown_length = content_stats['initial_markdown_chars']
 chars_filtered = content_stats['filtered_chars_removed']
 stats_summary = f'Content processed: {original_html_length:,} HTML chars → {initial_markdown_length:,} Markdown chars, {chars_filtered:,} characters filtered'

```

```

#####if start_from_char > 0:
#####stats_summary += f' (started from char {start_from_char:,})'
#####if truncated:
#####chunk_info = f'chunk {chunk.chunk_index + 1} of {chunk.total_chunks}, '
#####stats_summary += f' → {len(content):,} final chars ({chunk_info}use start_from_char={content_stats
#####elif chars_filtered > 0:
#####stats_summary += f' (filtered {chars_filtered:,} chars of noise)'

Sanitize surrogates from content to prevent UTF-8 encoding errors
#####content = sanitize_surrogates(content)
#####query = sanitize_surrogates(query)

--- Structured extraction path ---
#####if structured_model is not None:
#####assert output_schema is not None
#####system_prompt = """
You are an expert at extracting structured data from the markdown of a webpage.

<input>
You will be given a query, a JSON Schema, and the markdown of a webpage that has been filtered to remove
</input>

<instructions>
- Extract ONLY information present in the webpage. Do not guess or fabricate values.
- Your response MUST conform to the provided JSON Schema exactly.
- If a required field's value cannot be found on the page, use null (if the schema allows it) or an empty
- If the content was truncated, extract what is available from the visible portion.
</instructions>
""".strip()

#####schema_json = json.dumps(output_schema, indent=2)
#####prompt = (
#####f'<query>\n{query}\n</query>\n\n'
#####f'<output_schema>\n{schema_json}\n</output_schema>\n\n'
#####f'<content_stats>\n{stats_summary}\n</content_stats>\n\n'
#####f'<webpage_content>\n{content}\n</webpage_content>'
#####)

#####try:
#####response = await asyncio.wait_for(
#####page_extraction_llm.ainvoke(
#####[SystemMessage(content=system_prompt), UserMessage(content=prompt)],
#####output_format=structured_model,
#####),
#####timeout=120.0,
#####)

response.completion is a pydantic model instance
#####result_data: dict = response.completion.model_dump(mode='json') # type: ignore[union-attr]
#####result_json = json.dumps(result_data)

#####current_url = await browser_session.get_current_page_url()
#####extracted_content = f'<url>\n{current_url}\n</url>\n<query>\n{query}\n</query>\n<structured_result>

#####from system.tools.extraction.views import ExtractionResult

#####extraction_meta = ExtractionResult(
#####data=result_data,
#####schema_used=output_schema,
#####is_partial=truncated,
#####source_url=current_url,
#####content_stats=content_stats,
#####)

Simple memory handling
#####MAX_MEMORY_LENGTH = 10000
#####if len(extracted_content) < MAX_MEMORY_LENGTH:
#####memory = extracted_content
#####include_extracted_content_only_once = False
#####else:
#####file_name = await file_system.save_extracted_content(extracted_content)
#####memory = f'Query: {query}\nContent in {file_name} and once in <read_state>.'

```

```

include_extracted_content_only_once = True

logger.info(f'■ {memory}')
return ActionResult(
 extracted_content=extracted_content,
 include_extracted_content_only_once=include_extracted_content_only_once,
 long_term_memory=memory,
 metadata={'structured_extraction': True, 'extraction_result': extraction_meta.model_dump(mode='j')
})
except Exception as e:
 logger.debug(f'Error in structured extraction: {e}')
 raise RuntimeError(str(e))

--- Free-text extraction path (default) ---
system_prompt = """
You are an expert at extracting data from the markdown of a webpage.

<input>
You will be given a query and the markdown of a webpage that has been filtered to remove noise and adversarial content.
</input>

<instructions>
- You are tasked to extract information from the webpage that is relevant to the query.
- You should ONLY use the information available in the webpage to answer the query. Do not make up information.
- If the information relevant to the query is not available in the page, your response should mention that.
- If the query asks for all items, products, etc., make sure to directly list all of them.
- If the content was truncated and you need more information, note that the user can use start_from_character to get more content.
</instructions>

<output>
- Your output should present ALL the information relevant to the query in a concise way.
- Do not answer in conversational format - directly output the relevant information or that the information is not available.
</output>
""".strip()

prompt = f'<query>\n{query}\n</query>\n\n<content_stats>\n{stats_summary}\n</content_stats>\n\n<webpage_content>\n{webpage_content}\n</webpage_content>'

try:
 response = await asyncio.wait_for(
 page_extraction_llm.ainvoke([SystemMessage(content=system_prompt), UserMessage(content=prompt)]),
 timeout=120.0,
)

 current_url = await browser_session.get_current_page_url()
 extracted_content = (
 f'<url>\n{current_url}\n</url>\n\n<query>\n{query}\n</query>\n\n<result>\n{response.completion}\n</result>'
)

 # Simple memory handling
 MAX_MEMORY_LENGTH = 10000
 if len(extracted_content) < MAX_MEMORY_LENGTH:
 memory = extracted_content
 include_extracted_content_only_once = False
 else:
 file_name = await file_system.save_extracted_content(extracted_content)
 memory = f'Query: {query}\nContent in {file_name} and once in <read_state>.'
 include_extracted_content_only_once = True

 logger.info(f'■ {memory}')
 return ActionResult(
 extracted_content=extracted_content,
 include_extracted_content_only_once=include_extracted_content_only_once,
 long_term_memory=memory,
)
except Exception as e:
 logger.debug(f'Error extracting content: {e}')
 raise RuntimeError(str(e))

--- Page search and exploration tools (zero LLM cost) ---

@self.registry.action(
 """Search page text for a pattern (like grep). Zero LLM cost, instant. Returns matches with surrounding text.

```

```

 param_model=SearchPageAction,
)
 async def search_page(params: SearchPageAction, browser_session: BrowserSession):
 js_code = _build_search_page_js(
 pattern=params.pattern,
 regex=params.regex,
 case_sensitive=params.case_sensitive,
 context_chars=params.context_chars,
 css_scope=params.css_scope,
 max_results=params.max_results,
)

 cdp_session = await browser_session.get_or_create_cdp_session()
 result = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': js_code, 'returnByValue': True, 'awaitPromise': True},
 session_id=cdp_session.session_id,
)

 if result.get('exceptionDetails'):
 error_text = result['exceptionDetails'].get('text', 'Unknown JS error')
 return ActionResult(error=f'search_page failed: {error_text}')

 data = result.get('result', {}).get('value')
 if data is None:
 return ActionResult(error='search_page returned no result')

 if isinstance(data, dict) and data.get('error'):
 return ActionResult(error=f'search_page: {data["error"]}')
```

```

 formatted = _format_search_results(data, params.pattern)
 total = data.get('total', 0)
 memory = f'Searched page for "{params.pattern}": {total} match{"es" if total != 1 else ""} found.'
 logger.info(f'█ {memory}')
 return ActionResult(extracted_content=formatted, long_term_memory=memory)

 @self.registry.action(
 """Query DOM elements by CSS selector (like find). Zero LLM cost, instant. Returns matching elements"""
)
 param_model=FindElementsAction,
)
 async def find_elements(params: FindElementsAction, browser_session: BrowserSession):
 js_code = _build_find_elements_js(
 selector=params.selector,
 attributes=params.attributes,
 max_results=params.max_results,
 include_text=params.include_text,
)

 cdp_session = await browser_session.get_or_create_cdp_session()
 result = await cdp_session.cdp_client.send.Runtime.evaluate(
 params={'expression': js_code, 'returnByValue': True, 'awaitPromise': True},
 session_id=cdp_session.session_id,
)

 if result.get('exceptionDetails'):
 error_text = result['exceptionDetails'].get('text', 'Unknown JS error')
 return ActionResult(error=f'find_elements failed: {error_text}')

 data = result.get('result', {}).get('value')
 if data is None:
 return ActionResult(error='find_elements returned no result')

 if isinstance(data, dict) and data.get('error'):
 return ActionResult(error=f'find_elements: {data["error"]}')
```

```

 formatted = _format_find_results(data, params.selector)
 total = data.get('total', 0)
 memory = f'Found {total} element{"s" if total != 1 else ""} matching "{params.selector}".'
 logger.info(f'█ {memory}')
 return ActionResult(extracted_content=formatted, long_term_memory=memory)

 @self.registry.action(
 """Scroll by pages. REQUIRED: down=True/False (True=scroll down, False=scroll up, default=True). Opt

```

```

param_model=ScrollAction,
)
async def scroll(params: ScrollAction, browser_session: BrowserSession):
 try:
 # Look up the node from the selector map if index is provided
 # Special case: index 0 means scroll the whole page (root/body element)
 node = None
 if params.index is not None and params.index != 0:
 node = await browser_session.get_element_by_index(params.index)
 if node is None:
 # Element does not exist
 msg = f'Element index {params.index} not found in browser state'
 return ActionResult(error=msg)

 direction = 'down' if params.down else 'up'
 target = f'element {params.index}' if params.index is not None and params.index != 0 else ''

 # Get actual viewport height for more accurate scrolling
 try:
 cdp_session = await browser_session.get_or_create_cdp_session()
 metrics = await cdp_session.cdp_client.send.Page.getLayoutMetrics(session_id=cdp_session.session_id)

 # Use cssVisualViewport for the most accurate representation
 css_viewport = metrics.get('cssVisualViewport', {})
 css_layout_viewport = metrics.get('cssLayoutViewport', {})

 # Get viewport height, prioritizing cssVisualViewport
 viewport_height = int(css_viewport.get('clientHeight') or css_layout_viewport.get('clientHeight', 0))

 logger.debug(f'Detected viewport height: {viewport_height}px')
 except Exception as e:
 viewport_height = 1000 # Fallback to 1000px
 logger.debug(f'Failed to get viewport height, using fallback 1000px: {e}')

 # For multiple pages (>=1.0), scroll one page at a time to ensure each scroll completes
 if params.pages >= 1.0:
 import asyncio

 num_full_pages = int(params.pages)
 remaining_fraction = params.pages - num_full_pages

 completed_scrolls = 0

 # Scroll one page at a time
 for i in range(num_full_pages):
 try:
 pixels = viewport_height # Use actual viewport height
 if not params.down:
 pixels = -pixels

 event = browser_session.event_bus.dispatch(
 ScrollEvent(direction=direction, amount=abs(pixels), node=node)
)
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)
 completed_scrolls += 1

 # Small delay to ensure scroll completes before next one
 await asyncio.sleep(0.15)

 except Exception as e:
 logger.warning(f'Scroll {i + 1}/{num_full_pages} failed: {e}')
 # Continue with remaining scrolls even if one fails

 # Handle fractional page if present
 if remaining_fraction > 0:
 try:
 pixels = int(remaining_fraction * viewport_height)
 if not params.down:
 pixels = -pixels

 event = browser_session.event_bus.dispatch(

```

```

 ScrollEvent(direction=direction, amount=abs(pixels), node=node)
)
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)
 completed_scrolls += remaining_fraction

 except Exception as e:
 logger.warning(f'Fractional scroll failed: {e}')

 if params.pages == 1.0:
 long_term_memory = f'Scrolled {direction} {target} {viewport_height}px'.replace(' ', ' ')
 else:
 long_term_memory = f'Scrolled {direction} {target} {completed_scrolls:.1f} pages'.replace(' ', ' ')
 else:
 # For fractional pages <1.0, do single scroll
 pixels = int(params.pages * viewport_height)
 event = browser_session.event_bus.dispatch(
 ScrollEvent(direction='down' if params.down else 'up', amount=pixels, node=node)
)
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)
 long_term_memory = f'Scrolled {direction} {target} {params.pages} pages'.replace(' ', ' ')

 msg = f'■ {long_term_memory}'
 logger.info(msg)
 return ActionResult(extracted_content=msg, long_term_memory=long_term_memory)
 except Exception as e:
 logger.error(f'Failed to dispatch ScrollEvent: {type(e).__name__}: {e}')
 error_msg = 'Failed to execute scroll action.'
 return ActionResult(error=error_msg)

 @self.registry.action(
 '',
 param_model=SendKeysAction,
)
 async def send_keys(params: SendKeysAction, browser_session: BrowserSession):
 # Dispatch send keys event
 try:
 event = browser_session.event_bus.dispatch(SendKeysEvent(keys=params.keys))
 await event
 await event.event_result(raise_if_any=True, raise_if_none=False)
 memory = f'Sent keys: {params.keys}'
 msg = f'■ {memory}'
 logger.info(msg)
 return ActionResult(extracted_content=memory, long_term_memory=memory)
 except Exception as e:
 logger.error(f'Failed to dispatch SendKeysEvent: {type(e).__name__}: {e}')
 error_msg = f'Failed to send keys: {str(e)}'
 return ActionResult(error=error_msg)

 @self.registry.action('Scroll to text.')
 async def find_text(text: str, browser_session: BrowserSession): # type: ignore
 # Dispatch scroll to text event
 event = browser_session.event_bus.dispatch(ScrollToTextEvent(text=text))

 try:
 # The handler returns None on success or raises an exception if text not found
 await event.event_result(raise_if_any=True, raise_if_none=False)
 memory = f'Scrolled to text: {text}'
 msg = f'■ {memory}'
 logger.info(msg)
 return ActionResult(extracted_content=memory, long_term_memory=memory)
 except Exception as e:
 # Text not found
 msg = f'Text '{text}' not found or not visible on page"
 logger.info(msg)
 return ActionResult(
 extracted_content=msg,
 long_term_memory=f"Tried scrolling to text '{text}' but it was not found",
)

 @self.registry.action(

```

```

 """Take a screenshot of the current viewport. If file_name is provided, saves to that file and returns
 """Otherwise, screenshot is included in the next browser_state observation.'',
 param_model=ScreenshotAction,
):
 async def screenshot(
 params: ScreenshotAction,
 browser_session: BrowserSession,
 file_system: FileSystem,
):
 """Take screenshot, optionally saving to file."""
 if params.file_name:
 # Save screenshot to file
 file_name = params.file_name
 if not file_name.lower().endswith('.png'):
 file_name = f'{file_name}.png'
 file_name = FileSystem.sanitize_filename(file_name)

 screenshot_bytes = await browser_session.take_screenshot(full_page=False)
 file_path = file_system.get_dir() / file_name
 file_path.write_bytes(screenshot_bytes)

 result = f'Screenshot saved to {file_name}'
 logger.info(f'█ {result}. Full path: {file_path}')
 return ActionResult(
 extracted_content=result,
 long_term_memory=f'{result}. Full path: {file_path}',
 attachments=[str(file_path)],
)
 else:
 # Flag for next observation
 memory = 'Requested screenshot for next observation'
 logger.info(f'█ {memory}')
 return ActionResult(
 extracted_content=memory,
 metadata={'include_screenshot': True},
)

 ## PDF Actions

 @self.registry.action(
 """Save the current page as a PDF file. Returns the file path of the saved PDF. '
 """Use this to capture the full page content (including content below the fold) as a printable document
 param_model=SaveAsPdfAction,
)
 async def save_as_pdf(
 params: SaveAsPdfAction,
 browser_session: BrowserSession,
 file_system: FileSystem,
):
 """Save the current page as a PDF using CDP Page.printToPDF."""
 import base64
 import re

 # Paper format dimensions in inches (width, height)
 paper_sizes: dict[str, tuple[float, float]] = {
 'letter': (8.5, 11),
 'legal': (8.5, 14),
 'a4': (8.27, 11.69),
 'a3': (11.69, 16.54),
 'tabloid': (11, 17),
 }

 paper_key = params.paper_format.lower()
 if paper_key not in paper_sizes:
 paper_key = 'letter'
 paper_width, paper_height = paper_sizes[paper_key]

 cdp_session = await browser_session.get_or_create_cdp_session(focus=True)

 result = await asyncio.wait_for(
 cdp_session.cdp_client.send.Page.printToPDF(
 params={

```

```

#####'printBackground': params.print_background,
#####'landscape': params.landscape,
#####'scale': params.scale,
#####'paperWidth': paper_width,
#####'paperHeight': paper_height,
#####'preferCSSPageSize': True,
#####},
#####session_id=cdp_session.session_id,
#####),
#####timeout=30.0,
#####)

####pdf_data = result.get('data')
####assert pdf_data, 'CDP Page.printToPDF returned no data'

####pdf_bytes = base64.b64decode(pdf_data)

Determine filename
####if params.file_name:
#####file_name = params.file_name
####else:
#####try:
#####page_title = await asyncio.wait_for(browser_session.get_current_page_title(), timeout=2.0)
#####safe_title = re.sub(r'[\w\s-]', '', page_title).strip()[:50]
#####file_name = safe_title if safe_title else 'page'
#####except Exception:
#####file_name = 'page'

####if not file_name.lower().endswith('.pdf'):
#####file_name = f'{file_name}.pdf'
####file_name = FileSystem.sanitize_filename(file_name)

####file_path = file_system.get_dir() / file_name
Handle duplicate filenames
####if file_path.exists():
#####base, ext = os.path.splitext(file_name)
#####counter = 1
#####while (file_system.get_dir() / f'{base} ({counter}){ext}').exists():
#####counter += 1
#####file_name = f'{base} ({counter}){ext}'
#####file_path = file_system.get_dir() / file_name

####async with await anyio.open_file(file_path, 'wb') as f:
#####await f.write(pdf_bytes)

####file_size = file_path.stat().st_size
####msg = f'Saved page as PDF: {file_name} ({file_size:,} bytes)'
####logger.info(f'■ {msg}. Full path: {file_path}')

####return ActionResult(
#####extracted_content=msg,
#####long_term_memory=f'{msg}. Full path: {file_path}',
#####attachments=[str(file_path)],
#####)

Dropdown Actions

####@self.registry.action(
#####'',
#####param_model=GetDropdownOptionsAction,
#####)
####async def dropdown_options(params: GetDropdownOptionsAction, browser_session: BrowserSession):
#####"""Get all options from a native dropdown or ARIA menu"""
Look up the node from the selector map
#####node = await browser_session.get_element_by_index(params.index)
#####if node is None:
#####msg = f'Element index {params.index} not available - page may have changed. Try refreshing browser'
#####logger.warning(f'■ {msg}')
#####return ActionResult(extracted_content=msg)

Dispatch GetDropdownOptionsEvent to the event handler
####event = browser_session.event_bus.dispatch(GetDropdownOptionsEvent(node=node))

```



```

■■■dropdown_data = await event.event_result(timeout=3.0, raise_if_none=True, raise_if_any=True)

■■■if not dropdown_data:
■■■raise ValueError('Failed to get dropdown options - no data returned')

■■■# Use structured memory from the handler
■■■return ActionResult(
■■■extracted_content=dropdown_data['short_term_memory'],
■■■long_term_memory=dropdown_data['long_term_memory'],
■■■include_extracted_content_only_once=True,
■■■)

■■■@self.registry.action(
■■■'Set the option of a <select> element.',
■■■param_model=SelectDropdownOptionAction,
■■■)
■■■async def select_dropdown(params: SelectDropdownOptionAction, browser_session: BrowserSession):
■■■"""Select dropdown option by the text of the option you want to select"""
■■■# Look up the node from the selector map
■■■node = await browser_session.get_element_by_index(params.index)
■■■if node is None:
■■■msg = f'Element index {params.index} not available - page may have changed. Try refreshing browser'
■■■logger.warning(f'■■■ {msg}')
■■■return ActionResult(extracted_content=msg)

■■■# Dispatch SelectDropdownOptionEvent to the event handler
■■■from system.browser.events import SelectDropdownOptionEvent

■■■event = browser_session.event_bus.dispatch(SelectDropdownOptionEvent(node=node, text=params.text))
■■■selection_data = await event.event_result()

■■■if not selection_data:
■■■raise ValueError('Failed to select dropdown option - no data returned')

■■■# Check if the selection was successful
■■■if selection_data.get('success') == 'true':
■■■# Extract the message from the returned data
■■■msg = selection_data.get('message', f'Selected option: {params.text}')
■■■return ActionResult(
■■■extracted_content=msg,
■■■include_in_memory=True,
■■■long_term_memory=f"Selected dropdown option '{params.text}' at index {params.index}",
■■■)
■■■else:
■■■# Handle structured error response
■■■# TODO: raise BrowserError instead of returning ActionResult
■■■if 'short_term_memory' in selection_data and 'long_term_memory' in selection_data:
■■■return ActionResult(
■■■extracted_content=selection_data['short_term_memory'],
■■■long_term_memory=selection_data['long_term_memory'],
■■■include_extracted_content_only_once=True,
■■■)
■■■else:
■■■# Fallback to regular error
■■■error_msg = selection_data.get('error', f'Failed to select option: {params.text}')
■■■return ActionResult(error=error_msg)

■■■# File System Actions

■■■@self.registry.action(
■■■'Write content to a file. By default this OVERWRITES the entire file - use append=true to add to an
■■■FILENAME RULES: Use only letters, numbers, underscores, hyphens, dots, parentheses. Spaces are auto
■■■SUPPORTED EXTENSIONS: .txt, .md, .json, .jsonl, .csv, .html, .xml, .pdf, .docx. '
■■■'CANNOT write binary/image files (.png, .jpg, .mp4, etc.) - do not attempt to save screenshots as fi
■■■'For PDF files, write content in markdown format and it will be auto-converted to PDF.'
■■■)
■■■async def write_file(
■■■file_name: str,
■■■content: str,
■■■file_system: FileSystem,
■■■append: bool = False,
■■■trailing_newline: bool = True,

```

```

 leading_newline: bool = False,
):
 if trailing_newline:
 content += '\n'
 if leading_newline:
 content = '\n' + content
 if append:
 result = await file_system.append_file(file_name, content)
 else:
 result = await file_system.write_file(file_name, content)

 # Log the full path where the file is stored (use resolved name)
 resolved_name, _ = file_system._resolve_filename(file_name)
 file_path = file_system.get_dir() / resolved_name
 logger.info(f'█ {result} File location: {file_path}')

 return ActionResult(extracted_content=result, long_term_memory=result)

 @self.registry.action(
 'Replace specific text within a file by searching for old_str and replacing with new_str. Use this f
)
 async def replace_file(file_name: str, old_str: str, new_str: str, file_system: FileSystem):
 result = await file_system.replace_file_str(file_name, old_str, new_str)
 logger.info(f'█ {result}')
 return ActionResult(extracted_content=result, long_term_memory=result)

 @self.registry.action(
 'Read the complete content of a file. Use this to view file contents before editing or to retrieve d
)
 async def read_file(file_name: str, available_file_paths: list[str], file_system: FileSystem):
 if available_file_paths and file_name in available_file_paths:
 structured_result = await file_system.read_file_structured(file_name, external_file=True)
 else:
 structured_result = await file_system.read_file_structured(file_name)

 result = structured_result['message']
 images = structured_result.get('images')

 MAX_MEMORY_SIZE = 1000
 # For images, create a shorter memory message
 if images:
 memory = f'Read image file {file_name}'
 elif len(result) > MAX_MEMORY_SIZE:
 lines = result.splitlines()
 display = ''
 lines_count = 0
 for line in lines:
 if len(display) + len(line) < MAX_MEMORY_SIZE:
 display += line + '\n'
 lines_count += 1
 else:
 break
 remaining_lines = len(lines) - lines_count
 memory = f'{display}{remaining_lines} more lines...' if remaining_lines > 0 else display
 else:
 memory = result
 logger.info(f'█ {memory}')
 return ActionResult(
 extracted_content=result,
 long_term_memory=memory,
 images=images,
 include_extracted_content_only_once=True,
)

 # Intelligent content reading

 @self.registry.action(
 'Intelligently read long content to find specific information. Works on current page (source="page")
 param_model=ReadContentAction,
)
 async def read_long_content(
 params: ReadContentAction,

```

```

#####browser_session: BrowserSession,
#####page_extraction_llm: BaseChatModel,
#####available_file_paths: list[str],
#####):
#####import re

#####from system.llm.messages import UserMessage

#####goal = params.goal
#####context = params.context
#####source = params.source
#####max_chars = 50000

#####async def extract_search_terms(goal: str, context: str) -> list[str]:
"""Use LLM to extract search terms from goal."""
prompt = f"""Extract 3-5 key search terms from this goal that would help find relevant sections.
Return only the terms, one per line, no numbering or bullets.

Goal: {goal}

Context: {context}"""
response = await page_extraction_llm.ainvoke([UserMessage(content=prompt)])
return [term.strip() for term in response.completion.strip().split('\n') if term.strip()][:5]

#####def search_text(content: str, pattern: str, context_chars: int = 100) -> list[dict]:
"""Search content for pattern, return matches with positions."""
try:
regex = re.compile(pattern, re.IGNORECASE)
except re.error:
regex = re.compile(re.escape(pattern), re.IGNORECASE)

matches = []
for match in regex.finditer(content):
start = max(0, match.start() - context_chars)
end = min(len(content), match.end() + context_chars)
matches.append(
{
'position': match.start(),
'snippet': content[start:end],
}
)
return matches

#####def chunk_content(content: str, chunk_size: int = 2000) -> list[dict]:
"""Split content into chunks with positions."""
chunks = []
for i in range(0, len(content), chunk_size):
chunks.append(
{
'start': i,
'end': min(i + chunk_size, len(content)),
'text': content[i : i + chunk_size],
}
)
return chunks

#####try:
if source.lower() == 'page':
Read from current webpage
from system.dom.markdown_extractor import extract_clean_markdown

Clear DOM cache and wait for page to settle before extracting
if browser_session._dom_watchdog:
browser_session._dom_watchdog.clear_cache()

wait_time = browser_session.browser_profile.wait_for_network_idle_page_load_time
await asyncio.sleep(wait_time)

content, _ = await extract_clean_markdown(browser_session=browser_session, extract_links=False)
source_name = 'current page'

if not content:

```

```

#####return ActionResult(
#####extracted_content='Error: No page content available',
#####long_term_memory='Failed to read page: no content',
#####)

#####else:
Read from file
#####file_path = source

Validate file path against whitelist (available_file_paths + downloaded files)
#####allowed_paths = set(available_file_paths or [])
#####allowed_paths.update(browser_session.downloaded_files)
#####if file_path not in allowed_paths:
#####return ActionResult(
#####extracted_content=f'Error: File path not in available_file_paths: {file_path}. '
#####f'The user must add this path to available_file_paths when creating the Agent.',
#####long_term_memory=f'Failed to read: file path not allowed: {file_path}',
#####)

#####if not os.path.exists(file_path):
#####return ActionResult(
#####extracted_content=f'Error: File not found: {file_path}',
#####long_term_memory='Failed to read: file not found',
#####)

#####ext = os.path.splitext(file_path)[1].lower()
#####source_name = os.path.basename(file_path)

#####if ext == '.pdf':
Read PDF directly using pypdf
#####import pypdf

#####reader = pypdf.PdfReader(file_path)
#####num_pages = len(reader.pages)

Extract all page text
#####page_texts: list[str] = []
#####total_chars = 0
#####for page in reader.pages:
#####text = page.extract_text() or ''
#####page_texts.append(text)
#####total_chars += len(text)

If PDF is small enough, return it all
#####if total_chars <= max_chars:
#####content_parts = []
#####for i, text in enumerate(page_texts, 1):
#####if text.strip():
#####content_parts.append(f'--- Page {i} ---\n{text}')
#####content = '\n\n'.join(content_parts)

#####memory = f'Read {source_name} ({num_pages} pages, {total_chars:,} chars) for goal: {goal[:50]}'
#####logger.info(f'█ {memory}')
#####return ActionResult(
#####extracted_content=f'PDF: {source_name} ({num_pages} pages)\n\n{content}',
#####long_term_memory=memory,
#####include_extracted_content_only_once=True,
#####)

PDF too large - use intelligent extraction
#####logger.info(f'PDF has {total_chars:,} chars across {num_pages} pages, using intelligent extraction')

Extract search terms from goal
#####search_terms = await extract_search_terms(goal, context)

Search and score pages by relevance
#####page_scores: dict[int, int] = {} # 1-indexed page -> score
#####for term in search_terms:
#####try:
#####term_pattern = re.compile(re.escape(term), re.IGNORECASE)
#####except re.error:
#####continue

```

```

#####for i, text in enumerate(page_texts, 1):
#####if term_pattern.search(text):
#####page_scores[i] = page_scores.get(i, 0) + 1

Select pages: always include page 1, then most relevant
#####pages_to_read = [1]
#####sorted_pages = sorted(page_scores.items(), key=lambda x: -x[1])
#####for page_num, _ in sorted_pages:
#####if page_num not in pages_to_read:
#####pages_to_read.append(page_num)

Build result respecting char limit, truncating pages if needed
#####content_parts = []
#####chars_used = 0
#####pages_included = []
#####for page_num in sorted(set(pages_to_read)):
#####text = page_texts[page_num - 1]
#####page_header = f'--- Page {page_num} ---\n'
#####remaining = max_chars - chars_used
#####if remaining < len(page_header) + 50:
#####break # no room for meaningful content
#####page_content = page_header + text
#####if len(page_content) > remaining:
#####page_content = page_content[: remaining - len('\n[...truncated]')] + '\n[...truncated]'
#####content_parts.append(page_content)
#####chars_used += len(page_content)
#####pages_included.append(page_num)

#####content = '\n\n'.join(content_parts)
#####memory = f'Read {source_name} ({len(pages_included)} relevant pages of {num_pages}) for goal: {goal}'
#####logger.info(f'█ {memory}')
#####return ActionResult(
#####extracted_content=f'PDF: {source_name} ({num_pages} pages, showing {len(pages_included)} relevant pages)',
#####long_term_memory=memory,
#####include_extracted_content_only_once=True,
#####)

#####else:
Text file
#####async with await anyio.open_file(file_path, 'r', encoding='utf-8', errors='ignore') as f:
#####content = await f.read()

Check if content fits in budget
#####if len(content) <= max_chars:
#####memory = f'Read {source_name} ({len(content):,} chars) for goal: {goal[:50]}'
#####logger.info(f'█ {memory}')
#####return ActionResult(
#####extracted_content=f'Content from {source_name} ({len(content):,} chars):\n\n{content}',
#####long_term_memory=memory,
#####include_extracted_content_only_once=True,
#####)

Content too large - use intelligent extraction
#####logger.info(f'Content has {len(content):,} chars, using intelligent extraction')

Extract search terms from goal
#####search_terms = await extract_search_terms(goal, context)

Search for each term and score chunks
#####chunks = chunk_content(content, chunk_size=2000)
#####chunk_scores: dict[int, int] = {} # chunk index -> relevance score

#####for term in search_terms:
#####matches = search_text(content, term)
#####for match in matches:
Find which chunk this match belongs to
#####for i, chunk in enumerate(chunks):
#####if chunk['start'] <= match['position'] < chunk['end']:
#####chunk_scores[i] = chunk_scores.get(i, 0) + 1
#####break

#####if not chunk_scores:

```

```

No matches - return first max_chars
truncated = content[:max_chars]
memory = f'Read {source_name} (truncated to {max_chars:,} chars, no matches for search terms)'
logger.info(f'█ {memory}')
return ActionResult(
extracted_content=f'Content from {source_name} (first {max_chars:,} of {len(content):,} chars):\n'
long_term_memory=memory,
include_extracted_content_only_once=True,
)

Sort chunks by relevance and collect most relevant ones
sorted_chunks = sorted(chunk_scores.items(), key=lambda x: -x[1])

Always include first chunk for context
selected_indices = {0} # Start with first chunk
for chunk_idx, _ in sorted_chunks:
selected_indices.add(chunk_idx)

Build result from selected chunks in order
result_parts = []
total_chars = 0
for i in sorted(selected_indices):
chunk = chunks[i]
if total_chars + len(chunk['text']) > max_chars:
break
if i > 0 and (i - 1) not in selected_indices:
result_parts.append('\n[...]\n') # Indicate gap
result_parts.append(chunk['text'])
total_chars += len(chunk['text'])

result_content = ''.join(result_parts)
memory = f'Read {source_name} ({len(selected_indices)} relevant sections of {len(chunks)}) for goal'
logger.info(f'█ {memory}')

return ActionResult(
extracted_content=f'Content from {source_name} (relevant sections, {total_chars:,} of {len(content):,} chars):\n'
long_term_memory=memory,
include_extracted_content_only_once=True,
)

except Exception as e:
error_msg = f'Error reading content: {str(e)}'
logger.error(error_msg)
return ActionResult(extracted_content=error_msg, long_term_memory=error_msg)

@self.registry.action(
"""Execute browser JavaScript. Best practice: wrap in IIFE (function(){...})() with try-catch for safety"""
terminates_sequence=True,
)
async def evaluate(code: str, browser_session: BrowserSession):
Execute JavaScript with proper error handling and promise support

cdp_session = await browser_session.get_or_create_cdp_session()

try:
Validate and potentially fix JavaScript code before execution
validated_code = self._validate_and_fix_javascript(code)

Always use awaitPromise=True - it's ignored for non-promises
result = await cdp_session.cdp_client.send.Runtime.evaluate(
params={'expression': validated_code, 'returnByValue': True, 'awaitPromise': True},
session_id=cdp_session.session_id,
)

Check for JavaScript execution errors
if result.get('exceptionDetails'):
exception = result['exceptionDetails']
error_msg = f'JavaScript execution error: {exception.get("text", "Unknown error")}'

Enhanced error message with debugging info
enhanced_msg = f'JavaScript Execution Failed: {error_msg}'

```

```

Validated Code (after quote fixing):
{validated_code[:500]}{'...' if len(validated_code) > 500 else ''}
"""

#####logger.debug(enhanced_msg)
#####return ActionResult(error=enhanced_msg)

Get the result data
#####result_data = result.get('result', {})

Check for wasThrown flag (backup error detection)
#####if result_data.get('wasThrown'):
#####msg = f'JavaScript code: {code} execution failed (wasThrown=true)'
#####logger.debug(msg)
#####return ActionResult(error=msg)

Get the actual value
#####value = result_data.get('value')

Handle different value types
#####if value is None:
Could be legitimate null/undefined result
#####result_text = str(value) if 'value' in result_data else 'undefined'
#####elif isinstance(value, (dict, list)):
Complex objects - should be serialized by returnByValue
#####try:
#####result_text = json.dumps(value, ensure_ascii=False)
#####except (TypeError, ValueError):
Fallback for non-serializable objects
#####result_text = str(value)
#####else:
Primitive values (string, number, boolean)
#####result_text = str(value)

#####import re

#####image_pattern = r'(data:image/[^;]+;base64,[A-Za-z0-9+/=]+)'
#####found_images = re.findall(image_pattern, result_text)

#####metadata = None
#####if found_images:
Store images in metadata so they can be added as ContentPartImageParam
#####metadata = {'images': found_images}

Replace image data in result text with shorter placeholder
#####modified_text = result_text
#####for i, img_data in enumerate(found_images, 1):
#####placeholder = '[Image]'
#####modified_text = modified_text.replace(img_data, placeholder)
#####result_text = modified_text

Apply length limit with better truncation (after image extraction)
#####if len(result_text) > 20000:
#####result_text = result_text[:19950] + '\n... [Truncated after 20000 characters]'

Don't log the code - it's already visible in the user's cell
#####logger.debug(f'JavaScript executed successfully, result length: {len(result_text)}')

Memory handling: keep full result in extracted_content for current step,
but use truncated version in long_term_memory if too large
#####MAX_MEMORY_LENGTH = 10000
#####if len(result_text) < MAX_MEMORY_LENGTH:
#####memory = result_text
#####include_extracted_content_only_once = False
#####else:
#####memory = f'JavaScript executed successfully, result length: {len(result_text)} characters.'
#####include_extracted_content_only_once = True

Return only the result, not the code (code is already in user's cell)
#####return ActionResult(
#####extracted_content=result_text,
#####long_term_memory=memory,

```

```

#####include_extracted_content_only_once=include_extracted_content_only_once,
#####metadata=metadata,
#####)

#####except Exception as e:
CDP communication or other system errors
error_msg = f'Failed to execute JavaScript: {type(e).__name__}: {e}'
logger.debug(f'JavaScript code that failed: {code[:200]}...')
return ActionResult(error=error_msg)

def _validate_and_fix_javascript(self, code: str) -> str:
 """Validate and fix common JavaScript issues before execution"""

 import re

 ##### Pattern 1: Fix double-escaped quotes (\\\" -> \")
 ##### fixed_code = re.sub(r'\\\"', '\"', code)

 ##### Pattern 2: Fix over-escaped regex patterns (\\\\d -> \\d)
 ##### Common issue: regex gets double-escaped during parsing
 ##### fixed_code = re.sub(r'\\\\([dDsSwWbBnrftv])', r'\\1', fixed_code)
 ##### fixed_code = re.sub(r'\\\\([. * + ? ^ $ { } () | [\]])', r'\\1', fixed_code)

 ##### Pattern 3: Fix XPath expressions with mixed quotes
 ##### xpath_pattern = r'document\.evaluate\s*\(\s*"([^\"]*)" \s*, '

 def fix_xpath_quotes(match):
 ##### xpath_with_quotes = match.group(1)
 ##### return f'document.evaluate(`{xpath_with_quotes}`,'

 ##### fixed_code = re.sub(xpath_pattern, fix_xpath_quotes, fixed_code)

 ##### Pattern 4: Fix querySelector/querySelectorAll with mixed quotes
 ##### selector_pattern = r'(querySelector(?:All)?)\s*\(\s*"([^\"]*)" \s*\)'

 def fix_selector_quotes(match):
 ##### method_name = match.group(1)
 ##### selector_with_quotes = match.group(2)
 ##### return f'{method_name}(`{selector_with_quotes}`)'

 ##### fixed_code = re.sub(selector_pattern, fix_selector_quotes, fixed_code)

 ##### Pattern 5: Fix closest() calls with mixed quotes
 ##### closest_pattern = r'\.closest\s*\(\s*"([^\"]*)" \s*\)'

 def fix_closest_quotes(match):
 ##### selector_with_quotes = match.group(1)
 ##### return f'.closest(`{selector_with_quotes}`)'

 ##### fixed_code = re.sub(closest_pattern, fix_closest_quotes, fixed_code)

 ##### Pattern 6: Fix .matches() calls with mixed quotes (similar to closest)
 ##### matches_pattern = r'\.matches\s*\(\s*"([^\"]*)" \s*\)'

 def fix_matches_quotes(match):
 ##### selector_with_quotes = match.group(1)
 ##### return f'.matches(`{selector_with_quotes}`)'

 ##### fixed_code = re.sub(matches_pattern, fix_matches_quotes, fixed_code)

 ##### Note: Removed getAttribute fix - attribute names rarely have mixed quotes
 ##### getAttribute typically uses simple names like "data-value", not complex selectors

 ##### Log changes made
 ##### changes_made = []
 ##### if r'\"' in code and r'\"' not in fixed_code:
 ##### changes_made.append('fixed escaped quotes')
 ##### if '`' in fixed_code and '`' not in code:
 ##### changes_made.append('converted mixed quotes to template literals')

 ##### if changes_made:
 ##### logger.debug(f'JavaScript fixes applied: {", ".join(changes_made)}')

```



```

 return fixed_code

def _register_done_action(self, output_model: type[T] | None, display_files_in_done_text: bool = True):
 if output_model is not None:
 self.display_files_in_done_text = display_files_in_done_text

 @self.registry.action(
 'Complete task with structured output.',
 param_model=StructuredOutputAction[output_model],
)
 async def done(params: StructuredOutputAction, file_system: FileSystem, browser_session: BrowserSession):
 # Exclude success from the output JSON
 # Use mode='json' to properly serialize enums at all nesting levels
 output_dict = params.data.model_dump(mode='json')

 attachments: list[str] = []

 # 1. Resolve any explicitly requested files via files_to_display
 if params.files_to_display:
 for file_name in params.files_to_display:
 file_content = file_system.display_file(file_name)
 if file_content:
 attachments.append(str(file_system.get_dir() / file_name))

 # 2. Auto-attach actual session downloads (CDP-tracked browser downloads)
 # but NOT user-supplied whitelist paths from available_file_paths
 session_downloads = browser_session.downloaded_files
 if session_downloads:
 existing = set(attachments)
 for file_path in session_downloads:
 if file_path not in existing:
 attachments.append(file_path)

 return ActionResult(
 is_done=True,
 success=params.success,
 extracted_content=json.dumps(output_dict, ensure_ascii=False),
 long_term_memory=f'Task completed. Success Status: {params.success}',
 attachments=attachments,
)

 else:
 @self.registry.action(
 'Complete task.',
 param_model=DoneAction,
)
 async def done(params: DoneAction, file_system: FileSystem):
 user_message = params.text

 len_text = len(params.text)
 len_max_memory = 100
 memory = f'Task completed: {params.success} - {params.text[:len_max_memory]}'
 if len_text > len_max_memory:
 memory += f' - {len_text - len_max_memory} more characters'

 attachments = []
 if params.files_to_display:
 if self.display_files_in_done_text:
 file_msg = ''
 for file_name in params.files_to_display:
 file_content = file_system.display_file(file_name)
 if file_content:
 file_msg += f'\n\n{file_name}:\n{file_content}'
 attachments.append(file_name)
 if file_msg:
 user_message += '\n\nAttachments:'
 user_message += file_msg
 else:
 logger.warning('Agent wanted to display files but none were found')
 else:
 for file_name in params.files_to_display:

```

```

#####file_content = file_system.display_file(file_name)
#####if file_content:
#####attachments.append(file_name)

#####attachments = [str(file_system.get_dir() / file_name) for file_name in attachments]

#####return ActionResult(
#####is_done=True,
#####success=params.success,
#####extracted_content=user_message,
#####long_term_memory=memory,
#####attachments=attachments,
#####)

def use_structured_output_action(self, output_model: type[T]):
self._output_model = output_model
self._register_done_action(output_model)

def get_output_model(self) -> type[BaseModel] | None:
"""Get the output model if structured output is configured."""
return self._output_model

Register -----

def action(self, description: str, **kwargs):
"""Decorator for registering custom actions

@param description: Describe the LLM what the function does (better description == better function call)
"""
return self.registry.action(description, **kwargs)

def exclude_action(self, action_name: str) -> None:
"""Exclude an action from the tools registry.

This method can be used to remove actions after initialization,
useful for enforcing constraints like disabling screenshot when use_vision != 'auto'.

Args:
action_name: Name of the action to exclude (e.g., 'screenshot')
"""
self.registry.exclude_action(action_name)

def _register_click_action(self) -> None:
"""Register the click action with or without coordinate support based on current setting."""
Remove existing click action if present
if 'click' in self.registry.actions:
del self.registry.actions['click']

if self._coordinate_clicking_enabled:
Register click action WITH coordinate support
@self.registry.action(
"""Click element by index or coordinates. Use coordinates only if the index is not available. Either
param_model=ClickElementAction,
)
async def click(params: ClickElementAction, browser_session: BrowserSession):
Validate that either index or coordinates are provided
if params.index is None and (params.coordinate_x is None or params.coordinate_y is None):
return ActionResult(error='Must provide either index or both coordinate_x and coordinate_y')

Try index-based clicking first if index is provided
if params.index is not None:
return await self._click_by_index(params, browser_session)
Coordinate-based clicking when index is not provided
else:
return await self._click_by_coordinate(params, browser_session)
else:
Register click action WITHOUT coordinate support (index only)
@self.registry.action(
"""Click element by index.
param_model=ClickElementActionIndexOnly,
)
async def click(params: ClickElementActionIndexOnly, browser_session: BrowserSession):

```

```

return await self._click_by_index(params, browser_session)

def set_coordinate_clicking(self, enabled: bool) -> None:
 """Enable or disable coordinate-based clicking.

 When enabled, the click action accepts both index and coordinate parameters.
 When disabled (default), only index-based clicking is available.

 This is automatically enabled for models that support coordinate clicking:
 - claude-sonnet-4-5
 - claude-opus-4-5
 - gemini-3-pro
 - browser-use/* models

 Args:
 enabled: True to enable coordinate clicking, False to disable
 """
 if enabled == self._coordinate_clicking_enabled:
 return # No change needed

 self._coordinate_clicking_enabled = enabled
 self._register_click_action()
 logger.debug(f'Coordinate clicking {"enabled" if enabled else "disabled"}')

Act -----
@observe_debug(ignore_input=True, ignore_output=True, name='act')
@time_execution_sync('--act')
async def act(
 self,
 action: ActionModel,
 browser_session: BrowserSession,
 page_extraction_llm: BaseChatModel | None = None,
 sensitive_data: dict[str, str | dict[str, str]] | None = None,
 available_file_paths: list[str] | None = None,
 file_system: FileSystem | None = None,
 extraction_schema: dict | None = None,
) -> ActionResult:
 """Execute an action"""

 for action_name, params in action.model_dump(exclude_unset=True).items():
 if params is not None:
 # Use Laminar span if available, otherwise use no-op context manager
 if Laminar is not None:
 span_context = Laminar.start_as_current_span(
 name=action_name,
 input={
 'action': action_name,
 'params': params,
 },
 span_type='TOOL',
)
 else:
 # No-op context manager when lmnr is not available
 from contextlib import nullcontext

 span_context = nullcontext()

 with span_context:
 try:
 result = await self.registry.execute_action(
 action_name=action_name,
 params=params,
 browser_session=browser_session,
 page_extraction_llm=page_extraction_llm,
 file_system=file_system,
 sensitive_data=sensitive_data,
 available_file_paths=available_file_paths,
 extraction_schema=extraction_schema,
)
 except BrowserError as e:
 logger.error(f'Action {action_name} failed with BrowserError: {str(e)}')
 result = handle_browser_error(e)

```

```

except TimeoutError as e:
 logger.error(f'Action {action_name} failed with TimeoutError: {str(e)}')
 result = ActionResult(error=f'{action_name} was not executed due to timeout.')
except Exception as e:
 # Log the original exception with traceback for observability
 logger.error(f'Action '{action_name}' failed with error: {str(e)}')
 result = ActionResult(error=str(e))

if Laminar is not None:
 Laminar.set_span_output(result)

if isinstance(result, str):
 return ActionResult(extracted_content=result)
elif isinstance(result, ActionResult):
 return result
elif result is None:
 return ActionResult()
else:
 raise ValueError(f'Invalid action result type: {type(result)} of {result}')
 return ActionResult()

def __getattr__(self, name: str):
 """
 Enable direct action calls like tools.navigate(url=..., browser_session=...).
 This provides a simpler API for tests and direct usage while maintaining backward compatibility.
 """
 # Check if this is a registered action
 if name in self.registry.actions:
 from typing import Union

 from pydantic import create_model

 action = self.registry.actions[name]

 # Create a wrapper that calls act() to ensure consistent error handling and result normalization
 async def action_wrapper(**kwargs):
 # Extract browser_session (required positional argument for act())
 browser_session = kwargs.get('browser_session')

 # Separate action params from special params (injected dependencies)
 special_param_names = {
 'browser_session',
 'page_extraction_llm',
 'file_system',
 'available_file_paths',
 'sensitive_data',
 'extraction_schema',
 }

 # Extract action params (params for the action itself)
 action_params = {k: v for k, v in kwargs.items() if k not in special_param_names}

 # Extract special params (injected dependencies) - exclude browser_session as it's positional
 special_kwargs = {k: v for k, v in kwargs.items() if k in special_param_names and k != 'browser_session'}

 # Create the param instance
 params_instance = action.param_model(**action_params)

 # Dynamically create an ActionModel with this action
 # Use Union for type compatibility with create_model
 DynamicActionModel = create_model(
 'DynamicActionModel',
 __base__=ActionModel,
 **{name: (Union[action.param_model, None], None)}, # type: ignore
)

 # Create the action model instance
 action_model = DynamicActionModel(**{name: params_instance})

 # Call act() which has all the error handling, result normalization, and observability
 # browser_session is passed as positional argument (required by act())
 return await self.act(action=action_model, browser_session=browser_session, **special_kwargs) # ty

```

```

 return action_wrapper

 # If not an action, raise AttributeError for normal Python behavior
 raise AttributeError(f"'{type(self).__name__}' object has no attribute '{name}'")

Alias for backwards compatibility
Controller = Tools

class CodeAgentTools(Tools[Context]):
 """Specialized Tools for CodeAgent agent optimized for Python-based browser automation.

 Includes:
 - All browser interaction tools (click, input, scroll, navigate, etc.)
 - JavaScript evaluation
 - Tab management (switch, close)
 - Navigation actions (go_back)
 - Upload file support
 - Dropdown interactions

 Excludes (optimized for code-use mode):
 - extract: Use Python + evaluate() instead
 - find_text: Use Python string operations
 - screenshot: Not needed in code-use mode
 - search: Use navigate() directly
 - File system actions (write_file, read_file, replace_file): Use Python file operations instead
 """

 def __init__(
 self,
 exclude_actions: list[str] | None = None,
 output_model: type[T] | None = None,
 display_files_in_done_text: bool = True,
):
 """Default exclusions for CodeAgent agent
 if exclude_actions is None:
 """
 exclude_actions = [
 # 'scroll', # Keep for code-use
 'extract', # Exclude - use Python + evaluate()
 'find_text', # Exclude - use Python string ops
 # 'select_dropdown', # Keep for code-use
 'dropdown_options', # Keep for code-use
 'screenshot', # Exclude - not needed
 'search', # Exclude - use navigate() directly
 # 'click', # Keep for code-use
 # 'input', # Keep for code-use
 # 'switch', # Keep for code-use
 # 'send_keys', # Keep for code-use
 # 'close', # Keep for code-use
 # 'go_back', # Keep for code-use
 # 'upload_file', # Keep for code-use
 # Exclude file system actions - CodeAgent should use Python file operations
 'write_file',
 'read_file',
 'replace_file',
]

 def __init__(
 self,
 exclude_actions=exclude_actions,
 output_model=output_model,
 display_files_in_done_text=display_files_in_done_text,
):
 """Override done action for CodeAgent with enhanced file handling
 self._register_code_use_done_action(output_model, display_files_in_done_text)

 def _register_code_use_done_action(
 self, output_model: type[T] | None, display_files_in_done_text: bool
):
 """Register enhanced done action for CodeAgent that can read files from disk."""
 if output_model is not None:
 # Structured output done - use parent's implementation
 return

```

```

 ## Override the done action with enhanced version
 @self.registry.action(
 'Complete task.',
 param_model=DoneAction,
)
 async def done(params: DoneAction, file_system: FileSystem):
 user_message = params.text

 len_text = len(params.text)
 len_max_memory = 100
 memory = f'Task completed: {params.success} - {params.text[:len_max_memory]}'
 if len_text > len_max_memory:
 memory += f' - {len_text - len_max_memory} more characters'

 attachments = []
 if params.files_to_display:
 if self.display_files_in_done_text:
 file_msg = ''
 for file_name in params.files_to_display:
 file_content = file_system.display_file(file_name)
 if file_content:
 file_msg += f'\n\n{file_name}:\n{file_content}'
 attachments.append(file_name)
 elif os.path.exists(file_name):
 # File exists on disk but not in FileSystem - just add to attachments
 attachments.append(file_name)
 if file_msg:
 user_message += '\n\nAttachments:'
 user_message += file_msg
 else:
 logger.warning('Agent wanted to display files but none were found')
 else:
 for file_name in params.files_to_display:
 file_content = file_system.display_file(file_name)
 if file_content:
 attachments.append(file_name)
 elif os.path.exists(file_name):
 attachments.append(file_name)

 ## Convert relative paths to absolute paths - handle both FileSystem-managed and regular files
 resolved_attachments = []
 for file_name in attachments:
 if os.path.isabs(file_name):
 # Already absolute
 resolved_attachments.append(file_name)
 elif file_system.get_file(file_name):
 # Managed by FileSystem
 resolved_attachments.append(str(file_system.get_dir() / file_name))
 elif os.path.exists(file_name):
 # Regular file in current directory
 resolved_attachments.append(os.path.abspath(file_name))
 else:
 # File doesn't exist, but include the path anyway for error visibility
 resolved_attachments.append(str(file_system.get_dir() / file_name))
 attachments = resolved_attachments

 return ActionResult(
 is_done=True,
 success=params.success,
 extracted_content=user_message,
 long_term_memory=memory,
 attachments=attachments,
)

 ## Override upload_file for code agent with relaxed path validation
 @self.registry.action(
 'Upload a file to a file input element. For code-use mode, any file accessible from the current dire',
 param_model=UploadFileAction,
)
 async def upload_file(
 params: UploadFileAction,
 browser_session: BrowserSession,

```

```

#####available_file_paths: list[str],
#####file_system: FileSystem,
#####):
Path validation logic for code-use mode:
1. If available_file_paths provided (security mode), enforce it as a whitelist
2. If no whitelist, for local browsers just check file exists
3. For remote browsers, allow any path (assume it exists remotely)

If whitelist provided, validate path is in it
#####if available_file_paths:
#####if params.path not in available_file_paths:
Also check if it's a recently downloaded file
#####downloaded_files = browser_session.downloaded_files
#####if params.path not in downloaded_files:
Finally, check if it's a file in the FileSystem service (if provided)
#####if file_system is not None and file_system.get_dir():
Check if the file is actually managed by the FileSystem service
The path should be just the filename for FileSystem files
#####file_obj = file_system.get_file(params.path)
#####if file_obj:
File is managed by FileSystem, construct the full path
#####file_system_path = str(file_system.get_dir() / params.path)
#####params = UploadFileAction(index=params.index, path=file_system_path)
#####else:
If browser is remote, allow passing a remote-accessible absolute path
#####if not browser_session.is_local:
#####pass
#####else:
#####msg = f'File path {params.path} is not available. To fix: add this file path to the available'
#####logger.error(f'##### {msg}')
#####return ActionResult(error=msg)
#####else:
If browser is remote, allow passing a remote-accessible absolute path
#####if not browser_session.is_local:
#####pass
#####else:
#####msg = f'File path {params.path} is not available. To fix: add this file path to the available'
#####logger.error(f'##### {msg}')
#####return ActionResult(error=msg)

For local browsers, ensure the file exists on the local filesystem
#####if browser_session.is_local:
#####if not os.path.exists(params.path):
#####msg = f'File {params.path} does not exist'
#####return ActionResult(error=msg)

Get the selector map to find the node
#####selector_map = await browser_session.get_selector_map()
#####if params.index not in selector_map:
#####msg = f'Element with index {params.index} does not exist.'
#####return ActionResult(error=msg)

#####node = selector_map[params.index]

Helper function to find file input near the selected element
#####def find_file_input_near_element(
#####node: EnhancedDOMTreeNode, max_height: int = 3, max_descendant_depth: int = 3
#####) -> EnhancedDOMTreeNode | None:
"""Find the closest file input to the selected element."""

#####def find_file_input_in_descendants(n: EnhancedDOMTreeNode, depth: int) -> EnhancedDOMTreeNode | None:
#####if depth < 0:
#####return None
#####if browser_session.is_file_input(n):
#####return n
#####for child in n.children_nodes or []:
#####result = find_file_input_in_descendants(child, depth - 1)
#####if result:
#####return result
#####return None

#####current = node

```

```

#####for _ in range(max_height + 1):
Check the current node itself
if browser_session.is_file_input(current):
return current
Check all descendants of the current node
result = find_file_input_in_descendants(current, max_descendant_depth)
if result:
return result
Check all siblings and their descendants
if current.parent_node:
for sibling in current.parent_node.children_nodes or []:
if sibling is current:
continue
if browser_session.is_file_input(sibling):
return sibling
result = find_file_input_in_descendants(sibling, max_descendant_depth)
if result:
return result
current = current.parent_node
if not current:
break
return None

Try to find a file input element near the selected element
file_input_node = find_file_input_near_element(node)

Highlight the file input element if found (truly non-blocking)
if file_input_node:
create_task_with_error_handling(
browser_session.highlight_interaction_element(file_input_node),
name='highlight_file_input',
suppress_exceptions=True,
)

If not found near the selected element, fallback to finding the closest file input to current scroll
if file_input_node is None:
logger.info(
f'No file upload element found near index {params.index}, searching for closest file input to scroll'
)

Get current scroll position
cdp_session = await browser_session.get_or_create_cdp_session()
try:
scroll_info = await cdp_session.cdp_client.send(Runtime.evaluate(
params={'expression': 'window.scrollY || window.pageYOffset || 0'}, session_id=cdp_session.session_id
))
current_scroll_y = scroll_info.get('result', {}).get('value', 0)
except Exception:
current_scroll_y = 0

Find all file inputs in the selector map and pick the closest one to scroll position
closest_file_input = None
min_distance = float('inf')

for idx, element in selector_map.items():
if browser_session.is_file_input(element):
Get element's Y position
if element.absolute_position:
element_y = element.absolute_position.y
distance = abs(element_y - current_scroll_y)
if distance < min_distance:
min_distance = distance
closest_file_input = element

if closest_file_input:
file_input_node = closest_file_input
logger.info(f'Found file input closest to scroll position (distance: {min_distance}px)')

Highlight the fallback file input element (truly non-blocking)
create_task_with_error_handling(
browser_session.highlight_interaction_element(file_input_node),
name='highlight_file_input_fallback',

```



```

#####suppress_exceptions=True,
#####)
#####else:
#####msg = 'No file upload element found on the page'
#####logger.error(msg)
#####raise BrowserError(msg)
TODO: figure out why this fails sometimes + add fallback hail mary, just look for any file input

Dispatch upload file event with the file input node
#####try:
#####event = browser_session.event_bus.dispatch(UploadFileEvent(node=file_input_node, file_path=params.p
#####await event
#####await event.event_result(raise_if_any=True, raise_if_none=False)
#####msg = f'Successfully uploaded file to index {params.index}'
#####logger.info(f'■ {msg}')
#####return ActionResult(
extracted_content=msg,
long_term_memory=f'Uploaded file {params.path} to element {params.index}',
#####)
#####except Exception as e:
logger.error(f'Failed to upload file: {e}')
raise BrowserError(f'Failed to upload file: {e}')

```

## ***.\system\tools\utils.py***

```

"""Utility functions for browser tools."""

from system.dom.service import EnhancedDOMTreeNode

def get_click_description(node: EnhancedDOMTreeNode) -> str:
 """Get a brief description of the clicked element for memory."""
 parts = []

 ## Tag name
 parts.append(node.tag_name)

 ## Add type for inputs
 if node.tag_name == 'input' and node.attributes.get('type'):
 input_type = node.attributes['type']
 parts.append(f'type={input_type}')

 ### For checkboxes, include checked state
 if input_type == 'checkbox':
 is_checked = node.attributes.get('checked', 'false').lower() in ['true', 'checked', '']
 ## Also check AX node
 if node.ax_node and node.ax_node.properties:
 for prop in node.ax_node.properties:
 if prop.name == 'checked':
 is_checked = prop.value is True or prop.value == 'true'
 break
 state = 'checked' if is_checked else 'unchecked'
 parts.append(f'checkbox-state={state}')

 ## Add role if present
 if node.attributes.get('role'):
 role = node.attributes['role']
 parts.append(f'role={role}')

 ### For role=checkbox, include state
 if role == 'checkbox':
 aria_checked = node.attributes.get('aria-checked', 'false').lower()
 is_checked = aria_checked in ['true', 'checked']
 if node.ax_node and node.ax_node.properties:
 for prop in node.ax_node.properties:
 if prop.name == 'checked':
 is_checked = prop.value is True or prop.value == 'true'
 break

```

```

state = 'checked' if is_checked else 'unchecked'
parts.append(f'checkbox-state={state}')

For labels/spans/divs, check if related to a hidden checkbox
if node.tag_name in ['label', 'span', 'div'] and 'type=' not in ' '.join(parts):
 # Check children for hidden checkbox
 for child in node.children:
 if child.tag_name == 'input' and child.attributes.get('type') == 'checkbox':
 # Check if hidden
 is_hidden = False
 if child.snapshot_node and child.snapshot_node.computed_styles:
 opacity = child.snapshot_node.computed_styles.get('opacity', '1')
 if opacity == '0' or opacity == '0.0':
 is_hidden = True

 if is_hidden or not child.is_visible:
 # Get checkbox state
 is_checked = child.attributes.get('checked', 'false').lower() in ['true', 'checked', '']
 if child.ax_node and child.ax_node.properties:
 for prop in child.ax_node.properties:
 if prop.name == 'checked':
 is_checked = prop.value is True or prop.value == 'true'
 break
 state = 'checked' if is_checked else 'unchecked'
 parts.append(f'checkbox-state={state}')
 break

Add short text content if available
text = node.get_all_children_text().strip()
if text:
 short_text = text[:30] + ('...' if len(text) > 30 else '')
 parts.append(f'"{short_text}"')

Add key attributes like id, name, aria-label
for attr in ['id', 'name', 'aria-label']:
 if node.attributes.get(attr):
 parts.append(f'{attr}={node.attributes[attr][:20]}')

return ' '.join(parts)

```

## **`.lsystem\tools\views.py`**

```

from typing import Generic, TypeVar

from pydantic import BaseModel, ConfigDict, Field
from pydantic.json_schema import SkipJsonSchema

Action Input Models
class ExtractAction(BaseModel):
 query: str
 extract_links: bool = Field(
 default=False, description='Set True to true if the query requires links, else false to safe tokens'
)
 start_from_char: int = Field(
 default=0, description='Use this for long markdowns to start from a specific character (not index in l'
)
 output_schema: SkipJsonSchema[dict | None] = Field(
 default=None,
 description='Optional JSON Schema dict. When provided, extraction returns validated JSON matching thi'
)

class SearchPageAction(BaseModel):
 pattern: str = Field(description='Text or regex pattern to search for in page content')
 regex: bool = Field(default=False, description='Treat pattern as regex (default: literal text match)')
 case_sensitive: bool = Field(default=False, description='Case-sensitive search (default: case-insensiti'
)
 context_chars: int = Field(default=150, description='Characters of surrounding context per match')

```

```

css_scope: str | None = Field(default=None, description='CSS selector to limit search scope (e.g. "div#main")')
max_results: int = Field(default=25, description='Maximum matches to return')

```

```

class FindElementsAction(BaseModel):
 selector: str = Field(description='CSS selector to query elements (e.g. "table tr", "a.link", "div.procedure")')
 attributes: list[str] | None = Field(
 default=None,
 description='Specific attributes to extract (e.g. ["href", "src", "class"]). If not set, returns tag and text content'
)
 max_results: int = Field(default=50, description='Maximum elements to return')
 include_text: bool = Field(default=True, description='Include text content of each element')

```

```

class SearchAction(BaseModel):
 query: str
 engine: str = Field(
 default='duckduckgo', description='duckduckgo, google, bing (use duckduckgo by default because less censored)'
)

```

```

Backward compatibility alias
SearchAction = SearchAction

```

```

class NavigateAction(BaseModel):
 url: str
 new_tab: bool = Field(default=False)

```

```

Backward compatibility alias
GoToUrlAction = NavigateAction

```

```

class ClickElementAction(BaseModel):
 index: int | None = Field(default=None, ge=1, description='Element index from browser_state')
 coordinate_x: int | None = Field(default=None, description='Horizontal coordinate relative to viewport')
 coordinate_y: int | None = Field(default=None, description='Vertical coordinate relative to viewport')
 expect_download: bool = Field(default=False, description='set True if expecting a download, False otherwise')
 click_count: int = 1 # TODO

```

```

class ClickElementActionIndexOnly(BaseModel):
 model_config = ConfigDict(title='ClickElementAction')

 index: int = Field(ge=1, description='Element index from browser_state')

```

```

class InputTextAction(BaseModel):
 index: int = Field(ge=0, description='from browser_state')
 text: str
 clear: bool = Field(default=True, description='1=clear, 0=append')

```

```

class DoneAction(BaseModel):
 text: str = Field(description='Final user message in the format the user requested')
 success: bool = Field(default=True, description='True if user_request completed successfully')
 files_to_display: list[str] | None = Field(default=[])

```

```

T = TypeVar('T', bound=BaseModel)

```

```

def _hide_internal_fields_from_schema(schema: dict) -> None:
 """Remove internal fields from the JSON schema to avoid collisions with user models."""
 props = schema.get('properties', {})
 props.pop('success', None)
 props.pop('files_to_display', None)

```

```

class StructuredOutputAction(BaseModel, Generic[T]):
 model_config = ConfigDict(json_schema_extra=_hide_internal_fields_from_schema)

```

```

■success: bool = Field(default=True, description='True if user_request completed successfully')
■data: T = Field(description='The actual output data matching the requested schema')
■files_to_display: list[str] | None = Field(default=[])

```

```

class SwitchTabAction(BaseModel):
 ■tab_id: str = Field(min_length=4, max_length=4, description='4-char id')

```

```

class CloseTabAction(BaseModel):
 ■tab_id: str = Field(min_length=4, max_length=4, description='4-char id')

```

```

class ScrollAction(BaseModel):
 ■down: bool = Field(default=True, description='down=True=scroll down, down=False scroll up')
 ■pages: float = Field(default=1.0, description='0.5=half page, 1=full page, 10=to bottom/top')
 ■index: int | None = Field(default=None, description='Optional element index to scroll within specific e

```

```

class SendKeysAction(BaseModel):
 ■keys: str = Field(description='keys (Escape, Enter, PageDown) or shortcuts (Control+o)')

```

```

class UploadFileAction(BaseModel):
 ■index: int
 ■path: str

```

```

class NoParamsAction(BaseModel):
 ■model_config = ConfigDict(extra='ignore')

```

```

 ■# Optional field required by Gemini API which errors on empty objects in response_schema
 ■description: str | None = Field(None, description='Optional description for the action')

```

```

class ScreenshotAction(BaseModel):
 ■model_config = ConfigDict(extra='ignore')

 ■file_name: str | None = Field(
 ■■default=None,
 ■■description='If provided, saves screenshot to this file and returns path. Otherwise screenshot is inc
 ■)

```

```

class SaveAsPdfAction(BaseModel):
 ■file_name: str | None = Field(
 ■■default=None,
 ■■description='Output PDF filename (without path). Defaults to page title. Extension .pdf is added autom
 ■)
 ■print_background: bool = Field(default=True, description='Include background graphics and colors')
 ■landscape: bool = Field(default=False, description='Use landscape orientation')
 ■scale: float = Field(default=1.0, ge=0.1, le=2.0, description='Scale of the webpage rendering (0.1 to 2
 ■paper_format: str = Field(
 ■■default='Letter',
 ■■description='Paper size: Letter, Legal, A4, A3, or Tabloid',
 ■)

```

```

class ReadContentAction(BaseModel):
 ■"""Action for intelligent reading of long content."""

 ■goal: str = Field(description='What to look for or extract from the content')
 ■source: str = Field(
 ■■default='page',
 ■■description='What to read: "page" for current webpage, or a file path',
 ■)
 ■context: str = Field(default='', description='Additional context about the task')

```

```

class GetDropdownOptionsAction(BaseModel):
 ■index: int
class SelectDropdownOptionAction(BaseModel):

```

```

index: int
text: str = Field(description='exact text/value')

```

## ***.\system\tools\extraction\schema\_utils.py***

```

"""Converts a JSON Schema dict to a runtime Pydantic model for structured extraction."""

import logging
from typing import Any

from pydantic import BaseModel, ConfigDict, Field, create_model

logger = logging.getLogger(__name__)

Keywords that indicate composition/reference patterns we don't support
_UNSUPPORTED_KEYWORDS = frozenset(
 [
 '$ref',
 'allOf',
 'anyOf',
 'oneOf',
 'not',
 '$defs',
 'definitions',
 'if',
 'then',
 'else',
 'dependentSchemas',
 'dependentRequired',
]
)

Primitive JSON Schema type → Python type
_PRIMITIVE_MAP: dict[str, type] = {
 'string': str,
 'number': float,
 'integer': int,
 'boolean': bool,
 'null': type(None),
}

class _StrictBase(BaseModel):
 model_config = ConfigDict(extra='forbid', validate_by_name=True, validate_by_alias=True)

 def _check_unsupported(schema: dict) -> None:
 """Raise ValueError if the schema uses unsupported composition keywords."""
 for kw in _UNSUPPORTED_KEYWORDS:
 if kw in schema:
 raise ValueError(f'Unsupported JSON Schema keyword: {kw}')

 def _resolve_type(schema: dict, name: str) -> Any:
 """Recursively resolve a JSON Schema node to a Python type.

 Returns a Python type suitable for use as a field type in pydantic.create_model.
 """
 _check_unsupported(schema)

 json_type = schema.get('type', 'string')

 # Enums – constrain to str (Literal would be stricter but LLMs are flaky)
 if 'enum' in schema:
 return str

 # Object with properties → nested pydantic model
 if json_type == 'object':

```

```

 properties = schema.get('properties', {})
 if properties:
 return _build_model(schema, name)
 return dict

 ## Array
 if json_type == 'array':
 items_schema = schema.get('items')
 if items_schema:
 item_type = _resolve_type(items_schema, f'{name}_item')
 return list[item_type]
 return list

 ## Primitive
 base = _PRIMITIVE_MAP.get(json_type, str)

 ## Nullable
 if schema.get('nullable', False):
 return base | None

 return base

_PRIMITIVE_DEFAULTS: dict[str, Any] = {
 'string': '',
 'number': 0.0,
 'integer': 0,
 'boolean': False,
}

def _build_model(schema: dict, name: str) -> type[BaseModel]:
 """Build a pydantic model from an object-type JSON Schema node."""
 _check_unsupported(schema)

 properties = schema.get('properties', {})
 required_fields = set(schema.get('required', []))
 fields: dict[str, Any] = {}

 for prop_name, prop_schema in properties.items():
 prop_type = _resolve_type(prop_schema, f'{name}_{prop_name}')

 if prop_name in required_fields:
 default = ...
 elif 'default' in prop_schema:
 default = prop_schema['default']
 elif prop_schema.get('nullable', False):
 # _resolve_type already made the type include None
 default = None
 else:
 # Non-required, non-nullable, no explicit default.
 # Use a type-appropriate zero value for primitives/arrays;
 # fall back to None (with | None) for enums and nested objects
 # where no in-set or constructible default exists.
 json_type = prop_schema.get('type', 'string')
 if 'enum' in prop_schema:
 # Can't pick an arbitrary enum member as default – use None
 # so absent fields serialize as null, not an out-of-set value.
 prop_type = prop_type | None
 default = None
 elif json_type in _PRIMITIVE_DEFAULTS:
 default = _PRIMITIVE_DEFAULTS[json_type]
 elif json_type == 'array':
 default = []
 else:
 # Nested object or unknown – must allow None as sentinel
 prop_type = prop_type | None
 default = None

 field_kwargs: dict[str, Any] = {}
 if 'description' in prop_schema:
 field_kwargs['description'] = prop_schema['description']

```

```

 if isinstance(default, list) and not default:
 fields[prop_name] = (prop_type, Field(default_factory=list, **field_kwargs))
 else:
 fields[prop_name] = (prop_type, Field(default, **field_kwargs))

 return create_model(name, __base__=StrictBase, **fields)

def schema_dict_to_pydantic_model(schema: dict) -> type[BaseModel]:
 """Convert a JSON Schema dict to a runtime Pydantic model.

 The schema must be ``{"type": "object", "properties": {...}, ...}``.
 Unsupported keywords ($ref, allOf, anyOf, oneOf, etc.) raise ValueError.

 Returns:
 A dynamically-created Pydantic BaseModel subclass.

 Raises:
 ValueError: If the schema is invalid or uses unsupported features.
 """
 _check_unsupported(schema)

 top_type = schema.get('type')
 if top_type != 'object':
 raise ValueError(f'Top-level schema must have type "object", got {top_type!r}')

 properties = schema.get('properties')
 if not properties:
 raise ValueError('Top-level schema must have at least one property')

 model_name = schema.get('title', 'DynamicExtractionModel')
 return _build_model(schema, model_name)

```

## **`.\system\tools\extraction\views.py`**

```

"""Pydantic models for the extraction subsystem."""

from typing import Any

from pydantic import BaseModel, ConfigDict, Field

class ExtractionResult(BaseModel):
 """Metadata about a structured extraction, stored in ActionResult.metadata."""

 model_config = ConfigDict(extra='forbid')

 data: dict[str, Any] = Field(description='The validated extraction payload')
 schema_used: dict[str, Any] = Field(description='The JSON Schema that was enforced')
 is_partial: bool = Field(default=False, description='True if content was truncated before extraction')
 source_url: str | None = Field(default=None, description='URL the content was extracted from')
 content_stats: dict[str, Any] = Field(default_factory=dict, description='Content processing statistics')

```

## **`.\system\tools\extraction\__init__.py`**

```

from system.tools.extraction.schema_utils import schema_dict_to_pydantic_model
from system.tools.extraction.views import ExtractionResult

__all__ = ['schema_dict_to_pydantic_model', 'ExtractionResult']

```

## **.lsystem\tools\registry\service.py**

```
import asyncio
import functools
import inspect
import logging
import re
from collections.abc import Callable
from inspect import Parameter, iscoroutinefunction, signature
from types import UnionType
from typing import Any, Generic, Optional, TypeVar, Union, get_args, get_origin

import pyotp
from pydantic import BaseModel, Field, RootModel, create_model

from system.browser import BrowserSession
from system.filesystem.file_system import FileSystem
from system.llm.base import BaseChatModel
from system.observability import observe_debug
from system.telemetry.service import ProductTelemetry
from system.tools.registry.views import (
 ■ActionModel,
 ■ActionRegistry,
 ■RegisteredAction,
 ■SpecialActionParameters,
)
from system.utils import is_new_tab_page, match_url_with_domain_pattern, time_execution_async

Context = TypeVar('Context')

logger = logging.getLogger(__name__)

class Registry(Generic[Context]):
 ■"""Service for registering and managing actions"""

 ■def __init__(self, exclude_actions: list[str] | None = None):
 ■■self.registry = ActionRegistry()
 ■■self.telemetry = ProductTelemetry()
 ■■# Create a new list to avoid mutable default argument issues
 ■■self.exclude_actions = list(exclude_actions) if exclude_actions is not None else []

 ■def exclude_action(self, action_name: str) -> None:
 ■■"""Exclude an action from the registry after initialization.

 ■■If the action is already registered, it will be removed from the registry.
 ■■The action is also added to the exclude_actions list to prevent re-registration.
 ■■"""
 ■■# Add to exclude list to prevent future registration
 ■■if action_name not in self.exclude_actions:
 ■■■self.exclude_actions.append(action_name)

 ■■# Remove from registry if already registered
 ■■if action_name in self.registry.actions:
 ■■■del self.registry.actions[action_name]
 ■■■logger.debug(f'Excluded action "{action_name}" from registry')

 ■def _get_special_param_types(self) -> dict[str, type | UnionType | None]:
 ■■"""Get the expected types for special parameters from SpecialActionParameters"""
 ■■# Manually define the expected types to avoid issues with Optional handling.
 ■■# we should try to reduce this list to 0 if possible, give as few standardized objects to all the act.
 ■■# but each driver should decide what is relevant to expose the action methods,
 ■■# e.g. CDP client, 2fa code getters, sensitive_data wrappers, other context, etc.
 ■■return {
 ■■■'context': None, # Context is a TypeVar, so we can't validate type
 ■■■'browser_session': BrowserSession,
 ■■■'page_url': str,
 ■■■'cdp_client': None, # CDPClient type from cdp_use, but we don't import it here
 ■■■'page_extraction_llm': BaseChatModel,
 ■■■'available_file_paths': list,
```



```

 """'has_sensitive_data': bool,
 """'file_system': FileSystem,
 """'extraction_schema': None, # dict | None, skip type validation
 """}

 def _normalize_action_function_signature(
 self,
 func: Callable,
 description: str,
 param_model: type[BaseModel] | None = None,
) -> tuple[Callable, type[BaseModel]]:
 """
 Normalize action function to accept only kwargs.

 Returns:
 - Normalized function that accepts (*_, params: ParamModel, **special_params)
 - The param model to use for registration
 """
 sig = signature(func)
 parameters = list(sig.parameters.values())
 special_param_types = self._get_special_param_types()
 special_param_names = set(special_param_types.keys())

 ## Step 1: Validate no **kwargs in original function signature
 ## if it needs default values it must use a dedicated param_model: BaseModel instead
 for param in parameters:
 if param.kind == Parameter.VAR_KEYWORD:
 raise ValueError(
 f"Action '{func.__name__}' has **{param.name} which is not allowed. "
 f"Actions must have explicit positional parameters only."
)

 ## Step 2: Separate special and action parameters
 action_params = []
 special_params = []
 param_model_provided = param_model is not None

 for i, param in enumerate(parameters):
 ## Check if this is a Type 1 pattern (first param is BaseModel)
 if i == 0 and param_model_provided and param.name not in special_param_names:
 ## This is Type 1 pattern - skip the params argument
 continue

 if param.name in special_param_names:
 ## Validate special parameter type
 expected_type = special_param_types.get(param.name)
 if param.annotation != Parameter.empty and expected_type is not None:
 ## Handle Optional types - normalize both sides
 param_type = param.annotation
 origin = get_origin(param_type)
 if origin is Union:
 args = get_args(param_type)
 ## Find non-None type
 param_type = next((arg for arg in args if arg is not type(None)), param_type)

 ## Check if types are compatible (exact match, subclass, or generic list)
 types_compatible = (
 param_type == expected_type
 or (
 inspect.isclass(param_type)
 and inspect.isclass(expected_type)
 and issubclass(param_type, expected_type)
)
 or
 ## Handle list[T] vs list comparison
 (expected_type is list and (param_type is list or get_origin(param_type) is list))
)

 if not types_compatible:
 expected_type_name = getattr(expected_type, '__name__', str(expected_type))
 param_type_name = getattr(param_type, '__name__', str(param_type))
 raise ValueError(

```

```

#####f"Action '{func.__name__}' parameter '{param.name}: {param_type_name}' "
#####f"conflicts with special argument injected by tools: '{param.name}: {expected_type_name}'"
#####)
#####special_params.append(param)
#####else:
#####action_params.append(param)

Step 3: Create or validate param model
###if not param_model_provided:
#####Type 2: Generate param model from action params
#####if action_params:
#####params_dict = {}
#####for param in action_params:
#####annotation = param.annotation if param.annotation != Parameter.empty else str
#####default = ... if param.default == Parameter.empty else param.default
#####params_dict[param.name] = (annotation, default)

#####param_model = create_model(f'{func.__name__}_Params', __base__=ActionModel, **params_dict)
#####else:
No action params, create empty model
#####param_model = create_model(
#####f'{func.__name__}_Params',
#####__base__=ActionModel,
#####)
#####assert param_model is not None, f'param_model is None for {func.__name__}'

Step 4: Create normalized wrapper function
###@functools.wraps(func)
###async def normalized_wrapper(*args, params: BaseModel | None = None, **kwargs):
#####"""Normalized action that only accepts kwargs"""
Validate no positional args
#####if args:
#####raise TypeError(f'{func.__name__}() does not accept positional arguments, only keyword arguments')

Prepare arguments for original function
#####call_args = []
#####call_kwargs = {}

Handle Type 1 pattern (first arg is the param model)
#####if param_model_provided and parameters and parameters[0].name not in special_param_names:
#####if params is None:
#####raise ValueError(f'{func.__name__}() missing required 'params' argument')
For Type 1, we'll use the params object as first argument
#####pass
#####else:
Type 2 pattern - need to unpack params
If params is None, try to create it from kwargs
#####if params is None and action_params:
Extract action params from kwargs
#####action_kwargs = {}
#####for param in action_params:
#####if param.name in kwargs:
#####action_kwargs[param.name] = kwargs[param.name]
#####if action_kwargs:
Use the param_model which has the correct types defined
#####params = param_model(**action_kwargs)

Build call_args by iterating through original function parameters in order
#####params_dict = params.model_dump() if params is not None else {}

#####for i, param in enumerate(parameters):
Skip first param for Type 1 pattern (it's the model itself)
#####if param_model_provided and i == 0 and param.name not in special_param_names:
#####call_args.append(params)
#####elif param.name in special_param_names:
This is a special parameter
#####if param.name in kwargs:
#####value = kwargs[param.name]
Check if required special param is None
#####if value is None and param.default == Parameter.empty:
#####if param.name == 'browser_session':
#####raise ValueError(f'Action {func.__name__} requires browser_session but none provided.')

```

```

#####elif param.name == 'page_extraction_llm':
#####raise ValueError(f'Action {func.__name__} requires page_extraction_llm but none provided.')
#####elif param.name == 'file_system':
#####raise ValueError(f'Action {func.__name__} requires file_system but none provided.')
#####elif param.name == 'page':
#####raise ValueError(f'Action {func.__name__} requires page but none provided.')
#####elif param.name == 'available_file_paths':
#####raise ValueError(f'Action {func.__name__} requires available_file_paths but none provided.')
#####elif param.name == 'file_system':
#####raise ValueError(f'Action {func.__name__} requires file_system but none provided.')
#####else:
#####raise ValueError(f"{func.__name__}() missing required special parameter '{param.name}'")
#####call_args.append(value)
#####elif param.default != Parameter.empty:
#####call_args.append(param.default)
#####else:
Special param is required but not provided
#####if param.name == 'browser_session':
#####raise ValueError(f'Action {func.__name__} requires browser_session but none provided.')
#####elif param.name == 'page_extraction_llm':
#####raise ValueError(f'Action {func.__name__} requires page_extraction_llm but none provided.')
#####elif param.name == 'file_system':
#####raise ValueError(f'Action {func.__name__} requires file_system but none provided.')
#####elif param.name == 'page':
#####raise ValueError(f'Action {func.__name__} requires page but none provided.')
#####elif param.name == 'available_file_paths':
#####raise ValueError(f'Action {func.__name__} requires available_file_paths but none provided.')
#####elif param.name == 'file_system':
#####raise ValueError(f'Action {func.__name__} requires file_system but none provided.')
#####else:
#####raise ValueError(f"{func.__name__}() missing required special parameter '{param.name}'")
#####else:
This is an action parameter
#####if param.name in params_dict:
#####call_args.append(params_dict[param.name])
#####elif param.default != Parameter.empty:
#####call_args.append(param.default)
#####else:
#####raise ValueError(f"{func.__name__}() missing required parameter '{param.name}'")

Call original function with positional args
###if iscoroutinefunction(func):
###return await func(*call_args)
###else:
###return await asyncio.to_thread(func, *call_args)

Update wrapper signature to be kwargs-only
###new_params = [Parameter('params', Parameter.KEYWORD_ONLY, default=None, annotation=Optional[param_model])

Add special params as keyword-only
###for sp in special_params:
###new_params.append(Parameter(sp.name, Parameter.KEYWORD_ONLY, default=sp.default, annotation=sp.annot

Add **kwargs to accept and ignore extra params
###new_params.append(Parameter('kwargs', Parameter.VAR_KEYWORD))

###normalized_wrapper.__signature__ = sig.replace(parameters=new_params) # type: ignore[attr-defined]

###return normalized_wrapper, param_model

@time_execution_sync('--create_param_model')
###def _create_param_model(self, function: Callable) -> type[BaseModel]:
###"""Creates a Pydantic model from function signature"""
###sig = signature(function)
###special_param_names = set(SpecialActionParameters.model_fields.keys())
###params = {
name: (param.annotation, ... if param.default == param.empty else param.default)
for name, param in sig.parameters.items()
if name not in special_param_names
###}
TODO: make the types here work
###return create_model(

```

```

 """f'{function.__name__}_parameters',
 """base__=ActionModel,
 """**params, # type: ignore
 """
)

def action(
 self,
 description: str,
 param_model: type[BaseModel] | None = None,
 domains: list[str] | None = None,
 allowed_domains: list[str] | None = None,
 terminates_sequence: bool = False,
):
 """Decorator for registering actions"""
 # Handle aliases: domains and allowed_domains are the same parameter
 if allowed_domains is not None and domains is not None:
 raise ValueError("Cannot specify both 'domains' and 'allowed_domains' - they are aliases for the same")

 final_domains = allowed_domains if allowed_domains is not None else domains

 def decorator(func: Callable):
 # Skip registration if action is in exclude_actions
 if func.__name__ in self.exclude_actions:
 return func

 # Normalize the function signature
 normalized_func, actual_param_model = self._normalize_action_function_signature(func, description, param_model)

 action = RegisteredAction(
 name=func.__name__,
 description=description,
 function=normalized_func,
 param_model=actual_param_model,
 domains=final_domains,
 terminates_sequence=terminates_sequence,
)
 self.registry.actions[func.__name__] = action

 # Return the normalized function so it can be called with kwargs
 return normalized_func

 return decorator

@observe_debug(ignore_input=True, ignore_output=True, name='execute_action')
@time_execution_async('--execute_action')
async def execute_action(
 self,
 action_name: str,
 params: dict,
 browser_session: BrowserSession | None = None,
 page_extraction_llm: BaseChatModel | None = None,
 file_system: FileSystem | None = None,
 sensitive_data: dict[str, str | dict[str, str]] | None = None,
 available_file_paths: list[str] | None = None,
 extraction_schema: dict | None = None,
) -> Any:
 """Execute a registered action with simplified parameter handling"""
 if action_name not in self.registry.actions:
 raise ValueError(f'Action {action_name} not found')

 action = self.registry.actions[action_name]
 try:
 # Create the validated Pydantic model
 validated_params = action.param_model(**params)
 except Exception as e:
 raise ValueError(f'Invalid parameters {params} for action {action_name}: {type(e)}: {e}') from e

 if sensitive_data:
 # Get current URL if browser_session is provided
 current_url = None
 if browser_session and browser_session.agent_focus_target_id:

```

```

 try:
 # Get current page info from session_manager
 target = browser_session.session_manager.get_target(browser_session.agent_focus_target_id)
 if target:
 current_url = target.url
 except Exception:
 pass
 validated_params = self._replace_sensitive_data(validated_params, sensitive_data, current_url)

 # Build special context dict
 special_context = {
 'browser_session': browser_session,
 'page_extraction_llm': page_extraction_llm,
 'available_file_paths': available_file_paths,
 'has_sensitive_data': action_name == 'input' and bool(sensitive_data),
 'file_system': file_system,
 'extraction_schema': extraction_schema,
 }

 # Only pass sensitive_data to actions that explicitly need it (input)
 if action_name == 'input':
 special_context['sensitive_data'] = sensitive_data

 # Add CDP-related parameters if browser_session is available
 if browser_session:
 # Add page_url
 try:
 special_context['page_url'] = await browser_session.get_current_page_url()
 except Exception:
 special_context['page_url'] = None

 # Add cdp_client
 special_context['cdp_client'] = browser_session.cdp_client

 # All functions are now normalized to accept kwargs only
 # Call with params and unpacked special context
 try:
 return await action.function(params=validated_params, **special_context)
 except Exception as e:
 raise

 except ValueError as e:
 # Preserve ValueError messages from validation
 if 'requires browser_session but none provided' in str(e) or 'requires page_extraction_llm but none':
 e
):
 raise RuntimeError(str(e)) from e
 else:
 raise RuntimeError(f'Error executing action {action_name}: {str(e)}') from e
 except TimeoutError as e:
 raise RuntimeError(f'Error executing action {action_name} due to timeout.') from e
 except Exception as e:
 raise RuntimeError(f'Error executing action {action_name}: {str(e)}') from e

 def _log_sensitive_data_usage(self, placeholders_used: set[str], current_url: str | None) -> None:
 """Log when sensitive data is being used on a page"""
 if placeholders_used:
 url_info = f' on {current_url}' if current_url and not is_new_tab_page(current_url) else ''
 logger.info(f'Using sensitive data placeholders: {", ".join(sorted(placeholders_used))}{url_info}')

 def _replace_sensitive_data(
 self, params: BaseModel, sensitive_data: dict[str, Any], current_url: str | None = None
) -> BaseModel:
 """
 Replaces sensitive data placeholders in params with actual values.

 Args:
 params: The parameter object containing <secret>placeholder</secret> tags
 sensitive_data: Dictionary of sensitive data, either in old format {key: value}
 or new format {domain_pattern: {key: value}}
 current_url: Optional current URL for domain matching
 Returns:

```

```

BaseModel: The parameter object with placeholders replaced by actual values
#####
secret_pattern = re.compile(r'<secret>(.*?)</secret>')

Set to track all missing placeholders across the full object
all_missing_placeholders = set()
Set to track successfully replaced placeholders
replaced_placeholders = set()

Process sensitive data based on format and current URL
applicable_secrets = {}

for domain_or_key, content in sensitive_data.items():
if isinstance(content, dict):
New format: {domain_pattern: {key: value}}
Only include secrets for domains that match the current URL
if current_url and not is_new_tab_page(current_url):
it's a real url, check it using our custom allowed_domains scheme://*.example.com glob matching
if match_url_with_domain_pattern(current_url, domain_or_key):
applicable_secrets.update(content)
else:
Old format: {key: value}, expose to all domains (only allowed for legacy reasons)
applicable_secrets[domain_or_key] = content

Filter out empty values
applicable_secrets = {k: v for k, v in applicable_secrets.items() if v}

def recursively_replace_secrets(value: str | dict | list) -> str | dict | list:
if isinstance(value, str):
1. Handle tagged secrets: <secret>label</secret>
matches = secret_pattern.findall(value)
for placeholder in matches:
if placeholder in applicable_secrets:
generate a totp code if secret is suffixed with bu_2fa_code
if placeholder.endswith('bu_2fa_code'):
totp = pyotp.TOTP(applicable_secrets[placeholder], digits=6)
replacement_value = totp.now()
else:
replacement_value = applicable_secrets[placeholder]

value = value.replace(f'<secret>{placeholder}</secret>', replacement_value)
replaced_placeholders.add(placeholder)
else:
Keep track of missing placeholders
all_missing_placeholders.add(placeholder)

2. Handle literal secrets: "user_name" (no tags)
This handles cases where the LLM forgets to use tags but uses the exact placeholder name
if value in applicable_secrets:
placeholder_name = value
if placeholder_name.endswith('bu_2fa_code'):
totp = pyotp.TOTP(applicable_secrets[placeholder_name], digits=6)
value = totp.now()
else:
value = applicable_secrets[placeholder_name]
replaced_placeholders.add(placeholder_name)

return value
elif isinstance(value, dict):
return {k: recursively_replace_secrets(v) for k, v in value.items()}
elif isinstance(value, list):
return [recursively_replace_secrets(v) for v in value]
return value

params_dump = params.model_dump()
processed_params = recursively_replace_secrets(params_dump)

Log sensitive data usage
self._log_sensitive_data_usage(replaced_placeholders, current_url)

Log a warning if any placeholders are missing
if all_missing_placeholders:

```

```

 logger.warning(f'Missing or empty keys in sensitive_data dictionary: {", ".join(all_missing_placeholders)}')

 return type(params).model_validate(processed_params)

 @time_execution_sync('--create_action_model')
 def create_action_model(self, include_actions: list[str] | None = None, page_url: str | None = None) -> Union[ActionModel, EmptyActionModel]:
 """Creates a Union of individual action models from registered actions,
 used by LLM APIs that support tool calling & enforce a schema.

 Each action model contains only the specific action being used,
 rather than all actions with most set to None.
 """
 from typing import Union

 # Filter actions based on page_url if provided:
 # if page_url is None, only include actions with no filters
 # if page_url is provided, only include actions that match the URL

 available_actions: dict[str, RegisteredAction] = {}
 for name, action in self.registry.actions.items():
 if include_actions is not None and name not in include_actions:
 continue

 # If no page_url provided, only include actions with no filters
 if page_url is None:
 if action.domains is None:
 available_actions[name] = action
 continue

 # Check domain filter if present
 domain_is_allowed = self.registry._match_domains(action.domains, page_url)

 # Include action if domain filter matches
 if domain_is_allowed:
 available_actions[name] = action

 # Create individual action models for each action
 individual_action_models: list[type[BaseModel]] = []

 for name, action in available_actions.items():
 # Create an individual model for each action that contains only one field
 individual_model = create_model(
 f'{name.title().replace("_", "")}ActionModel',
 __base__=ActionModel,
 __slots__={
 'name': (
 action.param_model,
 Field(description=action.description),
) # type: ignore
 },
)
 individual_action_models.append(individual_model)

 # If no actions available, return empty ActionModel
 if not individual_action_models:
 return create_model('EmptyActionModel', __base__=ActionModel)

 # Create proper Union type that maintains ActionModel interface
 if len(individual_action_models) == 1:
 # If only one action, return it directly (no Union needed)
 result_model = individual_action_models[0]
 else:
 # Meaning the length is more than 1
 # Create a Union type using RootModel that properly delegates ActionModel methods
 union_type = Union[tuple(individual_action_models)] # type: ignore : Typing doesn't understand that

 class ActionModelUnion(RootModel[union_type]): # type: ignore
 def get_index(self) -> int | None:
 """Delegate get_index to the underlying action model"""
 if hasattr(self.root, 'get_index'):
 return self.root.get_index() # type: ignore

```

```

 return None

 def set_index(self, index: int):
 """Delegate set_index to the underlying action model"""
 if hasattr(self.root, 'set_index'):
 self.root.set_index(index) # type: ignore

 def model_dump(self, **kwargs):
 """Delegate model_dump to the underlying action model"""
 if hasattr(self.root, 'model_dump'):
 return self.root.model_dump(**kwargs) # type: ignore
 return super().model_dump(**kwargs)

 # Set the name for better debugging
 ActionModelUnion.__name__ = 'ActionModel'
 ActionModelUnion.__qualname__ = 'ActionModel'

 result_model = ActionModelUnion

 return result_model # type: ignore

 def get_prompt_description(self, page_url: str | None = None) -> str:
 """Get a description of all actions for the prompt

 If page_url is provided, only include actions that are available for that URL
 based on their domain filters
 """
 return self.registry.get_prompt_description(page_url=page_url)

```

## ***.\system\tools\registry\views.py***

```

from collections.abc import Callable
from typing import TYPE_CHECKING, Any

from pydantic import BaseModel, ConfigDict

from system.browser import BrowserSession
from system.filesystem.file_system import FileSystem
from system.llm.base import BaseChatModel

if TYPE_CHECKING:
 pass

class RegisteredAction(BaseModel):
 """Model for a registered action"""

 name: str
 description: str
 function: Callable
 param_model: type[BaseModel]

 # If True, this action is known to change the page (e.g. navigate, search, go_back, switch).
 # multi_act() will abort remaining queued actions after executing a terminates_sequence action.
 terminates_sequence: bool = False

 # filters: provide specific domains to determine whether the action should be available on the given URL
 domains: list[str] | None = None # e.g. ['*.google.com', 'www.bing.com', 'yahoo.*']

 model_config = ConfigDict(arbitrary_types_allowed=True)

 def prompt_description(self) -> str:
 """Get a description of the action for the prompt in unstructured format"""
 schema = self.param_model.model_json_schema()
 params = []

 if 'properties' in schema:
 for param_name, param_info in schema['properties'].items():

```



```

Build parameter description
#####param_desc = param_name

Add type information if available
#####if 'type' in param_info:
#####param_type = param_info['type']
#####param_desc += f'={param_type}'

Add description as comment if available
#####if 'description' in param_info:
#####param_desc += f' ({param_info["description"]})'

#####params.append(param_desc)

Format: action_name: Description. (param1=type, param2=type, ...)
#####if params:
#####return f'{self.name}: {self.description}. ({", ".join(params)})'
#####else:
#####return f'{self.name}: {self.description}'

class ActionModel(BaseModel):
 ##### "Base model for dynamically created action models"

 ##### this will have all the registered actions, e.g.
 ##### click_element = param_model = ClickElementParams
 ##### done = param_model = None
 #####
 #####model_config = ConfigDict(arbitrary_types_allowed=True, extra='forbid')

 #####def get_index(self) -> int | None:
 ##### ##### "Get the index of the action"
 ##### ##### {'clicked_element': {'index':5}}
 ##### #####params = self.model_dump(exclude_unset=True).values()
 ##### #####if not params:
 ##### ##### #####return None
 ##### #####for param in params:
 ##### ##### #####if param is not None and 'index' in param:
 ##### ##### ##### #####return param['index']
 ##### #####return None

 #####def set_index(self, index: int):
 ##### ##### "Overwrite the index of the action"
 ##### ##### Get the action name and params
 ##### #####action_data = self.model_dump(exclude_unset=True)
 ##### #####action_name = next(iter(action_data.keys()))
 ##### #####action_params = getattr(self, action_name)

 ##### ##### Update the index directly on the model
 ##### #####if hasattr(action_params, 'index'):
 ##### ##### #####action_params.index = index

class ActionRegistry(BaseModel):
 ##### "Model representing the action registry"

 #####actions: dict[str, RegisteredAction] = {}

 #####@staticmethod
 #####def _match_domains(domains: list[str] | None, url: str) -> bool:
 ##### #####
 ##### #####Match a list of domain glob patterns against a URL.

 ##### #####Args:
 ##### #####domains: A list of domain patterns that can include glob patterns (* wildcard)
 ##### #####url: The URL to match against

 ##### #####Returns:
 ##### #####True if the URL's domain matches the pattern, False otherwise
 ##### #####

 ##### #####if domains is None or not url:

```

```

 return True

 # Use the centralized URL matching logic from utils
 from system.utils import match_url_with_domain_pattern

 for domain_pattern in domains:
 if match_url_with_domain_pattern(url, domain_pattern):
 return True
 return False

 def get_prompt_description(self, page_url: str | None = None) -> str:
 """Get a description of all actions for the prompt

 Args:
 page_url: If provided, filter actions by URL using domain filters.

 Returns:
 A string description of available actions.
 - If page is None: return only actions with no page_filter and no domains (for system prompt)
 - If page is provided: return only filtered actions that match the current page (excluding unfiltered)
 """
 if page_url is None:
 # For system prompt (no URL provided), include only actions with no filters
 return '\n'.join(action.prompt_description() for action in self.actions.values() if action.domains is None)

 # only include filtered actions for the current page URL
 filtered_actions = []
 for action in self.actions.values():
 if not action.domains:
 # skip actions with no filters, they are already included in the system prompt
 continue

 # Check domain filter
 if self._match_domains(action.domains, page_url):
 filtered_actions.append(action)

 return '\n'.join(action.prompt_description() for action in filtered_actions)

class SpecialActionParameters(BaseModel):
 """Model defining all special parameters that can be injected into actions"""

 model_config = ConfigDict(arbitrary_types_allowed=True)

 # optional user-provided context object passed down from Agent(context=...)
 # e.g. can contain anything, external db connections, file handles, queues, runtime config objects, etc
 # that you might want to be able to access quickly from within many of your actions
 # browser-use code doesn't use this at all, we just pass it down to your actions for convenience
 context: Any | None = None

 # browser-use session object, can be used to create new tabs, navigate, access CDP
 browser_session: BrowserSession | None = None

 # Current page URL for filtering and context
 page_url: str | None = None

 # CDP client for direct Chrome DevTools Protocol access
 cdp_client: Any | None = None # CDPCClient type from cdp_use

 # extra injected config if the action asks for these arg names
 page_extraction_llm: BaseChatModel | None = None
 file_system: FileSystem | None = None
 available_file_paths: list[str] | None = None
 has_sensitive_data: bool = False
 extraction_schema: dict | None = None

 @classmethod
 def get_browser_requiring_params(cls) -> set[str]:
 """Get parameter names that require browser_session"""
 return {'browser_session', 'cdp_client', 'page_url'}

```

# Directory: tests

## .ltests\test\_agent.py

```
import pytest
import asyncio
import os
from dotenv import load_dotenv

from system.agent.service import Agent
from system.browser.session import BrowserSession
from system.browser.profile import BrowserProfile
from system.llm import ChatGoogle

import logging

Load environment variables
load_dotenv()

Set up logging so the user can see the agent's thoughts in the console
logging.basicConfig(level=logging.INFO)

@pytest.mark.asyncio
async def test_agent_integration():
 """Verify that the MIRA Agent can successfully initialize and execute a command."""

 api_key = os.getenv("GOOGLE_API_KEY")
 if not api_key:
 pytest.skip("GOOGLE_API_KEY not found in environment. Skipping Agent integration test.")

 print("\nStarting Agent Integration Test...")

 # Initialize the specific LLM component from MIRA's abstraction layer
 llm = ChatGoogle(model="gemini-2.5-flash", api_key=api_key)

 # Initialize an isolated browser session (headless=False so you can see it work)
 profile = BrowserProfile(headless=False, disable_security=True, user_data_dir='/tmp/mock_agent_user_')
 session = BrowserSession(browser_profile=profile)

 # Initialize the Agent
 agent = Agent(
 task="Go to Wikipedia, search for 'Artificial Intelligence', find the first paragraph, and summarize it",
 llm=llm,
 browser_session=session
)

 try:
 await session.start()

 # Run the agent execution loop
 history = await agent.run()

 # Verify the history and final state
 assert history.is_successful(), "The agent did not successfully finish the task."

 final_result = history.final_result()
 assert final_result is not None, "Final result should not be None."
 assert "Success" in final_result, f"Agent failed to find the target. Final result was: {final_result}"

 # Format a summary for the user
 print("\n\n" + "="*50)
 print("AGENT EXECUTION SUMMARY")
 print("="*50)
 print(f"Total Steps Taken: {len(history.history)}")
 for i, step in enumerate(history.history):
 if set_action := step.model_output:
```

```

 print(f"Step {i+1}: Action -> {list(set_action.action[0].model_dump(exclude_unset=True))}")
 if set_action.current_state.thinking:
 print(f" Thought: {set_action.current_state.thinking}")

 print("-" * 50)
 print(f"■ Final Result: {final_result}")
 print("=" * 50 + "\n")
except Exception as e:
 pytest.fail(f"Agent run failed throwing an exception: {str(e)}")
finally:
 await session.stop()

```

## ***.ltests\test\_browser\_session.py***

```

import asyncio
import pytest
from system.browser.profile import BrowserProfile
from system.browser.session import BrowserSession
from system.browser.events import NavigateToUrlEvent

@pytest.mark.asyncio
async def test_browser_launch_and_navigate():
 # Initialize a basic browser session (headless for testing)
 profile = BrowserProfile(headless=True, disable_security=True, user_data_dir='/tmp/mock_user_data_dir')
 session = BrowserSession(browser_profile=profile)

 try:
 await session.start()

 # Navigate using the event system
 # Dispatch returns an Event object (or coroutine), we can await it if needed
 event = NavigateToUrlEvent(url="https://example.com")
 await session.event_bus.dispatch(event)
 await asyncio.sleep(2) # Give it an extra second to settle

 # Request state via the session helper
 state = await session.get_browser_state_summary(include_screenshot=False)

 assert "example.com" in state.url
 assert len(state.tabs) > 0

 finally:
 # Cleanly stop the browser
 await session.stop()

```

## ***.ltests\test\_dom.py***

```

import pytest
import asyncio
from system.browser.profile import BrowserProfile
from system.browser.session import BrowserSession
from system.browser.events import NavigateToUrlEvent

@pytest.mark.asyncio
async def test_dom_interaction_and_extraction():
 """Verify the DOM module correctly parsing elements and returning standard formatting."""
 profile = BrowserProfile(headless=True, disable_security=True, user_data_dir='/tmp/mock_user_data_dir')
 session = BrowserSession(browser_profile=profile)

 try:
 await session.start()

 # Navigate to a very simple website with guaranteed structure
 event = NavigateToUrlEvent(url="https://example.com")

```

```

await session.event_bus.dispatch(event)
await asyncio.sleep(2) # Wait for page to settle fully

Request full state, demanding DOM inclusion
state = await session.get_browser_state_summary(include_screenshot=False)

1. Assert DOM State Object Exists
assert state.dom_state is not None, "DOM state was completely empty"

2. Extract string representation of the parsed DOM
dom_markdown = state.dom_state.llm_representation()

3. Assert HTML structure exists in the parsed output
Example Domain has an <h1> tag with "Example Domain" and an <a> tag linking to iana.org
assert "Example Domain" in dom_markdown, "Could not find the main header text in DOM extraction"
assert "Learn more" in dom_markdown, "Could not find the hyperlink text in DOM extraction"

4. Assert Interactive Elements are Mapped
Is there at least one actionable element (the <a> link)?
assert len(state.dom_state.selector_map) > 0, "No interactive elements were mapped by the dom_wa

Check if any mapped element has an accessible name mentioning "Learn more"
found_link = False
for node_id, element_node in state.dom_state.selector_map.items():
 if "Learn more" in element_node.get_meaningful_text_for_llm():
 found_link = True
 break

assert found_link, "The href link to IANA was parsed, but not correctly stored in the interactiv

finally:
 await session.stop()

```

## ***.ltests/test\_llm.py***

```

import asyncio
import os
import sys
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from system.llm import ChatGoogle, UserMessage
from system.agent.views import ActionModel
from pydantic import BaseModel, ConfigDict
from dotenv import load_dotenv

Load environment variables from the MIRA/.env file
load_dotenv()

We need a small mock action since we haven't built Controller yet.
class MockClickAction(ActionModel):
 model_config = ConfigDict(extra='forbid')
 click_element_id: int

class MockActionOutput(BaseModel):
 model_config = ConfigDict(arbitrary_types_allowed=True, extra='forbid')
 action: list[MockClickAction]

async def test_llm_abstraction():
 print("Testing LLM Abstraction Layer...")

 api_key = os.getenv("GOOGLE_API_KEY")
 if not api_key:
 print("■■■ GOOGLE_API_KEY not found in environment. Skipping real API call.")
 return

 llm = ChatGoogle(model="gemini-2.5-flash", api_key=api_key)

 messages = [

```

```

 UserMessage(content="You are a web agent. Please click on element ID 42.")
]

 print("Sending prompt to Gemini expecting structured 'MockActionOutput'...")
 try:
 response = await llm.ainvoke(messages, output_format=MockActionOutput)
 print("■ Success!")
 print(f"Output: {response.completion.action}")
 except Exception as e:
 print(f"■ Failed: {e}")

if __name__ == "__main__":
 asyncio.run(test_llm_abstraction())

```

### ***.ltests/test\_profile.py***

```

import pytest
from system.browser.profile import BrowserProfile

def test_browser_profile_default_args():
 # We must provide user_data_dir as get_args() enforces it
 profile = BrowserProfile(headless=True, disable_security=True, user_data_dir='/tmp/mock_user_data_dir')
 args = profile.get_args()

 # Check core anti-bot args or security override args
 # (Just verifying the function successfully generates a list of arguments)
 assert any('--no-sandbox' in arg for arg in args) or any('--disable-web-security' in arg for arg in args)

def test_browser_profile_chrome_instance_path():
 profile = BrowserProfile(executable_path="/usr/bin/google-chrome")
 assert str(profile.executable_path) == "/usr/bin/google-chrome"

```

### ***.ltests/test\_views.py***

```

import pytest
from pydantic import ValidationError
from system.browser.views import (
 TabInfo,
 BrowserError,
 BrowserStateHistory,
 NetworkRequest,
 PaginationButton
)

def test_tab_info_serialization():
 # Test that the target_id and parent_target_id serialize correctly (returns last 4 chars)
 tab = TabInfo(
 url="https://example.com",
 title="Example",
 target_id="1234567890abcdef",
 parent_target_id="abcdef1234567890"
)

 dumped = tab.model_dump(by_alias=True)
 assert dumped["tab_id"] == "cdef"
 assert dumped["parent_tab_id"] == "7890"

def test_tab_info_validation():
 # Test strictness of fields forbidding extra
 with pytest.raises(ValidationError):
 TabInfo(
 url="https://example.com",
 title="Example",
 tab_id="123",

```

```

 extra_field="not allowed"
)

def test_browser_error_formatting():
 # Test the string representation
 err = BrowserError("Something went wrong", details={"code": 500})
 assert "Something went wrong" in str(err)
 assert "500" in str(err)

def test_browser_state_history_to_dict():
 # Test the to_dict method
 history = BrowserStateHistory(
 url="https://example.com",
 title="Test Title",
 tabs=[
 TabInfo(url="https://example.com/1", title="Tab 1", target_id="1111"),
 TabInfo(url="https://example.com/2", title="Tab 2", target_id="2222")
],
 interacted_element=None,
 screenshot_path="/tmp/fake_path.png"
)

 data = history.to_dict()
 assert data["url"] == "https://example.com"
 assert data["title"] == "Test Title"
 assert len(data["tabs"]) == 2
 assert data["screenshot_path"] == "/tmp/fake_path.png"
 assert len(data["interacted_element"]) == 1
 assert data["interacted_element"][0] is None

```

## **`.ltests\robustness\base.py`**

```

import asyncio
import os
import logging
from dotenv import load_dotenv

from system.agent.service import Agent
from system.browser.session import BrowserSession
from system.browser.profile import BrowserProfile
from system.llm import ChatGoogle

def get_llm():
 load_dotenv()
 api_key = os.getenv("GOOGLE_API_KEY")
 if not api_key:
 raise ValueError("GOOGLE_API_KEY not found in environment")
 return ChatGoogle(model="gemini-2.5-flash", api_key=api_key)

async def run_test(task_description: str, **agent_kwargs):
 print(f"\n[TEST SUITE executing task]")
 print(f"Task: {task_description}")
 print("=" * 50)

 llm = get_llm()
 profile = BrowserProfile(headless=False, disable_security=True, user_data_dir='/tmp/mira_robustness_')
 session = BrowserSession(browser_profile=profile)

 agent = Agent(task=task_description, llm=llm, browser_session=session, **agent_kwargs)
 try:
 await session.start()
 history = await agent.run()

 final_result = history.final_result()
 print("\n\n" + "="*50)
 print("AGENT EXECUTION SUMMARY")
 print("="*50)

```

```

print(f"Total Steps Taken: {len(history.history)}")
if history.usage:
 print(f"Total Tokens Used: {history.usage.total_tokens} (Prompt: {history.usage.total_prompt_tokens})")
for i, step in enumerate(history.history):
 if set_action := step.model_output:
 print(f"Step {i+1}: Action -> {list(set_action.action[0].model_dump(exclude_unset=True))}")
 if set_action.current_state.thinking:
 print(f" Thought: {set_action.current_state.thinking}")
print("-" * 50)
print(f"■ Final Result: {final_result}")
print("=" * 50 + "\n")
except Exception as e:
 print(f"■ Test threw an exception: {e}")
finally:
 await session.stop()

```

### ***.\tests\robustness\test\_1a\_static.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://en.wikipedia.org/wiki/Python_(programming_language) and extract the summary paragraph."
 asyncio.run(run_test(task))

```

### ***.\tests\robustness\test\_1b\_dynamic.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://news.ycombinator.com/. Scroll to the bottom and click the 'More' link to load more stories."
 asyncio.run(run_test(task))

```

### ***.\tests\robustness\test\_1c\_ecommerce.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://www.amazon.com. Search for 'wireless mouse', apply the '4 Stars & Up' customer filter, and list the top 5 results."
 asyncio.run(run_test(task))

```

### ***.\tests\robustness\test\_1d\_iframe.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_iframe. There is an iframe on the page. Extract its content."
 asyncio.run(run_test(task))

```



### ***.ltests\robustness\test\_1d\_macro.py***

```
import asyncio
import os
from pathlib import Path
from system.agent.service import Agent
from system.agent.workflow import WorkflowTemplate
from base import run_test

async def verify_macro():
 macro_path = Path("C:/Users/maham/tmp/mock_macro_workflow.json")
 if not macro_path.exists():
 print("■ Mock macro not found. Please run the ecommerce autonomous test first.")
 return

 print(f"■ Loading workflow macro from {macro_path}")
 with open(macro_path, "r", encoding="utf-8") as f:
 workflow_json = f.read()

 template = WorkflowTemplate.model_validate_json(workflow_json)

 # We pass it to the agent.
 # The LLM is still required in the signature, but shouldn't be invoked unless fallback occurs
 # Notice we provide the ORIGINAL task description so the LLM executes the same goal
 task_goal = template.original_task if template.original_task else "Execute the saved workflow macro."
 print(f"■ Resuming original semantic goal: {task_goal}")

 agent = await run_test(
 task_description=task_goal,
 workflow_template=template
)

 print("\n■ Macro Execution Test complete.")

if __name__ == "__main__":
 asyncio.run(verify_macro())
```

### ***.ltests\robustness\test\_2a\_multitab.py***

```
import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Open a tab to https://en.wikipedia.org/wiki/Cat. Then open a new tab to https://en.wikipedia.org/wiki/Dog."
 asyncio.run(run_test(task))
```

### ***.ltests\robustness\test\_2b\_form.py***

```
import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://www.google.com/travel/flights. Set the departure city to 'JFK' (New York) and the destination city to 'LAX' (Los Angeles)."
 asyncio.run(run_test(task))
```

### ***.ltests\robustness\test\_2c\_math.py***

```
import asyncio
```

```

from base import run_test

if __name__ == "__main__":
 task = "Go to https://www.calculator.net/. Using the on-screen buttons, calculate (45 * 12) + 300, a
 asyncio.run(run_test(task))

```

### ***.ltests\robustness\test\_3a\_blocker.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://nowsecure.nl/, which is protected by Cloudflare. Try to read the page content.
 asyncio.run(run_test(task))

```

### ***.ltests\robustness\test\_3b\_overlays.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://www.forbes.com/. You will likely encounter an overlay, intrusive ad, or cookie
 asyncio.run(run_test(task))

```

### ***.ltests\robustness\test\_3c\_missing.py***

```

import asyncio
from base import run_test

if __name__ == "__main__":
 task = "Go to https://example.com and click the 'Sign Up' or 'Login' button. When you realize it dem
 asyncio.run(run_test(task))

```

### ***.ltests\robustness\test\_4a\_followup.py***

```

import asyncio
from system.agent.service import Agent
from system.browser.session import BrowserSession
from system.browser.profile import BrowserProfile
from base import get_llm
from system.logging_config import setup_logging

async def run_interactive_chat():
 """
 Tests Multi-turn Follow-ups interactively:
 Reuses the same BrowserSession and injects the previous AgentState
 so the agent "remembers" what it was doing and can act on the current page
 based on dynamic user input.
 """
 setup_logging()
 llm = get_llm()

 print("\n[TEST SUITE executing interactive chat session]")
 print("Type 'exit' or 'quit' to end the session.")
 print("=" * 50)
 # We use a single persistent session for all agent runs

```

```

keep_alive=True ensures the browser doesn't auto-close when an agent run finishes
profile = BrowserProfile(headless=False, disable_security=True, user_data_dir='/tmp/mira_followup_t')
session = BrowserSession(browser_profile=profile)

previous_state = None
turn_counter = 1

try:
 await session.start()

 while True:
 # 1. Get user input (using run_in_executor to not block the async loop)
 task = await asyncio.get_event_loop().run_in_executor(None, input, "\n■ You: ")

 if task.lower().strip() in ['exit', 'quit']:
 print("\nEnding session...")
 break

 if not task.strip():
 continue

 print(f"\n[TURN {turn_counter}] Executing task...")

 # 2. Configure Agent state
 if previous_state:
 # Mark it as a follow-up task so it prefers the current page context
 previous_state.follow_up_task = True

 # 3. Create Agent with shared session and previous state
 agent = Agent(
 task=task,
 llm=llm,
 browser_session=session,
 injected_agent_state=previous_state
)

 # 4. Run Agent
 history = await agent.run()

 print("\n" + "-" * 50)
 print(f"■ Result: {history.final_result()}")
 print("-" * 50)

 # 5. Save state for next turn
 previous_state = agent.state
 turn_counter += 1

except Exception as e:
 print(f"\n■ Encountered an error: {e}")
finally:
 # We must explicitly clean up the session
 print("\nCleaning up browser session...")
 try:
 await session.kill()
 except:
 pass

if __name__ == "__main__":
 asyncio.run(run_interactive_chat())

```